

React быстро

АЗАТ МАРДАН
МОРТЕН БАРКЛУНД



ВТОРОЕ ИЗДАНИЕ



MANNING

React Quickly

SECOND EDITION

MORTEN BARKLUND

AZAT MARDAN



MANNING

SHELTER ISLAND

React быстро

ВТОРОЕ ИЗДАНИЕ

АЗАТ МАРДАН
МОРТЕН БАРКЛУНД



Санкт-Петербург • Москва • Минск

2024

ББК 32.988.02-018
УДК 004.738.5
M25

Мардан Азат, Барклунд Мортен

M25 React быстро. 2-е межд. изд. — СПб.: Питер, 2024. — 512 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-2220-2

React предельно упрощает создание привлекательных и надежных интерфейсов для веб-приложений. Эта великолепная библиотека JavaScript имеет модульную архитектуру, что позволяет легко создавать, объединять и тестировать компоненты. React идеально подходит для небольших прототипов, корпоративных сайтов и других подобных решений.

«React быстро. 2-е издание» предлагает уникальный подход к освоению фреймворка React. Более 80 компактных примеров проведут читателя от изучения основ работы к созданию довольно сложных приложений. В книге подробно описаны многие функциональные компоненты, хуки React и средства доступности веб-приложений, а также представлены интересные проекты для отработки новых навыков.

Книга предназначена для разработчиков, имеющих опыт создания веб-приложений на базе JavaScript.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.988.02-018
УДК 004.738.5

Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1633439290 англ.

Authorized translation of the English edition © 2023 Manning Publications.
This translation is published and sold by permission of Manning Publications,
the owner of all rights to publish and sell the same.

ISBN 978-5-4461-2220-2

© Перевод на русский язык ООО «Прогресс книга», 2024
© Издание на русском языке, оформление ООО «Прогресс книга», 2024
© Серия «Библиотека программиста», 2024

<https://t.me/portalToIT>

Краткое содержание

Глава 1. Знакомство с React.....	31
Глава 2. Первые шаги с React.....	59
Глава 3. Знакомство с JSX.....	104
Глава 4. Функциональные компоненты.....	151
Глава 5. Состояния и их роль в интерактивной природе React	190
Глава 6. Эффекты и жизненный цикл компонентов React.....	243
Глава 7. Хуки как основа веб-приложений	280
Глава 8. Обработка событий в React.....	297
Глава 9. Работа с формами в React	348
Глава 10. Расширенные хуки React для масштабирования	394
Глава 11. Проект: меню сайта	435
Глава 12. Проект: таймер	466
Глава 13. Проект: менеджер задач.....	489

Оглавление

Отзывы о первом издании	17
Предисловие	20
Благодарности	22
О книге	24
Для кого эта книга	24
Структура книги	25
Исходный код	26
Программные требования	26
Об авторах	28
Иллюстрация на обложке	29
От издательства	30
Глава 1. Знакомство с React	31
1.1. Преимущества React	33
1.1.1. Простота	34
1.1.2. Скорость и удобство тестирования	40
1.1.3. Экосистема и сообщество	42

1.2. Недостатки React	42
1.3. Как React интегрируется в веб-приложение	43
1.3.1. Одностраничные приложения и React	45
1.3.2. Стек React	47
1.4. Первый код React: Hello world	49
1.4.1. Результат	50
1.4.2. Написание приложения	51
1.4.3. Установка и запуск веб-сервера	54
1.4.4. Переход на локальный веб-сайт	55
1.5. Вопросы	56
Ответы	57
Итоги	57
Глава 2. Первые шаги с React	59
2.1. Создание нового приложения React	60
2.1.1. Команды проектов React	64
2.1.2. Структура файлов	66
2.1.3. Шаблоны	67
2.1.4. Достоинства и недостатки	68
2.2. О примерах в книге	70
2.3. Вложение элементов	71
2.3.1. Иерархия узлов	73
2.3.2. Простое вложение	74
2.3.3. Одноуровневые элементы	76
2.4. Создание нестандартных компонентов	81
2.5. Работа со свойствами	85
2.5.1. Одно свойство	86
2.5.2. Несколько свойств	88
2.5.3. Специальное свойство: children	92
2.6. Структура приложения	96

2.7. Вопросы.....	101
Ответы.....	102
Итоги.....	103

Глава 3. Знакомство с JSX 104

3.1. Что такое JSX и чем он хорош?	105
3.1.1. До и после JSX	106
3.1.2. Сочетание HTML и JavaScript	106
3.2. Логика JSX	109
3.2.1. Создание элементов с JSX	109
3.2.2. Использование JSX с нестандартными компонентами	111
3.2.3. Многострочные объекты JSX	113
3.2.4. Вывод переменных в JSX	114
3.2.5. Работа со свойствами в JSX.....	116
3.2.6. Условные конструкции в JSX	120
3.2.7. Комментарии в JSX	128
3.2.8. Списки объектов JSX.....	129
3.2.9. Фрагменты в JSX.....	132
3.3. Транспилиция JSX	135
3.4. Возможные проблемы React и JSX.....	136
3.4.1. Самозакрывающиеся элементы	136
3.4.2. Специальные символы.....	137
3.4.3. Преобразование строк	138
3.4.4. Атрибут style.....	140
3.4.5. Зарезервированные имена class и for	141
3.4.6. Атрибуты из нескольких слов	142
3.4.7. Значения логических атрибутов.....	142
3.4.8. Пробелы.....	145
3.4.9. Атрибуты data-	148
3.5. Вопросы.....	149
Ответы	150
Итоги.....	150

Глава 4. Функциональные компоненты..... 151

4.1. Сокращенная запись компонентов React	153
4.1.1. Пример приложения	153
4.1.2. Деструктуризация свойств	158
4.1.3. Значения по умолчанию	160
4.1.4. Сквозные свойства.....	163
4.2. Сравнение типов компонентов.....	166
4.2.1. Преимущества функциональных компонентов	167
4.2.2. Недостатки функциональных компонентов.....	168
4.2.3. Незначимые факторы при выборе компонентов	168
4.2.4. Выбор типа компонента.....	169
4.3. Когда не стоит использовать функциональные компоненты.....	170
4.3.1. Границы ошибок	170
4.3.2. Код с компонентами на базе классов.....	171
4.3.3. Библиотека требует использования компонентов на базе классов	172
4.3.4. <code>getSnapshotBeforeUpdate</code>	172
4.4. Преобразование компонента на базе класса в функциональный компонент	173
4.4.1. Версия 1: только <code>render()</code>	174
4.4.2. Версия 2: вспомогательный метод	177
4.4.3. Версия 3: реальный метод класса	181
4.4.4. Версия 4: конструктор	184
4.4.5. Повысить сложность — значит усложнить преобразование.....	187
4.5. Вопросы.....	188
Ответы	189
Итоги.....	189

Глава 5. Состояния и их роль в интерактивной природе React 190

5.1. Почему в приложениях React важно состояние?	192
5.1.1. Состояние компонента React	193
5.1.2. Где хранить состояние?	194

5.1.3. Что можно хранить в состоянии компонента	196
5.1.4. Что не стоит хранить в состоянии.....	197
5.2. Добавление состояния в функциональный компонент	198
5.2.1. Импортирование и использование хука	201
5.2.2. Инициализация состояния	204
5.2.3. Деструктуризация значения состояния и сеттера.....	212
5.2.4. Использование значений состояния	213
5.2.5. Присваивание значения.....	216
5.2.6. Использование множественных состояний.....	228
5.2.7. Область видимости состояния	231
5.3. Компоненты на базе классов с состоянием.....	236
5.3.1. Сходство с хуком useState.....	237
5.3.2. Отличия от хука useState	239
5.4. Вопросы.....	240
Ответы	241
Итоги.....	241

Глава 6. Эффекты и жизненный цикл компонентов React..... 243

6.1. Выполнение эффектов в компонентах	244
6.1.1. Выполнение эффекта при монтировании	246
6.1.2. Выполнение эффекта при монтировании и очистки при демонтировании	249
6.1.3. Выполнение очистки при демонтировании	252
6.1.4. Выполнение эффекта при некоторых рендерингах	254
6.1.5. Выполнение эффекта и очистки при отдельных рендерингах.....	257
6.1.6. Синхронное выполнение эффекта	261
6.2. Рендеринг.....	264
6.2.1. Рендеринг при монтировании	265
6.2.2. Рендеринг при рендеринге родительского компонента	267
6.2.3. Рендеринг при обновлении состояния	269
6.2.4. Рендеринг внутри функций.....	271

6.3. Жизненный цикл компонента на базе класса	274
6.3.1. Методы жизненного цикла	275
6.3.2. Унаследованные методы жизненного цикла	275
6.3.3. Преобразование методов жизненного цикла в хуки.....	276
6.4. Вопросы.....	277
Ответы	278
Итоги.....	279
Глава 7. Хуки как основа веб-приложений	280
7.1. Компоненты с состоянием	281
7.1.1. Простые значения состояния с useState	282
7.1.2. Создание сложного состояния с useReducer	282
7.1.3. Сохранение значения без повторного рендеринга с использованием useRef	283
7.1.4. Упрощенное многокомпонентное состояние с использованием useContext	288
7.1.5. Низкоприоритетные обновления состояния с использованием useDeferredValue и useTransition	288
7.2. Эффекты компонентов.....	289
7.3. Оптимизация производительности за счет сокращения количества повторных рендерингов	290
7.3.1. Мемоизация произвольного значения с использованием useMemo	291
7.3.2. Мемоизация функций с использованием useCallback.....	291
7.3.3. Создание стабильных идентификаторов DOM с использованием useId.....	291
7.4. Создание сложных библиотек компонентов	291
7.4.1. Создание API компонентов с использованием useImperativeHandle.....	292
7.4.2. Улучшенная отладка хуков с использованием useDebugValue	292
7.4.3. Синхронизация внешних данных с использованием useSyncExternalStore	293

7.4.4. Выполнение эффекта перед рендерингом с использованием useInsertionEffect	293
7.5. Два ключевых принципа хуков	294
7.6. Вопросы	294
Ответы	295
Итоги	296

Глава 8. Обработка событий в React 297

8.1. Обработка событий DOM в React	299
8.1.1. Базовая обработка событий в React	300
8.2. Обработчики событий	305
8.2.1. Определение обработчиков событий	306
8.2.2. Объекты событий	308
8.2.3. Объекты событий React	309
8.2.4. Длительное хранение объектов синтетических событий	312
8.3. Фазы событий и распространение	315
8.3.1. Как фазы и распространение работают в браузере	319
8.3.2. Обработка фаз событий в React	323
8.3.3. Нетипичное распространение событий	323
8.3.4. Невсплывающие события DOM	324
8.4. Действия по умолчанию и как их избежать	325
8.4.1. Действие события по умолчанию	326
8.4.2. Как избежать выполнения действий по умолчанию	326
8.4.3. Другие события по умолчанию	328
8.5. Общие сведения об объектах событий React	329
8.6. Функции обработчиков событий из свойств	330
8.7. Генераторы обработчиков событий	333
8.8. Ручное прослушивание событий DOM	334
8.8.1. Прослушивание событий окна и документа	335
8.8.2. Неподдерживаемые события HTML	338
8.8.3. Объединение обработки событий React и DOM	340

8.9. Вопросы	345
Ответы	346
Итоги	346
Глава 9. Работа с формами в React	348
9.1. Управляемый и неуправляемый ввод	350
9.2. Контроль управляемого ввода	351
9.2.1. Фильтрация ввода	354
9.2.2. Ввод с применением маски	357
9.2.3. Ввод множества похожих значений	359
9.2.4. Отправка данных форм	366
9.2.5. Другие виды полей ввода	373
9.2.6. Другие свойства	381
9.3. Контроль неуправляемого ввода	382
9.3.1. Возможности	386
9.3.2. Поля ввода файлов	390
9.4. Вопросы	391
Ответы	392
Итоги	392
Глава 10. Расширенные хуки React для масштабирования	394
10.1. Разрешение компонентам обращаться к значениям	396
10.1.1. React Context	400
10.1.2. Контекстные состояния	404
10.1.3. Подробнее о React Context	407
10.2. Работа со сложным состоянием	415
10.2.1. Взаимозависимое состояние	417
10.3. Нестандартные хуки	424
10.3.1. Как образуется нестандартный хук?	425
10.3.2. Когда использовать нестандартный хук?	426
10.3.3. Где найти нестандартные хуки?	431

10.4. Вопросы.....	432
Ответы	433
Итоги.....	433

Глава 11. Проект: меню сайта 435

11.1. Заготовка меню.....	438
11.1.1. Выходная разметка HTML.....	439
11.1.2. Иерархия компонентов.....	439
11.1.3. Значки.....	440
11.1.4. CSS.....	441
11.1.5. Шаблон.....	442
11.1.6. Исходный код.....	444
11.1.7. В браузере.....	445
11.2. Рендеринг статического меню.....	446
11.2.1. Цель упражнения.....	446
11.2.2. Выходная разметка HTML.....	446
11.2.3. Дерево компонентов	447
11.2.4. Исходный код.....	448
11.2.5. В браузере.....	449
11.3. Домашнее задание: динамическое меню.....	450
11.3.1. Цель шага.....	450
11.3.2. Подсказки для решения.....	451
11.3.3. Иерархия компонентов.....	451
11.3.4. Что дальше?	453
11.4. Домашнее задание: получение данных из контекста.....	453
11.4.1. Цель шага.....	453
11.4.2. Подсказки для решения.....	454
11.4.3. Иерархия компонентов.....	455
11.4.4. Что дальше?	455
11.5. Домашнее задание: дополнительная ссылка.....	457
11.5.1. Цель шага.....	457
11.5.2. Подсказки для решения.....	458

11.5.3. Иерархия компонентов.....	463
11.5.4. Что дальше?	464
11.6. Несколько слов напоследок	465
Итоги.....	465
Глава 12. Проект: таймер	466
12.1. Заготовка для таймера	469
12.1.1. Выходная разметка HTML.....	470
12.1.3. Структура проекта.....	473
12.1.4. Исходный код.....	474
12.1.5. Запуск приложения	477
12.2. Добавление простой кнопки запуска/остановки таймера	478
12.2.1. Цель упражнения.....	478
12.2.2. Иерархия компонентов.....	479
12.2.3. Обновленная структура проекта	479
12.2.4. Исходный код.....	481
12.2.5. Запуск приложения	485
12.3. Домашнее задание: инициализация таймера произвольным временем.....	485
12.4. Домашнее задание: сброс таймеров	486
12.5. Домашнее задание: множественные таймеры	487
Итоги.....	488
Глава 13. Проект: менеджер задач	489
13.1. Заготовка менеджера задач	493
13.1.1. Иерархия компонентов.....	493
13.1.2. Структура проекта.....	493
13.1.3. Исходный код.....	494
13.1.4. Запуск приложения	496
13.2. Простой список задач.....	496
13.2.1. Цель упражнения.....	497
13.2.2. Иерархия компонентов.....	497

13.2.3. Обновленная структура проекта	498
13.2.4. Исходный код.....	500
13.2.5. Запуск приложения	506
13.3. Домашнее задание: этапы и прогресс выполнения задач	506
13.4. Домашнее задание: приоритеты.....	508
13.5. Домашнее задание: перетаскивание	509
13.6. Заключение	510
Итоги.....	510

Отзывы о первом издании

«Единое окно» для всех, кому нужно пошаговое руководство не только по React, но и по экосистеме сопутствующих средств, концепций и библиотек.

— *Питер Купер (Peter Cooper), редактор JavaScript Weekly*

Идеально подходит как для новичков React, так и для самых опытных специалистов.

— *Мэтью Хек (Matthew Heck), TechChange*

Захватывающая книга, в которой теория соседствует с практикой!

— *Дейн Балиа (Dane Balia), Entelect*

Отличное руководство, чтобы начать работать с React... быстро!

— *Арт Бергквист (Art Bergquist), Cognetic Technologies*

Книгу легко читать. Она написана простым языком, помогающим шаг за шагом разбираться в каждой концепции.

— *Израэл Моралес (Israel Morales), SavvyCard*

Наконец-то я понял, как работать с React, и это потрясающе.

— *Питер Хэмптон (Peter Hampton), Университет Ольстера*

18 Отзывы о первом издании

«React быстро» — отличный ресурс для тех, кто хочет побыстрее включиться в работу с React. Очень основательный и актуальный материал. Обращусь к нему, когда начну работать над следующим приложением.

— Натан Бэйли (*Nathan Bailey*), *SpringboardAuto.com*

Если вы только начинаете работать с библиотекой React и действительно хотите ее освоить, эта книга — именно то, что вам нужно.

— Ричард Хо (*Richard Kho*), *Capital One*

Моей жене и сыну — они мотивируют меня становиться лучше и лучше как человек и как писатель. Как сказал Майкл Дж. Фокс, «семья — это не главное. Это всё».

— *Мортен Барклунд*

Посвящаю своему деду Халиту Хамитову. Спасибо за твою доброту и честность. Ты навсегда останешься в моей памяти — как все, чему ты меня научил, как наши поездки на дачу и наши сражения в шахматы.

— *Азат Мардан*

Предисловие

Перед вами — настоящая история любви! Мальчик знакомится с библиотекой и пропадает в ней! Это любовь с первого взгляда, и библиотека отвечает взаимностью. Они будут жить долго и счастливо, жизнеутверждающая музыка, хэппи-энд.

Я занимаюсь разработкой веб-приложений более десяти лет, но мне всегда как будто чего-то не хватало. Я работал с JavaScript, jQuery и даже Angular 1.0, но они не заполнили пробелов. Будем честны: кому захочется писать спагетти-код, который развалится уже через полгода?

Но однажды я наткнулся на React, и все тут же встало на свои места. Это было мгновенное притяжение, которому я не мог сопротивляться. Как только я впервые увидел эту библиотеку, я уже знал, что мы созданы друг для друга.

Внезапно все обрело смысл. Работа с компонентами и управление потоками данных в React стали именно тем, чего я искал. Кто знал, что писать код может быть так интересно? Я ринулся изучать все, что только можно узнать о React. Я читал документацию, смотрел обучающие ролики и делал проекты с таким азартом, что стирал пальцы о клавиатуру. Ладно, это небольшое преувеличение, но вы поняли, о чем я.

Я просто не мог оторваться от новой парадигмы. Мне хотелось переписать весь код, который я раньше писал. Когда же я стал глубже погружаться в философию разработки, лежащую в основе библиотеки, мое влечение только усилилось и переросло в отношения всей жизни.

Так случилось, что спустя несколько лет после перехода на React я познакомился с обаятельным Азатом — родственной душой с неукротимой страстью к этой замечательной библиотеке. Представьте мой восторг, когда я узнал, что Азат воплотил свою любовь к React в большой книге — первом издании «React быстро».

Это издание запало мне в душу. Его практические примеры, четкая структура и стиль изложения, рассчитанный на начинающих, идеально подходили для любого энтузиаста React. Сообщество не скупилось на похвалы автору, и вполне заслуженно.

Но React, как и любое юное дарование, продолжал развиваться и взрослеть. И в один знаменательный день 2019 года в него были добавлены хуки, принятые разработчиками с энтузиазмом и ставшие настоящей инновацией. Это был поворотный момент, который произвел революцию в разработке приложений.

Тогда мы с Азатом решили объединиться и перенести книгу в новую эпоху, сохранив ее актуальность и популярность. Мы поставили перед собой задачу подготовить второе издание, отражающее все современные возможности React. Структура и дух первого издания не изменились, но мы обновили его, чтобы оно шло в ногу со временем. И теперь мы рады поделиться с миром своей любовью к React!

– *Мортен Барклунд*

Благодарности

Выражаем глубочайшую благодарность команде издательства, которая помогла воплотить эту книгу в реальность. Прежде всего это наш главный редактор Фрэнсис Лефковиц (Frances Lefkowitz), неустанно помогавшая придать форму материалу и отшлифовать текст, чтобы он был полезен читателям всех уровней. Ее бесценные наблюдения и подсказки сыграли важнейшую роль в подготовке книги, которой мы действительно гордимся.

Также хотим выразить особую благодарность ведущему редактору издательства Энди Уолдрону (Andy Waldron); он верил в этот проект с самого начала и помог нам ориентироваться в сложном мире издательского дела.

Отдельное спасибо нашему научному редактору Нинославу Черкезу (Ninoslav Čerkez); он представил подробную техническую обратную связь и помог проследить, чтобы примеры кода и объяснения были точными и актуальными. Спасибо корректору Крису Виллануэве (Chris Villanueva) за внимательность, благодаря которой он обнаружил множество ошибок и неточностей.

Не можем не упомянуть литературного редактора Джули Макнами (Julie McNamee) за ее бесподобную работу над рукописью. Она не эксперт по грамматике, а настоящий мастер языка, которая позаботилась, чтобы в тексте не осталось лишних слов и он стал более лаконичным и выразительным. Спасибо, Джули!

Спасибо вам, наши рецензенты: Эмит Ламба (Amit Lamba), Андрес Саку (Andres Saco), Бернар Фуэнтес (Bernard Fuentes), Брендан О'Хара (Brendan O'Hara), Brent Бойлан (Brent Boylan), Крис Виллануэва (Chris Villanueva), Данило Зекович (Danilo Zekovic'), Дерик Хичкок (Derick Hitchcock), Фернандо Бернардино (Fernando Bernardino), Франсиско Ривас (Francisco Rivas), Ганеш Сваминатан (Ganesh Swaminathan), Харш Равал (Harsh Raval), Джеймс (James Bishop), Джейсон Хейлз (Jason Hales), Джефф Смит (Jeff Smith), Джон Пантоя

(John Pantoja), Картикеяраджан Раджендран (Karthikeyarajan Rajendran), Келум Прабат Сенанаяке (Kelum Prabath Senanayake), Кент Спиллнер (Kent Spillner), Ларри Кай (Larry Cai), Мэтт Деймел (Matt Deimel), Маттео Баттиста (Matteo Battista), Мишель Уильямсон (Michelle Williamson), Мик Уилсон (Mick Wilson), Миранда Вурр (Miranda Whurr), Нитендра Бхосле (Nitendra Bhosle), Нуран Махмуд (Nouran Mahmoud), Патрис Мальдаг (Patrice Maldague), Питер Гиселинк (Pieter Gyselinck), Ричард Харриман (Richard Harriman), Ричард Тобиас (Richard Tobias), Родни Вайс (Rodney Weis), Роман Жу́жа (Roman Zhuzha), Сайоа Пикадо Фернандес (Saioa Picado Fernández), Сантош Джозеф (Santosh Joseph), Теофанис Деспудис (Thefanis Despoudis) и Ив Дорфсман (Yves Dorfsman) — ваши рекомендации помогли сделать эту книгу лучше.

Спасибо всем за вашу преданность, усердную работу и практический опыт. Без вас мы бы не справились.

О книге

Эта книга призвана помочь начинающему разработчику React стать опытным мастером. В ней подробно описываются основы React, и материал построен так, чтобы и начинающие, и опытные разработчики могли освоить базовые концепции популярной библиотеки, такие как JavaScript XML (JSX), компоненты, состояние, хуки, события и элементы форм.

Книга будет чрезвычайно полезна любому, кто хочет создавать приложения React, независимо от его уровня знаний. Она понятно и подробно рассказывает о базовых концепциях React и поможет разработчику писать чистый код, который легко обслуживать, понимать и расширять.

Второе издание «React быстро» научит всем фундаментальным принципам, необходимым для проектирования понятных, эффективных, легко обновляемых веб-приложений с использованием React. Как вы увидите, в React сложилась огромная экосистема средств и библиотек. После чтения книги и проработки примеров вы можете решить продолжить совершенствовать свои навыки с прицелом на карьерный рост. В таком случае вам может пригодиться книга «Job-Ready React» Мортена Барклунда (Manning, 2024), которая расширяет навыки и методологии, описанные здесь. Она подготовит вас к будущей работе и познакомит с библиотеками более высокого уровня, а также с приемами и средствами из арсенала профессиональных разработчиков React.

ДЛЯ КОГО ЭТА КНИГА

Новички найдут в книге практические примеры и упражнения, которые помогут им разработать первые приложения React, а также как следует разобраться

в принципах работы React. В книге приводится пошаговое руководство по созданию приложений React с нуля, с практическими примерами и упражнениями, подкрепляющими теорию.

Для опытных разработчиков книга станет полезным справочником и напомнит фундаментальные концепции React. Она будет особенно полезна тем, кто изучал React без системы или хочет углубить свои знания лучших практик React.

Чтобы извлечь максимум пользы из книги, желательно иметь опыт работы с HTML, CSS и JavaScript, но быть экспертом в этих областях не обязательно. Что еще важнее, вам не понадобится знание React — мы начнем с самого начала и дойдем до того момента, когда вы станете уверенно строить сложные приложения!

СТРУКТУРА КНИГИ

Книга включает десять глав, посвященных отдельным темам; за ними следуют три главы проектов. Первые десять глав расположены в естественной последовательности. Читателям без опыта работы с React мы рекомендуем знакомиться с ними по порядку, чтобы обучение было наиболее эффективным.

Если у вас уже есть опыт работы с React, вы можете сразу переходить к интересующей вас главе. В главах с 1-й по 4-ю читатель знакомится с React и ее основными концепциями: структурой компонентов, JSX и функциональными компонентами. Если эти темы вам уже знакомы, эти главы можно пропустить.

В главах с 5-й по 7-ю описаны различные хуки, включая базовые хуки состояния (глава 5), хуки эффектов (глава 6) и прочие (глава 7). В главах 8 и 9 рассматриваются события и формы соответственно. Чтобы освоить приемы работы с полями ввода в формах, просто необходимо отлично разбираться в событиях.

Глава 10 обобщает все рассмотренные концепции. В ней разбираются некоторые нетривиальные компоненты и логические паттерны, которые могут встретиться в более сложных приложениях.

Наконец, в главах с 11-й по 13-ю вы можете проверить новые знания на трех проектах с возрастающей сложностью. Мы создадим интерактивное меню для сайта, таймер и менеджер задач с расширенной функциональностью. Если вы уже знакомы с React, попробуйте самостоятельно реализовать эти проекты, чтобы понять пробелы в своих знаниях.

Исходный код

Весь исходный код, использованный в книге, доступен на GitHub, а также на справочном сайте книги. На нем же доступна система, которая позволяет не только просмотреть и загрузить исходный код для каждого примера, но и запустить полученное приложение прямо в браузере без загрузки каких-либо компонентов. Репозиторий GitHub находится по адресу <https://github.com/rq2e/rq2e>, а программа просмотра исходного кода — по адресу <https://reactquickly.dev/browse>.

Исполняемые фрагменты кода можно загрузить с электронной версии этой книги (liveBook) по адресу <https://livebook.manning.com/book/react-quickly-second-edition>. Полный код примеров из книги также доступен для загрузки на веб-сайте Manning по адресу www.manning.com/books/react-quickly-second-edition.

Книга содержит множество примеров исходного кода как в виде нумерованных листингов, так и встроенных в обычный текст. В обоих случаях исходный код оформляется моноширинным шрифтом, чтобы он отличался от исходного текста.

Во многих случаях исходный код отформатирован; мы добавили разрывы строк и изменили отступы, чтобы листинги помещались на странице. Впрочем, иногда даже этого оказывалось недостаточно, и в листинги пришлось включать маркеры продолжения строк (➡). Многие листинги снабжены выносками, поясняющими моменты, на которые следует обратить внимание.

Программные требования

Чтобы использовать и запускать примеры и проекты в этой книге, вам понадобятся:

- среда командной строки с установленной последней версией Node.js и npm;
- текстовый редактор.

Вот и все! А теперь мы покажем, как быстро настроить среду командной строки и выбрать текстовый редактор, чтобы подготовиться к первому упражнению главы 1.

Среда командной строки с Node.js и npm

Сначала необходимо убедиться, что у вас установлены совместимые версии Node.js и npm. Для запуска примеров из книги понадобится Node.js версии 12 и выше.

Для Windows:

- Откройте окно командной строки или PowerShell, нажав Windows + R, затем введите команду `cmd` или `powershell` в диалоговом окне **Открыть**.
- Введите в командной строке `node -v` и нажмите Enter.
- Если у вас установлена библиотека Node.js, команда выведет номер версии.

Для macOS и Unix:

- Откройте приложение **Терминал**.
- Введите в терминале `node -v` и нажмите Enter.
- Если у вас установлена библиотека Node.js, команда выведет номер версии.

Если библиотека Node.js не установлена или ее версия старше версии 12, перейдите по адресу <https://nodejs.org/en/download>, загрузите пакет для своей операционной системы и выполните инструкции по установке.

Опытные пользователи могут работать с любым другим менеджером пакетов для установки Node.js. Важно установить версию не ниже 12.

Текстовый редактор

Скорее всего, вы уже умеете работать с текстовым редактором и используете его — с учетом знаний JavaScript, HTML и CSS, необходимых для того, чтобы извлечь максимум пользы из книги. Впрочем, если он у вас не установлен, воспользуйтесь одним из популярных вариантов, совместимых с большинством платформ:

- Sublime Text: www.sublimetext.com/download (бесплатная пробная версия).
- Brackets: <https://brackets.io/> (с открытым исходным кодом, бесплатный).
- Visual Studio Code: <https://code.visualstudio.com/> (бесплатный).

Форум liveBook

Приобретая книгу «React быстро», 2-е изд., вы получаете бесплатный доступ к веб-форуму издательства Manning (на английском языке), на котором можно оставлять комментарии о книге, задавать технические вопросы и получать помощь от автора и других пользователей. Чтобы получить доступ к форуму, откройте страницу <https://livebook.manning.com/book/react-quickly-second-edition/discussion>. Информацию о форумах Manning и правилах поведения на них см. на <https://livebook.manning.com/discussion>.

В рамках своих обязательств перед читателями издательство Manning предоставляет ресурс для содержательного общения читателей и авторов. Эти обязательства не подразумевают конкретную степень участия автора, которое остается добровольным (и неоплачиваемым). Задавайте автору хорошие вопросы, чтобы он не терял интереса к происходящему! Форум и архивы обсуждений доступны на веб-сайте издательства, пока книга продолжает издаваться.

ОБ АВТОРАХ



Мортен Барклунд, независимый разработчик, выступавший в качестве ведущего в разных проектах, в том числе в проекте React с открытым кодом, финансируемом Google. Мортен имеет ученую степень в computer science, полученную в Техническом университете Дании; уже более двух десятков лет он активно участвует в жизни сообщества веб-разработчиков и успел поработать над сотнями проектов.



Азат Мардан — автор таких бестселлеров о JavaScript, React и Node.js, как «React быстро», 1-е изд.; «Practical Node.js»; «Pro Express.js», «Full Stack JavaScript» и «TypeScript Mistakes». Он является приглашенным профессором Технологического университета, ментором стартапов и разработчиком или ведущим разработчиком проектов с опытом работы в стартапах и крупных корпорациях, включая YouTube, Google, Capital One, Indeed и DocuSign. Азат преподавал на многих семинарах и курсах, включая курс по edX, который прослушали более 40 000 студентов по всему миру. Азат — лауреат премии Microsoft MVP (Most Valuable Professional) в области технологий разработки и занял 239-е место по активности участия на GitHub в мире. Он выступал с докладами более чем на 30 международных конференциях, в том числе совместно с такими известными технологическими специалистами, как Дуглас Крокфорд (Douglas Crockford), Джефф Этвуд (Jeff Atwood) (один из создателей Stack Overflow), Джим Ягельски (Jim Jagielski) (создатель Apache), Скотт Хансельман (Scott Hanselman) и Дениз Купер (Danese Cooper).

Иллюстрация на обложке

Иллюстрация под названием *Homme Baschkir* («Башкир»), помещенная на обложку, взята из вышедшего в 1788 году каталога национальных костюмов, составленного Жаком Грассе де Сен-Совером (Jacques Grasset de Saint-Sauveur). Каждая иллюстрация этого каталога тщательно прорисована и раскрашена от руки.

В прежние времена по одежде человека можно было легко определить, где он живет и какова его профессия или положение в обществе. Manning отдает дань изобретательности и инициативности компьютерных технологий, используя для своих изданий обложки, демонстрирующие богатое вековое разнообразие региональных культур, оживающее на изображениях из собраний, подобных этому.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

<https://t.me/portalToIT>

Знакомство с React

В ЭТОЙ ГЛАВЕ

- ✓ Что такое React?
- ✓ React для решения задач
- ✓ Интеграция React в веб-приложения
- ✓ Пишем первое приложение React: Hello World

React — воистину революционный инструмент. Многие разработчики даже не знают о том, что он им необходим, но стоит один раз попробовать, и они уже не могут с ним расстаться. Именно так случилось с авторами этой книги и многими другими энтузиастами веб-разработки. React пользуется невероятной популярностью — и на то есть веские причины.

Чтобы заниматься веб-разработкой в начале 2000-х, все, что было нужно, — HTML и серверный язык (такой, как Perl или PHP). Старые добрые времена, когда для обычной отладки кода на фронтенд приходилось набивать его вызовами `alert()`! Время шло, интернет-технологии развивались, и сложность сайтов возросла в разы. Сайты превратились в веб-приложения со сложными интерфейсами, бизнес-логикой и уровнями данных, которые требовали постоянных изменений и обновлений, часто в реальном времени.

Для решения проблем с построением сложных пользовательских интерфейсов (UI, User Interface) было написано много библиотек шаблонов JavaScript. Однако все они требовали, чтобы разработчики придерживались классического разделения обязанностей, то есть разделения стилей (CSS), данных и структуры (HTML) и динамических взаимодействий (JavaScript), и не удовлетворяли современных потребностей. (Помните DHTML?)

React же предлагает новый подход, при правильном применении упрощающий разработку клиентской части. React — мощная библиотека пользовательского интерфейса — предоставляет альтернативу, принятую многими крупными компаниями, такими как Facebook, Netflix и Airbnb, которые сочли ее перспективной. Вместо создания одноразовых шаблонов для пользовательских интерфейсов React позволяет создавать на JavaScript UI-компоненты, которые можно снова и снова использовать на сайтах.

Вам нужен элемент для вывода капчи или выбора даты? Определите в React компонент `<Captcha />` или `<DatePicker />`, который можно добавить к форме: простой подключаемый компонент, содержащий всю функциональность и логику для взаимодействия с бэкендом. Понадобилось поле с автозаполнением, которое обращается с асинхронным запросом к базе данных после того, как пользователь введет четыре буквы или больше? Определите компонент `<Autocomplete charNum="4"/>` для асинхронного запроса. Вы даже можете выбрать, будет ли такое поле иметь интерфейс текстового поля или же вместо него будет использоваться другой нестандартный элемент формы, возможно, `<Autocomplete textbox="..." />`.

Такой подход не нов. Идея построения пользовательских интерфейсов из компонентов появилась давно, но React стал первым, кто предоставил такую возможность с использованием чистого JavaScript без шаблонов. И такой метод оказалось проще обслуживать, повторно использовать и расширять.

React — превосходная библиотека для создания пользовательских интерфейсов, и ее просто необходимо включить в свой веб-инструментарий. Тем не менее она не решает всех задач фронтенд-разработки. В этой главе рассматриваются плюсы и минусы использования React в приложениях и возможность интеграции React в существующий стек веб-разработки.

В этой книге разбираются лишь основы React. Мы хотим, чтобы читатель прочно усвоил базовые концепции и принципы библиотеки React, не углубляясь в изучение сопутствующих или более сложных тем. Сосредоточившись исключительно на React, читатели смогут как следует разобраться в возможностях библиотеки и будут готовы применить полученные знания в разработке разнообразных веб-проектов.

1.1. ПРЕИМУЩЕСТВА REACT

Создатели каждой новой библиотеки или фреймворка утверждают, что их детище по определенным параметрам превосходит своих предшественников. Вначале была библиотека jQuery, и она стала огромным шагом вперед, позволив писать межбраузерный код на чистом JavaScript. Если вам доводилось работать с JavaScript на заре веб-программирования, вы знаете, что один вызов AJAX мог занимать много строк кода, так как разработчику приходилось учитывать специфику Internet Explorer и WebKit-браузеров. С jQuery же стало достаточно одного-единственного вызова: `$.ajax()`. В те дни jQuery иногда даже называли фреймворком — но не сейчас! Теперь под фреймворком понимают что-то более масштабное и мощное.

Точно так же дело обстояло с Backbone, а затем и с Angular; каждое новое поколение фреймворков JavaScript приносило что-то новое. В этом React не уникален. Новое в нем то, что React переосмысливает некоторые базовые концепции, используемые многими популярными фреймворками: например, идею о необходимости шаблонов.

Ниже перечислены некоторые преимущества React перед другими библиотеками и фреймворками, существовавшими на момент появления React:

- *Простые приложения* — React использует компонентную архитектуру с чистым JavaScript, декларативный стиль программирования и мощные, удобные для разработчика абстракции DOM (и не только DOM, но и iOS, Android и т. д.).
- *Быстрый пользовательский интерфейс* — React обеспечивает выдающуюся производительность благодаря своей виртуальной модели DOM и алгоритму интеллектуального согласования (smart-reconciliation algorithm). Одно из дополнительных преимуществ — возможность тестирования без запуска браузера с отсутствующим графическим интерфейсом.
- *Сокращение объема кода* — огромное сообщество React и гигантская экосистема компонентов предоставляют в распоряжение разработчика множество разнообразных библиотек и компонентов. Это важный момент при выборе фреймворка, используемого для разработки.

Благодаря целому ряду особенностей React был проще в использовании по сравнению с другими фреймворками для фронтенда, доступными на первых порах его существования. Однако с момента выхода React появилось много новых фреймворков. Отчасти из-за популярности React разработчики некоторых из них

руководствовались аналогичными концепциями или стремились реализовать те же преимущества, слегка изменяя их параметры. Одни фреймворки были вдохновлены той же идеей, но работали совершенно иначе, тогда как другие были очень похожи на React, но обладали более узкой функциональностью и иногда требовали написания дополнительного кода, но в других случаях заметно сокращали объем кода приложения.

Разберем, что сделало React таким популярным. Это особенности, которые были уникальны для этого фреймворка на момент его выхода, а сейчас появились и у других современных фреймворков. Разберем их подробнее, один за другим. Начнем с невероятной простоты использования.

1.1.1. Простота

Концепция простоты в компьютерных технологиях высоко ценится как разработчиками, так и пользователями. Тем не менее она подразумевает не только простоту использования. Простое решение может быть более сложным в реализации, но в конечном итоге оно оказывается более элегантным и эффективным, а кажущееся на первый взгляд простым нередко оказывается сложным. Простота тесно связана с принципом KISS (Keep It Simple, Stupid, то есть «не усложняй»¹). Суть в том, что простые системы лучше работают.

Подход React позволяет строить более простые решения с применением кардинально улучшенного процесса веб-разработки. Когда мы начинали работать с React, это стало для нас масштабным сдвигом, который можно было сравнить с переходом от простого JavaScript без фреймворков на jQuery.

В React эта простота достигается благодаря следующим особенностям:

- *Декларативный стиль* (в отличие от императивного) — React отдает предпочтение декларативному стилю вместо императивного, автоматически обновляя представления.
- *Компонентная архитектура, использующая чистый JavaScript*, — React не использует для своих компонентов предметно-ориентированные языки (DSL, Domain-Specific Languages), только чистый JavaScript. Кроме того, отсутствует искусственное разделение работы над одной функциональностью.

¹ [https://ru.wikipedia.org/wiki/KISS_\(принцип\)](https://ru.wikipedia.org/wiki/KISS_(принцип)).

- *Мощные абстракции* — в React существует упрощенный механизм взаимодействия с DOM, который позволяет нормализовать обработку событий и другие интерфейсы, работающие одинаково в разных браузерах.

Рассмотрим все эти особенности по очереди.

Декларативный стиль

Декларативный стиль означает, что разработчик определяет, *что должно получиться*, а не *как это делать* шаг за шагом (как в императивном стиле). Но почему декларативный стиль лучше? Его главное преимущество — уменьшение сложности, упрощение чтения и понимания кода.

Различия между императивным и декларативным стилем программирования быстро переходят в область чистой теории. Декларативное программирование, в своем максимальном проявлении, становится слишком сложным для понимания без знания таких сложных абстрактных концепций, как монады и функторы. Вот несколько возможных различий между двумя стилями:

- *Выражения вместо команд* — программирование в императивном стиле часто работает с независимыми командами, которые сами по себе изменяют состояние программы, тогда как в декларативном программировании используются выражения, которые в сочетании друг с другом направляют логику выполнения.
- *Зарезервированные слова* — в императивном стиле программирования часто используются многочисленные зарезервированные слова (`for`, `while`, `switch`, `if`, `else` и т. д.), тогда как в декларативном программировании для достижения тех же результатов используются методы массивов, стрелочные функции, обращения к объектам, булевы выражения и тернарные операторы.
- *Композиция функций* — в императивном стиле программирования часто используются независимые вызовы функций и методов, тогда как программирование в декларативном стиле использует композицию функций для определения новых выражений на базе существующих и создания небольших обобщенных логических блоков, композиция которых приводит к нужному результату.
- *Изменяемость* — в императивном программировании часто используются изменяемые объекты и операции с существующими структурами, тогда как в декларативном программировании используются неизменяемые данные, а новые структуры создаются на базе уже существующих (вместо их модификации).

Продemonстрируем различия между стилями на простом примере. Требуется создать функцию `countGoodPasswords`, которая для заданного списка паролей возвращает количество «хороших» (сильных) паролей. Хорошим будем считать пароль длиной не менее 9 символов.

Это очень простая задача, которую можно решить на любом языке программирования несколькими разными способами. Для некоторых языков естественнее и удобнее какой-то определенный стиль, но JavaScript занимает особое место, так как он принадлежит обоим мирам. В нем задача может решаться как императивно, так и декларативно.

Начнем с (очень) наивного императивного решения:

```
function countGoodPasswords(passwords) {
  ➔ const goodPasswords = [];
  for (let i = 0; i < passwords.length; i++) {
    ➔ const password = passwords[i];
    if (password.length < 9) {
      continue;
    }
    ➔ goodPasswords.push(password);
  }
  return goodPasswords.length;
}
```

Зарезервированное слово управляет логикой программы

Изменяет существующий объект

Новая команда изменяет состояние программы

Конечно, этот пример немного искусственный, и даже в полностью императивной парадигме программирования программа может быть намного короче.

Реализуем тот же пример в парадигме декларативного программирования:

```
function countGoodPasswords(passwords) {
  return passwords.filter(p => p.length >= 9).length;
}
```

Мы приходим прямо к цели всего одной командой, которая манипулирует с объектом за несколько шагов, применяя композицию функций для достижения нужного результата. Исходный массив фильтруется с переходом к временному значению — массиву, содержащему только хорошие пароли. Однако этот массив нигде не сохраняется; мы переходим сразу к следующему шагу, на котором определяется длина полученного массива.

Впрочем, это был довольно типичный код JavaScript. А как насчет React? React применяет тот же декларативный подход при построении пользовательских интерфейсов. Сначала разработчик описывает элементы интерфейса

в декларативном стиле. А если в представлениях, генерируемых этими элементами, произойдут изменения, React позаботится об обновлении. Вот так!

Удобство декларативного стиля React в полной мере проявляется при необходимости вносить изменения в представление. Они называются изменениями *внутреннего состояния*. При изменении состояния React соответствующим образом обновляет представление.

ПРИМЕЧАНИЕ О том, как работают состояния, говорится в главе 4.

Компонентная архитектура с использованием чистого JavaScript

Компонентная архитектура (CBA, component-based architecture) существовала еще до появления React. Разделение обязанностей, слабая связанность (coupling) и повторное использование кода лежат в основе этого подхода, имеющего массу преимуществ: разработчики, в том числе веб-разработчики, обожают компонентную архитектуру. Структурным элементом CBA в React является класс компонента. Как и другие CBA, он обладает многими преимуществами, главное из которых — повторное использование (ведь так вы пишете меньше кода!).

Главное, чего не хватало до появления React, — реализации этой архитектуры на чистом JavaScript. Работая с Angular, Backbone, Ember и большинством других MVC-подобных фреймворков для фронтенда, вы используете один файл для JavaScript, а другой файл для шаблона (в Angular употребляется термин *директивы* для компонентов).

Использование двух языков (и двух и более файлов) для одного компонента создает ряд проблем. Разделение HTML и JavaScript хорошо работает, если разметка HTML рендерится на сервере, а JavaScript используется только для эффектов типа мигающего текста. Теперь *одностраничные приложения* (SPA, Single Page Applications) обрабатывают сложный пользовательский ввод и выполняют рендер в браузере. Это означает, что HTML и JavaScript жестко связываются в функциональном отношении. Для разработчиков было бы удобнее не разделять HTML и JavaScript в ходе работы над частью проекта (компонентом).

Во внутренней реализации React использует *виртуальную модель DOM* для определения различий (дельты) между текущим содержимым браузера и новым представлением. Этот процесс называется *сравнением* (diffing) в DOM или *соотнесением состояния с представлением* (когда они перестают различаться). Это означает, что разработчикам не нужно явно изменять представление; им достаточно обновить состояние, а представление будет обновляться автоматически по мере надобности. Вы увидите, как эта концепция снова и снова неявно

используется в книге. Мы никогда не манипулируем DOM напрямую, а поручаем React выполнять эту работу за нас.

И наоборот, с jQuery обновления приходится реализовывать в императивном стиле. Манипулируя DOM, разработчик может на программном уровне изменять веб-страницу или ее отдельные части (более вероятный сценарий) без повторной перерисовки всей страницы. Собственно, при вызове методов jQuery происходят именно операции с DOM.

Чтобы оценить, какую поддержку обеспечивает используемый фреймворк, взгляните на рис. 1.1. На одном конце шкалы находится «фреймворк», который вообще вам не помогает. Напишите свое приложение на простом JavaScript, и вы окажетесь в этой точке. Использование jQuery несколько упрощает манипуляции с DOM, но при обновлениях вы все равно не получите никакой поддержки от фреймворка. Вам придется вручную обновлять представления jQuery при обновлении данных jQuery.



Рис. 1.1. Какую помощь оказывает фреймворк? jQuery не делает ничего; Angular делает все. Для некоторых разработчиков React занимает золотую середину между этими крайностями

На другом конце шкалы находятся такие фреймворки, как Angular, — еще один очень популярный фреймворк, сравнимый с React во всех отношениях. Однако Angular работает по совершенно иному принципу, а за кулисами происходит гораздо больше «волшебства». Часто вы просто описываете, как компоненты взаимодействуют друг с другом, а Angular пытается правильно связать их. Проблема в том, что если что-то работает не так, вы часто теряете возможность точного управления. Вы многого не видите, и это приводит к излишнему усложнению.

React находит ту «золотую середину», в которой фреймворк берет на себя большую часть рутинной работы по связыванию компонентов, но не лишает вас возможности точного управления, необходимого для создания сложных веб-приложений. Разумеется, это мнение субъективно, но его разделяют многие разработчики.

Мощные абстракции

React предоставляет ряд замечательных абстракций, которые значительно упрощают жизнь разработчика:

- Синтетические события, абстрагирующие браузерные различия в платформенных событиях.
- JavaScript XML (JSX) абстрагирует JS DOM.
- Независимость от браузера обеспечивает возможность рендера в небраузерных средах (например, на сервере).

В React реализована мощная абстракция модели событий браузера. Иначе говоря, React скрывает нижележащие интерфейсы и предоставляет нормализованные/синтезированные методы и свойства. Например, при создании события `onClick` в React обработчик события получает не платформенный объект события для конкретного браузера, а синтетический объект события, который является оберткой для платформенных объектов событий. Вы можете рассчитывать на одинаковое поведение синтетических событий независимо от того, в каком браузере будет выполняться код. React также содержит набор синтетических событий для событий касания, которые отлично подходят для построения веб-приложений для мобильных устройств.

JSX — один из неоднозначных элементов React. Для некоторых разработчиков абстракция JSX — сильный аргумент в пользу React, тогда как для других она становится препятствием, иногда даже непреодолимым.

Если вы знакомы с Angular, значит, вам уже приходилось писать большой объем кода JavaScript в коде шаблона, потому что в современной веб-разработке простая разметка HTML слишком статична и вряд ли будет применяться на практике сама по себе. Мы рекомендуем дать React шанс и протестировать JSX на практике.

JSX представляет собой *синтаксический сахар* на базе JavaScript, позволяющий записывать элементы React на JavaScript в HTML-подобной записи с `<>`. React хорошо сочетается с JSX, поскольку разработчикам становится проще реализовать и прочитать код. JSX можно считать своего рода мини-языком, который компилируется в платформенный JavaScript. Таким образом, JSX не выполняется в браузере, а используется как исходный код для компиляции. Рассмотрим компактный фрагмент, написанный на JSX:

```
if (user.session) {
  return <a href="/logout">Logout</a>;
} else {
  return <a href="/login">Login</a>;
}
```

Даже если вы загрузите файл JSX в браузере при помощи библиотеки среды выполнения, которая сразу компилирует JSX в платформенный JavaScript, вы все равно выполняете не JSX, а JavaScript. В этом смысле JSX напоминает

CoffeeScript. Эти языки компилируются в платформенный JavaScript с улучшенным синтаксисом и функциональностью по сравнению с обычным JavaScript.

Мы понимаем, что для кого-то сама идея чередования HTML с кодом JavaScript выглядит странно. Всем новичкам React (как и нам когда-то) требуется время, чтобы свыкнуться с этой идеей, потому что они подсознательно ожидают увидеть лавину сообщений о синтаксических ошибках. Да, использовать JSX не обязательно. По этим двум причинам мы не рассматриваем JSX до главы 3. Однако поверьте на слово: это очень мощный язык. Стоит к нему привыкнуть, и вам будет трудно с ним расстаться.

Другой пример абстракции DOM в React — возможность рендера элементов React на сервере. Она может пригодиться для улучшения поисковой оптимизации (SEO) и/или производительности.

При рендере компонентов React на сервере варианты не ограничиваются DOM или строками HTML. Возможны гибридные решения, в которых шаблоны с контентом проходят рендер на сервере, после чего заново заполняются живыми данными в браузере. Эта опция более подробно рассматривается в разделе 1.3. И если говорить о DOM, одним из самых заманчивых преимуществ React является превосходная производительность.

1.1.2. Скорость и удобство тестирования

Помимо необходимых обновлений DOM, фреймворк может выполнить ненужные обновления, которые только снижают производительность сложных пользовательских интерфейсов. Этот эффект особенно заметен и неприятен для пользователя при большом количестве динамических UI-элементов на веб-странице.

С другой стороны, виртуальная модель DOM React существует только в памяти JavaScript. Каждый раз, когда происходит изменение в данных, React сначала сравнивает различия по своей виртуальной модели DOM; только когда библиотека знает, что в рендере произошли изменения, она обновляет фактическую модель DOM. На рис. 1.2 изображена высокоуровневая схема работы виртуальной модели DOM React при изменении данных.

В конечном итоге React обновляет только те части, для которых это абсолютно необходимо, чтобы внутреннее состояние (виртуальная модель DOM) и представление (реальная модель DOM) не отличались. Например, если присутствует элемент `<p>` и текст дополняется состоянием компонента, то обновлен

будет только текст (то есть `innerHTML`), а не сам элемент. Таким образом обеспечивается повышение производительности по сравнению с повторным рендерингом целых наборов элементов или даже целых страниц (рендеринг на стороне сервера).

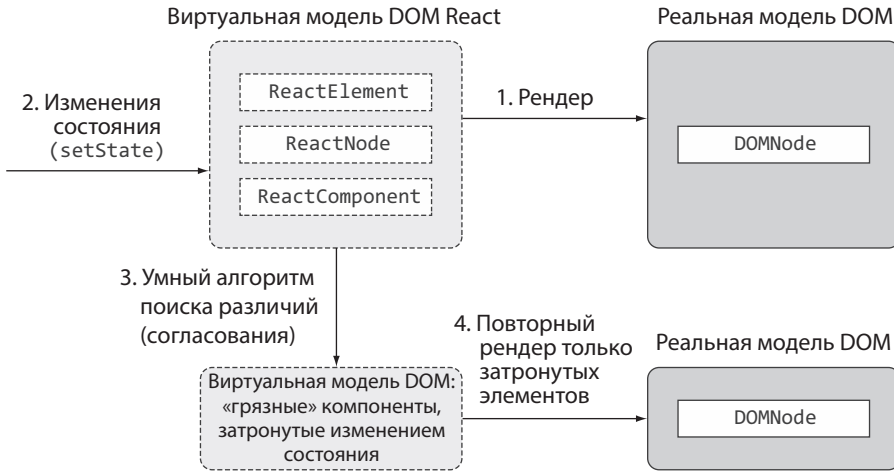


Рис. 1.2. После того как рендеринг компонента завершен, при изменении состояния он сравнивается с виртуальной моделью DOM, находящейся в памяти, и в случае необходимости рендерится заново

ПРИМЕЧАНИЕ

Если вас интересует техническая сторона алгоритмов и нотация «О-большое», следующие две статьи отлично объясняют, как команде React удалось превратить задачу $O(n^3)$ в задачу $O(n)$:

- Reconciliation на сайте React (<http://mng.bz/PQ9X>);
- React's Diff Algorithm Кристофера Шедо (Christopher Chedeau) (<http://mng.bz/68L4>).

Еще одно преимущество виртуальной модели DOM — возможность проведения модульного тестирования без браузеров, не имеющих графического интерфейса, например PhantomJS (<http://phantomjs.org>). Существует ряд библиотек, включая Jest и React Testing Library, которые позволяют тестировать компоненты React прямо из командной строки! Модульное тестирование компонентов React более подробно рассматривается в следующих главах.

1.1.3. Экосистема и сообщество

Наконец, React поддерживается разработчиками такого гиганта, как Facebook, а также их коллегами из Instagram. Как и в случае с Angular и некоторыми другими библиотеками, поддержка технологии крупной компанией обеспечивает превосходные условия для тестирования (так как приложение открывается в миллионах браузеров), создает уверенность в будущем и повышает скорость, с которой участники сообщества добавляют новый контент. Конечно, существует риск того, что, если Facebook внезапно захочет двигаться в ином направлении, которое вам не понравится, вы окажетесь в тупике, так что тщательно продумайте свой выбор.

Для React существует огромный объем качественного контента. Если вам понадобится какой-то компонент или интерфейс, достаточно вбить в поиске интернета запрос «react [имя_компонента]», и более чем в 95 % случаев вы найдете что-нибудь, заслуживающее внимания.

История ПО с открытым исходным кодом показывает, что маркетинг подобных проектов не менее важен для их успеха, чем сам код. Если у проекта убогий сайт, не хватает документации и примеров, а логотип выглядит уродливо, большинство разработчиков проигнорируют его, особенно сейчас, когда библиотек JavaScript развелось как грибов после дождя. Разработчики привередливы и не станут выбирать кога в мешке.

«Нельзя судить о книге по ее обложке», — скажете вы. На первый взгляд эта истина опровергает сказанное выше. Но к сожалению, большинство людей — и разработчики не исключение — склонны к предубеждениям, особенно когда это касается позиционирования продукта. К счастью, у React за спиной отличная техническая репутация. И раз уж речь зашла о книгах и обложках — надеемся, вы купили эту книгу не из-за обложки!

1.2. НЕДОСТАТКИ REACT

Конечно, почти все на свете имеет недостатки. В том числе и React, но их список для каждого свой. Некоторые особенности, например декларативный стиль в отличие от императивного, в высшей степени субъективны. Для кого-то это достоинство, а для кого-то недостаток. Ниже приведен наш список недостатков React (как и любые списки такого рода, он необъективен):

- *React не является полноценным фреймворком на все случаи жизни* (как швейцарский нож). Разработчикам приходится использовать React с такими библиотеками, как Redux или XState, чтобы добиться функциональности, срав-

нимой с Angular или Ember. Это может стать преимуществом, если вам нужна минималистская UI-библиотека для интеграции с существующим стеком.

- *Стеки React требуют сопровождения и непрерывного управления пакетами.* Так как вы никогда не работаете с React в отдельности, а почти всегда объединяете ее с другими пакетами, вам придется постоянно следить за зависимостями и использованием нужных версий пакетов. В крупных проектах это может добавить немало лишней работы.
- *React использует нетрадиционный подход к веб-разработке*, а JSX и функциональное программирование могут отпугнуть новичков. Особенно на первых порах React и похожим фреймворкам не хватало практических приемов, хороших книг, обучающих курсов и источников информации. Подробнее о JSX мы поговорим в главе 3.
- *React поддерживает только одностороннее связывание.* Хотя одностороннее связывание лучше для сложных приложений, поскольку оно устраняет основную сложность, некоторые разработчики (особенно Angular-разработчики), привыкшие к двустороннему связыванию, увидят, что им приходится писать чуть больше кода. Мы объясним, как работает одностороннее связывание по сравнению с двусторонним связыванием Angular, в главе 9, при рассмотрении работы с данными форм.
- *React в чистом виде не является реактивным* (в смысле реактивного программирования и архитектур, которые в большей степени управляются событиями, более гибкие и отзывчивые). Разработчикам приходится использовать другие инструменты, такие как библиотека React Query, для обеспечения бесшовной интеграции с внешним контентом и быстрого времени отклика. Также разработчикам приходится слегка менять подход к созданию приложений React; в противном случае попытки «затолкать круглый React в квадратную архитектуру» приводят к появлению плохо написанных приложений.

Продолжая введение в React, посмотрим, как React интегрируется в веб-приложение.

1.3. КАК REACT ИНТЕГРИРУЕТСЯ В ВЕБ-ПРИЛОЖЕНИЕ

Вариантов сайтов существует множество, и React можно использовать для создания интерактивного контента во многих из них — либо как замену существующих технологий, либо как механизм добавления новой функциональности на сайт. React можно использовать как на «классических» сайтах, которые в основном

рендерятся на сервере, так и в клиентских веб-приложениях, также называемых одностраничными приложениями (SPA), о которых мы уже упоминали.

Базовая библиотека React в первую очередь является UI-библиотекой. Базовая библиотека совместима с другими UI-библиотеками, но несовместима напрямую с более полнофункциональными фреймворками веб-приложений (такими, как Angular). Впрочем, в сочетании с другими библиотеками, разработанными командой React или сторонними поставщиками (например, React Router и Redux), React может стать полноценным конкурентом для любого фреймворка веб-приложений.

Если вы используете другой фреймворк SPA (Angular, Vue, Ember, Backbone и т. д.) для рендеринга веб-приложения, вероятно, вам придется полностью заменить его стеком на базе React. Очень трудно, практически невозможно, создать гибридное SPA-приложение, у которого, например, одни части генерируются средствами Angular, а другие — средствами React.

Можно использовать React только для части пользовательского интерфейса, если на сайте присутствуют небольшие интерактивные UI-элементы (*виджеты*). В таком случае можно постепенно заменять виджеты небольшими приложениями React, не внося других изменений. Существующие виджеты могут быть написаны на простом JavaScript, на базе jQuery и даже Angular или сходного фреймворка. В процессе преобразования виджетов на React вы сможете оценить разные варианты и поймете, какой из них лучше вам подходит.

Разработка фронтенда на React не зависит от части бэкенда. Другими словами, чтобы использовать React, вам не придется зависеть от бэкенда на базе JavaScript (Node или Deno). Ничто не мешает использовать React с другими технологиями бэкенда, такими как Java, Ruby, Go или Python. В конце концов, React — UI-библиотека. Ее можно интегрировать с любым бэкендом и любой библиотекой данных фронтенда (Backbone, Angular, Meteor и т. д.).

Другой популярный сценарий применения React — генераторы статических сайтов. В такой конфигурации React используется для локального определения веб-сайта в заданной среде, а затем при развертывании на живом сервере выполняется рендеринг в веб-сайт на базе простого HTML, где JavaScript используется по минимуму, только для добавления интерактивности. Все шаблоны и т. п. будут преобразованы. Изначально такое решение было популярно для небольших и относительно нечасто обновляемых сайтов, например блогов.

Недавние достижения в рендеринге React на стороне сервера повысили популярность решений с предварительным рендерингом даже для больших SPA-приложений с частыми обновлениями. В таких случаях подойдут популярные фреймворки на базе React, такие как Next.js или Remix. Подобные решения относятся к категории *веб-приложений с частичным рендерингом на стороне*

сервера, в которых код React выполняется как на сервере, так и в клиенте. Например, можно выполнить частичный рендер списка на сервере и добавить поддержку интерактивной фильтрации и сортировки на стороне клиента. Возможно, кого-то такое описание напугает, но с новыми фреймворками, такими как Next.js и Remix, такие задачи решаются относительно просто.

Подытожим, как React может интегрироваться в веб-приложения. React чаще всего используется в следующих случаях:

- Как UI-библиотека в одностраничных приложениях на базе стека, связанного с React, например React+React Router+Redux.
- Как подключаемый виджет в любом стеке фронтенда (например, как компонент ввода с автозаполнением на сайте, созданном с использованием комбинации технологий).
- Как статический веб-сайт, рендеринг которого осуществляется при развертывании для предоставления редко обновляемого контента.
- Как веб-сайт с частичным рендерингом на стороне сервера или одностраничное приложение, построенное на базе более мощного фреймворка, которое теоретически может получать контент от внешней CMS (например, WordPress или Contentful).
- Как UI-библиотека в мобильных приложениях с использованием React Native или десктопных приложениях с использованием Electron.

React хорошо сочетается с другими фронтенд-технологиями, но в основном используется как часть одностраничной архитектуры (SPA). О том, какое место React занимает в SPA, расскажем в следующем разделе.

1.3.1. Одностраничные приложения и React

Одностраничные приложения (SPA) — это особый тип веб-сайтов. Сайт относится к категории SPA, если значительная часть его функциональности (не только вывод информации) доступна непосредственно в браузере. Примеры: Facebook, Google Docs, Gmail и т. д.

SPA-приложения строятся с применением многих технологий, и React лишь одна из возможных частей стека. Более того, одной React недостаточно; чтобы React-приложение могло работать автономно, понадобится как минимум несколько других технологий. В этом разделе мы разберемся, что собой представляет SPA в целом, а затем покажем, какое место React занимает в этой структуре.

Другое название для архитектуры SPA — *толстый клиент*; браузер, выступая в роли клиента, содержит основную логику и выполняет такие функции, как

рендеринг HTML, проверка данных, изменения пользовательского интерфейса и т. д. Сравните с «тонким клиентом», в котором браузер используется только для отображения информации, заранее сгенерированной сервером. В тонком клиенте браузер выполняет минимальный объем работы.

На рис. 1.3 показано общее представление типичной архитектуры SPA независимо от используемой технологии. На диаграмме изображена высокоуровневая архитектура с пользователем, браузером и сервером. Пользователь формулирует запрос, а также выполняет действия ввода, такие как нажатие кнопок, перетаскивание, наведение указателя мыши и т. д.

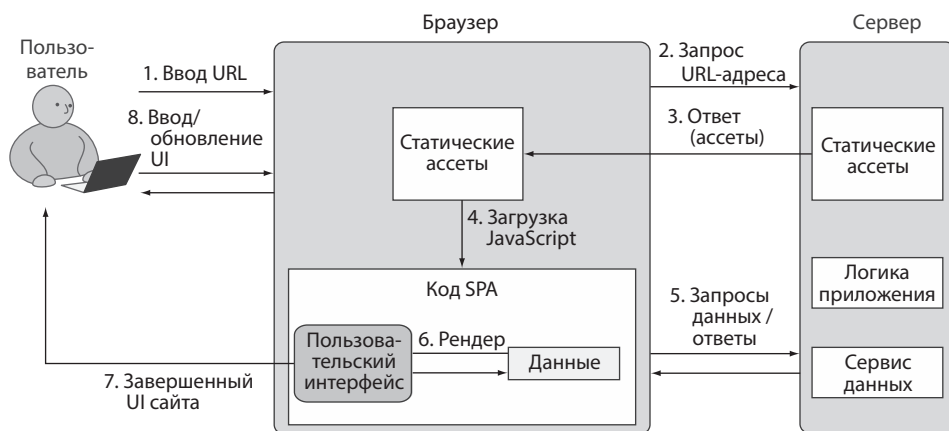


Рис. 1.3. Обобщенная архитектура SPA

Рассмотрим типичный процесс от начала до конца, следуя по порядку шагов, представленному на рис. 1.3:

1. Пользователь вводит URL-адрес в браузере, чтобы открыть новую страницу.
2. Браузер отправляет URL-запрос серверу.
3. Сервер отвечает статическими ассетами — HTML, CSS и JavaScript. В большинстве случаев разметка HTML минимальна, то есть это всего лишь заготовка веб-страницы. Обычно в это время на экране появляется сообщение «Loading...» («Загрузка...») и/или GIF-изображение спиннера.
4. Статические ассеты включают код JavaScript для SPA. При загрузке этот код отправляет дополнительные запросы данных.
5. Данные возвращаются в JSON, XML или любом другом формате.
6. После того как одностраничное приложение получит данные, оно выполняет рендеринг отсутствующей разметки HTML (блок «Пользовательский интерфейс»).

фейс» на рисунке). Другими словами, рендер UI осуществляется в браузере средствами SPA-приложения, наполняющего шаблоны данными. Этот процесс называется *гидратацией* (hydration).

7. По завершении рендера браузер обновляет контент, и пользователь может работать со страницей.
8. Пользователь видит красивую веб-страницу. Он может взаимодействовать со страницей («Ввод» на рисунке), инициируя новые запросы от SPA к серверу, и цикл шагов 2–6 продолжается. На этой стадии возможна маршрутизация в браузере, если SPA реализует ее; это означает, что навигация по новому URL-адресу инициирует не загрузку новой страницы с сервера, а повторный рендер SPA в браузере.

Подведем итог: в SPA большая часть рендера UI происходит в браузере. От браузера и к нему передаются только данные. Сравните с «классическим» веб-сайтом (то есть не SPA), при котором весь рендер, или прорисовка, выполняется на сервере. React интегрируется в архитектуру SPA на шагах 6 и 8, где выполняется рендеринг контента, основанный на данных, а также обработка пользовательского ввода и обновление контента на основании обновленных данных.

1.3.2. Стек React

React не является полноценным фреймворком JavaScript для фронтенда. Библиотеку React можно назвать минималистской в том смысле, что она решает только одну задачу (рендеринг реактивных UI) и старается делать это очень хорошо. React не привязывает выполнение таких операций, как моделирование данных, стилевое оформление или маршрутизация, к определенной схеме (то есть не является категоричным). По этой причине разработчикам приходится использовать React в паре с библиотекой маршрутизации и/или моделирования.

Хотя React можно применять как меньшую часть стека, разработчики чаще выбирают использование React-центричного стека, который состоит как из ядра React, так и из библиотек данных, маршрутизации и стилей, созданных специально для использования с React, включая следующие:

- Библиотеки моделей данных и бэкенда, такие как TanStack Query (<https://tanstack.com/query/latest>), Redux (<http://redux.js.org>), Recoil.js (<https://recoiljs.org/>), XState (<https://xstate.js.org/>) и Apollo (www.apollographql.com/).
- Библиотека маршрутизации — часто используется React Router (<https://github.com/reactjs/react-router>) или аналогичный маршрутизатор, реализованный во многих фреймворках.

- Библиотеки стилей — либо заранее определенный набор компонентов с примененными стилями, такими как Material UI (<https://mui.com/>) или Bootstrap (<https://react-bootstrap.github.io/>), либо библиотека, обеспечивающая удобство работы с CSS внутри компонентов React, например Styled-Components (<https://styled-components.com/>), Vanilla Extract (<https://vanilla-extract.style/>) или даже Tailwind CSS (<https://tailwindcss.com/>).

Фреймворки React для веб-сайтов

Другую категорию фреймворков React составляют полноценные серверные фреймворки, которые берут на себя многие задачи. Такие фреймворки делятся на две категории, но иногда фреймворк может относиться к обеим из них:

- Генераторы статических сайтов (SSG).
- Динамический React, генерируемый на сервере (SSR).

SSG представляют собой фреймворки, которые генерируют полностью статический веб-сайт, готовый к развертыванию на любом хосте для статических сайтов, что требует минимальных усилий и затрат на хостинг. Такое решение особенно популярно для небольших личных сайтов (например, блогов), но оно применяется и для сайтов небольших компаний и даже интернет-магазинов (которые не обязательно часто обновлять).

Фреймворки SSR более сложны. Они обеспечивают предварительный рендеринг приложения React на сервере перед передачей разметки HTML по каналу связи в браузеры посетителей. Это означает, что они хорошо подходят для SEO, обеспечивают хорошую совместную используемость и обладают множеством других преимуществ. Ниже названы три таких фреймворка:

- Gatsby — очень популярный фреймворк для блогеров, который также может подойти для разных видов статических веб-сайтов.
- Next.js — вероятно, самый популярный фреймворк React для веб-сайтов. Пригодится как для небольших статических сайтов, так и для огромных динамических сайтов.
- Remix — относительно новый представитель этого семейства, очень быстро набирает популярность на сверхбыстрых динамических сайтах React.

Все эти фреймворки — и многие, многие другие — являются разными расширениями React, каждое из которых функционирует в своих парадигмах. Они добавляют новую функциональность на базе React и иногда поставляются вместе с наборами компонентов React, которые помогут создать сайт, раскрывающий весь потенциал фреймворка.

Экосистема библиотек React растет с каждым днем. Кроме того, способность React описывать компоненты (автономные фрагменты пользовательского интерфейса) обеспечивает возможность повторного использования кода. Множество компонентов упаковываются в виде прм-модулей.

Отличный (и тщательно отобранный) список компонентов React для разных целей можно найти на странице <https://github.com/brillout/awesome-react-components>. В этом списке есть все, от UI-компонентов (включая множество элементов форм) до полных UI-фреймворков или вспомогательных средств разработки и инструментов тестирования.

Итак, вы узнали, что такое React, стек React и какое место эта библиотека занимает в высокоуровневых SPA-приложениях. Теперь можно использовать инструменты, построенные на базе React, для генерирования сложных веб-сайтов. Пора взяться за дело и написать первый код React.

1.4. ПЕРВЫЙ КОД REACT: HELLO WORLD

Рассмотрим первый код React: типичнейший пример, встречающийся при изучении языков программирования, — приложение Hello world. (Если этого не сделать, боги программирования нас покарают!)

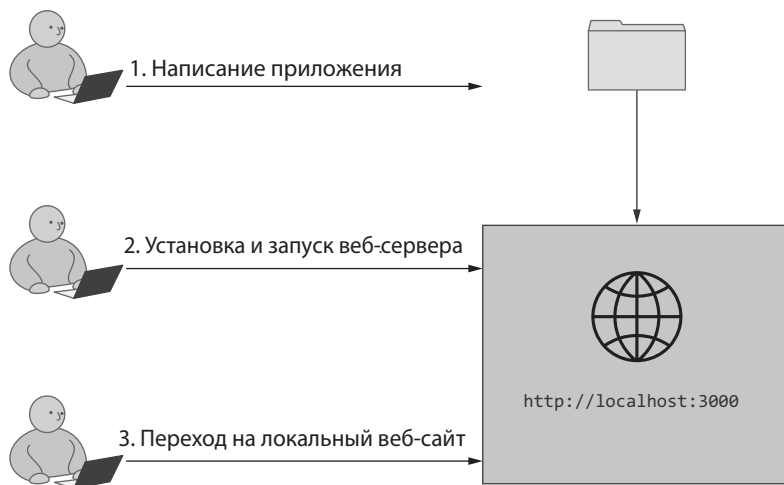


Рис. 1.4. Процесс создания первого приложения React включает всего три простых шага

Чтобы взяться за работу, вам кое-что понадобится. К счастью, так как мы разрабатываем приложение, которое выполняется в браузере, можно обойтись без разных компиляторов или библиотек. Вот краткий список того, что вам понадобится для работы:

- Текстовый редактор.
- Умение пользоваться системным терминалом.
- Установленная копия `npm` версии 5.2 и выше (если учесть, что версия 5.2 появилась в июле 2017 года, скорее всего, ваша копия `npm` подойдет, — конечно, если она у вас есть).
- Установленный современный браузер (подойдет любая новая версия Edge, Firefox, Chrome или Safari).

Если у вас есть все вышеперечисленное, можно переходить к первому примеру. При работе над другими примерами в дальнейшем вам также, скорее всего, не понадобится ничего, что выходило бы за рамки этого списка.

1.4.1. Результат

Проект будет выводить текст «Hello world!!!» (`<h1>`) на веб-странице. На рис. 1.5 показан результат (хотя, возможно, вы не испытываете большого энтузиазма и ограничитесь одним восклицательным знаком).

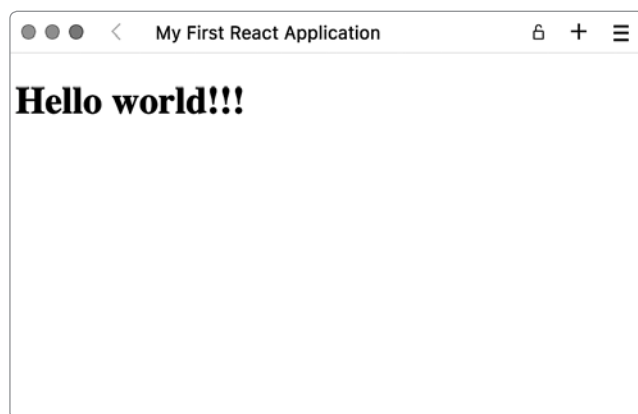


Рис. 1.5. Приложение «Hello World»

JSX в примере еще не используется, только простой код JavaScript. (Более того, JSX будет использоваться, начиная лишь с главы 3.)

ИЗУЧАЕМ REACT БЕЗ JSX

Хотя все разработчики React пишут на JSX, в браузерах выполняется только стандартный код JavaScript; код JSX непонятен браузеру. Вот почему полезно начинать знакомство с кодом React на стандартном JavaScript. Другая причина, по которой мы начинаем со стандартного JS, — показать, что JSX является необязательным, хотя, по сути, и стандартным языком React. Наконец, предварительная обработка JSX требует некоторого инструментария, но она упростит всю настройку, потому что вы будете меньше обращать внимание на то, что происходит под капотом, и займетесь более интересным делом — написанием отличных компонентов React.

Мы хотим, чтобы вы как можно быстрее начали работать с React, не тратя слишком много времени на подготовку в этой главе. В главе 2 вы узнаете, с чего начинается работа над новым приложением, а в главе 3 мы добавим JSX.

1.4.2. Написание приложения

Проект настолько прост, что он состоит из единственного файла HTML. Этот файл будет включать ссылки на новейшие версии React 18 (самая стабильная версия на момент написания книги) библиотек React Core и ReactDOM. Конечно, в него также включен небольшой фрагмент кода JavaScript для прорисовки очень простого приложения, которое мы строим.

Разметка `index.html` очень проста. Она начинается с включения библиотек в `<head>`. В элементе `<body>` создается контейнер `<div>` с идентификатором `root` и элементом `<script>` (в котором позднее будет размещен код приложения), как показано в листинге 1.1.

Листинг 1.1. Загрузка библиотек и кода React

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First React Application</title>
    <script
      ↗ src="//unpkg.com/react@18/umd/react.development.js"> | Импортирует
      ↘ </script> | библиотеку React
    <script src="//unpkg.com/react-dom@18/umd
      ↗ /react-dom.development.js"></script> | Импортирует библиотеку ReactDOM
    </head>
    <body>
      <div id="root"></div>
      <script type="text/javascript">
        ...
      </script>
    </body>
  </html>
```

Определяет пустой элемент `<div>` для монтирования пользовательского интерфейса React

Начало кода React для приложения Hello World

Создает узел `script`, который будет содержать код JavaScript

Просто введите этот код в текстовом редакторе и сохраните его в файле с именем `index.html` где-нибудь на компьютере.

Зачем создавать узел `<div>`, почему бы не выполнить рендер элемента React прямо в элементе `<body>`? Потому что это может создать конфликт с другими библиотеками и браузерными расширениями, которые манипулируют с телом документа. Более того, попытавшись присоединить элемент непосредственно к телу документа, вы получите предупреждение:

```
Warning: createRoot(): Creating roots directly with document.body is
  ➡ discouraged, ....
```

Вот еще одна сильная сторона React: полезные предупреждения и сообщения об ошибках!

ПРИМЕЧАНИЕ Предупреждения и сообщения об ошибках React не включаются в итоговую сборку в целях сокращения информационного шума, повышения безопасности и уменьшения размера дистрибутива. Итоговая сборка представляет собой минифицированный файл из библиотеки React Core, например `react.min.js`. Версия для разработки с предупреждениями и сообщениями об ошибках не минифицирована (например, `react.development.js`), как в этом примере.

Включая библиотеки в файл HTML, вы получаете доступ к глобальным объектам React и ReactDOM: `window.React` и `window.ReactDOM`. Вам понадобятся два метода этих объектов: для создания элемента (React) и для рендера его в контейнере `<div>` (ReactDOM), как показано в листинге 1.2. Чтобы создать элемент React, достаточно вызвать `React.createElement(elementName, data, children)` с тремя аргументами:

- `elementName` — тег HTML в виде строки (например, `'h1'`) или класс нестандартного компонента в виде объекта. Нестандартные компоненты в книге еще не рассматривались, мы начнем создавать их в главе 2.
- `data` — данные в форме атрибутов и свойств элемента. Пока свойства не нужны, поэтому в этом аргументе передается `null`. Мы вернемся к использованию свойств в главе 2.
- `children` — дочерние элементы или внутренний контент HTML/text (например, `Hello world!!!`).

Листинг 1.2. Создание и прорисовка элемента h1

```
const reactElement = React.createElement(
  'h1',
  null,
  'Hello world!!!'
);
```

Создает элемент React h1
с текстом "Hello world!!!"

```
const domNode = document.getElementById('root');
const root = ReactDOM.createRoot(domNode);
root.render(reactElement);
```

Прорисовывает элемент h1 в корневом контейнере

Создает корневой контейнер для приложения React, связанный с конкретным элементом DOM

Получает ссылку на элемент DOM с идентификатором root на странице

Код в листинге 1.2 заключается в тег `<script>` в файле HTML, созданном ранее, вместо многоточия, которое изначально было помещено в этот тег в качестве заполнителя. В этом листинге программа получает элемент React и сохраняет ссылку на этот объект в переменной `reactElement`. Переменная `reactElement` не содержит реальный узел DOM, это экземпляр компонента React `h1` (элемент). Ему можно присвоить любое имя, например `helloWorldHeading`. Иначе говоря, React предоставляет абстракцию над DOM.

После того как элемент будет создан и сохранен в переменной, создается контейнер приложения React (с именем `root`) на основе элемента DOM при помощи метода `ReactDOM.createRoot()`. Наконец, элемент React рендерится в корне при помощи метода `root.render()`, как показано в листинге 1.2.

При желании все эти этапы можно объединить в один вызов. Результат будет тем же, но в этом случае не используются три лишние переменные, как показано в листинге 1.3.

Листинг 1.3. Реализация с одной командой

```
ReactDOM
  .createRoot(document.getElementById('root'))
  .render(React.createElement('h1', null, 'Hello world!'));
```

Мы будем использовать развернутую форму из листинга 1.2, и полный файл HTML должен выглядеть, как показано в листинге 1.4.

Листинг 1.4. Создание и рендеринг элемента h1

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First React Application</title>
    <script src="//unpkg.com/react@18/umd/react.development.js"></script>
    <script src="//unpkg.com/react-dom@18/umd/react-
      dom.development.js"></script>
  </head>
  <body>
    <div id="root"></div>
    <script type="text/javascript">
```

```

const reactElement = React.createElement(
  "h1",
  null,
  "Hello world!!!"
);
const domNode = document.getElementById("root");
const root = ReactDOM.createRoot(domNode);
root.render(reactElement);
</script>
</body>
</html>

```

Вставленный код JavaScript
находится в нужном месте

Когда файл HTML будет готов, необходимо отправить контент в браузер, чтобы увидеть его работу.

1.4.3. Установка и запуск веб-сервера

На следующем шаге страница HTML передается браузеру. Что для этого понадобится? Нельзя ли просто открыть файл HTML в браузере? Из-за ограничений использования разных источников открыть в браузере файл, находящийся на жестком диске, чтобы он мог обращаться к контенту в других доменах (например, библиотекам React, загруженным из <https://unpkg.com>), не получится. Браузеры просто не позволяют это сделать. Вы можете попытаться открыть файл в браузере двойным щелчком, но откроется пустая страница.

В связи с этим контент придется отправлять при помощи локального веб-сервера разработки. Звучит страшно, но в наши дни это на удивление простая операция.

Если вы настроили узел так, как рекомендовалось во введении, этого будет достаточно для продолжения работы. Просто введите следующую команду в папке, в которой сохранен файл `index.html`:

```
$ npx serve
```

Вот и все. Возможно, вам будет предложено установить пакет (если вы еще этого не делали, просто подтвердите установку нажатием Enter), но через несколько секунд программа сообщит, что все готово и веб-сервер работает.

ЛОКАЛЬНЫЙ ВЕБ-СЕРВЕР РАЗРАБОТКИ

К сожалению, в самом первом примере вам придется настроить собственный локальный веб-сервер. Хотя эта задача очень проста, необходимость решать ее сейчас немного раздражает.

Если по какой-то причине приведенная команда не работает, можно использовать один из других способов настройки локального веб-сервера из текущей папки.

Если у вас установлен Node, попробуйте использовать следующую команду:

```
$ npx http-server -p 3000
```

Или, если на компьютере имеется Python 2, введите следующую команду:

```
$ python -m SimpleHTTPServer 3000
```

А если у вас установлен Python 3, команда будет выглядеть так (возможно, придется ввести python3 вместо python — это зависит от конфигурации):

```
$ python -m http.server 3000
```

Наконец, если у вас имеется локальная рабочая конфигурация PHP, попробуйте выполнить следующую команду:

```
$ php -S localhost:3000
```

Каждая из этих команд запускает на компьютере локальный веб-сервер в той папке, в которой была выполнена команда. В результате файл HTML будет доступен по адресу `http://localhost:3000`.

1.4.4. Переход на локальный веб-сайт

Когда веб-сервер заработает, вы сможете перейти в браузере на сайт по следующему адресу:

```
http://localhost:3000
```

Теперь вы видите, как работает приложение, и это должно быть похоже на рис. 1.5 в начале этого раздела.

На рис. 1.6 изображена вкладка **Elements** в инструментах разработчика в браузере с выбранным элементом `<h1>`. Вы понимаете, что React что-то изменил, потому что в исходном файле HTML в корневом узле не было элемента `<h1>` — он был пустым.

Поздравляем! Вы только что реализовали свое первое приложение React!

Начиная со следующей главы, мы не будем создавать свои приложения React таким способом. Вместо этого мы будем использовать небольшой инструмент, который быстро сгенерирует и настроит основную структуру приложений React за нас, что намного упростит процесс. Кроме того, это решит проблему доставки контента, чтобы больше не приходилось беспокоиться о веб-серверах.

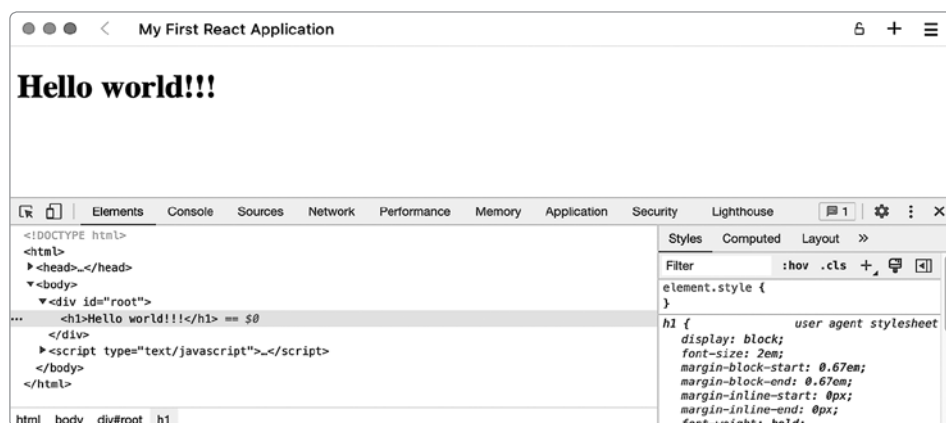


Рис. 1.6. Анализ веб-приложения Hello World, сгенерированного React

Отдельный файл JavaScript

Код JavaScript можно абстрагировать в отдельный файл, а не включать сценарий напрямую в файл HTML (см. листинг 1.1). Например, можно создать файл с именем `script.js`, а затем скопировать в него весь код из листинга 1.2 или 1.3. Затем в файле HTML необходимо добавить ссылку на файл `script.js` после `<div id="root">`, не включая сам сценарий:

```
<div id="root"></div>
<script src="script.js"></script>
```

1.5. ВОПРОСЫ

1. React — это полноценный фреймворк, и многие приложения можно создавать, не используя только React. *Да* или *нет*?
2. Какую основную задачу решает React?
 - а) Получение данных с сервера.
 - б) Создание красивых виджетов HTML.
 - в) Рендеринг динамических данных на уровне пользовательского интерфейса.

3. Какой из следующих методов осуществляет рендеринг компонентов React в DOM (внимание, это каверзный вопрос)?
 - a) `ReactDOM.appendRoot(...).render()`
 - b) `ReactDOM.renderRoot(...).render()`
 - c) `ReactDOM.createRoot(...).render()`
 - d) `ReactDOM.launchRoot(...).render()`
4. Чтобы использовать React в одностраничных приложениях, необходим Node.js на сервере. *Да* или *нет*?
5. Для рендеринга элементов React на веб-странице необходимо добавить файл `react-dom.js`. *Да* или *нет*?

ОТВЕТЫ

5. *Да*. Потребуется библиотека `ReactDOM`.
4. *Нет*. Можно использовать любую технологию бэкенда.
3. `ReactDOM.createRoot(...).render()`.
интерфейса (вариант с).
2. Хотя в React можно создавать красивые виджеты HTML, главной задачей React является *рендеринг динамических данных на уровне пользовательского интерфейса*.
1. *Нет*. При создании подавляющего большинства приложений React почти всегда используются другие фреймворки или библиотеки.

ИТОГИ

- React для веб-приложений включает библиотеки React Core и ReactDOM. Библиотека React Core ориентирована на единообразие построения и совместного использования композитных UI-компонентов, использующих JavaScript и (не обязательно) JSX. С другой стороны, чтобы работать с React в браузере, можно использовать библиотеку ReactDOM, которая содержит методы для рендеринга DOM, а также рендеринга на стороне сервера.
- React использует декларативный стиль; это всего лишь представление, или слой пользовательского интерфейса.

- React использует компоненты, которые создаются вызовом `ReactDOM.createRoot()`.
- Для разработки и организации пользовательских интерфейсов в React используется чистый JavaScript.
- Хотя использовать JSX (HTML-подобный синтаксис для объектов React) на базе React не обязательно, его используют все.
- React может интегрироваться в веб-стек на разных уровнях, от небольшого виджета на странице до основы всего сайта.
- React не подходит на все случаи жизни; это всего лишь UI-прослойка веб-приложения, которое состоит из множества частей. React часто используется совместно с библиотеками данных (такими, как Redux или XState).

2

Первые шаги с React

<https://t.me/portalToIT>

В ЭТОЙ ГЛАВЕ

- ✓ Создание проекта React
- ✓ Вложение элементов
- ✓ Создание класса компонента
- ✓ Работа со свойствами

В этой главе вы узнаете, как создать новый проект React и нестандартные компоненты для рендеринга HTML. На концепциях, рассмотренных в этой главе, будет строиться весь дальнейший материал.

Сначала вы узнаете, как создать новый проект React. При этом мы покажем, как создавать собственные проекты React и как использовать систему шаблонов React для быстрой реализации всех примеров и проектов, над которыми мы будем работать в этой книге. Просто невероятно: как всего одной строкой можно загрузить готовый к выполнению и настроенный код!

В начале работы над первым проектом React мы разберем некоторые фундаментальные концепции React, которые вы будете часто использовать, в том

числе элементы, компоненты и свойства. Если коротко: элементы являются экземплярами компонентов, которым могут передаваться свойства. В каких ситуациях их нужно применять и почему? Ответов на эти вопросы придется подождать до раздела 2.3, а сейчас речь пойдет о том, как создать новое веб-приложение React.

ПРИМЕЧАНИЕ Исходный код примеров этой главы доступен по адресу <https://rq2e.com/ch02>. Как будет показано в разделе 2.2, вам не придется ничего загружать вручную. Вы можете воспроизвести все примеры этой и последующих глав прямо в командной строке всего одной командой.

2.1. СОЗДАНИЕ НОВОГО ПРИЛОЖЕНИЯ REACT

В этом разделе мы познакомимся с волшебной командной строкой, в которой все примеры React будут работать как часы. Всего тремя короткими командами за пару минут вы сможете загрузить полностью функциональную заготовку веб-приложения React, откомпилировать его, запустить через веб-сервер и посмотреть в браузере (схема процесса изображена на рис. 2.1).

Если у вас установлена современная версия Node (и `npm`), как рекомендовалось ранее, введите в терминале следующую команду:

```
$ npm create-react-app name-of-app
```

ПРИМЕЧАНИЕ `npm` — это не опечатка. `npm` — программа для исполнения пакетов, поставляемая с `npm`. Она позволяет выполнять команды с использованием пакетов, находящихся только в папке проекта, и/или команды, которые будут загружаться динамически в случае необходимости.

Выполните эту команду, чтобы настроить новое приложение React. При первом выполнении `npm` запросит подтверждение на загрузку утилиты `create-react-app` (просто нажмите Enter). Эта стадия соответствует шагам 1.a и 1.b на рис. 2.1. При последующих выполнениях команды вопросов уже не будет.

ПРИМЕЧАНИЕ В дальнейшем для программы `create-react-app` мы будем использовать сокращение `CRA`.

Команда создает новую папку с переданным именем (`name-of-app` в предыдущем примере). Внутри папки утилита инициализирует новый проект Git, загружает

необходимые ресурсы для приложения, после чего загружает и локально устанавливает все зависимости, необходимые для проекта.

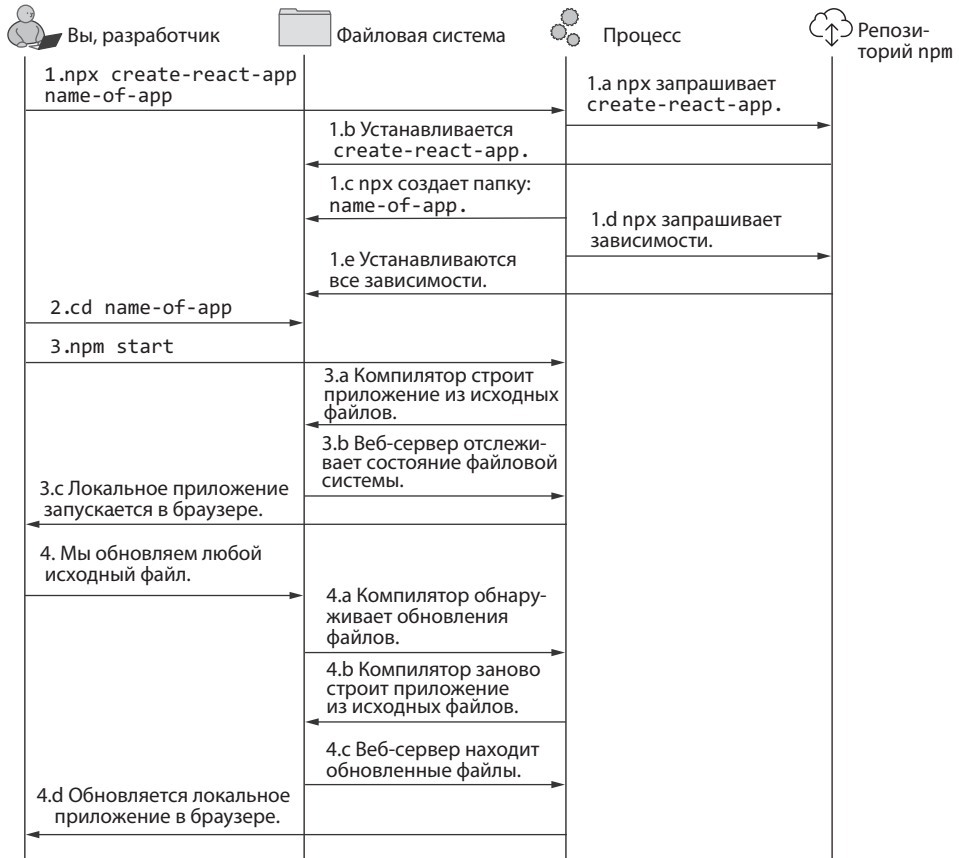


Рис. 2.1. Три команды для создания рабочего приложения React с нуля. Если после создания обновить исходные файлы, система автоматически перекомпилирует и обновит приложение в браузере

Команда еще некоторое время работает — около 1–3 минут в зависимости от сложности проекта и состояния сети. Результат ее выполнения должен выглядеть примерно так:

```

Success! Created name-of-app at <folder>
Inside that directory, you can run several commands:

```

name-of-app и <folder>
заменяются реальным именем
и путем к папке проекта

```

→ npm start
  Starts the development server.

  npm run build
    Bundles the app into static files for production.

  npm test
    Starts the test runner.

  npm run eject
    Removes this tool and copies build dependencies, configuration
    files and scripts into the app directory. If you do this, you
    can't go back!

  We suggest that you begin by typing:

  cd name-of-app ← Эта команда переходит в папку
  npm start       вновь созданного проекта

  Happy hacking! ← Спасибо! Оставайтесь на светлой стороне!

  Первая из четырех команд, которые можно выполнить
  в приложении (о том, что она делает, мы расскажем позже)

```

Три другие команды будут описаны в следующем подразделе

Замечательно. Если в выводе появилась команда `yarn` вместо `npm`, не беспокойтесь. Почему — читайте во врезке.

Альтернативы npm

Для проектов JavaScript существует ряд популярных менеджеров пакетов, которые используют те же репозитории пакетов и структуру, но немного другие команды. Самым популярным менеджером, безусловно, остается `npm`, но среди альтернатив выделяются `Yarn` и `pnpm`. `npm` заранее устанавливается с Node; это менеджер пакетов по умолчанию, который используют многие разработчики.

Тем не менее можно выбрать другой менеджер пакетов, с упрощенной структурой команд. Для целей этой книги различия между `npm` и альтернативами несущественны, если не считать синтаксиса при вводе команд. Если у вас установлен менеджер `Yarn` или `pnpm`, то, скорее всего, установлен и `npm`, так что вы будете использовать этот вариант.

Если вы хотите использовать один из других упомянутых менеджеров пакетов, изучите документацию с описанием команд:

- Yarn: <https://classic.yarnpkg.com/lang/en/docs/cli/run/>
- pnpm: <https://pnpm.io/cli/run>

Выполним инструкции из фрагмента кода выше и запустим следующие две команды:

```
$ cd name-of-app  
$ npm start
```

Начинается третья часть волшебства. Запускается сервер разработки React, который компилирует все используемые файлы и ресурсы (действие 3.a на рис. 2.1) и запускает локальный веб-сервер разработки (действие 3.b на рис. 2.1). Через несколько секунд в командной строке выводится сообщение, которое выглядит примерно так:

```
Compiled successfully!  
You can now view name-of-app in the browser.  
Local:      http://localhost:3000
```

Более того, приложение уже должно запуститься в браузере, так как команда также открывает окно браузера с соответствующим URL (действие 3.c на рис. 2.1). Если этого не произошло, просто введите адрес `localhost:3000` в браузере, чтобы увидеть приложение. В окне браузера выводится приложение React (как показано на рис. 2.2), созданное по шаблону. Этот шаблон используется по умолчанию для новых приложений React, для которых не выбран конкретный шаблон. Шаблоны более подробно рассматриваются в разделе 2.1.3.

Заметим, что последняя команда `npm start` работает непрерывно и остается активной в терминале. Она отслеживает исходные файлы, перекомпилирует все приложение при изменении исходных файлов и даже перезагружает браузер обновленным приложением (действия 4–4.d на рис. 2.1)! Чистая магия.

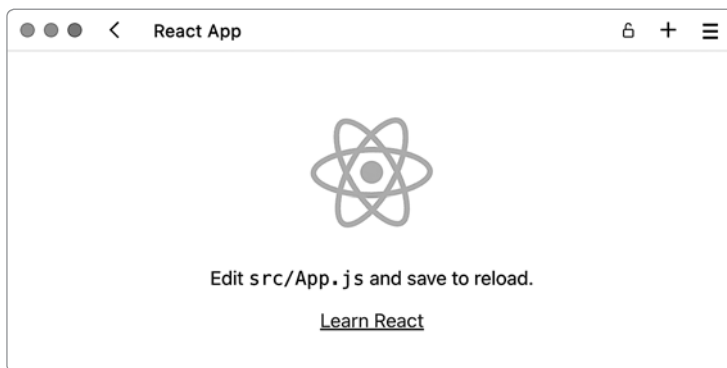


Рис. 2.2. Приложение React по умолчанию, запущенное в новом проекте React. Скорее всего, у фактического приложения будет темная тема с белым текстом на черном фоне. Приведенный снимок экрана инвертирован для удобства печати

Если вам потребуется прервать работу этой команды, просто нажмите `Ctrl+C` в терминале, чтобы вернуться к обычному приглашению терминала. Однако в этом случае приложение перестанет работать, потому что вы остановили локальный веб-сервер.

Возможно, вы заметили, что в предыдущем выводе при создании приложения указана не только команда `start`, которую мы использовали, но и три другие команды: `build`, `test` и `eject`. Все четыре команды мы подробно рассмотрим в следующем подразделе.

2.1.1. Команды проектов React

Итак, исходный код приложения React находится в системе и вам, вероятно, потребуется взаимодействовать с ним. Нетрудно предположить две основные операции: проверить проект в процессе разработки и развернуть приложение на веб-сервере. Возможно, вы также захотите выполнить все тесты для приложения, чтобы убедиться, что все работает как задумано. Наконец, может потребоваться выйти за рамки CRA, чтобы работать с ядром на более низком уровне. CRA абстрагирует некоторые процессы, о которых на первых порах вам знать не обязательно, но когда приложение станет более мощным, вам понадобится работать с внутренней конфигурацией приложения. Для этих четырех целей новое приложение React, созданное с использованием CRA, поддерживает следующие четыре команды:

- **start** — запускает локальный веб-сервер разработки и непрерывно компилирует проект при обнаружении изменений, поставляя его любому локальному браузеру.
- **build** — компилирует все ресурсы в рабочий пакет, готовый к развертыванию на подходящем веб-хосте.
- **test** — запускает тестировщик, который проводит все модульные тесты, предусмотренные проектом.
- **eject** — предоставляет доступ к внутренним процессам и возможность полной настройки конфигурации.

Рассмотрим эти команды по очереди и обсудим, как и когда их использовать.

start

Команда **start** — основная команда, которая используется при каждом запуске нового проекта или загрузке текущего для продолжения работы с ним. В начале

сеанса разработки выполните команду **start** в терминале, после чего вы сможете писать код в редакторе, причем обновленный контент будет автоматически от-правляться в браузер.

Команда **start** непрерывно строит проект в фоновом режиме, используя версию React для разработки (development version) и ее вспомогательные библиотеки. Эта версия отличается от рабочей версии React (production version), которая используется в команде **build**. Версия разработки включает намного более содержательные сообщения об ошибках и предупреждения, а также средства отладки приложения, выполняемого в браузере. Однако в связи с этим версия разработки React намного больше по размеру, так что использовать ее для выпуска приложения не стоит. Она только увеличит размер приложения без необходимости и затруднит его использование.

Команда **start** также перезагружает приложение в браузере в процессе работы, но не просто перезагружает окно браузера целиком, а действует намного тоньше. React старается перезагрузить только изменившиеся фрагменты логики, а остальные компоненты оставляет в исходном виде. Например, если вы нажали кнопку, чтобы свернуть секцию, которая открыта по умолчанию при запуске приложения, React сможет внедрить обновленный код при сохранении состояния в браузере, так что секция останется свернутой при обновлении логики.

build

Эта команда выполняется, когда вы готовы развернуть приложение на реальном веб-сервере, чтобы пользователи могли с ним работать. При выполнении команды **build** используется рабочая версия React, намного более компактная и оптимизированная для развертывания. Результат сборки помещается в папку **/build**.

По умолчанию на этом все, но при желании можно настроить прямое развертывание в облачном хостинге из команды **build**. Чтобы узнать, как это сделать, обращайтесь к документации провайдера облачного веб-хостинга. В книге эта команда не используется, так как мы будем применять другой шаблон для развертывания приложений в проекте, а развертывание в облаке рассматривать как дополнительную возможность.

test

Чтобы запустить все модульные тесты, предусмотренные в проекте, выполните команду **test**. Ее можно выполнить и с пустым шаблоном по умолчанию, потому что даже этот шаблон включает файл с тестом.

eject

Команда `eject` может быть опасной, поскольку она необратима. Выполнив ее, вы получите доступ к расширенному набору настраиваемых параметров React, но потеряете возможность автоматического обновления всех задействованных инструментов. В книге эта команда не рассматривается, но мы кратко обсудим ее в разделе 2.1.4, при анализе достоинств и недостатков.

2.1.2. Структура файлов

При создании проекта с помощью CRA почти всегда используется одна и та же структура файлов. Нестандартные шаблоны могут ее менять, но на практике это встречается редко. Структура включает следующие важные элементы:

```
/
public/
  index.html
src/
  index.js
  App.js
package.json
```

Если у вас есть эти две папки и четыре файла, можно двигаться дальше.

Папка `public` предназначена для файлов, которые будут поставляться напрямую через веб-сервер. К их числу относится файл `index.html`, который предоставляет все приложение, а также двоичные файлы, которые не нужно упаковывать в приложение, например контент, требуемый файлом `index.html` напрямую (значок сайта, CSS (Cascading Style Sheets), шрифты или графика для совместного использования) и большие файлы (видео, изображения и т. д.).

В папке `src` (источник) будет сохранен весь упакованный код JavaScript, а также весь остальной контент, который должен быть упакован в один пакет. В основном это код JavaScript, но теоретически здесь также могут храниться CSS, значки, небольшие изображения, файлы JSON и т. д. Упаковка начинается с файла `index.js` в папке источника. Основное приложение обычно размещается в файле с именем `App.js` или `app.js`, как вам больше нравится, но в остальном вы можете проявить гибкость. Некоторые шаблоны разделяют содержимое папки `src` по вложенным папкам, что необходимо для организации больших проектов.

Основной файл конфигурации проекта называется `package.json`, как того требуют `npm` и `Yarn`. Это начальный файл проекта, который также определяет зависимости и команды, которые вы можете выполнять, как описано в разделе 2.1.1.

Корневая папка часто содержит множество других файлов конфигурации, необходимых для библиотек, включаемых в проект. Нередко встречаются нестандартные шаблоны с 20+ других конфигурационных файлов в корне проекта. Рассмотрим подробнее, что такое нестандартные шаблоны и для чего они нужны.

2.1.3. Шаблоны

Приложение по умолчанию на рис. 2.2 выглядит симпатично, но пользы от него не много. Шаблон по умолчанию позволит создать простое веб-приложение в подобном стиле, но всегда ли это именно то, что нужно? Если вы хотите создать веб-приложение на базе конкретного технологического стека или использовать React особым образом, скорее всего, стоит выбрать другой стартовый шаблон с правильными настройками.

При использовании CRA можно задать используемый шаблон. По умолчанию используется шаблон с (вращающимся) логотипом React, который вы уже видели. Если вы хотите выбрать другой шаблон, передайте его в аргументе:

```
$ npx create-react-app name-of-app --template name-of-template
```

Можно использовать только имя уже существующего шаблона; если шаблон не существует, приложение остановится. Часто разработчики вообще не считают нужным выбирать шаблон и работают с шаблоном по умолчанию. Но если вы знаете, что вам нужна особая конфигурация или код должен начинаться с конкретного состояния, используйте шаблон, который предусматривает все это. Примеры часто используемых шаблонов:

- Минимальные шаблоны с еще меньшей функциональностью, чем у шаблона по умолчанию, например `--template minimal`. Этот шаблон не включает графику, CSS, тесты, основные веб-компоненты и другие удобные функции, входящие в шаблон по умолчанию.
- Разновидности шаблона по умолчанию или минимального шаблона, использующие TypeScript, например `--template typescript` или `--template minimal-typescript`. Могут быть полезны для создания нового проекта, использующего TypeScript.
- Сложные шаблонные конфигурации, созданные другими разработчиками, полезные, если у вас имеется стек определенных зависимостей, уже встроенных в создаваемое приложение. Например `--template redux-typescript`, изначально включающий Redux и TypeScript, или `--template rb`, популярный шаблон React, включающий множество проверенных библиотек, в том числе

Redux с Redux-Saga, компонентами со стилевым оформлением, ESLint, husky и многие другие.

Одна из самых ценных особенностей системы шаблонов для CRA — полная децентрализация системы. Любой желающий может опубликовать пакет в `npm` и структурировать его таким образом, что его можно будет использовать в качестве основы для построения приложений. Конечно, это обстоятельство становится и одним из недостатков системы. Непонятно, насколько полезен найденный шаблон `npm` и что он делает. В такой ситуации стоит довериться коллективному разуму — если какая-то вещь популярна, то скорее всего, она полезна.

Одно из преимуществ факта, что любой разработчик может публиковать свои шаблоны в `npm`, заключается в том, что к числу таких разработчиков принадлежат и авторы этой книги. Мы будем использовать нестандартные шаблоны React для всех примеров и проектов нашей книги. Вскоре мы вернемся к этому вопросу, а пока обсудим достоинства и недостатки CRA.

2.1.4. Достоинства и недостатки

Использование CRA для создания новых приложений React имеет много преимуществ, но как обычно, у этих преимуществ есть последствия. Многие сильные стороны уже упоминались выше, но на всякий случай перечислим их еще раз:

- *Простота* — новые приложения создаются с меньшими усилиями. Вы получаете транспилиацию JavaScript XML (JSX), упаковку, тестирование, автоматическую перезагрузку и многое другое практически бесплатно, и вам не приходится разбираться с многочисленными взаимозависимостями.
- *Обновляемость* — переходить на новые версии React и всех остальных используемых библиотек легко. Мы еще не говорили о том, как это делать, но это несложно. Просто выполните команду `npm install --exact react-scripts@VERSION`, чтобы обновить весь проект до конкретной версии сценариев React. Подробности см. в журнале изменений `react-scripts`.
- *Сообщество* — благодаря изобилию доступных шаблонов CRA и простому способу создания новых шаблонов, скорее всего, вы найдете готовый шаблон с подходящей комбинацией инструментов и вам не придется разбираться, как их правильно объединить.
- *Настройка* — у разработчика остается возможность добавить любые другие плагины и библиотеки, необходимые для его проекта, поверх подходящих

шаблонов. Ваш проект должен взаимодействовать с Google Maps и Amazon Web Services (AWS)? Просто добавьте их библиотеки, и можно приступать к работе.

Тем не менее у шаблонов есть и недостатки. Некоторые из них можно упомянуть вкратце или вообще не обращать на них внимания, но иногда приходится искать другие конфигурации, которые не может предоставить CRA. Рассмотрим некоторые из таких случаев:

- *Понимание* — не настраивая проект с нуля, вы не будете знать его подробности. Если вам требуется уникальная конфигурация, а до сих пор вы полагались на CRA, у вас могут возникнуть трудности, потому что вы никогда не обращали внимания на мелочи. Но такова двойственная природа абстракций: выигрыш обеспечивается за счет того, что вам не приходится разбираться в подробностях реализации.
- *Контроль* — вы теряете контроль над тем, какие библиотеки используются в проекте. CRA в настоящее время использует webpack и BabelJS для упаковки и транспиляции JSX, но это ни в коем случае не означает, что других вариантов быть не может. Недавно появились такие инструменты, как esbuild, Bun, SWC и Rome, частично предназначенные для тех же целей, но вы не сможете легко переключиться на них. Вам приходится использовать технологический стек, который сейчас за вас выбирает CRA. С другой стороны, это можно считать преимуществом, потому что когда другой инструмент станет стандартным вариантом и, возможно, даже превзойдет Babel, CRA перейдет на него — и вам не придется ломать над ним голову. Если вы привязаны к конкретному стеку, вам придется создавать проект с нуля. Другой вариант — использовать команду `eject`, как описано в разделе 2.1.1, которая открывает доступ к дополнительным возможностям настройки и контроля в ущерб удобству обновления.
- *Интеграция* — если вы хотите интегрировать приложение в конфигурацию на стороне сервера, CRA вам с этим не поможет. Для проектов, базирующихся на фреймворках веб-сайтов, как описано в главе 1, придется использовать конфигурации, предоставляемые этими фреймворками, а не CRA.

Взвесив все перечисленные плюсы и минусы, мы пришли к выводу, что CRA идеально подойдет для начинающих разработчиков. CRA упрощает работу и избавляет от головной боли. А когда у вас накопится опыт, можно начинать эксперименты с другими инструментами. Именно по этой причине мы использовали CRA в примерах и проектах этой книги.

2.2. О ПРИМЕРАХ В КНИГЕ

Как уже говорилось, мы используем CRA для всех проектов и почти всех примеров в этой книге. Единственным исключением останется первый пример, приведенный в главе 1.

Имена всех шаблонов, созданных для этой книги, имеют следующую структуру:

rqXX-ИМЯ

Конечно, `rq` — сокращение от «React Quickly» («React быстро»). `xx` заменяется номером главы, а последняя часть представляет собой короткое имя каждого примера. Для каждого примера и проекта, использующего CRA, имя шаблона и краткое описание его использования приводятся во врезке следующего вида:

Репозиторий: `rq02-nesting`

Этот пример находится в репозитории `rq02-nesting`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq02-nesting --template rq02-nesting
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq02-nesting>

Иногда примеры содержат несколько вариаций исходного кода; в таких случаях каждая разновидность приводится с отдельным шаблоном, как показано выше. Также в описаниях некоторых примеров приводятся рекомендации для домашней работы. В таких случаях шаблон будет указан как отправная точка для домашней работы, а другой шаблон будет содержать *одно* из возможных решений. Вы можете взять шаблон решения за образец или сравнить его с собственным решением. Все домашние задания имеют бесконечное количество решений, и даже если ваша работа не совпадает с шаблоном, это не значит, что она неверна, — просто она другая.

Для простоты имя шаблона часто может использоваться как имя приложения. Допустим, вы начинаете работу над следующим примером в книге. Шаблону

присвоено имя `rq02-nesting`, поэтому оно также будет использоваться в качестве имени веб-приложения:

```
$ npx create-react-app rq02-nesting --template rq02-nesting
```

Просто введите эту команду в консоли — и все готово для работы над решением задачи. Можно действовать иначе: просто прочитать главу и просмотреть код в листингах из книги. Если вам что-то покажется странным или захочется запустить код и немного поэкспериментировать, создайте решения на базе шаблонов и наблюдайте за работой примеров.

А теперь можно перейти к следующему разделу, в котором речь пойдет о вложении.

2.3. ВЛОЖЕНИЕ ЭЛЕМЕНТОВ

Пора вернуться к созданию приложений React — в конце концов, именно этим мы и собираемся заниматься. И для начала слегка усложним задачу по сравнению с полезным, но очень упрощенным примером из главы 1. В ней вы научились создавать элементы React. Напомним, что для этой цели используется метод `React.createElement()`. Например, элемент ссылки может создаваться так:

```
const reactLinkElement = React.createElement("a",
  { href: "http://react.dev" },
  "React Website"
)
```

Все хорошо, пока вы создаете только один элемент. Проблема в том, что большинство пользовательских интерфейсов обычно содержат несколько элементов, иначе как бы вы включили в страницу хоть какую-то информацию, кроме одного абзаца? Решением для создания многоэлементных структур с иерархической природой является вложение элементов. В предыдущей главе вы реализовали свой первый код React, создав один элемент React и отрендерив его в DOM методом `ReactDOM.createRoot().render()`:

```
const title = React.createElement("h1", null, "Hello world!");
const domElement = document.getElementById("root");
ReactDOM.createRoot(domElement).render(title);
```

Важно заметить, что `ReactDOM.createRoot().render()` получает в аргументе только один (корневой) элемент — `reactElement` в данном примере. Полученное приложение показано на рис. 2.3.

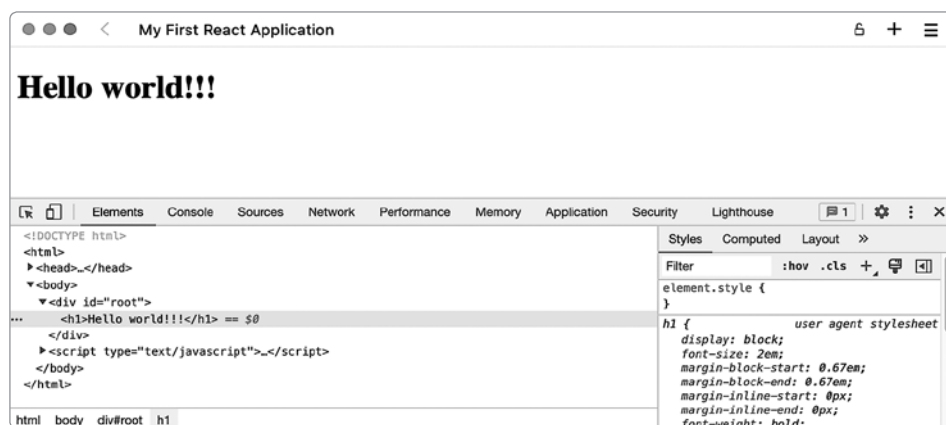


Рис. 2.3. В браузере рендерится один элемент заголовка. Мы открыли средства разработчика, чтобы показать структуру нижележащей разметки HTML. О том, как открыть средства разработчика, можно узнать из документации браузера; скорее всего, это делается нажатием сочетания клавиш `Ctrl+Alt+I` / `Cmd+Opt+I`

Репозиторий: `rq02-nesting`

Этот пример находится в репозитории `rq02-nesting`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq02-nesting --template rq02-nesting
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq02-nesting>

При извлечении шаблона `rq02-nesting` вы получите приведенное выше приложение, но на этот раз с использованием `CRA`, и вам не нужно добавлять библиотеки вручную и писать HTML, как делалось в главе 1.

Вспомните, что при использовании `createElement` в третьем аргументе передается дочерний элемент. В данном случае это простой текст. Однако этот текст на самом деле является еще одним элементом, по крайней мере в полученной модели DOM. В React он не имеет конкретного типа, но при этом до определенной степени функционирует как элемент. Это отношение можно представить на очень простой диаграмме, как показано на рис. 2.4.

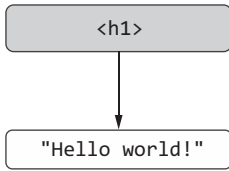


Рис. 2.4. Серый прямоугольник представляет настоящий элемент React, а белый — элемент text

2.3.1. Иерархия узлов

Прежде чем разбираться с созданием сложных структур HTML, необходимо определиться с базовой терминологией. Документ HTML часто представляется в виде перевернутого дерева, как показано на рис. 2.5. Узлы дерева обычно описываются в терминах семейных отношений (родитель, потомок и т. д.).

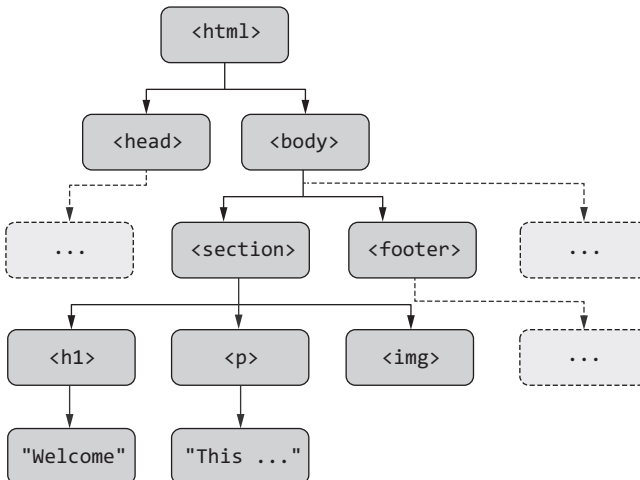


Рис. 2.5. Структура документа HTML в виде перевернутого дерева, узлы которого находятся друг с другом в отношениях, подобных семейным, — родитель, потомок и т. д.

При описании структуры дерева используется следующая терминология:

- *Узел* — к этой категории относится любой компонент дерева, включая как элементы HTML, так и текстовые узлы. Все прямоугольники на рис. 2.5 являются узлами. Два нижних прямоугольника — текстовые узлы, а все остальные — узлы элементов.
- *Корень* — первый (самый верхний) узел является корнем дерева. На рис. 2.5 корневым является узел `<html>`.

- *Родитель* — узел, находящийся непосредственно над заданным узлом, является его родителем. Каждый узел дерева имеет только одного родителя. Узел, находящийся над родителем, может называться прародителем, и т. д. На рис. 2.5 родительским узлом `<body>` является узел `<html>`. У корневого узла нет родителя; это единственный узел дерева без родителя.
- *Дочерний узел* — любой узел, находящийся непосредственно под заданным узлом, называется дочерним по отношению к этому узлу. Узел может иметь несколько дочерних узлов. Дочерними узлами `<section>` являются узлы `<h1>`, `<p>` и `<img>`. Не у всех узлов есть дочерние узлы; например, элемент `<img>` не имеет дочерних узлов. У текстовых узлов никогда не бывает дочерних узлов.
- *Одноуровневый узел* — два узла, имеющих одного родителя, называются одноуровневыми. Узел `<p>` имеет два одноуровневых узла: `<h1>` и `<img>`.
- *Предки* — родитель узла, его родитель, родитель его родителя и т. д. (вплоть до корня) называются предками узла. Узел `<h1>` имеет трех предков: `<section>`, `<body>` и `<html>`.
- *Потомки* — дочерние узлы заданного узла, все их дочерние узлы, дочерние узлы их дочерних узлов и т. д. называются потомками узла. У узла `<section>` пять потомков: три непосредственных дочерних узла, а также два текстовых узла, которые являются дочерними для первых двух.
- *Вложение* — процесс организации узлов дерева и принятия решений о том, какие узлы должны являться дочерними по отношению к другим узлам, в результате чего создается дерево документа. На рис. 2.5 мы решили вложить узлы `<h1>`, `<p>` и `<img>` в узел `<section>`.

2.3.2. Простое вложение

Предположим, вам нужно, чтобы слово «world» в строке «Hello world!» выводилось курсивом, но при этом содержалось в элементе `h1`. Как показано на рис. 2.6, для этого следует создать элемент `em` со строкой "world" в качестве дочернего элемента и другой элемент `h1` с тремя дочерними элементами:

- строкой "Hello " (обратите внимание на пробел в конце);
- уже упомянутым элементом `em`;
- строкой "!".

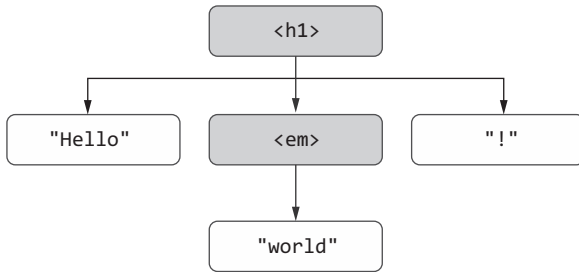


Рис. 2.6. Для рендеринга приветственного сообщения с выделением потребуются два элемента React и три текстовых элемента

При использовании `React.createElement` это будет выглядеть так:

```
const world = React.createElement("em", null, "world");
const title = React.createElement("h1", null, "Hello ", world, "!")
```

← `createElement` с тремя аргументами
`createElement` с пятью аргументами, последние три — его дочерние элементы

Как видите, теперь в `createElement` передаются пять аргументов: сначала тип элемента, затем свойства и, наконец, дочерние узлы элемента. При вызове можно передать любое количество аргументов для дочерних узлов. Также можно передать дочерние узлы в виде массива:

```
const title = React.createElement("h1", null, ["Hello ", world, "!"])
```

В нашем случае нет смысла помещать элементы в массив, прежде чем передавать их в аргументе, но если бы у нас уже был массив элементов, можно было бы просто передать его в аргументе. Если обобщить все сказанное (без использования массива), скрипт целиком будет выглядеть, как показано в листинге 2.1.

Листинг 2.1. Выделение слова `world`

```
import React from "react";
import ReactDOM from "react-dom/client";
const world = React.createElement("em", null, "world");
const title = React.createElement("h1", null, "Hello ", world, "!");
const domElement = document.getElementById("root");
ReactDOM.createRoot(domElement).render(title);
```

На рис. 2.7 показано итоговое представление в браузере.

Но что, если потребуется разместить элемент *после* `h1`, а не *внутри*? Одноуровневые элементы рассматриваются в следующем разделе.

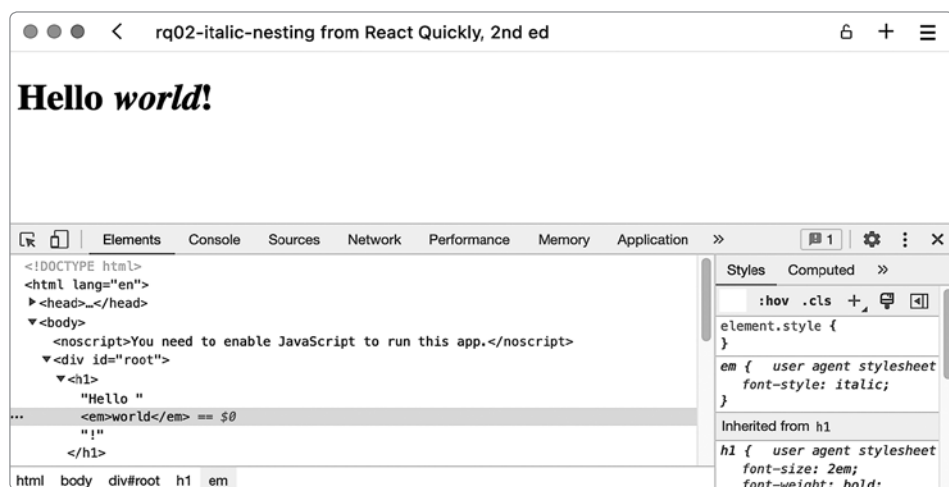


Рис. 2.7. Выделенное приветствие в браузере. Обратите внимание на базовую структуру HTML в инструментах разработчика

Репозиторий: rq02-nesting-italic

Этот пример находится в репозитории `rq02-nesting-italic`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq02-nesting-italic --template rq02-nesting-italic
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq02-nesting-italic>

2.3.3. Одноуровневые элементы

Часто на верхнем уровне располагается только один элемент React. Это относится и к методу `ReactDOM.createRoot().render()` — только один элемент можно отрендерить в DOM в качестве корневого. Далее вы также увидите, что нестандартные компоненты могут возвращать только один элемент.

Но что, если требуется вывести заголовок, *а затем* ссылку после него, как в предыдущем примере (рис. 2.8)? Это будут два разных элемента, расположенных рядом друг с другом, и их нельзя напрямую отрендерить вызовом `ReactDOM.createRoot().render()`.

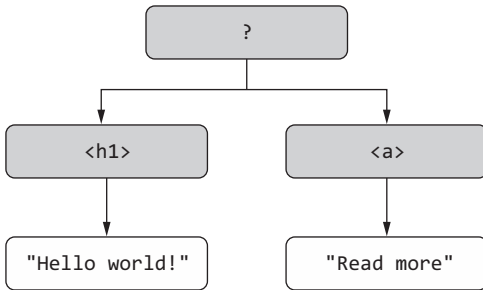


Рис. 2.8. Два одноуровневых элемента React, которые требуется отрендерить в корне

Вместо этого придется упаковать их в другой элемент (который займет место ? на рис. 2.8). В таком случае доступны два варианта. Первый — использовать нейтральный элемент DOM; он реализуется просто, но добавляет «физический» элемент в выходную разметку HTML. Альтернативное решение — использовать элемент React **Fragment**, который работает как любой другой элемент, но не выводит никакую разметку HTML. Различия между этими двумя решениями представлены на рис. 2.9.

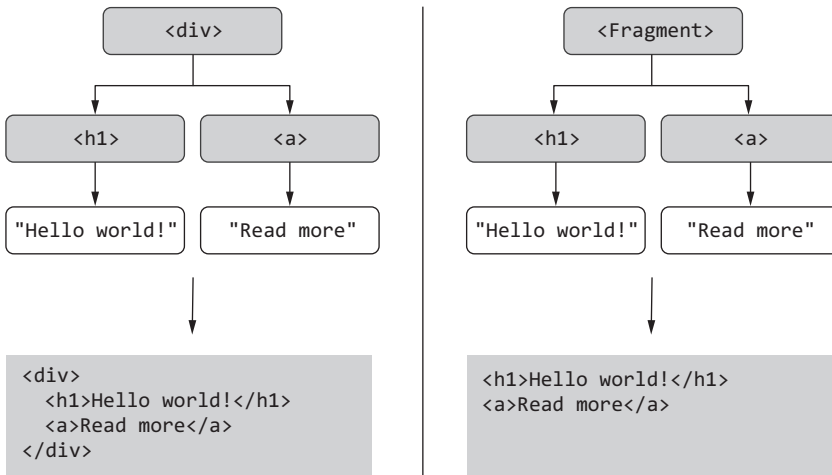
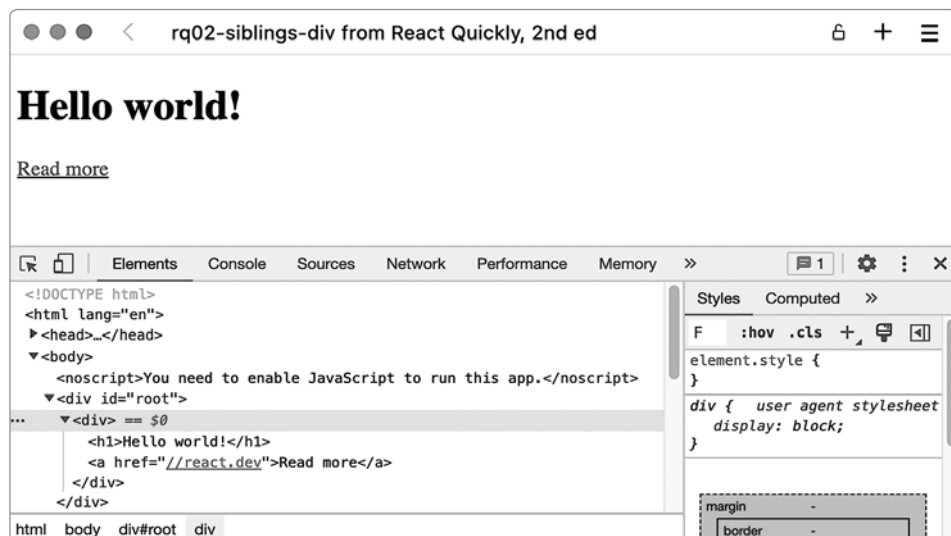


Рис. 2.9. Два способа создания одноуровневых элементов с разным выводом

Если вы предпочитаете использовать нейтральный элемент DOM, сгруппируйте элементы в `<div>`, как показано в листинге 2.2. В результате получите разметку HTML, представленную на рис. 2.10.

Листинг 2.2. Два элемента в группирующем контейнере

```
import React from "react";
import ReactDOM from "react-dom/client";
const title = React.createElement("h1", null, "Hello world!");
const link = React.createElement("a", { href: "//react.dev" }, "Read more");
const group = React.createElement("div", null, title, link);
const domElement = document.getElementById("root");
ReactDOM.createRoot(domElement).render(group);
```

**Рис. 2.10.** Заголовок и ссылка в группирующем элементе**Репозиторий: rq02-siblings-div**

Этот пример находится в репозитории `rq02-siblings-div`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq02-siblings-div --template rq02-siblings-div
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq02-siblings-div>

Контейнер `<div>` обычно хорошо подходит для контента блочного уровня, а `<span>` используется для контента встраиваемого уровня. Однако не обязательно

использовать «реальный» элемент. Можно создать пустой элемент React, который существует только для группировки других элементов и не выводится в разметке HTML-страницы. Это можно сделать с помощью чудесного компонента `React.Fragment`, который может использоваться для группировки (листинг 2.3).

Листинг 2.3. Два элемента во фрагменте

```
import React from "react";
import ReactDOM from "react-dom/client";
const title = React.createElement("h1", null, "Hello world!");
const link = React.createElement("a", { href: "//react.dev" }, "Read more");
const group = React.createElement(
  React.Fragment, null, title, link
);
const domElement = document.getElementById("root");
ReactDOM.createRoot(domElement).render(group);
```

Обратите внимание
на использование `React.
Fragment` в первом
аргументе `createElement`

Репозиторий: `rq02-siblings-fragment`

Этот пример находится в репозитории `rq02-siblings-fragment`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq02-siblings-frag --template rq02-siblings-fragment
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq02-siblings-fragment>

Результат выполнения этого кода в браузере показан на рис. 2.11.

Также можно отрендерить весь элемент одной командой:

```
const group = React.createElement(
  React.Fragment,
  null,
  React.createElement(
    "h1",
    null,
    "Hello world!",
  ),
  React.createElement(
    "a",
    { href: "//react.dev" },
    "Read more",
  ),
);
```

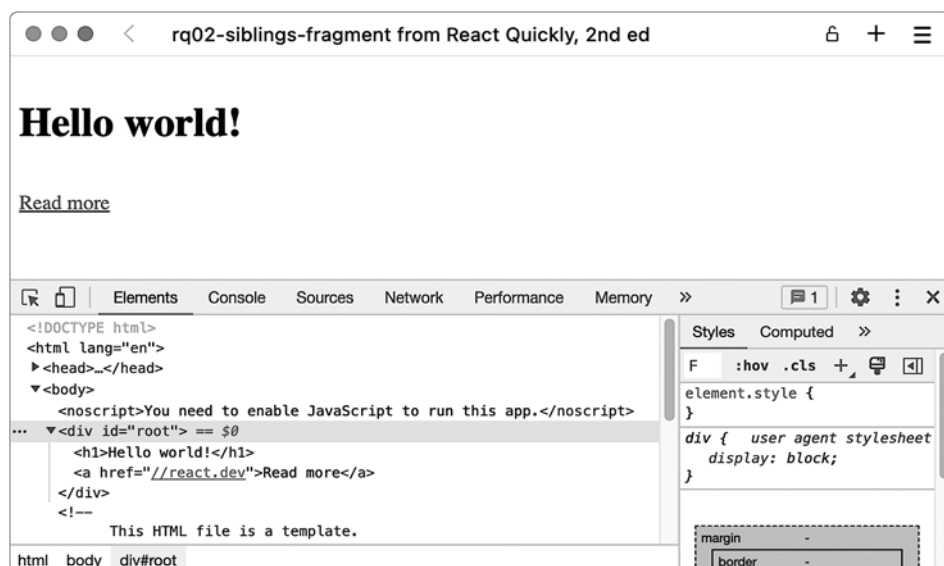


Рис. 2.11. Заголовок и ссылка без группирующего элемента

Этот код работает так же, как приведенный выше, просто в нем используется меньше переменных. Кто-то решит, что этот способ более очевиден, а другим он покажется хуже читаемым.

Скорее всего, до сих пор вы в основном передавали строковые значения в первом параметре `createElement()`. Однако первый параметр может получать значения двух типов, как только что было показано для фрагментов:

- Стандартный тег HTML в виде строки, например "h1", "div" или "p" (без угловых скобок). Имя записывается в нижнем регистре.
- Компонент React в виде ссылки (не в виде строки). Имя обычно начинается с прописной буквы.

Первый подход рендерит стандартные элементы HTML. В качестве имени тега HTML может использоваться любая строка независимо от того, имеет ли она смысл в браузере по умолчанию. Таким образом, хотя в основном можно использовать обычные элементы HTML, такие как `div`, `main`, `section` и т. д., ничто не мешает создать элемент `tiny-horse`, который будет рендериться в браузере в виде `<tiny-horse>`. Такой элемент не имеет смысла и стилевого оформления по умолчанию, но он будет работать.

Во втором варианте можно передать компонент React в виде ссылки. Под ней понимается не имя компонента React в виде строки, а прямая ссылка на компонент, о котором идет речь. Вы уже видели один подобный пример, когда мы использовали `React.Fragment`. В следующем разделе мы покажем, как создать собственные нестандартные компоненты.

2.4. СОЗДАНИЕ НЕСТАНДАРТНЫХ КОМПОНЕНТОВ

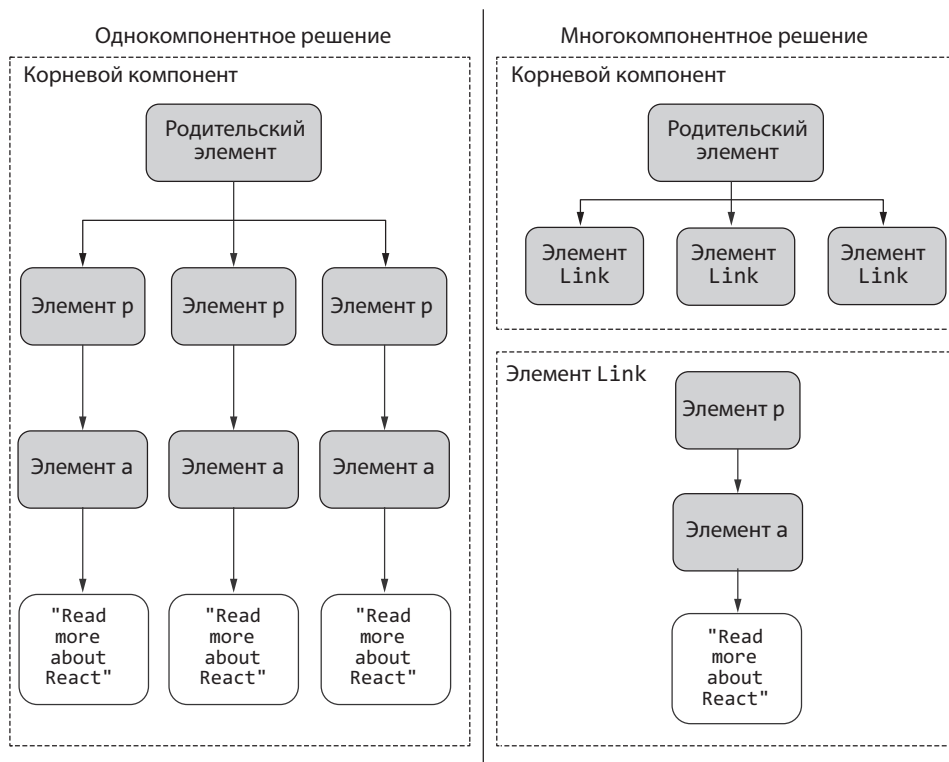
После вложения элементов в React появляется следующая проблема: количество элементов стремительно растет. Приходится использовать компонентную архитектуру (CBA), описанную в главе 1, которая позволяет повторно использовать код за счет разбиения функциональности на слабо связанные составляющие. На помощь приходят *классы компонентов*, или просто *компоненты*, как их часто называют для краткости (не путайте с веб-компонентами).

Стандартные теги HTML можно рассматривать как строительные блоки. Их можно использовать для построения собственных классов компонентов React, которые затем используются для создания нестандартных элементов (экземпляров классов). Используя нестандартные элементы, можно инкапсулировать и абстрагировать логику в импортируемых классах (составных компонентах, пригодных для повторного использования). Эта абстракция позволяет повторно использовать пользовательские интерфейсы в больших сложных приложениях, а также в различных проектах. Примеры такого рода — компоненты с автозаполнением, панели, кнопки, меню и т. д.

В следующем примере нужно создать три одинаковые ссылки. Вообще говоря, создание одинаковых ссылок не имеет особого смысла, но пока вы еще не знаете, как настроить их, поэтому мы ограничимся этим сценарием. Итак, создадим три ссылки, содержащие текст «Read more about React» и ведущие на сайт React по адресу www.react.dev. Также требуется упаковать каждую ссылку в абзац, чтобы они выводились в разных строках.

Эту задачу можно решить двумя способами. Первый — действовать наивно и создать три идентичные копии элементов, второй — действовать более эффективно, создать компонент ссылки, пригодный для повторного использования, а затем — три его экземпляра, как показано на рис. 2.12.

Начнем с первого подхода, в котором используется один компонент с ручным копированием. Требуется создать три независимые ссылки в независимых абзацах; для этого можно воспользоваться пространственным решением из листинга 2.4.

**Рис. 2.12.** Два подхода к созданию повторяющихся элементов**Листинг 2.4.** Три повторяющиеся ссылки

```
import React from "react";
import ReactDOM from "react-dom/client";
const link1 = React.createElement(
  "a", { href: "//react.dev" }, "Read more about React"
);
const p1 = React.createElement("p", null, link1);
const link2 = React.createElement(
  "a", { href: "//react.dev" }, "Read more about React"
);
const p2 = React.createElement("p", null, link2);
const link3 = React.createElement(
  "a", { href: "//react.dev" }, "Read more about React"
);
const p3 = React.createElement("p", null, link3);
const group = React.createElement(React.Fragment, null, p1, p2, p3);
const domElement = document.getElementById("root");
ReactDOM.createRoot(domElement).render(group);
```

Результат выполнения этого кода в браузере изображен на рис. 2.13; это именно то, что требовалось.

Но в листинге 2.4 много повторений, что, конечно, нежелательно. Вся суть React и аналогичных фреймворков заключается в том, чтобы полностью избежать повторений. А для этого потребуется нестандартный компонент!

Нестандартный компонент представляет собой именованный объект, который содержит другие элементы и экземпляры компонентов. Таким образом, в данном случае мы можем создать один компонент `Link`, который рендерит нужную ссылку, и тогда в код можно будет включить три экземпляра компонента `Link` вместо «низкоуровневых» элементов `<p>` и `<a>` со всеми свойствами.

Класс компонента React создается расширением класса `React.Component` в синтаксисе ES6 `class ПОТОМОК extends РОДИТЕЛЬ`. Для создания класса компонента `Link` будет использоваться команда `class Link extends React.Component`.

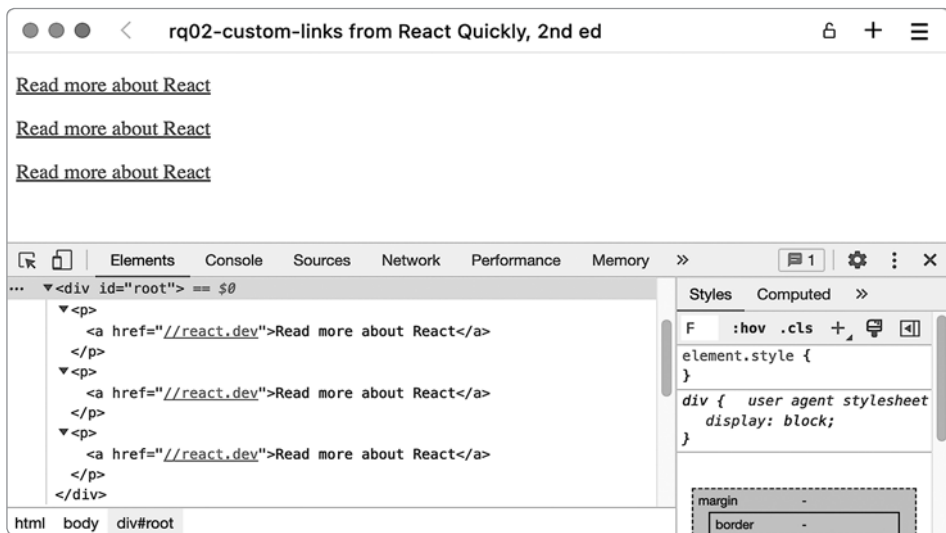


Рис. 2.13. Три идентичные ссылки в приложении

Единственная обязательная часть, которую необходимо реализовать для нового класса, — метод `render()`. Этот метод должен возвращать один корневой элемент, созданный методом `createElement()`, который создается на базе другого нестандартного класса компонента или тега HTML. В любом варианте при желании можно использовать вложенные элементы — при условии, что корневой элемент будет только один.

Листинг 2.5. Создание и рендеринг класса компонента React

```
import React from "react";
import ReactDOM from "react-dom/client";
class Link extends React.Component {
  render() {
    return React.createElement(
      "p",
      null,
      React.createElement(
        "a",
        { href: "//react.dev" },
        "Read more about React"
      )
    );
  }
}
const link1 = React.createElement(Link);
const link2 = React.createElement(Link);
const link3 = React.createElement(Link);
const group = React.createElement(
  React.Fragment, null, link1, link2, link3
);
const domElement = document.getElementById("root");
ReactDOM.createRoot(domElement).render(group);
```

Определяет класс компонента React с именем Link

Создает метод render() в виде выражения (функция, возвращающая один элемент)

Возвращает новый элемент с контентом, необходимым для этого компонента

Создает экземпляры нового компонента Link

Репозиторий: rq02-custom-links

Этот пример находится в репозитории `rq02-custom-links`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq02-custom-links --template rq02-custom-links
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq02-custom-links>

По общепринятому соглашению имена переменных, содержащих компоненты React, начинаются с прописной буквы. В обычном JavaScript такое требование отсутствует. В приведенном коде вместо `Link` можно использовать имя класса `someLink`, и решение будет работать. Но так как такое требование существует в JSX (о котором речь пойдет в следующей главе), мы применяем его и в этом случае.

По аналогии с `ReactDOM.createRoot().render()`, метод `render()` в компоненте класса может возвращать только один элемент. Если вам нужно вернуть

несколько одноуровневых элементов, упакуйте их в компонент-контейнер — либо элемент HTML, либо фрагмент React. Теперь при выполнении этого кода в браузере вы получите такую же разметку HTML, как и раньше (рис. 2.13).

Новый код получается намного более компактным. Лишние повторы удалены, и мы изолировали часть кода, которую можно повторно использовать столько раз, сколько потребуется. Вот она — мощь повторного использования компонентов! Оно ускоряет разработку и сокращает количество ошибок. Компоненты также обладают свойствами, событиями жизненного цикла, состоянием, событиями DOM и другими функциями, обеспечивающими их интерактивность и автономность; эти темы рассматриваются в следующих главах.

А пока мы получили одинаковые ссылки. Но разве не здорово иметь возможность задать значения атрибутов, чтобы изменять их содержимое и/или поведение по отдельности? Это можно делать при помощи свойств, которые рассматриваются в следующем разделе.

2.5. РАБОТА СО СВОЙСТВАМИ

Свойства — краеугольный камень декларативного стиля, используемого в React. Свойства можно рассматривать как неизменяемые значения внутри элемента. Они позволяют модифицировать элементы, используемые в представлениях: например, можно изменить URL-адрес ссылки, передав новое значение свойства:

```
React.createElement("a", { href: "//react.dev" }, "React");
```

Помните, что свойства неизменяемы в пределах своих компонентов. Родитель задает свойства своих дочерних элементов при их создании. Дочерний элемент не должен изменять свои свойства. Например, свойство `PROPERTY_NAME` со значением `VALUE` может передаваться компоненту типа `Link` следующим образом:

```
React.createElement(Link, { PROPERTY_NAME: VALUE });
```

Свойства очень похожи на атрибуты HTML (как показывает `href` в ссылке из примера в начале этого раздела). Это одно из их применений, но есть и другое: свойства элемента можно использовать в коде по своему желанию. Возможные варианты использования свойств:

- для рендеринга стандартных атрибутов HTML элемента: `href`, `title`, `style`, `class` и т. д.;
- как нестандартные инструкции для компонентов, чтобы выполнять их рендеринг по отдельности.

Для обращения к объекту свойств внутри компонента можно использовать синтаксис `this.props`. Это фиксированный (неизменяемый) объект, то есть значения из него можно только читать, но не присваивать.

Неизменяемые объекты в JavaScript

Во внутренней реализации React использует вызов `Object.freeze()` — внутреннюю функцию JavaScript, делающую объект `this.props` неизменяемым. Чтобы проверить, объявлен ли объект неизменяемым, можно воспользоваться методом `Object.isFrozen()`. Например, можно проверить, вернет ли следующая команда `true`:

```
class Test extends React.Component {  
  render() {  
    console.log(Object.isFrozen(this.props))  
    return React.createElement("div")  
  }  
}
```

Подробности довольно сложны, но пока достаточно знать, что никогда не следует редактировать или добавлять свойства внутри самого компонента. Это нужно делать только в родительском контексте.

2.5.1. Одно свойство

Начнем с очень простого примера: в ссылках, созданных выше, должно выводиться название фреймворка. То есть в одной ссылке должен выводиться текст «Read more about React», в другой — «Read more about Vue» и в третьей — «Read more about Angular», как показано на рис. 2.14.

Для это необходимо сделать следующее:

1. Передать свойство экземплярам компонента.
2. Использовать свойство внутри компонента.

Начнем с передачи нового свойства экземплярам ссылок. Вместо

```
const link1 = React.createElement(Link);
```

мы будем передавать во втором аргументе объект, содержащий всего одно свойство:

```
const link1 = React.createElement(Link, { framework: "React" });
```

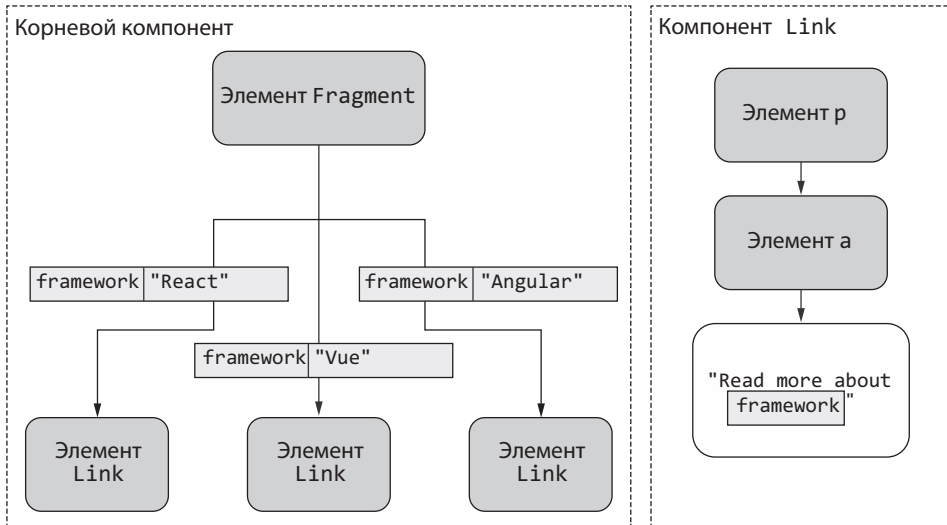


Рис. 2.14. Передача свойств компонентам и использование свойства внутри компонента

Мы используем имя переменной `framework`. Это выбор, который мы сделали сами как создатели компонента. Остается лишь использовать то же имя переменной на втором шаге.

Теперь необходимо использовать переданное свойство внутри класса. Так как переменной присвоено имя `framework`, обращаться к ней следует с синтаксисом `this.props.framework`. Результат выполнения представлен в листинге 2.6.

Листинг 2.6. Экземпляры Link с разным текстом

```

import React from "react";
import ReactDOM from "react-dom/client";
class Link extends React.Component {
  render() {
    return React.createElement(
      "p",
      null,
      React.createElement(
        "a",
        { href: "//react.dev" },
        `Read more about ${this.props.framework}`
      )
    );
  }
}
  
```

Текстовое содержимое ссылки строится объединением `this.props.framework` со статическим контентом. Обратите внимание на использование обратных апострофов для формирования строки с переменной. Эту функциональность обеспечивает JavaScript, а не React

```

const link1 = React.createElement(Link, {
  framework: "React"
});
const link2 = React.createElement(Link, {
  framework: "Vue"
});
const link3 = React.createElement(Link, {
  framework: "Angular"
});
const group = React.createElement(
  React.Fragment, null, link1, link2, link3
);
const domElement = document.getElementById("root");
ReactDOM.createRoot(domElement).render(group);

```

В первом экземпляре компонента ссылки для свойства `framework` используется строка "React"

Во втором экземпляре компонента ссылки для свойства `framework` используется строка "Vue"

В третьем экземпляре компонента ссылки для свойства `framework` используется строка "Angular"

На рис. 2.15 показана работа этого решения.

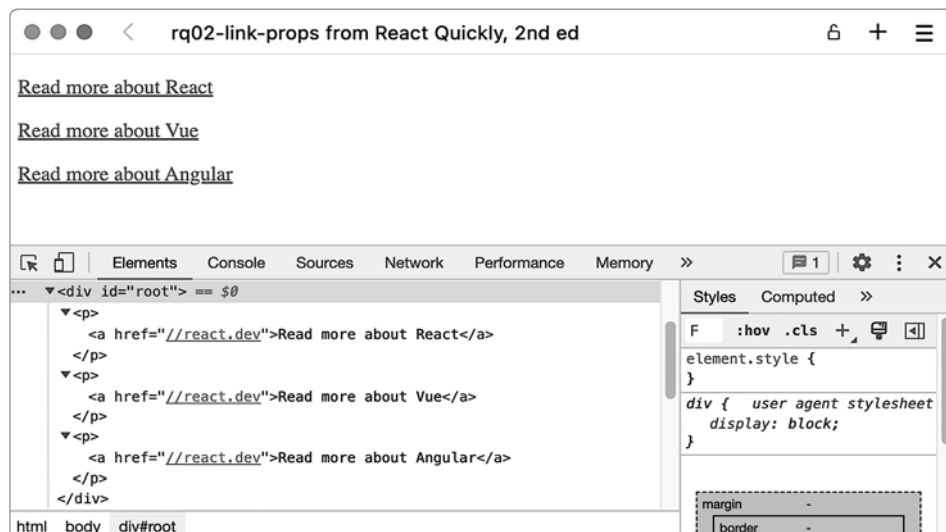


Рис. 2.15. Три ссылки с разным текстом, каждая из которых находится в отдельном абзаце

2.5.2. Несколько свойств

Вы, должно быть, заметили, что все ссылки продолжают указывать на один и тот же URL-адрес сайта React. Конечно, это нам не подходит, потому что URL должны быть разными. Используя тот же подход, мы просто добавляем новое свойство `url` и используем его внутри компонента, а также в экземплярах

компонента. Эта возможность наглядно представлена на рис. 2.16 и реализована в коде в листинге 2.7.

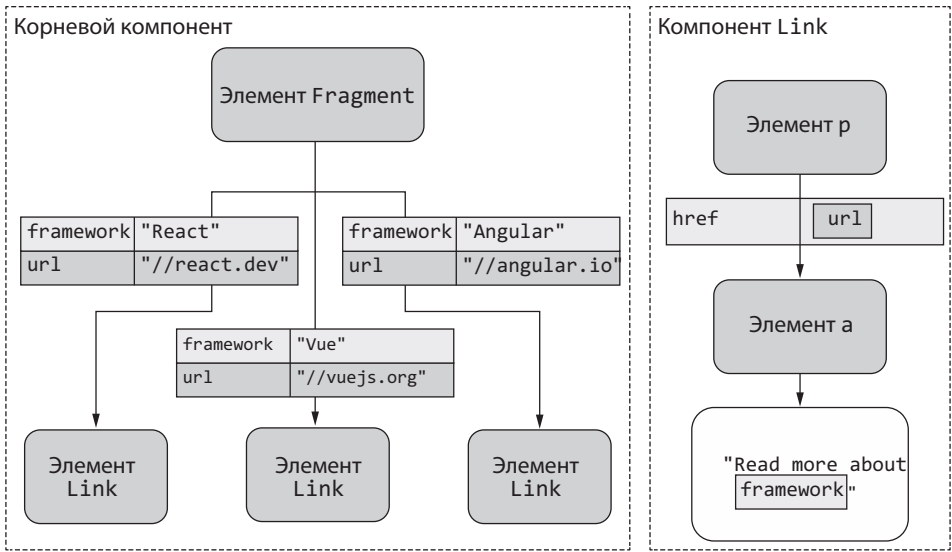


Рис. 2.16. Передача двух свойств компонентам

Листинг 2.7. Экземпляры Link с разным текстом и URL-адресами

```
import React from "react";
import ReactDOM from "react-dom/client";
class Link extends React.Component {
  render() {
    const link = React.createElement(
      "a",
      { href: this.props.url },
      `Read more about ${this.props.framework}`
    );
    return React.createElement("p", null, link);
  }
}

const link1 = React.createElement(Link, {
  framework: "React",
  url: "//react.dev",
});
const link2 = React.createElement(Link, {
  framework: "Vue",
  url: "//vuejs.org",
});
const link3 = React.createElement(Link, {
  framework: "Angular",
```

Свойство url используется для задания свойства href элемента <a>

Для React используется URL https://react.dev

Для Vue используется URL https://vuejs.org

```
url: "//angular.io", ← Для Angular используется URL https://angular.io
});
const group = React.createElement(
  React.Fragment, null, link1, link2, link3
);
const domElement = document.getElementById("root");
ReactDOM.createRoot(domElement).render(group);
```

Репозиторий: rq02-link-props

Этот пример находится в репозитории `rq02-link-props`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq02-link-props --template rq02-link-props
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq02-link-props>

На рис. 2.17 показано, как это приложение выглядит в браузере. Как видите, мы можем использовать свойства обоих нестандартных компонентов (используемых внутри компонента для настройки возвращаемой структуры) и элементов HTML (которые задают атрибуты HTML).

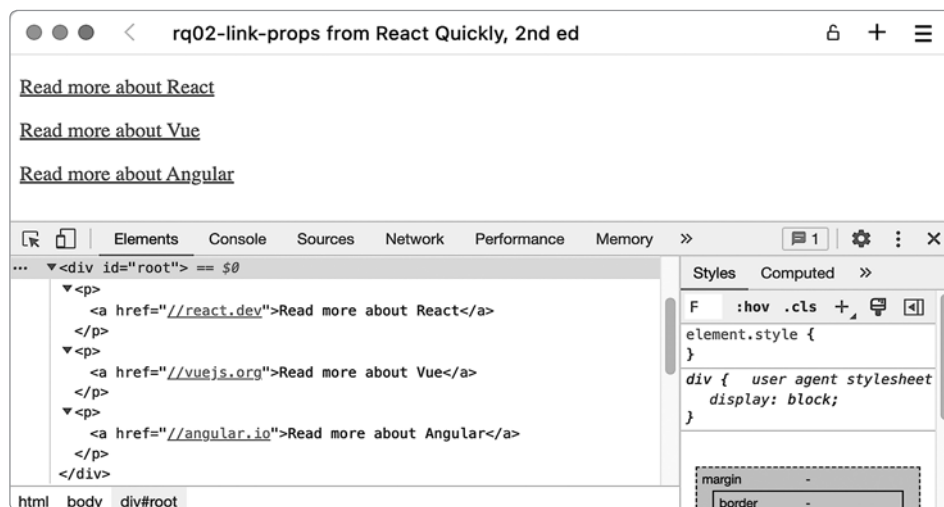


Рис. 2.17. Три ссылки с разным текстом и разными URL

Что произойдет, если перепутать элементы и задать нестандартное свойство для элемента HTML? React все равно отрендерит его. До React 16 недействительные свойства отфильтровывались, но поскольку современные веб-приложения часто используют другие сторонние библиотеки, которые могут зависеть от нестандартных свойств, React 16 и более поздних версий позволяет использовать любые указанные свойства.

На основании значения свойства можно полностью изменить отрендеренные элементы. Например, можно проверить свойство `framework` и вернуть большой заголовок со ссылкой для имени фреймворка "React":

```
class Link extends React.Component {
  render() {
    const link = React.createElement(
      "a",
      { href: this.props.url },
      `Read more about ${this.props.framework}`
    );
    if (this.props.framework === "React") {
      return React.createElement("h1", null, link);
    }
    return React.createElement("p", null, link);
  }
}
```

Создает элемент ссылки и сохраняет его в переменной

Проверяет, содержит ли фреймворк текст "React"

Если да, возвращает элемент h1 с содержащейся в нем ссылкой

В противном случае возвращает элемент абзаца с содержащейся в нем ссылкой

Также этот пример отлично демонстрирует, что элементы React содержат самый обычный JavaScript. Вы можете создать элемент и сохранить его в переменной, а затем использовать эту переменную так, как считаете нужным. Также можно реализовать условное ветвление стандартными средствами JavaScript. Если отрендерить новый компонент в браузере, ссылки неожиданно перестают быть идентичными, как видно из рис. 2.18.

В этой главе мы рассмотрели несколько модификаций очень простой разметки HTML, которая почти бесполезна сама по себе. Но начиная с малого, мы постепенно подходим к более сложным темам. Поверьте: у классов компонентов очень широкие возможности.

Очень важно знать, как React работает в обычных событиях JavaScript, если вы (как и многие разработчики React) собираетесь использовать JSX. Ведь в конечном итоге в браузерах все равно выполняется стандартный код JS, а иногда появляется необходимость понимать результаты транспиляции JSX в JS. Забегая вперед, скажем, что мы будем использовать JSX, — эта тема рассматривается в следующей главе. Но сначала необходимо кое-что сказать о структуре приложения React.

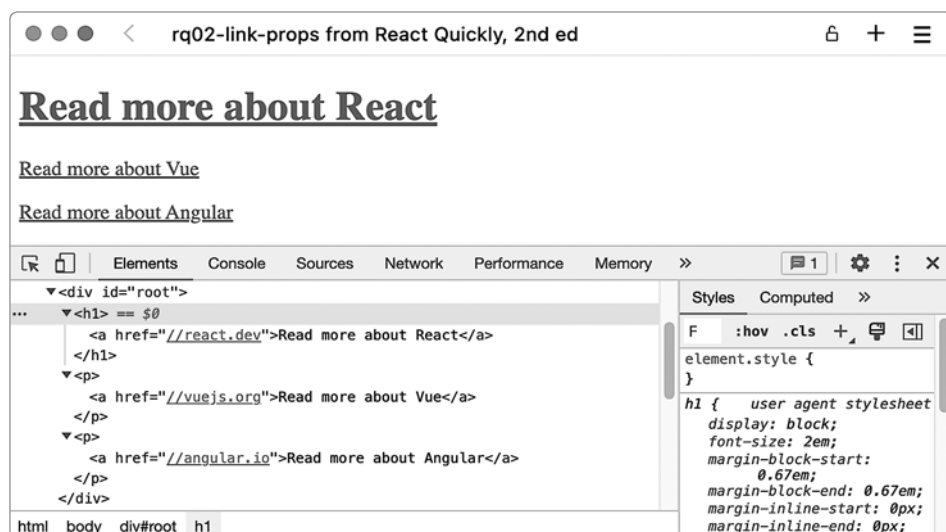


Рис. 2.18. Выводятся три ссылки, но ссылка на React выделяется как самая важная

2.5.3. Специальное свойство: children

Элементы React получают специальное свойство `children`. Его не удастся задать обычным образом, но можно использовать как любое другое свойство.

Немного изменим наш пример и создадим список ссылок, в которых выводятся только названия фреймворков без предшествующего текста «Read more about», как показано на рис. 2.19.

Теперь пойдем еще дальше. Предположим, вам требуется вывести фреймворк для React жирным шрифтом. Вы уже знаете, как выделить элемент жирным шрифтом, — для этого достаточно упаковать его в элемент:

```
React.createElement("strong", null, "React");
```

Но как передать это в свойстве? Создать узел для фреймворка React:

```
const boldReact = React.createElement("strong", null, "React");
const link1 = React.createElement(
  Link,
  { framework: boldReact, url: "//react.dev" }
);
```

Создает элемент React в переменной с именем `boldReact`

Передает эту переменную в свойстве `framework` для элемента `Link`

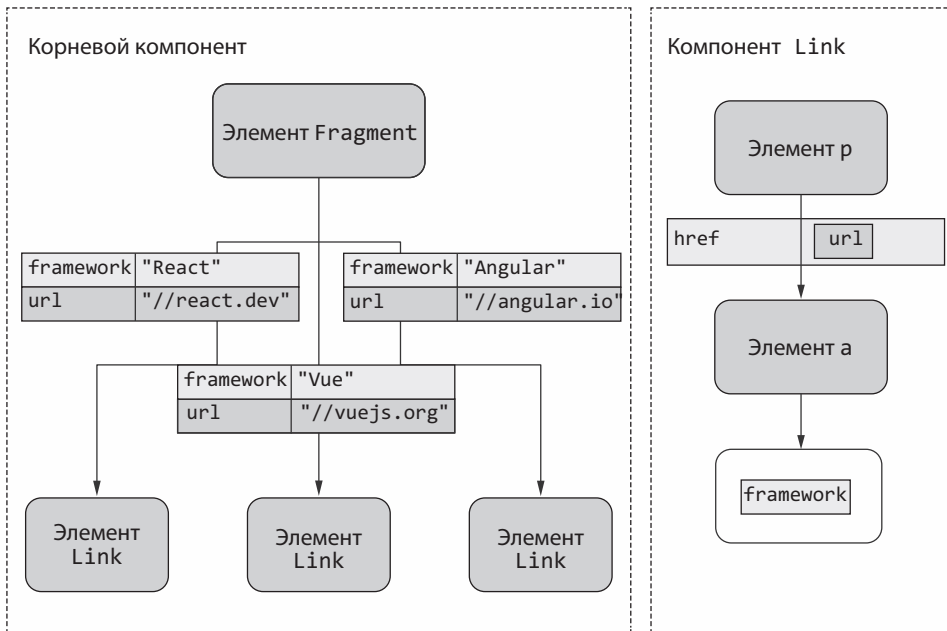


Рис. 2.19. Новая структура со ссылками, содержащими только названия фреймворков

Впрочем, это немного странно. Мы создаем элементы, но передаем их как свойства, чего обычно не делаем. Нельзя ли создать элемент и передать его как дочерний элемент?

Вспомним, что аргументы `React.createElement`, начиная с третьего, являются дочерними по отношению к элементу. Хотя мы не использовали эту возможность для нестандартных компонентов, она существует. Все узлы, передаваемые как дочерние для нестандартного элемента, доступны в свойстве `this.props.children`. Это свойство содержит либо один узел (если передан один дочерний элемент), либо массив узлов (если переданы множественные дочерние элементы).

Итак, изменим корневой компонент, чтобы он содержал три ссылки, у которых текст передается не в свойстве с именем `framework`, а как дочерний узел. Текст первой ссылки должен выводиться жирным шрифтом, как показано на рис. 2.20, а затем реализовано в листинге 2.8.

Листинг 2.8. Передача текста ссылки в дочернем узле

```

import React from "react";
import ReactDOM from "react-dom/client";
class Link extends React.Component {
  render() {
    return React.createElement(
      "p",
      null,
      React.createElement(
        "a",
        { href: this.props.url },
        this.props.children ← Свойство children используется так же, как и любое другое
      )
    );
  }
}
const boldReact = React.createElement("strong", null, "React");
const link1 = React.createElement(
  Link,
  { url: "//react.dev" },
  boldReact
);
const link2 = React.createElement(
  Link,
  { url: "//vuejs.org" },
  "Vue"
);
const link3 = React.createElement(
  Link,
  { url: "//angular.io" },
  "Angular"
);
const group = React.createElement(
  React.Fragment, null, link1, link2, link3
);
const domElement = document.getElementById("root");
ReactDOM.createRoot(domElement).render(group);

```

Теперь каждому нестандартному компоненту передается только одно именованное свойство, а именно свойство `url`. Но теперь также передается дочерний узел, который становится свойством `children`. Для React это вложенный элемент

Каждому компоненту передается только одно именованное свойство `url`. Для Vue и Angular дочерний узел представляет собой обычный текстовый узел

Запустив это приложение в браузере, получим результат, показанный на рис. 2.21. Различия между обычным свойством и свойством `children` могут показаться несущественными, но когда в следующей главе мы начнем использовать JSX, вы увидите, что при правильном использовании это свойство имеет большое значение.

Репозиторий: rq02-links-children

Этот пример находится в репозитории `rq02-links-children`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq02-links-children --template rq02-links-children
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq02-links-children>

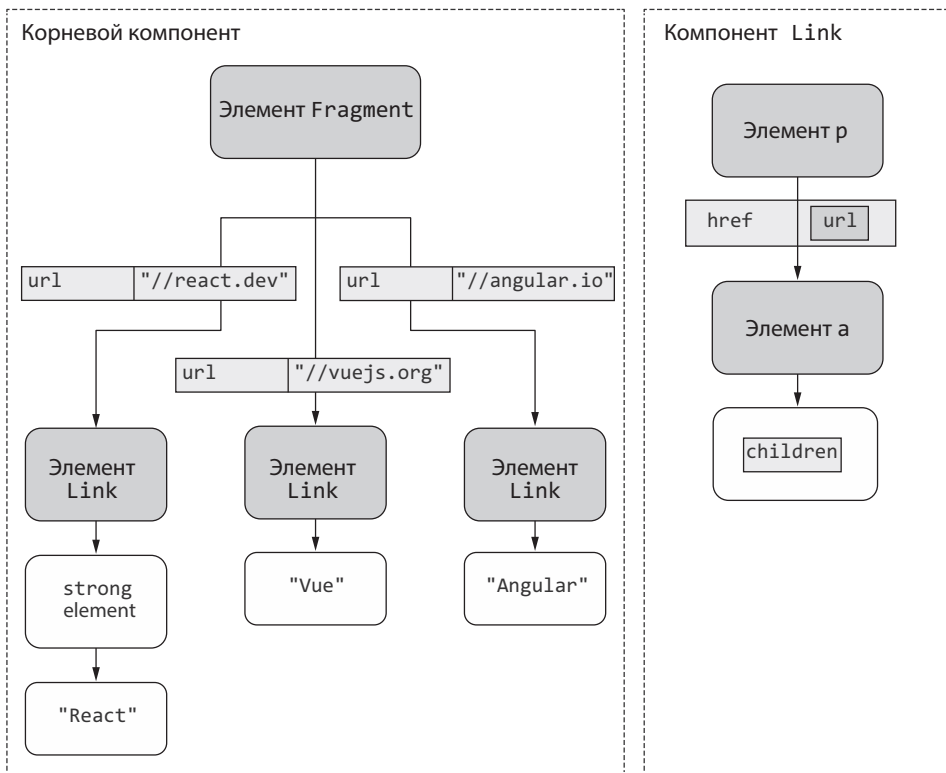


Рис. 2.20. Дерево компонентов при передаче текста ссылки в дочернем узле вместо обычного свойства

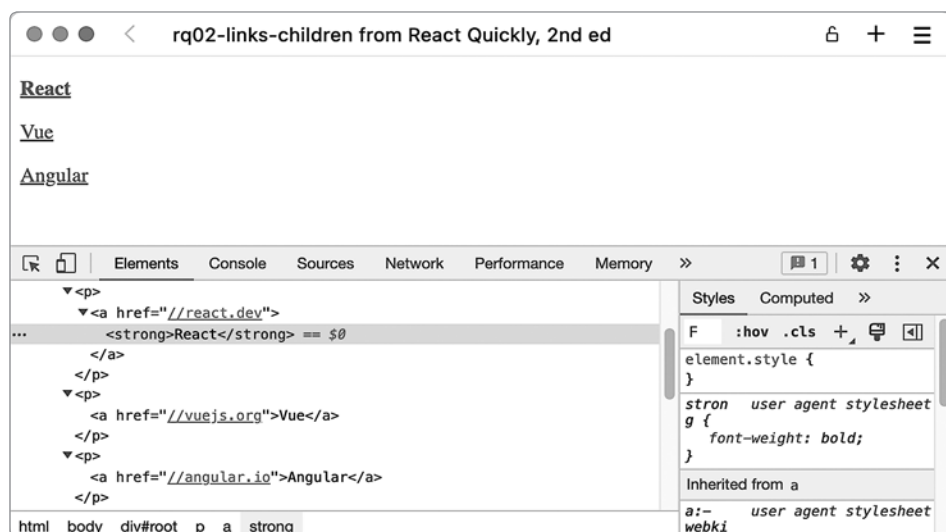


Рис. 2.21. Компоненты ссылки используют свойство `children`, первая ссылка выделена жирным шрифтом

2.6. СТРУКТУРА ПРИЛОЖЕНИЯ

Начиная со следующей главы, мы будем строить приложения по единой схеме, с легко узнаваемыми закономерностями. Также мы будем следовать стандартной структуре, которую предоставляет шаблон CRA по умолчанию.

В предыдущем примере этой главы приложение было размещено непосредственно в файле `index.js` папки с исходным кодом. В дальнейшем в качестве корневого элемента приложений будет использоваться нестандартный компонент `App`, который будет рендериться как единственный дочерний элемент в браузере. Это означает, что мы не будем работать с файлом `src/index.js`. Он останется неизменным для всех приложений, в которых будет использоваться CRA.

Для демонстрации перепишем наше приложение с тремя ссылками так, чтобы в нем использовались два новых компонента: корневой компонент `App` и компонент `Link`. Второй компонент мы используем в первом трижды. Наконец, деконструируем некоторые свойства из пространства имен `React`, чтобы немного сократить определение компонента. Поместим весь код в файл `src/App.js` (рис. 2.22 и листинг 2.9).

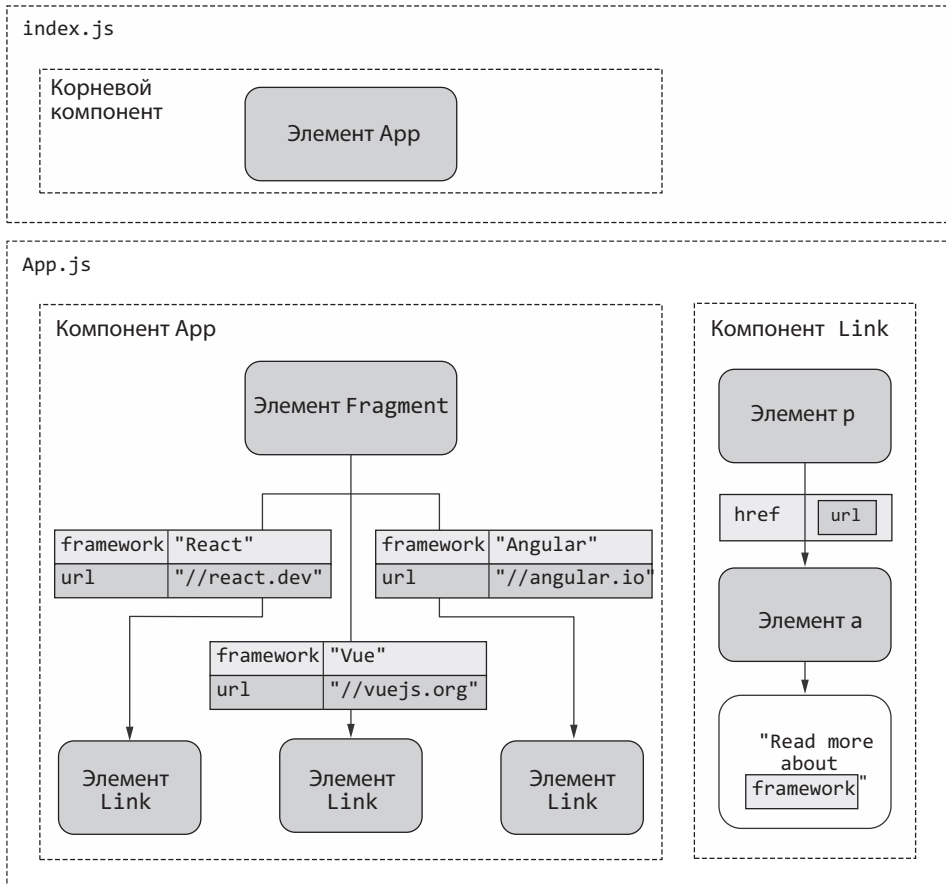


Рис. 2.22. Новая структура файлов с двумя компонентами в App.js

Листинг 2.9. Код приложения в файле src/App.js

```
import React, { Fragment, Component } from "react";
class Link extends Component {
  render() {
    return React.createElement(
      "p",
      null,
      React.createElement(
        "a",
        { href: this.props.url },
        `Read more about ${this.props.framework}`
      )
    );
  }
}
```

Деструктурирует импорт React для прямого обращения к Fragment и Component

Компонент Link выглядит точно так же, как в предыдущем листинге

```

    )
  };
}
}
class App extends Component {
  render() {
    const link1 = React.createElement(Link, {
      framework: "React",
      url: "///react.dev",
    });
    const link2 = React.createElement(Link, {
      framework: "Vue",
      url: "///vuejs.org",
    });
    const link3 = React.createElement(Link, {
      framework: "Angular",
      url: "///angular.io",
    });
    return React.createElement(
      Fragment, null, link1, link2, link3
    );
  }
}
export default App;

```

← Новый компонент App, который рендерит корень приложения

Компонент App возвращает один элемент, как должны делать все компоненты

← В App.js компонент App экспортируется как один доступный ассет в этом файле

Затем мы изменяем файл `src/index.js`, чтобы импортировать компонент App из `App.js` и отрендерить его в корневом элементе DOM, как показано в листинге 2.10.

Листинг 2.10. Файл `src/index.js`

```

import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
createRoot(document.getElementById("root"))
  .render(React.createElement(App));

```

← Импортирует приложение из `App.js` и сохраняет его в локальной переменной `App`

← Создает один корневой элемент из элемента HTML с `id="root"`

← Рендерит компонент App в корне

Новый файл `src/index.js` фактически готов. Нам больше не потребуется редактировать его; для настройки будущих приложений редактируется только файл `src/App.js`.

Репозиторий: `rq02-links-app`

Этот пример находится в репозитории `rq02-links-app`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq02-links-app --template rq02-links-app
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq02-links-app>

С ростом приложения хранить все в одном файле `src/App.js` становится неудобно. В дальнейшем можно создать новые файлы и импортировать их по мере необходимости. Хотя обычно создается один файл на компонент, имя которого соответствует имени компонента (включая прописную первую букву), это не обязательно. Если для функционирования компоненту нужно несколько других небольших компонентов, можно выбрать, помещать ли весь код в один файл, как `Link` и `App` в листинге 2.10, или разбить его на несколько файлов.

Теперь посмотрим, как создать тот же пример с компонентами `App` и `Link` в разных файлах. Структура файлов представлена на диаграмме на рис. 2.23. Также необходимо обновить файл `src/App.js`, чтобы импортировать компонент `Link` из `src/Link.js`, как показано в листинге 2.11.

Листинг 2.11. Один компонент на файл: `src/App.js`

```
import React, { Fragment, Component } from "react";
import Link from "../Link"; ← Импортирует компонент Link из другого файла
class App extends Component {
  render() {
    const link1 = React.createElement(Link, {
      framework: "React",
      url: "//react.dev",
    });
    const link2 = React.createElement(Link, {
      framework: "Vue",
      url: "//vuejs.org",
    });
    const link3 = React.createElement(Link, {
      framework: "Angular",
      url: "//angular.io",
    });
    return React.createElement(Fragment, null, link1, link2, link3);
  }
}
export default App;
```

Затем необходимо создать новый файл `src/Link.js`, содержащий только компонент `Link`, и не забыть экспортировать его в конце файла.

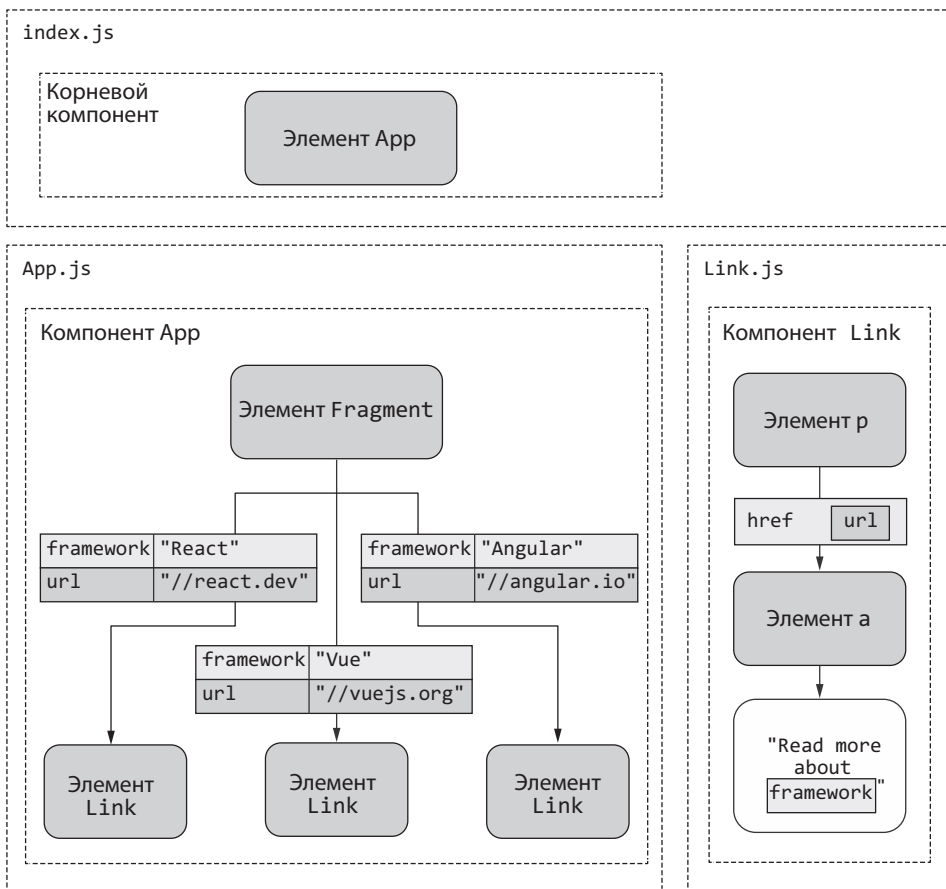


Рис. 2.23. Структура файлов при использовании отдельного файла для каждого компонента

Листинг 2.12. Один компонент на файл: src/Link.js

```
import React, { Component } from "react";
class Link extends Component {
  render() {
    return React.createElement(
      "p",
      null,
      React.createElement(
        "a",
        { href: this.props.url },
        `Read more about ${this.props.framework}`
      )
    );
  }
}
```

Теперь в этом файле содержится только определение компонента Link

```

    );
  }
}
export default Link; ← Не забудьте экспортировать компонент в конце

```

Репозиторий: rq02-links-app-alt

Этот пример находится в репозитории `rq02-links-app-alt`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq02-links-app-alt --template rq02-links-app-alt
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq02-links-app-alt>

В книге будут использоваться оба подхода. В некоторых главах приложения будут небольшими и компактными и весь код будет размещаться в `src/App.js`. Но потом приложения станут больше и одного файла будет недостаточно. Также мы будем создавать файлы не только для компонентов, но и для других сущностей. В частности, для часто используемых функций или другой общей функциональности, которая требуется во многих файлах. Мы также будем использовать отдельные файлы для нестандартных хуков, когда займемся этой темой в главе 10.

Когда проекты расширяются еще больше, в проект вводится иерархия папок. Официальных стандартов по использованию папок для организации проектов React не существует, так что команды часто разрабатывают собственные схемы.

2.7. ВОПРОСЫ

1. Какие из следующих команд могут использоваться для создания классов компонентов React?
 - a) `const Name = React.createComponent()`
 - b) `class Name extends React.Component`
 - c) `const Name = React.createElement()`
 - d) `class Name extends React.Class`

2. Какой из следующих атрибутов или методов компонента React является единственным обязательным?
 - a) `function`
 - b) `return`
 - c) `name`
 - d) `render`
 - e) `class`
3. Какие из следующих конструкций могут использоваться для обращения к свойству `url` компонента?
 - a) `this.properties.url`
 - b) `this.data.url`
 - c) `this.props.url`
 - d) `url`
4. Свойства React являются неизменяемыми в контексте текущего компонента. *Да или нет?*
5. Классы компонентов React позволяют разработчикам создавать пользовательские интерфейсы, пригодные для повторного использования. *Да или нет?*

ОТВЕТЫ

1. `class NAME extends React.Component`, потому что `React.Class` не существует, а остальные варианты используются для создания экземпляров компонентов, а не определения компонентов.
2. `render()` — единственный обязательный метод. Кроме того, `function`, `return`, `render` и `class` не являются допустимыми именами методов.
3. `this.props.url`, потому что только `this.props` возвращает объект свойств.
4. *Да.* Изменить свойство внутри самого компонента невозможно.
5. *Да.* Разработчики используют новые компоненты для создания пользовательских интерфейсов, пригодных для повторного использования.

ИТОГИ

- Новые приложения React можно создавать при помощи командной строки `create-react-app`. Она позволяет быстро приступить к работе с мощным начальным набором полезных библиотек.
- Новые проекты React можно создавать по заданному шаблону. Ко всем примерам в книге прилагается шаблон, чтобы можно было просмотреть пример локально при помощи трех коротких команд и не приходилось ничего искать или загружать напрямую. Мы также предоставим ссылки для загрузки примеров.
- Чтобы использовать вложение элементов React, следует включать вложенные вызовы `createElement()` друг в друга. Также можно формировать одноуровневые узлы, используя третий, четвертый и т. д. аргументы `createElement()`.
- Элементы можно создавать на базе обычных имен узлов HTML, передавая имя узла HTML в первом аргументе `createElement()`.
- Если вы хотите изменить полученные элементы при помощи свойств, передайте их в объекте во втором аргументе `createElement()`.
- Чтобы использовать компонентную архитектуру, или СВА (одна из особенностей React), можно создавать нестандартные компоненты. Они могут использовать свойства во внутренней реализации через переменную `this.props`. Дочерние узлы передаются в специальном свойстве `children`.
- Во всех примерах этой книги используется очень простая структура файлов.

3

Знакомство с JSX

В ЭТОЙ ГЛАВЕ

- ✓ JSX и его преимущества
- ✓ Использование JSX для ускорения и упрощения реализации нестандартных компонентов
- ✓ Возможные проблемы React и JSX

JavaScript XML (JSX) — это синтаксическое расширение JavaScript. Теперь это один из тех самых компонентов, которые делают React замечательной штукой, но стал он таким далеко не сразу.

Пример использования JSX в JavaScript:

```
const link = <a href="//react.dev">React</a>;
```

JSX — элемент, заключенный в угловые скобки: `<a href="//react.dev">React</a>`. Это не строка, не шаблонный литерал и не HTML. Это объект JavaScript, созданный с использованием синтаксического расширения, называемого JSX. Он значительно ускоряет создание элементов React, делая их более компактными, а также упрощает их чтение, причем второе преимущество не менее важно, чем первое.

JSX создан только для разработчиков. Сам по себе он никак не ускоряет и не улучшает веб-приложения. JSX преобразуется в тот же код, который получается и без JSX.

Хотя использовать JSX в принципе не обязательно, считается, что писать компоненты React можно только с ним. Команды, не использующие JSX, существуют, но их подавляющее меньшинство.

В этой главе мы сначала разберем причины использования JSX, затем обсудим разные варианты практического применения JSX и, наконец, рассмотрим некоторые нюансы, на которые необходимо обращать внимание при использовании JSX. Попутно также затронем процесс преобразования JSX в JavaScript, называемый *транспилицией* (о нем уже упоминалось в главе 2). К счастью, вам она почти не понадобится.

ПРИМЕЧАНИЕ Исходный код примеров этой главы доступен по адресу <https://rq2e.com/ch03>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке всего одной командой.

3.1. ЧТО ТАКОЕ JSX И ЧЕМ ОН ХОРОШ?

JSX — расширение JavaScript, предоставляющее «синтаксический сахар» для вызовов функций и построения объектов, особенно `React.createElement()` (иначе говоря, он упрощает процесс написания кода, сохраняя его функциональность). На первый взгляд код JSX может показаться чем-то вроде шаблонизатора или HTML, но это не так. JSX строит элементы React, позволяя использовать все преимущества JavaScript. JSX — отличный инструмент для записи компонентов React. Его основные плюсы:

- *Улучшенное взаимодействие с разработчиком* (DX, Developer Experience) — код проще читать благодаря большей выразительности XML-подобного синтаксиса, лучше подходящего для представления вложенных декларативных структур.
- *Улучшенные сообщения об ошибках* — React по умолчанию предполагает использование JSX и выводит полезные сообщения об ошибках. Если JSX не используется, эти сообщения будут запутывать разработчика, так как в них будет упоминаться не тот синтаксис, который он реально использует.
- *Ускорение кода* — при преобразовании JSX в JavaScript транспилятор одновременно оптимизирует код, благодаря чему итоговый код JavaScript выполняется быстрее, чем если бы он был введен вручную.

- *Повышение эффективности работы команд* — неопытным разработчикам (например, веб-дизайнерам) проще менять код, потому что JSX напоминает разметку HTML, которая им уже знакома.
- *Снижение количества синтаксических ошибок* — разработчики набирают на клавиатуре меньше кода, а это значит, что они сделают меньше ошибок.

И хотя язык JSX не обязателен к применению, он хорошо сочетается с React, и создатели React рекомендуют использовать его (а мы присоединяемся к этой рекомендации). Команду, которая использует React без JSX, найти непросто. И хотя мы не можем утверждать, что *все* последние проекты React используют JSX, мы почти уверены, что это действительно так.

3.1.1. До и после JSX

Чтобы продемонстрировать выразительность JSX, приведем фрагмент, необходимый для создания элемента с несколькими нестандартными компонентами, за которыми следует ссылка:

```
const element = <main>
  <Title>Welcome</Title>
  <Carousel images={6} />
  <a href="/blog">Go to the blog</a>
</main>;
```

Этот код идентичен следующему фрагменту, реализованному без JSX:

```
const element = React.createElement(
  'main',
  null,
  React.createElement(Title, null, 'Welcome'),
  React.createElement(Carousel, {images: 6}),
  React.createElement('a', {href: "/blog"}, 'Go to the blog'),
);
```

Вероятно, вы согласитесь, что версия с JSX намного понятнее. Она похожа на HTML, который очень легко читается, и отчасти совпадает с разметкой HTML, которая будет отрендерена (конечно, не считая нестандартных компонентов).

3.1.2. Сочетание HTML и JavaScript

Фактически JSX представляет собой мини-язык с XML-подобным синтаксисом; однако этот язык изменил подход к написанию UI-компонентов. В прошлом разработчики писали разметку HTML — и код JavaScript для контроллеров

и представлений — в MVC-подобном стиле, переключаясь между файлами. Такой подход был обусловлен разделением обязанностей на ранней стадии развития технологий. Он хорошо работал, пока веб-страницы состояли из статической разметки HTML, небольшого количества CSS и крошечного фрагмента JS для создания мигающего текста.

Времена изменились. В наши дни создаются высокоинтерактивные пользовательские интерфейсы, а JavaScript и HTML тесно связаны друг с другом в реализации различных аспектов функциональности. Это нарушает принцип разделения обязанностей (SoC), который считается одним из фундаментальных принципов большей части современной разработки. Согласно ему, несвязанные элементы должны разделяться, а взаимосвязанные — находиться рядом друг с другом. Если вы предпочитаете соблюдать этот принцип, код следует разбить так, чтобы каждая изолированная часть реализовала одну (и только одну!) обязанность, и использовать эти «части» в разных сочетаниях. Если разделить шаблон и логику представлений, работающие только в сочетании друг с другом, получится, что две сущности, которые должны быть объединены, на самом деле разделены без необходимости.

React исправляет нарушенный принцип разделения обязанностей (SoC, separation of concerns), объединяя описание пользовательского интерфейса и логики JavaScript; а JSX-код напоминает HTML, его проще читать и писать. Уже только по одной этой причине стоит использовать React и JSX для написания пользовательских интерфейсов.

Код JSX компилируется различными преобразователями (инструментами) в стандартный ECMAScript (рис. 3.1). Вы, должно быть, знаете, что JavaScript также является стандартом ECMAScript, но JSX в эту спецификацию не входит и не имеет определенной семантики. Это означает, что при попытке откомпилировать JavaScript со встроенным JSX обычным компилятором JavaScript без предварительной транспиляции JSX вы получите сообщения об ошибках. JSX сам по себе не является действительным кодом JavaScript, и его нельзя напрямую откомпилировать компилятором JavaScript.

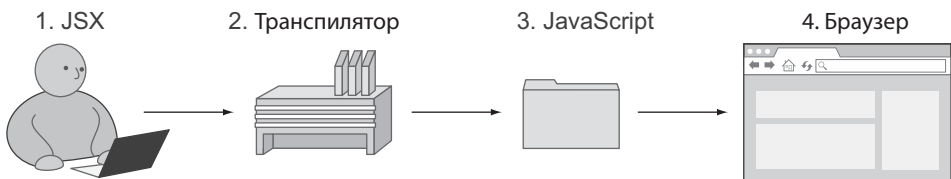


Рис. 3.1. JSX транпилируется в обычный JavaScript

ПРИМЕЧАНИЕ Мы называем преобразование *транспилицией*, а не *компиляцией*, потому что код транслируется с одного исходного языка (JSX) на другой исходный язык (JavaScript). В свою очередь, полученный код JavaScript интерпретируется «реальным» компилятором, который выполняет код. Транспилиция — всего лишь преобразование синтаксиса, а не интерпретация кода.

При выполнении приложения React браузер видит только команды `React.createElement`, необходимые для генерирования нужной структуры. JSX существует только в редакторе. Транспилатор преобразует файлы, содержащие JSX, в «чистый» JavaScript с многочисленными вызовами `React.createElement()`, облегчая жизнь разработчикам.

Возникает резонный вопрос: зачем возиться с JSX? Справедливо. Если учесть, насколько нелогично выглядит код JSX для новичков, неудивительно, что многие разработчики отвернулись от этой замечательной технологии. Например, в следующем фрагменте JSX в коде JavaScript присутствуют угловые скобки, которые выглядят очень странно:

```
const title = <h1>Hello</h1>;
```

Одна из самых удобных особенностей JSX — сокращенные формы записи для `React.createElement(NAME, ...)`. Вместо того чтобы снова и снова записывать этот вызов функции, достаточно использовать `<NAME />`. И как уже говорилось, чем меньше символов вы вводите, тем меньше ошибок совершите. В JSX на первое место ставится взаимодействие с разработчиками, чтобы им было проще создавать компоненты и приложения за меньшее время и с меньшим количеством ошибок.

Основная причина, по которой используется JSX, в том, что многим код в угловых скобках (`< >`) читать проще, чем код с большим количеством команд `React.createElement()`. А если привыкнуть рассматривать `<NAME />` не как XML, а как синоним для кода JavaScript, вся кажущаяся нелогичность синтаксиса JSX исчезнет. Знание и использование JSX может сильно повлиять на разработку компонентов React и, соответственно, приложений на базе React.

Как уже говорилось, код JSX необходимо транспилировать в стандартный код JavaScript перед тем, как выполнять его в браузере. В большинстве ситуаций вручную этим заниматься не придется, но на всякий случай мы рассмотрим некоторые транспилаторы в разделе 3.3. А пока постараемся глубже разобраться в JSX и понять его работу.

3.2. ЛОГИКА JSX

Посмотрим, как работать с JSX. Вы можете прочитать этот раздел и обращаться к нему в дальнейшем или (если вам интересно запустить некоторые примеры на своем компьютере) продолжить работу над примерами с использованием приведенных шаблонов `create-react-app` (CRA). С CRA транспилиция JSX, по сути, прилагается бесплатно, так что самостоятельно ее настраивать не придется.

3.2.1. Создание элементов с JSX

Объекты `ReactElement` в JSX создавать достаточно просто. В табл. 3.1 приведены некоторые примеры JavaScript, которые использовались в предыдущих примерах, и их эквиваленты в JSX.

Таблица 3.1. Код JavaScript и его эквиваленты JSX

JavaScript	Эквивалент JSX
<code>React.createElement('h1')</code>	<code><h1 /></code>
<code>React.createElement('h1', null, 'Welcome',)</code>	<code><h1> Welcome </h1></code>
<code>React.createElement(Title, null, 'Welcome',)</code>	<code><Title> Welcome </Title></code>
<code>React.createElement(Title, {size: 6}, 'Welcome')</code>	<code><Title size="6"> Welcome </Title></code>
<code>React.createElement(Title, {size: 6}, 'Welcome to ', React.createElement('strong', null, 'Narnia',),)</code>	<code><Title size="6"> Welcome to Narnia </Title></code>

В коде JSX атрибуты и их значения (например, `size={6}`) берутся из второго аргумента `createElement()`. Мы подробно рассмотрим работу со свойствами позже.

А пока рассмотрим пример элементов JSX без свойств. Ниже приведен один из первых примеров из предыдущей главы, обновленный до рекомендованной структуры с использованием нестандартного компонента `App`. Это просто элемент `h1` с текстом «Hello world!», в котором слово «world» выделено курсивом.

Листинг 3.1. Выделение приветствия без JSX

```
import React, { Component } from 'react';
class App extends Component {
  render() {
    return React.createElement(
      'h1',
      null,
      'Hello ',
      React.createElement('em', null, 'world'),
      '!',
    );
  }
}
export default App;
```

Реализация на JSX выглядит намного проще.

Листинг 3.2. Выделение приветствия с JSX

```
import React, { Component } from 'react';
class App extends Component {
  render() {
    return <h1>Hello <em>World</em>!</h1>;
  }
}
export default App;
```

Объекты, созданные в синтаксисе JSX, можно сохранять в переменных, потому что JSX всего лишь синтаксическое улучшение `React.createElement()`. В следующем примере перед возвратом управления ссылка на сгенерированный элемент сохраняется в переменной:

```
const title = <h1>Hello <em>World</em>!</h1>;
return title;
```

Код полностью идентичен строке 4 листинга 3.2; в нем просто используется дополнительная переменная перед возвратом.

3.2.2. Использование JSX с нестандартными компонентами

В предыдущем примере используется тег JSX `<h1>`, который также является стандартным тегом HTML. При работе с нестандартными компонентами применяется такой же синтаксис. Единственное различие заключается в том, что имя класса компонента должно начинаться с буквы в верхнем регистре, как в `<Title />`.

В листинге 3.3 приведена улучшенная версия приложения с тремя ссылками из главы 2, переписанная на JSX. В этом случае создается новый класс компонента, а JSX используется для создания элемента на его базе. Помните наш пример с `Link` из предыдущей главы? Код без JSX выглядит, как показано в листинге 3.3 (преобразован к рекомендованной структуре `App`).

Листинг 3.3. Три идентичные ссылки без JSX

```
import React, { Component, Fragment } from 'react';
class Link extends Component {
  render() {
    return React.createElement(
      'p',
      null,
      React.createElement(
        'a',
        {href: '//react.dev'},
        'Read more about React',
      ),
    );
  }
}
class App extends Component {
  render() {
    const link1 = React.createElement(Link);
    const link2 = React.createElement(Link);
    const link3 = React.createElement(Link);
    const group = React.createElement(Fragment, null, link1, link2, link3);
    return group;
  }
}
export default App;
```

С использованием JSX код приходит к листингу 3.4. Если выполнить его в браузере, результат будет таким же, как на рис. 2.13 в главе 2; напомним его на рис. 3.2.

Листинг 3.4. Три идентичные ссылки с JSX

```
import { Component, Fragment } from 'react';
class Link extends Component {
  render() {
    return (
      <p>
        <a href="//react.dev">Read more about React</a>
      </p>
    );
  }
}
class App extends Component {
  render() {
    return (
      <Fragment>
        <Link />
        <Link />
        <Link />
      </Fragment>
    );
  }
}
export default App;
```

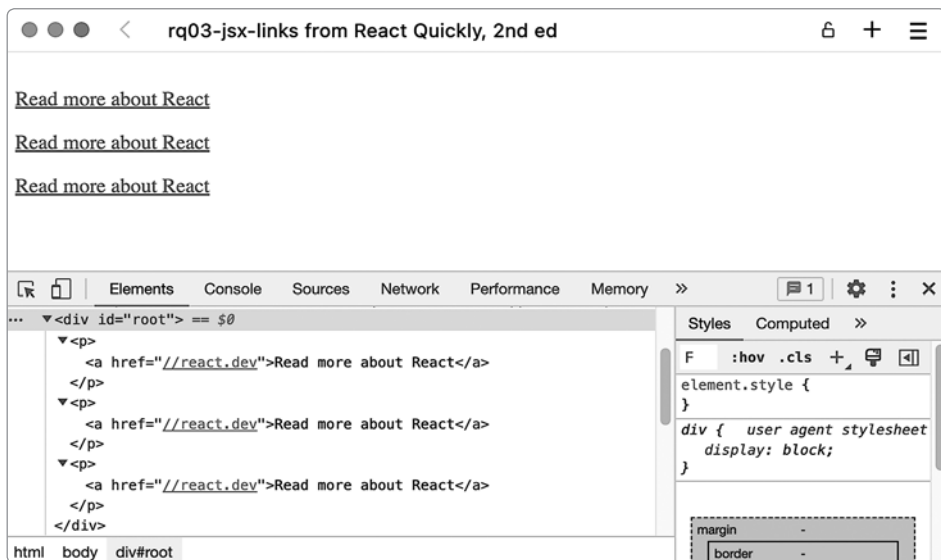
Создает компонент с именем Link, экземпляр которого может быть создан позднее с использованием записи JSX <Link />

Открывающая круглая скобка, которая начинает возвращаемое многострочное выражение JSX

Фрагменты React являются такими же элементами, как любые другие, и могут рендериться с использованием JSX

Закрывающая круглая скобка, которая завершает возвращаемое многострочное выражение JSX

Три идентичных экземпляра компонента Link

**Рис. 3.2.** Три идентичные ссылки в приложении, записанные на JSX

Репозиторий: rq03-jsx-links

Этот пример находится в репозитории `rq03-jsx-links`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq03-jsx-links --template rq03-jsx-links
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq03-jsx-links>

3.2.3. Многострочные объекты JSX

Вы, вероятно, заметили круглые скобки после возвращаемого многострочного объекта JSX в листинге 3.4. Они обязательны, если многострочный объект JSX начинается, например, в отдельной строке после `return`. Так создаются многострочные объекты JSX, которые не начинаются в той же строке.

```
return (
  <main>
    <h1>Hello world</h1>
  </main>
);
```

Как вариант, можно начать корневой элемент в одной строке с `return`, тогда круглые скобки `()` не обязательны. Например, так:

```
return <main>
  <h1>Hello world</h1>
</main>;
```

Недостатком второго варианта можно считать то, что открывающий тег `<main>` становится менее заметным: его легко пропустить. Выбор за вами. Мы будем применять только первый вариант, чтобы не создавать путаницы.

То же относится к любому другому применению многострочных объектов JSX, например, при сохранении их в переменной. Для этого также используются круглые скобки:

```
const message = (
  <main>
    <h1>Hello world</h1>
  </main>
);
```

3.2.4. Вывод переменных в JSX

Написанные компоненты должны уметь изменять свое представление на основе кода. Например, компонент для отображения даты/времени должен отображать текущую дату и время, а не жестко фиксированное значение.

При работе с React в JavaScript для подстановки переменных в строки приходится прибегать к строковым шаблонам в обратных апострофах или, еще хуже, к конкатенации. Например, чтобы использовать переменную как строку в компоненте `DateTimeNow` без JSX, придется написать следующий код:

```
class DateTimeNow extends React.Component {
  render() {
    const dateTimeNow = new Date().toLocaleString()
    return React.createElement(
      'span',
      null,
      `Current date and time is ${dateTimeNow}.`
    )
  }
}
```

В JSX же для динамического вывода переменных используются фигурные скобки `{}`, которые значительно сокращают код:

```
class DateTimeNow extends React.Component {
  render() {
    const dateTimeNow = new Date().toLocaleString()
    return <span>Current date and time is {dateTimeNow}</span>
  }
}
```

Ссылаясь на переменную, которая является элементом React (созданным с использованием JSX), можно напрямую вставить любой другой фрагмент JSX в текущем контексте:

```
const now = <date>{dateTimeNow}</date>;
const message = <p>Today is {now}</p>;
```

Этот вариант равнозначен прямой вставке элемента:

```
const message = <p>Today is <date>{dateTimeNow}</date></p>;
```

Вставляемые переменные также могут быть свойствами, а не локально определенными переменными:

```
<p>Hello {this.props.userName}, today is {dateTimeNow}</p>
```

Кроме того, можно вызывать методы компонента, которые вы создали сами. Это стандартный способ изоляции блоков функциональности, как показано в листинге 3.5.

Листинг 3.5. ButtonList с использованием метода

```
import { Component } from 'react';
class ButtonList extends Component {
  getButton(text) {
    return (
      <button disabled={this.props.disabled}>{text}</button>
    );
  }
  render() {
    return (
      <aside>
        {this.getButton('Up')}
        {this.getButton('Down')}
      </aside>
    );
  }
}
```

Определяет метод `getButton`, который получает аргумент `text` для ярлыка на кнопке

Кнопка зависит от другого свойства, передаваемого компоненту

Вызывает метод для вставки кнопки с нужным текстом

Конечно, пример в листинге 3.5 сильно упрощен, так как в большинстве случаев в подобных ситуациях будет использоваться дополнительный компонент. Тем не менее в некоторых сценариях методы компонентов могут оказаться полезными. Этот пример всего лишь показывает, что методы компонента можно вызывать прямо в JSX. В частности, в фигурных скобках можно выполнять произвольные выражения JavaScript, например отформатировать выводимую дату:

```
<p>Today is {new Date(Date.now()).toLocaleTimeString()}.</p>
```

А теперь перепишем приветствие с выделенным словом, чтобы оно сохранялось в переменной перед выводом (листинг 3.6). Затем разберемся, как работать со свойствами в JSX.

Листинг 3.6. Выделение приветствия с использованием JSX и переменной

```
import { Component } from 'react';
class App extends Component {
  render() {
    const world = <em>World</em>;
    return <h1>Hello {world}!</h1>;
  }
}
export default App;
```

3.2.5. Работа со свойствами в JSX

Эта тема уже затрагивалась, когда я представлял JSX: свойства элементов определяются с использованием синтаксиса атрибутов, а именно: запись `key1=value1 key2=value2...` в теге JSX используется для определения как атрибутов HTML, так и свойств компонента React. В этом отношении она напоминает синтаксис атрибутов в HTML/XML.

Иначе говоря, если вам понадобится передать свойства, запишите их в JSX так же, как вы бы это сделали в обычной разметке HTML. Кроме того, рендеринг стандартных атрибутов HTML осуществляется путем задания свойств элементов (см. раздел 2.3) в элементе React с тегом HTML. Например, следующий код задает стандартный атрибут HTML `href` для якоря `<a>`:

```
return <a href="// react.dev">Let's do React!</a>;
```

Этот же способ применяется для задания свойств нестандартных компонентов. Например, компонент `Link` из предыдущей главы можно использовать в JSX следующим образом:

```
return <Link url="//react.dev" framework="React" />;
```

Конечно, фиксированным значениям атрибутов не хватает гибкости. Если вы хотите повторно использовать компонент `Link`, атрибут `href` должен изменяться, чтобы он каждый раз отражал новый адрес. Это называется динамическим присваиванием значений в отличие от их жесткой фиксации. Сейчас мы пойдем дальше и рассмотрим компонент, который может использовать динамически сгенерированные значения для атрибутов. Эти значения можно брать из свойств компонентов (`this.props`). Дальше все просто. Требуется только использовать фигурные скобки (`{}`) внутри угловых скобок (`<>`) для передачи динамических значений свойств элементам.

Предположим, вы строите компонент, который будет использоваться для ссылок на учетные записи пользователей. Для тега `<a>` нужно будет задать значения нескольких атрибутов, но значения `href` и `title` должны быть разными, поэтому они не могут жестко фиксироваться в коде. Создадим динамический компонент `ProfileLink`, который рендерит ссылку, используя свойства `url` и `label` для `href` и `title` соответственно. Для передачи свойств в `<a>` используются `{}`:

```
class ProfileLink extends React.Component {  
  render() {  
    return (  
      <a  
        href={this.props.url}  

```

```

        title={this.props.label}
        target="_blank">Profile
      </a>
    );
  }
}

```

Откуда берутся значения свойств? Они определяются при создании `ProfileLink`, то есть в компоненте, который создает `ProfileLink` (в его родителе). Например, так передаются значения `url` и `label` при создании экземпляра `ProfileLink`, в результате чего генерируется тег `<a>` со следующими значениями:

```
<ProfileLink url="/users/johnny" label="Profile for Johnny" />
```

В предыдущей главе говорилось о том, что при рендере стандартных элементов (`<h>`, `<p>`, `<div>`, `<a>` и т. д.) React рендерит все свойства, даже если они не имеют семантического смысла в спецификации HTML. Это не особенность JSX, а стандартное поведение React.

Если у вас есть объект со свойствами, которые необходимо по очереди отрендерить на элементе, это можно сделать следующим образом:

```

return (
  <Post
    id={post.id}
    title={post.title}
    content={post.content}
  />
);

```

Такое решение отлично работает, и оно безопасно. Но если у вас имеется объект со значениями и вы хотите отрендерить их *все*, это можно сделать при помощи оператора расширения (`spread`):

```
return <Post {...post} />;
```

Заметим, что при этом будет отрендерено *каждое* свойство объекта `post`, неважно, требуется это или нет. Используйте этот процесс, если вы твердо уверены, что объект содержит только нужные вам свойства, или по крайней мере позаботьтесь о том, чтобы все лишние свойства игнорировались.

Таким способом можно даже отрендерить все свойства, переданные компоненту, в другом элементе внутри этого компонента, используя расширение `this.props`:

```
return <input value={this.value} {...this.props} />;
```

Впрочем, этот способ небезопасен, так как он позволяет родительскому компоненту передавать произвольные значения, которые заменяют любые значения, переданные ранее. Если `this.props` содержит свойство `value`, то оно заменит свойство `value`, заданное в компоненте перед расширением. Будьте очень осторожны при расширении объектов, особенно при расширении всех свойств, передаваемых компоненту. Мы вернемся к оператору расширения в следующей главе и рассмотрим некоторые типичные примеры его использования.

Специальное свойство: `children`

Вспомните, что в предыдущей главе было представлено специальное свойство `children`, которое выглядит как свойство только внутри нестандартного компонента, но не снаружи. При использовании JSX работать со свойством `children` становится намного удобнее. В примере с дочерними узлами из главы 2 иерархия компонентов выглядела, как показано на рис. 3.3.

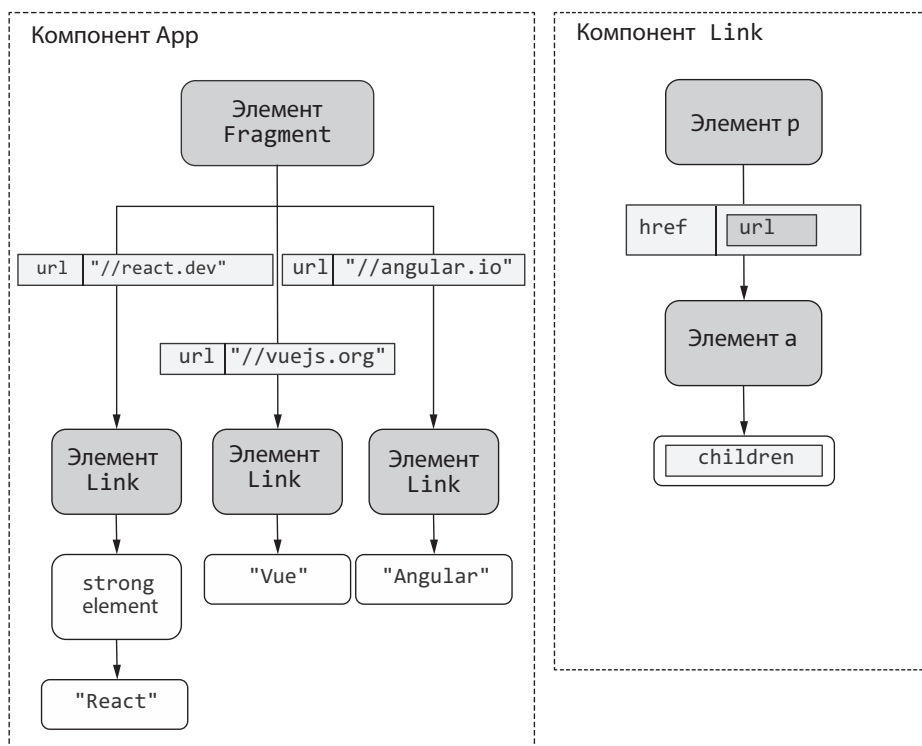


Рис. 3.3. Дерево компонентов при использовании дочерних узлов для представления содержимого ссылки

Реализуем этот пример на JSX. Вы уже знаете, что необходимо сделать.

Листинг 3.7. Список ссылок с дочерними узлами в JSX

```
import { Fragment, Component } from "react";
class Link extends Component {
  render() {
    return (
      <p>
        <a href={this.props.url}>
          {this.props.children}
        </a>
      </p>
    );
  }
}
class App extends Component {
  render() {
    return (
      <Fragment>
        <Link url="//react.dev">
          <strong>React</strong>
        </Link>
        <Link url="//vuejs.org">Vue</Link>
        <Link url="//angular.io">Angular</Link>
      </Fragment>
    );
  }
}
export default App;
```

Мы все еще используем свойство `url`, как раньше, а также `this.props.children`, как если бы это было любое другое свойство

Обратите внимание, как элегантно дочерние узлы добавляются в JSX. Они практически не отличаются от остального кода

Репозиторий: rq03-children

Этот пример находится в репозитории `rq03-children`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq03-children --template rq03-children
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq03-children>

Различия между использованием свойств и дочерних узлов неожиданно становятся намного более явными. Содержимое ссылки также можно было передать в виде свойства, но это выглядело бы довольно некрасиво. При использовании традиционного подхода со свойствами это выглядело бы так:

```
<Link
  url="//react.dev"
  content={<strong>React</strong>}
/>
```

Но с дочерними узлами код принимает следующий вид:

```
<Link url="//react.dev">
  <strong>React</strong>
</Link>
```

Любой разработчик, без сомнения, выберет второй вариант.

3.2.6. Условные конструкции в JSX

Условная логика в программировании всегда важна. Например, если пользователь прошел процедуру входа, на экране выводится информация его учетной записи; в противном случае отображается форма для ввода логина. Так как JSX представляет собой обычный код JavaScript, по сути, в нем можно использовать те же конструкции, как и при создании условного ветвления в обычном коде. Тем не менее появились некоторые стандартные схемы, которые многие разработчики применяют для организации ветвления в компонентах React, использующих JSX:

- Быстрый возврат при пустом рендеринге.
- Тернарный оператор для рендеринга альтернативных элементов.
- Логический оператор И (&&) для рендеринга необязательных элементов.
- Объектные отображения для рендеринга разнообразных элементов.
- Дополнительные компоненты для более сложного ветвления.

Последовательно рассмотрим все эти схемы, чтобы вы лучше понимали, как применяется условное ветвление в JSX и нестандартных компонентах React.

Быстрый возврат при пустом рендеринге

Представьте, что ваш компонент рендерит нечто актуальное только в случае истинности некоторого условия. Например, компонент для обратного отсчета рендерит значение только тогда, когда количество оставшихся секунд больше нуля.

Если компонент ничего не рендерит, из него можно просто вернуть `null`. Однако для оптимизации компонентов стоит сделать это как можно раньше, чтобы по возможности ускорить выполнение. Наша цель — выйти из простейшего

случая как можно быстрее, чтобы избежать лишних вычислений или создания объектов JSX, если они не понадобятся.

Создание компонента `Countdown` может выглядеть так:

```
class Countdown extends Component {
  render() {
    const seconds = this.props.remaining % 60;
    const minutes = Math.floor(this.props.remaining / 60);
    const message = <p>{minutes}:{seconds}</p>;
    if (seconds > 0 || minutes > 0) {
      return message;
    } else {
      return null;
    }
  }
}
```

Здесь нет ошибки — код работает, и он полностью функционален. Но вы увидите, что многие разработчики применяют быстрый выход, если компонент ничего не рендерит. Пустой рендеринг можно обнаружить еще до вычисления количества секунд и минут и создания объекта JSX:

```
class Countdown extends Component {
  render() {
    if (this.props.remaining === 0) {
      return null;
    }
    const seconds = this.props.remaining % 60;
    const minutes = Math.floor(this.props.remaining / 60);
    return <p>{minutes}:{seconds}</p>;
  }
}
```

Здесь также используется тот факт, что при возврате из блока `if` блок `else` не понадобится. Можно сказать, что присутствие `else` неявно — весь код после блока `if` выполняется только в том случае, если условие оказалось ложным.

Тернарный оператор для альтернатив

Другой очень распространенный случай в компонентах React — рендеринг разных элементов в зависимости от истинности или ложности некоторого условия. Например, в корзине должен выводиться список товаров, если покупатель выбрал хотя бы один товар, или сообщение об отсутствии товаров, если покупатель не выбрал ничего.

В JSX для этого можно использовать переменную, которой присваиваются разные значения в обычном блоке `if/else`. Однако такое решение получается

громоздким, и в React чаще используется тернарный оператор. Если конструкция `if/else` является командой, тернарный оператор представляет собой выражение и, как следствие, может встраиваться напрямую в JSX:

```
<p>User is {this.props.isOnline ? 'Online' : 'Offline'}</p>
```

Пользуясь этим, можно задать компонент корзины следующим образом:

```
class ShoppingCart extends Component {
  render() {
    return (
      <aside>
        <h1>Shopping cart</h1>
        {this.props.items.length === 0 ? (
          <p>Your cart is empty. Go buy something!</p>
        ) : (
          <CartItem items={this.props.items} />
        )}
      </aside>
    );
  }
}
```

Логические операторы для рендеринга необязательных элементов

Еще одна распространенная схема — выполнить рендеринг элемента, если некоторое условие истинно, но ничего не рендерить, если оно ложно. Например, рядом с именем проверенного пользователя должна отображаться галочка почета, а с именами непроверенных юзеров не должно отображаться ничего. Для этого можно воспользоваться логическим оператором `И` и тем фактом, что логические операторы используют ускоренное вычисление, возвращая управление, как только становится точно известно, что все выражение истинно. Таким образом, при обработке `a && b` JavaScript возвращает `a`, если значение `a` *квазиложно*, или `b`, если `a` *квазиистинно*. Если `a` квазиистинно, то уже неважно, какое значение содержит `b`; в любом случае будет возвращено именно оно. Это можно объединить с тем фактом, что React рендерит `false` как пустую строку (подробнее об этом позже).

Этим фактом можно воспользоваться для условного рендеринга элементов, чтобы логический оператор `И` возвращал `false` только в том случае, если пользователь не прошел проверку, или элемент React, если пользователь был проверен:

```
class UserName extends Component {
  render() {
    return (
```

```

    <p>
      {this.props.username}
      {this.props.isVerified && <Checkmark />}
    </p>
  );
}
}
}

```

Квазиистинность

В JavaScript *квазиистинное* (truthy) значение преобразуется в true при вычислении в логическом контексте. Например, в команде if

```

if (someVariable) {
  // Выполняется тогда и только тогда,
  // когда значение someVariable квазиистинно.
}

```

значение квазиистинно, если оно не квазиложно. Это не шутка, а настоящее принятое определение.

Существуют всего шесть квазиложных значений:

- false
- 0
- "" (пустая строка)
- null
- Undefined
- NaN (не число)

Эта схема довольно часто встречается в компонентах React, поэтому ее желательно знать.

Использование объектов для выбора

До сих пор мы имели дело со случаями «рендеринг элемента или ничего» или «рендеринг одного или другого элемента». Но что, если нужно отрендерить более двух типов элементов в зависимости от условия? В этом сценарии требуется отрендерить значок в зависимости от статуса сообщения в блоге. Если сообщение находится в состоянии черновика, рендерится значок черновика. Если пост опубликован, рендерится значок публикации. А если сообщение находится в любом другом состоянии (которым, как выясняется, оказывается только удаленное состояние), рендерится значок мусорной корзины.

В принципе, можно использовать вложение тернарных операторов, чтобы начать с проверки условия `status === "draft"`; если условие не выполняется, то проверяется условие `status === "published"`; а если не выполняется и оно, мы считаем сообщение удаленным:

```
class PostStatus extends Component {
  render() {
    return this.props.status === "draft" ?
      <DraftIcon /> :
      this.props.status === "published" ?
        <PublishedIcon /> :
        <TrashIcon />;
  }
}
```

Это решение работает, но красивым его не назовешь. Альтернатива — воспользоваться командой `switch` и просто возвращать разные значения в разных случаях. Однако существует и другое решение, более соответствующее декларативному стилю: в нем используется объект со свойствами для разных случаев, приводящими к разным результатам:

```
const status2icon = {
  draft: <DraftIcon />,
  published: <PublishedIcon />,
  deleted: <TrashIcon />,
};
class PostStatus extends Component {
  render() {
    return status2icon[this.props.status];
  }
}
```

Коротко и красиво, не так ли? Однако стоит отметить, что этот код не обрабатывает ситуацию, в которой статус не совпадает ни с одним из состояний. В предыдущей версии компонент выводил значок мусорной корзины, если сообщение не было ни черновым, ни опубликованным, но теперь корзина выводится только для сообщений со статусом удаленных.

Чтобы обработать ситуацию, в которой статус имеет непредвиденное значение, необходимо добавить в конец логическое ИЛИ, чтобы в случае, если при индексировании объекта не был получен никакой результат, все равно что-нибудь выводилось. Например, в любом непредвиденном случае должна выводиться мусорная корзина:

```
class PostStatus extends Component {
  render() {
```

```

    return status2icon[this.props.status] || status2icon.deleted;
  }
}

```

Вероятно, эта схема применяется в React реже остальных, но она может встречаться в простых ситуациях, подобных тем, которые мы рассмотрели.

Использование дополнительных компонентов для сложного ветвления

Описанные выше сценарии покрывают только простейшие случаи ветвления. Но что, если в компоненте содержится более сложная логика?

Представим компонент покупательской корзины с кнопками в нижней части. Требуется реализовать следующую бизнес-логику, которую требует заказчик:

- Если пользователь выполнил вход, выводится только кнопка оформления заказа **Checkout**.
- Если пользователь еще не выполнил вход, выводится кнопка **Login**, а также кнопка оформления заказа гостем **Checkout as Guest**.
- Если какой-то товар отсутствует на складе или корзина пуста, кнопка **Checkout** или **Checkout as Guest** блокируется.
- Если пользователь выполнил вход, но еще не указал номер платежной карты, отображается кнопка **Add Credit Card**.
- Если пользователь выполнил вход, ввел номер карты и указал адрес, рядом с кнопкой **Checkout** должна появиться кнопка покупки в один клик **One-Click Buy**. Эта кнопка блокируется по той же логике, что и кнопка **Checkout**.

Теперь реализуем все эти требования, используя описанные выше приемы.

Листинг 3.8. Сложная покупательская корзина

```

import { Component, Fragment } from "react";
class ShoppingCart extends Component {
  render() {
    const hasItems = this.props.items.length > 0;
    const isLoggedIn = this.props.user !== null;
    const hasCreditCard = isLoggedIn && this.props.user.creditcard !== null;
    const hasAddress = isLoggedIn && this.props.user.address !== null;
    const isAvailable = this.props.items.every((item) => !item.outOfStock);
    return isLoggedIn ? ( ← Первый тернарный оператор
      hasCreditCard ? ( ← Второй тернарный оператор

```

```

<Fragment>
  <button disabled={!hasItems || !isAvailable}>
    Checkout
  </button>
  {hasAddress && (
    <button
      disabled={!hasItems || !isAvailable}
    >
      One-click buy
    </button>
  )}
</Fragment>
) : (
  <button>Add credit card</button>
)
) : (
  <Fragment>
    <button>Login</button>
    <button disabled={!hasItems || !isAvailable}>
      Checkout as guest
    </button>
  </Fragment>
);
}
}
class App extends Component {
  render() {
    const items = [1, 2, 3];
    const user = { creditcard: null, address: true };
    return <ShoppingCart items={items} user={user} />;
  }
}
export default App;

```

Логическая операция И для
необязательного рендеринга кнопки

Повторяющаяся
логика для
заблокированной
кнопки

Репозиторий: rq03-cart-single

Этот пример находится в репозитории `rq03-cart-single`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq03-cart-single --template rq03-cart-single
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq03-cart-single>

Кажется, код покрывает все. Однако вложенные условные конструкции и дубликаты атрибутов усложняют его. В таких ситуациях часто лучше разбить код на несколько возможных вариантов.

Создадим новые компоненты `<UserButtons />` и `<GuestButtons />`. На верхнем уровне можно выбрать, какой компонент должен использоваться в каждом конкретном случае, а затем включить необходимые дополнительные проверки и условные конструкции в каждый компонент.

Листинг 3.9. Упрощенная многокомпонентная корзина

```
import { Component, Fragment } from "react";
class UserButtons extends Component {
  render() {
    const hasCreditCard = this.props.user.creditcard !== null;
    const hasAddress = this.props.user.address !== null;
    const disabled = !this.props.canCheckout;
    return hasCreditCard ? ( ← Тернарные операторы
      <Fragment>
        <button disabled={disabled}>Checkout</button>
        {hasAddress && ( ← Логическое И для необязательного рендеринга
          <button disabled={disabled}>One-click buy</button>
        )}
      </Fragment>
    ) : (
      <button>Add credit card</button>
    );
  }
}
class GuestButtons extends Component {
  render() {
    return (
      <Fragment>
        <button>Login</button>
        <button disabled={!this.props.canCheckout}>
          Checkout as guest
        </button>
      </Fragment>
    );
  }
}
class ShoppingCart extends Component {
  render() {
    const hasItems = this.props.items.length > 0;
    const isLoggedIn = this.props.user !== null;
    const isAvailable = this.props.items.every((item) => !item.outOfStock);
    const canCheckout = hasItems && isAvailable;
    return isLoggedIn ? ( ← Тернарные операторы
      <UserButtons user={this.props.user} canCheckout={canCheckout} />
    ) : (
      <GuestButtons canCheckout={canCheckout} />
    );
  }
}
class App extends Component {
```

```
render() {
  const items = [1, 2, 3];
  const user = { creditcard: null, address: true };
  return <ShoppingCart items={items} user={user} />;
}
}
export default App;
```

Репозиторий: rq03-cart-multi

Этот пример находится в репозитории `rq03-cart-multi`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq03-cart-multi --template rq03-cart-multi
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq03-cart-multi>

Это решение работает точно так же и имеет точно такую же сложность, но каждый компонент намного проще, и каждый компонент легко понять по отдельности, читая его код. Можно пойти дальше и разбить компонент `<UserButtons>` на два компонента: «есть платежная карта» и «нет платежной карты».

Конечно, нужно признать, что чем больше компонентов, тем больше кода, а чем больше кода, тем выше затраты памяти и нагрузка на процессор (в общем случае), так что последний пример расходует чуть больше ресурсов, чем первый. Впрочем, в большинстве приложений эти различия пренебрежимо малы и качество кода часто важнее мелких оптимизаций.

3.2.7. Комментарии в JSX

Так как JSX записывается внутри JavaScript, в элементах JSX можно использовать обычные комментарии JavaScript:

```
// Это заголовок страницы
const title = <h1>Hello world!</h1>;
```

Но если вы используете очень длинные сегменты кода JSX, можно добавить встроенные комментарии. При этом способе не всегда удастся использовать обычные комментарии JavaScript.

Чтобы добавить комментарии в JSX между тегами, заключите стандартные комментарии JavaScript `/**/` или `//` в `{}`:

```
const content = (
  <div>
    { /* Выглядит как обычный комментарий JS */ }
    { /* Можно писать комментарий
      в несколько строк */ }
    { // Или одной строкой }
  </div>
);
```

Можно добавлять комментарии JavaScript напрямую, используя `/**/` или `//` внутри тегов:

```
const content = (
  <div>
    <input
      /* Этот элемент
      рендерится, потому что... */
      name={this.props.name} // Тут оставлен важный комментарий
    />
  </div>
);
```

Отметим, что при добавлении обычного однострочного комментария между тегами в фигурных скобках необходимо вставить символ новой строки до завершения фигурных скобок. Вот такой код работать не будет:

```
const content = (
  <div>
    { // Это НЕ работает! }
  </div>
);
```

Такая запись приведет к ошибке компиляции, потому что закрывающая фигурная скобка считается частью комментария и у открывающей фигурной скобки не будет парной закрывающей скобки. В этом случае парсер выдаст сообщение об ошибке.

3.2.8. Списки объектов JSX

В элементах React часто встречается один прием: отображение массива элементов на массив объектов JSX, которые должны возвращаться в компоненте. Предположим, необходимо создать компонент для рендеринга раскрывающегося

списка. Требуется передать список вариантов в массиве строк новому компоненту `<Select />`. Так выглядит желаемая реализация:

```
class App extends Component {
  render() {
    const items = ['apples', 'pears', 'playstations'];
    return <Select items={items} />;
  }
}
```

Тогда компонент `Select` должен правильно рендерить `<select>` с элементами `<option>` в HTML. Как это сделать? Наивное решение — просто декларативно отобразить элементы из строк в объекты JSX, как показано в листинге 3.10.

Листинг 3.10. Наивная реализация select

```
import { Component } from "react";
class App extends Component {
  render() {
    const items = ["apples", "pears", "playstations"];
    return <Select items={items} />;
  }
}
class Select extends Component {
  render() {
    return (
      <select>
        {this.props.items.map((item) => (
          <option>{item}</option>
        ))}
      </select>
    );
  }
}
export default App;
```

← Для каждого элемента массива items
 ← Возвращает элемент JSX

Репозиторий: rq03-naive-select

Этот пример находится в репозитории `rq03-naive-select`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq03-naive-select --template rq03-naive-select
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq03-naive-select>

Подобная попытка решить задачу выглядит вполне достойно. Однако при выполнении в браузере будет выдано предупреждение:

Warning : Each child in a list should have a unique "key" prop.¹

Приложение работает, но вы получите предупреждение об отсутствующем свойстве `key`. Пожалуй, на этом этапе изучения React еще рано говорить об использовании свойства `key`; оно применяется для отслеживания перемещений элемента в отрендеренной модели DOM. Если перемещается один и тот же элемент, React будет повторно использовать тот же элемент; но если React не знает, тот это элемент или нет, он будет удалять все старые элементы и заново создавать новые при каждом рендеринге списка.

Для целей данного примера можно просто использовать значение `item` как свойство `key` корневого элемента, возвращаемого в отображенном массиве. Так мы приходим к коду из листинга 3.11.

Листинг 3.11. Верная реализация select

```
import { Component } from "react";
class App extends Component {
  render() {
    const items = ["apples", "pears", "playstations"];
    return <Select items={items} />;
  }
}
class Select extends Component {
  render() {
    return (
      <select>
        {this.props.items.map((item) => (
          <option key={item}>{item}</option>
        ))}
      </select>
    );
  }
}
export default App;
```

← Мы добавили свойство `key` в элемент `<option>`

Свойство `key` — внутреннее свойство React, которое никогда не рендерится в DOM. Рекомендуется хранить в свойстве `key` уникальный идентификатор элемента, а не просто индекс элемента в массиве (если элементы перемещаются в массиве, индексы меняются, хотя сами элементы остаются неизменными, что препятствует повторному использованию элемента).

¹ Предупреждение: каждый дочерний узел в списке должен иметь уникальное свойство "key".

Репозиторий: rq03-correct-select

Этот пример находится в репозитории `rq03-correct-select`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq03-correct-select --template rq03-correct-select
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq03-correct-select>

Значения ключей должны быть уникальными. При рендеринге списка с не-уникальными ключами вы будете получать в консоли предупреждения о дублирующихся ключах.

ПРИМЕЧАНИЕ Ключи локальны для конкретного массива, так что они должны быть уникальными внутри отдельного массива, а не между всеми массивами приложения или даже компонента. Разные массивы объектов JSX могут содержать одинаковые ключи — важно лишь, чтобы одинаковых ключей не было в одном массиве.

Как уже отмечалось, эта возможность довольно сложна, чтобы объяснить ее на этом этапе, так что пока просто запомните: если вы видите в консоли предупреждение об отсутствующем свойстве `key` или повторяющихся ключах, причина именно такова, как мы указали.

3.2.9. Фрагменты в JSX

Мы уже не раз обращались к компонентам JSX. Они используются для экспортирования нескольких элементов на одном уровне в ситуациях, когда допустим только один элемент. Что-то похожее мы уже проделывали для включения заголовка со ссылкой:

```
import { Fragment } from 'react';
...
return (
  <Fragment>
    <h1>Hello and welcome</h1>
    <a href="/blog">Go to the blog</a>
  </Fragment>
);
```

Тем не менее, начиная с React 16.2 (и Babel 7), поддерживается и более короткий синтаксис. Теперь даже не нужно импортировать компонент `Fragment`:

```
return (
  <>
    <h1>Hello and welcome</h1>
    <a href="/blog">Go to the blog</a>
  </>
);
```

Новый сокращенный синтаксис использует пустой тег для рендеринга элементов. При этом синтаксис с `<></>` не может получать никакие атрибуты или свойства. Впрочем, единственным свойством, которое, может быть, захочется применить, будет свойство `key`, поскольку рендерится список элементов, в котором каждый элемент содержит более одного элемента JSX в корне.

Классический сценарий — список определений. Он задается в HTML следующим образом:

```
<dl>
  <dt>Term A</dt>
  <dd>Description of Term A.</dd>
  <dt>Term B</dt>
  <dd>Description of Term B.</dd>
</dl>
```

Как видите, каждая запись требует присутствия двух одноуровневых элементов в списке для рендеринга (`<dt>` и `<dd>`).

Например, если создать приложение, выводящее названия трех пород собак с краткой информацией о каждой, необходимо отобразить названия пород и определения на два элемента. Для этого их надо обернуть во фрагмент, но, поскольку фрагмент должен иметь свойство `key`, приходится применять литеральный компонент `Fragment`, и к сожалению, при этом не удастся использовать сокращенный синтаксис, о котором говорилось выше. Код реализации см. в листинге 3.12.

Листинг 3.12. Список определений пород собак

```
import { Component, Fragment } from "react";
class App extends Component {
  render() {
    const list = [
      { breed: "Chihuahua", description: "Small breed of dog." },
      { breed: "Corgi", description: "Cute breed of dog." },
      { breed: "Cumberland Sheepdog", description: "Extinct breed of dog." },
    ];
```

```

    return <Breeds list={list} />;
  }
}
class Breeds extends Component {
  render() {
    return (
      <dl>
        {this.props.list.map(
          ({ breed, description }) => (
            <Fragment key={breed}>
              <dt>{breed}</dt>
              <dd>{description}</dd>
            </Fragment>
          )
        )}
      </dl>
    );
  }
}
export default App;

```

Использует деструктуризацию для простого обращения к свойствам элемента списка

Так как требуется свойство `key`, приходится использовать компонент `Fragment`. Обратите внимание: в качестве ключа можно было использовать `breed`, так как это значение однозначно определяет каждый элемент массива

Репозиторий: `rq03-dog-breeds`

Этот пример находится в репозитории `rq03-dog-breeds`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq03-dog-breeds --template rq03-dog-breeds
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq03-dog-breeds>

Запустив приложение в браузере, получаем именно тот список определений, который был нужен (рис. 3.4). Использование фрагментов с ключами не настолько экзотический случай, как может показаться, поэтому очень полезно узнать его уже на этом этапе.

Итак, вы познакомились с JSX и его преимуществами. Оставшаяся часть главы будет посвящена инструментам JSX и возможным ловушкам, которых необходимо избегать.

Прежде чем продолжить, следует уяснить, что для правильного функционирования любого проекта JSX код JSX должен быть откомпилирован. Браузеры не могут выполнять JSX напрямую — только JavaScript, поэтому необходимо

транспилировать код JSX в обычный код JS (см. рис. 3.1). К счастью, это намного проще, чем может показаться на первый взгляд.

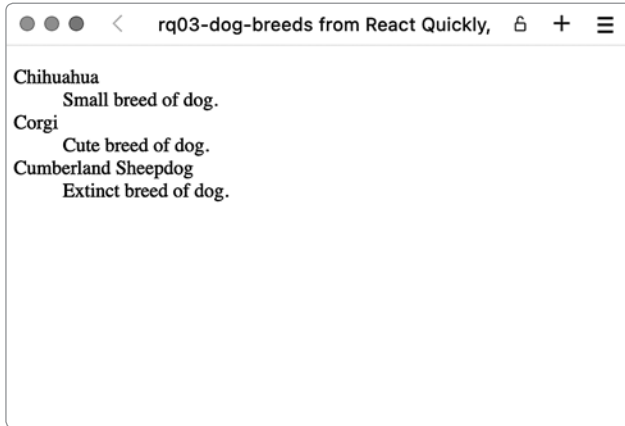


Рис. 3.4. Список определений подмножества пород собак с кратким описанием

3.3. ТРАНСПИЛИЦИЯ JSX

Для запуска проектов и примеров в этой книге самостоятельная настройка транспилятора (или большинства других позиций технологического стека) не требуется. Сказанное относится к большинству проектов, с которыми вы столкнетесь на практике, потому что в существующих проектах уже имеется рабочий технологический конвейер, и новые проекты могут базироваться на документированных описаниях лучших практик того фреймворка, с которым вы хотите работать.

Но можно и настроить проект React с нуля. Для этого придется настроить транспилятор JSX. Самый популярный инструмент для транспиляции JSX — Babel, однако существуют и другие, которые также заслуживают внимания. Некоторые альтернативы могут входить в пакет средств сборки, который упрощает сопровождение всей конфигурации, или служат простой заменой Babel. Среди альтернатив можно выделить SWC, Sucrase и esbuild. О том, как ими пользоваться, можно узнать из документации:

- Babel: <https://babeljs.io/>
- SWC: <https://swc.rs/>
- Sucrase: <https://sucrase.io/>
- esbuild: <https://esbuild.github.io/>

Мы рекомендуем не тратить много времени на самостоятельную настройку транспилятора JSX. Классные ребята из React создали CRA, который сделает почти все, что нам нужно в этой книге. Возможно, вы захотите заняться нестандартной настройкой в собственных проектах, но для книги будет достаточно того, что умеет делать наша подборка замечательных инструментов.

3.4. ВОЗМОЖНЫЕ ПРОБЛЕМЫ REACT И JSX

В этом разделе рассматриваются некоторые особые случаи и аномалии, о которых нужно знать при использовании JSX:

- Самозакрывающиеся (одиночные) теги необходимы для листовых узлов.
- Специальные символы записываются в литеральной форме.
- Преобразования строк выглядят нетипично.
- Атрибут `style` является объектом.
- Некоторые атрибуты имеют зарезервированные имена, и их необходимо переименовать.
- Имена атрибутов, состоящие из нескольких слов, записываются в «верблюжьем регистре».
- Логические атрибуты обрабатываются не так, как в HTML.
- Некоторые пробелы теряются (но не все).
- Возможность добавлять атрибуты `data-` при необходимости.

3.4.1. Самозакрывающиеся элементы

JSX требует включать закрывающий слеш (/) либо в закрывающий тег, либо, при отсутствии дочерних элементов, в конец одиночного тега. Например, следующая запись верна:

```
<a href="//react.dev">React</a>

```

Следующая запись ошибочна, так как в обоих узлах отсутствует конечный тег:

```
<a href="//react.dev">React

```

Вы, вероятно, знаете, что формат HTML менее требователен. Браузеры игнорируют отсутствие слеша или конечного тега и прекрасно рендерят элемент

без них. В этом нетрудно убедиться: создайте файл HTML, содержащий только разметку `<button>Press me`. Кнопка отобразится нормально!

3.4.2. Специальные символы

Сущности HTML представляют собой коды для отображения специальных символов: знаков авторского права, дефисов, длинных тире, кавычек и т. д. Несколько примеров:

```
&copy;
&mdash;
&ldquo;
```

Эти коды можно вывести так же, как любую другую строку, в текстовом контенте внутри узла или в атрибуте узла. Для примера возьмем статический JSX (текст, определяемый в коде без переменных или свойств):

```
<span>&copy;&mdash;&ldquo;</span>
<input value="&copy;&mdash;&ldquo;" />
```

Но если вы попытаетесь выводить сущности HTML в `<span>` динамически (из переменной или свойства), то получите прямой вывод (`©—“`), а не специальные символы. Например, такой код работать не будет:

```
// Антипаттерн. НЕ работает!
const specialChars = '&copy;&mdash;&ldquo;';
<span>{specialChars}</span>
<input value={specialChars}/>
```

React/JSX автоматически экранирует опасную разметку HTML, что хорошо для безопасности (безопасность по умолчанию — классная штука!). Чтобы вывести специальные символы, придется использовать один из следующих способов:

- Скопировать специальный символ прямо в исходный код (проследите, чтобы использовалась кодировка UTF-8). При работе со специальными символами рекомендуется именно этот способ.
- Экранировать специальный символ префиксом `\u` и использовать код Юникода (информация доступна на сайте fileformat.info).
- Преобразовать код символа в строку вызовом `String.fromCharCode(charCodeNumber)`.
- Использовать специальное свойство `dangerouslySetInnerHTML` для задания внутренней разметки HTML (это опасный способ, и применять его не рекомендуется).

Чтобы разобраться в последнем способе (это крайнее средство: если на «Титанике» все плохо, бегите к шлюпкам!), взгляните на следующий код:

```
const specialChars = '&copy;&mdash;&ldquo;';
<span dangerouslySetInnerHTML={{__html: specialChars}}/>
```

У создателей React явно есть чувство юмора, раз они назвали свойство `dangerouslySetInnerHTML`!

3.4.3. Преобразование строк

Когда React выводит значение переменной (или выражения в общем), в каком виде оно рендерится? React может рендерить одно из двух: либо строку, которая становится строковым контентом между элементами, либо элемент, который становится просто элементом, как если бы он был отрендерен напрямую. Но как «что-то» преобразуется в строку, если это не элемент? В этом случае React ведет себя нетипично, так как учитывает тип выражения, рендеринг которого проводится. Таблица 3.2 дает представление о возможном выводе для разных примитивных значений в JavaScript.

Таблица 3.2. Рендеринг различных типов в React

Тип	Вывод
"string"	"string"
""	""
3.4	"3.4"
0	"0"
NaN	"NaN"
Number.POSITIVE_INFINITY	"Infinity"
Number.NEGATIVE_INFINITY	"-Infinity"
true	"true"
false	""
undefined	""
null	""

Здесь можно заметить несколько неожиданностей. Что самое важное, `false` преобразуется в пустую строку, а `true` — в `"true"`. Таким образом, четыре

квазиложных значения (пустая строка, `false`, `null` и `undefined`) становятся пустыми строками.

А как насчет значения `0`, которое также является квазиложным? Оно превращается в `0`. Было бы странно, если бы в компонентах нельзя было вывести `0`, так что это в какой-то степени необходимо. Наконец, `NaN` также превращается в `"NaN"`, а не в пустую строку. Обычно это помогает в отладке вычислений: увидев `NaN`, вы знаете, что где-то допущена ошибка, но если вы ничего не видите, то поиски займут больше времени.

Этот факт — `false` не рендерит ничего, но `0` что-то рендерит — особенно важен при использовании логического И для рендеринга необязательных элементов, о чем говорилось выше. Возможно, вы привыкли использовать в JavaScript конструкции вида:

```
if (items.length) {
  hasItems = true;
}
```

Здесь `items.length` используется в качестве условия для команды `if`, потому что мы знаем, что значение `0` в любом случае квазиложно, поэтому писать `items.length > 0` не обязательно — истинность выражения от этого не изменится.

Тем не менее поступать так в JSX не стоит. Предположим, необходимо отрендерить в корзине кнопку `Checkout`, если в корзине есть хотя бы один товар, но при отсутствии товаров ничего рендериться не должно:

```
class ShoppingCart extends Component {
  render() {
    return (
      <aside>
        <h1>Shopping cart</h1>
        <CartItem items={this.props.items} />
        {this.props.items.length && (
          <button>Checkout</button>
        )}
      </aside>
    );
  }
}
```

Не делайте так, потому что использование длины массива как условия напрямую в выражении И приведет к проблемам

Такое решение работает, если в корзине более `0` товаров. Но что произойдет, если там `0` товаров? Логическое выражение И, выделенное в аннотации, применяет ускоренное вычисление и возвращает первое квазиложное значение в его исходном виде. Кроме того, поскольку длина массива равна `0`, итоговое значение выражения неожиданно равно `0`, и оно рендерится в документе как `"0"`

(см. табл. 3.2). Таким образом, если использовать приведенный код, для пустой корзины внезапно внизу будет выведен "0", что довольно странно.

Всегда проверяйте, что длина массива больше 0, чтобы гарантировать, что результат имеет логический тип. А лучше сохранить сравнение в другой переменной, тогда читать код будет еще проще:

```
class ShoppingCart extends Component {
  render() {
    const hasItems = this.props.items.length > 0;
    return (
      <aside>
        <h1>Shopping cart</h1>
        <CartItem items={this.props.items} />
        {hasItems && <button>Checkout</button>}
      </aside>
    );
  }
}
```

Сохраняет сравнение в переменной с гарантированно логическим типом

Использует эту переменную для условного рендеринга необязательного элемента

Во время разработки об этом нередко забывают, поэтому частенько в приложении обнаруживаются посторонние нули. Они почти всегда оказываются результатом такого рода выражений.

3.4.4. Атрибут style

Атрибут `style` в JSX работает не так, как в обычной разметке HTML. В JSX вместо строки должен передаваться объект JavaScript, а свойства CSS должны записываться в «верблюжем регистре». Несколько примеров:

- `background-image` превращается в `backgroundImage`;
- `font-size` превращается в `fontSize`;
- `font-family` превращается в `fontFamily`.

Объект JavaScript можно сохранить в переменной или отрендерить его «на месте» с двойными фигурными скобками (`{{...}}`). Двойные фигурные скобки необходимы, поскольку одна пара нужна для JSX, а другая — для объектного литерала JavaScript.

Предположим, имеется объект и размер его шрифта:

```
const smallFontSize = { fontSize: '10pt' };
```

В разметке JSX можно использовать объект `smallFontSize`:

```
<input style={smallFontSize} />
```

или задать больший размер шрифта (30pt), передав значение напрямую без дополнительной переменной

```
<input style={{ fontSize: '30pt' }} />
```

Рассмотрим другой пример прямой передачи стилей. На этот раз `<span>` назначается красная рамка:

```
<span style={{
  borderWidth: '1px',
  borderStyle: 'solid',
  borderColor: 'red',
}}>Red velvet cake is delicious</span>
```

Значение `border` может быть и таким:

```
<span style={{border: '1px red solid'}}>Ней</span>
```

Главная причина, по которой стили представляют не строками CSS, а объектами JavaScript, — так их быстрее обрабатывает React при применении изменений к представлениям.

3.4.5. Резервированные имена `class` и `for`

React (и JSX) принимают любые атрибуты, которые представляют собой стандартные атрибуты HTML, за исключением `class` и `for`. Эти имена являются зарезервированными словами в JavaScript/ECMAScript (для создания классов и циклов `for` соответственно), а JSX преобразуется в JavaScript. Таким образом, по аналогии с тем, как нельзя создать переменную с именем `for` или любым другим зарезервированным словом, нельзя создать и атрибуты с этими именами (во всяком случае, не напрямую).

Вместо них следует использовать `className` и `htmlFor` соответственно. Например, если необходимо применить к элементу имя класса `"hidden"`, придется использовать атрибут `className`:

```
<p className="hidden">...</p>
```

Если понадобится создать метку для элемента формы, используйте `htmlFor`:

```
<input type="checkbox" id={this.props.id} value="hasCorgi" />
<label htmlFor={this.props.id}>Corgi?</label>
```

Оба атрибута достаточно легко запомнить, а если вы забудете о них, компилятор выдаст сообщение об ошибке.

3.4.6. Атрибуты из нескольких слов

Как и два зарезервированных имени, о которых шла речь в предыдущем разделе, другие атрибуты HTML в React тоже переименовываются. И иногда в новых именах разобраться довольно сложно.

Любой атрибут, состоящий более чем из одного английского слова, переименовывается по схеме «верблюжьего регистра». Это выглядит нормально для атрибутов SVG (Scalable Vector Graphics), использующих дефисы, таких как `clip-path` или `fill-opacity`. Использовать атрибуты с дефисами в JSX невозможно, поэтому они переименовываются в `clipPath` и `fillOpacity` соответственно.

Однако то же самое происходит с атрибутами HTML, которые не используют дефисы, но обычно записываются в нижнем регистре, что может создать путаницу. Если ввести

```
return <video autoplay>...</video>;
```

в JSX, эта команда работать не будет, потому что атрибут называется `autoplay` (и может записываться строчными буквами в HTML), а в React нужно использовать «верблюжий регистр» и назвать его `autoPlay`. Для кого-то это может стать неприятным сюрпризом. То же самое справедливо для многих свойств, которые часто используются в HTML.

Вместо того чтобы выводить предупреждение о пропущенных свойствах, React незаметно фильтрует их. Таким образом, вы можете даже не догадываться о том, что имя введено неверно, пока не поймете, что видео не воспроизводится автоматически (поскольку необходим `autoPlay` вместо `autoplay`), `iframe` не поддерживает полноэкранный режим (необходим `allowFullscreen` вместо `allowfullscreen`) или в поле ввода не ограничивается количество символов (`maxLength` вместо `maxlength`).

3.4.7. Значения логических атрибутов

Некоторые атрибуты (такие, как `disabled`, `required`, `checked`, `autofocus` и `readOnly`) относятся только к элементам форм. Здесь важнее всего помнить, что значение атрибута должно задаваться в выражении JavaScript (то есть внутри `{}`), а не в строке. Например, `{false}` используется для разрешения ввода:

```
<input disabled={false} />
```

Но не используйте значение `"false"`, потому что оно пройдет проверку на истинность (непустая строка считается истинным значением в JavaScript, как упоминалось в разделе 3.2.6). Дело в том, что строка `"false"` не является ни одним из шести квазиложных значений; это непустая строка, которая является квазиистинным значением, так что в результате будет получено значение `true` и React отрендерит поле ввода как отключенное (`disabled` будет присвоено значение `true`):

```
<input disabled="false" /> // Don't do this!
```

Если опустить значение после свойства, то React будет считать, что оно равно `true`:

```
<input required />
```

Это то же самое, что присвоить `true` вручную, поэтому просто используйте приведенный выше код вместо `required={true}`.

Для многих атрибутов в HTML отсутствие значения означает присваивание ему `false`, так что если вы хотите присвоить атрибуту именно `true` или `false`, просто включите его без указания значения или опустите. Чтобы задать значение в зависимости от содержимого переменной, используйте выражение:

```
<input readOnly={!isEditable} />
```

ПРИМЕЧАНИЕ Обращайте внимание на особенность записи атрибутов из нескольких слов. Этот логический атрибут в React называется `readOnly`, а не `readonly`, как в HTML.

Нестандартные компоненты с логическими свойствами

Все сказанное относится и к созданию собственных компонентов. Если у вас есть нестандартный компонент, которому нужно передать логическое свойство, можно просто использовать свойство из `this.props`, как если бы оно было логическим, а React присвоит ему `true`, если так указано при использовании.

Например, можно создать компонент, который будет выводить сообщение с сигналом для пользователя. Это может быть либо сообщение об ошибке, либо предупреждение. Чтобы управлять уровнем сигнала, мы добавляем флаг `isError`; если он равен `true`, к сообщению добавляется значок тревоги. Затем компонент используется для вывода двух разных сигналов в приложении — ошибки и предупреждения (листинг 3.13). При выполнении в браузере вы увидите, что сообщения выводятся верно (рис. 3.5).

Листинг 3.13. Передача и получение логических свойств в JSX

```
import { Component } from 'react';
class Alert extends Component {
  render() {
    return (
      <p>
        {this.props.isError && '⚠'}
        {this.props.children}
        {this.props.isError && '⚠'}
      </p>
    );
  }
}
class App extends Component {
  render() {
    return (
      <main>
        <Alert>We are almost out of cookies</Alert>
        <Alert isError>
          We are completely out of ice cream
        </Alert>
      </main>
    );
  }
}
export default App;
```

← Задаёт свойству `isError` значение `true`, для чего свойство просто включается в JSX без значения

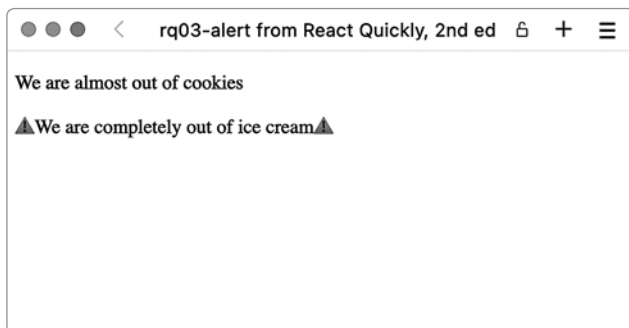


Рис. 3.5. Первое сообщение является предупреждением, а второе определенно указывает на ошибку

Репозиторий: rq03-alert

Этот пример находится в репозитории `rq03-alert`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq03-alert --template rq03-alert
```


На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq03-alert>

3.4.8. Пробелы

Если вы хотите добавить пробелы между компонентами (например, при включении в предложение слова, выделенного жирным шрифтом), необходимо тщательно следить за размещением символов новой строки. Допустим, вы хотите написать заголовок с выделенным словом в середине: например, "All corgis are awesome"¹, но слово "corgis" должно быть выделено курсивом. В листинге 3.14 показано, как могла бы выглядеть наивная реализация в JSX.

Листинг 3.14. Наивная реализация сообщения с частичным выделением

```
import { Component } from 'react';
class App extends Component {
  render() {
    return (
      <h1>
        All   ← Какой-то простой текст
        <em>corgis</em>   ← Затем узел JSX, а затем...
        are awesome ← ...еще немного простого текста
      </h1>
    );
  }
}
export default App;
```

Репозиторий: rq03-bad-whitespace

Этот пример находится в репозитории rq03-bad-whitespace. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq03-bad-whitespace --template rq03-bad-whitespace
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq03-bad-whitespace>

¹ Все корги милые. — *Примеч. пер.*

Все логично, не так ли? Запустите это приложение при помощи CRA и посмотрите, как оно выглядит в браузере. Результат показан на рис. 3.6.

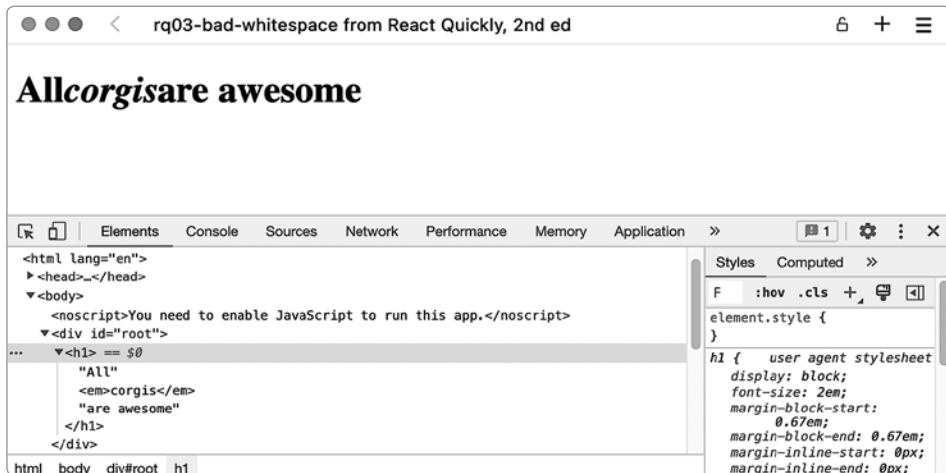


Рис. 3.6. Сообщение с неправильно расставленными пробелами

Что-то явно пошло не так. Пробелы вокруг слова `corgis` пропали. Что произошло? Обычно символы новой строки и табуляции JSX не считает пробелами. Когда выше в этой главе приводился код JSX

```
return (
  <main>
    <h1>Hello world</h1>
  </main>
);
```

нам не были нужны пробелы между элементами `<main>` и `<h1>`. Код был просто отформатирован в несколько строк для красоты, а не потому, что мы хотели вывести в браузере лишние пробелы. Итак, если между элементами JSX встречаются пробелы, включающие символы новой строки, то все они удаляются. При этом неважно, что в конце простой текстовой строки в листинге 3.14 находятся «нормальные» пробелы. Если между элементами располагается символ новой строки, удаляются все пробелы.

Как же поступить? Отрендерить пробелы можно двумя способами:

- Вообще не использовать символы новой строки между элементами.
- Добавить пробелы как выражения в коде.

Второе решение может показаться излишне сложным, но иногда оно единственно возможное. Рассмотрим первое решение — отказ от символов новой строки — на практике.

Листинг 3.15. Сообщение с частичным выделением без символов новой строки

```
import { Component } from 'react';
class App extends Component {
  render() {
    return (
      <h1>
        All <em>corgis</em> are awesome
      </h1>
    );
  }
}
export default App;
```

Внутри заголовка символов новой строки нет

Символы новой строки находятся до и после заголовка

Обратите внимание: до и после сообщения допустимы символы новой строки (потому что пробелы здесь могут быть свернуты — они нас не интересуют). Мы просто не хотим, чтобы символы новой строки появлялись там, где должны быть реальные пробелы. А теперь рассмотрим выражения-пробелы в листинге 3.16.

Листинг 3.16. Сообщение с частичным выделением с выражениями-пробелами

```
import { Component } from 'react';
class App extends Component {
  render() {
    return (
      <h1>
        All
        {" "}
        <em>corgis</em>
        {" "}
        are awesome
      </h1>
    );
  }
}
export default App;
```

Пробелы, вставленные как выражения

Здесь пробелы добавляются в фигурных скобках. Тем самым мы заставляем ядро JSX включить пробелы как реальные элементы и не рассматривать их как

часть незначимых пропусков, обычно существующих между элементами. Часто разработчики добавляют такие пробелы в виде выражений в конце строки перед символом новой строки, как показано в листинге 3.17.

Листинг 3.17. Сообщение с частичным выделением с меньшим количеством строк

```
import { Component } from 'react';
class App extends Component {
  render() {
    return (
      <h1>
        All{" "}
        <em>corgis</em>{" "}
        are awesome
      </h1>
    );
  }
}
export default App;
```

Выражения-пробелы присоединяются в конце строк

Репозиторий: rq03-good-whitespace

Этот пример находится в репозитории `rq03-good-whitespace`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq03-good-whitespace --template
  rq03-good-whitespace
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq03-good-whitespace>

Оба листинга корректно выводят сообщение, показанное на рис. 3.7, справедливость которого никто не может оспорить.

3.4.9. Атрибуты `data-`

Иногда требуется передать дополнительные данные с использованием узлов DOM. Хотя модель DOM не должна использоваться как база данных или локальное хранилище, иногда это бывает необходимо для передачи переменных

сторонним библиотекам. Если вам требуется создать нестандартные атрибуты и отрендерить их, используйте префикс `data-`.

Например, так выглядит действительный нестандартный атрибут `data-object-id`, который React сгенерирует в представлении (разметка HTML будет совпадать со следующим кодом JSX):

```
<li data-object-id={object.id}>...</li>
```

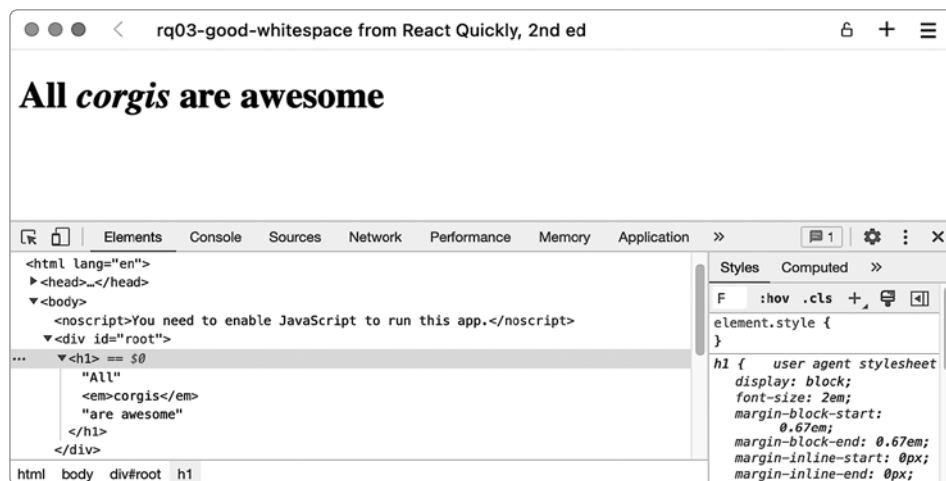


Рис. 3.7. Теперь корги действительно милые

3.5. ВОПРОСЫ

1. Какие из следующих конструкций используются для вывода переменной JavaScript в JSX: `=`, `<%= %>`, `{ }` или `<?= ?>`?
2. Атрибут `class` не разрешен в JSX. *Да* или *нет*?
3. Для атрибутов, значение которых не задано, по умолчанию используется значение `false`. *Да* или *нет*?
4. Встроенный атрибут стиля в JSX представляет собой объект JavaScript, а не строку, в отличие от других атрибутов. *Да* или *нет*?
5. Если вам нужно включить логику `if/else` в JSX, можно использовать ее в фигурных скобках `{ }`. Например, код `JSX class={if (!this.props.isAdmin) return 'hide'}` является верным. *Да* или *нет*?

ОТВЕТЫ

1. Для переменных и выражений используется `{}`.
2. `La.class` — зарезервированная команда JavaScript. По этим причинам в JSX используется `className`.
3. *Нет.* Для атрибута, значение которого не задано явно, по умолчанию используется значение `true`.
4. *Да.* `style` является объектом по соображениям производительности.
5. *Нет.* Во-первых, `class` не является правилами именования атрибутов; следует использовать `className`. Кроме того, вместо `if return` (что в данном контексте все равно неверно) следует использовать тернарный оператор или логические выражения. Можно использовать такую запись: `className={this.props.isAdmin || 'hide'}`.

ИТОГИ

- JSX — всего лишь «синтаксический сахар» для таких методов React, как `React.createElement`.
- Вместо стандартных атрибутов HTML `class` и `for` следует использовать `className` и `htmlFor`.
- Атрибут `style` получает объект JavaScript, а не строку, как с обычными атрибутами HTML.
- Тернарные и логические операторы — лучшие способы реализации команд `if/else`.
- Задачи вывода переменных, написания комментариев и сущностей HTML решаются достаточно легко и просто.
- Некоторые атрибуты HTML и SVG, состоящие из нескольких слов, в React именуются иначе. Обратите внимание на эти атрибуты и не забудьте проверить, правильно ли преобразованы ваши атрибуты в HTML.
- Код JSX необходимо транспилировать в JavaScript перед выполнением в браузере, но заниматься этим придется очень редко. Тем не менее при необходимости для этого можно использовать ряд инструментов, например Babel — самый популярный инструмент на момент написания книги.

4

Функциональные компоненты

В ЭТОЙ ГЛАВЕ

- ✓ Знакомство с функциональными компонентами
- ✓ Сравнение функциональных компонентов с компонентами на базе классов
- ✓ Выбор между двумя типами определений компонентов
- ✓ Преобразование компонента на базе класса в функциональный компонент

Довольно долго в React использовались компоненты на базе классов, но в какой-то момент у простейших компонентов появилась альтернатива. Функциональные компоненты предоставляют более компактный, а в некоторых отношениях и более простой способ записи компонентов React; теперь они обладают таким же набором функциональных возможностей, как и их «родственники» на базе классов.

Термин «*функциональный*» в данном случае обозначает не наличие функциональности (да и кому нужен компонент *без* функциональности?), а определение самого компонента в виде функции JavaScript (вместо класса JavaScript). Изначально функциональные компоненты были менее мощными, чем компоненты

на базе классов, но когда в React 16.9 появились хуки, функциональные компоненты неожиданно стали не менее, а то и более производительными, чем их аналоги на базе классов. Многие разработчики React полностью перешли на функциональные компоненты, так как сейчас команда React рекомендует именно этот вариант.

Компоненты на базе классов все еще полностью поддерживаются в React и, скорее всего, в ближайшее время никуда не исчезнут. Они также очень часто применяются на практике по нескольким причинам:

- Не весь код на базе классов прошел рефакторинг и все еще нуждается в обслуживании.
- Некоторые старые библиотеки документируют взаимодействие только с компонентами на базе классов, поэтому коду приходится использовать эти компоненты, чтобы нормально взаимодействовать с библиотекой.
- Некоторые разработчики React с большим стажем привыкли пользоваться компонентами на базе классов, отлично в них разбираются и поэтому предпочитают работать с ними.
- Переход от компонентов на базе классов на функциональные компоненты привел к существенному изменению модели их жизненного цикла. В некоторых ситуациях применение старого подхода на базе классов может упростить жизненный цикл с повторным рендерингом.
- Небольшое подмножество базовой функциональности React возможно только при использовании компонентов на базе классов (в частности, границы ошибок).

Функциональные компоненты не только продолжают существовать, но и захватят мир — по крайней мере, мир React. Все признаки указывают на то, что функциональные компоненты станут основной технологией программирования на React. Они значительно упрощают жизнь разработчиков и не имеют недостатков (ну почти).

В этой главе вы узнаете, что собой представляют функциональные компоненты, чем они отличаются от компонентов на базе классов (и в чем они похожи), как выбрать подходящую разновидность компонентов для проекта и как преобразовать компонент на базе класса в функциональный компонент.

ПРИМЕЧАНИЕ Исходный код примеров этой главы доступен по адресу <https://rq2e.com/ch04>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке одной командой.

4.1. СОКРАЩЕННАЯ ЗАПИСЬ КОМПОНЕНТОВ REACT

В этом разделе мы расскажем о функциональных компонентах и их новых возможностях. Эти возможности представляют собой «синтаксический сахар», который часто обуславливается способностями современного JavaScript, а не является специфической функциональностью React. Тем не менее мы опишем их в этой главе, потому что все они будут использоваться в дальнейшем. Все они фактически стандартны, очень часто встречаются в коде React и касаются упрощения написания компонентов и взаимодействия с ними:

- Упрощение доступа к свойствам с использованием деструктуризации.
- Упрощение интерфейса компонента за счет использования значений по умолчанию.
- Упрощение интерфейса компонента за счет использования сквозных свойств.

Все вместе эти возможности позволяют писать простые предметные компоненты React, используя компактные определения.

4.1.1. Пример приложения

Создадим небольшое приложение React: меню со списком ссылок из обычной разметки HTML. Мы построим очень простой фрагмент HTML с меню веб-сайта, но это один из основных структурных элементов каждого веб-приложения.

На этом примере мы покажем, что даже при незначительном усложнении компонентов три возможности, перечисленные выше, помогают упростить компоненты как снаружи, так и внутри. Дерево компонентов показано на рис. 4.1.

Вывод приложения в браузере показан на рис. 4.2.

Сначала мы создадим приложение с компонентами на базе классов. Затем создадим те же компоненты с использованием функций. Когда будут готовы обе версии, мы обсудим, какой вариант лучше. Учтите, что этот выбор субъективен — единственно верного решения не существует. У вас есть полное право выбрать тот способ, который вы сочтете более удобным для себя.

Реализация с использованием классов

В приложении используются три компонента: `<App/>`, `<Menu />` внутри `<App/>` и `<MenuItem />` внутри `<Menu />`. Пока поместим весь код в один файл `App.js`, см. листинг 4.1.

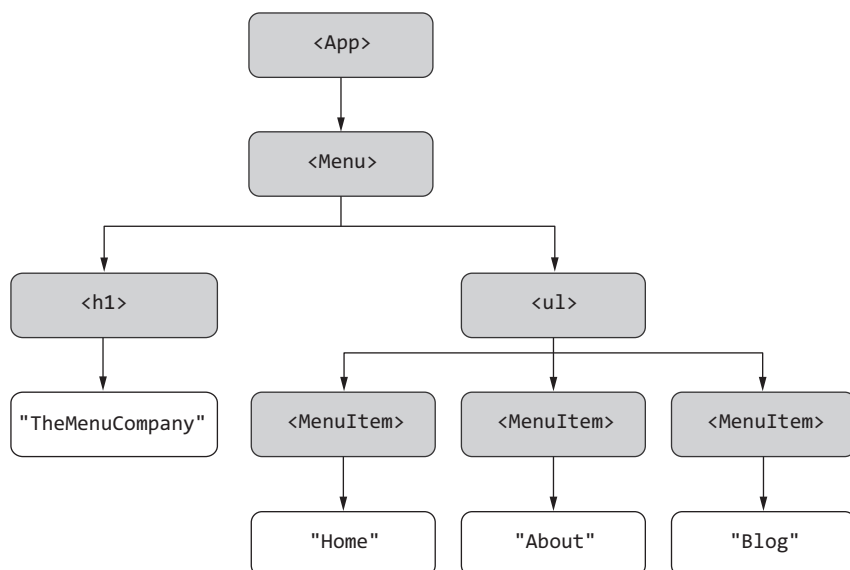


Рис. 4.1. Диаграмма приложения меню представляет общую структуру компонентов от `<App>` наверху до текстовых узлов на нижнем уровне

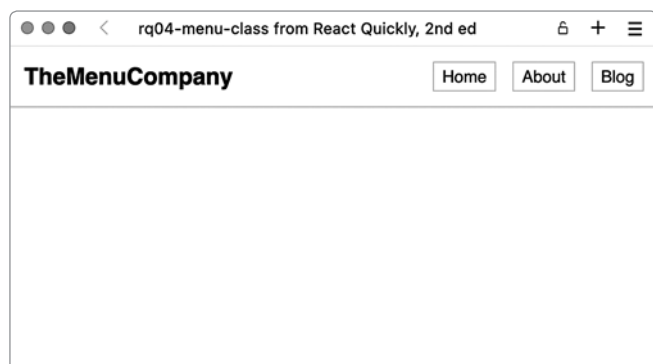


Рис. 4.2. Приложение меню в браузере, состоящее из простой разметки HTML с ограниченным стилевым оформлением

Листинг 4.1. Приложение меню с использованием классов**Определяет новый компонент**

```

import { Component } from "react";
import "../App.css";
class App extends Component {
  render() {
    return (
      <main>
        <Menu />
      </main>
    );
  }
}
class Menu extends Component {
  render() {
    return (
      <nav className="navbar">
        <h1 className="title">TheMenuCompany</h1>
        <ul className="menu">
          <MenuItem label="Home" href="/" />
          <MenuItem label="About" href="/about/" />
          <MenuItem label="Blog" href="/blog" />
        </ul>
      </nav>
    );
  }
}
class MenuItem extends Component {
  render() {
    return (
      <li className="menu-item">
        <a
          className="menu-link"
          href={this.props.href}
        >
          {this.props.label}
        </a>
      </li>
    );
  }
}
export default App;

```

Импортирует файл CSS для стиливого оформления приложения

Создает экземпляр другого нестандартного компонента без передачи свойств

Передаёт свойства нестандартному компоненту

Использует свойства, переданные нестандартному компоненту

Использует стандартный тег HTML

Репозиторий: rq04-menu-class

Этот пример находится в репозитории `rq04-menu-class`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-menu-class --template rq04-menu-class
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-menu-class>

В приложении используются только механизмы, которые вы уже знаете: вложение компонентов, встроенные компоненты HTML и нестандартные компоненты, передача свойств дочерним компонентам и обращение к свойствам, переданным нестандартным компонентам.

Да, в этом приложении также импортируется файл CSS. Эта возможность изначально поддерживается CRA, и она также встречается в шаблоне по умолчанию при создании нового приложения `create-react-app` (CRA), как было показано в главе 2.

Реализация с использованием функций

В этом примере мы с головой окунемся в неизведанное и попробуем разобраться на ходу. Посмотрим, как то же приложение выглядит с использованием функциональных компонентов.

Листинг 4.2. Приложение меню с использованием функций

```
import "./App.css";
function App() {
  return (
    <main>
      <Menu />
    </main>
  );
}
function Menu() {
  return (
    <nav className="navbar">
      <h1 className="title">TheMenuCompany</h1>
      <ul className="menu">
        <MenuItem label="Home" href="/" />
        <MenuItem label="About" href="/about/" />
      </ul>
    </nav>
  );
}
```

Эти два функциональных компонента не получают никаких аргументов

```
    <MenuItem label="Blog" href="/blog" />
  </ul>
</nav>
);
}
function MenuItem(props) {
  return (
    <li className="menu-item">
      <a className="menu-link" href={props.href}>
        {props.label}
      </a>
    </li>
  );
}
export default App;
```

Этот функциональный компонент получает (один) аргумент

Репозиторий: rq04-menu-function

Этот пример находится в репозитории `rq04-menu-function`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-menu-function --template rq04-menu-function
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-menu-function>

Просто невероятно, как все просто. Чтобы создать функциональный компонент, нужно создать функцию, которая возвращает JavaScript XML (JSX), — и все.

Обратиться к свойствам, переданным компоненту, можно через единственный аргумент, переданный функции, — неизменяемый объект свойств. Он работает по тому же принципу, что и `this.props` в компонентах на базе классов.

Подойдет любая функция

Как было показано в аннотациях к листингу 4.2, какие-то функциональные компоненты получают аргумент `props`, а какие-то не получают. Кроме того, мы использовали для определения функции версию в форме команды (то есть `function name() {}`), но это не обязательно. Любое значение, которое может быть выполнено в виде функции, возвращающей JSX, может использоваться как компонент.

Их даже можно определять, встраивая в другие компоненты. Обычно такая практика не приветствуется, но иногда может быть полезной.

```
const App = function() {
  const EmptyMenu = () => { return <nav /> };
  return (
    <main>
      <EmptyMenu />
    </main>
  );
}
```

Функциональное выражение, использующее стрелочную запись

Функциональное выражение, использующее ключевое слово «function»

Здесь один компонент, App, определяется с использованием функционального выражения, а другой компонент, EmptyMenu, определяется прямо в теле функции с использованием стрелочной записи. Пока список пуст (в нем нет ни одного элемента меню), но он показывает, как легко создавать компоненты.

Последнюю функцию можно еще упростить при помощи неявного возврата:

```
const EmptyMenu = () => <nav />;
```

Да, это абсолютно рабочий компонент React. Конечно, он очень прост и делает не много (пока), но это компонент React.

Мы еще вернемся к тому, как принцип «любая функция может быть компонентом» может быть полезен, в дальнейших главах. А пока просто запомните, что компоненты записываются определенным образом по соглашению, а не из-за ограничений фреймворка.

4.1.2. Деструктуризация свойств

В предыдущем примере MenuItem функциональный компонент принимал свойства в объекте props и позднее обращался к ним, например, в форме props.label. Более распространенное решение, применяемое многими разработчиками React, — прямая *деструктуризация* свойств в сигнатуре функции. Деструктуризацией называется компактный механизм извлечения частей составного значения. В JavaScript деструктуризация объекта обычно выполняется в следующей форме:

```
const someObject = { a: 1, b: 2, c: 3 };
const { a, b } = someObject; ← Деструктуризует «someObject» на части «a» и «b»
```

Это выражение присваивает значение someObject.a переменной a. Аналогичным образом значение someObject.b присваивается переменной b. Значение

`someObject`.с игнорируется, так как оно не деструктуризируется в нашем выражении.

Также можно выполнить деструктуризацию при передаче объектных аргументов функции:

```
function log({ message, level }) {
  console.log(level.toUpperCase(), "Message:", message);
}
log({ message: "Unknown product", level: "error" });
```

В результате в консоли выводится следующее сообщение:

```
ERROR Message: Unknown product1
```

В таком простом примере может возникнуть вопрос, почему бы не использовать два разных аргумента:

```
function log(message, level) { ...
```

Но с усложнением функций и добавлением новых аргументов использование одного объекта значительно упрощает вызов функции с переменным количеством аргументов. Это избавляет от необходимости запоминать, что `level` — это пятый аргумент, и т. д.

В функциональных компонентах React свойства всегда передаются в первом (и единственном) аргументе определяющих функций. Метод деструктуризации объекта-аргумента может использоваться для того, чтобы сделать определение компонента еще более понятным, как показано в листинге 4.3. Учтите, что в примере приведена лишь часть файла `App.js`.

Листинг 4.3. MenuItem с деструктуризацией аргументов

```
...
function MenuItem({ href, label }) { ← Деструктуризирует аргумент
  return (                               в определении функции
    <li className="menu-item">
      <a className="menu-link" href={href}>
        {label}
      </a>
    </li>
  );
}
...
```

Позволяет использовать свойства без обращения к объекту props

¹ ОШИБКА: неизвестный продукт.

Репозиторий: rq04-menu-destruct

Этот пример находится в репозитории `rq04-menu-destruct`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-menu-destruct --template rq04-menu-destruct
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-menu-destruct>

Конечно, этот пример полностью идентичен выполнению деструктуризации в отдельной строке внутри определения функции в листинге 4.4.

Листинг 4.4. MenuItem с явной деструктуризацией

```
...
function MenuItem(props) {
  const { href, label } = props;
  return (
    <li className="menu-item">
      <a className="menu-link" href={href}>
        {label}
      </a>
    </li>
  );
}
...
```

Аргумент `props` все еще передается без деструктуризации

Однако его деструктуризация выполняется отдельной командой в первой строке компонента

После этого можно использовать свойства как отдельные переменные, как раньше

В этой книге мы используем подход, показанный в листинге 4.3, с деструктуризацией аргументов непосредственно в определении компонента. Он также часто встречается на практике, так как многие разработчики React применяют это соглашение. Но как уже говорилось, это всего лишь соглашение; можно делать и по-другому.

4.1.3. Значения по умолчанию

У использования деструктурированных свойств есть еще одно преимущество: возможность задавать значения по умолчанию. Предположим, ссылка на блог в меню должна открываться в новом окне (или вкладке) браузера, а другие ссылки должны открываться в том же окне. Для этого можно добавить новое свойство `target`, которое должно быть указано в меню. Здесь тоже приводится только фрагмент `App.js`.

Листинг 4.5. Команды меню со свойством target

```

...
function Menu() {
  return (
    <nav className="navbar">
      <h1 className="title">TheMenuCompany</h1>
      <ul className="menu">
        <MenuItem
          label="Home" href="/" target="_self"
        />
        <MenuItem
          label="About" href="/about/" target="_self"
        />
        <MenuItem
          label="Blog" href="/blog" target="_blank"
        />
      </ul>
    </nav>
  );
}
function MenuItem({ label, href, target }) {
  return (
    <li className="menu-item">
      <a
        className="menu-link"
        href={href}
        target={target}
      >
        {label}
      </a>
    </li>
  );
}
...

```

Добавляет новое свойство в каждый экземпляр компонента элемента меню

Получает новое свойство в результате деструктуризации внутри компонента

Назначает свойство соответствующему элементу JSX как атрибут

Однако можно заметить, что есть смысл установить открывание ссылки в том же окне как поведение по умолчанию и не указывать его в меню. Для реализации этого поведения можно воспользоваться значениями по умолчанию в определении функции. Помните, что данная функциональность не является особенностью React — это обычная функциональность JavaScript.

Листинг 4.6. Элементы меню со свойством target, заданным по умолчанию

```

...
function Menu() {
  return (
    <nav className="navbar">
      <h1 className="title">TheMenuCompany</h1>
      <ul className="menu">
        <MenuItem label="Home" href="/" />
        <MenuItem label="About" href="/about/" />
      </ul>
    </nav>
  );
}
...

```

Указывать свойство target не обязательно, если не нужно переопределять настройку по умолчанию

```

    <MenuItem
      label="Blog" href="/blog" target="_blank"
    />
  </ul>
</nav>
);
}
function MenuItem({ label, href, target = "_self" }) {
  return (
    <li className="menu-item">
      <a
        className="menu-link"
        href={href}
        target={target}
      >
        {label}
      </a>
    </li>
  );
}
...

```

Но переопределить значение по умолчанию легко

Определяет значение по умолчанию с использованием встроенной записи JavaScript для значений по умолчанию при деструктуризации

Репозиторий: rq04-menu-default

Этот пример находится в репозитории `rq04-menu-default`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-menu-default --template rq04-menu-default
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-menu-default>

Упорядочение свойств

Свойства компонента можно задавать в любом порядке. Хотя в JavaScript принято указывать свойства со значениями по умолчанию в конце определения, ничто не мешает делать по-другому.

Например, следующая строка обычно не рекомендуется, но вполне допустима:

```
function MenuItem({ label, target="_self", href }) {
```

А рекомендованный порядок выглядит так:

```
function MenuItem({ label, href, target="_self" }) {
```

Сказанное относится только к порядку свойств, не имеющих значений по умолчанию, и свойств со значениями по умолчанию — оба списка свойств не имеют внутреннего упорядочения свойств, так что вы или ваша команда можете выбрать любые правила на свое усмотрение.

4.1.4. Сквозные свойства

Дополним наш пример. Предположим, что разные элементы должны обладать разными дополнительными свойствами:

- Ссылке **Home** никакие дополнительные свойства не нужны.
- Ссылка **About** должна содержать идентификатор "about-link".
- Ссылка **Blog** должна содержать идентификатор "blog-link".

Реализуем эту задачу, используя механизм, который вы уже успели изучить. Для отсутствующих значений установим значения по умолчанию.

Листинг 4.7. Элементы меню с несколькими значениями по умолчанию

```
...
function Menu() {
  return (
    <nav className="navbar">
      <h1 className="title">TheMenuCompany</h1>
      <ul className="menu">
        <MenuItem label="Home" href="/" />
        <MenuItem label="About" href="/about/" id="about-link" />
        <MenuItem
          label="Blog" href="/blog" target="_blank" id="blog-link"
        />
      </ul>
    </nav>
  );
}
function MenuItem({ label, href, target = "_self", id=null }) {
  return (
    <li className="menu-item">
      <a className="menu-link" href={href} target={target} id={id}>
        {label}
      </a>
    </li>
  );
}
...
```

Запись повторяется. Мы получаем набор аргументов только для того, чтобы передать их одному элементу, причем имя и все остальное остается неизменным.

Синтаксис REST

Можно указать, что некоторые аргументы должны обрабатываться особым образом, а все остальные аргументы должны передаваться целевому элементу. Для этого используется другая современная концепция JavaScript — *синтаксис rest*. При деструктуризации объекта можно использовать синтаксис *rest*, обозначаемый тремя точками. Он определяет объект, которому будут присвоены все оставшиеся свойства, не присвоенные ранее:

```
const someObject = { a: 1, b: 2, c: 3, d: 4 };
const { a, b, ...otherAttrs } = someObject;
```

Два свойства, *c* и *d*, на которые мы еще не ссылались в команде деструктуризации, передаются как свойства в новый объект с именем *otherAttrs*. Следовательно, приведенный фрагмент кода равнозначен следующему:

```
const a = 1;
const b = 2
const otherAttrs = { c: 3, d: 4 };
```

Синтаксис *rest* можно использовать в определении компонентной функции:

```
function MyComponent({ a, b, ...rest }) {
  // a = 1, b = 2, rest = { c: 3 }
}
// Затем:
<MyComponent a="1" b="2" c="3" />
```

Все оставшиеся свойства можно сохранить в объекте, которому обычно присваивается имя *rest*. Теперь остается использовать этот объект и применить все содержащиеся в нем свойства к элементу в выводе.

Вы уже видели, как назначить свойства элементу JSX из объекта, но давайте повторим эту операцию с использованием оператора расширения *spread*:

```
const extraProps = { target: "_blank", id: "link" }
return <a href="/blog/" {...extraProps} />
```

Не забудьте упаковать расширение в фигурные скобки, иначе оно работать не будет.

Применение *rest* на практике

Вернемся к нашему примеру. Требуется сохранить свойства *label* и *href*, переданные компоненту `<MenuItem />`, не обращая внимания на остальные. Если компоненту передаются любые другие свойства, их нужно передать целевому

элементу. Учитывая эти вводные, компонент будет выглядеть, как показано в листинге 4.8.

Листинг 4.8. Элементы меню с операторами rest и spread

```
...
function MenuItem({ label, href, ...rest }) {
  return (
    <li className="menu-item">
      <a className="menu-link" href={href} {...rest}>
        {label}
      </a>
    </li>
  );
}
...
```

В этой строке «...» — синтаксис rest

В этой строке «...» — оператор spread

Репозиторий: rq04-menu-rest

Этот пример находится в репозитории rq04-menu-rest. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-menu-rest --template rq04-menu-rest
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-menu-rest>

Стало намного симпатичнее. Не нужно задавать все лишние свойства, которые нас на самом деле не интересуют. Любой другой компонент может передать то, что ему нужно, за исключением нескольких свойств (в данном случае `label` и `href`), которые требуют особой обработки. Заметим, что `href` даже можно было не указывать как свойство, так как оно будет включено в переменную `rest`, но мы все равно указываем его, потому что это обязательное свойство, которое всегда должны указывать потребители этого компонента. Кстати, это тоже соглашение, а не требование React.

Стоит отметить ряд моментов: как видите, `rest` и `spread` похожи. В обоих случаях перед именем переменной ставятся три точки. Тем не менее используются они по-разному: один — для деструктуризации, другой — для присваивания. Они имеют одинаковую природу (и поэтому одинаково выглядят), но это все же разные операторы.

Использование имени переменной `rest` для дополнительных параметров — распространенное соглашение, но ни в коем случае не обязательное требование. Вы увидите, что этого соглашения придерживаются многие разработчики, но ничто не мешает от него отойти, если это покажется вам обоснованным.

Кроме того, это не специфическая функция React, а всего лишь полезный артефакт языка JavaScript, популярный среди разработчиков React. Мы тоже будем использовать его в следующих главах.

Rest и упорядочение свойств

Синтаксис `rest` должен быть последним элементом деструктуризации объекта, поэтому должен находиться в конце списка свойств. В сочетании со свойствами по умолчанию (что, конечно, возможно) типичный вариант упорядочения выглядит так:

1. Свойства без значений по умолчанию.
2. Свойства со значениями по умолчанию.
3. Rest.

Пример с использованием всех трех видов:

```
function MenuItem({ label, href, target="_self", ...rest }) {
```

4.2. СРАВНЕНИЕ ТИПОВ КОМПОНЕНТОВ

На этом этапе изучения React различия между функциональными компонентами и компонентами на базе классов могут показаться второстепенными и даже незначительными. По сути, все сводится к отличию следующего компонента на базе класса:

```
class Menu extends Component {
  render() {
    return <nav />;
  }
}
```

от следующего функционального компонента:

```
function Menu() {
  return <nav />;
}
```

Когда мы займемся более сложными компонентами, особенно при работе с обратными вызовами и состоянием, мы увидим, что ситуация усложнится, а различий

между функциональными компонентами и их аналогами на базе классов станет больше. При переходе к композиции компонентов и повторному использованию обобщенной функциональности для двух типов компонентов формируются свои паттерны, между которыми нет практически ничего общего.

Выбор типа компонентов определяет характер всей дальнейшей работы в React, но откровенно говоря, особого выбора не осталось. С очень большой долей вероятности вы будете использовать функциональные компоненты, если только особенности конкретного проекта не предполагают обратного. Тем не менее в этом разделе мы рассмотрим как достоинства, так и недостатки функциональных компонентов, а также некоторые факторы, которые на самом деле *не влияют* на выбор.

4.2.1. Преимущества функциональных компонентов

Ниже приведен список основных субъективных преимуществ функциональных компонентов:

- *Компактность* — функциональные компоненты чаще содержат меньше строк кода и меньший объем чистого шаблонного кода, чем их аналоги на базе классов. При реализации функциональных компонентов требуется вводить меньше символов.
- *Удобочитаемость* — отследить происхождение некоторого свойства в компоненте на базе класса через несколько уровней составных высокоуровневых компонентов оказывается намного сложнее, чем сделать то же в функциональном компоненте, использующем хуки (новую возможность React 16.8, которая будет рассмотрена в следующих нескольких главах). Как правило, функциональные компоненты намного проще читать и они намного понятнее даже при быстром изучении.
- *Чистота* — чистоту функции (чистая функция не имеет побочных эффектов и не зависит ни от какой информации, кроме своих аргументов) проще определить, а побочные эффекты нечистых функций проще обнаружить благодаря существованию хуков. Чистоту компонента на базе класса или ее отсутствие обычно определить труднее, что может значительно усложнить отладку и понимание кода.
- *Простота* — функции являются фундаментальной частью любого языка программирования и даже математической теории. Теоретические средства, используемые для описания, работы, композиции и объяснения функций, намного совершеннее своих аналогов, существующих для классов. Классы также являются фундаментальной частью многих языков программирова-

ния, но они остаются абстракциями существенно более высокого уровня, чем простые функции.

- *Тестируемость* — возможность выделения частей функциональности в независимые хуки часто заметно упрощает модульное тестирование функциональных компонентов, так как их можно разбить на меньшие блоки и легко протестировать каждый блок по отдельности.
- *Популярность* — выбор в пользу функциональных компонентов сам по себе оказывается преимуществом. Многие разработчики React предпочитают использовать функциональные компоненты; большинство новых проектов разрабатывается в экосистеме функциональных компонентов; и подавляющее большинство нового контента о React (видеоролики, учебники, книги и т. д.) посвящено исключительно функциональным компонентам.

Заметим, что все эти преимущества относятся к взаимодействию с разработчиками. Выбор типа компонента никак не улучшает и не ухудшает конечный продукт — веб-приложение, доступное для пользователей. Речь идет почти исключительно об упрощении написания, обслуживания и отладки компонентов для пользователей: именно в этих аспектах синтаксис функциональных компонентов по-настоящему раскрывает свой потенциал.

Как правило, функциональные компоненты более элегантны, более компактны, и — что важнее — намного более понятны. Конечно, отчасти это всего лишь субъективное мнение авторов, но его разделяют многие разработчики React, что наглядно подтверждается примерами общедоступного кода на GitHub и в других похожих репозиториях.

4.2.2. Недостатки функциональных компонентов

У функциональных компонентов нет прямых недостатков. Это справедливо для любой возможности, которую можно создать как в функциональном компоненте, так и в компоненте на базе класса.

4.2.3. Незначимые факторы при выборе компонентов

Некоторые факторы, важные для разработчиков, команд и структурных подразделений, в действительности вообще не играют роли при выборе типа компонентов. Например:

- *Скорость* — при выполнении простого компонента в функциональной форме или его реализации на базе класса ощутимых различий в скорости нет. Средства, которые делают каждый компонент (а следовательно, все приложение)

быстрым и динамичным, для двух этих типов компонентов немного разные. Возможно, кто-то скажет, что в функциональных компонентах эти средства более прозрачны и понятны, но их аналоги существуют и для компонентов на базе классов, так что любой компонент может быть быстрым при правильной оптимизации или медленным при плохой оптимизации.

- *Компонуемость* — несмотря на все различия в паттернах проектирования, возможности повторного использования и компонуемость функциональности одинаково высоки и имеют широкую поддержку для обоих типов компонентов.
- *Удобство использования* — для конечного пользователя, посещающего ваше веб-приложение, от выбора того или иного типа ничего не зависит. Опыт взаимодействия пользователя не влияет на это решение.
- *Доступность* — обеспечение доступности компонентов React само по себе является самостоятельным навыком, но оно не зависит от того, в какой форме написан компонент.
- *Надежность* — правильность или надежность кода компонента не зависит от типа компонента. Надежность — следствие грамотной разработки, а не выбора инструментария.
- *Удобство обслуживания* — сейчас еще очень рано говорить о том, что компоненты на базе классов устарели, поэтому можно ожидать, что оба типа компонентов будут полностью поддерживаться React во всех будущих версиях.

Все эти аспекты разработки важны, и все же они напрямую не зависят от выбора типа компонента, скорее они зависят от квалификации и видения разработчика или команды разработчиков.

4.2.4. Выбор типа компонента

Краткий ответ на вопрос: «Какой тип компонентов выбрать для проекта?» довольно прост: *функциональные компоненты*. Развернутый ответ предполагает уточнение: *если нет очень веских причин этого не делать*.

По нашему мнению, подкрепленному богатым опытом, всегда лучше применять новейшую стабильную версию любой технологии, а для React это определенно функциональные компоненты, а не компоненты на базе классов. Функциональные компоненты существуют уже достаточно давно, так что в большинстве новых разработок применяются именно функциональные компоненты и их окружение (особенно хуки), и многие другие разработчики также будут постепенно переходить на функциональные компоненты. Тем не менее иногда стоит подумать

об использовании компонентов на базе классов. Примеры таких ситуаций рассматриваются в следующем разделе.

4.3. КОГДА НЕ СТОИТ ИСПОЛЬЗОВАТЬ ФУНКЦИОНАЛЬНЫЕ КОМПОНЕНТЫ

Как уже говорилось, почти все, что можно сделать в компоненте на базе класса, можно сделать и в функциональном компоненте (кроме границ ошибок). Впрочем, кроме технических, существуют и другие причины, по которым можно отдать предпочтение компонентам на базе классов.

В этом разделе рассматриваются случаи, в которых обоснован сознательный отказ от использования функциональных компонентов:

- Требуется создать границу ошибки, чтобы обрабатывать ошибки, происходящие на нижних уровнях дерева рендеринга.
- Основной код приложения состоит из компонентов на базе классов, и новый код должен хорошо интегрироваться с уже имеющимся.
- Используется библиотека, специализированная для компонентов на базе классов.
- Требуется использовать встроенную функциональность React `getSnapshotBeforeUpdate`.

Все эти ситуации перечислены по убыванию вероятности их возникновения. Если учесть, что первый пункт списка — нестандартная ситуация, которая актуальна только для самых больших и сложных баз данных, скорее всего, вы никогда не столкнетесь ни с одним из этих исключений. Тем не менее мы рассмотрим их в следующих подразделах.

4.3.1. Границы ошибок

Установление границ ошибок становится актуальной задачей для любого кода React, достигшего определенного уровня сложности, поэтому, скорее всего, вы столкнетесь с ней при работе с большой кодовой базой. На момент написания книги эту задачу было невозможно решить без использования компонентов на базе классов. Пока что нет даже планов преобразования функциональности границ ошибок в хуки или что-то похожее, что позволило бы решить ее в терминах функциональных компонентов.

Граница ошибок позволяет определить резервные действия в случае, если дочерний компонент выдает ошибку JavaScript. Конечно, всегда следует стараться, чтобы в приложении не было необработанных ошибок. Но со временем код усложняется, ввод изменяется, API эволюционирует, становится труднее обеспечить покрытие кода тестами и при его выполнении могут возникать ошибки. Граница ошибок гарантирует, что при их возникновении конечный пользователь получит не только ваши извинения, но и как минимум аккуратно отформатированное сообщение об ошибке. Полезно также регистрировать ошибки в каком-нибудь инструменте аналитики.

Для обработки ошибок, происходящих в дочерних компонентах, существуют два метода React API. Первый — `getDerivedStateFromError` — позволяет установить внутренний флаг, указывающий на то, что необходимо изменить рендеринг компонента из-за ошибки, произошедшей в другом месте. Второй — `componentDidCatch` — служит для получения фактической ошибки вместе с трассировкой стека и другой информацией, позволяющей зарегистрировать ошибку для целей отладки. Мы не станем подробно рассказывать о том, как работают эти методы, так как эта тема выходит за рамки книги, но если они вам понадобятся, по обоим методам разработана довольно подробная документация React.

Если потребуется перехватывать ошибки в дереве компонентов, вам придется использовать компонент на базе классов по крайней мере для одного компонента. При этом 99 % компонентов могут оставаться функциональными, невзирая на наличие одной-двух границ ошибок на базе классов.

4.3.2. Код с компонентами на базе классов

Представьте, что вас приняли на должность разработчика в компании, активно работающей со старой кодовой базой React. Это большое приложение — возможно, с сотнями и даже тысячами компонентов — и обширный набор сложных функциональных аспектов.

Вам поручено добавить новую функциональность в небольшую часть этого приложения. Хотя добавление функциональных компонентов к компонентам на базе классов проблем не вызывает, разработчики могут сильно удивиться тому, что одна часть компонентов написана в одном стиле, а другая — в другом.

Рефакторинг всего кода с переходом на функциональные компоненты может быть сложной задачей, тем не менее желательно поставить такую цель в долгосрочной перспективе. Однако в течение переходного периода код будет преобразован лишь частично, и вам потребуется где-то оставить классы, а где-то использовать функциональные компоненты.

По мере взросления React компоненты на базе классов уходят в прошлое и используются все реже. Если вы окажетесь в ситуации, описанной выше, руководствуйтесь здравым смыслом и плывите по течению. Не настаивайте на преобразованиях, пока к ним не будет готова вся команда, и не идите против принятых в ней схем.

4.3.3. Библиотека требует использования компонентов на базе классов

Пожалуй, этот сценарий можно считать гипотетическим, так как мы не смогли найти библиотеку, требующую исключительно компонентов на базе классов, но это не означает, что ее не существует. Вы вполне можете оказаться в ситуации, в которой взаимодействие со сторонней функциональностью потребует использования компонентов на базе классов.

Самый вероятный случай — использование старой библиотеки, которая еще не обновлялась с момента появления хуков React, а в ее примерах и руководствах все еще используются компоненты на базе классов. Это не означает, что такую библиотеку нельзя использовать с хуками; просто вам придется делать это на свой страх и риск и вы не сможете воспользоваться документацией библиотеки, если что-то пойдет не так. Если библиотека требует использования компонентов на базе классов, скорее всего, так происходит потому, что ее документация устарела, но нельзя исключать, что существует библиотека, которая вообще не работает с хуками.

Если вы столкнетесь с любым из описанных сценариев, лучший выход — поискать более современную библиотеку. За четыре года, прошедших с момента появления хуков, многое изменилось — не только запись. Возможно, вы обнаружите, что библиотека, о которой идет речь, отстала во многих отношениях, а то и вовсе уже не обслуживается.

4.3.4. `getSnapshotBeforeUpdate`

В React API существует еще одна встроенная функция, которой нет в мире хуков: `getSnapshotBeforeUpdate`. Это очень специфическая функциональность с крайне узкой областью применения; мы не будем углубляться в подробности. Проблему ее использования легко обойти при помощи хуков, для этого достаточно немного изменить структуру компонентов.

Но если от вас требуют использовать именно эту функциональность, избежать этого не удастся (хотя кто вообще устанавливает такие странные

требования?). Если на самом деле вам требуется только решить проблему, в которой `getSnapshotBeforeUpdate` присутствует в компоненте на базе класса, вы сможете использовать вариант с функциональными компонентами.

Мы упомянули этот метод просто для полноты картины, а не из-за того, что он применяется часто. Быстрый поиск на GitHub обнаруживает только семь репозиторий, в которых упоминается этот конкретный метод, причем три из них — старые и необслуживаемые демоверсии. Таким образом, этот метод обслуживает функциональность, которая с большой вероятностью просто вообще исчезнет из React API, а не будет обновлена до функционального эквивалента.

4.4. ПРЕОБРАЗОВАНИЕ КОМПОНЕНТА НА БАЗЕ КЛАССА В ФУНКЦИОНАЛЬНЫЙ КОМПОНЕНТ

Преобразование простого компонента на базе класса в функциональный компонент уже было показано в листингах 4.1 и 4.2. В этом разделе мы более подробно изучим его, сгладим некоторые шероховатости и подготовимся к дальнейшему путешествию. Мы еще вернемся к этому преобразованию при добавлении более сложной функциональности в компоненты в следующих главах.

Чтобы потренироваться в преобразовании, создадим еще одно веб-приложение: галерею с изображениями и их названиями. Это простое визуальное приложение без интерактивности (мы еще не знаем, как ее добавить), которое демонстрирует особенности внутреннего устройства компонентов, так что мы будем использовать разные приемы для преобразования компонентов. Вывод приложения в браузере показан на рис. 4.3.

Мы создадим четыре версии компонента, в следующей последовательности:

- Версия 1, использующая только метод `render`.
- Версия 2, использующая вспомогательный метод.
- Версия 3, использующая вторичный метод с обращением к классу.
- Версия 4, использующая конструктор для инициализации вычислений.

Мы пройдем через все эти итерации, чтобы показать, как компоненты на базе классов преобразуются в функциональные компоненты, по мере того как классы используют все более сложные паттерны, требующие немного разных решений в функциональном эквиваленте. Наконец, мы обсудим, как с усложнением компонента преобразование «один в один» становится все более сложным, а то и почти невозможным.

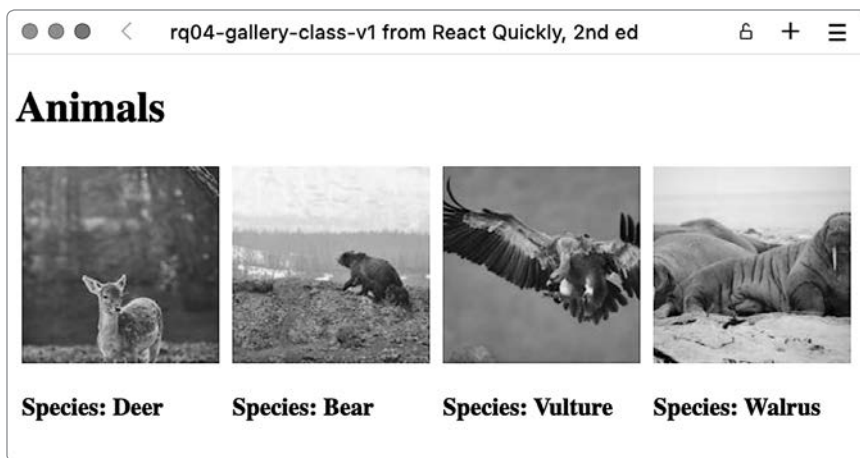


Рис. 4.3. Приложение-галерея в браузере с простыми изображениями и их названиями

4.4.1. Версия 1: только render()

Первая итерация с классами реализуется точно так же, как меню, приведенное ранее в этой главе.

Оригинал

Мы используем три компонента, каждый из которых использует свой метод `render` для возвращения кода JavaScript.

Листинг 4.9. Галерея v1 с использованием классов

```
import { Component } from "react";
class App extends Component {
  render() {
    return (
      <main>
        <h1>Animals</h1>
        <Gallery />
      </main>
    );
  }
}
class Gallery extends Component {
  render() {
    return (
      <section style={{ display: "flex" }}>
        <Image index="1003" title="Deer" />
```

Три класса
компонентов
содержат только
функцию `render`
без других методов

```

    <Image index="1020" title="Bear" />
    <Image index="1024" title="Vulture" />
    <Image index="1084" title="Walrus" />
  </section>
);
}
}
class Image extends Component {
  render() {
    return (
      <figure style={{ margin: "5px" }}>
        <img
          src={`//picsum.photos/id/${this.props.index}/150/150/`}
          alt={this.props.title}
        />
        <figcaption>
          <h3>Species: {this.props.title}</h3>
        </figcaption>
      </figure>
    );
  }
}
export default App;

```

Три класса компонентов
содержат только функцию
render без других методов

Репозиторий: rq04-gallery-class-v1

Этот пример находится в репозитории `rq04-gallery-class-v1`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-gallery-class-v1 --template
  rq04-gallery-class-v1
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-gallery-class-v1>

Преобразование

Преобразуя такой простой компонент в функциональный, мы напрямую преобразуем метод `render` класса в функцию, имя которой совпадает с именем класса. Для класса

```

class MyComponent extends Component {
  render() {
    ...
  }
}

```

результат будет выглядеть так:

```
function MyComponent() {
  ..
}
```

Остается проследить, что прямая деструктуризация `props` выполняется в определении компонента вместо обращения через `this.props`, как раньше. Так мы приходим к результату из листинга 4.10.

Листинг 4.10. Галерея v1 с использованием функций

```
function App() {
  return (
    <main>
      <h1>Animals</h1>
      <Gallery />
    </main>
  );
}
function Gallery() {
  return (
    <section style={{ display: "flex" }}>
      <Image index="1003" title="Deer" />
      <Image index="1020" title="Bear" />
      <Image index="1024" title="Vulture" />
      <Image index="1084" title="Walrus" />
    </section>
  );
}
function Image({ index, title }) {
  return (
    <figure style={{ margin: "5px" }}>
      <img
        src={`//picsum.photos/id/${index}/150/150`}
        alt={title}
      />
      <figcaption>
        <h3>Species: {title}</h3>
      </figcaption>
    </figure>
  );
}
export default App;
```

Определение компонента заменилось функцией

Это определение компонента также получает деструктуризованные свойства

Ссылка на свойство заменяется простой переменной вместо свойства объекта

Пока ничего удивительного. Мы используем только приемы, которые уже разбирали в этой главе:

- Функциональный компонент определяется как функция, возвращающая JSX.

- Если требуется получить свойства, деструктурируем их в определении функции.
- Если требуется обратиться к свойствам, делаем это напрямую с использованием деструктурированных переменных.

Репозиторий: `rq04-gallery-function-v1`

Этот пример находится в репозитории `rq04-gallery-function-v1`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-gallery-function-v1 --template rq04-gallery-function-v1
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-gallery-function-v1>

В следующих подразделах разберем определение компонента изображения, чтобы увидеть разные версии только этого компонента. Для краткости в листингах будет приводиться только компонент изображения.

4.4.2. Версия 2: вспомогательный метод

В этой версии реализации вы узнаете, что делать, если класс изображения содержит еще один метод, который служит вспомогательной функцией при рендеринге. Свойство `src` элемента `<img />` получается слишком длинным и неудобным, и код JSX будет выглядеть намного проще, если рендерить URL вспомогательным методом.

Оригинал

Дополним реализацию галереи на базе классов небольшими улучшениями в коде. В этой итерации предположим, что разработчик приложения хочет визуально разгрузить следующие строки исходного компонента из листинга 4.9.

```
<img  
  src={`//lorempixel.com/200/100/animals/${this.props.index}/`}   
  alt={this.props.title}  
>
```

и привести их к более простому виду:

```
<img
  src={this.getImageSource(this.props.index)}
  alt={this.props.title}
/>
```

Для этого необходимо определить метод класса `getImageSource`, который получает аргумент, индекс и возвращает строку с URL:

```
getImageSource(index) {
  return `//lorempixel.com/200/100/animals/${index}/`;
}
```

После всех изменений компонент изображения приходит к виду, приведенному в листинге 4.11.

Листинг 4.11. Галерея v2 с использованием классов (фрагмент)

```
...
class Image extends Component {
  getImageSource(index) { ← Определяет новый метод в классе
    return `//picsum.photos/id/${index}/150/150/`;
  }
  render() {
    return (
      <figure style={{ margin: "5px" }}>
        <img
          src={this.getImageSource(this.props.index)} ← Вызывает новый метод
          alt={this.props.title}                     с передачей свойства
        />                                             в аргументе
        <figcaption>
          <h3>Species: {this.props.title}</h3>
        </figcaption>
      </figure>
    );
  }
}
...
```

Репозиторий: `rq04-gallery-class-v2`

Этот пример находится в репозитории `rq04-gallery-class-v2`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-gallery-class-v2 --template
  rq04-gallery-class-v2
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-gallery-class-v2>

Отметим, что в этом листинге приведен только компонент изображения. Компоненты приложения и галереи остаются неизменными. Мы не повторяем их ни в листинге 4.11, ни в приведенном ниже преобразовании. Теперь задача сводится к преобразованию нового компонента на базе класса, использующего несколько методов, в функциональный компонент.

Преобразование

Чтобы преобразовать эту функцию, необходимо понимать, что метод класса в объектно-ориентированном смысле является не методом класса, а всего лишь вспомогательной функцией. Собственно, функцию можно вынести полностью за пределы класса и получить тот же результат.

Представьте, что листинг 4.11, приведенный выше, выглядит как листинг 4.12.

Листинг 4.12. Галерея v2 с использованием классов и функции (фрагмент)

```
...
function getImageSource(index) {
  return `//picsum.photos/id/${index}/150/150/`;
}
class Image extends Component {
  render() {
    return (
      <figure style={{ margin: "5px" }}>
        <img
          src={getImageSource(this.props.index)}
          alt={this.props.title}
        />
        <figcaption>
          <h3>Species: {this.props.title}</h3>
        </figcaption>
      </figure>
    );
  }
}
...
```

Метод выделен в отдельную независимую функцию за пределами класса

Вызывает функцию как любую другую функцию, а не как метод класса

Это решение работает точно так же, как предыдущее, потому что метод `getImageSource` не использует никакие данные, которые были бы до-

ступны только внутри класса. Другими словами, функция была чистой, зависела только от ввода, но не от другой внешней информации и не имела внешних связей.

Преобразовать новый компонент на базе класса с использованием вспомогательной функции теперь почти так же просто, как раньше. Вспомогательная функция не меняется, преобразуется только сам компонент.

Листинг 4.13. Галерея v2 с использованием функций (фрагмент)

```
...
function getImageSource(index) {
  return `//picsum.photos/200/100/animals/${index}/`;
}
function Image({ index, title }) {
  return (
    <figure style={{ margin: "5px" }}>
      <img src={getImageSource(index)} alt={title} />
      <figcaption>
        Species: {title}
      </figcaption>
    </figure>
  );
}
...
```

Вспомогательная функция размещается за пределами определения компонента

Вызывается как любая другая функция

Репозиторий: rq04-gallery-function-v2

Этот пример находится в репозитории rq04-gallery-function-v2. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-gallery-function-v2 --template
  rq04-gallery-function-v2
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-gallery-function-v2>

Такое решение намного проще и короче предыдущей итерации из листинга 4.10. Тег `<img />` гораздо легче читать, а детали генерирования фактического URL-адреса были перемещены в отдельную целевую функцию.

Решение основано на том, что метод заведомо является чистым; иначе говоря, он не использует никакую внешнюю информацию, а зависит только от своих

аргументов. А если это условие не выполняется? Рассмотрим эту ситуацию в следующем подразделе.

4.4.3. Версия 3: реальный метод класса

А теперь взглянем на метод класса из предыдущего примера под другим углом. Предположим, разработчик, реализующий компонент, решил использовать тот факт, что метод является частью класса и, как следствие, может напрямую обращаться к свойствам компонента.

Оригинал

В таком случае методу не нужно зависеть от передачи индекса в аргументе. Он может напрямую прочитать индекс из свойств компонента, использующего `this.props`:

```
getImageSource() {  
  return `//lorempixel.com/200/100/animals/${this.props.index}`;  
}
```

Теперь при использовании этого метода передавать аргумент не обязательно; достаточно просто вызвать метод. В результате получаем определение компонента из листинга 4.14.

Листинг 4.14. Галерея v3 с использованием классов (фрагмент)

```
...  
class Image extends Component {  
  getImageSource() {  
    return `//picsum.photos/id/${  
      this.props.index  
    }/150/150/`;   
  }   
  render() {  
    return (  
      <figure style={{ margin: "5px" }}>  
        <img  
          src={this.getImageSource()}   
          alt={this.props.title}  
        />  
        <figcaption>  
          <h3>Species: {this.props.title}</h3>  
        </figcaption>  
      </figure>  
    );  
  }  
}
```

На этот раз метод использует объект `props` напрямую

Теперь метод может вызываться без передачи аргумента

Репозиторий: rq04-gallery-class-v3

Этот пример находится в репозитории `rq04-gallery-class-v3`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-gallery-class-v3 --template rq04-gallery-class-v3
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-gallery-class-v3>

Теперь метод класса действительно является методом класса и зависит от внешней информации. Что делать? Если кратко, в функциональных компонентах у методов класса прямого аналога не существует, тем не менее, того же результата можно добиться другими способами.

Существуют два основных способа преобразования таких компонентов на базе классов в функциональные компоненты, и у каждого из них есть свои плюсы и минусы:

- Преобразование метода в чистую функцию и ее перемещение за пределы компонента.
- Создание локальной функции внутри компонента.

В следующих подразделах мы рассмотрим оба способа и сравним их.

Преобразование с использованием чистой функции

Вариант 1 — запомнить предыдущую версию изображения и попытаться ограничить рост сложности и количества взаимосвязей. В данном случае все просто: цель — удалить все прямые обращения к свойствам компонентов или другой локальной информации компонента и передавать их в аргументах функции. Это приведет нас к `getImageSource`, уже встречавшейся в версии 2, где она получала аргумент и возвращала строку.

Реализация будет выглядеть точно так же, как в листинге 4.13. Однако представьте, что метод более сложный и в нем используется большое количество свойств:

```
getImageSource() {
  const { width, height, index } = this.props;
  return `//picsum.photos/id/${index}/${width}/${height}/`;
}
```

Когда мы используем этот метод при рендеринге компонента на базе класса, все выглядит неплохо:

```
return (  
  ...  
  <img src={this.getImageSource()} alt={this.props.title} />  
  ...  
);
```

Функция довольно компактная, а вся сложность обращения к разным свойствам перемещена в метод.

Если преобразовать ее в чистую функцию, неожиданно приходится передавать множество аргументов, что повышает ее сложность. В функциональном компоненте с чистой функцией пришлось бы передавать все свойства, и это будет выглядеть так:

```
return (  
  ...  
  <img  
    src={getImageSource(width, height, index)}  
    alt={title}  
  />  
  ...  
);
```

Этот вызов уже не такой аккуратный и изолированный, как предыдущий, но преобразование по-прежнему работает.

Преобразование с использованием локальной функции

В варианте 2 метод класса преобразуется в локальную функцию внутри функционального компонента. Результат показан в листинге 4.15.

Листинг 4.15. Галерея v3 с использованием функций (фрагмент)

```
...  
function Image({ index, title }) {  
  const getImageSource = () =>  
    `//picsum.photos/id/${index}/150/150/`;  
  return (  
    <figure style={{ margin: "5px" }}>  
      <img src={getImageSource()} alt={title}/>  
      <figcaption>  
        <h3>Species: {title}</h3>  
      </figcaption>  
    </figure>  
  );  
}  
...
```

Определяет внутри компонента локальную функцию, которая может обращаться к свойствам

Вызывает как любую другую функцию

Репозиторий: rq04-gallery-function-v3

Этот пример находится в репозитории `rq04-gallery-function-v3`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-gallery-function-v3 --template rq04-gallery-function-v3
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-gallery-function-v3>

Так как определение `getImageSource` превратилось в локальную функцию внутри компонента, ему доступны свойства, передаваемые компоненту, и не нужно передавать все свойства вспомогательной функции. Недостаток такого подхода в том, что каждый раз при создании нового компонента мы создаем новую локальную функцию. В примере с четырьмя компонентами это не так важно, но представьте большое, сложное приложение с тысячами и даже миллионами экземпляров некоторых компонентов. При наличии миллионов экземпляров исходного компонента на базе класса, определяемого в листинге 4.14, использовалось бы всего одно определение метода `getImageSource`, которое не занимает много памяти.

Однако при использовании функционального компонента, определенного в листинге 4.15, каждый его экземпляр будет содержать локально определенную функцию и каждая функция будет занимать блок памяти программы. Обычно это не создает особых проблем, но означает, что эти две реализации в чем-то различаются.

При преобразовании компонента на базе класса с дополнительными методами классов можно использовать любой из вариантов, описанных выше. Главное, помнить преимущества и недостатки обоих методов. В нашем примере приемлемы оба варианта, но иногда один вариант оказывается предпочтительнее другого в зависимости от обстоятельств.

4.4.4. Версия 4: конструктор

Как говорилось выше, в компонент на базе класса можно добавить метод-конструктор. Как правило, конструктор используется для инициализации

атрибутов, которые сохраняют одно значение на всем сроке жизни компонента, независимо от переданных свойств. Дело в том, что конструктор выполняется только один раз, при первом создании компонента, а не при каждом обновлении свойств компонента или повторном рендеринге компонента по другим причинам.

Повторный рендеринг компонентов подробнее мы рассмотрим позже. А пока просто запомните, что конструктор вызывается только один раз за время жизни компонента, поэтому в него не следует включать функциональность, зависящую от свойств, которые могут измениться в будущем.

В этом примере в компонент изображения добавляется конструктор, генерирующий случайный идентификатор, который нужно применить к элементу. Например, это может понадобиться, чтобы подключить элемент к внешней библиотеке или сослаться на него с использованием свойств ARIA (Accessible Rich Internet Applications) для обеспечения доступности.

Если бы идентификатор создавался в методе `render`, то он генерировался бы заново при каждом рендеринге компонента. Вместо этого идентификатор создается в конструкторе, чтобы гарантировать его неизменность на всем сроке жизни компонента.

Листинг 4.16. Галерея v4 с использованием классов (фрагмент)

```
...
class Image extends Component {
  constructor(props) {
    super(props);
    this.id =
      `image-${Math.floor(Math.random() * 1000000)}`;
  }
  render() {
    return (
      <figure style={{ margin: "5px" }} id={this.id}>
        <img
          src={`//picsum.photos/id/${this.props.index}/150/150/`}
          alt={this.props.title}
        />
        <figcaption>
          <h3>Species: {this.props.title}</h3>
        </figcaption>
      </figure>
    );
  }
}
...
```

Вызов `super(props)` обязателен в конструкторе компонента на базе класса; в противном случае компонент работать не будет

Создаем переменную класса со сгенерированным уникальным идентификатором

Затем используем идентификатор в методе `render`, извлекая его из класса

Репозиторий: rq04-gallery-class-v4

Этот пример находится в репозитории `rq04-gallery-class-v4`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-gallery-class-v4 --template rq04-gallery-class-v4
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-gallery-class-v4>

Заметим, что идентификатор просто сохраняется в виде свойства непосредственно в экземпляре класса, для чего применяется запись `this.id`. Мы не помещаем его в `this.props` по двум причинам: (1) это невозможно (объект фиксированный) и (2) это не свойство, переданное компоненту, — мы вычислили его сами. Как же привести компонент к функциональной форме, используя приемы, которые вы уже знаете? Никак! Необходимые для этого средства — хуки (прежде всего `useMemo`) — будут описаны в главе 7, а пока придется подождать.

Проблема в том, что, в отличие от компонентов на базе классов, у которых есть конструктор, выполняемый однократно при первом создании компонента, и отдельный метод `render`, который выполняется при каждом рендеринге компонента (включая повторные), функциональный компонент содержит только один метод, выполняемый при каждом рендеринге компонента, включая первый. В функциональном компоненте первый рендеринг ничем не отличается от всех последующих.

Вы еще не видели компонент, который выполняет повторный рендеринг, но пока просто поверьте: почти все компоненты, которые вы будете писать на React, должны рендериться многократно. Если компонент рендерится только один раз, то скорее всего, это очень простой компонент, не имеющий внутренней логики или состояния. Например, логотип веб-приложения можно определить в простом, никогда не изменяющемся компоненте. Жизненный цикл компонента и рендеринг подробно рассматриваются в главе 6.

Чтобы понять, как генерируется идентификатор при использовании функционального компонента, взгляните на листинг 4.17. В этом листинге хук `useMemo` используется для генерирования уникального идентификатора при первом рендеринге компонента, после чего вычисленный результат повторно используется при каждом последующем рендеринге.

Листинг 4.17. Галерея v4 с использованием функций (фрагмент)

```
import { useMemo } from 'react'; ← Импортирует хук
...                               из пакета React
function Image({ index, title }) {
  const id = useMemo(() =>
    `image-${Math.floor(Math.random() * 1000000)}`,
    []);
  return (
    <figure style={{ margin: "5px" }} id={id}>
      <img src={`//picsum.photos/id/${index}/150/150/`} alt={title} />
      <figcaption>
        <h3>Species: {title}</h3>
      </figcaption>
    </figure>
  );
}
```

Применяет хук и добавляет "волшебный" пустой массив, чтобы хук выполнялся только один раз. Этот массив, называемый массивом зависимостей, рассматривается в главе 6

Репозиторий: rq04-gallery-function-v4

Этот пример находится в репозитории rq04-gallery-function-v4. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq04-gallery-function-v4 --template rq04-gallery-function-v4
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq04-gallery-function-v4>

Сейчас мы не будем подробно рассказывать, как работает это решение, но это логический эквивалент инициализации переменной в конструкторе. В главе 7 вы больше узнаете о хуках, рендеринге и мемоизации. Этот метод потребует частичного переписывания компонента, и чтобы его освоить, придется подумать немного нестандартно, но все примеры, приведенные выше, легко преобразуются в функциональные компоненты.

4.4.5. Повысить сложность — значит усложнить преобразование

До сих пор мы рассматривали очень простые примеры. В них не было ни взаимодействий, ни состояния. Такие опции, как фильтрация картинок животных

и вывод подробной информации об изображаемом животном по щелчку, требуют более сложных компонентов React, а те, в свою очередь, требуют более сложной логики для преобразования в функциональный компонент.

Когда мы станем рассматривать более сложные возможности функциональных компонентов React, мы также кратко опишем, как бы они выглядели в компоненте на базе класса. Кроме того, вы узнаете, как преобразовать компонент, в котором используются эти возможности, в эквивалентный функциональный компонент.

А пока просто запомните, что хотя всю функциональность (кроме очень редких исключений, перечисленных в разделе 4.3) можно преобразовать из компонента на базе класса в функциональный компонент, сделать это не всегда просто. Подобно тому как нам приходилось принимать неочевидные решения при преобразовании компонентов с использованием методов класса, на других уровнях сложности будут появляться новые решения, предпочтительные в той или иной ситуации. А если создатель приложения объединит все эти возможности, получится очень сложный компонент, который придется полностью переработать, чтобы он мог работать в функциональном мире.

4.5. ВОПРОСЫ

1. Каковы возможности функциональных компонентов в отношении функциональности React и создаваемых приложений по сравнению с возможностями компонентов на базе классов — *более узкие, более широкие или одинаковые?*
2. Сколько аргументов передается функциональному компоненту?
3. Какое из следующих утверждений *не является* преимуществом функциональных компонентов?
 - a) *Функциональные компоненты компактнее, чем компоненты на базе классов.*
 - b) *Функциональные компоненты быстрее, чем компоненты на базе классов.*
 - c) *Функциональные компоненты понятнее, чем компоненты на базе классов.*
4. Вы начинаете работать над новым приложением. Какой тип компонентов выбрать: функциональные компоненты или компоненты на базе классов — при прочих равных условиях?
5. Преобразовать компонент на базе класса в функциональный компонент всегда просто. *Да или нет?*

ОТВЕТЫ

1. Возможности функциональных компонентов и компонентов на базе классов всегда *одинаковы*. Любое приложение, которое можно построить с использованием одного типа, можно построить и с другим типом.
2. Функциональные компоненты получают *один* аргумент: *функцию* для рендеринга.
3. *Функциональные компоненты не превосходят компоненты на базе типов по скорости*. Хотя в любой наивной реализации могут существовать небольшие различия в скорости, компоненты обоих типов могут работать очень быстро при правильной оптимизации или затормозить все приложение, если они написаны криво. Выбор типа компонента сам по себе не влияет на скорость приложения.
4. При прочих равных условиях в любом новом проекте следует использовать *функциональные компоненты*.
5. *Нет*. Преобразовать простой компонент на базе класса в функциональный компонент обычно труда не составит, но с ростом сложности компонента усложняется и его преобразование.

ИТОГИ

- Функциональные компоненты — это компоненты React, написанные другим способом; альтернатива компонентам на базе классов.
- Любая функция JavaScript, возвращающая JSX, является функциональным компонентом, но по соображениям единообразия функциональные компоненты обычно записывают в определенном стиле.
- Для упрощения определения функциональных компонентов часто используются некоторые приемы JavaScript, включая деструктуризацию, значения по умолчанию, синтаксис *rest* и оператор *spread*.
- Функциональные компоненты как минимум не уступают компонентам на базе классов по всем показателям.
- В некоторых отношениях функциональные компоненты удобнее для разработчиков, но эти преимущества не влияют на конечный продукт, не зависящий от типа компонента.
- Если у вас есть выбор, в React рекомендуется использовать функциональные компоненты.
- В общем случае компоненты на базе классов можно преобразовать в функциональные компоненты, но такое преобразование может потребовать значительных усилий и рефакторинга существующей функциональности.

5

Состояния и их роль в интерактивной природе React

В ЭТОЙ ГЛАВЕ

- ✓ Роль состояния компонентов
- ✓ Использование состояния в функциональных компонентах
- ✓ Преобразование компонентов на базе классов, обладающих состоянием, в функциональные компоненты

Все компоненты, которые мы до сих пор создавали, получали свойства и рендерили разметку HTML на основании этих свойств. Например, можно передать кнопке свойство `label`, чтобы кнопка отображалась с заданным текстом. Однако текст кнопки не будет изменяться при наступлении какого-то события: например, **Вкл** не перейдет в положение **Выкл** при переключении. Дело в том, что в приложении пока не реализована возможность реагировать на происходящие события и сохранять информацию о динамических изменениях.

Вывод компонентов, которые мы до сих пор создавали, зависит только от их свойств. Другими словами, компоненты являются «чистыми» в терминологии функционального программирования. Они не имеют других входных данных

и побочных эффектов. Если вы передадите одному компоненту одни и те же свойства, вы всегда получите один и тот же результат и никакого другого.

Все это хорошо, и именно это нам нужно, — но это немного скучно. Такие компоненты отлично подходят для представления данных, но они бесполезны, если вам понадобится создать интерактивное приложение. Если вы хотите, чтобы в приложении что-то обновлялось нажатием кнопки или при заполнении текстового поля, эту информацию необходимо где-то сохранить и передать ее другому компоненту, чтобы тот мог отреагировать. Представьте форму для ввода учетных данных. Когда пользователь вводит адрес электронной почты и пароль, эту информацию необходимо где-то сохранить, чтобы вывести сообщение об ошибке в случае неверного ввода. Когда пользователь нажимает кнопку **Send**, данные передаются на удаленный сервер.

Компоненты, которые зависят только от своих свойств и не имеют другой внутренней логики, также называются *компонентами без состояния*. Альтернативой является компонент *с состоянием*. В этом контексте под *состоянием* понимается способность изменяться со временем посредством использования внутренних переменных. Один компонент может в один момент времени иметь одно внутреннее состояние, которое приводит к одному выводу JavaScript XML (JSX), а в другой — перейти в другое состояние, которое приводит к другому выводу. Представьте кнопку, которая может иметь активное состояние и неактивное. Информация о том, активна кнопка или нет, является состоянием кнопки, а компонент, содержащий состояние, называется компонентом *с состоянием*.

В этой главе вы узнаете, что собой представляет приложение с состоянием и что делает компонент с состоянием в таком приложении. Для большей наглядности мы продемонстрируем, как задать, обновить и использовать состояние в функциональном компоненте. Несмотря на то что API очень прост и состоит из единственной функции `useState`, описание будет довольно длинным.

В конце главы мы кратко обсудим, как работает присваивание, обновление и использование состояния в компонентах на базе классов. Эти операции выполняются похожим, но все же иным способом, и есть ряд важных моментов, которые необходимо учитывать.

При обсуждении компонентов на базе классов вы также узнаете, как происходит преобразование компонентов с состоянием на базе классов в функциональные компоненты с состоянием. Это может быть полезно, если вы работаете со старым кодом, все еще использующим компоненты на базе классов, и вам нужно обновить их до функций, чтобы пользоваться новейшими технологиями. Эта информация также пригодится, если вы найдете в интернете примеры и учебники, которые

учат делать что-то полезное с компонентом на базе класса. Существуют тысячи старых, но все еще полезных учебников, но чтобы применить приведенные в них рекомендации в современном коде, требуется провести кое-какие преобразования.

ПРИМЕЧАНИЕ Исходный код примеров этой главы доступен по адресу <https://rq2e.com/ch05>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке одной командой.

5.1. ПОЧЕМУ В ПРИЛОЖЕНИЯХ REACT ВАЖНО СОСТОЯНИЕ?

Состояние играет важную роль при создании любых интерактивных приложений. Если приложение не имеет состояния, это означает, что оно абсолютно статично, — такое приложение вообще не может изменяться после открытия в браузере. Этот режим может подойти для сообщения в блоге или рецепта, но если вы хотите, чтобы пользователь мог вводить свои регистрационные данные, обновлять изображение, щелкать на элементах или еще как-то взаимодействовать с приложением, чтобы влиять на выводимую информацию, приложение должно обладать состоянием.

Компоненты React обладают *индивидуальным состоянием*. Хранение состояния в компонентах обеспечивает существование состояния у приложения React в целом.

Заметим, что хотя почти все *приложения* React обладают состоянием, не все *компоненты* обладают состоянием. Приложение может содержать лишь несколько компонентов с состоянием, но эти компоненты управляют состоянием всего приложения и обновляют все компоненты без состояния при необходимости. Хотя привести общие оценки очень трудно, можно предположить, что не более трети компонентов приложения будет обладать состоянием, а по мере роста и усложнения приложения их доля, скорее всего, будет убывать. Представьте дерево компонентов условного приложения, изображенное на рис. 5.1. На диаграмме темные компоненты обладают состоянием, тогда как у светлых компонентов состояния нет.

React не предоставляет инструментов, предназначенных для присвоения состояния приложению в целом. Приложение React определяется как сумма его компонентов. Чтобы приложение обладало состоянием, им должны обладать какие-то из его компонентов.

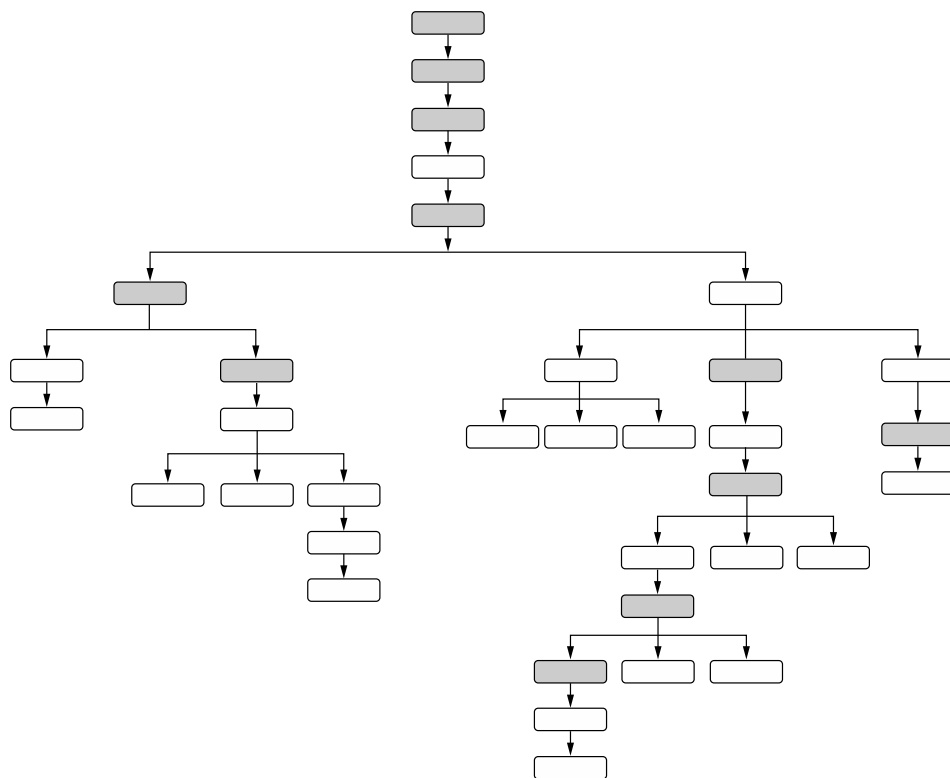


Рис. 5.1. У темных компонентов есть состояние; светлые компоненты состояния не имеют. Обратите внимание: компоненты с состоянием часто располагаются в верхней части дерева, тогда как компоненты без состояния чаще встречаются ближе к листовым узлам

5.1.1. Состояние компонента React

Тип компонента определяется наличием у него состояния:

- *Компонент с состоянием* — компонент с состоянием не зависит от контекста и может обновляться на основании внутренних триггеров.
- *Компонент без состояния* — компонент без состояния может изменяться или обновляться только тогда, когда родительский компонент предоставит ему новые свойства.

Состояние компонента React — механизм, позволяющий сохранять в компоненте значения, которые могут изменяться со временем. Представьте различия между

компонентом часов, который может выводить текущее время на основании переданного ему свойства, и компонентом часов, который может обновлять себя каждую секунду и непрерывно выводить текущее время. Во втором случае компонент должен иметь возможность хранить текущее время суток (а также средства для изменения этого значения).

Различия между этими двумя подходами представлены на рис. 5.2.

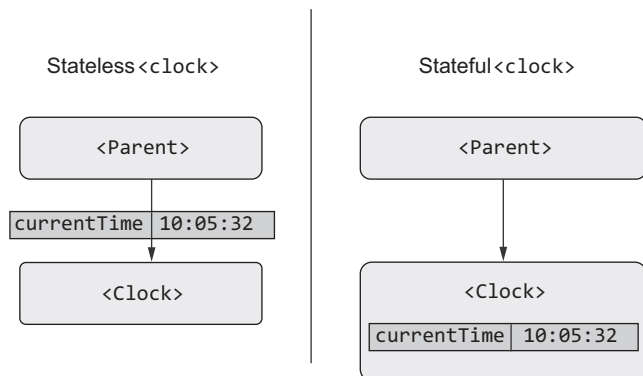


Рис. 5.2. Чтобы часы без состояния могли показывать текущее время, их должен каждую секунду обновлять родительский компонент, тогда как часы с состоянием могут обновлять себя и родителю не приходится этим заниматься

Обратите внимание: чтобы часы без состояния на рис. 5.2 действительно работали, родитель должен обладать состоянием, потому что текущее время все равно должно где-то храниться. Конечно, родительский компонент тоже может не иметь состояния, но тогда состояние придется вынести на более высокий уровень дерева.

5.1.2. Где хранить состояние?

Итак, приложение должно обладать состоянием. Где же его разместить? Его принято хранить как можно ближе к компонентам, которые в нем нуждаются.

Представим приложение, которое содержит меню верхнего уровня (header) с действующими и активными часами (clock), основным разделом с несколькими страницами, которые можно обновлять в процессе навигации по странице, и футер (footer) со статическими ссылками. Состояние часов должно где-то храниться — либо внутри самого компонента часов, либо в другом компоненте на более высоком уровне. Если определить структуру

приложения так, как показано на рис. 5.3, появляется выбор места для хранения состояния часов.

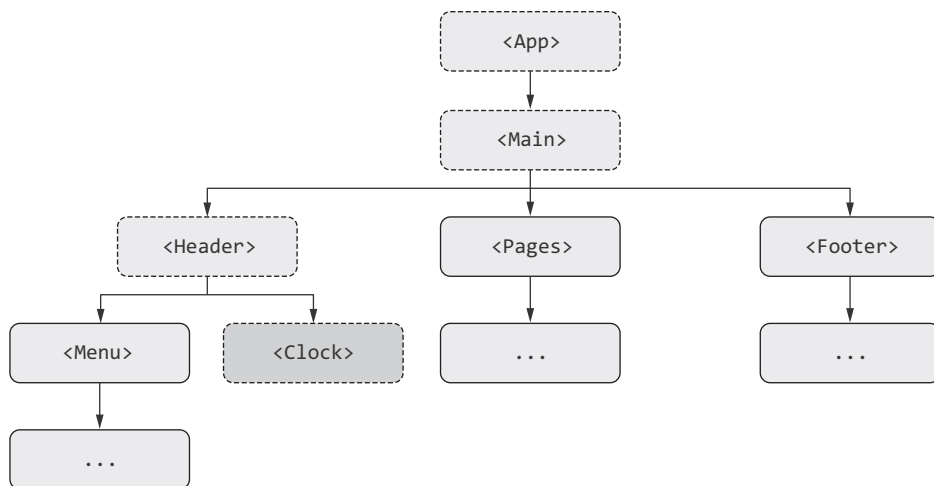


Рис. 5.3. Состояние часов необходимо только для компонента, выделенного темно-серым цветом. Состояние часов может храниться в любом компоненте в пунктирной рамке (это компонент часов и все его предки)

В этом примере состояние часов логично хранить в самом компоненте часов. Никакому другому компоненту не нужно знать текущее время, так что состояние можно локализовать в том компоненте, которому оно необходимо.

С другой стороны, предположим, также необходимо хранить состояние страницы, которая отображается в приложении. Эта информация необходима как в компоненте `Pages`, который должен отображать активную страницу, так и в компоненте `Menu`, который должен отображать ссылку на текущую страницу на выделенном фоне.

Анализ дерева документа на рис. 5.4 показывает, что информацию можно хранить либо в `Main`, либо в `App`, так как только эти два компонента содержат оба компонента, которым требуется интересующее нас состояние.

Где же разместить состояние текущей страницы — в `Main` или в `App`? Решайте сами, так как это, скорее всего, дело вкуса. Хотя существует практическая рекомендация хранить состояние как можно «ниже» в дереве документа (то есть в `Main`), у этого компонента может быть множество других обязанностей. Тогда разумно поместить информацию в родительский компонент `App` для оптимальной организации.

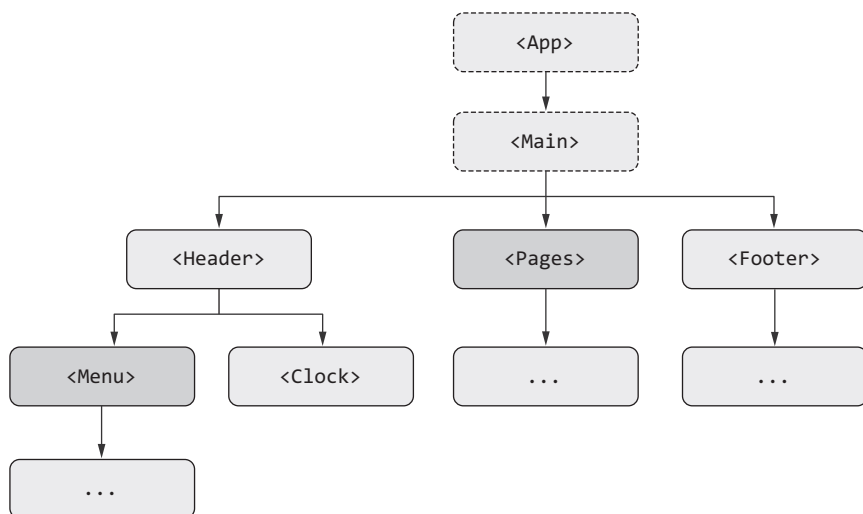


Рис. 5.4. Состояние текущей страницы необходимо только для двух компонентов, выделенных темно-серым цветом. Состояние часов может храниться в двух компонентах в пунктирной рамке — их общих предках

5.1.3. Что можно хранить в состоянии компонента

В общем случае любое состояние, используемое в веб-приложении, относится к одной из трех категорий:

- Данные приложения.
- Состояние UI.
- Данные форм.

Эта классификация не является специфическим правилом или артефактом React, а следует из принципов работы приложений с состоянием.

Разные типы данных хранятся по-разному. Мы рассмотрим каждый тип и поговорим о том, как правильно хранить и использовать данные. Могут существовать и другие категории состояния компонента, но большинство из них попадает в одну из трех указанных.

Данные приложения

К категории данных приложения относятся данные, с которыми пользователь работает, которые он читает или обновляет. Если вы создаете веб-приложение, для входа в которое пользователь должен вводить свои учетные данные, информация

о пользователе относится к данным приложения. Если пользователь может войти и просмотреть список тренировок в фитнес-центре, записаться на занятие и т. д., все эти данные также относятся к данным приложения.

Данные приложения чаще всего хранятся на его глобальном уровне. Если существует компонент, в котором выводится список тренировок в фитнес-центре, то список доступных занятий можно сохранить локально в компоненте, но это также будет означать, что вся информация о доступных тренировках будет потеряна при демонтировании компонента и ее придется заново загружать с сервера при повторном монтировании. Эти два термина, *монтирование* и *демонтирование*, более подробно рассматриваются в следующей главе.

Лучшим решением часто оказывается создание в компоненте долгосрочного хранилища данных, чтобы после однократной загрузки данные оставались в приложении. Позже мы рассмотрим разные возможные решения, основанные на использовании встроенной функциональности React (а именно контекстов React).

Состояние UI

Под состоянием UI понимается текущее состояние компонентов UI (пользовательского интерфейса): какая вкладка активна в настоящий момент, открыто меню или нет, и т. д. В общем случае это временные данные, которые не предназначены для долгосрочного хранения, а только помогают веб-приложению правильно рендерить нужные элементы.

Значения состояния UI чаще всего хранятся с максимально возможной локализацией. Информация о том, открыто меню или нет, актуальна только внутри компонента меню, так что ее можно хранить как локальное состояние только внутри этого компонента.

Данные форм

Как будет показано в главе 9, данные форм — это еще один очень частый способ применения состояния компонентов. Пока пользователь взаимодействует с формой: вводит данные, переходит от одного поля формы к другому, — текущее состояние формы часто хранится в локальном состоянии компонента, которое включает все поля формы.

5.1.4. Что не стоит хранить в состоянии

Некоторые данные лучше никогда не хранить в состоянии, например такие:

- *Значения, которые не изменяются* — в эту категорию входят не только константы («магические числа»), но и данные конфигурации, загружаемые

при запуске приложения. Если значение не изменяется, не делайте его переменной.

- *Копии других значений состояния* — всегда старайтесь поддерживать единый авторитетный источник данных. Если в глобальном состоянии приложения есть какие-то данные, их одновременное хранение в локальном состоянии какого-то компонента только вызовет путаницу (если только вы не позволите пользователю локально обновить данные в форме).
- *Дубликаты данных* — если в состоянии хранятся две версии одних и тех же данных, возможно, эти данные стоит консолидировать. Например, если в состоянии хранятся имя, фамилия и сочетание имени и фамилии, то при изменении любого из этих атрибутов придется обновлять как минимум два из этих значений. Намного лучше хранить в состоянии только основные значения, имя и фамилию и при необходимости вычислять полное имя на основании данных состояния.

Конечно, существует много других данных, которые никогда не стоит включать в состояние, но полный список будет слишком длинным. Приведенный список содержит основные ловушки, которые у вас наверняка возникнет соблазн применить (но хочется верить, что вы с ним справитесь).

5.2. ДОБАВЛЕНИЕ СОСТОЯНИЯ В ФУНКЦИОНАЛЬНЫЙ КОМПОНЕНТ

До сих пор мы обсуждали, почему, где и что хранить в состоянии компонента, но все еще не разобрались, как же все это делать. Хранение состояния в функциональном компоненте имеет невероятно простой API, который иногда становится источником радости, а иногда — дикой головной боли. Так как этот API работает на очень низком уровне, возможно, вам придется добавить ему функциональности для большего удобства; однако с его помощью реализовать компоненты с состоянием в простейших сценариях очень легко. *Правда.*

Предлагаю не медлить и понаблюдать за этим API в действии. Для этого создадим простейший возможный компонент с состоянием — счетчик нажатий на кнопку. Для этого необходимо инициализировать счетчик, вывести текущее значение и увеличивать счетчик с каждым нажатием. Остался очень важный последний шаг. Нельзя просто обновить любую текущую переменную и надеяться, что компонент правильно отрендерится. Необходимо сообщить React, что значение обновилось, а это значит, что необходимо обратиться к React-специфичному API. Простая блок-схема процесса изображена на рис. 5.5.



Рис. 5.5. Блок-схема логики компонента-счетчика. Переменная инициализируется и выводится, а нажатием кнопки значение увеличивается и React передается информация об обновлении компонента, чтобы вывести новое значение в компоненте

Чтобы сделать все это в функциональном компоненте, необходимо использовать из пакета React функцию с именем `useState`. Функция получает исходное значение и возвращает текущее состояние и функцию обновления. Добавим необходимые составляющие React-специфичного API, как показано на рис. 5.6.

Полный код приведен в листинге 5.1. Значения, передаваемые и возвращаемые хуком `useState`, будут рассмотрены позже, так что пока не обращайте особого внимания на функцию `setCounter`. В свое время мы расскажем о ней.

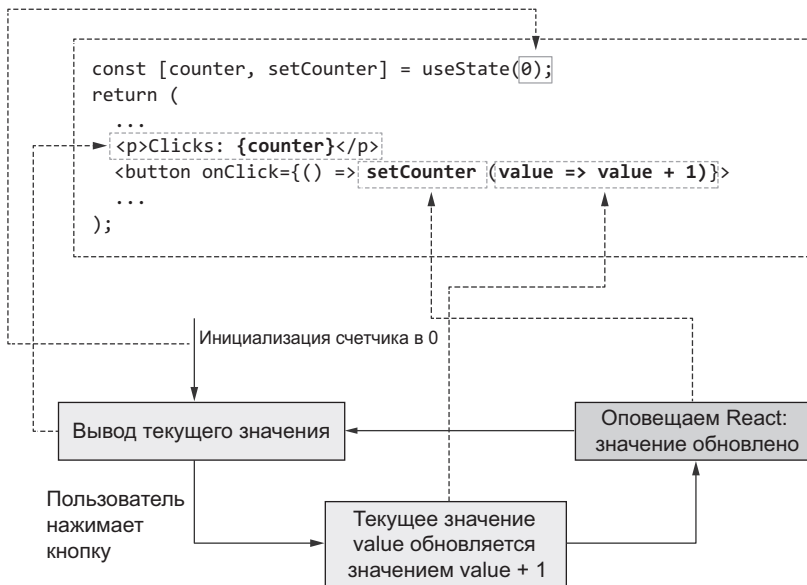


Рис. 5.6. Блок-схема работы с состоянием и строк кода, относящихся к каждому действию на диаграмме. Пунктирные линии со стрелками соотносят концепцию с конкретным блоком кода, реализующим поставленную цель

Листинг 5.1. Полностью работоспособный счетчик

```
import { useState } from "react";
function Counter() {
  const [counter, setCounter] = useState(0);
  return (
    <main>
      <p>Clicks: {counter}</p>
      <button
        onClick={() => setCounter((value) => value + 1)}
      >
        Increment
      </button>
    </main>
  );
}
function App() {
  return <Counter />;
}
export default App;
```

Импортирует функцию `useState` из пакета `React`

Инициализирует новое состояние и получает текущее значение с сеттером

Выводит значение через текущее состояние

Обновляет значение через сеттер

Попробуем выполнить код в браузере и понажимать кнопку, как показано на рис. 5.7.

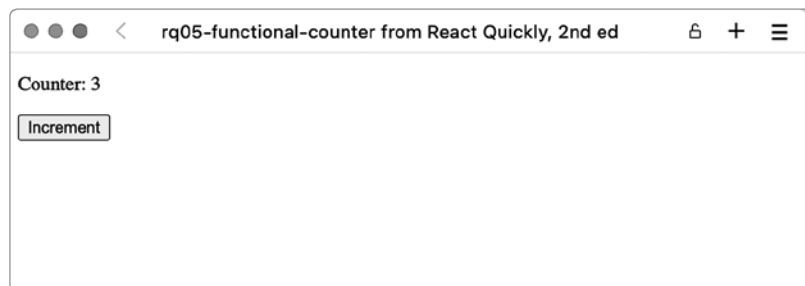


Рис. 5.7. Счетчик после трех нажатий. Впрочем, их можно продолжить, ограничений нет (вернее, пороговое значение равно 9 007 199 254 740 991, но вряд ли вы до него доберетесь)

Репозиторий: rq05-functional-counter

Этот пример находится в репозитории `rq05-functional-counter`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq05-functional-counter --template rq05-functional-counter
```


На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq05-functional-counter>

Нам предстоит изучить довольно большой объем материала, поэтому будем разбирать его по этапам:

- Импорт функции `useState` из пакета `React`.
- Вызов `useState` в функциональном компоненте с передачей исходного значения.
- Деструктуризация ответа вызова `useState` в виде двух элементов массива:
 - В первом элементе передается текущее значение.
 - Во втором элементе передается сеттер.
- Использование текущего значения по усмотрению.
- При необходимости обновить состояние — вызов сеттера с функцией или простым значением.

Мы рассмотрим эти этапы в следующих подразделах. Вы также узнаете, как использовать несколько экземпляров `useState` для создания более сложных компонентов.

Да, кстати, `useState` — это хук. Это первый и самый простой из хуков, которые появились в `React 16.8` и изменили все. *Хуки* (hooks) — специальные функции, с которыми нельзя работать как с любой другой функцией. Некоторые из них мы рассмотрим в этом разделе, но подробно теме хуков будет посвящена глава 6.

5.2.1. Импорт и использование хука

`useState` является хуком. *Хук* — обобщенный термин для обозначения специальных функций, существующих в `React 16.8` и выше. `React` поставляется с набором встроенных хуков, которые считаются хуками, потому что так говорит `React`. Они не объединены общим предназначением и не предоставляют пересекающуюся функциональность, а используются для «подключения» к базовой функциональности `React`, и чтобы они работали правильно, потребуется быть очень внимательными.

То, что `useState` является хуком, на самом деле очень легко понять, потому что функция начинается с `use*`. В современном `React` принято соглашение, по которому любая функция, начинающаяся с `use*`, является хуком, а не-хуки никогда не должны начинаться с этого слова.

Что же такого особенного в хуках? Хуки прокладывают путь *от* компонентов к внутренним механизмам React. С ними можно сделать много того, что невозможно без дополнительного уровня доступа. Функциональный компонент — всего лишь функция, так что без более глубокого доступа ничего серьезнее, чем управление одним рендерингом, сделать не удастся.

React включает 15 хуков (в версии React 18). Они представляют собой низкоуровневые блоки, которые могут объединяться для создания продвинутых компонентов. Возможно, со временем в React API добавятся новые встроенные хуки, так что к тому моменту, когда вы будете читать книгу, их число может превысить 15.

На базе хуков React можно создавать собственные хуки. В таком случае им также следует присваивать имена, начинающиеся с `use*`. Например, для приведенного выше компонента можно создать хук `useCounter`. Нестандартные хуки рассматриваются в главе 10.

Правила хуков

В компоненте необходимо всегда использовать один и тот же хук. Более того, при каждом рендеринге компонента необходимо использовать одни и те же хуки в одинаковом порядке. На первый взгляд это странно, но это требуется для правильной работы функции в React.

Под «всегда использовать» мы имеем в виду, что один и тот же хук должен всегда вызываться при каждом рендеринге компонента, то есть при каждом выполнении функции, определяющей компонент. Это означает, что хук не может, например, выполняться условно в блоке `if` или располагаться после необязательной команды `return`.

Представьте разновидность компонента счетчика, в которой вы передаете свойство компоненту для обозначения того, должен ли он быть видимым. Это можно сделать так, как показано на рис. 5.8.

Реализация в коде следующая:

```
function Counter({ isVisible }) {
  if (!isVisible) {
    return null;
  }
  const [counter, setCounter] = useState(0);
  return (
    ...
  );
}
```

Если значение `isVisible` ложно, сразу возвращается `null`

Только если значение `isVisible` отлично от `false`, состояние инициализируется с использованием хука `useState`. Это ошибка!

Нет, так нельзя! В чем подвох? Функция-хук `useState` вызывается не каждый раз, а только в некоторых случаях. Если свойству `isVisible` присваивается `true`

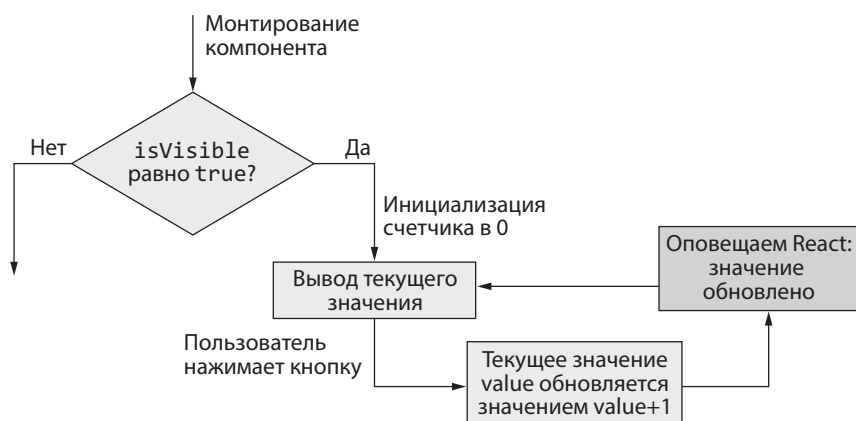


Рис. 5.8. Можно ли сначала проверить свойство и, если оно ложно, просто проигнорировать инициализацию состояния?

при одном рендеринге, то хук вызывается, но если при следующем рендеринге оно окажется равным `false`, то хук не будет вызван. И это не просто мелкий дефект — он полностью нарушит работоспособность приложения. React выдает сообщение об ошибке следующего вида:

React Hook "useState" is called conditionally. React Hooks must be called in the exact same order in every component render. Did you accidentally call a React Hook after an early return?¹

По этой причине иногда приходится писать на первый взгляд неоптимальный код. Все хуки нужно разместить до любых попыток вернуть что-либо из компонента, как показано на рис. 5.9.

Реализация будет следующей:

```

function Counter({ isVisible }) {
  const [counter, setCounter] = useState(0);
  if (!isVisible) {
    return null;
  }
  return (
    ...
  );
}
  
```

← Инициализирует две переменные, которые могут никогда не понадобиться

← Что-то возвращается только после выполнения всех хуков

¹ Хук React "useState" вызывается условно. Хуки React должны вызываться в одинаковом порядке при каждом рендеринге компонента. Возможно, вызов хука React осуществлен случайно после быстрого возврата?

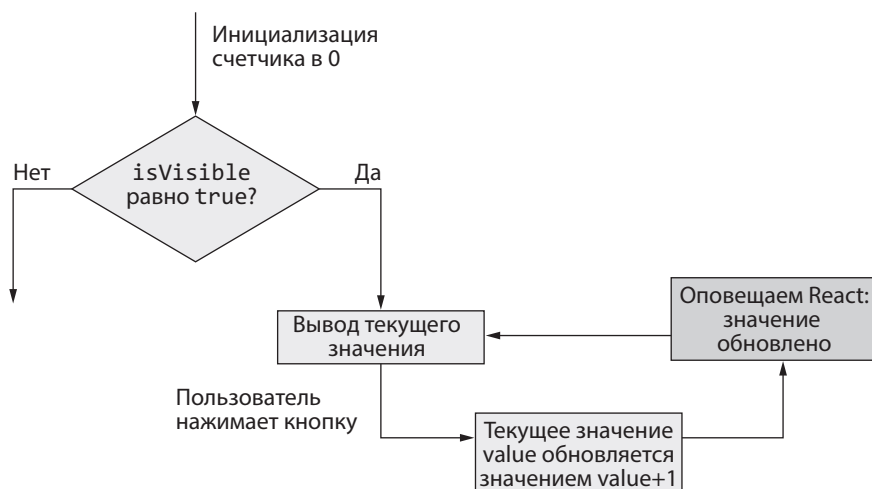


Рис. 5.9. Необходимо инициализировать состояние до возможной отмены рендеринга, даже если оно вообще не используется

Это также означает, что хуки никогда не должны выполняться условно (например, внутри блока `if`) или в цикле (потому что это означало бы, что количество вызовов хуков может изменяться) и никогда не должны вызываться в обратном вызове или в обработчике событий (при вызове они должны выполняться напрямую в теле компонента). Некоторые примеры таких ограничений приводятся в следующих разделах; там же вы узнаете, как обойти эти ограничения, чтобы достичь желаемой цели.

Вы узнаете о хуках подробнее в следующей главе, когда мы расскажем об их внутреннем устройстве и о том, как их правильно использовать.

5.2.2. Инициализация состояния

При вызове `useState` необходимо передать исходное значение; иначе будет передано `undefined`. Важно только значение, передаваемое `useState` при первом вызове для каждого экземпляра компонента. При повторном рендеринге, независимо от его причины, исходное значение игнорируется.

Самая очевидная цель передачи исходного значения — определение эталонного значения в компоненте. Каким должно быть состояние компонента при первом монтировании? Если это должно быть динамическое значение, передаваемое в виде свойства, используйте это свойство. Если это должно быть любое статическое значение, передайте его. В 99 % случаев в качестве исходного значения

будет использоваться либо статическое значение (очень часто `null`), либо свойство. Примеры инициализации будут рассмотрены в оставшейся части раздела.

Исходное значение

У каждого состояния существует исходное значение. В нашем примере счетчику присваивается исходное значение `0`, но конечно, оно может быть и другим. С таким же успехом можно инициализировать его значением `10`, `100` или даже динамическим значением.

Предположим, требуется создать модификацию счетчика, в которой значение может инициализироваться некоторым передаваемым свойством. Создадим приложение с тремя разными экземплярами этого счетчика, инициализированными разными начальными значениями. Полученное дерево компонентов изображено на рис. 5.10.

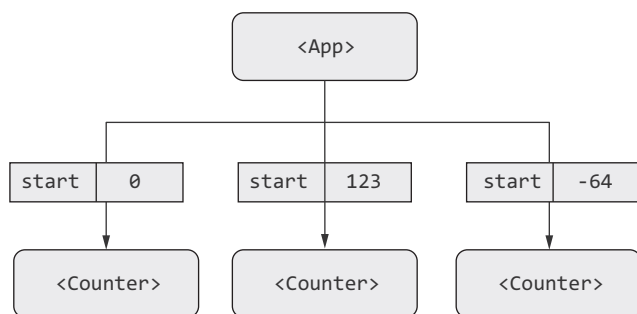


Рис. 5.10. Теперь приложение создает три счетчика, инициализированных разными значениями — просто для красоты

Возможная реализация представлена в листинге 5.2. На рис. 5.11 показано, как выглядит результат в браузере.

Листинг 5.2. Три счетчика

```

import { useState } from "react";
function Counter({ start }) {
  const [counter, setCounter] = useState(start);
  return (
    <main>
      <p>Counter: {counter}</p>
      <button onClick={() => setCounter(value => value + 1)}>
        Increment
      </button>
    </main>
  );
}
function App() {

```

Компоненту передается свойство с именем `start`

Это свойство используется для инициализации состояния

```

return (
  <>
    <Counter start={0} />
    <Counter start={123} />
    <Counter start={-64} />
  </>
);
}
export default App;

```

Три экземпляра счетчика
с тремя разными
исходными значениями

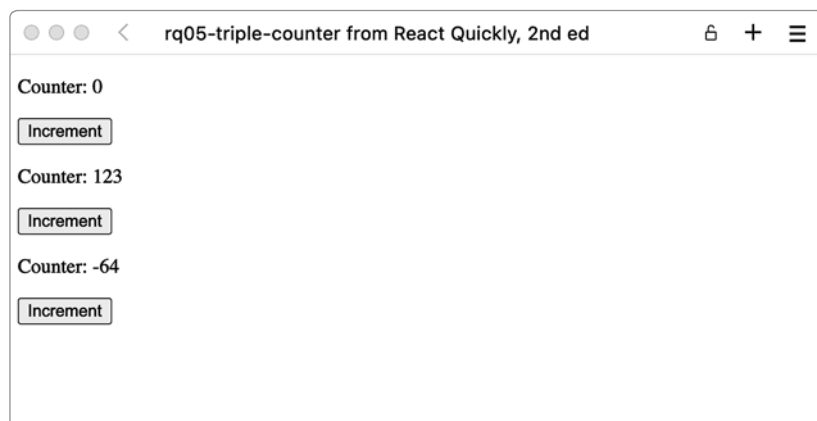


Рис. 5.11. Три счетчика до начала нажатий кнопки

Репозиторий: **rq05-triple-counter**

Этот пример находится в репозитории `rq05-triple-counter`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq05-triple-counter --template rq05-triple-counter
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq05-triple-counter>

Назовем несколько часто встречающихся статических исходных значений, кроме чисел:

- `true` или `false` для *логических значений* — если меню скрыто до нажатия кнопки, состояние `isMenuVisible` инициализируется значением `false`.

- *Пустая строка*, "" — если в форме присутствует текстовое поле для ввода адреса зарегистрированной электронной почты, его состояние инициализируется пустым значением, чтобы поле было пустым до того, как пользователь начнет ввод.
- `null` — отличный вариант заполнителя при наличии сложного значения, которому еще ничего не присвоено.

Для присваивания динамических исходных значений чаще всего используются свойства. В частности, мы воспользовались этой возможностью в предыдущей реализации счетчика. Свойство также будет использоваться, например, в компоненте для изменения имени. Текущее имя передается в виде свойства, а состояние инициализируется на основании полученных данных.

Состояние также можно инициализировать значением из `cookie` или другого локального хранилища. Для формы ввода учетных данных состояние поля с адресом электронной почты можно инициализировать последним адресом, использованным на той же форме и хранящимся в `cookie`.

Важно только первое значение

Создадим новую модификацию счетчика с переменным начальным значением. На этот раз вне счетчика добавим новую кнопку, которая будет изменять исходное значение счетчика. Таким образом, вместо повторной инициализации значения в 0 по нажатию кнопки начальное значение счетчика будет уменьшаться на 10, как показано на диаграмме 5.12. Фактически мы создаем компонент с состоянием поверх другого компонента с состоянием.

Вместо того чтобы с ходу браться за реализацию этого решения, продумаем некоторые возможные сценарии. Что произойдет в следующем случае?

1. Счетчик инициализируется начальным значением 0.
2. Вы нажимаете кнопку, чтобы уменьшить начальное значение, поэтому теперь оно равно -10.
3. Обновится ли счетчик в значение -10?

Представим более широкий сценарий:

1. Счетчик инициализируется начальным значением 0.
2. Вы нажимаете кнопку **Increment** счетчика, чтобы увеличить его значение на 1.
3. Затем вы нажимаете кнопку, чтобы уменьшить начальное значение (которое было равно 0), поэтому теперь оно равно -10.
4. Обновится ли счетчик в значение -10? Или он обновится в значение -9?

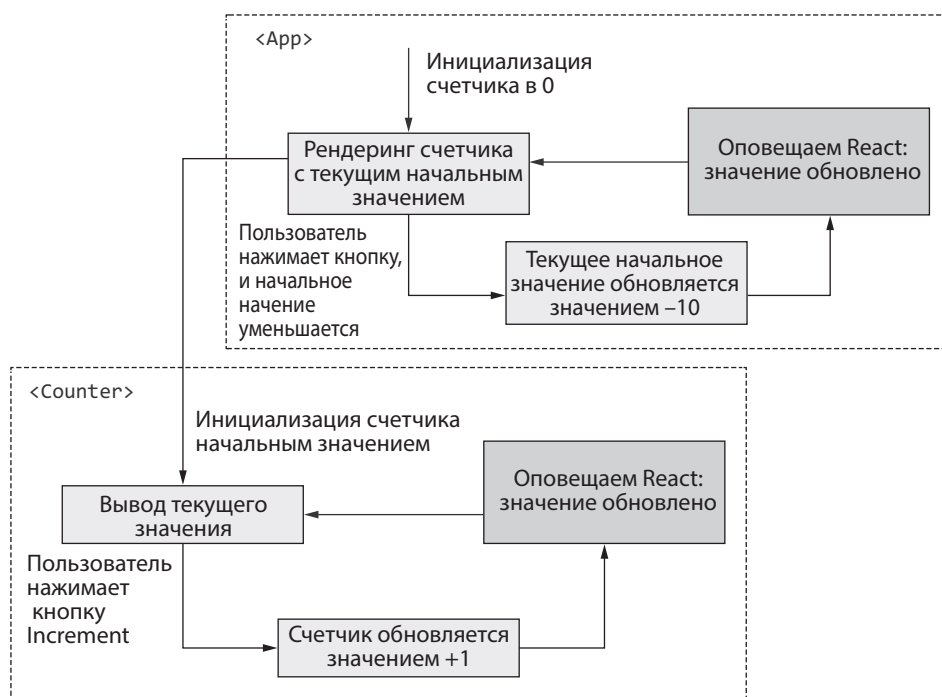


Рис. 5.12. Теперь состоянием обладает как приложение, так и счетчик, и состояние приложения должно использоваться для инициализации счетчика

На самом деле оба описанных сценария бессмысленны, потому что, как говорилось выше, только первое значение, переданное в `useState`, используется как значение для состояния. Если исходное значение изменится при последующем рендеринге, состояние при этом не обновляется. Это и хорошо, и плохо одновременно. Хорошо, потому что в противном случае счетчик всегда бы имел одно и то же значение, так как при каждом рендеринге передавалось бы то же значение. Плохо, потому что настроить изменение значения в зависимости от переданного параметра не удастся (по крайней мере таким способом).

В нашем случае не совсем понятно, что должно произойти при снижении начального значения после начала отсчета. Важно заранее точно определить желаемый результат, прежде чем писать реализацию в коде.

Можно сделать так, чтобы значение состояния обновлялось в зависимости от свойства, но для этого потребуются другие хуки (в частности, хук `useEffect`), о которых речь пойдет в следующей главе.

Функция-инициализатор

В некоторых случаях требуется задать в качестве исходного значения результат некоторого вычисления. Предположим, имеется поле для ввода пароля, которое нужно инициализировать хорошим, сильным паролем, но после того, как пользователь начнет вводить текст, поле должно просто принять введенный пароль. Где-то в коде присутствует затратная функция `generatePassword`, которая используется для создания исходного пароля. На рис. 5.13 представлена диаграмма работы приложения.

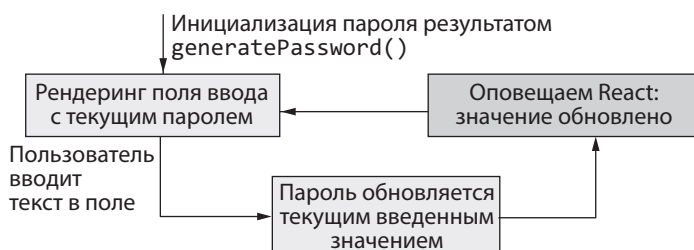


Рис. 5.13. Логика использования функции для генерирования исходного значения

Если реализовать эту схему с использованием исходного значения, результат будет выглядеть примерно так:

```
function Password() {
  const [password, setPassword] = useState(generatePassword());
  ...
}
```

Однако функция `generatePassword()` будет вызываться при каждом рендеринге (потому что она выполняется при каждом рендеринге), но возвращаемое значение будет игнорироваться при всех вызовах, кроме первого, как объяснялось выше. Это может быть сложная функция, которая выполняет несколько алгоритмов с затратными вычислениями, поэтому лучше не выполнять ее, если возвращаемое значение не используется.

В такой ситуации исходным значением может быть функция, которая возвращает реальное исходное значение. Тогда функция исходного значения будет вызвана только при первом рендеринге и будет игнорироваться в дальнейшем, как показано на рис. 5.14.

В общем виде это можно сделать так:

```
const [password, setPassword] = useState(() => generatePassword());
```

а в нашем конкретном случае намного проще:

```
const [password, setPassword] = useState(generatePassword);
```

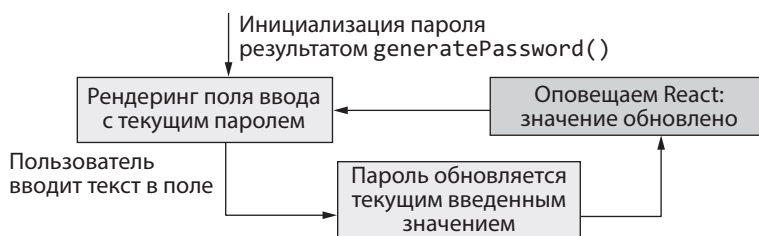


Рис. 5.14. В этом случае функция `generatePassword` не вызывается напрямую. Вместо этого мы указываем хуку вызывать функцию только при необходимости (то есть только при первом рендеринге)

Так как `generatePassword` уже является функцией, ее можно передать в таком виде. Тем не менее, если функция получает аргумент (например, длину сгенерированного пароля), приходится использовать первый вариант:

```
const [password, setPassword] = useState(() => generatePassword(12));
```

Инициализация функцией

А если состояние является функцией? Если передать функцию в исходном значении, она будет вызываться, так как же сохранить функцию как исходное значение? Создайте другую функцию, которая будет возвращать первую. Звучит немного странно, но смысл в этом есть.

Предположим, существует компонент-калькулятор, выполняющий некоторые математические операции (например, сложение, вычитание и умножение) с двумя значениями, вводимыми в два поля. Вычисление выполняется функцией, которая получает два значения и возвращает один ответ. Это можно реализовать в форме типа-перечисления:

```
const OPERATORS = {
  ADDITION: (a, b) => a + b,
  SUBTRACTION: (a, b) => a - b,
  PRODUCT: (a, b) => a * b,
};
function Calculator() {
  const [operator, setOperator] = useState(OPERATORS.ADDITION);
  ...
}
```

Смотрится красиво, но не работает. Процесс изображен на рис. 5.15.

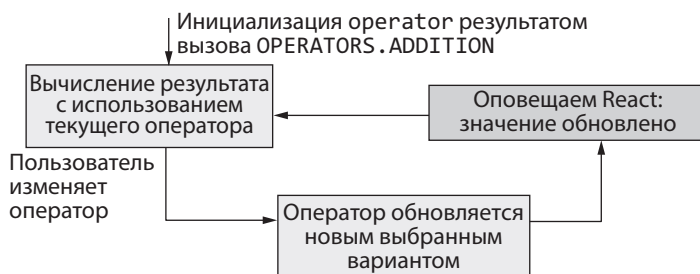


Рис. 5.15. Так как функция передается в качестве исходного значения, React рассматривает ее как функцию-инициализатор и вызывает ее — как делал и раньше

Если включить этот фрагмент кода в компонент, `operator` инициализируется значением `NaN`. Конечно, так происходит потому, что `useState` вызывается с функцией в качестве инициализатора, и поэтому вызывает функцию, но эта функция не знает, что делать без аргументов, и просто возвращает `NaN`. Нужна функция, которая возвращает оператор, как показано на рис. 5.16.



Рис. 5.16. На этот раз функция также передается в качестве исходного значения, но эта функция возвращает нужное значение (которым также оказывается функция)

Возможная реализация выглядит так:

```
function Calculator() {  
  const [operator, setOperator] = useState(() => OPERATORS.ADDITION);  
  ...  
}
```

Это абсолютно нормальная рабочая конструкция. Мы еще вернемся к ней, когда речь пойдет о функциях-сеттерах.

5.2.3. Деструктуризация значения состояния и сеттера

Если вам нужен компонент с состоянием, используйте хук `useState`. Хук возвращает значение, которое деструктуризируется в значение состояния и сеттер:

```
const [value, setter] = useState(initial);
```

Эта конструкция практически обязательна. То же можно сделать и другими способами, но все используют хук `useState` именно так. Если вы будете действовать так же, ваш код будет понятнее другим разработчикам. Это всего лишь необходимое соглашение при использовании этого хука. Другие хуки работают так же, и вам нужно просто привыкнуть к этой записи.

Возвращаемое значение `useState`

Возвращаемое значение хука `useState` выглядит непонятно. Хук возвращает массив из двух элементов. Первый элемент содержит текущее значение состояния, а второй — сеттер. «Принять» это значение и изменить его для своих целей можно разными способами. Можно сохранить возвращенный массив в переменной и обозначать два элемента, `value[0]` и `value[1]` соответственно, а можно скопировать их в две другие переменные. Но рекомендованный и самый популярный способ — деструктуризовать массив непосредственно при присваивании возвращаемого значения переменной и назначить двум возвращаемым значениям любые имена по выбору:

```
const [counter, setCounter] = useState(0);
```

Большинство разработчиков React применяют такую запись практически на автомате. В сообществе принято использовать хук `useState` именно так, и через какое-то время вы перестанете об этом задумываться. Единственное, на что стоит обратить внимание, — выбор имен двух деструктуризованных переменных. Стандартная схема — присвоить имя значению состояния по имени компонента, в котором оно хранится, и присвоить сеттеру такое же имя, но с префиксом `set*`. Именно так мы поступили выше с `counter` и `setCounter` соответственно.

Команды часто разрабатывают собственные стандарты выбора имен или заимствуют их у других команд, но предложенную схему можно считать безопасным стандартом по умолчанию. Единственная сложность может быть связана с логическими значениями состояния. Например, если значение состояния имеет имя `isCollapsed`, то сеттер получит имя `setIsCollapsed`, что странно выглядит на английском, поэтому некоторые предпочтут назвать его `setCollapsed` и пропустить префикс `is*` или `has*`, часто присваиваемый логическим переменным.

Почему `useState` возвращает массив

Итак, вы поняли, что `useState` возвращает массив. Но почему `useState` возвращает массив с двумя несвязанными значениями? Ведь это явно не список!

Представьте, что вы — разработчик ядра React, который занимается созданием хука `useState`. Функция `useState` должна возвращать два значения: текущее состояние, которое может иметь произвольный тип, а второе — сеттер, который может получать любое значение и даже функцию обновления.

В JavaScript нет кортежей или структур, в которых можно «элегантно» объединять разные типы. Казалось бы, можно вернуть объект с двумя свойствами и согласовать их имена (например, `obj.value` и `obj.set`). Деструктурировать их ничуть не сложнее:

```
const { value, set } = useState(0); // На самом деле не работает!
```

Но если вы решили хранить несколько состояний в одном компоненте, вам придется часто переименовывать их. Даже если состояние только одно, возможно, ему все равно лучше присвоить более содержательное имя, а деструктуризация именованных свойств в объекте в разные локальные переменные получается менее краткой, чем с массивами:

```
const {  
  value: counter,  
  set: setCounter,  
} = useState(0); // Тоже не работает
```

Все это приводит к увеличению объема кода и ненужным накладным расходам. Поэтому, вместо того чтобы возвращать более четко определенный объект с двумя именованными свойствами, разработчики React выбрали массив ради простоты использования.

Так как разработчики React отлично знают функцию `useState` и часто используют ее в работе, они привыкают к необычному синтаксису и не обращают на него внимания. Но мы согласны, что он выглядит странно.

5.2.4. Использование значений состояния

Вспомните компонент-счетчик, о котором мы говорили выше. Что произойдет, если изменить функциональность кнопки `Increment`, чтобы вместо увеличения значения она присваивала ему строку `"hi there"`? Теперь это не число, а строка. Логика показана на рис. 5.17.

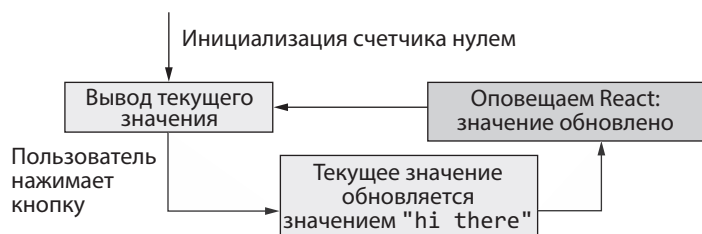


Рис. 5.17. По нажатию кнопки текущему значению счетчика присваивается строка

Попробуем реализовать эту логику:

```

import { useState } from "react";
function Counter() {
  const [counter, setCounter] = useState(0);
  return (
    <main>
      <p>Counter: {counter}</p>
      <button onClick={() => setCounter("hi there")}>
        Increment
      </button>
    </main>
  );
}
  
```

Инициализация счетчика числом

Вывод текущего значения счетчика

По нажатию изменяет значение счетчика на строку

Этот код работает. Если нажать кнопку, получим результат, показанный на рис. 5.18.

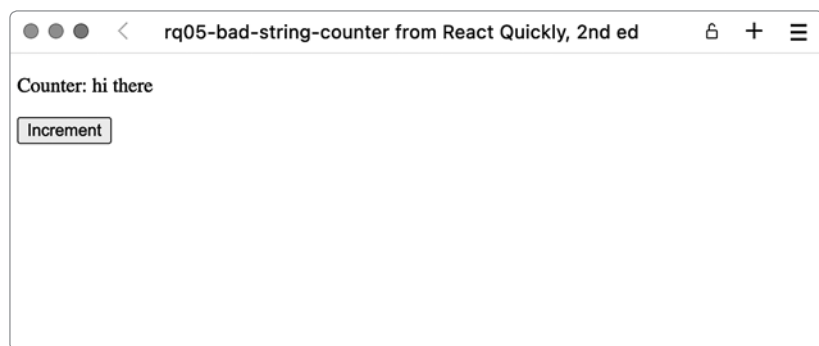


Рис. 5.18. Значение «счетчика» стало строкой, и эта строка выводится в компоненте, потому что мы не проверяем, что значение является числом

Менять тип состояния обычно довольно бессмысленно, — впрочем, как и менять тип любой другой переменной.

Хук `useState` возвращает то состояние, которое вы укажете. Вы можете изменить тип, сложность и т. д. Вы полностью управляете значением. Сначала используется значение, которое было передано в качестве исходного, а после этого — значения, передаваемые сеттеру.

Впрочем, в большинстве сценариев тип состояния изменять не следует. Как и с любой другой переменной в коде, полезно сохранять последовательные типы, хотя JavaScript и не устанавливает таких ограничений. Например, можно инициализировать значение `null`, а позднее присвоить ему число; при этом `null` означает, что конкретное число еще неизвестно, так что инициализация нулем может привести к ошибкам восприятия, например, если в значении хранится возраст. Даже если пользователь еще не указал свой возраст в форме, это не значит, что он равен нулю. Это можно считать изменением типа; изначально используется `null`, которое затем заменяется числом.

Конечно, в состоянии можно хранить литералы объектов, что может быть удобно для взаимосвязанных значений, которые всегда вместе обновляются или используются. Например, можно создать компонент-загрузчик, который выводит индикатор загрузки файла как в процентах, так и в текстовом виде (доля загруженных байтов от общего количества):

```
function Loader() {
  const [progress, setProgress] = useState(null);
  const someCallback = () => {
    ...
    setProgress({ loaded, total });
  };
  if (!progress) {
    return null;
  }
  const { loaded, total } = progress;
  return (
    <h2>{Math.floor(100 * loaded / total)}%</h2>
    <p>Loaded { loaded } out of {total}</p>
  );
}
```

Приведенный пример неполон: никакие данные в нем не загружаются, поэтому понадобится дополнительная логика для загрузки данных и проверки значений. Тем не менее он демонстрирует хранение взаимосвязанных значений в одном значении состояния.

Далее мы поговорим о том, как использовать несколько состояний, чтобы не пытаться втиснуть все состояния в одно значение. Размещать несколько значений в одном состоянии можно только в том случае, если значения тесно связаны друг с другом, как в предыдущем примере с `Loader`.

5.2.5. Присваивание значения

Задать значение состояния относительно несложно — это делается так же, как задается исходное значение, с теми же хитростями и обходными решениями. Состояние можно обновить, задавая либо статическое значение, либо функцию обновления, которая возвращает новое задаваемое значение.

Присваивание статического значения

Создадим простой компонент-«аккордеон» с возможностью сворачивания и разворачивания его содержимого. В заголовке присутствуют две кнопки: «+» и «-» соответственно. Кнопка + разворачивает компонент и показывает его содержимое, а кнопка – сворачивает компонент и скрывает содержимое. Логика компонента представлена на рис. 5.19, а ее реализация приведена в листинге 5.3.

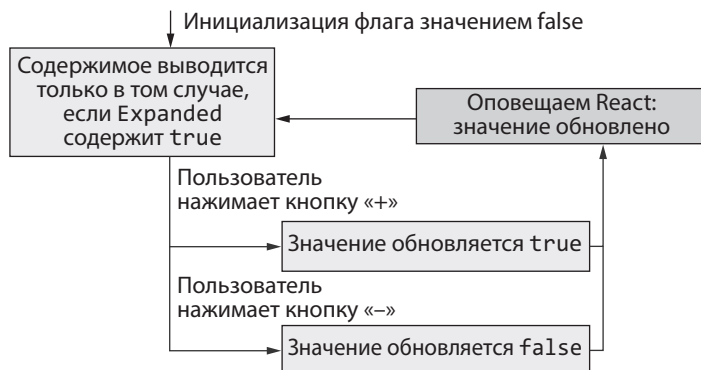


Рис. 5.19. Блок-схема логики компонента-«аккордеона». Логической переменной присваивается true или false в зависимости от того, какая кнопка была нажата

Листинг 5.3. Простой компонент-«аккордеон»

```

import { useState } from "react";
function Accordion() {
  const [isExpanded, setExpanded] = useState(false);
  return (
    <main>
      <h2 style={{ display: "flex", gap: "6px" }}>
        Secret password
        <button onClick={() => setExpanded(false)}>
          -
        </button>
        <button onClick={() => setExpanded(true)}>
          +
        </button>
      </h2>
    </main>
  );
}
  
```

Инициализация состояния значением false

Нажатие кнопки вызывает сеттер с передачей true или false


```
    </h2>
    {isExpanded && (
      <p>
        Password: <code>hunter2</code>.
      </p>
    )}
  </main>
);
}
function App() {
  return <Accordion />;
}
export default App;
```

Выводит скрытое содержимое компонента, если флаг содержит true

Репозиторий: rq05-accordion

Этот пример находится в репозитории rq05-accordion. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq05-accordion --template rq05-accordion
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq05-accordion>

На рис. 5.20 показан результат в браузере. Пример демонстрирует возможность использования сеттера со статическим значением. Кнопка «минус» всегда задает состоянию значение `false` независимо от того, сколько раз она нажата. Так как состоянию задается фиксированное значение, проверять текущее значение не нужно.

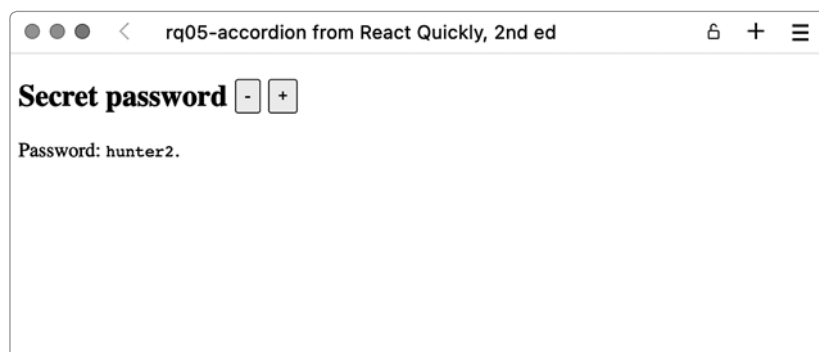


Рис. 5.20. Нажатие кнопки + раскрывает скрытое содержимое компонента

Присваивание через функцию обновления

В качестве значения можно указать как непосредственное значение, что мы только что сделали, так и функцию обновления, которая возвращает новое значение. Если использовать функцию обновления, текущее состояние будет передаваться в аргументе.

Пример использования функции обновления мы уже видели:

```
const [counter, setCounter] = useState(0);
...
<button onClick={() => setCounter((value) => value + 1)}>
```

Значение в состоянии обновляется через простую функцию-инкремент, которая получает аргумент и возвращает аргумент +1.

Присваивание функции

Чтобы присвоить состоянию функцию, придется использовать такое же обходное решение, как с исходным значением. Нам понадобится функция, возвращающая функцию-оператор.

Таким образом, добавив в рассмотренный пример с калькулятором кнопки для выбора оператора, мы реализуем полное приложение. Логика представлена на рис. 5.21, а ее реализация приведена в листинге 5.4.

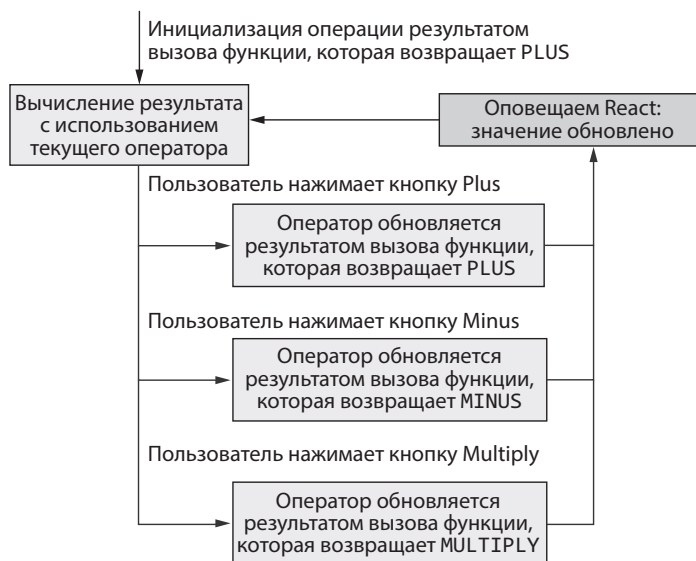


Рис. 5.21. Расширенный пример с калькулятором теперь содержит три кнопки для смены оператора

Листинг 5.4. Простой калькулятор

```

import { useState } from "react";
const PLUS = (a, b) => a + b;
const MINUS = (a, b) => a - b;
const MULTIPLY = (a, b) => a * b;
function Calculator({ a, b }) {
  const [operator, setOperator] = useState(() => PLUS);
  return (
    <main>
      <h1>Calculator</h1>
      <button
        onClick={() => setOperator(() => PLUS)}
      >
        Plus
      </button>
      <button
        onClick={() => setOperator(() => MINUS)}
      >
        Minus
      </button>
      <button
        onClick={() => setOperator(() => MULTIPLY)}
      >
        Multiply
      </button>
      <p>
        Result of applying operator to {a} and {b}:
        <code> {operator(a, b)} </code>
      </p>
    </main>
  );
}
function App() {
  return <Calculator a={7} b={4} />;
}
export default App;

```

Инициализирует состояние функцией, возвращающей функцию-оператор по умолчанию

Обновляет состояние функцией, возвращающей функцию выбранного оператора

Можно вызвать значение состояния как функцию, так как теперь оно всегда является функцией

Репозиторий: rq05-calculator

Этот пример находится в репозитории `rq05-calculator`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq05-calculator --template rq05-calculator
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq05-calculator>

Калькулятор (пусть и не самый мощный) показан на рис. 5.22.

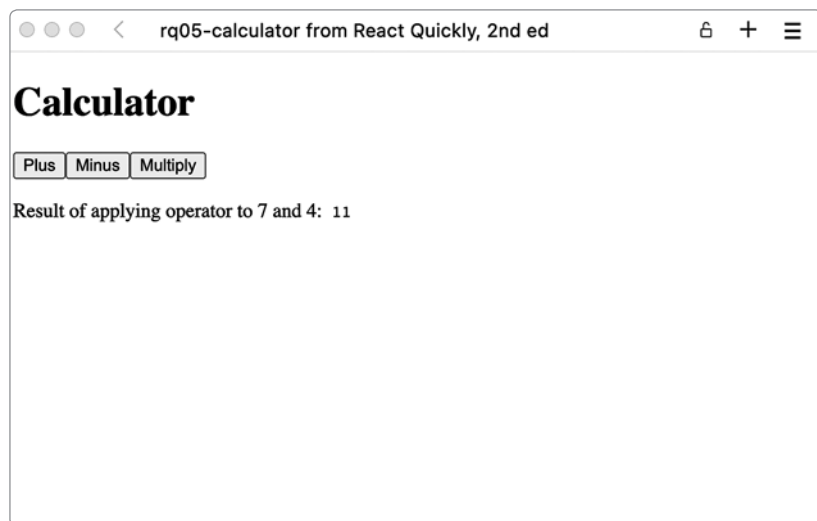


Рис. 5.22. Калькулятор с операцией по умолчанию PLUS. Оператор нигде не отображается, выводится только результат применения оператора к двум операндам

Присваивание и рендеринг

Что произойдет, если продолжить нажимать кнопку **Plus** на калькуляторе? Будет ли компонент рендериться заново, в результате чего каждый раз будет выполняться вычисление? Каждый раз состоянию присваивается одно и то же значение, так для чего это делать?

На самом деле компонент заново рендериться не будет. React включает встроенную оптимизацию, так что `useState` ожидает конца текущего цикла для обновления компонента. Функция проверяет, изменилось ли значение, а затем повторно рендерит компонент только в том случае, если оно изменилось. Из-за этого могут возникнуть ситуации, в которых сеттер состояния вызывается, но повторный рендеринг не выполняется (так как ничего не изменилось, повторный рендеринг считается излишним).

Расширим счетчик, описанный выше, и добавим в него кнопку **Reset**, которая обнуляет счетчик. Блок-схема обновленной версии изображена на рис. 5.23.

Реализация этой логики приведена в листинге 5.5. Новый счетчик с возможностью сброса показан на рис. 5.24.



Рис. 5.23. Новая кнопка Reset обнуляет счетчик независимо от предыдущего значения

Листинг 5.5. Счетчик с возможностью сброса

```

import { useState } from "react";
function Counter() {
  const [counter, setCounter] = useState(0);
  return (
    <main>
      <p>Counter: {counter}</p>
      <button onClick={() => setCounter((val) => val + 1)}>
        Increment
      </button>
      <button onClick={() => setCounter(0)}> ← Нажатием кнопки Reset счетчик обнуляется
        Reset
      </button>
    </main>
  );
}
function App() {
  return <Counter />;
}
export default App;
  
```

Репозиторий: rq05-reset-counter

Этот пример находится в репозитории `rq05-reset-counter`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq05-reset-counter --template rq05-reset-counter
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq05-reset-counter>

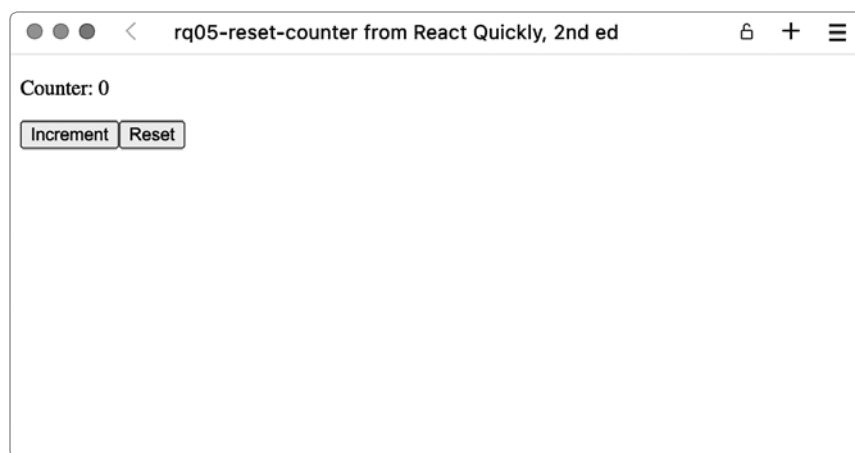


Рис. 5.24. Счетчик с возможностью сброса, который только что был обнулен

Кнопка **Reset** сбрасывает значение счетчика до 0. Если снова нажать кнопку, ничего не происходит. Но как определить, был компонент повторно отрендерен или нет? Узнать ответ поможет очень полезный плагин для Chrome, Firefox и современных версий Edge, называемый React Developer Tools. Его можно загрузить из соответствующих хранилищ:

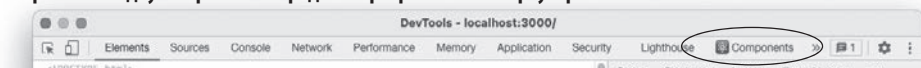
- Chrome и Edge: <http://mng.bz/wvoq>
- Firefox: <http://mng.bz/qrYw>

Этот плагин позволяет увидеть, когда выполняется рендеринг любого компонента. Инструкции по использованию React Developer Tools приведены на рис. 5.25.

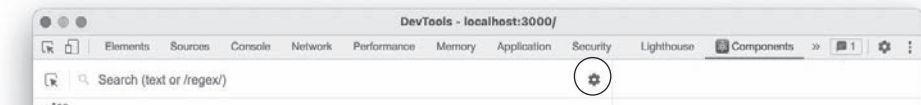
Завершив установку плагина, вернитесь к приложению счетчика и нажмите кнопку **Increment**. Вы увидите, что синяя рамка вокруг всего компонента коротко мигает при каждом увеличении счетчика. Примерный вид приложения показан на рис. 5.26.

Если нажать кнопку **Reset**, когда счетчик не равен 0, вы увидите, как синяя рамка мигнет, потому что компонент был отрендерен. Но если нажать кнопку **Reset**, когда счетчик уже равен 0, синяя рамка не появляется. React понимает, что если состояние не изменилось, то и вывод компонента не изменился (или по крайней мере не должен измениться).

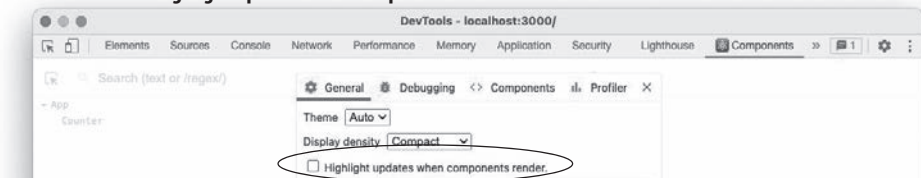
1. Откройте вкладку Components в средствах разработчика в браузере.



2. Нажмите на шестеренку, обозначенную кружком настроек (это не значок с шестеренкой в правом верхнем углу!), чтобы открыть панель.



3. Отметьте чекбокс Highlight updates when components render.



4. После этого настройки должны выглядеть так:

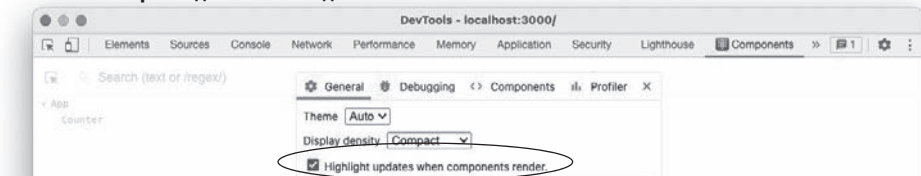


Рис. 5.25. Откройте вкладку Components в React Developer Tools, а затем — меню с шестеренкой и выберите Highlight Updates When Components Render

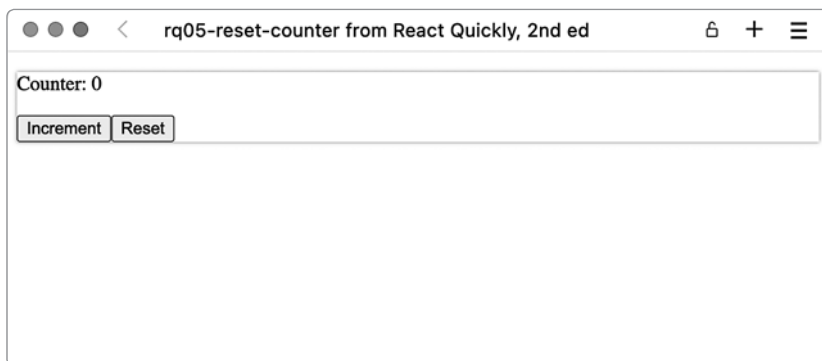


Рис. 5.26. Синяя рамка вокруг всего компонента показывает, что он был отрендерен из-за изменения состояния

Состоянию необходимо задать новое значение

Условие повторного рендеринга также означает, что если значению состояния присваивается тот же объект, который в нем уже содержится (даже если этот объект был изменен «изнутри»), ничего не произойдет, потому что объект не рендерится заново. Например, это может произойти, если в состоянии хранится массив. Если вы выполняете операции с массивом «на месте» и снова присваиваете тот же массив как значение состояния, компонент рендериться не будет, потому что ничего не изменилось (по крайней мере с точки зрения равенства ссылок).

Посмотрим, как это происходит на практике, и обсудим возможные решения. Для этого создадим простое приложение списка задач. У нас есть список элементов, которые можно пометить в списке как выполненные; при пометке элемента он удаляется из массива, а затем список рендерится снова.

Неверное решение — каждый раз присваивать состоянию один и тот же массив. Неважно, был ли массив изменен перед его повторным присваиванием состоянию, потому что React не проверяет значение состояния, а учитывает только ссылку. Логика такого решения схематически представлена на рис. 5.27.

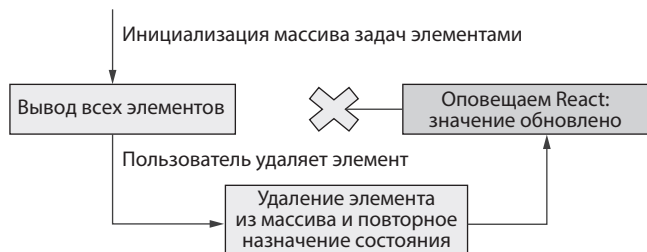


Рис. 5.27. Ошибочный способ использования массива в качестве значения состояния — каждый раз присваивать состоянию один и тот же массив. Проблема в том, что React не обнаружит никаких изменений и не выполнит повторный рендеринг компонента

Хотя вы уже понимаете, что именно здесь не так, все равно попробуем реализовать этот вариант. Это поможет убедиться, что это решение действительно не работает.

Листинг 5.6. Неработоспособное приложение со списком задач¹

```
import { useState } from "react";
function TodoApplication({ initialList }) {
```

¹ В итоге получится «Feed the plants» — накормить цветы; «Water the dishes» — полить посуду; «Clean the cat» — помыть кота. — *Примеч. пер.*


```
const [todos, setTodos] = useState(initialList);
return (
  <main>
    {todos.map((todo, index) => (
      <p key={todo}>
        {todo}
        <button
          onClick={() => {
            todos.splice(index, 1); ← Изменяет массив на месте
            setTodos(todos); ← Обновляет состояние
                              значением, которое в нем
                              уже содержится (хотя оно
                              уже было изменено)
          }}
        >
          x
        </button>
      </p>
    ))}
  </main>
);
}
function App() {
  const items = [
    "Feed the plants",
    "Water the dishes",
    "Clean the cat"
  ];
  return <TodoApplication initialList={items} />;
}
export default App;
```

Репозиторий: rq05-bad-todo

Этот пример находится в репозитории rq05-bad-todo. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq05-bad-todo --template rq05-bad-todo
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq05-bad-todo>

Проверим, как работает это решение. Попробуйте нажать кнопки **Delete**, показанные на рис. 5.28. Ничего не происходит. Если вы попытаетесь включить контуры обновления компонентов в плагине React Developer Tools, то увидите, что компонент заново не рендерится, потому что значение состояния остается идентичным по ссылке, хотя его содержимое было «обновлено».



Рис. 5.28. Приложение со списком задач выглядит нормально, но не работает — при нажатии кнопок ничего не происходит

По этой причине изменять состояние напрямую не только не рекомендуется, но и категорически не следует. Чтобы операция выполнялась правильно, необходимо присвоить значению состояния новый массив, который является дубликатом прежнего, но без исключенного элемента. Один из возможных способов — применить оператор `spread` к срезу массива до и после удаляемого элемента. Чтобы исправить модель и привести ее к виду, представленному на рис. 5.29, создадим новый массив и зададим *его* как новое состояние. Реализация в листинге 5.7 следует логике диаграммы.

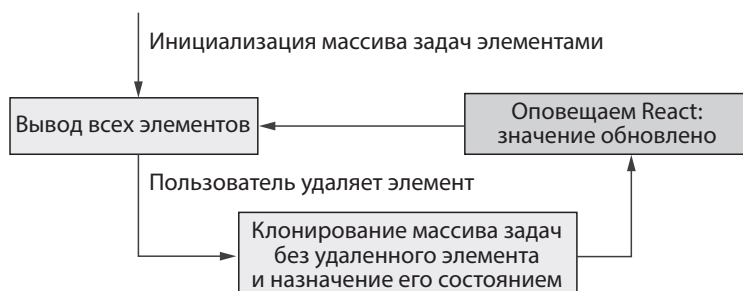


Рис. 5.29. Теперь новый массив передается сеттеру каждый раз, когда из списка удаляется элемент. React правильно определяет, что состояние изменилось, и рендерит компонент заново

Листинг 5.7. Верная реализация приложения со списком задач

```
import { useState } from "react";
function TodoApplication({ initialList }) {
  const [todos, setTodos] = useState(initialList);
  return (
    <main>
      {todos.map((todo, index) => (
        <p key={todo}>
          {todo}
          <button
            onClick={() => {
              setTodos((value) => [
                ...value.slice(0, index),
                ...value.slice(index + 1),
              ]);
            }}
          </button>
        </p>
      ))}
    </main>
  );
}
function App() {
  const items = ["Feed the plants", "Water the dishes", "Clean the cat"];
  return <TodoApplication initialList={items} />;
}
export default App;
```

Присваивает состоянию новый массив, который строится конкатенацией двух частей: среза прежнего массива от начала до элемента, предшествующего удаленному, и среза прежнего массива от элемента, следующего за удаленным, до конца массива

Репозиторий: rq05-proper-todo

Этот пример находится в репозитории `rq05-proper-todo`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq05-proper-todo --template rq05-proper-todo
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq05-proper-todo>

Выглядит как раньше, но теперь из списка действительно можно удалять элементы, как показано на рис. 5.30.

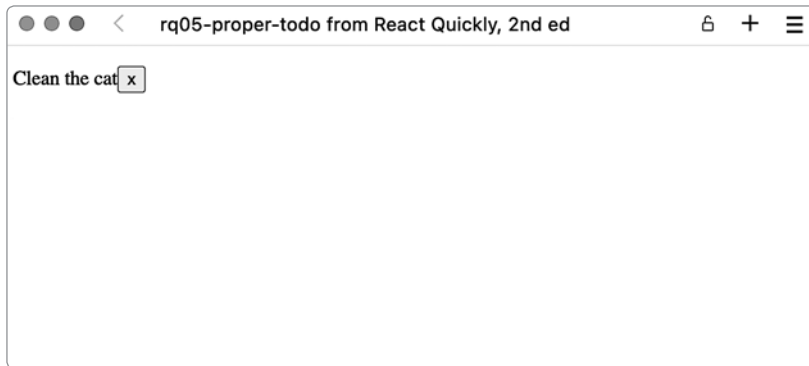


Рис. 5.30. Приложение со списком задач теперь работает: два элемента списка задач были помечены как завершённые

5.2.6. Использование множественных состояний

Мы уже несколько раз затрагивали эту тему, но давайте еще раз проговорим: да, в одном компоненте можно использовать несколько хуков `useState`, и вы достаточно часто будете это делать. Для примера доработаем новую версию приложения со списком задач. Пусть завершённые задачи не удаляются из массива, а помечаются как завершённые. Завершённые задачи будут выводиться в списке зачеркнутыми. Кроме того, добавим новый фильтр, позволяющий выводить все элементы или только незавершённые.

Чтобы отфильтровать список, необходимо хранить информацию о том, должны ли отфильтровываться завершённые элементы. Идеальное решение — добавить еще одно значение состояния, в котором хранится флаг фильтрации, как показано на рис. 5.31. Реализация приведена в листинге 5.8.

Листинг 5.8. Список задач с фильтром

```
import { useState } from "react";
function markDone(list, index) {
  return list.map(
    (item, i) => (i === index ? { ...item, done: true } : item)
  );
}
function TodoApplication({ initialList }) {
  const [todos, setTodos] = useState(initialList);
```

Создает небольшую вспомогательную функцию, которая получает массив объектов задач и возвращает новый массив с теми же объектами, не считая того, что один из них (обозначенный вторым аргументом) будет помечен как завершённый

По-прежнему инициализирует список задач с использованием хука `useState`

```

const [hideDone, setHideDone] = useState(false);
const filteredTodos = hideDone
  ? todos.filter(({ done }) => !done)
  : todos;
return (
  <main>
    <div style={{ display: "flex" }}>
      <button onClick={() => setHideDone(false)}>
        Show all
      </button>
      <button onClick={() => setHideDone(true)}>
        Hide done
      </button>
    </div>
    {filteredTodos.map((todo, index) => (
      <p key={todo.task}>
        {todo.done ? (
          <strike>{todo.task}</strike>
        ) : (
          {todo.task}
          <button
            onClick={() => setTodos((value) =>
              markDone(value, todo.index)
            )}
          </button>
        )}
        x
      </p>
    ))}
  </main>
);
}
function App() {
  const items = [
    { task: "Feed the plants", done: false, index: 0 },
    { task: "Water the dishes", done: false, index: 1 },
    { task: "Clean the cat", done: false, index: 2 },
  ];
  return <TodoApplication initialList={items} />;
}
export default App;

```

Использует флаг для обязательной фильтрации списка задач

Но теперь имеется второй экземпляр хука `useState` для нового флага фильтрации, которому по умолчанию присваивается `false`

Две кнопки вызывают сеттер фильтрации с `true` или `false`

Не забудьте использовать новый (возможно, отфильтрованный) список

Если задача завершена, она выводится зачеркнутой

Если задача не завершена, то рендерится кнопка, которая вызывает вспомогательную функцию и обновляет состояние списка задач

Создает список исходных элементов как список объектов, каждый из которых помечен как еще не заверченный. Помните о необходимости сохранить исходную позицию каждого элемента, так как индекс в отфильтрованном массиве будет отличаться от исходного индекса



Рис. 5.31. Теперь состояние можно обновлять тремя способами. Если элемент помечен как завершённый, все равно нужно создать новый массив, но в нем этот элемент должен быть помечен как завершённый. При переключении фильтра мы просто задаем новое значение соответствующего флага состояния

Репозиторий: `rq05-filter-todo`

Этот пример находится в репозитории `rq05-filter-todo`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq05-filter-todo --template rq05-filter-todo
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq05-filter-todo>

Приложение в действии показано на рис. 5.32. Попробуйте использовать разные кнопки и посмотрите, что получится.

Конечно, вы не ограничены двумя значениями состояния на один компонент. Вы можете использовать сколько угодно состояний, хотя если их будет больше

десяти, за ними будет сложнее уследить. При усложнении состояний мы рекомендуем использовать провайдеры контекста, редьюсеры или нестандартные хуки (или их комбинацию). Мы вернемся к этим средствам более высокого уровня в главе 10.

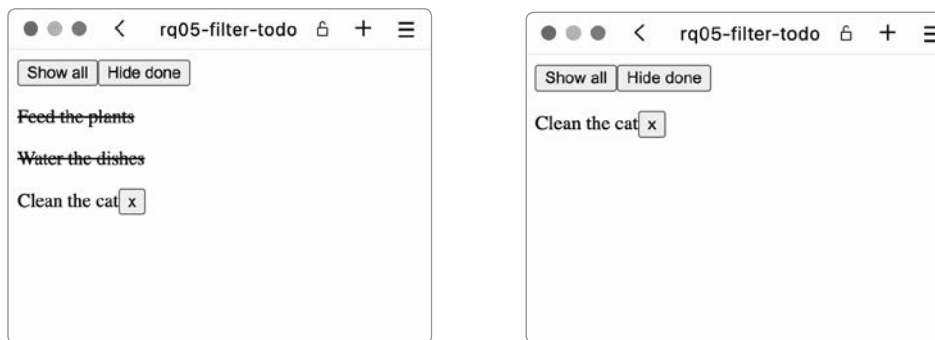


Рис. 5.32. После завершения двух простых задач можно выбрать предпочтительный вариант — видеть весь список и радоваться тому, что выполнено 67 % дел, или же видеть только оставшиеся задачи

5.2.7. Область видимости состояния

Во всех компонентах, которые мы до сих пор создавали, мы обращались к состоянию внутри самого компонента, а не за его пределами, где определяется состояние. Но что делать, если состояние должно охватывать несколько компонентов? Что, если требуется обратиться к значению в одном компоненте, но обновить его в другом? Мы уже намекали на такую возможность в начале главы, когда рассказывали о количестве компонентов с состоянием во всем дереве компонентов приложения, но еще не применяли ее на практике.

Воспользуемся свойствами для передачи значений состояния и сеттеров состояния соответствующим компонентам. Диаграмма состояния остается той же; отличается только дерево компонентов. Если в предыдущих примерах все делал один компонент, сейчас мы будем работать с группой компонентов. Компонент `ToDoApplication` все еще остается компонентом с состоянием, который содержит два значения состояния. В его работе будут участвовать компоненты `FilterButton` и `Task`, которые обеспечат рендеринг верхних кнопок фильтрации и отдельных задач в списке соответственно. На рис. 5.33 показано новое дерево компонентов, а также все его свойства.

Объединим все эти компоненты в одном приложении из листинга 5.9. И заодно немного улучшим внешний вид приложения, применив стили.

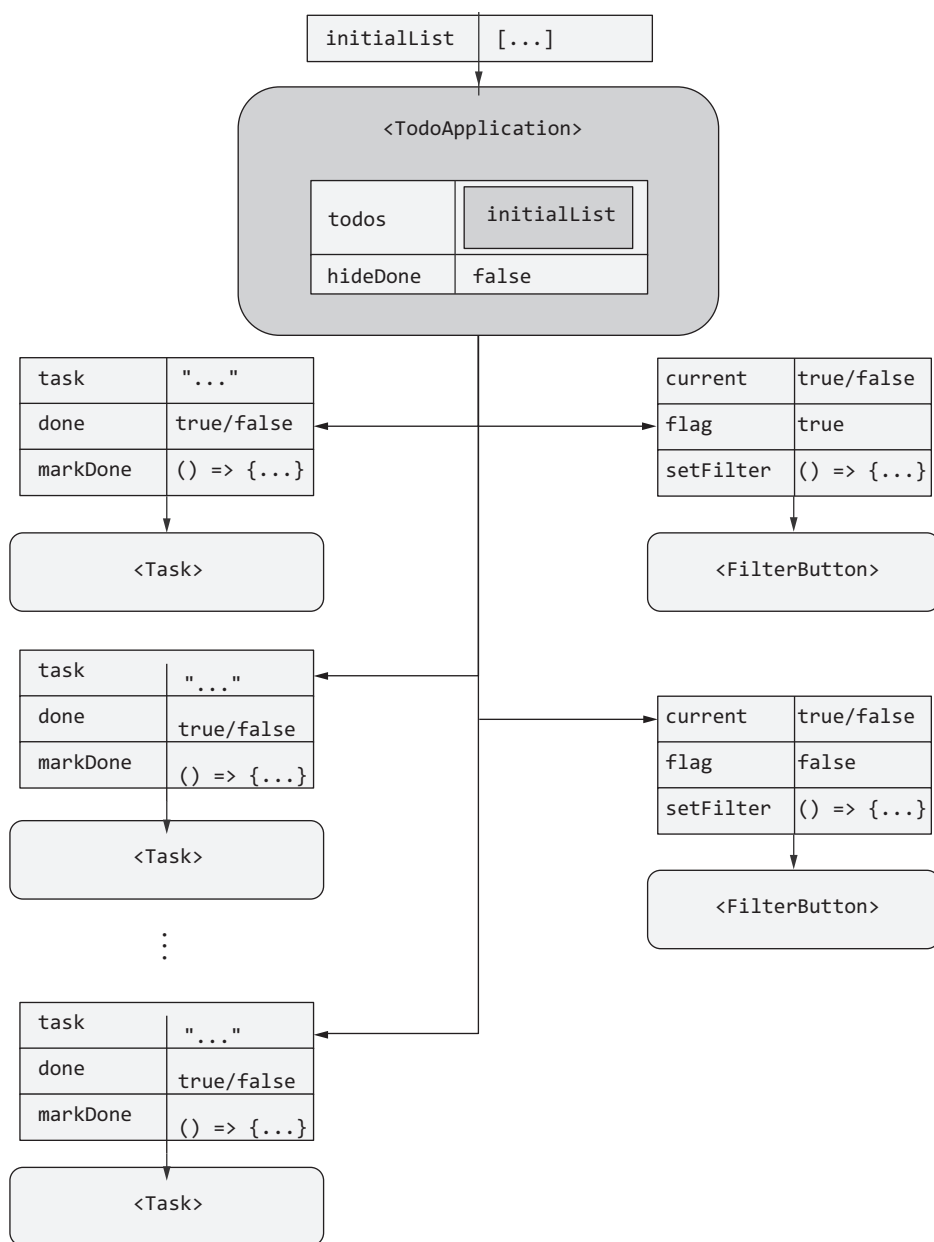


Рис. 5.33. Дерево компонентов многокомпонентного приложения со списком задач. Мы рендерим переменное количество экземпляров `Task`, по одному для каждого элемента списка, и всегда ровно две кнопки фильтрации

Листинг 5.9. Доработанное многокомпонентное приложение со списком задач

```

import { useState } from "react";
function markDone(list, index) {
  return list.map(
    (item, i) => (i === index ? { ...item, done: true } : item)
  );
}
function FilterButton(
  { current, flag, setFilter, children }
) {
  const style = {
    border: "1px solid dimgray",
    background: current === flag ? "dimgray" : "transparent",
    color: current === flag ? "white" : "dimgray",
    padding: "4px 10px",
  };
  return (
    <button
      style={style}
      onClick={() => setFilter(flag)}
    >
      {children}
    </button>
  );
}
function Task({ task, done, markDone }) {
  const paragraphStyle = {
    color: done ? "gray" : "black",
    borderLeft: "2px solid",
  };
  const buttonStyle = {
    border: "none",
    background: "transparent",
    display: "inline",
    color: "inherit",
  };
  return (
    <p style={paragraphStyle}>
      <button
        style={buttonStyle}
        onClick={done ? null : markDone}
      >
        {done ? "✓ " : "○ "}
      </button>
      {task}
    </p>
  );
}
function TodoApplication({ initialList }) {
  const [todos, setTodos] = useState(initialList);
  const [hideDone, setHideDone] = useState(false);

```

FilterButton получает четыре свойства и использует их для рендеринга кнопки

Свойство onClick кнопки вызывает переданный сеттер с переданным значением

Аналогичным образом компонент Task получает набор свойств, включая обратный вызов

На этот раз мы просто вызываем переданный обратный вызов нажатием кнопки, потому что он выполняет всю необходимую работу, но только в том случае, если элемент еще не завершен

```

const filteredTodos = hideDone
  ? todos.filter(({ done }) => !done)
  : todos;
return (
  <main>
    <div style={{ display: "flex" }}>
      <FilterButton
        current={hideDone}
        flag={false}
        setFilter={setHideDone}
      >
        Show all
      </FilterButton>
      <FilterButton
        current={hideDone}
        flag={true}
        setFilter={setHideDone}
      >
        Hide done
      </FilterButton>
    </div>
    {filteredTodos.map((todo, index) => (
      <Task
        key={todo.task}
        task={todo.task}
        done={todo.done}
        markDone={() => setTodos((value) =>
          markDone(value, todo.index)
        )}
      />
    ))}
  </main>
);
}
function App() {
  const items = [
    { task: "Feed the plants", done: false, index: 0 },
    { task: "Water the dishes", done: false, index: 1 },
    { task: "Clean the cat", done: false, index: 2 },
  ];
  return <TodoApplication initialList={items} />;
}
export default App;

```

Две кнопки фильтрации
в последнем компоненте
с почти одинаковыми
свойствами

Для каждого элемента
задачи создается экземпляр
компонента Task

markDone присваивается
та же функция обновления,
что и прежде

Репозиторий: rq05-nice-todo

Этот пример находится в репозитории `rq05-nice-todo`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq05-nice-todo --template rq05-nice-todo
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq05-nice-todo>

Готово! Перед вами первое законченное, работающее и четко структурированное приложение для React! Вывод такой же, как и прежде, но выглядит намного привлекательнее, как видно из рис. 5.34.



Рис. 5.34. Полностью готовое приложение со списком задач — и даже с привлекательным интерфейсом! Фильтр установлен в режим «Показать все задачи» (Show All), но если переключить его в режим «Скрыть завершенные задачи» (Hide Done), будет выведен только последний элемент, как и в предыдущей реализации

В основе этого приложения лежат те же принципы, что и в основе любого другого. Состояние хранится на одном уровне и передается другим компонентам там, где это необходимо, чтобы выводить нужный результат. В последнем приложении со списком задач состояние хранится «глобально» в компоненте `TodoApplication`, а не локально внутри каждого дочернего компонента.

Если потребуется добавить в дерево еще один компонент, который находится рядом со списком задач, но должен иметь доступ к тем же значениям состояния, как и список задач, нужно будет поднять состояния из компонента `TodoApplication` в компонент `App`, а затем передавать значения и сеттеры вниз

компоненту `TodoApplication`. Это более сложная работа, но в главе 10 мы представим удобное решение с использованием контекстов React.

5.3. КОМПОНЕНТЫ НА БАЗЕ КЛАССОВ С СОСТОЯНИЕМ

До сих пор речь шла о добавлении состояния к функциональным компонентам. Однако компоненты с состоянием существовали еще до появления хуков. Собственно, состояние было главной особенностью, встроенной в функциональность компонентов на базе классов.

В компонентах на базе классов работа с состоянием строится аналогично и включает четыре шага, представленных на рис. 5.5:

1. Инициализация состояния.
2. Вывод текущего значения.
3. Обновление состояния.
4. Оповещение React об изменении состояния.

Вы видели массу примеров реализации этих четырех шагов в функциональном компоненте. Теперь посмотрим, как их реализовать в компоненте на базе класса. API выглядит аналогично, но синтаксис несколько отличается; поведение также слегка изменяется. Впрочем, базовая концепция остается той же.

Код (частичного) компонента счетчика выглядит так:

```
class Counter extends Component {
  state = { counter: 0 }
  render() {
    return (
      ...
      <p>Clicks: {this.state.counter}</p>
      <button onClick={() =>
        this.setState({ counter: this.state.counter + 1 })
      ...
      });
  }
}
```

На рис. 5.35 приведена краткая схема соответствия между частями кода и фазами жизненного цикла состояния. Вместо того чтобы хранить значения в локальных

переменных, как это делается в функциональном компоненте, мы храним значения состояния в поле класса `this.state`.

В этом разделе вы сначала узнаете, что добавление состояния в компоненты на базе классов похоже на добавление состояния в функциональные компоненты, но с некоторыми отличиями в синтаксисе; затем мы разберем три больших изменения в поведении; и наконец, кратко рассмотрим процесс преобразования компонента на базе класса, обладающего состоянием, в функциональный компонент. Мы будем приводить не полные примеры, а только фрагменты, в которых есть отличия.

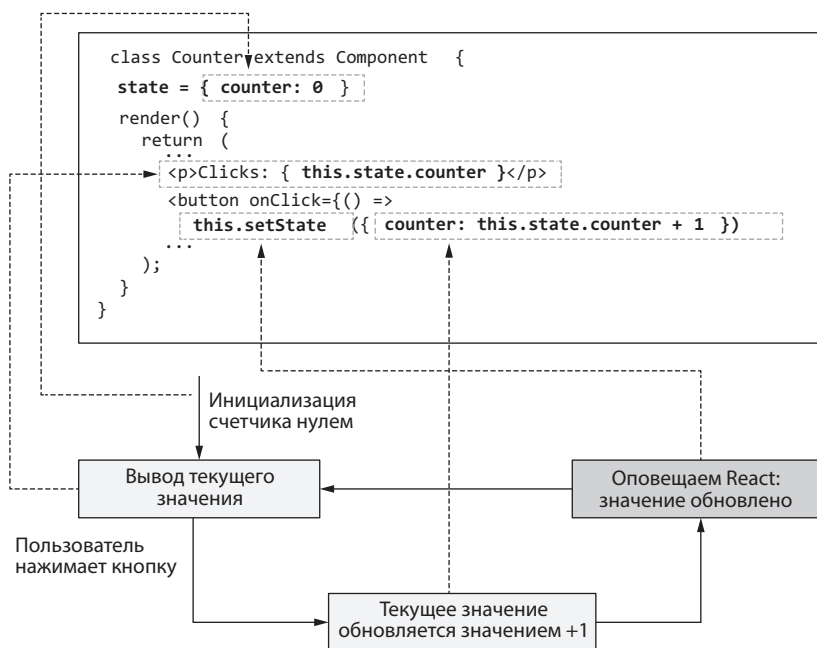


Рис. 5.35. Блок-схема данных в счетчике с добавлением кода. Пунктирные линии со стрелками соединяют состояние на блок-схеме с частью кода, ответственной за это действие

5.3.1. Сходство с хуком `useState`

Все, что мы до сих пор делали с функциональными компонентами, можно сделать и с компонентами на базе классов. Основные процессы те же — инициализация, обновление и вывод состояния, просто для них используется несколько иной синтаксис. Различия в синтаксисе коротко представлены в табл. 5.1.

Таблица 5.1. Состояние в функциональных компонентах и компонентах на базе классов

Функциональный компонент	Компонент на базе класса
<pre>const [counter, setCounter] = useState(0);</pre> Инициализация состояния статическим значением.	<pre>state = { counter: 0, }</pre> При инициализации состояния статическим значением можно использовать поле класса.
<pre>const [counter, setCounter] = useState(initialValue);</pre> Инициализация состояния динамическим значением из свойства.	<pre>constructor(props) { this.state = { counter: props.initialValue, }; }</pre> Обращение к свойству в конструкторе и инициализация состояния через <code>this.state</code>.
<pre><p> Counter: {counter} </p></pre>	<pre><p> Counter: {this.state.counter} </p></pre>
<pre>onClick={() => setCounter(0) }</pre> Если состоянию присваивается фиксированное значение, можно использовать нефункциональную разновидность.	<pre>onClick={() => this.setState({ counter: 0 }) }</pre> Здесь происходит то же самое, только с созданием объекта.
<pre>onClick={() => setCounter(value => value + 1) }</pre> При задании функции обновления мы просто используем старое значение и возвращаем новое значение с типом, указанным в состоянии.	<pre>onClick={() => this.setState(({ counter }) => ({ counter: counter + 1 })) }</pre> Если состоянию присваивается динамическое значение, вычисленное на основе текущего значения, можно использовать функциональную реализацию обновления, но в этом случае необходимо вернуть объект, полученный на основе прежнего объекта состояния.

5.3.2. Отличия от хука `useState`

Использование состояния в компонентах на базе классов в некоторых отношениях отличается от работы с состоянием в функциональных компонентах. Эти отличия весьма важны и влияют на специфику использования состояния в компонентах на базе классов. Основные отличия перечислены ниже:

- Можно иметь только один объект состояния, который всегда должен быть объектом.
- Компоненты *всегда заново рендерятся при обновлении*, даже если ничего не изменилось.
- При обновлении состояния происходит слияние объектов, что открывает возможность *частичных обновлений*.

В следующих подразделах мы рассмотрим и кратко проиллюстрируем все эти отличия.

Только один объект состояния

Как показано в табл. 5.1, состояние компонента на базе класса существует внутри объекта состояния. Даже если у вас есть всего одно значение (например, счетчик), его необходимо создать в объекте состояния, обновлять в объекте состояния и отображать из объекта состояния.

Следствием этого становится легкий переход от одного значения состояния к нескольким. Вы просто добавляете второе свойство в объект состояния, и по сути, это все. Введя состояние в компонент на базе класса, вы сможете без проблем поддерживать одно или несколько значений состояния.

Рендеринг компонентов при каждом обновлении состояния

В разделе 5.2.5 говорилось, что хук `useState` инициирует повторный рендеринг компонента только в том случае, если состояние действительно изменилось. Если вы присваиваете значению состояния `0`, а оно уже содержит `0`, компонент обновляться не будет. React предполагает, что компоненты являются чистыми и компонент будет рендерить одинаковые результаты, если состояние не обновлялось.

Но в прошлом все было иначе, и некоторые приложения действительно зависели от этого поведения. В компоненте на базе класса можно вызвать `setState` с теми же значениями и даже без значений, и React повторно отрендерит компонент.

Слияние объектов состояния

Так как состояние в компоненте на базе класса представляет собой один большой объект, который может иметь десятки значений состояния, очень неудобно их все помнить и задавать их значения.

Представьте, что для сброса счетчика используется следующий фрагмент:

```
this.setState({ counter: 0 });
```

Если в том же компоненте хранятся другие значения состояния, такой способ мог бы привести к сбросу или даже удалению остальных значений состояния, потому что они не включены в объект, передаваемый `setState`. Очень неприятно. В таком случае пришлось бы каждый раз копировать все существующие значения в новый объект — примерно так:

```
this.setState(  
  oldState => ({ ...oldState, counter: 0 })  
);
```

К счастью, делать это самостоятельно не нужно — это автоматически выполняет React для всех компонентов на базе классов. Когда вы передаете новый объект (или функцию обновления, которая возвращает объект) методу `setState`, React производит автоматическое слияние нового объекта с существующим объектом состояния. При этом выполняется именно та операция, которая показана в приведенном выше фрагменте, так что вам не нужно помнить о необходимости каждый раз выполнять ее вручную.

5.4. ВОПРОСЫ

1. Какие из следующих данных вы бы хранили в состоянии компонента?
 - a) Динамические данные приложения.
 - b) Свойства компонента.
 - c) Значения констант.
2. Какой из следующих фрагментов представляет правильный способ инициализации простого числового состояния в функциональном компоненте?
 - a) `const { value, setter } = useState(0);`
 - b) `const [ value, setter ] = useState(0);`
 - c) `const { value, setter } = useState({ value: 0 });`
 - d) `const [ value, setter ] = useState({ value: 0 });`

3. В каждом функциональном компоненте может определяться только один хук `useState`. *Да* или *нет*?
4. При обновлении значения состояния компонента через сеттер `useState` всегда выполняется повторный рендеринг компонента. *Да* или *нет*?
5. Какой из следующих фрагментов вы бы использовали для чтения одного числового значения из состояния в компоненте на базе класса?
 - a) `<p>Value: {this.state}.</p>B) <p>Value: {this.counter}.</p>`
 - b) `<p>Value: {this.state.counter}.</p>`
 - c) `<p>Value: {this.state.counter}.</p>`

ОТВЕТЫ

1. В состоянии определено `следует` хранить динамические данные приложения, но только не свойства (они уже хранятся в объекте свойств). Также в состоянии не `следует` хранить неизменяемые значения.
2. `const [ value, setter ] = useState(0);`. Исходное значение предоставляется `useState` в виде простого значения, а возвращаемое значение дефакто ирируется как массив, а не как объект.
3. *Нет*. В каждом компоненте можно определять столько хуков `useState`, сколько нужно.
4. *Нет*. Компонент рендерится только в том случае, если новое значение, передаваемое сеттеру, отличается от существующего. Для проверки используется равенство `ссылки`, так что даже если содержимое объекта обновилось, а сам объект остался прежним, повторный рендеринг не выполняется.
5. `<p>Value: {this.state.counter}.</p>`. Вспомните, что состояние в компоненте на базе класса всегда является объектом, а значения состояний представляются свойствами этого объекта.

ИТОГИ

- Состояние компонента обеспечивает интерактивность приложения. Без компонентов с состоянием в разработке далеко не уйти.
- Состояние может поддерживаться как в компонентах на базе классов, так и в функциональных компонентах.
- Состояние в функциональных компонентах инициализируется отдельными вызовами `useState` с отдельными сеттерами для каждого значения состояния.

- Значение хука `useState` можно инициализировать передачей статического значения, динамического значения или даже функции, возвращающей исходное значение.
- Значение хука `useState` можно обновить в любой момент, но только в обратном вызове или в другом хуке и никогда — напрямую в определении компонента.
- При обновлении значения хука `useState` можно либо задать новое значение напрямую, либо передать функцию, которая возвращает новое значение на основании старого значения.
- Состояние в компонентах на базе классов инициализируется как один объект и обновляется методом `setState`.
- Преобразование компонента на базе классов с состоянием в функциональный компонент с состоянием может потребовать серьезного рефакторинга, так как эти два метода значительно различаются.

Эффекты и жизненный цикл компонентов React

В ЭТОЙ ГЛАВЕ

- ✓ Выполнение эффектов внутри компонентов
- ✓ Полное описание жизненного цикла компонентов React
- ✓ Монтирование, демонтирование и рендеринг компонентов
- ✓ Методы жизненного цикла для компонентов на базе классов

Компоненты React используют JavaScript XML (JSX) для передачи информации пользователю в форме HTML. Однако чтобы компонент приносил пользу в приложении, он должен делать намного больше. Все, что происходит в React, происходит в том или ином компоненте, так что если приложению требуется назначить cookie, загрузить данные, обработать ввод на форме, вывести изображение с камеры пользователя, запустить или остановить таймер или использовать другие динамические возможности, одного JSX будет недостаточно.

Если компонент должен загружать данные с сервера, эффект необходимо запускать сразу же после загрузки компонента; но при повторном рендеринге компонента он уже запускаться не должен. С другой стороны, если требуется записать cookie с фамилией пользователя, введенной в поле формы входа, этот эффект должен инициализироваться каждый раз, когда пользователь что-нибудь

вводит в поле. А при выводе таймера он должен запускаться сразу же после загрузки компонента, но останавливаться при последующей выгрузке компонента, чтобы не расходовать ресурсы.

Во всех этих случаях применяются *эффекты* (effects) — функции, выполняемые в компоненте при определенных условиях. Чтобы запустить эффект, необходимо указать условия его выполнения. Но чтобы вы полностью поняли, как это делать, потребуется изучить тему жизненного цикла компонентов React.

Дадим формальное определение терминов, с которыми мы уже встречались, но которые до сих пор не определяли, — это монтирование, демонтирование и повторный рендеринг. Последний особенно важен. Когда и почему компоненты рендерятся повторно и как подключиться к этому процессу, чтобы управлять им или реагировать на него?

Наконец, кратко опишем работу методов жизненного цикла в компонентах на базе классов и сравним ее с работой в функциональных компонентах. В этой части различия между двумя типами компонентов проявляются еще ярче, чем мы видели до сих пор. Методы жизненного цикла компонента на базе класса очень сложны и запутанны по сравнению с простотой эффектов в функциональных компонентах. Нам предстоит многое узнать, так что за дело!

ПРИМЕЧАНИЕ Исходный код примеров этой главы доступен по адресу <https://rq2e.com/ch06>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке одной командой.

6.1. ВЫПОЛНЕНИЕ ЭФФЕКТОВ В КОМПОНЕНТАХ

Предположим, у вас есть компонент-таймер и вы хотите вывести количество секунд, прошедших с момента его монтирования. Первое, что приходит в голову, — создать интервал с помощью вызова `setInterval` в теле функции, который увеличивает значение состояния счетчика каждую секунду. Но при изменении значения состояния весь компонент рендерится заново, из-за чего будет создан другой интервал, так что компонент будет рендериться два раза в секунду; так будут созданы еще два интервала, рендерящих его четыре раза в секунду, и т. д. Очевидно, это решение не работает.

Другая возможная идея — установить тайм-аут вызовом `setTimeout`. В этом случае через 1 секунду после рендеринга компонента мы увеличиваем значение состояния счетчика, что, в свою очередь, станет причиной повторного рендеринга

и создания нового тайм-аута. Решение выглядит логично. Но что, если компонент будет отрендерен повторно по другим причинам? Повторный рендеринг компонента может быть инициирован изменением свойства или существованием нескольких значений состояния, которые могут изменяться независимо от счетчика. Если компонент демонтируется, потому что он уже не нужен, тайм-аут продолжит работать, и через секунду будет сделана попытка обновить несуществующий компонент. К сожалению, этот способ тоже не подходит.

Для решения этой проблемы React вводит *хук эффектов* с именем `useEffect` (обратите внимание на важный префикс `use*`, используемый в именах всех хуков). Эффект в хуке `useEffect` инициируется при изменении любого значения в наборе зависимостей. Более того, при выполнении эффекта в `useEffect` можно задать функцию очистки, которая будет выполняться в одном из двух случаев: перед повторным срабатыванием эффекта или при демонтировании компонента. На рис. 6.1 показана логика выполнения.

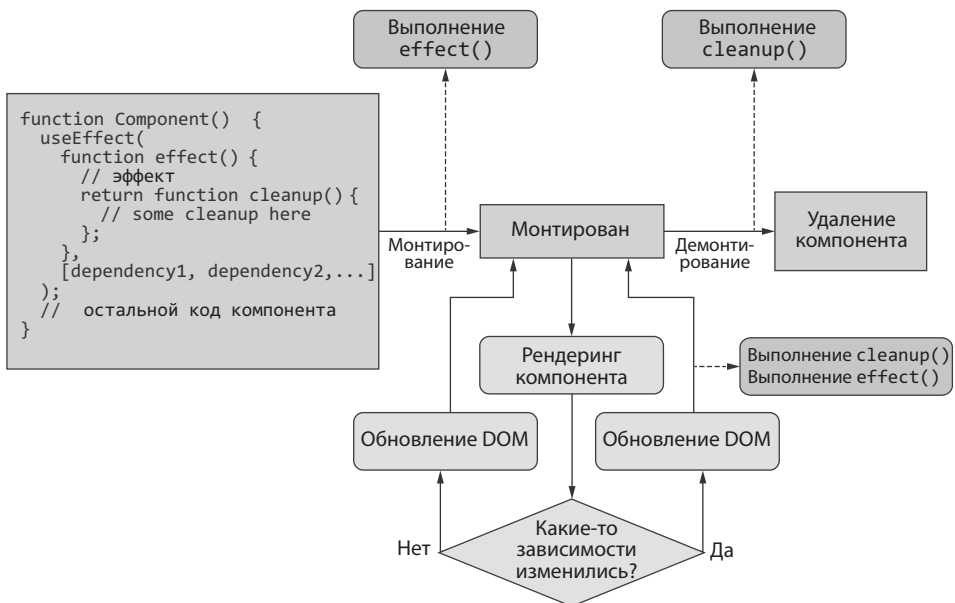


Рис. 6.1. Хук `useEffect` представлен как в виде кода, так и в виде блок-схемы. Хук содержит необязательный эффект и необязательную функцию очистки. Эффект выполняется при монтировании, а функция очистки — при демонтировании (если они определены, конечно). Более того, если эффект имеет массив зависимостей, очистка и эффект также выполняются при каждом изменении ссылки на значение в массиве зависимостей

Диаграмма получилась довольно сложной, поэтому мы разберем ее поэтапно и изучим функции, которые выполняют простые операции. Одна из хитростей заключается в том, что в вызове `useEffect` можно определить только эффект, только функцию очистки или и то и другое — как вам нужно. Более того, аккуратно заполнив массив зависимостей правильными значениями, можно инициировать выполнение эффекта и очистки в нужные моменты.

Существуют пять наиболее вероятных сценариев, в которых могут выполняться эффекты и функция очистки. Разберем их все и приведем примеры:

- В компоненте загружаются внешние данные. Чтобы правильно реализовать эту возможность в эффекте, он должен выполняться сразу же после монтирования компонента.
- Создается таймер с использованием интервала. Для этого такой эффект должен выполняться при монтировании компонента и удаляться при демонтировании.
- Требуется отслеживать момент закрытия диалогового окна независимо от того, как оно было закрыто. Чтобы правильно реализовать эту возможность, такой эффект должен выполняться только при демонтировании компонента.
- Требуется обновить заголовок окна (или вкладки) браузера названием текущей отображаемой страницы. Чтобы это делалось в эффекте, он должен выполняться при каждом изменении свойства заголовка, но не при изменении любых других свойств, если заголовок остается неизменным.
- Требуется запустить таймер, но только в том случае, если он активен, что обозначается флагом `isActive`. Для этого эффект и его очистка должны выполняться при каждом изменении флага `isActive`, но не при изменении других свойств или значений, если флаг `isActive` сохраняет прежнее значение.

6.1.1. Выполнение эффекта при монтировании

Предположим, вы создаете компонент раскрывающегося списка, который загружает выводимые данные с внешнего сервера. Загрузка данных должна совершаться как эффект, выполняемый при монтировании, и никогда не должна происходить повторно (потому что данные уже загружены). В таком сценарии актуальна только часть диаграммы, выделенная на рис. 6.2.

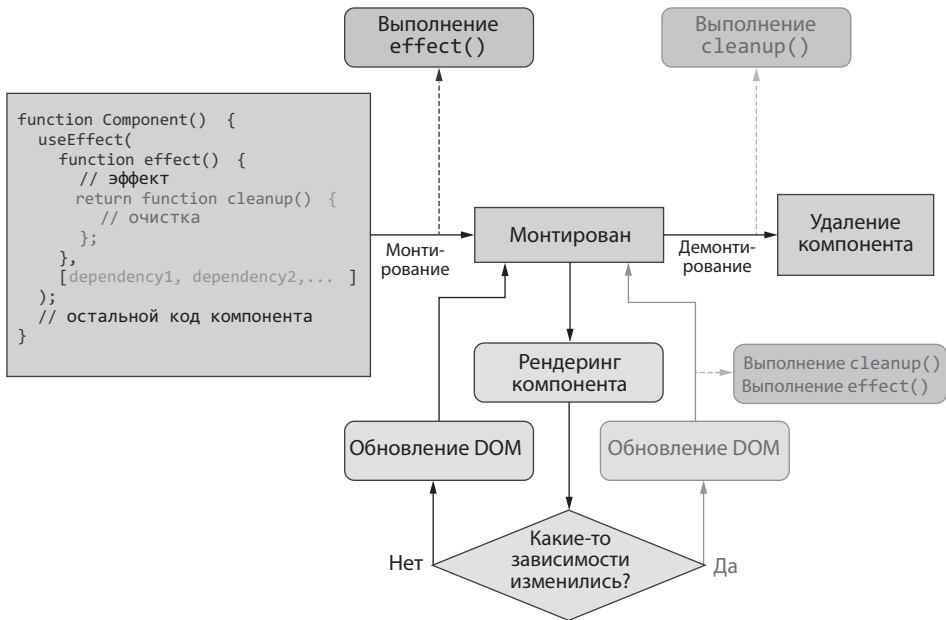


Рис. 6.2. Выполнение хука эффекта только при монтировании компонента. Обратите внимание: массив зависимостей остается пустым, а функция очистки не определяется. Это означает, что эффект будет выполняться только при монтировании компонента и никогда — при его повторном рендеринге

Код приведен в листинге 6.1, а результат его выполнения показан на рис. 6.3.

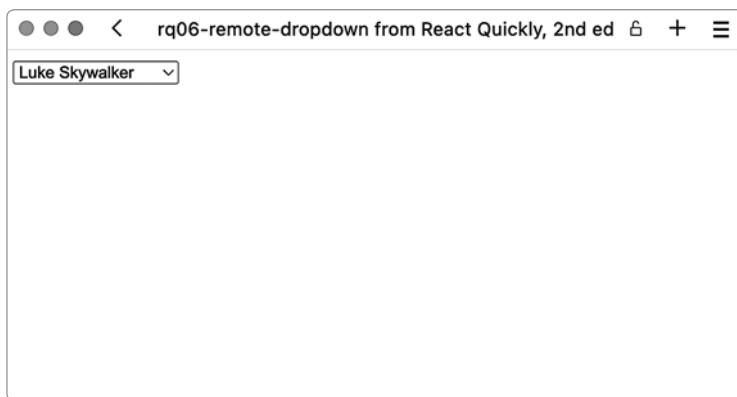


Рис. 6.3. Список персонажей «Звездных войн» в действии. Да пребудет с тобой источник данных!

Листинг 6.1. Загрузка данных раскрывающегося списка с удаленного сервера

```
import { useState, useEffect } from "react";
function RemoteDropdown() {
  const [options, setOptions] = useState([]);
  useEffect(() => {
    fetch("/ /www.swapi.tech/api/people")
      .then((res) => res.json())
      .then((data) => data.results)
      .then((characters) => characters.map(({ name }) => name))
      .then((names) => setOptions(names));
  }, []);
  return (
    <select>
      {options.map((option) => (
        <option key={option}>{option}</option>
      ))}
    </select>
  );
}
function App() {
  return <RemoteDropdown />;
}
export default App;
```

Состояние необходимо для хранения значений после загрузки вариантов

В хуке эффекта загружаются данные по указанному URL (список персонажей «Звездных войн»)

После парсинга результата значению состояния присваивается массив имен персонажей

Наконец, мы передаем пустой массив зависимостей, чтобы эффект срабатывал только при монтировании и никогда больше

Репозиторий: rq06-remote-dropdown

Этот пример находится в репозитории `rq06-remote-dropdown`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq06-remote-dropdown --template
  rq06-remote-dropdown
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq06-remote-dropdown>

Это классическая схема, которая часто встречается, например, в веб-приложении, загружающем данные, которые актуальны только в небольшой его части. Однако в этом сценарии кроется небольшая проблема. Что, если по какой-то причине компонент демонтируется до получения ответа от сервера: например, подключение к интернету нестабильно или сервер перегружен? Проблема решается при помощи функции очистки. Об этом будет рассказано в следующем разделе.

6.1.2. Выполнение эффекта при монтировании и очистке при демонтировании

Вам поручено создать компонент-секундомер. Он должен сразу же при монтировании запускать интервал, который просто инкрементно увеличивается с течением времени, а если компонент будет демонтирован (например, пользователь его закрыл) — остановить работу. Для этого потребуется эффект, который выполняется при монтировании и запускает функцию очистки при демонтировании. В таком сценарии актуальна только часть диаграммы, выделенная на рис. 6.4.

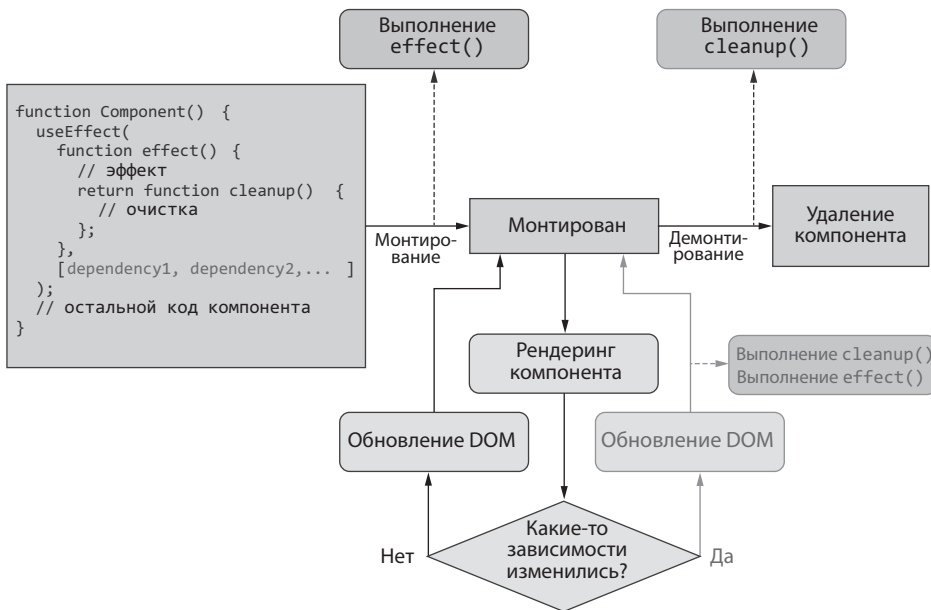


Рис. 6.4. Чтобы хук эффекта инициировал эффект и очистку только при монтировании и демонтировании компонента соответственно, необходимо добавить пустой массив зависимостей; благодаря этому эффект и очистка никогда не будут выполняться при повторном рендеринге компонента

Код приведен в листинге 6.2. На рис. 6.5 показан компонент в действии.

Листинг 6.2. Секундомер

```
import { useState, useEffect } from "react";
function Stopwatch() {
  const [seconds, setSeconds] = useState(0);
  useEffect(() => {
```

В функции эффекта запускается интервал, который должен каждую секунду увеличивать счетчик

```

const interval = setInterval(
  () => setSeconds((seconds) => seconds + 1),
  1000
);
return () => clearInterval(interval);
}, []);
return <h1>Seconds: {seconds}</h1>;
}
function App() {
  const [showWatch, setShowWatch] = useState(false);
  return (
    <>
      <button onClick={() => setShowWatch((b) => !b)}>Toggle watch</button>
      {showWatch && <Stopwatch />}
    </>
  );
}
export default App;

```

При помощи встроенной в браузер функции `setInterval` можно обеспечить вызов функции инкремента с постоянной частотой

Отменяет интервал в функции очистки с использованием встроенной функции `clearInterval`

Рендерит секундомер по условию, чтобы показать, что функция очистки выполняет свою работу

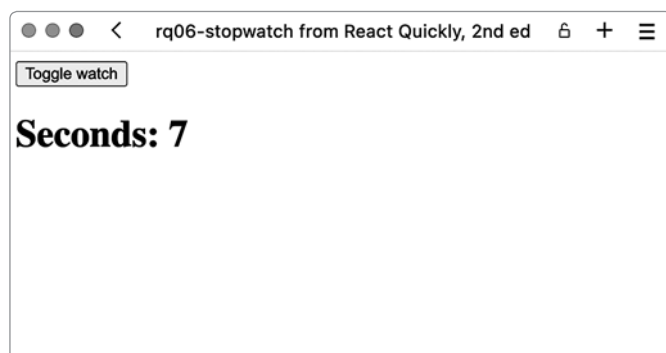


Рис. 6.5. Секундомер отсчитывает время

Репозиторий: rq06-stopwatch

Этот пример находится в репозитории `rq06-stopwatch`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq06-stopwatch --template rq06-stopwatch
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq06-stopwatch>

И хотя мы используем переменную `setSeconds` внутри эффекта, она не указывается как зависимость — это стабильная переменная, которая не изменяется. Функция обновления состояния, возвращаемая хуком `useState`, всегда остается одной и той же функцией по ссылке. Ее можно включить в массив; работа хука не изменится. Если эта часть покажется вам слишком сложной, просто включите функцию в массив.

События

Другой типичный пример использования эффекта только для монтирования и демонтирования — прослушивание событий. Например, компонент может обновляться только при изменении размера всей веб-страницы (для чего он прослушивает событие изменения размера) или при прокрутке определенного элемента (прослушивание события прокрутки). Примеры будут приведены в главе 8, посвященной событиям.

Отмена действия при демонтировании

Третий вариант сценария использования для монтирования и демонтирования расширяет пример из предыдущего раздела. Компонент `RemoteDropdown` загружает данные при монтировании, но что произойдет, если данные передаются недостаточно быстро и пользователь уже вышел из раздела приложения с раскрывающимся списком, не дождав ответа? Произойдет попытка обновить состояние компонента, который больше не существует!

Избежать этого можно двумя способами: либо отменить запрос в функции очистки (в JavaScript через `AbortController`), либо создать локальный флаг, который запоминает, находится ли компонент в процессе монтирования, и обновляет состояние компонента только в том случае, если флаг содержит `true`. В противном случае компонент просто игнорирует возвращенный ответ.

Отмена запроса с использованием `AbortController` при демонтировании реализуется примерно так:

```
useEffect(() => {
  const controller = new AbortController();
  fetch(url, { controller })
    .then(data => {
      // Обработка данных
    });
  return () => {
    controller.abort();
  };
}, []);
```

Создает контроллер отмены внутри эффекта

Не забудьте передать контроллер отмены в функцию fetch

В функции очистки мы запускаем контроллер отмены. Если запрос уже прошел, то при попытке отмены ничего не произойдет

Первый вариант отмены предпочтительнее, так как он позволяет просто отменить запрос, тем не менее это не всегда возможно. Если запрос по какой-то причине отменить нельзя, то чтобы следить за тем, монтирован ли компонент, можно использовать следующий код:

```
useEffect(() => {
  let mounted = true;
  fetch(url)
    .then(data => {
      if (!mounted) {
        return;
      }
      // Обработка данных
    });
  return () => {
    mounted = false;
  };
}, []);
```

Хранит в функции эффекта локальную переменную, которая инициализируется значением true и отражает информацию о том, что, по нашим данным, компонент монтирован

При поступлении данных мы сначала проверяем, что компонент все еще монтирован. Если нет, просто выполняется отмена запроса

Флаг переключается в функцию очистки, которая будет вызываться только при демонтировании компонента

Такое решение работает для любого типа отложенного обратного вызова, выполняемого в хуке эффекта. Это может быть обрабатываемое обещание (promise), сработавший тайм-аут или что-то подобное. Внутри эффекта локальной переменной присваивается false при демонтировании компонента, а далее остается лишь отменить обратный вызов при срабатывании.

6.1.3. Выполнение очистки при демонтировании

Представьте, что вы работаете над большим приложением с компонентом диалогового окна. В этом компоненте выводятся сигналы для пользователя. Диалоговое окно можно закрыть разными способами: нажатием кнопки x в углу, нажатием Escape на клавиатуре, кнопкой ОК в нижней части окна и т. д. Ваша задача — обеспечить вызов аналитической функции при закрытии диалогового окна. Можно вручную добавить маленький фрагмент кода для всех разных способов закрытия диалогового окна, но мы знаем, что вместо этого можно выполнить эффект при демонтировании компонента. В таком сценарии актуальна только часть диаграммы, выделенная на рис. 6.6.

Реализация может быть следующей:

```
function Dialog() {
  useEffect(
    () => () => trackEvent('dialog_dismissed'),
    [],
  );
  // Остальной код компонента
};
```

Двойная стрелочная запись необходима, так как функция эффекта должна возвращать функцию при выполнении

Учтите, что это неполный пример. Он предполагает, что диалоговое окно — это часть приложения со значительно большей функциональностью.

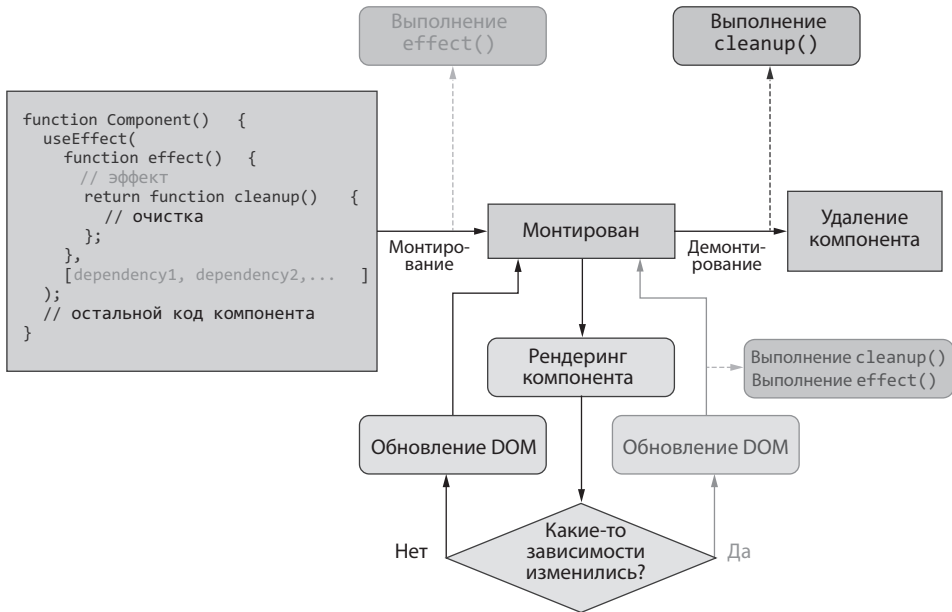


Рис. 6.6. Если важна только функция очистки, то задавать код эффекта не нужно; достаточно вернуть функцию из функции эффекта. С пустым массивом зависимостей этот код никогда не будет выполняться при повторном рендеринге компонента

Другой пример, в котором — какое совпадение! — также встречается диалоговое окно, — это управление фокусом. Когда вы используете клавиатуру для перехода к кнопке и нажимаете Enter, чтобы открыть диалоговое окно, то при следующем закрытии диалогового окна фокус ввода должен вернуться *к той же кнопке*, чтобы можно было продолжить переход к другим кнопкам интерфейса. При открытии диалогового окна фокус ввода должен перейти внутрь этого окна, но при его демонтаже он должен вернуться к элементу, которому он принадлежал перед открытием диалогового окна. Это можно сделать при помощи хука `useEffect`, для которого определена только функция очистки.

Кто-то скажет, что оба приведенных примера несколько надуманны или по крайней мере узкоспецифичны. Дело в том, что логика использования хука `useHook` только для функции очистки при демонтаже немного необычна и довольно редко встречается в реальных компонентах.

В намного более типичном сценарии функция очистки используется именно для той цели, на которую указывает ее название: очистки после вызова `useEffect`, который оставляет некоторую функциональность после демонтирования, чтобы избежать захвата ресурсов или утечки памяти в приложении. Подобный пример был приведен в предыдущем подразделе, и еще больше примеров мы разберем позже.

6.1.4. Выполнение эффекта при некоторых рендерингах

Согласитесь, хорошо, если бы заголовок вкладки в браузере автоматически обновлялся, когда пользователь переходит по блогу? Весь сайт блога создан на React, и в нем присутствует компонент, который динамически отображает любой пост в блоге. Заголовок документа изменяется в эффекте, который должен выполняться при каждом изменении заголовка блога и не должен — при изменении любых других свойств. В таком сценарии актуальна только часть диаграммы, выделенная на рис. 6.7. Реализация компонента приведена в листинге 6.3.

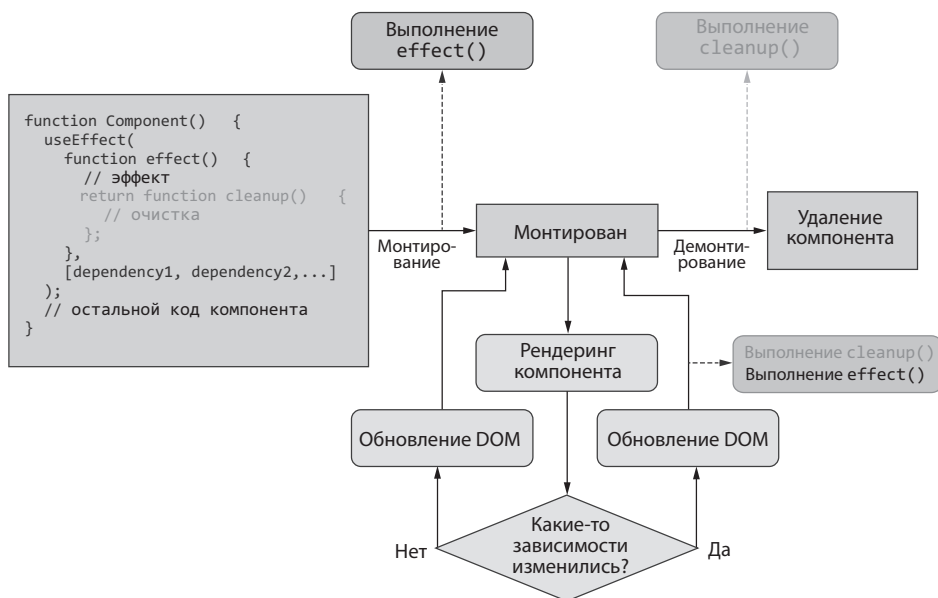


Рис. 6.7. На этот раз используется массив зависимостей. Эффект должен выполняться при монтировании и при каждом изменении определенного свойства. Однако он не должен выполняться при изменении других свойств, поэтому в массив зависимостей следует включить только соответствующие переменные

Листинг 6.3. Выполнение побочных эффектов в хуке

```
import { useEffect } from "react";
function BlogPost({ title, body }) {
  useEffect(() => {
    document.title = title;
  }, [title]);
  return (
    <article>
      <h1>{title}</h1>
      {body}
    </article>
  );
}
function App() {
  return (
    <main>
      <BlogPost title="First post" body={
        <p>Welcome to my cool website.</p>
      } />
    </main>
  );
}
export default App;
```

Эффект из `useEffect` назначает заголовок документа значение свойства `title`

Включение в массив зависимостей только `title` гарантирует, что заголовок документа обновится только при обновлении заголовка сообщения

Репозиторий: rq06-blog-title

Этот пример находится в репозитории `rq06-blog-title`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq06-blog-title --template rq06-blog-title
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq06-blog-title>

Это классический пример целей, для которых предназначен хук `useEffect`, — то есть выполнения побочных эффектов компонента. Заголовок документа невозможно обновить через `DOM`, поэтому его обновление должно быть побочным эффектом; в таких ситуациях `useEffect` — лучший выбор.

Обновление на основании свойства

Еще один типичный сценарий — обновление значения состояния на основании свойства. Возможно, вы помните из предыдущей главы, что при инициализации

состояния в `useState` значением свойства присваивание происходит только при первом рендеринге компонента после монтирования. Если компонент повторно рендерится с новым значением свойства, состояние уже не будет автоматически обновляться этим значением.

Проблему можно решить при помощи эффекта, который зависит от свойства и обновляет значение состояния в зависимости от этого свойства. Создадим очень простой компонент для ввода адреса электронной почты. Реализуем в нем возможность предварительного заполнения компонента адресом, полученным от родительского компонента, с использованием свойства.

Листинг 6.4. Обновление состояния на основании свойства

```
import { useEffect, useState } from "react";
function EmailInput({ value }) {
  const [email, setEmail] = useState("");
  useEffect(() => setEmail(value), [value]);
  return (
    <label>
      Email address:
      <input
        type="email"
        value={email}
        onChange={(evt) => setEmail(evt.target.value)}
      />
    </label>
  );
}
const EMAIL1 = "daffyduck@looneytunes.invalid";
const EMAIL2 = "bugsunny@looneytunes.invalid";
const EMAIL3 = "elmerfudd@looneytunes.invalid";
function App() {
  const [defaultEmail, setDefaultEmail] = useState(EMAIL1);
  return (
    <main>
      <button onClick={() => setDefaultEmail(EMAIL1)}>Use {EMAIL1}</button>
      <br />
      <button onClick={() => setDefaultEmail(EMAIL2)}>Use {EMAIL2}</button>
      <br />
      <button onClick={() => setDefaultEmail(EMAIL3)}>Use {EMAIL3}</button>
      <br />
      <EmailInput value={defaultEmail} />
    </main>
  );
}
export default App;
```

Создаем новое значение состояния, но ничем его не инициализируем

Это объясняется тем, что при каждом рендеринге из-за изменения значения свойства значению состояния (заново) присваивается значение свойства. Не забудьте добавить массив зависимостей, который содержит только свойство value

Поле ввода в этом компоненте обновляется новым способом (об этом далее в главе 8)

Наконец, значение состояния обновляется при каждом изменении входных данных

Возможно, этот сценарий трудно понять, но он вполне типичен для компонентов с контролем ввода.

Репозиторий: rq06-email-input

Этот пример находится в репозитории `rq06-email-input`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq06-email-input --template rq06-email-input
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq06-email-input>

6.1.5. Выполнение эффекта и очистки при отдельных рендерингах

На этот раз вместо секундомера, ведущего отсчет с возрастанием значений, мы создадим компонент с обратным отсчетом, при котором значения убывают. Отсчет можно приостанавливать и возобновлять. Для этого по-прежнему необходимо запустить интервал в эффекте, но нужно останавливать и запускать этот интервал при каждой приостановке и возобновлении отсчета соответственно. Для этого необходимо создать (в функции очистки) эффект с зависимостью. В таком сценарии актуальна вся диаграмма, изображенная на рис. 6.8.

Обратный счетчик — типичный пример компонента, в котором очистка должна выполняться при демонтировании. Этот компонент отличается от компонента-секундомера, рассмотренного выше, в котором отсчет времени можно приостанавливать, запускать и останавливать при необходимости, без демонтирования и повторного монтирования компонента (единственный способ остановить компонент, рассмотренный выше).

Компонент с обратным счетчиком будет инициализироваться начальным временем счетчика, то есть **10** в данном примере. В нем также есть кнопка **Reset**, которая позволяет сбросить счетчик до исходного значения в любой точке. Кроме того, кнопка **Pause/Resume** переключает режимы отсчета и паузы счетчика. Наконец, обратный отсчет каждую секунду уменьшает счетчик и останавливает его, когда значение достигает 0. Чтобы счетчик нельзя было снова запустить с 0, кнопка **Pause/Resume** блокируется при завершении обратного отсчета.

Выглядит сложно — посмотрим на блок-схему состояния компонента на рис. 6.9.

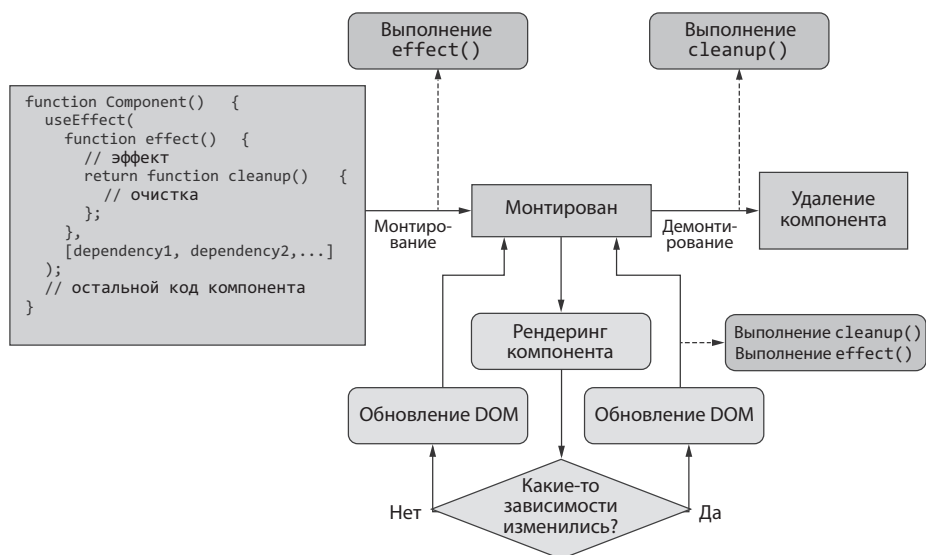


Рис. 6.8. Если задача — выполнять эффект и очистку при отдельных рендерингах, актуален весь эффект целиком

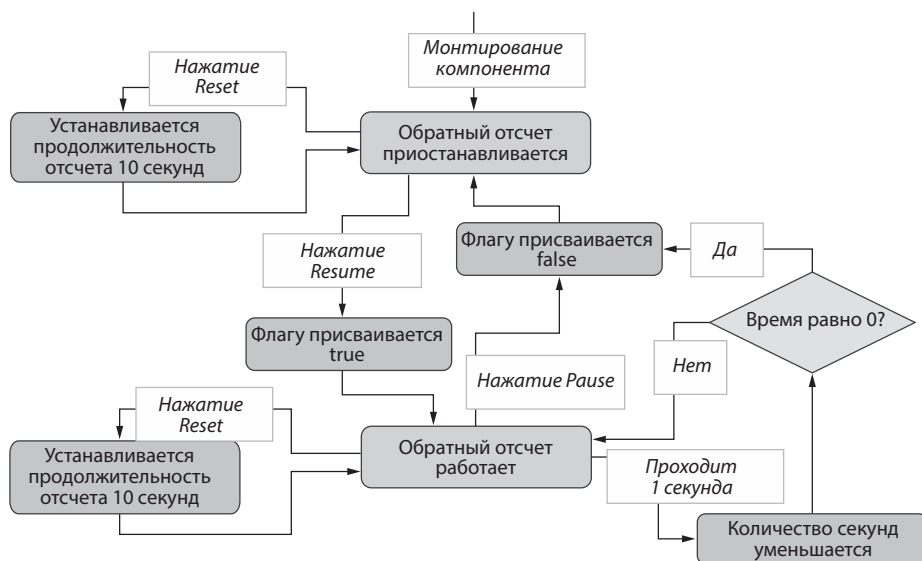


Рис. 6.9. Блок-схема: изменение во времени состояния компонента с обратным отсчетом при взаимодействии с пользователем. Обратите особое внимание, что кнопка Reset не останавливает и не запускает отсчет, а просто оставляет счетчик в текущем состоянии. Также обратите внимание на то, как отсчет останавливается по истечении времени

Реализация приведена в листинге 6.5. Результат показан на рис. 6.10.

Листинг 6.5. Интерактивный обратный отсчет

```
import { useEffect, useState } from "react";
function Countdown({ from }) {
  const [seconds, setSeconds] = useState(from);
  const [isRunning, setRunning] = useState(false);
  useEffect(() => {
    if (!isRunning) {
      return;
    }
    const interval = setInterval(
      () => {
        setSeconds((value) => {
          if (value <= 1) {
            setRunning(false);
          }
          return value - 1;
        }),
        1000
      );
    return () => clearInterval(interval);
  }, [isRunning]);
  return (
    <section>
      <h2>Time left: {seconds} seconds</h2>
      <button onClick={() => setSeconds(from)}>
        Reset
      </button>
      <button
        onClick={() => setRunning((v) => !v)}
        disabled={seconds === 0}
      >
        {isRunning ? "Pause" : "Resume"}
      </button>
    </section>
  );
}
function App() {
  return <Countdown from={10} />;
}
export default App;
```

Инициализирует seconds значением исходного свойства

Инициализирует флаг isRunning значением false

Прежде всего эффект проверяет, выполняется ли обратный отсчет, и если нет, незаметно отменяется (ничего не возвращая — очищать нечего)

Если обратный отсчет идет, определяем интервал, обновляющий значение состояния каждую секунду

При обновлении значения состояния проверяем, равно ли значение 1 (или меньше); если да, обратный отсчет необходимо остановить

Возвращает результат, на 1 меньший текущего значения счетчика

Предусматривает, что эффект вернет функцию очистки, которая полностью отменит интервал

В JSX компонента определяется кнопка, которая только сбрасывает счетчик (не изменяя значение флага отсчета)

Другая кнопка переключает значение флага, но не изменяет счетчик. При достижении счетчиком нуля кнопка блокируется

Текст на переключателе изменяется в зависимости от текущего состояния флага отсчета

Эффект должен зависеть от значения состояния isRunning. При каждом изменении этого значения выполняется наш эффект (а непосредственно перед ним выполняется очистка последнего эффекта)

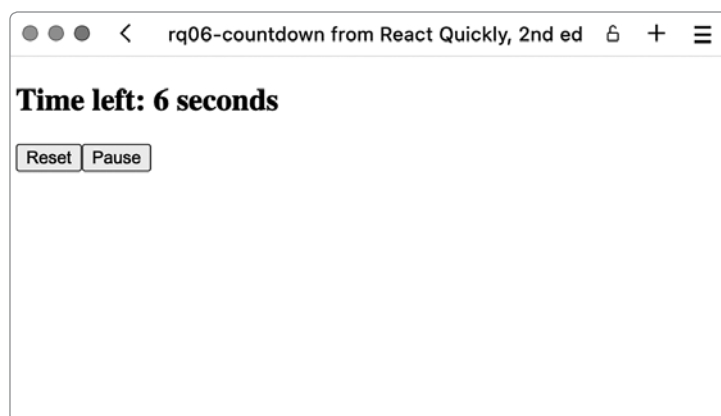


Рис. 6.10. Выполнение компонента с обратным отсчетом

Репозиторий: `rq06-countdown`

Этот пример находится в репозитории `rq06-countdown`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq06-countdown --template rq06-countdown
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq06-countdown>

В компоненте происходит довольно много всего, и мы используем тщательно подобранную комбинацию трех хуков, чтобы он работал правильно. В частности, можно заметить, что когда счетчик достигает нуля, мы не останавливаем интервал напрямую. В листинге 6.5 флаг отсчета переводится в состояние `false` вызовом `setRunning(false)`; . Это вызывает повторный рендеринг компонента, иницилируя повторное выполнение эффекта, потому что флаг `isRunning` указан в качестве зависимости для эффекта. При повторном выполнении эффекта функция очистки остановит интервал. Таким образом, перевод флага `isRunning` в состояние `false` косвенно приведет к остановке интервала, но только благодаря волшебству хука.

Компонент получился весьма нетривиальным, так что если вы не поймете его сразу, ничего страшного. Мы рекомендуем загрузить приведенный код

приложения и поэкспериментировать с ним. Попробуйте изменить его разные фрагменты и посмотрите, как устроено приложение и почему оно работает именно так, а не иначе.

6.1.6. Синхронное выполнение эффекта

А теперь поговорим о еще более гипотетической ситуации. Представьте, что вы создаете компонент, содержащий текст; требуется подсчитать общее количество букв и вывести это число. Текст полностью статичен, так что можно было бы подсчитать буквы вручную перед созданием компонента, но нужно быть уверенным, что компонент автоматически обновит счетчик букв, если в будущем текст изменится.

Одно из возможных решений — добавить в компонент значение состояния, содержащее счетчик букв, и инициализировать его нулем. Затем добавить в компонент эффект, который выполняется после рендеринга компонента; он подсчитывает все буквы и обновляет состояние. При повторном рендеринге компонента будет выведено правильное количество букв. Таким образом, логика хука эффекта объединяется с логикой хука состояния, как показано на рис. 6.11.

Проблема с логикой, описанной и представленной на рис. 6.11, заключается в том, что браузер обновляет пользовательский интерфейс и отображает его для пользователя до выполнения хука эффекта. Это значит, что пользователь на короткое время будет видеть, что компонент рендерит 0, прежде чем произойдет повторный рендеринг и будет выведено верное количество букв.

А что, если вместо этого выполнить эффект после того, как React сгенерирует необходимую разметку HTML, но до того, как браузер обновит UI и представит его пользователю? Удивительно, но... это можно сделать. Можно выполнить *хук эффекта макета* (layout effect hook), который делает две вещи не так, как обычный хук эффекта. Во-первых, он выполняется до того, как браузер обновит UI, и во-вторых, что не менее важно, сразу после генерирования DOM. Если React обнаружит обновление состояния из эффекта макета, он сразу же запустит повторный рендеринг компонента с обновленным состоянием. Эта ситуация показана на рис. 6.12. Заменяя `useEffect` на `useLayoutEffect` в этом конкретном случае, можно избежать вывода неверного значения.

Заметим, что `useEffect` подходит почти во всех случаях, а `useLayoutEffect` — только в нескольких особых ситуациях. Всегда сначала применяйте `useEffect`, и только если он не подходит, возможно, `useLayoutEffect` окажется именно тем, что нужно.

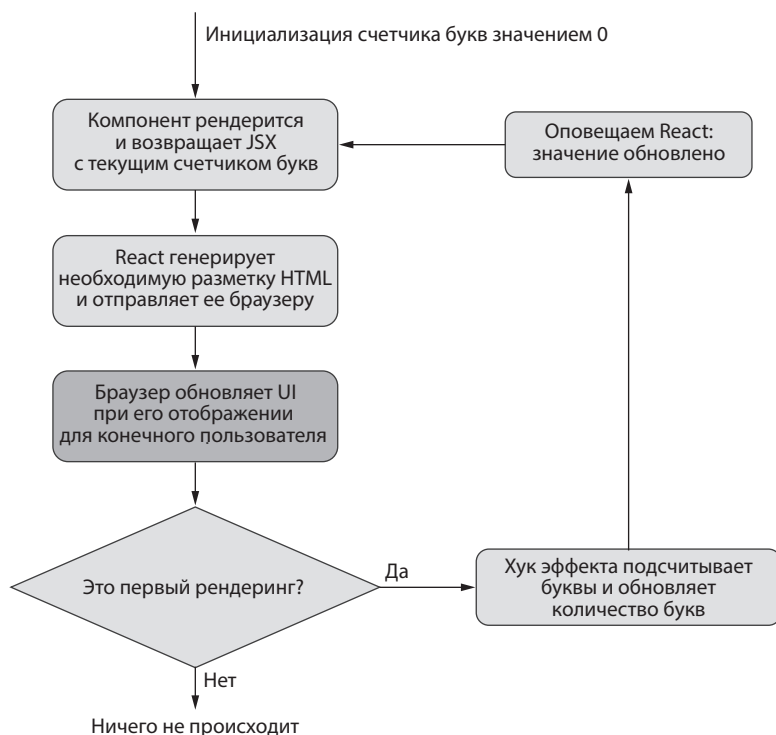


Рис. 6.11. Логика состояния при обновлении состояния и выполнении эффекта. Проблема возникает с темным блоком. При обновлении пользовательского интерфейса пользователь увидит начальное значение 0, прежде чем компонент быстро отрендерится заново и выведет верное количество букв

Технические детали эффекта макета

Хук `useLayoutEffect` можно считать разновидностью `useEffect`. Он идентичен `useEffect` во всех отношениях, кроме *момента* вызова. Как и `useEffect`, хук `useLayoutEffect` получает в аргументах функцию и массив зависимостей. При изменении любой зависимости выполняется функция очистки предыдущего эффекта (если она есть), после чего эффект выполняется для этого экземпляра с сохранением любой потенциальной возвращаемой функции очистки, срабатывающей в результате эффекта.

Различия между `useEffect` и `useLayoutEffect` в основном технические, но в целом сводятся ко времени вызова. `useLayoutEffect` вызывается синхронно в том же цикле выполнения, в котором компоненты рендерятся в DOM (но до того, как у браузера будет возможность *прорисовать* DOM в окне браузера). С другой

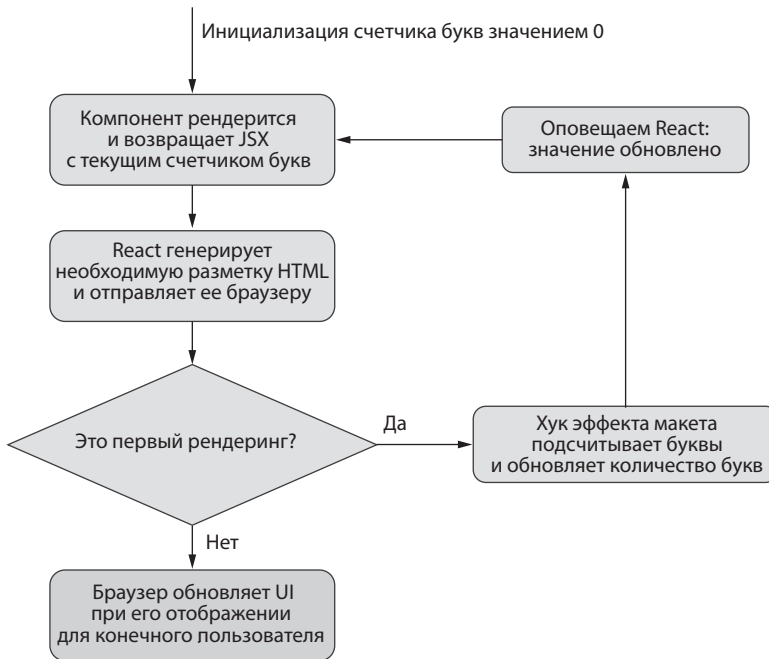


Рис. 6.12. Эффект макета будет выполняться до обновления UI в браузере, и новое состояние будет рендериться немедленно после обновления, поэтому UI сразу будет корректным

стороны, `useEffect` вызывается *асинхронно* в следующем цикле выполнения, в котором DOM прорисовывается в окне, а весь код CSS начинает работать и вычисляться. Диаграммы времени двух событий представлены на рис. 6.13.

Как видно из рис. 6.13, на предыдущих диаграммах выполнения `useEffect` были скрыты некоторые подробности. Обратите внимание: здесь `useEffect` и `useLayoutEffect` используются с одинаковыми зависимостями. Зависимости могут различаться, вследствие чего логика будет выполняться по-разному для разных рендеров: одни будут выполнять эффекты макетов и очистку, другие — обычные эффекты и очистку, а третьи могут выполнять и то и другое.

Одно из следствий синхронного выполнения эффекта макета после рендеринга заключается в том, что при выполнении сложных эффектов экран не будет обновляться, пока эффект не завершен. Пользовательский интерфейс фактически блокируется на время выполнения эффекта макета. По этой причине писать эффекты макетов следует крайне осторожно, чтобы они занимали как можно меньше тактов процессора.

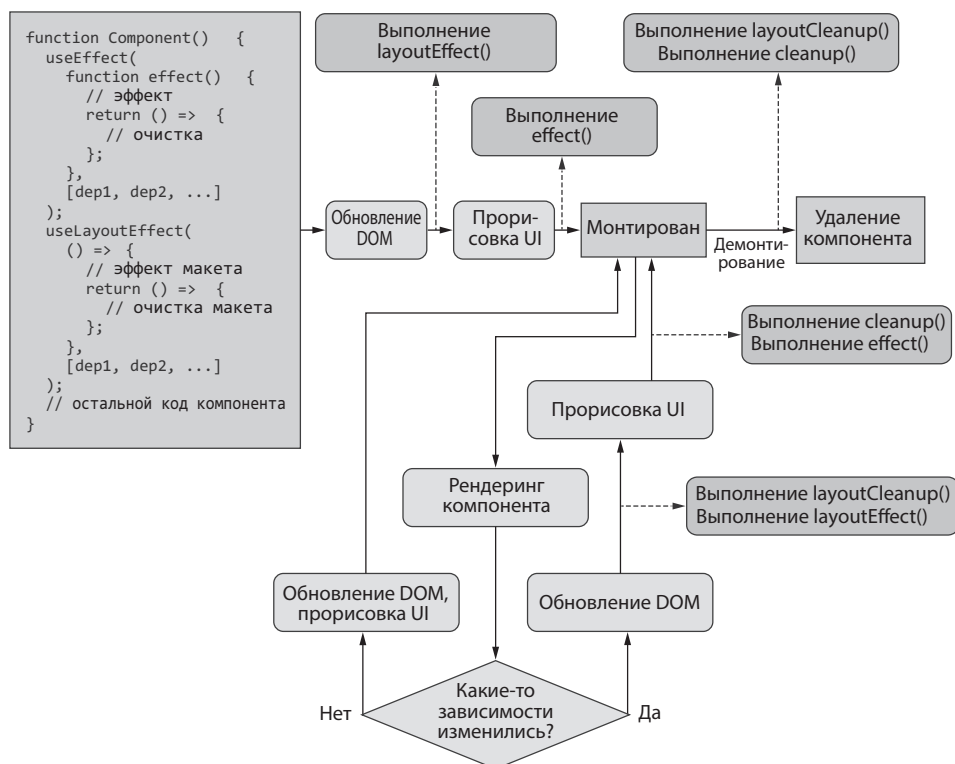


Рис. 6.13. Диаграмма времени выполнения `useLayoutEffect` и `useEffect`. Обратите внимание на то, что эффект макета выполняется сразу же после обновления DOM, но до того, как у браузера будет возможность разместить элементы с использованием CSS. (Простите, что на диаграмме слишком много всего!)

Если вы не до конца поняли, чем обычные эффекты отличаются от эффектов макетов, не переживайте. В 99 % случаев следует использовать хук `useEffect`. Только в очень редких ситуациях, когда требуется обновить DOM в эффекте перед его прорисовкой в окне, но после завершения рендеринга компонента, приходится использовать хук `useLayoutEffect`.

6.2. РЕНДЕРИНГ

В предыдущем разделе мы не раз говорили о повторном рендеринге компонентов. В этом разделе мы добавим больше технических подробностей, чтобы понять, что же означает (повторный) рендеринг компонента. Эти сведения не пригодятся

напрямую на практике, но они очень важны для понимания процессов, происходящих в приложении.

Рендеринг функционального компонента инициируется наступлением одного из трех событий:

- *монтирования* компонента (то есть раньше этого компонента не было в дереве, а теперь он появился);
- повторного рендеринга родительского компонента;
- обновления хуков с состоянием, которые использует компонент.

И это все. Если ничего из перечисленного не произошло, компонент повторно рендериться не будет — абсолютно точно. Если произошло *хотя бы* одно событие из этого списка, компонент *будет* отрендерен повторно, и это тоже абсолютно точно. Однако React может накапливать операции рендеринга и выполнять их после наступления нескольких из перечисленных событий, так что при изменении значения состояния и повторном рендеринге родительского компонента компонент может быть повторно отрендерен один раз или два. Этим управляет React в зависимости от неочевидных подробностей реализации. Мы подробно рассмотрим примеры каждого из этих сценариев, поговорим о том, как все происходит и что можно сделать.

Заметим, что речь идет о повторном рендеринге *всего* компонента. В компоненте могут присутствовать функции или обратные вызовы, которые рендерят некоторую часть вывода, и они могут рендериться повторно по миллиону разных причин. В частности, при использовании так называемых *рендер-пропсов* (render props), широко распространенных в старом коде и в разновидности React Context API, в которой не используются хуки. Рендер-пропсы встречаются и в современных и более сложных кодовых базах, так как они могут использоваться для частичного рендеринга контента в общем компоненте. Эта тема подробнее рассматривается в конце этого раздела.

6.2.1. Рендеринг при монтировании

Представьте, что у вас есть компонент, который загружает какие-то внешние данные. Можно воспользоваться примером раскрывающегося списка с удаленной загрузкой, описанным выше. При монтировании он загружает данные с удаленного сервера и сохраняет их локально в компоненте. При демонтировании данные теряются.

Рендеринг компонента при монтировании — самый простой и очевидный случай. Заметим, что если компонент включается в родительский компонент по

некоторому условию, он будет монтироваться и демонтироваться в зависимости от этого условия. Это не всегда то, что нужно.

Если знакомый нам компонент `RemoteDropdown` условно рендерится в родительском компоненте, что приводит к его многократной выгрузке и загрузке, внешние данные будут многократно загружаться и уничтожаться, что приведет к потере времени и неэффективному использованию канала. Хотя сетевое кэширование частично решает проблему, решить ее полностью можно двумя способами: либо перемещением хранения и выборки данных в компонент более высокого уровня, который всегда включается в приложение, либо изменением условного рендеринга компонента. Иногда этот подход встречается и в современных компонентах. Обычно условный рендеринг компонента выглядит примерно так:

```
return (
  <main>
    {hasDropdown && (<RemoteDropdown />)}
  </main>
);
```

Если логическое значение равно `true`, компонент монтируется. Если позже оно примет значение `false`, компонент демонтируется

Другой способ:

```
return (
  <main>
    <RemoteDropdown isVisible={hasDropdown} />
  </main>
);
```

Компонент монтируется всегда, а флаг просто переключается как свойство

Нужно внести изменения в компонент, чтобы флаг мог использоваться как индикатор необходимости рендеринга чего-либо:

```
function RemoteDropdown({ isVisible }) {
  const [options, setOptions] = useState([]);
  useEffect(() => {
    // Здесь происходит загрузка
  }, []);
  if (!isVisible) {
    return null;
  }
  // Остальной код компонента
};
```

Сначала добавляются все необходимые хуки

Только после обработки всех хуков можно проверить, нужно ли что-нибудь рендерить

Применять такой подход к условному рендерингу обычно не рекомендуется (в отличие от первого варианта). Тем не менее это решение может стать удобным, если вы не хотите, чтобы компонент монтировался и демонтировался снова и снова, — он будет оставаться в документе, но рендериться только в отдельных случаях.

6.2.2. Рендеринг при рендеринге родительского компонента

Не все, возможно, знают, но каждый дочерний компонент также рендерится при рендеринге его родительского компонента. Создадим простой пример со значком внутри кнопки:

```
function Icon() {
  return 
};
function Button() {
  const [enabled, setEnabled] = useState(false);
  const style = { border: `1px solid ${enabled ? "red" : "black"}` };
  return (
    <button style={style} onClick={() => setEnabled(b => !b)}>
      <Icon /> Toggle
    </button>
  );
}
```

Компонент значка предельно прост: он не изменяется и не обновляется ни при каких условиях

Компонент кнопки обладает внутренним состоянием и рендерится при каждом изменении состояния

Как вы думаете, что произойдет, если протестировать этот код в браузере? При каждом нажатии кнопки флаг `enabled` переключается и кнопка рендерится заново. А значок? Будет ли он рендериться повторно (а именно: будет ли функция с именем `Icon` выполняться повторно)? Да, будет. React не предполагает «чистоты» компонентов, потому что для этого нет никаких оснований. Поэтому React рендерит компонент каждый раз, когда рендерится его родитель. Если компонент получает свойства, React рендерит компонент каждый раз независимо от того, изменяются эти свойства или нет.

Представим другой сценарий и используем в нем это поведение. Смоделируем броски трех игровых кубиков. Код приведен в листинге 6.6, а результат его выполнения представлен на рис. 6.14.

Листинг 6.6. Моделирование бросков кубиков

```
import { useState } from "react";
function Die() {
  const style = {
    border: "2px solid black",
    display: "inline-block",
    width: "2em",
    height: "2em",
    textAlign: "center",
    lineHeight: 2,
  };
  const value = Math.floor(6 * Math.random());
```

Хотя компонент `Die` кажется чистым, в действительности он зависит от внешнего источника информации (`Math.random`) и (потенциально) возвращает новый результат при каждом рендеринге

```

    return <span style={style}>{value}</span>;
  }
}
function DiceRoller() {
  const [rolls, setRolls] = useState(1);
  return (
    <main>
      <h1>Rolls: {rolls}</h1>
      <button onClick={() => setRolls((r) => r + 1)}>
        Re-roll
      </button>
      <div>
        <Die />
        <Die />
        <Die />
      </div>
    </main>
  );
}
function App() {
  return <DiceRoller />;
}
export default App;

```

Компонент `DiceRoller` обладает состоянием

С нажатием кнопки увеличивается счетчик бросков, что инициирует полный рендеринг компонента. В результате будут отрендерены все дочерние компоненты, и мы получим новые результаты броска

Три разных экземпляра одного компонента `Die`, каждый из которых имеет собственный внутренний случайный источник данных

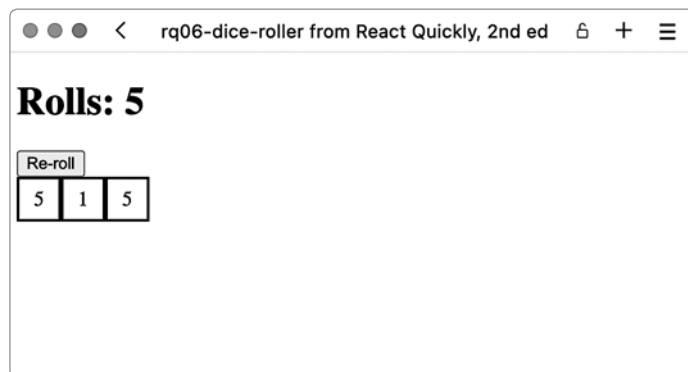


Рис. 6.14. Приложение после пяти бросков кубиков

Репозиторий: rq06-dice-roller

Этот пример находится в репозитории `rq06-dice-roller`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq06-dice-roller --template rq06-dice-roller
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq06-dice-roller>

На более общем уровне содержимое таких переменных (например, значение броска игрального кубика) следует хранить в состоянии компонента, чтобы не зависеть от его волшебного обновления при каждом рендеринге, как в приведенном примере. Пожалуйста, не делайте так. В рассмотренном примере представлен просто ужасный паттерн проектирования React. Например, мы не можем вывести сумму очков на кубиках в родительском компоненте, потому что ему неизвестны значения их дочерних компонентов.

Намного лучше использовать для родительского компонента другую структуру: сгенерировать три случайных числа и передавать их компонентам-кубикам в виде свойств. Но мы выбрали именно это решение, чтобы показать, что даже чистые (на первый взгляд) компоненты рендерятся при рендеринге их родительского компонента.

6.2.3. Рендеринг при обновлении состояния

При обновлении состояния внутри хука с состоянием (более подробно о хуках с состоянием см. в главе 7) происходит рендеринг компонента, использующего этот хук. Собственно, в этом заключается весь смысл обновления состояния, так что логика вполне очевидная и правильная.

Но если состояние содержит данные, которые часто обновляются, рендеринг может выполняться слишком часто. Лучше избегать постоянного рендеринга компонентов, поскольку он создает высокую нагрузку на процессор и/или память браузера, а еще раздражает пользователей.

Источником часто обновляемой информации может быть индикатор текущей позиции мыши. Пользователи могут перемещать мышь часто — много раз в секунду. Если вы используете несколько компонентов, хранящих позицию мыши в состоянии, у вас будет несколько компонентов, которые рендерятся много раз в секунду, что замедлит работу компьютера. Далее рассмотрим два разных примера с этой конфигурацией и возможности минимизации рендеринга при обновлениях состояния.

Хранение информации более высокого уровня

Представьте компонент, фон которого становится желтым, когда курсор находится в его левой половине, и синим — когда он находится в правой. Допустим,

ширина компонента — 200 пикселей, так что если курсор отстоит более чем на 100 пикселей от левого края, то считаем, что он находится в правой части; в противном случае он находится в левой части.

Первый вариант реализации — сохранять смещение курсора от левого края в значении состояния и при каждом рендеринге проверять, какой цвет фона должен отображаться:

```
function BlinkingBackground() {
  const [left, setLeft] = useState(0);
  const onMouseMove = (evt) =>
    setLeft(evt.nativeEvent.offsetX);
  const style = {
    backgroundColor: left < 100 ? "blue" : "red",
  };
  return <div style={style} onMouseMove={onMouseMove} />;
}
```

В состоянии хранится позиция мыши

Сохраняет позицию мыши в состоянии при перемещении мыши

Определяет, какой цвет должен использоваться, при каждом рендеринге компонента

Однако этот метод неэффективно запускает множество циклов рендеринга, потому что почти все время мышь будет перемещаться в одной половине компонента и компонент будет рендериться снова и снова для каждой ее позиции. Намного эффективнее просто хранить в состоянии признак того, где находится курсор — в левой или в правой половине. В таком случае компонент рендерится только при переходе курсора на другую половину:

```
function BlinkingBackground() {
  const [isLeft, setLeft] = useState(true);
  const onMouseMove = (evt) =>
    setLeft(evt.nativeEvent.offsetX < 100);
  const style = {
    backgroundColor: isLeft ? "blue" : "red"
  };
  return <div style={style} onMouseMove={onMouseMove} />;
}
```

В состоянии хранится только логический флаг

Сохраняет позицию мыши в состоянии при перемещении мыши

Определяет, какой цвет должен использоваться, при каждом рендеринге компонента

В этом примере используется тот факт, что React рендерит компонент при обновлении состояния только в том случае, если состояние действительно изменило значение. Сеттер вызывается с такой же частотой, как в предыдущем примере, но так как мы сохраняем только логическое значение, меняющееся с `true` на `false` лишь при прохождении курсора через центр компонента, большинство вызовов просто игнорируется, так как они не изменяют состояние компонента. В новом примере модель данных работает намного эффективнее, и теперь компонент рендерится только тогда, когда происходит событие, влияющее на вывод.

Прямые манипуляции с элементами DOM

В этом примере создается компонент, который перемещает элемент синхронно с перемещением курсора. Похоже, мы зашли в тупик. Как обновить стиль элемента в компоненте без сохранения его в состоянии компонента?

В таких случаях приходится действовать в обход React и напрямую обновлять DOM. Для этого понадобится ссылка на соответствующий элемент (для этого используется еще один хук, `useRef`, описанный в главе 7); при перемещении мыши стиль элемента обновляется напрямую:

```
function PhantomCursor() {
  const element = useRef();
  const onMouseMove = (evt) => {
    element.current.style.left =
      `${evt.nativeEvent.offsetX}px`;
    element.current.style.top =
      `${evt.nativeEvent.offsetY}px`;
  }
  return (
    <div style={{ position: "relative" }} onMouseMove={onMouseMove}>
      
    </div>
  );
}
```

Создает ссылку, указывающую на элемент DOM

Напрямую обновляет элемент DOM по ссылке при каждом перемещении мыши

Не забудьте установить ref на элемент, с которым будет выполняться операция

Компонент никогда не рендерится повторно. Он рендерится при монтировании, а затем остается как есть. Событие мыши изменяет внешний вид компонента, но уже не под контролем React. С точки зрения React это статический компонент, который вообще никогда не изменяется.

Проблема в том, что если вам понадобится использовать положение мыши для других задач приложения — вычислений, проверки коллизий и т. д., — ее все равно придется сохранить в состоянии. Однако и в этом случае следует по возможности стараться не обновлять состояние часто.

6.2.4. Рендеринг внутри функций

Компоненту не обязательно выполнять рендеринг внутри другого компонента; он может сделать это, например, внутри функции. В таком случае компонент будет рендериться при каждом выполнении функции. Иногда такая функция выполняется только при рендеринге родительского компонента, так что результат

остается неизменным; однако функция может выполняться и в другое время, и время рендеринга тоже будет разным.

Представьте компонент кнопки, в котором родитель задает значок для компонента. Кнопка может находиться либо в нажатом состоянии, либо в ненажатом. Иногда в этих состояниях должны использоваться разные значки. Можно заставить компонент получать два разных свойства для двух состояний или же передать функцию, которая получает состояние кнопки в аргументе и возвращает нужный значок.

Листинг 6.7. Кнопка с функцией получения значка

```
import { useState } from "react";
function Icon({ type }) {
  return <img src={`images/${type}.png`} width="16" alt="" />;
}
function Button({ label, getIcon }) {
  const [pressed, setPressed] = useState(false);
  return (
    <button onClick={() => setPressed((p) => !p)}>
      {getIcon(pressed)}
      {label}
    </button>
  );
}
function LockButton() {
  const getIcon = (pressed) =>
    pressed ? <Icon type="lock" /> : <Icon type="unlock" />;
  return <Button label="Lock" getIcon={getIcon} />;
}
function App() {
  return <LockButton />;
}
export default App;
```

Обобщенный компонент значка содержит изображение, загруженное из соответствующей папки

Кнопка вызывает функцию `getIcon` с текущим состоянием при каждом рендеринге

Функция `getIcon` определяется так, чтобы она возвращала один из двух значков

Репозиторий: rq06-push-button

Этот пример находится в репозитории `rq06-push-button`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq06-push-button --template rq06-push-button
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq06-push-button>

В этом решении компонент значка рендерится внутри функции, а не внутри компонента. Тем не менее функция вызывается только внутри компонента кнопки напрямую при рендеринге, поэтому все происходит так, как если бы значок был включен в рендеринг, просто это делается через функцию. Однако такой подход меняет некоторые представления о компонентах, и вам будет труднее оптимизировать этот код и даже разобраться, как он работает.

Того же результата можно добиться намного более привычным способом. Взгляните еще раз на функцию `getIcon`. Эта функция возвращает код JSX, зависящий от передаваемых аргументов. Звучит знакомо? Именно это делает функциональный компонент. А значит, схему можно немного изменить и преобразовать функцию `getIcon` в нестандартный компонент.

Листинг 6.8. Кнопка с компонентом значка

```
import { useState } from "react";
function Icon({ type }) {
  return <img src={` /images/${type}.png`} width="16" alt="" />;
}
function Button({ label, ButtonIcon }) {
  const [pressed, setPressed] = useState(false);
  return (
    <button onClick={() => setPressed((p) => !p)}>
      <ButtonIcon pressed={pressed} />
      {label}
    </button>
  );
}
function LockIcon({ pressed }) {
  return pressed ? <Icon type="lock" /> : <Icon type="unlock" />;
}
function LockButton() {
  return <Button label="Lock" ButtonIcon={LockIcon} />;
}
function App() {
  return <LockButton />;
}
export default App;
```

Компонент кнопки теперь рассчитывает получить свойство `ButtonIcon` (обратите внимание на прописную букву в начале) вместо функции `getIcon`, как в предыдущем варианте

Так как мы ожидаем получить компонент, его можно отрендерить как таковой прямо в теле

Теперь `getIcon` — не просто функция, а полноценный функциональный компонент (получающий свойства вместо простого аргумента)

Наконец, мы просто передаем `LockIcon` как свойство. Это вполне допустимо, хотя раньше мы этого не делали

Репозиторий: `rq06-push-button2`

Этот пример находится в репозитории `rq06-push-button2`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq06-push-button2 --template rq06-push-button2
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq06-push-button2>

Этот способ отлично работает — и выглядит намного чище! Конечно, оптимизацию можно продолжить (например, переместив тернарную условную конструкцию в свойство, которое изменяется внутри компонента `LockIcon`), но это выходит за рамки текущего примера.

Концепция передачи функций, рендерящих JSX, обозначается термином *рендер-пропсы* (render props). Она довольно часто применялась в прошлом. Однако при работе с функциональными компонентами почти всегда лучше преобразовать аргумент в полноценный компонент, как мы и делали. Это упрощает понимание логики передачи данных и подходит почти в 95 % случаев применения функций (то есть нефункциональных компонентов), рендерящих JSX.

React Context

Одна из немногих причин для рендеринга JSX в функциях — использование версии React Context API без хуков с компонентом `MyContext.Consumer`. Этот компонент получает функцию как дочерний компонент (концепция сама по себе необычная). Это довольно экзотический случай, и вы вряд ли встретитесь с ним в современном коде React с функциональными компонентами. Если же это случится, обратитесь к электронной документации React, чтобы узнать, как использовать React Context API. Или, еще лучше, по возможности преобразуйте компонент в функциональный и воспользуйтесь хуком `useContext` (о том, как использовать этот хук, рассказано в главах 7 и 10).

6.3. ЖИЗНЕННЫЙ ЦИКЛ КОМПОНЕНТА НА БАЗЕ КЛАССА

Когда компонент на базе класса монтируется, рендерится и демонтируется, вместо хуков для реагирования на разные стадии жизненного цикла компонента можно использовать методы. Имена методов описывают их предназначение и место в жизненном цикле, так что в большинстве случаев они достаточно понятны.

Некоторые методы жизненного цикла выполняются в нескольких событиях. Другие методы позволяют вмешаться в стандартное планирование обновлений компонентов React, если у вас есть внутренние данные, которых нет у React.

Прежде в React были и другие методы жизненного цикла, но в новых версиях React от них отказались из-за ненадежности их поведения.

По возможности всегда используйте функциональные компоненты, но на случай, если вы столкнетесь с компонентом на базе класса и захотите провести рефакторинг, чтобы преобразовать его в функциональный, приведем несколько общих рекомендаций. Учтите, что это не однозначное руководство к действию, и возможно, в вашем случае понадобится что-то переписать или переработать.

6.3.1. Методы жизненного цикла

При монтировании компонента вызываются следующие методы (в указанном порядке):

- `constructor()`
- `static getDerivedStateFromProps()`
- `render()`
- `componentDidMount()`

При обновлении компонента на базе класса (по любой из причин, перечисленных выше) вызываются следующие методы (в указанном порядке):

- `static getDerivedStateFromProps()`
- `shouldComponentUpdate()`
- `render()`
- `getSnapshotBeforeUpdate()`
- `componentDidUpdate()`

На самом деле все не совсем так. Метод `shouldComponentUpdate()` занимает особое место: если он определен, вы можете остановить цикл рендеринга возвратом `false`. На первый взгляд это кажется отличным способом сократить рендеринг, но реализовать это может быть очень непросто, а при неправильном использовании метода возможна рассинхронизация компонентов с их фактическим представлением DOM. При демонтировании компонента вызывается метод `componentDidUnmount()`.

6.3.2. Унаследованные методы жизненного цикла

Некоторые методы жизненного цикла, которые активно использовались в прошлом, все еще могут встретиться в старом коде. Они имели большую

популярность, но создавали массу проблем и были объявлены устаревшими. Эти методы теперь называются иначе, но все еще существуют в React, даже в React 18. Когда-нибудь они будут удалены и перестанут работать, но пока этого не случилось. Тем не менее их текущие имена предупреждают об их ненадежности всех желающих ими воспользоваться. *Раньше* эти методы назывались так:

- `componentWillMount()`
- `componentWillUpdate()`
- `componentWillReceiveProps()`

Теперь они называются так:

- `UNSAFE_componentWillMount()`
- `UNSAFE_componentWillUpdate()`
- `UNSAFE_componentWillReceiveProps()`

Префикс `UNSAFE` сигнализирует разработчикам, что этот метод лучше не использовать — или по крайней мере постараться избавиться от него как можно быстрее.

Мы не будем рассматривать функциональность этих методов, так как пользоваться ими категорически не рекомендуется. Если вы встретите их в коде, обращайтесь к электронной документации, чтобы воспроизвести необходимую функциональность без этих методов.

6.3.3. Преобразование методов жизненного цикла в хуки

Преобразовать компонент на базе класса может быть непросто. Вы уже видели, как решать некоторые проблемы, возникающие в ходе преобразования; с добавлением компонентов с состоянием задача только усложнилась. А с добавлением методов жизненного цикла появился риск впасть в отчаяние. Перечислим эти методы и опишем, как реализовать схожую функциональность с использованием хуков:

- `constructor()` — метод может быть реализован с использованием `useEffect()` без зависимостей либо для предварительного вычисления затратных значений — с использованием `useMemo()` без зависимостей.
- `getDerivedStateFromProps()` — может быть реализован с использованием хука `useEffect()` с соответствующими свойствами в качестве зависимостей.
- `render()` — весь функциональный компонент становится функцией рендеринга.

- `componentDidMount()` — метод в основном делает то же, что и хук `useEffect()`, но без зависимостей. Часто используется в сочетании с `componentDidUnmount()` — эквивалентной функции очистки для хука. Заметим, что для корректной работы `componentDidMount` выполняется синхронно, тогда как `useEffect` выполняется асинхронно, так что для достижения того же эффекта, возможно, придется использовать `useLayoutEffect`. Впрочем, в большинстве случаев с таким же успехом можно воспользоваться `useEffect`, потому что фактор синхронности редко имеет значение для этого метода жизненного цикла.
- `shouldComponentUpdate()` — у метода нет эквивалентного хука, но его также не обязательно применять, если вы используете хуки. Если вы хотите выполнять рендеринг функционального компонента как можно реже, используйте хуки мемоизации, о которых речь пойдет в следующей главе.
- `getSnapshotBeforeUpdate()` — экзотический метод, который редко применяется на практике. Он почти всегда используется для одной конкретной цели: сохранения позиции прокрутки некоторой части компонента *до* обновления компонента, чтобы можно было восстановить эту позицию *после* обновления компонента новыми данными. Это конкретное поведение можно эмулировать в функциональном компоненте, упаковав сеттер состояния в нестандартную функцию, которая сохраняет прежнюю позицию прокрутки в ссылке, прежде чем обновлять компонент и инициировать новый рендеринг.
- `componentDidUpdate()` — может эмулироваться хуком `useEffect` с зависимостями. В зависимостях задаются актуальные значения, которые изменились и вызвали изменившееся поведение, на которое нужно отреагировать.
- `componentDidUnmount()` — функциональность этого метода можно переместить в функцию очистки в хуке `useEffect` (или `useLayoutEffect`) без зависимостей. Часто используется для отмены подписок или интервалов, назначаемых при монтировании.

6.4. ВОПРОСЫ

1. В функциональных компонентах невозможно выполнять побочные эффекты, для этого подходят только компоненты на базе класса. *Да или нет?*
2. Когда можно выполнить эффект с использованием хука эффекта?
 - а) При *монтировании* компонента.
 - б) При *демонтировании* компонента.

- с) При *обновлении* компонента.
 - д) Во всех перечисленных случаях.
- 3. Если нужно, чтобы данные загружались в компоненте сразу же при его выводе, но затем не перезагружались даже при обновлении компонента, массив зависимостей должен:
 - а) пропускаться;
 - б) быть пустым;
 - с) содержать только URL данных.
- 4. При рендеринге родительского компонента дочерние компоненты будут повторно отрендерены только в том случае, если их свойства обновились. *Да или нет?*
- 5. Как выглядит правильный синтаксис хука эффекта, который выполняется при демонтировании компонента?
 - а) `useEffect(() => runOnUnmount(), [])`;
 - б) `useEffect(() => () => runOnUnmount(), [])`;
 - с) `useEffect(() => runOnUnmount())`;
 - д) `useEffect(() => () => runOnUnmount())`;

ОТВЕТЫ

1. *Нет*. При помощи хука `useEffect` (или `useLayoutEffect`) можно выполнять побочные эффекты и внутри функций анимационных компонентов.
2. Эффект можно выполнить для любого конкретного рендеринга компонента и даже при его демонтировании, так что все варианты верны.
3. Если нужно, чтобы эффект выполнялся только при демонтировании компонента, следует передать пустой массив зависимостей.
4. *Нет*. Каждый раз, когда компонент рендерится, рендерятся и все его дочерние компоненты независимо от того, изменились их свойства или нет.
5. `useEffect(() => () => runOnUnmount(), [])`. Эффект демонтирования (или оптимизация) должен выполняться функцией, так что двойная запись функций не требуется. Кроме того, массив зависимостей должен оставаться пустым, а не пропускаться.

ИТОГИ

- У каждого экземпляра компонента React существует собственный жизненный цикл.
- Хук `useEffect` чаще всего используется для выполнения побочных эффектов, актуальных для конкретного компонента, при его монтировании, рендеринге и демонтировании.
- Тщательно выбирая содержимое массива зависимостей, можно назначить точное время выполнения хука эффекта. Так можно создавать умные компоненты, взаимодействующие с браузером, сетью и пользователем множеством разных способов.
- Компоненты рендерятся в трех основных случаях, наступление которых определяет React: при монтировании компонента, при обновлении состояния компонента и при рендеринге родительского компонента.
- Компоненты на базе класса не могут использовать хуки. Они зависят от методов жизненного цикла для реализации похожего поведения. Эти методы можно преобразовать в хуки, но иногда это довольно сложно.



Хуки как основа веб-приложений

В ЭТОЙ ГЛАВЕ

- ✓ Подробнее о создании компонентов с состоянием
- ✓ Нетривиальные задачи, решаемые с использованием хуков
- ✓ Правила использования хуков

Хуки — это то, что движет современными приложениями React. Они образуют относительно небольшую, но очень важную часть React API. Кроме того, работать с хуками не так просто. В этой главе мы рассмотрим все хуки и то, что они делают, а также некоторые важные нюансы, которые необходимо знать об использовании хуков в целом.

Хуки занимают особую нишу в экосистеме React. На первый взгляд они совершенно не связаны по своей функциональности, но при более внимательном изучении выясняется, что у них есть общие черты и особенности поведения, которые необходимо учитывать при работе с ними. Можно сказать, что все они происходят от общего предка, хотя в результате эволюции перестали быть похожими друг на друга.

Именно по этой причине мы решили посвятить эту главу хукам. Таким образом, мы будем рассматривать разные темы, но так или иначе относящиеся

к использованию хуков. А напоследок объясним, как все эти хуки на самом деле связаны между собой, несмотря на то что предназначены для разных целей.

До сих пор вы изучили 3 хука: `useState` (в главе 5), `useEffect` и `useLayoutEffect` (в главе 6). На момент написания книги в React существует 15 встроенных хуков (в версии React 18), которые мы кратко рассмотрим, сгруппировав по функциональности:

- *Хуки состояния* — эти функции наделяют состоянием компоненты и приложения на нескольких уровнях сложности: `useState`, `useReducer`, `useRef`, `useContext`, `useDeferredValue` и `useTransition`.
- *Хуки эффектов* — эти функции относятся к выполнению эффектов внутри компонента на разных стадиях общего жизненного цикла компонента, а также при каждом отдельном цикле рендеринга: `useEffect` и `useLayoutEffect`.
- *Хуки мемоизации* — эти функции используются для оптимизации производительности. Они предотвращают повторное вычисление, если их составляющие не изменились. К этой категории относятся функции `useMemo`, `useCallback` и `useId`.
- *Библиотечные хуки* — нетривиальные функции, которые используются почти исключительно в больших библиотечных компонентах, предназначенных для распространения в сообществе или для внутреннего пользования в крупных организациях. Эти функции редко встречаются в приложениях меньшего и среднего размера: `useDebugValue`, `useImperativeHandle`, `useInsertionEffect` и `useSyncExternalStore`.

Эти 15 «базовых» хуков поставляются с React. Вы можете создавать новые хуки, но создать собственный базовый хук не получится. Можно создавать только хуки на основе уже существующих. Нестандартные хуки рассматриваются в главе 10.

В будущем в React могут появиться новые встроенные хуки. В React 18.0 были добавлены 5 новых хуков, а в следующих версиях после React 18 могут появиться и другие.

ПРИМЕЧАНИЕ Исходный код примеров этой главы доступен по адресу <https://rq2e.com/ch07>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке одной командой.

7.1. КОМПОНЕНТЫ С СОСТОЯНИЕМ

Компоненты с состоянием рассматривались в главе 5, но нелишним будет повторить. Без компонентов с состоянием — и, как следствие, приложений

с состоянием — не может обойтись ни одно сколько-нибудь интересное веб-приложение.

Приложение без состояния полностью статично. Такое приложение останется неизменным все время, пока оно открыто в браузере, и будет неизменным для каждого пользователя, который с ним работает. Если в приложении нужна система учетных данных пользователей, сеансы, интерактивность и изменяемость со временем, оно должно обладать состоянием.

Тем не менее не все компоненты с состоянием одинаковы — так же, как не одинаковы все состояния. Одно состояние хранится лишь короткое время, другое состояние полностью локально для конкретного компонента, третье действует в масштабе приложения. Кроме того, состояние может быть как одной переменной, так и огромным сложным переплетением независимых переменных, которые должны обновляться согласованно. В этом разделе рассматриваются некоторые нетипичные ситуации применения компонентов и приложений с состоянием, а также обсуждается, как решить поставленную задачу с помощью подходящего хука.

7.1.1. Простые значения состояния с `useState`

Хук `useState` — основа приложений с состоянием. Скорее всего, вы будете использовать именно этот хук, когда вам понадобится работать с состоянием, так что он однозначно важен.

Если у вас есть меню, которое может открываться и закрываться, его состояние хранится в локальном хуке `useState` внутри компонента меню. Это одно простое значение используется только внутри этого компонента и никак не связывается с другими значениями состояния в приложении.

`useState` достаточно подробно рассмотрен в главе 5, поэтому мы не будем здесь много о нем говорить. Но в оставшейся части раздела будут описаны более сложные сценарии, в которых одного `useState` окажется недостаточно или это будет неоптимально.

7.1.2. Создание сложного состояния с `useReducer`

Представьте, что у вас есть компонент с загрузкой данных и вы хотите узнать, успешно ли прошла загрузка, какое сообщение об ошибке получено в случае неудачи или какие данные получены в случае успеха. Значение сообщения об ошибке актуально только для неудачной загрузки. Если загрузка прошла успешно, то сообщение об ошибке неактуально, оно даже не должно задаваться — и наоборот для данных результата. Этот сценарий — пример *взаимозависимых*

состояний. Отдельные значения в состоянии зависят друг от друга, и часто сразу несколько значений должны обновляться одновременно.

`useReducer` — полезный хук, предназначенный именно для этой цели. Он представляет собой расширенную версию `useState`, позволяющую изменять состояние более сложным и управляемым способом (почти на уровне конечного автомата, но не совсем), если конфигурация слишком сложна, чтобы ее можно было нормально представить одним значением состояния.

`useReducer` позволяет генерировать новое состояние («преобразовывать») исключительно на основании текущего состояния и действия, получающего некоторые полезные данные. Концепция преобразования состояния известна из других фреймворков, таких как Redux (отсюда название), поэтому она уже знакома многим разработчикам React.

Обратите внимание, что хук `useReducer` никогда не является единственно возможным вариантом, — все, что можно сделать с его помощью, можно сделать и комбинацией более простых вызовов `useState`. Тем не менее во многих случаях эффективнее использовать преобразование, чем решение с несколькими отдельными состояниями, чтобы обеспечить более жесткую передачу данных и лучше контролировать процесс.

Некоторые примеры преобразования будут представлены в главе 10, когда мы перейдем к более сложным архитектурам приложений. Преобразование актуально только для относительно сложных потоков данных; оно очень редко встречается в простых приложениях, которые мы строим в этой книге.

7.1.3. Сохранение значения без повторного рендеринга с использованием `useRef`

Представьте, что вам нужно создать кнопку, которая работает только по двойному щелчку в пределах заданного числа миллисекунд. Для этого необходимо запомнить, сколько времени прошло между двумя последовательными щелчками. Сохранение данных в компоненте — именно та задача, для которой существует состояние. Имеется значение, которое должно сохраняться между рендерингами, но оно не используется для рендеринга. При первом щелчке кнопка никак не изменяется. Необходимо на какое-то время запомнить значение внутри компонента экземпляра, но это значение не будет использоваться для определения вывода компонента.

`useRef` — один из простейших и одновременно один из самых непонятых хуков в React. Это хук с *пассивным состоянием*; то есть он может содержать

состояние, но присваивание или обновление состояния не инициирует повторный рендеринг.

`useRef` используется для разных целей, включая сохранение значений между рендерингами, а также хранение ссылок на элементы DOM, задействованные в рендеринге. Второе применение очень важно (кстати, оно же объясняет выбор названия `useRef`); это самый простой и лучший механизм обращения к элементам DOM в компонентах.

Пассивные значения состояния

С помощью `useRef` можно запомнить значение, которое актуально между рендерингами компонента, но не влияет напрямую на результат работы компонента. Звучит сложно и даже непонятно. Для чего может понадобиться такое значение в компоненте? Для примера снова воссоздадим компонент-счетчик, но на этот раз с дополнительной функциональностью: кнопка **Increment** работает только по двойному щелчку.

Время последнего щелчка должно храниться где-то в компоненте, и это место должно сохраняться между рендерингами. Вы уже знаете, что такие значения можно хранить в состоянии, предоставляемом хуком `useState`. Схема сценария изображена на рис. 7.1, а реализация приведена в листинге 7.1.

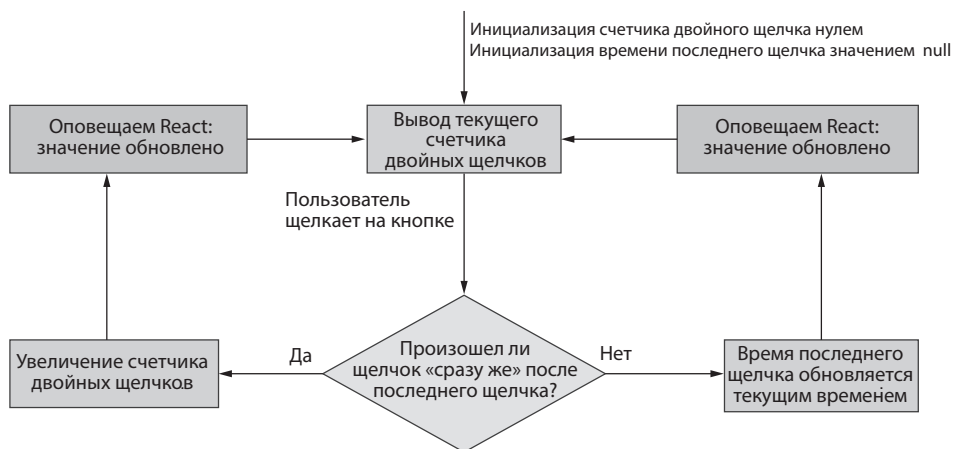


Рис. 7.1. По щелчку на кнопке выбирается один из двух путей выполнения в зависимости от того, следует ли второй щелчок за первым в очень коротком интервале времени

Листинг 7.1. Счетчик двойных щелчков с `useState`

```
import { useState } from "react";
const THRESHOLD = 300;
function DoubleClickCounter() {
  const [counter, setCounter] = useState(0);
  const [lastClickTime, setLastClickTime] = useState(null);
  const onClick = () => {
    const isDoubleClick =
      Date.now() - lastClickTime < THRESHOLD;
    if (isDoubleClick) {
      setCounter((value) => value + 1);
    } else {
      setLastClickTime(Date.now());
    }
  };
  return (
    <main>
      <p>Counter: {counter}</p>
      <button onClick={onClick}>Increment</button>
    </main>
  );
}
function App() {
  return <DoubleClickCounter />;
}
export default App;
```

Запоминает время последнего щелчка в значении состояния

Если с момента последнего щелчка прошло менее 300 мс, это двойной щелчок

Счетчик увеличивается только по двойному щелчку

Сохранение времени текущего щелчка, если это не двойной щелчок

Впрочем, такая схема не обязательна и только приводит к лишним повторным рендерингам. При вызове `setLastClickTime` React повторно рендерит компонент из-за изменения значения состояния. Однако JSX в компоненте не изменяется, и на экране будет отображен тот же вывод DOM. Код в листинге 7.1 работает, но он не оптимален.

Так как значение состояния используется только внутри компонента и компонент не должен рендериться заново только из-за обновления значения, можно воспользоваться ссылкой с использованием хука `useRef`. Просто вызовите хук `useRef` и сохраните результат в переменной.

Также можно передать необязательное исходное значение хуку. Чтобы прочитать или обновить текущее значение хука `useRef`, обратитесь к свойству `.current` возвращаемого значения хука. Сравните схему этого сценария на рис. 7.2 со сценарием на рис. 7.1 — экономится целый цикл рендеринга! Реализация представлена в листинге 7.2.

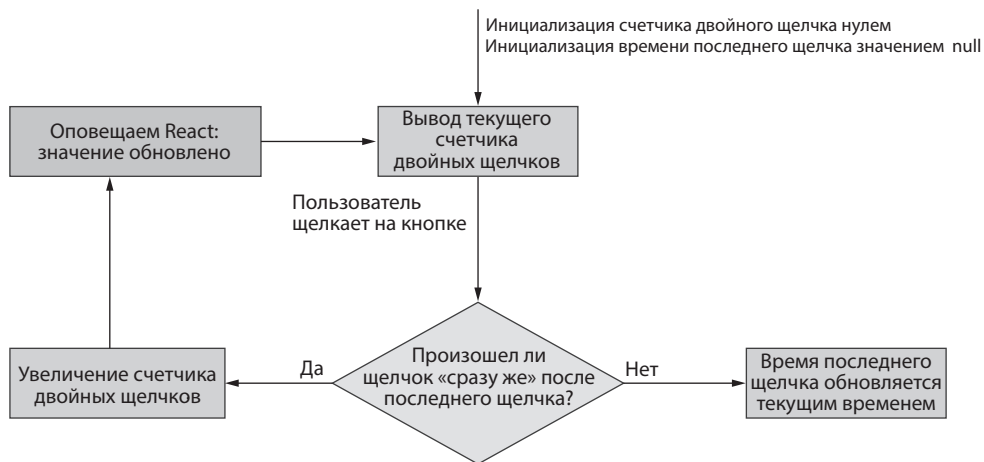


Рис. 7.2. На этот раз по первому щелчку на кнопке сохраняется время последнего щелчка, но это не приводит к повторному рендерингу, потому что это значение состояния не активное, а пассивное. То есть к значению можно будет обратиться позже (при этом или будущем рендеринге), но новый рендеринг при этом не запускается

Листинг 7.2. Счетчик двойных щелчков с useRef

```

import { useState, useRef } from "react";
const THRESHOLD = 300;
function DoubleClickCounter() {
  const [counter, setCounter] = useState(0);
  const lastClickTime = useRef(null);
  const onClick = () => {
    const isDoubleClick =
      Date.now() - lastClickTime.current < THRESHOLD;
    if (isDoubleClick) {
      setCounter((value) => value + 1);
    } else {
      lastClickTime.current = Date.now();
    }
  };
  return (
    <main>
      <p>Counter: {counter}</p>
      <button onClick={onClick}>Increment</button>
    </main>
  );
}
function App() {
  return <DoubleClickCounter />;
}
export default App;

```

Запоминает время последнего щелчка в значении useRef

Выполняет ту же проверку, что и раньше, только для обращения к значению теперь используется свойство .current

Обновляет текущее значение состояния через свойство .current

Репозиторий: rq07-double-counter

Этот пример находится в репозитории `rq07-double-counter`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq07-double-counter --template rq06-double-counter
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq07-double-counter>

Эта версия компонента намного лучше, потому что в ней нет лишнего повторного рендеринга. Кроме того, все будет работать, даже если компонент будет отрендерен повторно по другой причине.

Ссылки на элементы DOM

Как уже говорилось, `useRef` также используется для получения ссылок на элементы DOM. Этот прием неоднократно используется в оставшейся части книги. Мы будем применять его для получения ссылки на реальный элемент DOM, который рендерится в документе как следствие только что созданного элемента JSX. Синтаксис очень прост:

```
function Component() {
  const ref = useRef();
  return <div ref={ref} />;
}
```

Создает ссылку с использованием хука

Ссылке "присваивается" узел DOM; с таким синтаксисом при рендеринге компонента объекту `ref` будет присвоена ссылка на узел DOM

В этом случае ссылка ни для чего не используется, а только создается. Ее можно использовать в хуке эффекта, например, для вызова методов для элемента, чего нельзя сделать напрямую через свойства элемента DOM.

Например, следующий код автоматически передает фокус полю ввода при монтировании компонента:

```
function AutoFocusInput() {
  const ref = useRef();
  useEffect(() => ref.current.focus(), []);
  return <input ref={ref} />;
}
```

Создает эффект, который выполняется только при монтировании (пустой массив зависимостей), и передает фокус элементу через объект `ref`. Обратите внимание на использование синтаксиса `ref.current`, о котором говорилось выше

Последний вариант использования `useRef` более распространен (и именно для этой цели хук создавался). О `useRef` и свойстве `JSX ref` можно еще многое сказать, но пока оставим эту тему. Рассмотрим ее более подробно в следующих главах.

7.1.4. Упрощенное многокомпонентное состояние с использованием `useContext`

`useContext` относится к категории хуков состояния; это означает, что он работает аналогично `useState`. Но вместо того чтобы загружать и обновлять данные в локальном хранилище, `useContext` работает с хранилищем родительского компонента выше в дереве компонентов. Это версия React Context API, реализованная на основе хуков; ее практическое применение намного подробнее рассматривается в главе 10. А пока скажем, что это один из самых эффективных хуков, позволяющих создавать надежные архитектуры.

7.1.5. Низкоприоритетные обновления состояния с использованием `useDeferredValue` и `useTransition`

ПРИМЕЧАНИЕ Это довольно сложная, и притом совершенно новая тема. Все эти возможности появились только в React 18, так что поиск лучших практик все еще продолжается. Кроме того, вы вряд ли встретите в повседневной работе такой нетипичный случай. Мы не станем полностью описывать функциональность этих хуков, лишь кратко объясним, для чего они существуют. Если эта тема вам неинтересна, спокойно пропускайте ее.

Представьте, что вы создаете онлайн-редактор документов, схожий с Google Docs, содержащий многочисленные функции. На очереди добавление функции проверки орфографии. Для этого необходимо создать кнопку, однократное нажатие которой включает проверку орфографии, а повторное нажатие отключает ее. Активная кнопка имеет другой цвет фона.

Кнопка должна мгновенно реагировать на нажатие. Изображение должно меняться в пределах нескольких миллисекунд, чтобы пользователь понимал, что кнопка работает. Эту функциональность можно реализовать простым переключением свойства блокировки внутри кнопки при помощи сеттера `useState`.

Однако задачи этим не ограничиваются. При включении проверки орфографии все ошибки в документе должны подчеркиваться красной линией. Если пользователь работает над большим документом, поиск и выделение всех ошибок может оказаться весьма затратной операцией. Придется выполнить множество

операций со всеми словами в документе, чтобы найти ошибки и, возможно, даже предложить варианты исправления для каждого случая. Такие операции могут занимать десятые доли секунды и даже целые секунды.

Если внутренний флаг состояния внутри кнопки и глобальный флаг, инициирующий проверку орфографии, будут выполняться в React в одно время и с одинаковым приоритетом, внутренняя реализация React будет рассматривать оба обновления как одновременные и не станет ничего рендерить, пока они не завершатся. Это не очень хорошо, потому что кнопка проверки орфографии будет казаться нерабочей. Если пользователь нажимает кнопку и при этом ничего не происходит, он обычно нажимает ее еще раз. Внезапно кнопка срабатывает после завершения обработки, но поскольку пользователь успел сделать повторный щелчок, функциональность блокируется. Пользователю вряд ли понравится такое приложение.

Нельзя ли сообщить React, что обновление состояния кнопки имеет высокий приоритет и должно происходить немедленно, тогда как выделение всех ошибок в документе имеет более низкий приоритет и может отставать от нажатия кнопки на несколько циклов рендеринга? В React 18 появилась новая концепция *конкурентного режима* (Concurrent Mode), который предоставляет именно такую возможность. Хуки `useDeferredValue` и `useTransition` используются для назначения низкоприоритетных обновлений состояния с двух разных позиций. С учетом сложности этих хуков мы не станем рассматривать их в книге. Узнать о них больше можно из следующих источников:

- Статья: <http://mng.bz/jPAP>
- Видео: <http://mng.bz/WzM1>

7.2. ЭФФЕКТЫ КОМПОНЕНТОВ

Этот раздел будет совсем коротким, так что читайте не моргая, а не то полностью его пропустите! К *эффектам компонентов* относится группа хуков, предназначенная для выполнения побочных эффектов изнутри хуков. Цели могут быть такими:

- повлиять на определенный внешний аспект в зависимости от состояния компонента;
- обновить состояние компонента в зависимости от внешних факторов;
- повлиять на внешний аспект и обновить состояние компонента одновременно.

Два таких хука, `useEffect` и `useLayoutEffect`, уже были рассмотрены в предыдущей главе. Есть еще один подобный хук — `useInsertionEffect`, но он

зарезервирован для конкретных библиотек, поэтому его не рекомендуется использовать «обычным» разработчикам.

В этой главе мы не будем рассказывать больше о `useEffect` и `useLayoutEffect`, так как все, что нужно, мы уже сказали в главе 6. Последний хук эффекта, `useInsertionEffect`, будет кратко рассмотрен в разделе 7.4.

7.3. ОПТИМИЗАЦИЯ ПРОИЗВОДИТЕЛЬНОСТИ ЗА СЧЕТ СОКРАЩЕНИЯ КОЛИЧЕСТВА ПОВТОРНЫХ РЕНДЕРИНГОВ

ПРИМЕЧАНИЕ Это сложная тема, которая не обязательна для большинства простых приложений. Мы не будем углубляться в нее подробно, так как эти знания вам не понадобятся ни для первого, ни для второго, ни даже для десятого приложения. Весь потенциал этих хуков раскроется, только когда вы перейдете к работе над крупными приложениями с десятками и даже сотнями компонентов.

Если вы работаете над большим приложением, в котором существует множество «подвижных частей», данные обновляются из разных источников и имеются прослушватели событий для разных типов ввода, избыточный рендеринг компонентов может привести к снижению производительности. Когда ваши приложения выйдут на этот уровень, вернуть высокую скорость отклика им поможет мемоизация.

По своей сути *мемоизация* представляет собой кэширование результата некоторых вычислений, чтобы при повторении этих же вычислений возвращался кэшированный результат. Мемоизация может применяться в React несколькими способами, включая три хука, о которых мы поговорим в этом разделе.

Как уже было сказано, мы не станем подробно описывать эти хуки, так как их использование не обязательно для начинающих разработчиков React. Собственно, неправильное применение мемоизации может, наоборот, ухудшить производительность. Таким образом, с учетом нетипичной природы мемоизации мы вообще не применяем ее в этой книге и только вкратце опишем три хука, связанных с ней.

Узнать больше об оптимизации производительности React можно в книге «Job-Ready React» (Morten Barklund, Manning, 2024), в которой рассматриваются не только эти хуки и мемоизация вообще, но и другие приемы повышения производительности приложений.

7.3.1. Мемоизация произвольного значения с использованием `useMemo`

Предположим, вам требуется вывести криптографический хеш пароля, введенного в текстовом поле. Вычисление хеша — довольно затратная операция, так что если пароль не изменился, вычисления лучше не выполнять. Но компонент повторно рендерится несколько раз даже без изменения пароля. В таких и подобных случаях можно воспользоваться хуком `useMemo` для пересчета заданного значения в компоненте только при изменении его зависимостей.

7.3.2. Мемоизация функций с использованием `useCallback`

`useCallback` — специализированная версия `useMemo`, которая будет особенно полезной при использовании `useMemo` для мемоизации функции. Так как подобная задача встречается очень часто, для этой цели существует специальный хук `useCallback`, и обычно он применяется чаще, чем `useMemo`.

7.3.3. Создание стабильных идентификаторов DOM с использованием `useId`

Это еще более сложная тема, актуальная только для серверного React. Чтобы понять, когда использовать этот узкоспециализированный хук, необходимо обладать определенным запасом знаний.

`useId` гарантирует, что для двух абсолютно идентичных деревьев компонентов, если конкретный компонент внутри дерева вызывает `useId`, он будет получать один и тот же идентификатор независимо от того, на какой платформе выполняется хук. Это обеспечивает идентичную сгенерированную разметку HTML на клиенте и на сервере.

7.4. СОЗДАНИЕ СЛОЖНЫХ БИБЛИОТЕК КОМПОНЕНТОВ

Этот раздел включен только для полноты, чтобы описать все хуки React. Четыре хука, представленные в следующих подразделах, очень сложны и редко используются. Они предназначены для повторно используемых пакетов, например библиотек компонентов или модулей с открытым исходным кодом.

Последние два хука, упомянутые в этом разделе, появились в React 18 как следствие нового конкурентного режима. Некоторые библиотеки должны обновляться для правильного выполнения рендеринга в конкурентном режиме, чтобы избежать вычисления логики, которая не является строго

обязательной или оказывается преждевременной из-за конкурентности. Если вас интересуют более практические темы, пропустите этот раздел и переходите к разделу 7.5.

7.4.1. Создание API компонентов с использованием `useImperativeHandle`

Этот хук используется в высокоуровневых библиотеках компонентов, в которых требуется предоставить родительским компонентам API, адаптированный для вашего конкретного компонента или имитирующий встроенный элемент DOM для простоты использования. Он почти исключительно используется с `forwardRef`, что позволяет создавать собственные компоненты, получающие ссылки, но передающие их другим элементам или создающие специализированные ссылки.

Простой пример такого рода — обобщенный нестандартный компонент ввода, в котором родительский компонент должен передавать фокус нужному элементу. Предположим, что на экран выводится сообщение об ошибке «Поле не заполнено», и когда пользователь щелкает на этом сообщении, фокус передается нужному полю. Однако внутри компонента ввод может быть представлен элементами разных типов (текстовыми полями, текстовыми областями, списками выбора) и даже несколькими текстовыми полями ввода (представьте группу полей для ввода телефонного номера, включающую поля для ввода префикса кода страны и собственно номера).

Чтобы обобщить все подобные случаи и создать унифицированный API, можно воспользоваться хуком `useImperativeHandle`, предоставляющим метод `focus()` для компонента. Этот метод может использоваться в императивном коде (а в декларативном коде только через свойства), что гарантирует передачу фокуса нужному элементу при вызове.

Мы не будем углубляться в подробности работы этого хука или `forwardRef` — это продвинутая тема, выходящая за рамки настоящей главы. И все же полезно знать о существовании этого хука на случай, если вам потребуется создать сложный нестандартный компонент, предоставляющий нестандартный API по ссылке. Узнать больше можно в «исчерпывающем справочнике» по `useImperativeHandle`: <http://mng.bz/EQ00>.

7.4.2. Улучшенная отладка хуков с использованием `useDebugValue`

Этот хук предназначен только для удобства разработчика. Он никак не изменяет и не улучшает опыт взаимодействия пользователя с приложением.

Хук `useDebugValue` позволяет вывести нестандартное сообщение для других разработчиков, которые просматривают ваши нестандартные хуки в своих приложениях React при помощи плагина React Developer Tools в браузерах.

Обычно нестандартный хук выводит все свои внутренние состояния на панели React Developer Tools, но они могут быть непонятны тем, кого не интересует внутреннее строение нестандартных хуков. С хуком `useDebugValue` можно выводить только ту информацию, которая действительно представляет интерес для разработчика, использующего хук. Подробнее см. в статье [How to Use useDebugValue in React](http://mng.bz/N251) по адресу <http://mng.bz/N251>.

7.4.3. Синхронизация внешних данных с использованием `useSyncExternalStore`

В конкурентном режиме React может обновлять значение состояния с низким приоритетом, и во время вычисления последствий этого обновления может поступить неотложное обновление, вычисление которого должно производиться независимо от незавершенного обновления. Так как React работает по принципу параллелизма, в нем может существовать несколько полностью независимых экземпляров выполняемого приложения. Как следствие, при поступлении неотложного обновления React может проводить новые вычисления, основанные на предыдущем состоянии.

Если приложение использует внешнюю библиотеку для обновления состояния, эта библиотека должна иметь конкурентную логику состояния, чтобы тоже поддерживать несколько выполняемых одновременно экземпляров состояния. В React 18 именно для этой цели был введен хук `useSyncExternalStore`. Подробнее о нем можно прочитать в статье: <http://mng.bz/8r1K>.

7.4.4. Выполнение эффекта перед рендерингом с использованием `useInsertionEffect`

Если у вас есть библиотека, которая создает таблицы стилей или похожие узлы HTML в документе в качестве побочного эффекта рендеринга компонента, она должна поддерживать конкурентный режим, чтобы рендерить правильные узлы в нужное время. Для этой цели в React 18 появился хук `useInsertionEffect`.

Хотя может показаться, что это хук эффекта из той же категории, что и `useEffect` и `useLayoutEffect`, хук `useInsertionEffect` никогда не применяется к обычным компонентам. Он был создан только как следствие того, что некоторые библиотеки общего назначения должны обновляться с учетом особенностей конкурентного выполнения. Узнать больше можно из небольшой статьи о `useInsertionEffect`: <http://mng.bz/EQlq>.

7.5. ДВА КЛЮЧЕВЫХ ПРИНЦИПА ХУКОВ

При работе с хуками React необходимо соблюдать только два правила:

- Вызывайте хуки только *безусловно* на *верхнем уровне* функциональных компонентов.
- Вызывайте хуки только *внутри* функциональных компонентов.

Первое правило уже обсуждалось ранее: хуки можно использовать только непосредственно в компонентах и их всегда должно быть одинаковое количество. Это означает, что хуки никогда не должны вызываться внутри функции (в том числе внутри функции, вызываемой в хуке) или во вложенном блоке (условной конструкции или цикле) и в компоненте не должен происходить быстрый возврат, пока не завершится рендеринг всех хуков.

Второе правило кажется очевидным, но, возможно, не для всех: хуки можно использовать только внутри функциональных компонентов. Нельзя создать вспомогательную функцию или обратный вызов хука. Также хуки нельзя использовать внутри компонентов на базе классов.

Из этого правила существует только одно исключение: хуки можно использовать внутри других хуков, которые называются *нестандартными хуками*; нестандартные хуки могут использоваться внутри других нестандартных хуков, и т. д. Но нестандартные хуки можно использовать только либо внутри других нестандартных хуков, либо в компонентах, так что обойти правило не удастся — его можно только спрятать на один уровень (или несколько уровней) ниже. Нестандартные хуки рассматриваются в главе 10.

7.6. ВОПРОСЫ

1. В React всегда было и будет 15 хуков. *Да* или *нет*?
2. Какие из следующих хуков относятся к хукам состояния?
 - a) `useState`
 - b) `useValue`
 - c) `useId`
 - d) `useReducer`
3. `useMemo` — специализированная версия `useCallback`. *Да* или *нет*?
4. Хук не может вызываться в функции, если только это не функциональный компонент или нестандартный хук. *Да* или *нет*?

5. Какая из следующих конструкций недопустима?

- a)

```
function Component({ isVisible }) {
  if (!isVisible) return false;
  useEffect(() => { ... }, []);
  ...
}
```
- b)

```
function Component({ hasEffect }) {
  if (hasEffect) {
    useEffect(() => { ... }, []);
  }
  ...
}
```
- c)

```
function Component({ shouldRender }) {
  useEffect(() => { ... }, []);
  if (!shouldRender) return false;
  ...
}
```

ОТВЕТЫ

1. *Нет.* В React 16.8 появились первые 10 хуков, а в React 18.0 добавились еще 5.
2. `useState` и `useReducer` относятся к категории хуков состояния. `useValue` не является встроенным хуком (при желании вы можете создать нестандартный хук с таким именем), а `useId` уже используется для узкоспециализированной цели мемоизации.
3. *Нет.* Все наоборот; `useMemo` — обобщенный хук для мемоизации *любого значения*, тогда как `useCallback` — хук для мемоизации исключительно *функций*.
4. *Да.* Не пытайтесь вызывать хуки внутри функций, которые сами не являются функциями. Хотя на первый взгляд может показаться, что такое решение работает, но оно только создаст проблеммы, если одна из таких функций будет вызываться за пределами функционального компонента. Соблюдайте принципы работы с хуками!
5. Недопустимые конструкции — *a* и *b*. Только версия *c* представляет собой правильный компонент. В версиях *a* и *b* используется условный рендеринг хуков, а это запрещено.

ИТОГИ

- React содержит 15 встроенных хуков, но некоторые из них используются очень редко. Около 10 оставшихся хуков образуют базовый API, на основе которого строятся все приложения React.
- Хуки используются для разных целей, которые делают компоненты более эффективными и позволяют взаимодействовать с веб-страницей в целом. И хотя цели всех хуков очень сильно различаются, у всех хуков есть общие свойства.
- Хуки состояния обеспечивают поддержку состояния в приложении. В зависимости от сложности приложения и значений в состоянии можно использовать несколько разных хуков. С React 18 даже можно выполнять низкоприоритетные и высокоприоритетные обновления состояния, чтобы UI как можно быстрее реагировал на действия пользователя.
- Хуки эффектов используются для выполнения побочных эффектов внутри компонентов, как вы узнали в главе 6. При помощи массива зависимостей можно инициировать выполнение эффекта в нужные моменты времени.
- Хуки мемоизации используются для оптимизации рендеринга в React большого и сложного приложения.
- Библиотечные хуки предназначены для более сложного кода. Скорее всего, они не понадобятся для повседневных приложений.
- Используя хук, необходимо соблюдать два закона: хуки должны вызываться только на верхнем уровне компонента (нельзя использовать условные или циклические вызовы хуков) и только внутри функциональных компонентов (никаких хуков вне компонента, во вспомогательной функции и даже в компонентах на базе классов).

8

Обработка событий в React

В ЭТОЙ ГЛАВЕ

- ✓ Реакция на пользовательский ввод с использованием событий
- ✓ Обработка событий методом погружения и всплытия
- ✓ Управление действиями событий по умолчанию
- ✓ Присоединение прослушивателей событий непосредственно к DOM

Взаимодействие пользователей с веб-приложением JavaScript происходит с помощью событий. События могут инициироваться перемещением или щелчками мыши, касаниями и перетаскиваниями в сенсорном интерфейсе, нажатиями клавиш, прокруткой, копированием/вставкой, а также такими непрямыми взаимодействиями, как передача и снятие фокуса с элементов всего приложения.

До сих пор мы создавали приложения React, по минимуму взаимодействующие с пользователем. Мы обрабатывали нажатия кнопок, но не объясняли подробно, как работают события нажатий и как они обрабатываются. В этой главе, посвященной обработке событий, все изменится.

События можно рассматривать как способ обработки ввода от пользователя. Веб-приложение создает разметку JavaScript XML (JSX), которая преобразуется

в HTML. Затем пользователь взаимодействует с HTML, и результатом этих взаимодействий становятся события, передаваемые от элементов HTML приложению React. Простая логика передачи информации представлена на рис. 8.1.



Рис. 8.1. Передача информации между React и пользователем происходит через HTML. Представьте, что пользователь открывает страницу входа. Он вводит адрес электронной почты и пароль, браузер передает эти взаимодействия React в виде событий, затем приложение генерирует разметку JSX, необходимую для отображения зеленой галочки рядом с каждым заполненным полем ввода, а браузер рендерит соответствующую разметку HTML для пользователя

События также используются во внутренней работе браузера для обозначения изменений в элементах, например воспроизведения/приостановки/буферизации видео, завершения анимации, изменений узла DOM, загрузки данных (или неудачи при загрузке) и т. д. Существуют сотни возможных событий, и в любом интерактивном веб-приложении их довольно много. (Обо всех возможных событиях DOM можно прочесть, перейдя по ссылке <http://mng.bz/9D1j>.)

События в React можно обрабатывать двумя способами:

- использовать React для управления прослушивателем событий;
- добавлять и удалять прослушиватель событий вручную непосредственно в узле DOM.

Поручая работу с прослушивателями React, вы избавляетесь от большого объема однообразной работы и хлопот (а еще от возможных утечек памяти), но за это приходится расплачиваться некоторой потерей гибкости. Второй способ позволяет прослушивать разные виды событий и назначать прослушиватели любым узлам по желанию, но прослушивателями приходится управлять (и помнить о том, что их нужно удалять), а также обрабатывать платформенные события, которые могут различаться между браузерами.

В этой главе мы продемонстрируем оба подхода и обсудим, когда лучше применять тот или иной вариант. Заметим, что обработка событий *в React* проще, и поэтому рекомендуется использовать именно этот способ. По этой причине мы рассмотрим его намного подробнее.

Разбираясь, как прослушиваются события в интерфейсе React, мы поговорим о том, как обрабатываются события в React и как работать с React API для прослушивания конкретных событий, которые вам нужны. В частности, мы ответим на следующие вопросы:

- Какие события поддерживаются?
- Как создать функцию обработчика события?
- Какие объекты событий вы получаете?
- Как работают фазы и распространение событий?
- Как обрабатывать события в фазе погружения?
- Что такое действия по умолчанию и как их предотвратить?
- Когда следует сохранять события?
- Можно ли использовать свойства как обработчики событий?
- Что такое генераторы обработчиков событий?

Затем мы перейдем к ситуациям, в которых встроенной обработки событий React оказывается недостаточно и события приходится обрабатывать вручную в DOM. Мы также поделимся своими наблюдениями относительно того, как это лучше делать.

Разобравшись в событиях, мы сможем перейти к следующей главе и воспользоваться новым пониманием обработки событий для создания интерактивных полей ввода в формах и форм вообще — краеугольного камня многих веб-приложений.

ПРИМЕЧАНИЕ Исходный код примеров этой главы доступен по адресу <https://rq2e.com/ch08>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке одной командой.

8.1. ОБРАБОТКА СОБЫТИЙ DOM В REACT

События представляют собой важнейший механизм коммуникации в браузере между пользователем и скриптом, а также между разными элементами приложения. По этой причине правильная обработка событий становится одним из фундаментальных аспектов React в том смысле, что значительная часть базового API React выделяется именно для этой цели.

API очень простой. Если вы определяете свойство для элемента JSX, ссылающегося на узел HTML, и это свойство соответствует одному из известных событий из списка событий, поддерживаемых React, то React рассматривает это свойство как прослушиватель события, а не как атрибут DOM. Тогда React обеспечит

правильное добавление и удаление прослушивателя события при монтировании и демонтировании компонента.

8.1.1. Базовая обработка событий в React

Самое важное событие практически во всех веб-приложениях — событие щелчка. Несмотря на название, оно предназначено не только для получения щелчков мыши. Событие щелчка в HTML также выдается, когда пользователь касается кнопки (или ссылки) на сенсорном экране или активирует кнопку (или ссылку) клавишей Enter на клавиатуре.

Вернемся к нашему старому знакомому — компоненту-счетчику и внимательно рассмотрим, как в нем обрабатывается событие щелчка. Напомним, что в приложении имеется кнопка, а значение состояния инкрементно изменяется в ответ на щелчок. Для начала вспомним код этого простого приложения.

Листинг 8.1. Компонент-счетчик

```
import { useState } from "react";
function Counter() {
  const [counter, setCounter] = useState(0);
  const onClick = () =>
    setCounter((value) => value + 1);
  return (
    <>
      <h1>Value: {counter}</h1>
      <button onClick={onClick}>Increment</button>
    </>
  );
}
function App() {
  return <Counter />;
}
export default App;
```

Создает локальную переменную, которая представляет собой функцию, инкрементно изменяющую значение состояния при вызове

Присваивает локальную переменную свойству onClick кнопки

В этом примере мы обрабатываем событие щелчка для объекта HTML — в данном случае `<button>`. В любом элементе HTML щелчок вызывает именно это событие, так что этот элемент можно заменить на `<div>` или любой другой тип элемента.

Другое событие, которое можно прослушивать для всех объектов, — событие мыши (или указателя). Например, любой элемент может выдать событие `mousemove`, когда мышь перемещается внутри границ этого элемента. Это событие прослушивается аналогичным образом.

Создадим компонент, который выводит галочку, если мышь перемещается внутри элемента, или крестик, если мышь перестала двигаться на полсекунды или вышла за пределы элемента.

Для этого необходимо прослушивать событие `mousemove`. В React это означает присваивание функции свойству `onMouseMove` целевого элемента. В данном случае мы используем элемент `<section>` и выведем результат в заголовке внутри него. Реализация приведена в листинге 8.2, а результат выполнения кода показан на рис. 8.2.

Листинг 8.2. Детектор перемещений мыши

```
import { useState, useEffect } from "react";
function MouseStatus() {
  const [isMoving, setMoving] = useState(false);
  const onMouseMove = () => setMoving(true);
  useEffect(() => {
    if (!isMoving) return;
    const timeout = setTimeout(() => setMoving(false), 500);
    return () => clearTimeout(timeout);
  }, [isMoving]);
  return (
    <section onMouseMove={onMouseMove}>
      <h2>
        The mouse is {!isMoving && "not"} moving: {isMoving ? "✓" : "X"}
      </h2>
    </section>
  );
}
function App() {
  return <MouseStatus />;
}
export default App;
```

Создает локальную переменную, которая представляет собой функцию, устанавливающую флаг перемещения true при вызове

Присваивает локальную переменную соответствующему свойству элемента — на этот раз свойству `onMouseMove` элемента `section`

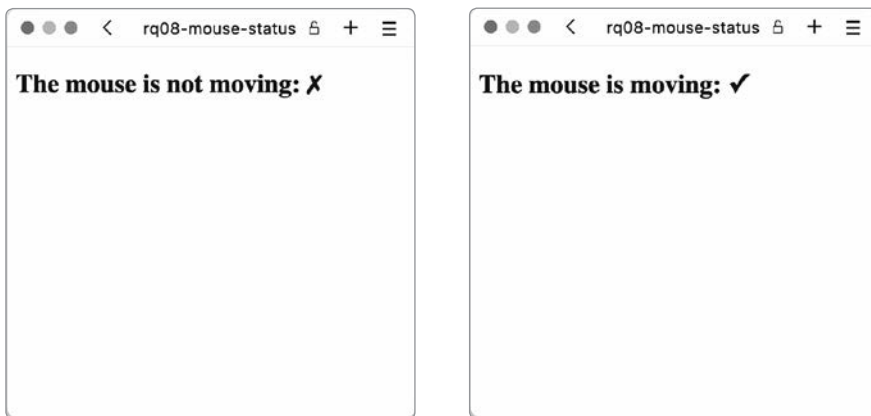


Рис. 8.2. Компонент проверки состояния мыши, когда мышь остается на месте и перемещается соответственно

Репозиторий: rq08-mouse-status

Этот пример находится в репозитории `rq08-mouse-status`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq08-mouse-status --template rq08-mouse-status
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq08-mouse-status>

Впрочем, не все события можно вызвать во всех типах элементов. В элементах видео (и аудио) вызывается событие воспроизведения, когда видео (или аудио) начинает воспроизводиться. В кнопках это событие не вызывается, потому что они не являются элементом, который воспроизводится.

Создадим приложение, которое отображает кнопку Play/Pause рядом с видео. Во время воспроизведения видео на кнопке выводится надпись Pause; при приостановке на кнопке выводится надпись Play.

Для этого понадобятся четыре прослушателя. Необходимо прослушивать события воспроизведения и приостановки объекта видео, а также событие щелчка для кнопки, но с двумя разными прослушателями в зависимости от того, воспроизводится видео или нет. Эта функциональность реализована в листинге 8.3. Запустив это приложение, вы увидите в браузере вывод как на рис. 8.3.

Листинг 8.3. Очень простой видеоплеер

```

Переводит флаг состояния в false,
когда видео приостанавливается

import { useState, useRef } from "react";
const VIDEO_SRC =
  "//images-assets.nasa.gov/video/One Small Step/One Small Step~orig.mp4";
function VideoPlayer() {
  const [isPlaying, setPlaying] = useState(false);
  const onPlay = () => setPlaying(true);
  const onPause = () => setPlaying(false);
  const onClickPlay = () => video.current.play();
  const onClickPause = () => video.current.pause();
  const video = useRef();
  return (
    <section>
      <video
        ref={video}
        src={VIDEO_SRC}
        data-bbox="129 634 853 888"/>

```

Устанавливает флаг состояния true, когда видео начинает воспроизводиться

Вызывает play по ссылке на элемент DOM, когда пользователь щелкает на кнопке, чтобы воспроизвести видео

Приостанавливает видео, когда пользователь щелкает на кнопке, а видео уже воспроизводится

```

    controls
      width="480"
      onPlay={onPlay}
      onPause={onPause}
    />
    <button onClick={
      isPlaying ? onClickPause : onClickPlay
    }>
      {isPlaying ? "Pause" : "Play"}
    </button>
  </section>
);
}
function App() {
  return <VideoPlayer />;
}
export default App;

```

Назначает два прослушателя событий для элемента `video` при помощи соответствующих свойств

Назначает один из прослушателей события щелчка кнопки свойству `onClick` в зависимости от флага



Рис. 8.3. Интерфейс при воспроизведении и приостановке видео соответственно

Чтобы прослушивать события в React, необходимо выполнить всего три условия:

- Знать, какое событие нужно прослушивать.
- Знать, для какого элемента должно вестись прослушивание.
- Назначить функцию прослушивания правильному свойству правильного элемента.

Репозиторий: `rq08-video-player`

Этот пример находится в репозитории `rq08-video-player`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq08-video-player --template rq08-video-player
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq08-video-player>

Собственно, это все. В оставшейся части главы мы разберем несколько примеров событий, и вы будете готовы к обработке любого сценария с событиями, который вам встретится в ходе работы.

События, поддерживаемые в React

В React можно прослушивать только события, поддерживаемые React. Обычно это ограничение не доставляет неудобств, поскольку почти все события DOM поддерживаются React. Полный список всех поддерживаемых событий приведен в табл. 8.1.

Таблица 8.1. Список событий, напрямую поддерживаемых в React

События буфера обмена	onCopy onCut onPaste
События композиции	onCompositionEnd onCompositionStart onCompositionUpdate
События клавиатуры	onKeyDown onKeyPress onKeyUp
События фокуса	onFocus onBlur
События форм	onChange onInput onInvalid onReset onSubmit
Общие события	onError onLoad
События мыши	onClick onContextMenu onDoubleClick onDrag onDragEnd onDragEnter onDragExit onDragLeave onDragOver onDragStart onDrop onMouseDown onMouseEnter onMouseLeave onMouseMove onMouseOut onMouseOver onMouseUp
События указателя	onPointerDown onPointerMove onPointerUp onPointerCancel onGotPointerCapture onLostPointerCapture onPointerEnter onPointerLeave onPointerOver onPointerOut
События выбора	onSelect
События касания	onTouchCancel onTouchEnd onTouchMove onTouchStart
События UI	onScroll
События колесика	onWheel

Мультимедийные события	onAbort onCanPlay onCanPlayThrough onDurationChange onEmptied onEncrypted onEnded onError onLoadedData onLoadedMetadata onLoadStart onPause onPlay onPlaying onProgress onRateChange onSeeked onSeeking onStalled onSuspend onTimeUpdate onVolumeChange onWaiting
События изображений	onLoad onError
События анимации	onAnimationStart onAnimationEnd onAnimationIteration
События переходов	onTransitionEnd
Другие события	onToggle

Однако некоторые события JavaScript не поддерживаются в React, в основном потому, что эти события выдаются объектами, которые не входят в DOM, а создаются в JavaScript. К этой категории относятся такие события, как подключения к сокетам и запросы объектов.

Среди других неподдерживаемых событий DOM можно отметить те, которые отправляются только узлами окна или документа. Они не поддерживаются React, потому что эти два узла никогда не присутствуют в приложениях React. React работает только внутри элемента документа и никогда не выходит за его пределы.

Отметим, что если для элемента JSX задается свойство, соответствующее известному типу события из табл. 8.1, React преобразует это свойство в прослушиватель для этого элемента, независимо от того, может в этом элементе вызываться такое событие или нет. Например, можно назначить прослушиватель события `onPlay` для элемента `<h1 />`, хотя это событие может вызываться только в элементах `<video />` и `<audio />`.

8.2. ОБРАБОТЧИКИ СОБЫТИЙ

Чтобы обработать события, просто назначьте любую функцию соответствующему свойству элемента JSX, который может выдавать это событие. Функции не обязательно иметь определенное поведение или получать особый аргумент. Функция обработчика события будет вызвана с одним аргументом — объектом события, но получать его не обязательно.

Так как не существует никаких ограничений или официально принятых лучших практик относительно того, как должна определяться функция обработчика

события, разработчики действуют по-разному. В этом разделе рассматриваются некоторые часто используемые варианты и соглашения.

8.2.1. Определение обработчиков событий

ПРИМЕЧАНИЕ В этом подразделе вы не узнаете ничего нового о React. В основном мы еще раз опишем разные способы определения функций в JavaScript внутри других функций. Если вы отлично разбираетесь в JavaScript, спокойно пропускайте этот раздел и переходите к разделу 8.2.2.

Функцию события можно определять любым способом. Если функция действительна, то она действительна и как обработчик события. Вот несколько типичных вариантов:

- Определить функцию как локальную переменную с использованием стрелочной функции.
- Определить функцию как локальную переменную с использованием функционального выражения.
- Определить функцию как встраиваемую при помощи стрелочной функции, напрямую назначаемой свойству.

Снова приведем компонент-счетчик с локальной переменной и стрелочной функцией:

```
function Counter() {
  const [counter, setCounter] = useState(0);
  const onClick = () => setCounter(c => c + 1);
  return (
    <>
      <h1>Value: {counter}</h1>
      <button onClick={onClick}>Increment</button>
    </>
  );
}
```

Создает переменную с `const` и назначает функцию с использованием синтаксиса стрелочной функции

Назначает переменную свойству `onClick`

Тот же компонент, но с функцией-обработчиком, определенной с использованием функционального выражения:

```
function Counter() {
  const [counter, setCounter] = useState(0);
  function onClick() {
    setCounter(c => c + 1);
  }
  return (
    <>
```

Создает функцию с использованием функционального выражения, которое определяет переменную как локальную

```

    <h1>Value: {counter}</h1>
    <button onClick={onClick}>Increment</button>
  </>
);
}

```

← Назначает переменную свойству onClick

И наконец, тот же компонент с обработчиком, определенным во встраиваемом виде с использованием синтаксиса стрелочной функции:

```

function Counter() {
  const [counter, setCounter] = useState(0); return (
    <>
      <h1>Value: {counter}</h1>
      <button onClick={() => setCounter(c => c + 1)}>
        Increment
      </button>
    </>
  );
}

```

← Создает встроенный обработчик события и напрямую назначает его соответствующему свойству элемента HTML

Второе решение (с функциональным выражением внутри компонента) выглядит необычно, но оно полностью рабочее. Мы не будем использовать этот синтаксис, и на практике он применяется не так часто.

Вы сами решаете, определять обработчик события в переменной или во встраиваемой форме в JSX. Многие разработчики используют оба этих варианта, и в этой книге мы поступим так же. Скорее всего, ваша команда найдет для себя оптимальную схему, а если вы работаете в одиночку, выберите ту, которая лучше подойдет для вас.

Стандартный способ — определять однострочные обработчики событий как встроенные, а многострочные — в отдельной переменной. Следовательно, можно сделать так:

```

return (
  <button onClick={() => {
    setCounter(count => count + 1);
    toggleState();
  }}>Button</button>
);

```

Но некоторым разработчикам такое решение кажется слишком громоздким, и они предпочитают определять многострочные обработчики событий в отдельной переменной перед возвращением JSX:

```

const onClick = () => {
  setCounter(count => count + 1);
  toggleState();
};
return <button onClick={onClick}>Button</button>;

```

8.2.2. Объекты событий

Когда обработчик вызывается из-за возникновения события, ему передается один аргумент — объект события. Так работает и обычный HTML, и JavaScript, и React.

Объекты событий React немного отличаются от других объектов, но мы займемся этими отличиями в следующем подразделе. А пока отметим, что общего у обычных объектов событий JavaScript и объектов событий React.

Снова попробуем построить компонент-счетчик с кнопками инкремента и декремента, но на этот раз воспользуемся одной функцией-обработчиком для обработки событий щелчков обеих кнопок. Это нужно, чтобы показать альтернативный вариант структурирования кода. Он не лучше и не хуже в отношении производительности, но некоторые разработчики предпочитают этот стиль предыдущему.

Чтобы реализовать эту схему, необходимо знать, какая кнопка вызвала событие, отправленное обработчику. Для этого следует обратиться к переданному объекту события. Он содержит свойство `.target`, ссылающееся на узел HTML, щелчок которого был сделан. Чтобы сравнить целевое свойство с реальным узлом, понадобится ссылка на один из узлов компонента. Реализация приведена в листинге 8.4.

Листинг 8.4. Инкремент и декремент в одном обработчике события

```
import { useState, useRef } from "react";
function Counter() {
  const [counter, setCounter] = useState(0);
  const increment = useRef();
  const onClick = (evt) => {
    const delta =
      evt.target === increment.current ? 1 : -1;
    setCounter((value) => value + delta);
  };
  return (
    <section>
      <h1>Value: {counter}</h1>
      <button ref={increment} onClick={onClick}>
        Increment
      </button>
      <button onClick={onClick}>Decrement</button>
    </section>
  );
}
function App() {
  return <Counter />;
}
export default App;
```

Прежде всего понадобится ссылка, чтобы обратиться к узлу HTML

Затем в одном обработчике события цель события сравнивается с узлом `increment`. Если кнопка не совпадает, значит, используется другая кнопка

Добавляет дельту к текущему хранимому значению

Назначает тот же обработчик события обеим кнопкам, но `ref` только для кнопки инкремента

Лучше ли это решение, чем то, в котором используются два отдельных обработчика событий? Субъективный вопрос. Оба решения хороши. В одних ситуациях лучше подходит одно, в других — другое. Выбор в основном зависит от предпочтений разработчика. Вы считаете код одного обработчика событий более понятным или предпочитаете создавать разные обработчики? Разницы в производительности нет, так что все зависит исключительно от выбора стиля.

Объекты событий всегда содержат свойство `target`, ссылающееся на цель события. Кроме того, у всех событий имеется свойство `type`. Значением этого свойства является тип инициированного события. Представьте, что один обработчик события был назначен как свойству `onMouseEnter`, так и свойству `onFocus` поля ввода. Тогда обработчик события сработает как при перемещении мыши над полем, так и при переходе к полю с клавиатуры. Чтобы узнать, какое событие произошло, достаточно проверить свойство `evt.type`.

Некоторые объекты событий содержат дополнительные свойства, специфические для типов событий. Например, объекты событий мыши всегда содержат свойства `.clientX` и `.clientY`, которые указывают, в какой позиции документа произошло событие мыши, а также свойства `.ctrlKey` и `.shiftKey`, указывающие, какие из управляющих клавиш были нажаты в момент события мыши. Впрочем, объекты событий мыши содержат много других свойств. Полный список всех объектов событий в React содержится в электронной документации по адресу <http://mng.bz/D4Zy>.

8.2.3. Объекты событий React

Не путайте обработчики событий React с «настоящими» обработчиками событий DOM. Обработчик событий DOM добавляется в узел DOM и получает объект события DOM при вызове. Обработчик события React не добавляется непосредственно в узел DOM и вызывается React с объектом события React, когда React обнаруживает событие заданного типа в этом узле. Сравните эти два механизма, изображенные на рис. 8.4.

Обратите внимание, что при добавлении прослушивателя к элементу JSX React нигде не добавляет новый прослушиватель. Он просто запоминает, что вам нужна информация о конкретном типе события для этого конкретного узла. React уже прослушивает все события на всех узлах, так что при возникновении события конкретного типа React проверяет, соответствует ли цель той, которую вы запросили; в случае совпадения React вызывает прослушиватель события с нестандартным объектом события React. Реализация в React новой системы событий поверх уже существующей системы, встроенной в браузер, объясняется двумя причинами: производительностью и последовательностью.

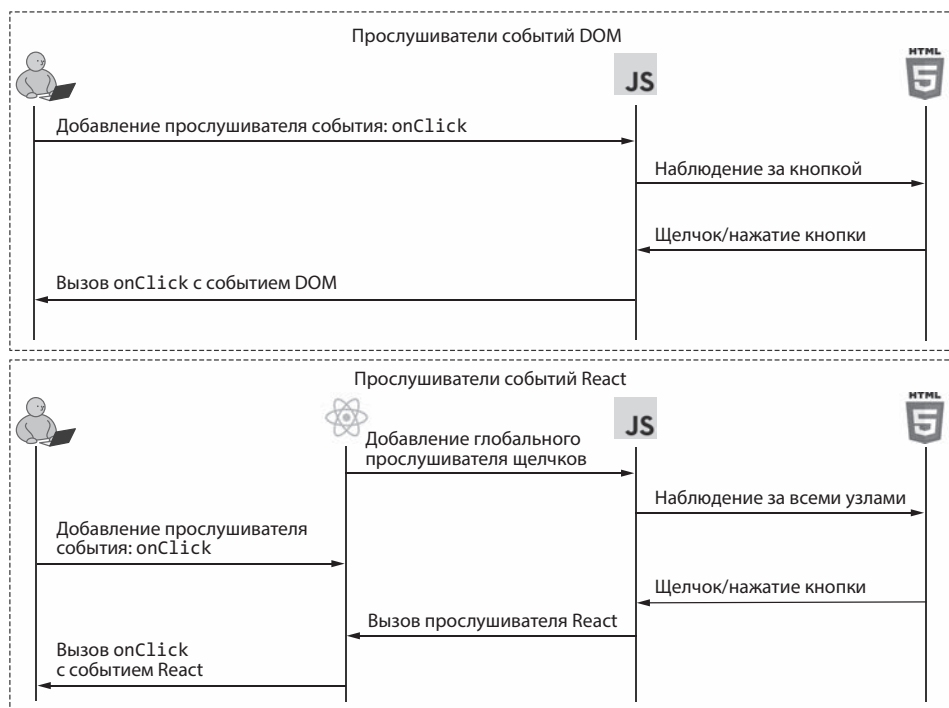


Рис. 8.4. React не добавляет прослушатель к отдельному узлу, а прослушивает все события для всех узлов, в отличие от платформенных прослушивателей DOM

Производительность

Как говорилось выше, React не добавляет прослушатели к отдельным узлам. React добавляет к документу один прослушатель для каждого типа событий, и это делается по соображениям производительности.

Выигрыш в производительности очевиден. Если добавить тысячу кнопок, а затем назначить прослушатель события щелчка для каждого узла в чистом JavaScript, это потребует значительных затрат памяти. Но если использовать для той же цели React, то React создаст только один прослушатель события щелчка для документа в целом, а потом при вызове проверит, совпадает ли цель с тем, что задал разработчик. Это значительно сокращает затраты памяти.

По этой причине вам не придется добавлять лишние прослушатели событий в React. Если вы реализуете веб-приложение на чистом JavaScript, то для сокращения количества прослушивателей потребуется создавать обходные пути. React сделает это за вас. Вам остается просто добавить столько прослушивателей,

сколько нужно, и вы можете быть уверены, что производительность приложения максимальна.

Последовательность

Несмотря на то что браузеры все больше стандартизируются, некоторые старые браузеры могут выполнять операции по-своему. Это особенно актуально для API-событий. Проблемы в основном возникают с браузерами в возрасте 5 лет и старше (старые версии Firefox, и особенно Internet Explorer 9 и более ранних версий), так что сейчас они не особенно важны, но эти браузеры все еще могут встречаться у пользователей.

Другой важный аспект последовательности довольно неожиданный. Некоторые события не стандартизованы, но при этом реализуются всеми браузерами. Например, события колесика мыши. Стандарта для этого события пока не существует, и в ближайшее время он вряд ли появится, но оно поддерживается всеми браузерами, поэтому React его тоже поддерживает. Из-за отсутствия стандарта при обработке события в разных браузерах используются отличающиеся имена. Так, в разных браузерах изменение позиции колесика прокрутки по оси x хранится в свойстве с именем `.deltaX` или `.wheelDeltaX`. Синтетическое событие колесика мыши в React решает эту проблему и всегда использует имя `.deltaX`. Аналогичная унификация применяется для других нестандартных свойств событий этого и других типов.

Таким образом, при обработке событий React вам вообще не нужно думать о различиях между браузерами. Вы можете полностью положиться на React и рассчитывать на то, что он разберется со всеми техническими деталями за вас. Так как старых версий браузеров становится все меньше и различия между браузерами постепенно исчезают, возможно, эта особенность системы синтетических событий React в какой-то момент станет неактуальна и ей на замену придут браузерные события.

API синтетических событий

Синтетические события React используют API на базе стандартной модели API, определенной в спецификации HTML. Это означает, что вы можете использовать все свойства и методы, которые ожидаете увидеть в событиях.

Все синтетические события используют набор общих свойств и методов, а более специализированные события содержат дополнительные специфические свойства. Например, все события содержат свойства `.type` и `.target`. Кроме того, все события содержат методы `.preventDefault()` и `.stopPropagation()`. Позже мы вернемся к тому, как они работают.

Отдельные типы событий также содержат дополнительные свойства, необходимые для конкретных событий, например свойства `.pageX` и `.pageY` для событий мыши и указателя, которые содержат координаты щелчков на странице.

ПРИМЕЧАНИЕ За подробной информацией о конкретных свойствах и методах обращайтесь к документации API синтетических событий React: <http://mng.bz/D4Zy>.

Доступ к платформенным событиям

Если по какой-то причине вам понадобится доступ к низкоуровневым платформенным событиям (например, если вы пишете код для конкретного браузера, который может включать в событие дополнительную информацию, полезную для приложения), вы можете обратиться к ним через свойство `.nativeEvent`. Это нестандартное свойство, которое является расширением API событий только для React.

8.2.4. Длительное хранение объектов синтетических событий

Сохранять события больше не нужно. Все, можно переходить к следующему разделу.

Погодите, что? Может показаться странным, но длительное хранение событий было актуально в React до версии 17, после которой надобность в нем отпала.

Все же, поскольку длительное хранение было часто используемой возможностью, которая устарела совсем недавно, мы рассмотрим его здесь на случай, если вы столкнетесь с ним на практике. Длительное хранение событий встречается в коде, который не был полностью обновлен при переходе на новые версии React, и даже в учебниках и руководствах по React, которые не прошли своевременного обновления.

Когда-то по соображениям производительности синтетические события React объединялись в пул, чтобы не приходилось создавать слишком много объектов одновременно. До версии 17 React не создавал новые объекты событий каждый раз, когда вызывалось событие. Вместо этого в React поддерживался внутренний пул (по сути, массив) событий; когда возникала необходимость отправки события, оно бралось из пула, а сразу после диспетчеризации события объект события возвращался в пул. Когда объект события возвращался в пул, происходила «очистка» события, то есть все его свойства сбрасывались и не имели определенного значения.

Для разработчика это означало, что событие, полученное в обработчике, нужно было *потребить* немедленно. Его нельзя было просто сохранить или отложить обращение к нему.

Представьте, что вы создаете счетчик, который может увеличиваться на значение, выбранное из раскрывающегося списка. Мы создали уже немало счетчиков, но это новая разновидность. В ней должно выводиться текущее значение счетчика (начиная с 0), а также раскрывающийся список со значениями от 1 до 5. Когда вы выбираете одно из значений, счетчик увеличивается на эту величину. При выборе нового значения раскрывающийся список снова увеличивается на новую величину, и т. д. Реализация приведена в листинге 8.5.

Листинг 8.5. Счетчик с раскрывающимся списком

```
import { useState } from "react";
function DropdownCounter() {
  const [counter, setCounter] = useState(0);
  const onChange = (evt) => setCounter(
    (value) => value + parseInt(evt.target.value)
  );
  const values = [1, 2, 3, 4, 5];
  return (
    <section>
      <h1>Counter: {counter}</h1>
      <select onChange={onChange}>
        {values.map((value) => (
          <option key={value} value={value}>
            {value}
          </option>
        ))}
      </select>
    </section>
  );
}
function App() {
  return <DropdownCounter />;
}
export default App;
```

Прибавляет выбранное значение к текущему значению счетчика в обработчике события изменения с использованием функции обновления

Назначает обработчик события элементу select

Репозиторий: rq08-persistence

Этот пример находится в репозитории `rq08-persistence`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq08-persistence --template rq08-persistence
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq08-persistence>

Решение работает, все хорошо. Однако если вы попытаете выполнить его в версиях React от 16.8 (когда появились хуки React) до 16.14 (последняя версия React 16 до появления React 17), оно работать не будет. Вместо этого `evt.target.value` выдаст ошибку в консоли, потому что значение `evt.target` не определено. Дело в том, что мы передаем функцию обновления сеттеру состояния, а эта функция обновления вызывается асинхронно. К тому моменту, когда функция будет вызвана, React уже вернет объект события в пул и очистит его, включая сброс `evt.target`. В React 16 проблему можно было решить одним из двух способов:

- сразу создать локальную копию значения из объекта события и использовать это значение асинхронно в функции обновления;
- использовать механизм длительного хранения события; в этом случае React знает, что этот конкретный объект события не следует возвращать в пул, исключает его как «одноразовый объект события» и создает другой объект события, который возвращает в пул вместо него.

Первый способ копирования нужных значений выглядит примерно так:

```
const onChange = (evt) => {
  const delta = parseInt(evt.target.value);
  setCounter((value) => value + delta);
};
```

Сначала копируем из объекта события значение, к которому необходимо обратиться позже

Затем используем это значение

Второй подход, в котором используется длительное хранение объекта события, выглядит так:

```
const onChange = (evt) => {
  evt.persist();
  setCounter((value) =>
    value + parseInt(evt.target.value));
};
```

Инструкция React: не использовать этот объект повторно, а сохранить его на неопределенное время для использования в будущем

Далее объект события может свободно использоваться даже в асинхронном коде

Вся эта суета с необходимостью длительного хранения событий при асинхронном использовании основательно раздражала. Случалось подобное не так часто и нередко приводило к путанице и ошибкам даже опытных разработчиков; это

стало одной из причин отказа от длительного хранения. Другая причина заключалась в том, что выигрыш в производительности за счет объединения событий в пулы снижался с ростом производительности устройств, так что оптимизация стала бессмысленной.

8.3. ФАЗЫ СОБЫТИЙ И РАСПРОСТРАНЕНИЕ

События не просто отправляются целевому объекту. Когда вы щелкаете на ссылке, она отправляет событие щелчка. Но если внутри ссылки находится элемент жирного текста (например, `<a href>A<strong>bold</strong> link</a>`), то на самом деле щелчок происходит на этом элементе. Затем ссылка выдает событие щелчка, потому что вы также щелкнули на элементе ссылки. Фактически вы «щелкаете» на всех родительских элементах элемента жирного текста. Этот механизм называется *распространением событий* (event propagation).

Чтобы лучше понять концепцию распространения событий, рассмотрим еще один пример. Требуется построить форму отправки обращений в службу поддержки, содержащую две области (наборы полей). Первая область содержит информацию о пользователе (имя и адрес электронной почты), а вторая — причину отправки запроса (тема и тело).

Так как форма должна быть как можно более удобной и привлекательной, мы наглядно выделяем область, в которой пользователь вводит данные в моменте. Примерный вид приложения показан на рис. 8.5.

Рис. 8.5. Готовая форма с вводом данных в обеих областях

Чтобы добиться желаемого результата, необходимо прослушивать события получения и потери фокуса для полей ввода. Когда поле получает фокус, то его область сохраняется как текущий обладатель фокуса. При потере фокуса полем никакая область фокусом не обладает. Для этого необходимо добавить два прослушивателя событий для каждого поля ввода в обеих областях. В данном примере каждая область содержит всего два поля ввода, поэтому всего используется восемь прослушивателей, — а если бы полей было намного больше? Пришлось бы повторять одни и те же два прослушивателя для каждого поля ввода. Если вам кажется, что это ужасно, вы абсолютно правы. Старайтесь избегать повторов. В данном примере это вполне возможно, потому что события всплывают!

Каждое событие в React *всплывает* по всем узлам в дереве документа, расположенным выше него. Чтобы узнать, какая область обладает фокусом, достаточно прослушивать, когда любой элемент внутри области получит фокус. Точно так же, чтобы узнать о потере фокуса элементом, достаточно знать, когда любой элемент внутри формы потеряет фокус. Используя этот прием, можно назначить прослушиватели получения фокуса для двух областей, а прослушиватель потери фокуса — для самой формы. Тогда для достижения цели хватит трех прослушивателей событий вместо восьми, которые почти не отличаются друг от друга. На рис. 8.6 показана итоговая структура JSX и места для размещения прослушивателей.

При передаче фокуса любому полю ввода событие сначала отправляется для самого поля ввода, но затем то же событие будет последовательно отправлено для каждого предка целевого элемента по порядку от родителя до корневого узла приложения React. Когда всплывающее событие достигнет набора полей, React вызовет прослушиватель события `onFocus`, размещенный в этой точке. Точно так же при потере фокуса полем ввода React вызовет прослушиватель `onBlur`, размещенный в элементе формы.

Теперь вы знаете, чего мы пытаемся добиться и как должен выглядеть код JSX. Остается лишь собрать все вместе в одном компоненте. Реализация приведена в листинге 8.6.

Листинг 8.6. Выделение областей формы

```
import { useState } from "react";
const FOCUS_NONE = 0;
const FOCUS_USER = 1;
const FOCUS_REQUEST = 2;
function getStyle(isActive) {
  return {
    display: "flex",
    flexDirection: "column",
```

Сначала добавляем вспомогательную функцию, которая генерирует стиль для области в зависимости от того, активна область или нет

```

    backgroundColor: isActive ? "oldlace" : "transparent",
  };
}
function Field({ label, children }) {
  return (
    <label>
      {label}:
      <br />
      {children}
    </label>
  );
}
function Contact() {
  const [focus, setFocus] = useState(FOCUS_NONE);
  const onUserFocus = () => setFocus(FOCUS_USER);
  const onRequestFocus = () => setFocus(FOCUS_REQUEST);
  const onBlur = () => setFocus(FOCUS_NONE);
  return (
    <form onBlur={onBlur}>
      <h1>Contact</h1>
      <fieldset>
        <div>
          onFocus={onUserFocus}
          style={getStyle(focus === FOCUS_USER)}
        </div>
        <legend>User</legend>
        <Field label="Name">
          <input />
        </Field>
        <Field label="Email">
          <input type="email" />
        </Field>
      </fieldset>
      <fieldset>
        <div>
          onFocus={onRequestFocus}
          style={getStyle(focus === FOCUS_REQUEST)}
        </div>
        <legend>Request</legend>
        <Field label="Subject">
          <input />
        </Field>
        <Field label="Body">
          <textarea />
        </Field>
      </fieldset>
    </form>
  );
}
function App() {
  return <Contact />;
}
export default App;

```

Затем необходимо запомнить, какая область обладает фокусом непосредственно сейчас (в начале ни одна область фокусом не обладает)

Создает три очень простых прослушателя

Назначает стиль каждой области в зависимости от того, обладает она фокусом или нет

Назначает прослушатели там, где они нужны

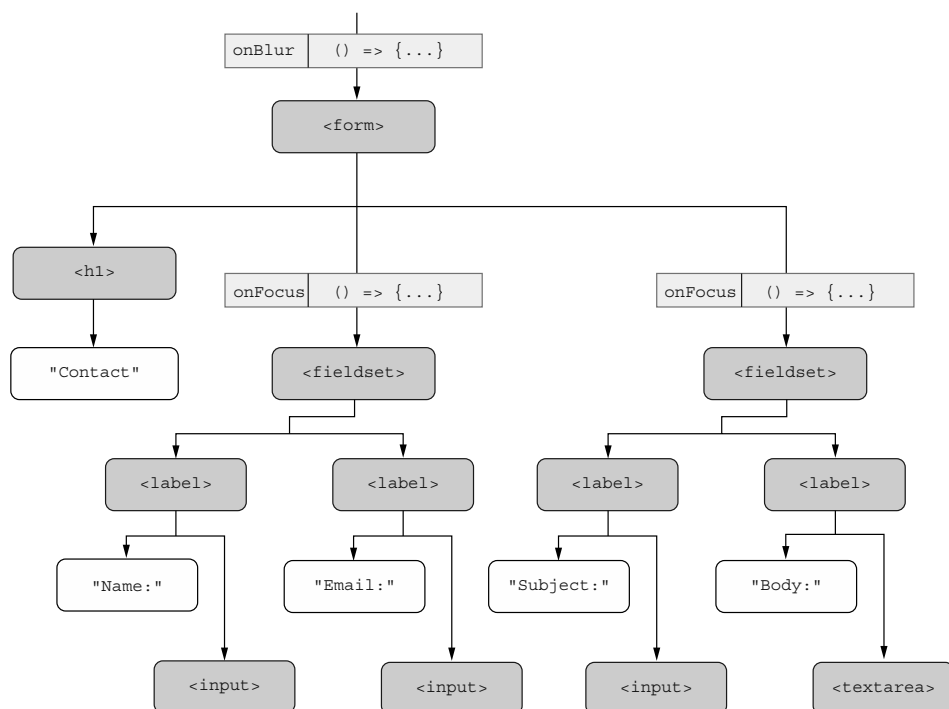


Рис. 8.6. Мы добавляем прослушиватель потери фокуса ко всей форме и прослушиватели получения фокуса для обоих наборов полей. Когда в любом поле ввода происходит событие (нижний ряд), событие перемещается вверх по дереву и обрабатывается подходящим обработчиком

Репозиторий: rq08-contact

Этот пример находится в репозитории rq08-contact. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq08-contact --template rq08-contact
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq08-contact>

Готово! Мы создали форму с оформлением стилем и с нетипичными прослушивателями передачи фокуса. Запустив код в браузере, вы получите вывод, который был показан на рис. 8.5.

В оставшейся части раздела мы более подробно обсудим технические детали распространения событий. Сначала рассмотрим события HTML и JavaScript вообще, а потом — события React в частности.

8.3.1. Как фазы и распространение работают в браузере

Как уже говорилось, события React поднимаются, или всплывают, по дереву элементов. События HTML тоже всплывают. При щелчке на кнопке каждый предок кнопки получит событие. А точнее, два события: одно *до* целевого элемента и одно *после* целевого элемента.

ПРИМЕЧАНИЕ Этот подраздел посвящен событиям HTML в целом, а не конкретно событиям React. Мы начинаем с этой темы, чтобы вы лучше поняли, как работают события в React. Далее мы обсудим фазы событий в React, которые немного отличаются.

Выше мы говорили о *всплытии* (bubbling) событий — о том, что происходит, когда предки отправляют событие *после* того, как оно отправлено целевому элементу. Помимо всплытия, существует и *погружение* (capturing) событий — то, что происходит *перед* передачей события целевому элементу. Схема диспетчеризации событий включает три фазы:

- *Фаза погружения* — события передаются всем родительским узлам DOM сверху вниз, начиная с элемента окна, через всех предков и завершая родителем целевого элемента.
- *Фаза цели* — событие передается самому целевому элементу.
- *Фаза всплытия* — события передаются всем родительским узлам DOM снизу вверх, начиная с родителя целевого элемента, через предков к элементу окна.

Эти процессы изображены на рис. 8.7. Вся концепция называется *распространением события*. Событие сначала распространяется через фазу погружения «вниз» к целевому элементу, а потом продолжает распространяться «вверх» к объекту окна в фазе всплытия.

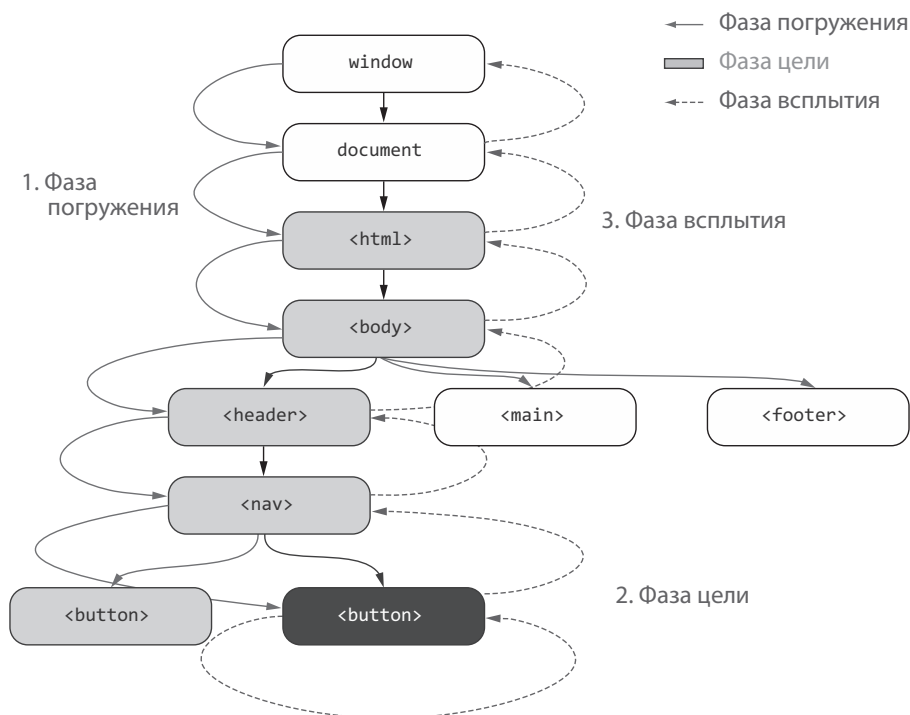


Рис. 8.7. По щелчку на черной кнопке браузер начинает распространять события по узлам документа, начиная с window и перемещаясь вниз по дереву документа в фазе погружения до цели, а затем перемещается вверх по дереву в фазе всплытия, пока не достигнет window

Если вы хотите прослушивать событие для конкретного элемента, можно указать, в какой фазе должно вестись прослушивание. По умолчанию событие прослушивается в фазе всплытия и цели, но можно добавить аргумент, чтобы события прослушивались в фазе погружения.

В JavaScript для добавления прослушивателя (фазы всплытия и цели) достаточно вызвать `addEventListener` с событием и функцией обратного вызова:

```
element.addEventListener("click", onClick);
```

Если требуется использовать прослушиватель в фазе погружения, необходимо добавить третий аргумент:

```
element.addEventListener("click", onClick, { capture: true });
```


Когда вы получаете событие, проверьте свойство `.eventPhase` объекта события, чтобы определить, к какой фазе он относится. Допустимые значения:

- `Event.CAPTURING_PHASE` (1) для фазы погружения.
- `Event.AT_TARGET` (2) для фазы цели.
- `Event.BUBBLING_PHASE` (3) для фазы всплытия.

В примере на рис. 8.7 будут отправлены 14 потенциальных событий в следующем порядке:

1. Фаза погружения:
 - a) Событие погружения передается в `window`.
 - b) Событие погружения передается в `document`.
 - c) Событие погружения передается элементу `<html>`.
 - d) Событие погружения передается элементу `<body>`.
 - e) Событие погружения передается элементу `<header>`.
 - f) Событие погружения передается элементу `<nav>`.
2. Фаза цели:
 - a) Целевое событие (зарегистрированное в качестве прослушивателя погружения) передается элементу `<button>`.
 - b) Целевое событие (зарегистрированное в качестве прослушивателя всплытия) передается элементу `<button>`.
3. Фаза всплытия:
 - a) Событие всплытия передается элементу `<nav>`.
 - b) Событие всплытия передается элементу `<header>`.
 - c) Событие всплытия передается элементу `<body>`.
 - d) Событие всплытия передается элементу `<html>`.
 - e) Событие всплытия передается в `document`.
 - f) Событие всплытия передается в `window`.

События 2.a и 2.b могут показаться знакомыми, но они будут сгруппированы сначала для передачи всем прослушивателям, определенным как прослушиватели погружения, а затем для передачи всем прослушивателям, определенным как прослушиватели всплытия. Конечно, одно событие для одной цели может

прослушиваться сразу несколькими прослушивателями. В этом случае события будут передаваться в порядке назначения прослушивателей.

На рис. 8.8 изображено упрощенное представление предыдущей схемы, включающее только три элемента, расположенных в порядке «сверху вниз».



Рис. 8.8. Три элемента в порядке «сверху вниз»

Представим, что мы добавили несколько прослушивателей к разным элементам в следующем порядке:

- A. Прослушиватель погружения добавляется к элементу `<nav>`.
- B. Прослушиватель всплытия добавляется к элементу `<button>`.
- C. Прослушиватель погружения добавляется к элементу `<button>`.
- D. Прослушиватель погружения добавляется к элементу `<header>`.
- E. Прослушиватель погружения добавляется к элементу `<nav>` (повторно).
- F. Прослушиватель погружения добавляется к элементу `<button>` (повторно).
- G. Прослушиватель всплытия добавляется к элементу `<nav>`.

Эти восемь прослушивателей (от A до G) будут вызываться в следующем порядке:

1. Прослушиватели погружения для `<header>`: D (`eventPhase=CAPTURING_PHASE`).
2. Прослушиватели погружения для `<nav>`: A, E (`eventPhase=CAPTURING_PHASE`).
3. Прослушиватели погружения для `<button>`: C, F (`eventPhase=AT_TARGET`).
4. Прослушиватели всплытия для `<button>`: B (`eventPhase=AT_TARGET`).
5. Прослушиватели всплытия для элемента `<nav>`: G (`eventPhase: BUBBLING_PHASE`).
6. Прослушиватели всплытия для элемента `<header>`: —.

Хотя прослушиватели самого целевого элемента будут срабатывать в порядке «сначала прослушиватели погружения, затем прослушиватели всплытия», все они будут вызваны в фазе события `AT_TARGET`, а не в фазах погружения и всплытия соответственно.

8.3.2. Обработка фаз событий в React

События в React не добавляются при добавлении прослушивателя к узлу с использованием метода. События в React добавляются назначением свойства элементу JSX, представляющему узел. Из-за этого невозможно добавить аргумент для передачи информации о том, в какой фазе должно прослушиваться событие. В React, как и в JavaScript, по умолчанию события добавляются как прослушиватели всплытия. Когда вы используете запись

```
<main onClick={onClickHandler}>
  ...
</main>
```

`onClickHandler` будет добавлен как прослушиватель фазы всплытия. Если вы хотите добавить прослушиватель события погружения, укажите после имени события суффикс `*Capture`. Например, для обработчика щелчка получается имя `onClickCapture`. Таким образом, во фрагменте

```
<main onClickCapture={handler1} onClick={handler4}>
  <button onClickCapture={handler2} onClick={handler3} />
</main>
```

обработчики будут вызваны в порядке `handler1`, `handler2`, `handler3` и затем `handler4`.

Обработчики погружения используются довольно редко. Скорее всего, вы никогда с ними не встретитесь или используете их пару раз в очень большом приложении, однако это отличный инструмент, который полезно иметь под рукой на случай, когда он действительно понадобится. Это своего рода кухонный комбайн от React — используется редко, но когда используется, это именно то, что нужно!

8.3.3. Нетипичное распространение событий

Четыре типа событий используют очень необычную логику распространения в React. Речь идет о парах `mouseenter/mouseLeave` и `pointerEnter/pointerLeave`. Эти события образуют пары, так как мышь или указатель входят в один элемент, выходя из другого элемента. Они всплывают от покинутого элемента к новому элементу и никогда не погружаются.

Детали, если они вам понадобятся, можно узнать в этой статье: <https://barklund.dev/mouseevents>. Впрочем, такие случаи крайне специфичны, и скорее всего, никогда вам не встретятся.

8.3.4. Невсплывающие события DOM

В DOM некоторые события вообще не всплывают, но при этом погружаются. Это события потери и получения фокуса. Однако в React для простоты оба события всплывают, как любые другие события. Представим следующую структуру в React:

```
<label onFocusCapture={handler1} onFocus={handler3}>  
  <input onFocus={handler2} />  
</label>
```

Если переместить курсор в поле ввода, три обработчика событий сработают в следующем порядке: `handler1`, `handler2` и `handler3`. Если реализовать то же самое без React и добавить прослушиватели событий с использованием JavaScript, `handler3` никогда не сработает, потому что у события этого типа фазы всплытия нет.

У этого явления в HTML есть технические обоснования, но так как оно сбивает с толку разработчиков (и о нем очень легко забыть), в React такие события просто всплывают, как и остальные. Разработчики React могут об этом не беспокоиться; события получения и потери фокуса используются как обычные события, что мы уже делали в начале раздела.

Почему события получения и потери фокуса не всплывают в HTML

Когда окно теряет фокус (пользователь перешел на другую вкладку в браузере или даже переключился на другую программу), объекту окна будет отправлено событие потери фокуса. Точно так же при получении фокуса окном, когда пользователь вернулся в окно/на вкладку браузера, объекту окна будет передано событие получения фокуса.

Если бы событие получения или потери фокуса для поля ввода или кнопки всплывало, то ему пришлось бы всплывать до объекта окна и разработчик мог бы перепутать событие с получением/потерей фокуса окном в результате действий пользователя. Два события можно было бы отличить по свойству `.target` события, но по принятому соглашению эти события просто не всплывают.

В React это не создает проблем, потому что прослушиватель событий React нельзя назначить объекту окна. Прослушиватели событий React могут назначаться только реальным элементам HTML (а объект `window` им не является) и только содержащимся внутри приложения (внутри элемента `<body>`). По этой причине события получения и потери фокуса в React всплывают.

8.4. ДЕЙСТВИЯ ПО УМОЛЧАНИЮ И КАК ИХ ИЗБЕЖАТЬ

Браузеры выполняют действия по умолчанию как следствие некоторых событий. В большинстве случаев эти действия полезны, но иногда — нет. В следующем примере будет показано такое нежелательное действие по умолчанию и то, как его избежать.

Представьте, что вы создаете в React форму входа для администратора. Форма содержит поле для ввода пароля и кнопку входа. Когда пользователь щелкает на кнопке, код должен проверить, совпадает ли введенный пароль с секретной строкой "platypus". Если строки совпадают, то любая секретная информация внутри приложения должна стать доступной для очевидно доверенного администратора. Начнем с листинга 8.7.

Листинг 8.7. Форма для администратора (с ошибкой?)

```
import { useState } from "react";
function Admin() {
  const [password, setPassword] = useState("");
  const [isAdmin, setAdmin] = useState(false);
  const onClick = () => {
    if (password === "platypus") {
      setAdmin(true);
    }
  };
  return (
    <>
      {isAdmin && <h1>Bacon is delicious!</h1>}
      <form>
        <input
          type="password"
          onChange={
            (evt) => setPassword(evt.target.value)
          }
        />
        <button onClick={onClick}>
          Login
        </button>
      </form>
    </>
  );
}
function App() {
  return <Admin />;
}
export default App;
```

Сохраняет введенный пароль в значении состояния

Сохраняет признак успешной проверки пользователя как администратора в другом значении состояния

Когда пользователь щелкает на кнопке, проверяет, совпадает ли введенный пароль с ожидаемым, и в случае совпадения обновляет состояние

Выводит условный JSX в зависимости от того, прошел ли пользователь проверку на администратора

Поле ввода обновляет пароль при изменении

По щелчку кнопки вызывается обработчик события

Но если запустить браузер, ввести что-нибудь в поле ввода и щелкнуть на кнопке, происходит неожиданное. Страница перезагружается, и поле ввода очищается. Это совсем не то, что нужно. В чем же дело?

8.4.1. Действие события по умолчанию

Если создать на веб-странице форму HTML с кнопкой и щелкнуть на этой кнопке, страница перезагрузится. Это поведение используется по умолчанию в HTML.

Предположим, вы сохранили следующую разметку HTML (обратите внимание: сейчас речь идет о простом HTML, а не о JSX) в файле и открыли файл в браузере:

```
<form>
  <button>Click me</button>
</form>
```

Щелчок на кнопке перезагружает страницу. Дело в том, что кнопка в форме вызывает отправку данных формой, а при отправке данных переменные внутри формы отправляются целевому URL-адресу формы. Это происходит, даже если в форме нет полей ввода и если у формы нет явно заданного целевого адреса (целевым URL-адресом по умолчанию является сама страница).

Зная это, мы понимаем, что не так с формой. Кнопка в приложении отправляет данные формы, а отправка данных формы по умолчанию инициирует перезагрузку страницы.

8.4.2. Как избежать выполнения действий по умолчанию

Чтобы форма работала правильно, необходимо внести два изменения. Во-первых, переместить обработчик события из щелчка на кнопке в отправку формы. Обработчик остается без изменений, мы просто назначаем его свойству `onSubmit` формы — вместо свойства `onClick` кнопки.

Во-вторых, необходимо предусмотреть, чтобы форма не выполняла действие по умолчанию, которое обычно совершается при отправке данных. Для этого мы вызываем `evt.preventDefault()` для объекта события, передаваемого обработчику. Реализация приведена в листинге 8.8.

Листинг 8.8. Форма для администратора (вроде бы исправленная?)

```
import { useState } from "react";
function Admin() {
  const [password, setPassword] = useState("");
  const [isAdmin, setAdmin] = useState(false);
  const onSubmit = (evt) => {
    evt.preventDefault();
    if (password === "platypus") {
      setAdmin(true);
    }
  };
  return isAdmin ? (
    <h1>Bacon is delicious!</h1>
  ) : (
    <form onSubmit={onSubmit}>
      <input
        type="password"
        onChange={(evt) => setPassword(evt.target.value)}
      />
      <button>Login</button>
    </form>
  );
}
function App() {
  return <Admin />;
}
export default App;
```

Принимает объект события в аргументе обработчика события, чтобы избежать действия по умолчанию

Вызывает метод `evt.preventDefault` в обработчике `submit` независимо от того, что еще происходит в обработчике

Связывает обработчик события с элементом формы

Репозиторий: rq08-admin

Этот пример находится в репозитории `rq08-admin`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq08-admin --template rq08-admin
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq08-admin>

Теперь форма работает как задумано. Мы избежали выполнения события по умолчанию в форме, и в результате встроенный в браузер обработчик события не активировался. Результат показан на рис. 8.9.

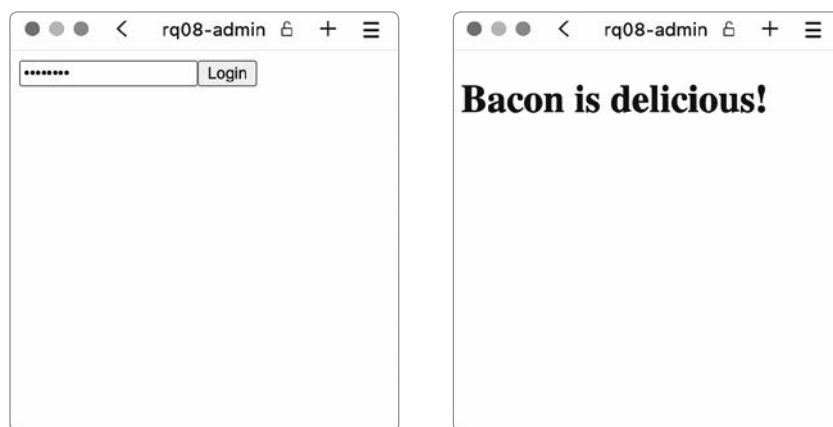


Рис. 8.9. Форма для администратора работает, и при вводе верного пароля перед пользователем открываются все тайны мироздания

Если нажать Enter в момент, когда фокус находится в поле ввода в форме, также содержащей кнопку **Submit**, будут отправлены данные формы. Если назначить обработчик события кнопке в виде `onClick`, отправка формы нажатием Enter с фокусом в поле ввода работать не будет, а страница перезагрузится из-за отправки формы. Перемещение обработчика в событие отправки данных формы позволяет обработать оба способа отправки формы.

ПРИМЕЧАНИЕ Конечно, этот пример нарушает все мыслимые стандарты веб-безопасности. Все, что происходит в React, может прочитать любой посетитель веб-страницы, и такая схема будет взломана за секунды. Обязательно используйте надлежащую архитектуру для создания безопасных схем входа.

8.4.3. Другие события по умолчанию

Для событий отправки данных формы действием по умолчанию является отправка всех значений, введенных на форме, по целевому URL-адресу. Это одно из событий по умолчанию, используемых в браузере, но, конечно, не единственное.

Щелчок по ссылке создает событие щелчка для элемента ссылки. Действием по умолчанию для этого события становится переход по ссылке и новому URL-адресу, заданному свойством `href`. И снова этого поведения по умолчанию можно избежать вызовом метода `.preventDefault()` для объекта события щелчка, полученного обработчиком. Это будет означать, что браузер не перейдет по целевому URL-адресу и просто ничего не произойдет.

Чтобы узнать, можно ли отменить событие, проверьте свойство `.cancellable`. Если значение равно `true`, можно вызвать `.preventDefault()`, чтобы прервать выполнение действий по умолчанию в браузере. Если значение равно `false`, вызов `.preventDefault()` возможен, но он ничего не сделает. Ниже перечислены основные события, которые можно отменить.

- События прокрутки — при отмене прокрутка не выполняется, а смещение прокрутки остается неизменным.
- События нажатия, удержания и отпускания клавиши — при отмене символ не вставляется (при вызове для поля ввода или текстового поля) или происходит то, что должно происходить при ненажатии клавиши (например, при отмене нажатия Page Up браузер не прокручивает страницу). С другой стороны, события ввода нельзя отменить при передаче постфактум (например, если пользователь что-то ввел или вставил текст из буфера).
- События начала перетаскивания и входа при перетаскивании — при отмене, соответственно, перетаскивание не выполняется вовсе или эффект перетаскивания не меняется. С другой стороны, нельзя отменить события выхода при перетаскивании и завершения перетаскивания.

В React действия по умолчанию и их отмена работают по той же схеме, что и в HTML. Поэтому узнать, какие события можно отменить и какие действия по умолчанию существуют, можно из любого электронного руководства HTML.

8.5. ОБЩИЕ СВЕДЕНИЯ ОБ ОБЪЕКТАХ СОБЫТИЙ REACT

Мы рассмотрели некоторые способы использования объекта события, который React отправляет обработчику события. В табл. 8.2 представлены свойства, общие для всех объектов событий. Многие из них подробно рассматриваются в этой главе. Это не все свойства объектов событий, но, на наш взгляд, самые важные из них.

Таблица 8.2. Важные свойства, общие для всех объектов событий в React и HTML

Свойство	Назначение
bubbles	Логическое значение всплытия события.
cancelable	Логическое значение возможности отмены события.
eventPhase	Числовое значение, указывающее, к какой фазе распространения принадлежит событие.

Таблица 8.2 (окончание)

Свойство	Назначение
preventDefault	Метод, запрещающий браузеру обрабатывать событие действием по умолчанию.
stopPropagation	Метод, запрещающий дальнейшее распространение события.
target	Целевой узел, для которого предназначено событие.
timestamp	Время создания события в миллисекундах.
type	Тип события, инициировавшего передачу объекта события.

8.6. ФУНКЦИИ ОБРАБОТЧИКОВ СОБЫТИЙ ИЗ СВОЙСТВ

При создании повторно используемых UI-элементов крайне важно придать им унифицированный внешний вид, чтобы их можно было использовать в других местах, не меняя каждый раз стилевое оформление. Создадим обобщенный компонент кнопки со стилевым оформлением, который можно неоднократно использовать в приложениях. Применим этот компонент для создания счетчика с кнопками **Increment** и **Decrement**, но со стилевым оформлением. Результат должен выглядеть, как показано на рис. 8.10, — обратите внимание на стильные кнопки. Структура приложения изображена на диаграмме JSX на рис. 8.11.



Рис. 8.10. Готовое приложение со счетчиком, увеличенным в несколько раз. Новые кнопки выглядят привлекательнее привычных стандартных, не так ли?

Как видите, мы передаем каждому экземпляру компонента кнопки функцию `handleClick`, которая внутри должна быть назначена обработчиком события щелчка кнопки. Реализация приведена в листинге 8.9.

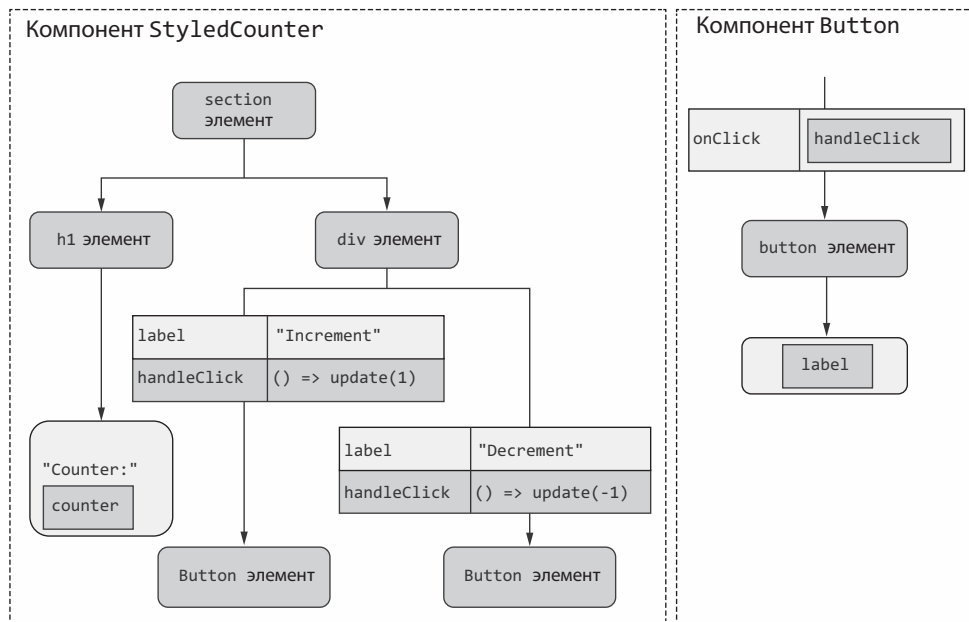


Рис. 8.11. Приложение включает два экземпляра компонента кнопки с немного различающимися свойствами

Листинг 8.9. Счетчик со стиливым оформлением кнопок

```

import { useState } from "react";
function Button({ handleClick, label }) {
  const buttonStyle = {
    color: "blue",
    border: "1px solid",
    background: "transparent",
    borderRadius: ".25em",
    padding: ".5em 1em",
    margin: ".5em",
  };
  return (
    <button style={buttonStyle} onClick={handleClick}>
      {label}
    </button>
  );
}
  
```

Назначает полученное свойство `handleClick` обработчиком события `onClick` внутри компонента кнопки

```
function StyledCounter() {
  const [counter, setCounter] = useState(0);
  const update = (d) => setCounter((v) => v + d);
  return (
    <section>
      <h1>Counter: {counter}</h1>
      <div>
        <Button
          handleClick={() => update(1)}
          label="Increment"
        />
        <Button
          handleClick={() => update(-1)}
          label="Decrement"
        />
      </div>
    </section>
  );
}
function App() {
  return <StyledCounter />;
}
export default App;
```



По щелчку на кнопках назначает свойству handleClick функцию, обновляющую состояние

Репозиторий: rq08-styled-counter

Этот пример находится в репозитории rq08-styled-counter. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq08-styled-counter --template rq08-styled-counter
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq08-styled-counter>

Выглядит неплохо, и в целом у нас получилось хорошее компактное структурированное решение. И все же что-то в нем смущает. При использовании компонентов-кнопок мы назначаем функцию, которая должна вызываться щелчком на кнопке. Тем не менее в листинге 8.9, поскольку мы назначаем функцию напрямую в свойстве onClick кнопки, функция вызывается с объектом события как первым и единственным аргументом. Иногда это хорошо, а иногда не очень. Объект события не должен быть доступен для внешнего компонента, потому что это подробность внутренней реализации компонента-кнопки.

Чтобы исключить это событие из вызова функции, необходимо создать другую функцию как обработчик события и при ее вызове вызвать свойство `handleClick` (без аргументов). Примерно так:

```
function Button({ handleClick, label }) {
  const buttonStyle = {...};
  const onClick = () => handleClick();
  return (
    <button style={buttonStyle} onClick={onClick}>
      {label}
    </button>
  );
}
```

По-прежнему получает свойство `handleClick`

Теперь создает локальную функцию, при вызове которой вызывается переданное свойство

Назначает обработчиком события эту локальную функцию, а не переданное свойство

Обратите внимание: прослушивателю события присвоено имя свойства `handle*`. Это обычная практика при передаче функций как свойств элементам, которые сами по себе являются не прослушивателями, а обратными вызовами; они вызываются прослушивателями событий или эффектами по необходимости. Этому свойству также можно присвоить имя `onClick`, но тогда оно будет выглядеть как прослушиватель события и пользователи будут ожидать, что оно работает как прослушиватель события (и ему в аргументе должен передаваться объект события).

Примеры свойств функций, вызываемых как обратные вызовы (либо напрямую как прослушиватели событий, либо внутри прослушивателей событий), в коде не редкость, так как это обычный способ проектирования библиотек повторно используемых компонентов UI. Подобная структура будет использоваться и в следующих главах. Мы будем именовать непосредственные обработчики событий (получающие объект события) по модели `on*`, а обратные вызовы (которые либо не получают никаких аргументов, либо получают нестандартные аргументы) по модели `handle*`.

8.7. ГЕНЕРАТОРЫ ОБРАБОТЧИКОВ СОБЫТИЙ

Если вы используете несколько функций обработчиков событий, между которыми не так много отличий, возможно, их стоит обобщить в *генератор обработчиков событий*. Вернемся к примеру счетчика с кнопками `Increment` и `Decrement`. Две разные функции были приведены к одной общей функции, которая обновляет значение на основании аргумента, после чего *эта* функция вызывалась с разными аргументами в обработчиках событий щелчков на двух кнопках:

```
function Counter() {
  const [counter, setCounter] = useState(0);
  const update = (delta) =>
    setCounter((c) => c + delta);
}
```

Обобщенная функция для обновления счетчика значением `delta`

```

return (
  <>
    <h1>Value: {counter}</h1>
    <button onClick={() => update(1)}> Increment </button>
    <button onClick={() => update(-1)}> Decrement </button>
  </>
);
}

```

← update вызывается с двумя разными значениями в обработчиках событий

Эту концепцию можно развить еще дальше. Обратите внимание: в обоих обработчиках событий все еще определяется функция, которая затем вызывает `update` (в обоих случаях используется стрелочное определение вида `() => update`). Это определение функции можно переместить в функцию `update` с применением *карированной функции* (curried function). Функция `update` преобразуется в *генератор* обработчиков событий, который возвращает обработчик события при вызове. Таким образом, это функция, которая возвращает другую функцию:

```

function Counter() {
  const [counter, setCounter] = useState(0);
  const update = (delta) => () =>
    setCounter((c) => c + delta);
  return (
    <>
      <h1>Value: {counter}</h1>
      <button onClick={update(1)}>Increment</button>
      <button onClick={update(-1)}>Decrement</button>
    </>
  );
}

```

Обобщенный генератор обработчиков событий для обновления счетчика значением delta

Вызов генератора обработчиков событий генерирует обработчик события с конкретным значением delta

Решение с генератором довольно экзотическое, но понимать всю его логику и не надо. Просто учтите, что этот подход применяется довольно многими разработчиками и часто встречается на практике. Мы вернемся к использованию генераторов обработчиков событий в следующей главе, посвященной обработке событий в формах, где вы еще раз потренируетесь применять эту концепцию.

8.8. РУЧНОЕ ПРОСЛУШИВАНИЕ СОБЫТИЙ DOM

Иногда требуется прослушивать события узлов, которые не находятся под прямым контролем React, или вручную управлять тем, нужно ли прослушивать события вообще. В обеих ситуациях можно прослушивать события напрямую

для узлов DOM в обычном коде JavaScript, обходя настройку прослушивателей React. Несколько примеров ситуаций, в которых вам может потребоваться ручное управление событиями:

- Прослушивание событий объекта окна или документа.
- Прослушивание событий узлов HTML, не включаемых напрямую в приложения React, — например, для `body`, который никогда не находится в приложениях React. Впрочем, это может быть и другой узел, не под управлением приложения React.
- Прослушивание событий объектов, не входящих в DOM, например запросов, сокетов или любых других объектов JavaScript.
- Прослушивание одного события конкретного узла, когда вас интересует только один экземпляр возникающего события.
- Условное прослушивание события узла.

Первые три примера реализуются только прямым прослушиванием событий узлов, но два последних можно реализовать в React. Однако в обоих случаях потребуются много дополнительной работы. В следующем подразделе вы узнаете, как применить ручное прослушивание событий DOM с выходом за пределы архитектуры React в каждой из ситуаций, представленных в списке.

8.8.1. Прослушивание событий окна и документа

Предположим, вам требуется вывести размер окна браузера в приложении. Чтобы вывести размер окна браузера при первом рендеринге компонента, можно использовать свойства `window.innerWidth` и `window.innerHeight`. Но если пользователь изменит размеры окна во время монтирования компонента, повторный рендеринг не произойдет и выводимое значение не обновится.

Чтобы компонент обновлялся при изменении размера окна, необходимо прослушивать событие изменения размеров для объекта окна. Так как этим событием React не управляет, необходимо присоединить прослушиватель напрямую к объекту окна вызовом `window.addEventListener`. Кроме того, необходимо проследить, чтобы в случае демонтирования компонента прослушиватель события был удален вызовом `window.removeEventListener`.

Если вспомнить, что говорилось в главе 6 о жизненных циклах компонентов, может показаться, что ситуация идеально подходит для хука `useEffect`, — и так и есть! Мы объединим его с хуком `useState` для получения компонента, логика которого изображена на рис. 8.12. Реализация приведена в листинге 8.10.

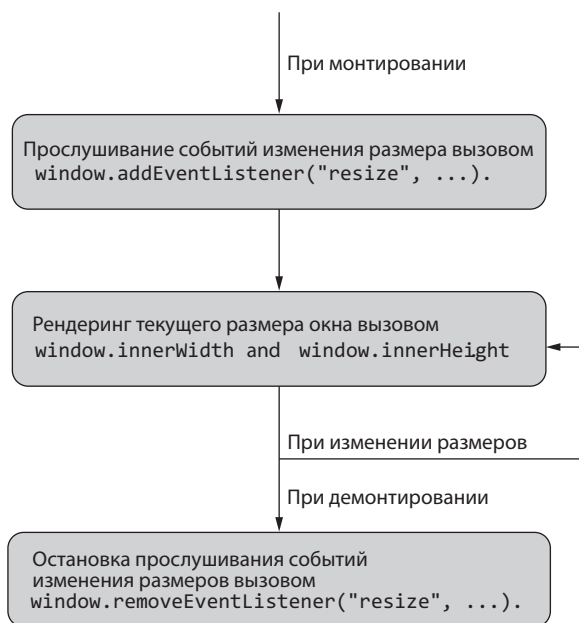


Рис. 8.12. Компонент `WindowSize` должен добавить прослушиватель при монтировании и удалить его при демонтировании. Пока компонент остается монтированным, браузер активирует обратный вызов при изменении размеров окна

Листинг 8.10. Вывод размеров окна

Настраивает хук эффекта

```
import { useState, useEffect } from "react";
function getWindowSize() {
  return `${window.innerWidth}x${window.innerHeight}`;
}
```

Сначала определяем небольшую вспомогательную функцию для получения выводимого текста размеров окна браузера

```
function WindowSize() {
  const [size, setSize] = useState(getWindowSize());
```

Использует вспомогательную функцию для инициализации значения состояния

```
  useEffect(() => {
    const onResize = () => setSize(getWindowSize());
    window.addEventListener("resize", onResize);
```

Функция назначается прослушивателем события непосредственно для объекта окна

Внутри этого хука определяем функцию, которая будет вызываться при изменении размеров окна

```
    return () =>
      window.removeEventListener("resize", onResize);
  }, [setSize]);
  return <h1>Window size: {size}</h1>;
```

Так как это хук эффекта, необходимо настроить зависимости. В данном случае это только функция `setSize`, которая заведомо стабильна, но мы все равно включаем ее для прозрачности

```
}
function App() {
  return <WindowSize />;
}
export default App;
```

Рендерит фактический размер окна в возвращаемом коде JSX

Хук эффекта возвращает функцию очистки, которая снова удаляет прослушиватель

Репозиторий: rq08-window-size

Этот пример находится в репозитории `rq08-window-size`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq08-window-size --template rq08-window-size
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq08-window-size>

Примерный вид приложения, запущенного в браузере, изображен на рис. 8.13.

Это очень простой пример прослушивания событий для постоянного объекта, такого как окно или документ. Такой подход часто применяется для событий, которые случаются только с этими двумя объектами, или же для перехвата всех событий определенного типа, которые всплывают до этих уровней.



Рис. 8.13. Приложение для вывода размеров окна в небольшом окне

Обратите внимание на интересное применение функции очистки в `useEffect`. Так как мы определяем функцию прослушателя внутри эффекта, добавляем прослушатель в эффект и — в случае, если его нужно удалить, — снова удаляем его (эта структура работает без учета зависимостей), а также в том случае, если функция монтируется и демонтируется несколько раз.

Однако стоит заметить, что поскольку мы не используем прием с однократным прослушиванием событий и ручным сохранением информации о том, кто что прослушивает, мы добавляем прослушатель в объект окна для каждого экземпляра компонента. Если этот элемент входит в длинный список похожих

элементов, мы будем добавлять новый прослушиватель для каждого нового элемента. Такой способ выглядит нелогично, если можно просто добавить один прослушиватель. При прямом добавлении событий к узлам DOM иногда приходится поработать над оптимизацией.

8.8.2. Неподдерживаемые события HTML

А теперь посмотрим, как прослушивать события DOM, которые не поддерживаются React, например события перехода. Это события назначения, запуска, завершения или отмены перехода CSS. Из этих четырех событий только событие завершения напрямую поддерживается в React с использованием свойства `onTransitionEnd`.

Создадим компонент, содержащий элемент с переходом. Текст должен выводиться переходом от красного к синему и обратно. Переход запускается двумя разными кнопками, которые задают цвет непосредственно для узла HTML при помощи объекта стиля данного узла. Затем должен выводиться заголовок с информацией о том, работает переход или нет.

Хотя событие `transitionend` можно прослушивать в React с использованием свойства `onTransitionEnd`, прослушивать событие `transitionstart` аналогичным образом не удастся. Поэтому для простоты будем прослушивать оба события с использованием обычного прослушивателя DOM — см. листинг 8.11. Результат выполнения кода показан на рис. 8.14.



Рис. 8.14. Щелкая на двух кнопках, можно увидеть, как цвет текста меняется с красного на синий и обратно на красный, а заголовок показывает, воспроизводится анимация или нет. Стоит заметить, что если щелкнуть на кнопке Go Red в тот момент, когда текст уже стал красным, переход не запускается, а заголовок не меняется

Листинг 8.11. События перехода

Также понадобится локальная переменная со ссылкой на элемент DOM, чтобы к элементу можно было обратиться из функции очистки

```
import { useState, useRef, useEffect } from "react";
function Transition() {
  const [isRunning, setRunning] = useState(false);
  const div = useRef();
  useEffect(() => {
    const onStart = () => setRunning(true);
    const onEnd = () => setRunning(false);
    const node = div.current;
    node.addEventListener("transitionstart", onStart);
    node.addEventListener("transitionend", onEnd);
    return () => {
      node.removeEventListener(
        "transitionstart", onStart
      );
      node.removeEventListener("transitionend", onEnd);
    };
  }, [setRunning]);
  return (
    <section>
      <h1>Transition is {!isRunning && "not"} running</h1>
      <div
        style={{ color: "red", transition: "color 1s linear" }}
        ref={div}
      >
        COLORFUL TEXT
      </div>
      <button onClick={() => (div.current.style.color = "blue")}>
        Go blue
      </button>
      <button onClick={() => (div.current.style.color = "red")}>
        Go red
      </button>
    </section>
  );
}
function App() {
  return <Transition />;
}
export default App;
```

Из-за необходимости ссылаться на элемент HTML используется хук useRef

Создает в хуке эффекта два обратных вызова, которые будут использоваться в качестве прослушивателей

Добавляет прослушиватели в хук эффекта непосредственно для элемента DOM

Задаёт свойству ref ссылку на целевой элемент

Удаляет те же прослушиватели из того же объекта при очистке

Репозиторий: rq08-window-size

Этот пример находится в репозитории rq08-transition. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq08-transition --template rq08-transition
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq08-transition>

Это пример одного из тех уникальных случаев, когда возникает необходимость прослушивать одно или несколько событий, не поддерживаемых напрямую в React. Да, эти события редко используются в приложениях, но это не значит, что они не могут использоваться вообще.

Приложения со сложными сценариями скорее будут использовать прямые прослушватели DOM для узлов HTML. В таких ситуациях прослушватели могут изменяться на основании других критериев и целесообразнее управлять прослушвателями событий вручную, не полагаясь на то, что JSX и React будут сами добавлять и удалять прослушватели. Пример такого рода будет рассмотрен ниже.

8.8.3. Объединение обработки событий React и DOM

В этом примере используется комбинация прослушвателей событий React с ручным управлением прослушвателями событий DOM. Создадим меню, которое открывается щелчком на кнопке и закрывается щелчком на области за пределами меню. Создание приложения будет состоять из двух итераций. Начнем с наивной реализации, потом найдем ошибку, исправим ее и реализуем компонент правильно.

А пока рассмотрим последовательность событий. Требуется прослушивать щелчки кнопки, открывающие меню. Вы уже знаете, как сделать это в React при помощи `onClick`. Но кроме этого, необходимо прослушивать события нажатия кнопки мыши (или указателя, если вы также перехватываете события касания) для объекта окна и назначить этот обработчик в хуке эффекта. Последовательность событий приведена на рис. 8.15, а реализация — в листинге 8.12.

Репозиторий: `rq08-naive-menu`

Этот пример находится в репозитории `rq08-naive-menu`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq08-naive-menu --template rq08-naive-menu
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq08-naive-menu>

Листинг 8.12. Раскрывающееся меню (наивная версия)

```

Добавляет прослушиватель
к объекту окна
import { useState, useEffect } from "react";
function Menu() {
  const [isExpanded, setExpanded] = useState(false);
  useEffect(() => {
    if (!isExpanded) {
      return;
    }
    const onWindowClick = () => setExpanded(false);
    window.addEventListener(
      "pointerdown", onWindowClick
    );
    return () => window.removeEventListener(
      "pointerdown", onWindowClick
    );
  }, [isExpanded]);
  return (
    <main>
      <button onClick={() => setExpanded(true)}>
        Show menu
      </button>
      {isExpanded && (
        <div style={{ border: "1px solid black", padding: "1em" }}>
          This is the menu
        </div>
      )}
    </main>
  );
}
function App() {
  return <Menu />;
}
export default App;

```

Хранит информацию о том, раскрыто меню или нет, в значении состояния (по умолчанию false)

Если меню не раскрыто, происходит быстрая отмена в хуке эффекта (делать ничего не нужно)

Если меню раскрыто, создается прослушиватель, который свернет меню по щелчку мышью в любой точке внутри окна

Снова удаляет прослушиватель при очистке

Кнопка меню переключает флаг раскрытия в состояние true

Рендерит меню, если флаг раскрытия содержит true

Так как `isExpanded` присутствует в массиве зависимостей, этот хук будет выполняться каждый раз, когда меню переходит из раскрытого состояния в свернутое и наоборот

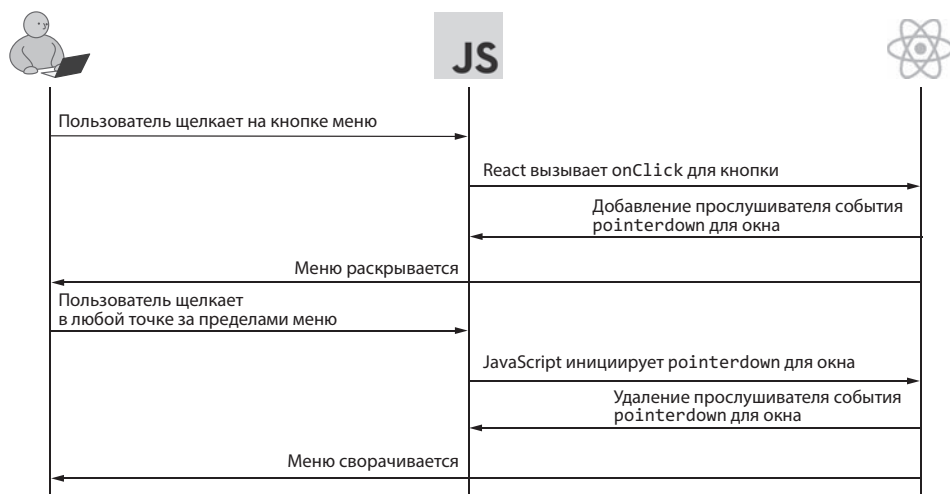


Рис. 8.15. Диаграмма логики работы компонента меню

На рис. 8.16 показано приложение, выполняемое в браузере. Однако возникает небольшая проблема. Если щелкнуть в области за пределами меню, когда оно раскрыто, меню закрывается как положено. Но если щелкнуть *внутри* меню, оно тоже закрывается. Так быть не должно. Пользователь должен иметь возможность взаимодействовать с меню, поскольку в будущем к меню будут добавлены кнопки или ссылки.

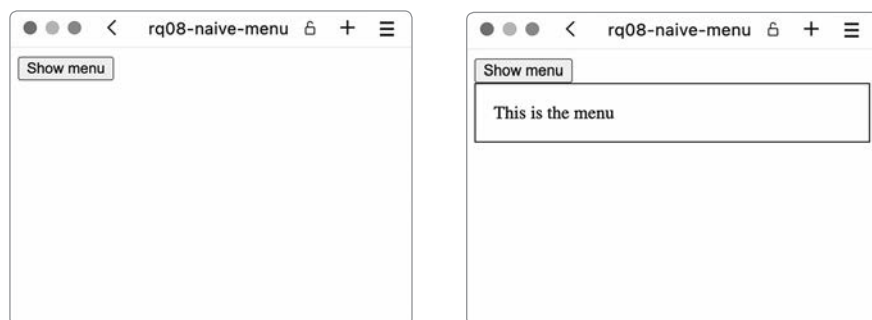


Рис. 8.16. Меню в раскрытом и свернутом виде соответственно

Меню должно закрываться только тогда, когда пользователь щелкает на области за пределами меню, но не по щелчку внутри меню. Остается понять, как выполнять операцию по щелчку мыши где угодно, кроме заданной области. Для этого мы используем три уже изученных приема:

- При раскрытии меню к объекту окна будет добавляться прослушиватель для любых событий `pointer-down`, происходящих с окном. При вызове меню будет сворачиваться, как и прежде.
- В новой версии мы также добавим прослушиватель события к самому меню. Он будет блокировать происходящие в нем события `pointerdown` от всплытия к объекту окна. Для этого мы будем останавливать распространение этих событий.
- Так как нам понадобится ссылка для узла DOM с меню, необходимо использовать хук `useRef`.

Объединение этих трех приемов гарантирует, что любые события нажатия кнопки мыши *внутри окна* (даже на элементах, неподконтрольных React) инициируют сворачивание меню, а нажатия кнопки мыши *внутри меню* его не инициируют, так как эти события не будут всплывать к объекту окна. Последовательность событий показана на рис. 8.17, а реализация приведена в листинге 8.13.

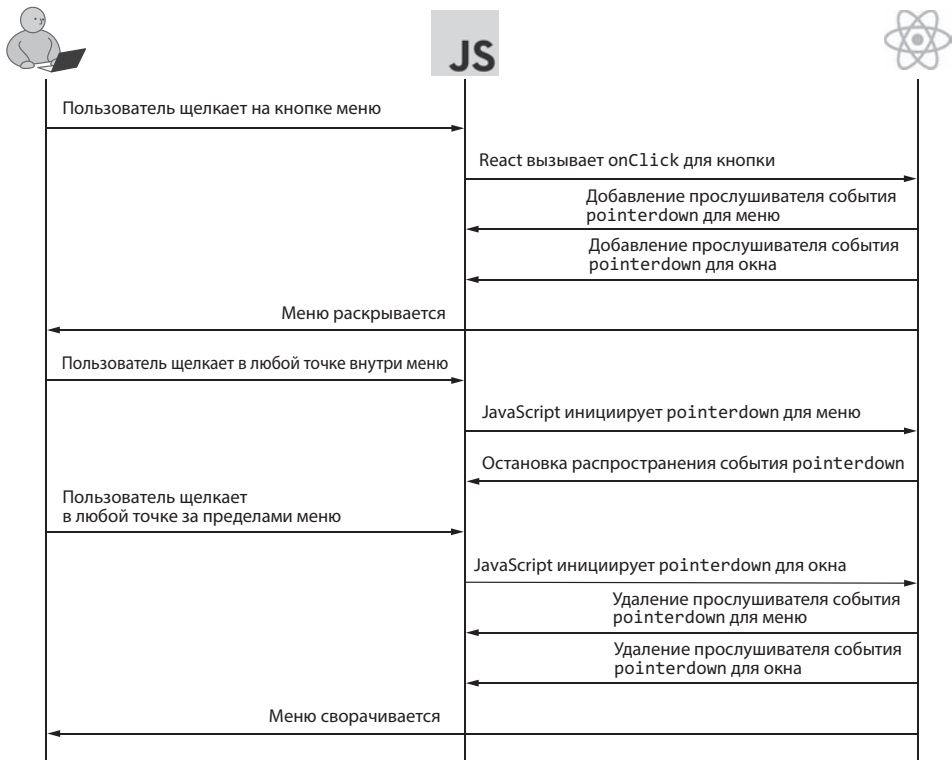


Рис. 8.17. Диаграмма логики работы компонента меню

Листинг 8.13. Раскрывающееся меню

Останавливает события указателя внутри самого меню, чтобы подавить распространение событий указателя и их выход за пределы узла меню

```
import { useRef, useState, useEffect } from "react";
function Menu() {
  const [isExpanded, setExpanded] = useState(false);
  useEffect(() => {
    if (!isExpanded) {
      return;
    }
    const onWindowClick = () => setExpanded(false);
    const onMenuClick = (evt) => evt.stopPropagation();
    const menu = menuRef.current;
    window.addEventListener("pointerdown", onWindowClick);
    menu.addEventListener("pointerdown", onMenuClick);
    return () => {
      window.removeEventListener(
        "pointerdown", onWindowClick
      );
      menu.removeEventListener(
        "pointerdown", onMenuClick
      );
    };
  }, [isExpanded]);
  const menuRef = useRef();
  return (
    <main>
      <button onClick={() => setExpanded(true)}>Show menu</button>
      {isExpanded && (
        <div
          ref={menuRef}
          style={{ border: "1px solid black", padding: "1em" }}
        >
          This is the menu
        </div>
      )}
    </main>
  );
}
function App() {
  return <Menu />;
}
export default App;
```

Прежде чем назначать прослушиватель для элемента меню, необходимо сохранить ссылку на этот элемент

Добавляет прослушиватель к элементу меню

Удаляет оба прослушивателя при очистке

Для сохранения ссылки на элемент меню понадобится useRef

Присваивает ссылку подходящему элементу JSX

Репозиторий: rq08-menu

Этот пример находится в репозитории `rq08-menu`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq08-menu --template rq08-menu
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq08-menu>

Запустив приложение в браузере, вы увидите то же самое, что и на рис. 8.16. Обратите внимание: раскрытое меню можно свернуть щелчком в любом месте, кроме самого меню (то есть внутри большого прямоугольника с черной рамкой).

Обратите внимание на использование разных хуков и даже объединение прослушивателей событий React с прослушивателями событий DOM для достижения желаемого результата. Все эти низкоуровневые элементы сочетаются в одном компоненте, который делает именно то, что нам нужно.

8.9. ВОПРОСЫ

1. Как правильно добавить прослушиватель события к кнопке JSX?
 - a) `<button click={onClick}>Click me</button>`
 - b) `<button click="onClick">Click me</button>`
 - c) `<button onClick={onClick}>Click me</button>`
 - d) `<button onClick="onClick">Click me</button>`
2. Обработчики событий React также могут назначаться вызовом `addEventListener` для элемента JSX. *Да* или *нет*?
3. Всплытие событий происходит редко и только для некоторых типов событий. *Да* или *нет*?
4. Вы не хотите, чтобы форма перезагружала страницу при отправке данных. Как вы поступите?
 - a) Назначите прослушиватель погружения.
 - b) Вызовете `evt.preventDefault()` для объекта события.

- с) Вручную назначите прослушиватель для узла HTML.
 - d) Вызовите `evt.stopPropagation()` для объекта события.
5. Невозможно прослушивать события узлов HTML, не находящихся внутри приложения React. *Да* или *нет*?

ОТВЕТЫ

1. `<button onClick={onClick}>Click me</click>`.
2. *Нет*. Обработчики событий React могут назначаться только с использованием свойств, например, `onClick`. Они не могут назначаться вызовом `addEventListener`.
3. *Нет*. Все события в React всплывают, даже те, которые не всплывают в HTML. Чтобы отменить действие по умолчанию, необходимо вызвать `evt.preventDefault()` для объекта события. Форма HTML инициирует перезагрузку страницы при отправке данных как действие по умолчанию.
5. *Нет*. Ручное назначение прослушивателей событий DOM может использоваться для прослушивания событий для любого узла HTML при условии, что на него есть ссылка.

ИТОГИ

- События играют важнейшую роль при создании интерактивных веб-приложений. События — это реакция приложения на пользовательский ввод.
- События также используются для взаимодействия между узлами HTML и приложением React, например, сообщения о загрузке ресурса или завершении воспроизведения видео.
- Прослушиватели событий React назначаются элементам JSX при помощи свойств. Прослушиватель события щелчка назначается в `onClick`, прослушиватель события вставки — в `onPaste` и т. д.
- Прослушиватели событий вызываются с объектом события, по которому можно определить, какое событие произошло, какой узел инициировал возникновение события, в какой фазе происходит обработка события, а также другие свойства конкретного события.

- Объекты событий также используются для прерывания нормального хода обработки событий — предотвращения действия по умолчанию браузера и/или остановки распространения события к другим прослушивателям событий.
- События распространяются от объекта окна вниз к целевому узлу и обратно вверх к объекту окна. Можно назначать прослушиватели для событий, проходящих вверх или вниз по дереву, например, чтобы прервать поток передачи сообщений или прослушивать события по нескольким целям.
- Можно назначать обычные прослушиватели событий объектам JavaScript и узлам HTML с использованием стандартного кода JavaScript. Иногда это необходимо, так как не все типы событий поддерживаются в React и не все узлы HTML доступны в React.

9

Работа с формами в React

В ЭТОЙ ГЛАВЕ

- ✓ Определение форм и элементов форм
- ✓ Отслеживание изменений в данных форм
- ✓ Обновление данных в элементах форм
- ✓ Обращение к элементам форм с использованием событий и ссылок
- ✓ Управляемый и неуправляемый ввод

Представьте веб-приложение без форм: вы не можете нигде залогиниться. Не можете ничего заказать в интернет-магазине. Не можете ни с кем пообщаться в чате. Наконец, вы даже не можете пожаловаться на приложение, потому что в нем нет формы для отправки сообщения!

Формы — основа многих интерактивных веб-приложений. Формы, в особенности элементы форм, являются основным средством получения данных от пользователя в полях ввода, чекбоксах, раскрывающихся списках, элементах отправки файлов и многих других элементах интерфейса.

Из-за важности форм любой приличный веб-фреймворк должен поддерживать работу с данными форм. React отлично работает с формами. Собственно, эффективная работа с данными форм стала одним из приоритетов React при его

создании, поскольку она была абсолютно необходима для функциональности, для которой разрабатывался React.

Работать с формами в React можно двумя способами. Можно доверить React управление состоянием формы и хранение текущих значений в состоянии компонента — способ, который считается предпочтительным, поскольку вся логика и потоки данных остаются в границах React. Часто лучше доверить управление всем приложением React, чем другим сторонам или приложениям. В таком случае контроль от автоматической обработки форм браузера переходит к React.

Также можно доверить HTML все управление состоянием формы, а в React только читать его при необходимости. К преимуществам этого варианта можно отнести использование встроенной обработки форм в браузере, но за это приходится расплачиваться потерей контроля в приложении. С неконтролируемым входным значением приходится разбираться пользователю и браузеру. Приложение не сможет (легко) гарантировать, что это значение будет соответствовать любым правилам, которые вы решите установить.

Эти два варианта поведения называются *управляемым* и *неуправляемым* соответственно. О том, к каким последствиям приводит выбор того или иного режима и как они влияют на выбор архитектуры, мы подробно расскажем в этой главе.

К концу этой главы вы научитесь строить сложные формы с разными видами полей ввода, включая текстовые поля, поля даты, числовые поля, диапазоны, кнопки и раскрывающиеся списки. На рис. 9.1 представлены все разновидности

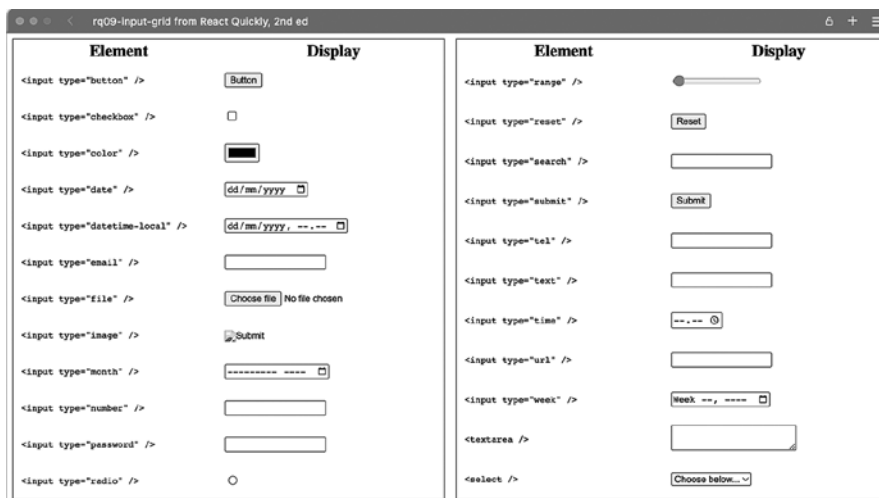


Рис. 9.1. Все элементы ввода HTML, которые могут использоваться в React

ввода, которые могут использоваться в React, и изучив эту главу, вы сможете применять их в приложениях React. Также вы научитесь определять, когда неуправляемая форма может быть лучше используемой по умолчанию управляемой формы, хотя такие ситуации встречаются не так часто.

ПРИМЕЧАНИЕ Исходный код примеров этой главы доступен по адресу <https://rq2e.com/ch09>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке одной командой.

9.1. УПРАВЛЯЕМЫЙ И НЕУПРАВЛЯЕМЫЙ ВВОД

При управляемом вводе React управляет выводимой информацией. React должен «подтвердить», что заданное изменение входящего значения приведет к изменению ввода. Неуправляемый ввод изменяется в зависимости от взаимодействий с пользователем, и React может только пассивно читать состояние, но не влиять на него и не изменять его.

Различия между двумя подходами представлены на рис. 9.2. В частности, обратите внимание на действия 4 и 5 на управляемой версии диаграммы. Эти два взаимодействия являются обязательными. С другой стороны, в неуправляемой версии действие 4.b не обязательно, а действие 5.b полностью отсутствует, потому что оно невозможно.

В обеих версиях диаграммы React может решить, каким должно быть исходное значение. Но только в управляемой версии диаграммы React может управлять значением после того, как пользователь начнет вводить данные.

Если контролировать ввод после исходного значения не требуется, можно использовать любой режим. Если ввод данных пользователем должен контролироваться (например, нужно отфильтровать ввод или применить к нему маску форматирования), работайте с управляемым вводом.

Ситуаций, в которых вы *обязательно* должны использовать только неуправляемый ввод, не существует, так что и причин для его использования тоже нет. Он в основном применяется по соображениям производительности. Примеры из следующих двух разделов показывают, как проверить, что ввод относится к заданному типу, а затем использовать его.

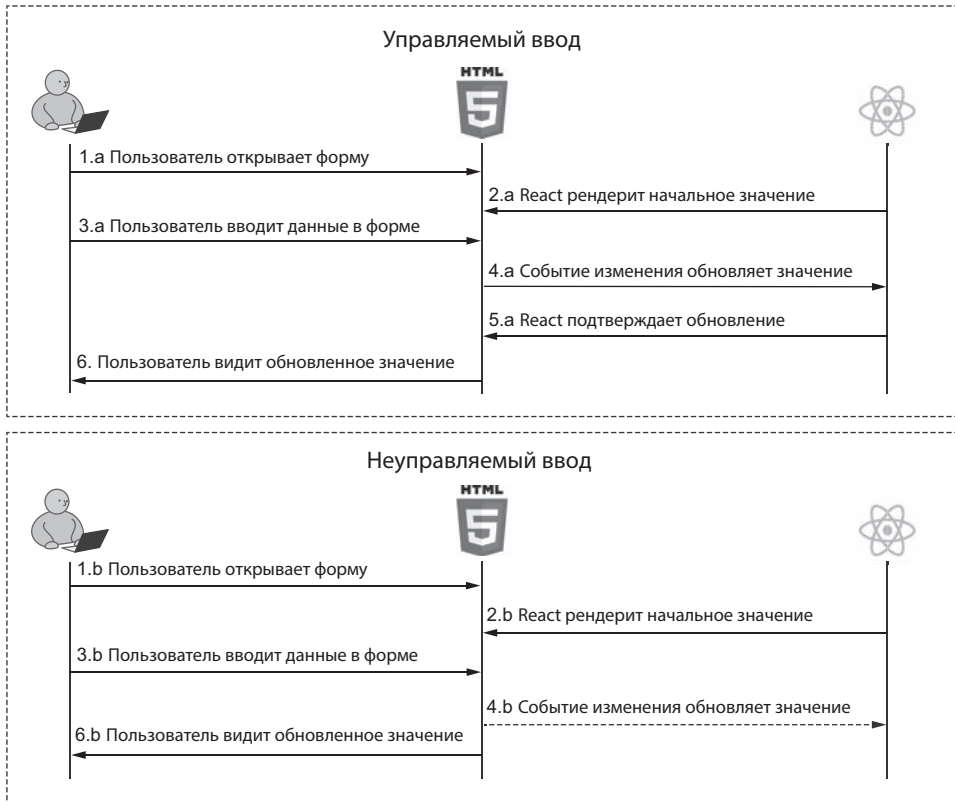


Рис. 9.2. Передача данных при управляемом и неуправляемом вводе отличается тем, что React не управляет значением после рендеринга ввода. Действие 5.a подчеркивает контроль React над вводом даже после взаимодействия с ним пользователя — React может перехватывать обновления, изменять их на ходу и даже игнорировать. При неуправляемом вводе (внизу) у React таких возможностей нет, и в форме отображается информация, которую вводит пользователь

9.2. КОНТРОЛЬ УПРАВЛЯЕМОГО ВВОДА

Для примера создадим очень простую форму ввода — калькулятор с двумя полями ввода и полем вывода, в котором выводится сумма двух значений. Для этого необходимо создать структуру JSX, представленную на рис. 9.3.

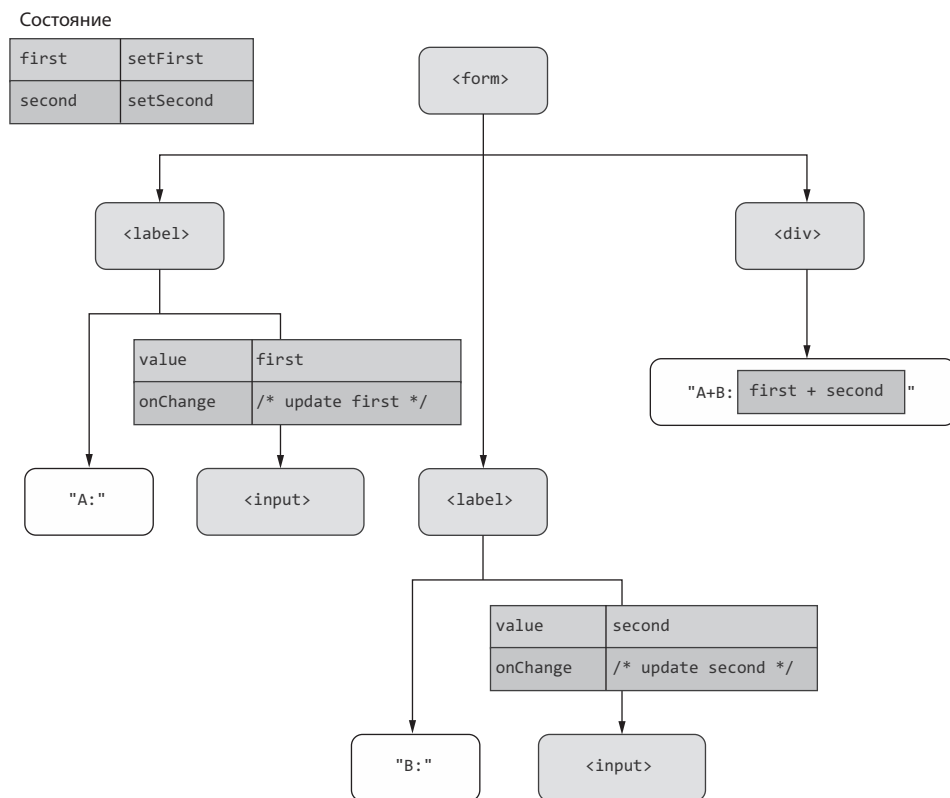


Рис. 9.3. Разметка JSX для калькулятора должна содержать два поля ввода. Обратите внимание: обоим полям ввода передаются два конкретных свойства: `value` и `onChange`

Как видно из диаграммы, мы задаем свойства `value` и `onChange` для полей ввода, что необходимо для управляемого ввода. Собственно, это можно считать определением управляемого ввода. Если значение задается напрямую в React, необходимо прослушивать событие изменения и обновлять значение, в результате чего ввод становится управляемым. Если значение не задается в React, прослушивать обновления не нужно, потому что ввод не управляется.

Кроме того, чтобы гарантировать, что входные значения являются числами, мы назовем им тип `"number"`. Реализация приложения приведена в листинге 9.1. Примерный вид приложения в браузере показан на рис. 9.4.

Листинг 9.1. Суммирование

```

import { useState } from "react";
function Sum() {
  const [first, setFirst] = useState(0);
  const [second, setSecond] = useState(0);
  const onChangeFirst = (evt) =>
    setFirst(evt.target.valueAsNumber);
  const onChangeSecond = (evt) =>
    setSecond(evt.target.valueAsNumber);
  return (
    <form style={{ display: "flex", flexDirection: "column" }}>
      <label>
        A:
        <input
          type="number"
          value={first}
          onChange={onChangeFirst}
        />
      </label>
      <label>
        B:
        <input
          type="number"
          value={second}
          onChange={onChangeSecond}
        />
      </label>
      <div>A+B: {first + second}</div>
    </form>
  );
}
function App() {
  return <Sum />;
}
export default App;

```

Инициализирует два значения состояния.
Инициализировать их нулями не обязательно;
подойдут любые начальные числа

Создает два почти одинаковых обработчика
изменений, которые обновляют разные значения
состояния на основании событий ввода

Присваивает правильные значения
и назначает прослушватели для
двух полей ввода

В конце выводит результат —
сумму двух значений состояния

Репозиторий: rq09-controlled-sum

Этот пример находится в репозитории `rq09-controlled-sum`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq09-controlled-sum --template rq09-controlled-sum
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq09-controlled-sum>

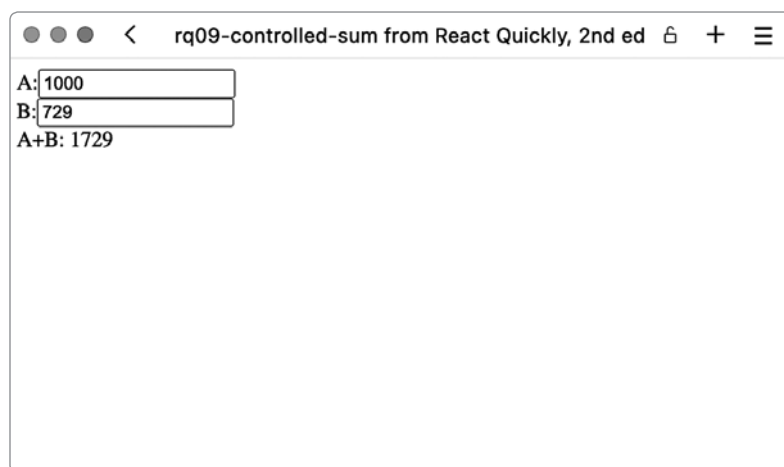


Рис. 9.4. Компонент суммирования в действии. Сумма 1000 и 729 действительно равна 1729

Обратите внимание, что для элементов ввода устанавливаются как `value`, так и `onChange` — это не совпадение. Это именно те два свойства, которые необходимо задать для использования управляемого ввода; кроме того, необходимо следить, чтобы значение изменялось при вызове обработчика изменения. Значение может изменяться и в другое время, но вы *должны* обновить значение при обновлении обработчика изменения. В противном случае информация, введенная пользователем, будет проигнорирована.

9.2.1. Фильтрация ввода

Если не обновлять значение состояния при вводе данных, поле ввода обновляться не будет. Конечно, это поведение можно использовать избирательно. Предположим, требуется добавить поле ввода для шестнадцатеричного кода цвета, а рядом с ним — маленький квадратик, который окрашивается в заданный цвет.

Шестнадцатеричный код цвета состоит из шести шестнадцатеричных цифр 0–9 и A–F. Так как в коде могут использоваться нецифровые символы, мы не можем воспользоваться типом `type="number"`, как делали раньше. В этом случае для поля ввода будет использоваться `type="text"`, но ввод необходимо фильтровать, чтобы только действительные символы попадали в текстовое поле после того, как пользователь введет данные. Блок-схема логики приведена на рис. 9.5.

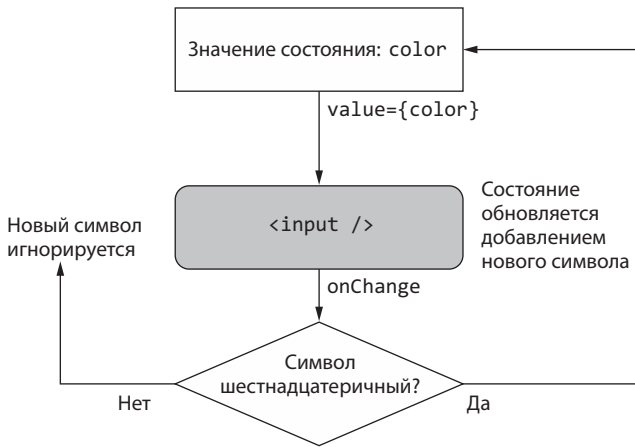


Рис. 9.5. Поток данных в компоненте вывода цвета включает фильтр в обработке события onChange. Любые символы, кроме шестнадцатеричных, исключаются из ввода, а затем значение «подтверждается» для JSX

ПРИМЕЧАНИЕ Мы знаем о существовании специального поля для выбора цвета, которое поддерживают все современные браузеры, — `<input type="color" />`. Но мы его не используем, так как это было бы слишком просто.

Для реализации этой логики просто применим фильтр в функции onChange перед заданием значения состояния. Тогда поле ввода будет обновляться при вводе допустимых символов, но игнорировать все недопустимые символы. Кроме того, все символы будут преобразовываться к верхнему регистру, чтобы текст выглядел привлекательнее. При выводе цвета отображаются только строки из шести символов. Если длина строки отличается от шести символов, отображается временный фон, обозначающий отсутствие действительного ввода. Реализация приведена в листинге 9.2. Результат выполнения кода в браузере показан на рис. 9.6.

Листинг 9.2. Вывод шестнадцатеричного кода цвета

```
import { useState } from "react";
const PLACEHOLDER = `conic-gradient(
  gray 0.25turn, white 0 0.5turn,
  gray 0 0.75turn, white 0 1turn
)`;
function HexColor() {
  const [color, setColor] = useState("BADA55");
  const onChange = (evt) =>
```

Определяет статический заполнитель, который использует конический градиент для отображения фона в клеточку

Инициализирует состояние допустимым кодом цвета

```

setColor(
  evt.target.value
    .replace(/[^0-9a-f]/gi, "")
    .toUpperCase()
);
const outputStyle = {
  width: "20px",
  border: "1px solid",
  background: color.length === 6
    ? `#${color}`
    : PLACEHOLDER,
};
return (
  <form style={{ display: "flex" }}>
    <label>
      Hex color:
      <input value={color} onChange={onChange} />
    </label>
    <span style={outputStyle} />
  </form>
);
}
function App() {
  return <HexColor />;
}
export default App;

```

В обработчике изменения проверяет текущее значение поля ввода после события, фильтрует ввод по регулярному выражению и преобразует результат к верхнему регистру

Когда требуется вывести значение цвета, мы сначала проверяем, что строка цвета содержит ровно шесть символов. Если проверка проходит, перед значением выводится значок решетки; в противном случае выводится заполнитель

Добавляет значение и обработчик изменения к полю ввода, как и прежде

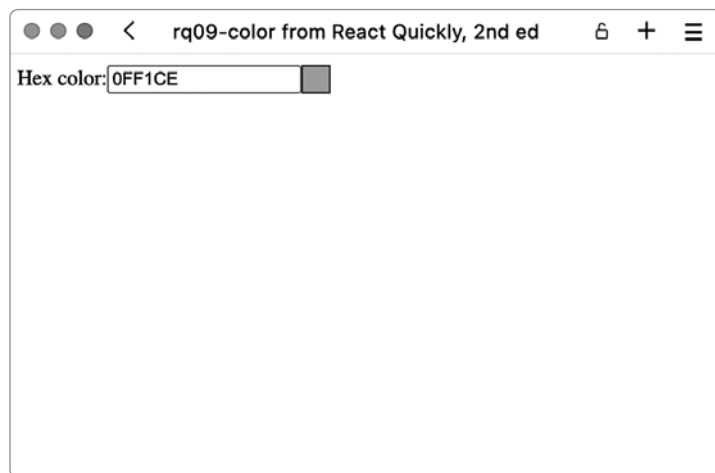


Рис. 9.6. Код 0FF1CE определяет допустимый цвет — приятный оттенок голубого

Репозиторий: rq09-color

Этот пример находится в репозитории `rq09-color`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq09-color --template rq09-color
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq09-color>

Обратите внимание: на диаграмме на рис. 9.5 упоминается проверка нового введенного символа, по результатам которой он добавляется или отбрасывается, но в фактическом исходном коде в обратном вызове `onChange` в листинге 9.2 проверяется не отдельный символ, а все входящее значение. Дело в том, что данные не обязательно вводятся посимвольно. Пользователь может вставить строку символов из буфера; в таком случае приходится проверять весь ввод вместо отдельного символа. Пользователь также может ввести новый символ в любом месте новой строки, что дополнительно усложнит проверку. Чтобы избежать лишней работы, мы всегда анализируем весь ввод и проверяем его по фильтру.

Также обратите внимание, что на этот раз значение состояния инициализируется непустой строкой. И это буквально все, что нужно, чтобы задать значение по умолчанию для поля ввода.

9.2.2. Ввод с применением маски

В более сложном варианте обработчика изменения к полю ввода применяется маска при вводе данных пользователем. В следующем примере добавляется поле для ввода номера тикета для условного веб-сайта. Эти условные номера определяются как три алфавитно-цифровых символа, за которыми следует дефис и еще три алфавитно-цифровых символа, например `R1S-T2U`.

Когда вы имеете дело с такими данными, по возможности помогайте пользователю с вводом. Для начала стоит преобразовать символы к верхнему регистру независимо от того, как их вводит пользователь (вы уже знаете, как это делается!). Кроме того, добавить дефис после первых трех символов. Наконец, ограничить ввод семью символами.

Звучит сложно, но на самом деле это не так. Достаточно изменить `onChange`, чтобы значение состояния обновлялось только допустимой и правильно отформатированной строкой, а любой другой ввод игнорировался. Фактический код JavaScript для решения этой задачи представляет собой набор операций форматирования строк, которые в совокупности реализуют нужную бизнес-логику.

На этот раз мы добавим к полю ввода заполнитель, который направляет действия пользователя в процессе ввода данных. Реализация всей логики приведена в листинге 9.3. Примерный вид приложения в браузере показан на рис. 9.7.



Рис. 9.7. Содержимое поля ввода Ticket Number до и после ввода значения. Обратите внимание на то, что после ввода трех символов автоматически вставляется дефис

Листинг 9.3. Ввод номера тикета

В этой версии реализована дополнительная проверка входящего значения, что приводит к разбиению его на две части, содержащие до трех символов

```
import { useState } from "react";
function TicketNumber() {
  const [ticketNumber, setTicketNumber] = useState("");
  const onChange = (evt) => {
    const [first = "", second = ""] = evt.target.value
      .replace(/[^0-9a-z]/gi, "")
      .slice(0, 6)
      .match(/.{0,3}/g);
    const value = first.length === 3
      ? `${first}-${second}`
      : first;
    setTicketNumber(value.toUpperCase());
  };
  const isValid = ticketNumber.length === 7;
  return (
    <form style={{ display: "flex" }}>
      <label>
        Ticket number:
        <input
```

Если первая часть состоит ровно из трех символов, мы помогаем пользователю, добавляя дефис в поле ввода

Если длина ввода составляет ровно семь символов, это должен быть допустимый номер тикета

```

    value={ticketNumber}
    onChange={onChange}
    placeholder="E.g. R1S-T2U"
  />
</label>
<span>{isValid ? "✓" : "X"}</span>
</form>
);
}
function App() {
  return <TicketNumber />;
}
export default App;

```

Добавляет все свойства к полю ввода, включая заполнитель

Выводит в конце значок, показывающий, прошел ввод проверку или нет

Репозиторий: rq09-ticket-no

Этот пример находится в репозитории `rq09-ticket-no`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq09-ticket-no --template rq09-ticket-no
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq09-ticket-no>

Заметим, что это решение неидеально. Если вы попытаетесь удалить некоторые символы нажатием Backspace, удалить дефис не удастся, потому что после его удаления код заметит, что строка состоит из трех символов, и автоматически снова добавит дефис. Создать ввод с маской может быть очень непросто. Существуют библиотеки и руководства, которые упрощают эту работу.

9.2.3. Ввод множества похожих значений

Если в форме требуется вводить много значений, создание отдельных значений состояния и обработчиков изменений для всех значений может быть однообразной и утомительной работой (к тому же приводящей к дублированию кода). Вместо этого можно создать одно значение состояния, содержащее все значения форм и обобщенный генератор обработчиков изменений, способный обрабатывать любой ввод.

Например, такая схема хорошо подходит для простой адресной формы с полями для ввода адресной строки 1, адресной строки 2, города, почтового индекса, региона и страны. Для всех данных используются простые поля ввода без проверки,

потому что в разных странах используются разные правила написания этих элементов. Мы просто разрешаем пользователю вводить в этих полях любые данные на его усмотрение.

Мы будем хранить в состоянии один объект со всеми актуальными данными формы, инициализируемыми пустыми строками. Так как все значения хранятся в одном объекте, индексируемом по ключу, мы можем использовать этот ключ для идентификации поля, обновляемого каждым обработчиком обновления. Итоговый код JSX напоминает частичное дерево JSX, изображенное на рис. 9.8.

Состояние

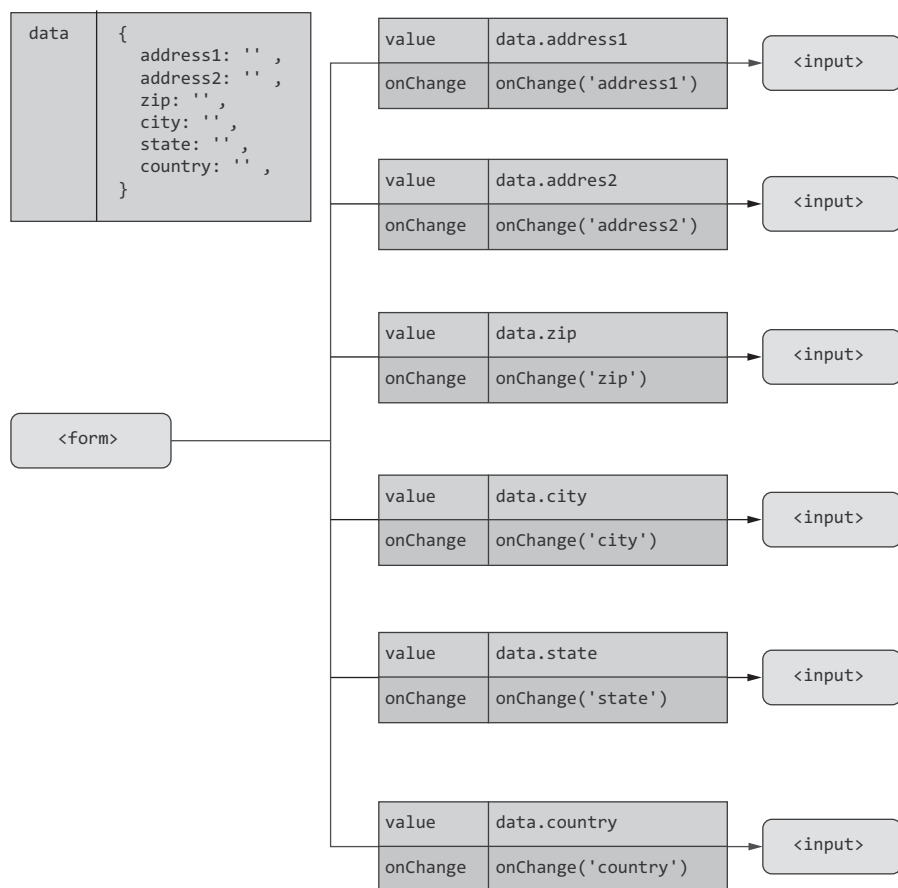


Рис. 9.8. Частичное дерево DOM (без надписей и других привычных элементов) состоит из полей ввода и их свойств. Дерево ориентировано слева направо, а не сверху вниз просто для того, чтобы все элементы поместились на странице

Теперь остается только создать функцию-генератор обработчиков событий, которая получает задаваемое свойство объекта и обновляет его входящим значением из объекта события при вызове обработчика события. Для этого необходимо создать функцию, которая возвращает функцию, как в главе 8. Реализация приведена в листинге 9.4. Примерный вид приложения в браузере показан на рис. 9.9.

Рис. 9.9. Форма полного адреса в браузере — не самая красивая, но полностью рабочая

Листинг 9.4. Адресная форма

```
import { useState } from "react";
function Address() {
  const [data, setData] = useState({
    address1: "",
    address2: "",
    zip: "",
    city: "",
    state: "",
    country: "",
  });
  const onChange = (key) => (evt) => {
    setData((oldData) =>
      ({ ...oldData, [key]: evt.target.value })
    );
  };
  return (
    <form style={{ display: "flex", flexDirection: "column" }}>
      <label>
        Address line 1:
        <input
```

В этой версии состояние представляет собой объект, содержащий все необходимые переменные

Функция onChange преобразуется в генератор, который получает ключ и возвращает обработчик события

При изменении ввода состояние обновляется полным прежним состоянием (чтобы не переопределять существующие значения), к которому добавляется новое значение с указанным ключом

```

        value={data.address1}
        onChange={onChange("address1")}
      />
    </label>
    <label>
      Address line 2:
      <input
        value={data.address2}
        onChange={onChange("address2")}
      />
    </label>
    <label>
      Zip:
      <input
        value={data.zip}
        onChange={onChange("zip")}
      />
    </label>
    <label>
      City:
      <input
        value={data.city}
        onChange={onChange("city")}
      />
    </label>
    <label>
      State:
      <input
        value={data.state}
        onChange={onChange("state")}
      />
    </label>
    <label>
      Country:
      <input
        value={data.country}
        onChange={onChange("country")}
      />
    </label>
  </form>
);
}
function App() {
  return <Address />;
}
export default App;

```

Применяет значение
и обработчик изменений
ко всем полям ввода

Репозиторий: rq09-address

Этот пример находится в репозитории rq09-address. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq09-address --template rq09-address
```

На следующем сайте можно посмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq09-address>

Использование свойства name

Можно развить рассмотренную идею. У элементов форм есть атрибут `name` с идентификатором поля, содержимое которого будет отправлено при использовании обычной формы HTML. Можно использовать это свойство для хранения ключа, который будет обновляться в обработчике изменения. В таком случае передавать ключ обработчику изменения не нужно, потому что он может проверить свойство `name` цели события. Структура JSX представлена на рис. 9.10 (сравните с рис. 9.8).

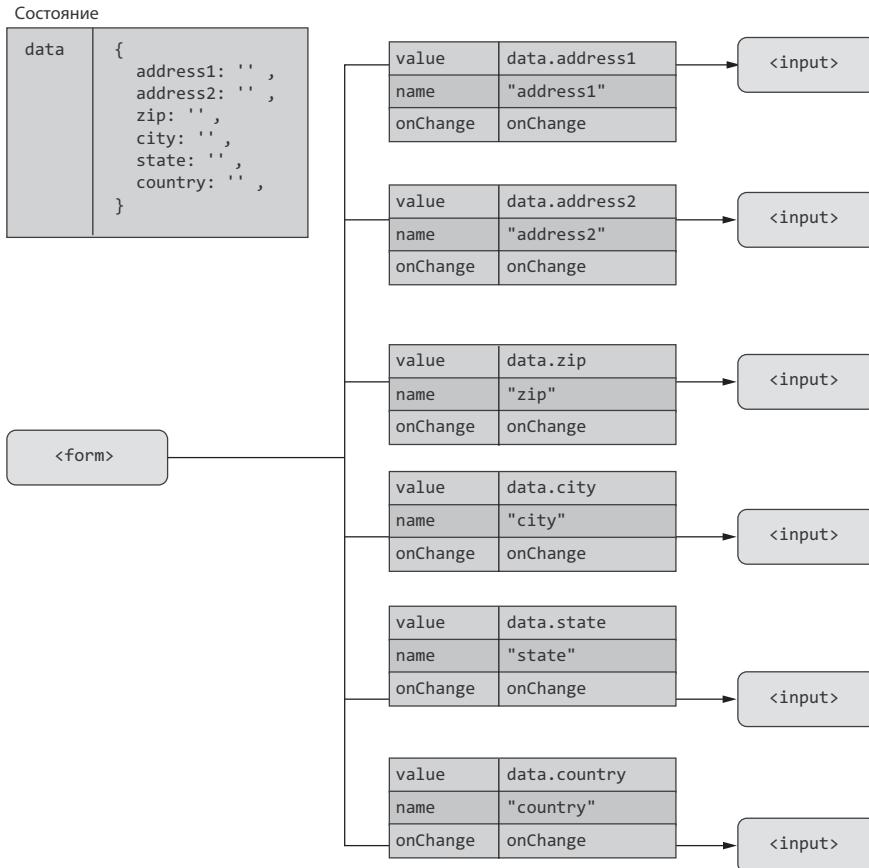


Рис. 9.10. Частичное дерево DOM с упрощенными обработчиками событий

Теперь один и тот же обработчик события может использоваться для каждого элемента. Обработчик события обновляет объект состояния ключом, взятым из атрибута `name` цели события, и значением, взятым из `value` цели события.

В новой версии также нужно проверить, что состояние содержит именно те данные, которые мы ожидаем. Для этого в конце добавим элемент `<pre>`, который выводит содержимое `data` в документе JSON. Реализация приведена в листинге 9.5. Примерный вид приложения в браузере показан на рис. 9.11.

Листинг 9.5. Упрощенная адресная форма

```
import { useState } from "react";
function Address() {
  const [data, setData] = useState({
    address1: "",
    address2: "",
    zip: "",
    city: "",
    state: "",
    country: "",
  });
  const onChange = (evt) => {
    const key = evt.target.name;
    const value = evt.target.value;
    setData((oldData) =>
      ({ ...oldData, [key]: value }));
  };
  return (
    <form style={{ display: "flex", flexDirection: "column" }}>
      <label>
        Address line 1:
        <input
          value={data.address1}
          name="address1"
          onChange={onChange}
        />
      </label>
      <label>
        Address line 2:
        <input
          value={data.address2}
          name="address2"
          onChange={onChange}
        />
      </label>
      <label>
        Zip:
        <input
          value={data.zip}
          name="zip"
          onChange={onChange}
        />
    </form>
  );
}
```

Функция `onChange` снова превращается в простой обработчик события, и мы извлекаем имя поля ввода из цели

Извлекает текущее значение поля ввода аналогичным образом

Обновляет объект состояния измененным вводом

Назначает свойство `name` и простой обработчик события каждому узлу ввода

```

</label>
<label>
  City:
  <input
    value={data.city}
    name="city"
    onChange={onChange}
  />
</label>
<label>
  State:
  <input
    value={data.state}
    name="state"
    onChange={onChange}
  />
</label>
<label>
  Country:
  <input
    value={data.country}
    name="country"
    onChange={onChange}
  />
</label>
<pre>{JSON.stringify(data, true, 2)}</pre>
</form>
);
}
function App() {
  return <Address />;
}
export default App;

```

Назначает свойство name и простой обработчик события каждому узлу ввода

Выводит удобное представление текущего состояния данных в формате JSON, чтобы мы видели, что все верно

rq09-smart-address from React Quickly

Address line 1: 1600 Pennsylvania Ave

Address line 2: Attn: Commander

Zip: 20500

City: Washington

State: DC

Country: USA

```

{
  "address1": "1600 Pennsylvania Ave",
  "address2": "Attn: Commander",
  "zip": "20500",
  "city": "Washington",
  "state": "DC",
  "country": "USA"
}

```

Рис. 9.11. Простая, но умная адресная форма работает именно так, как задумано!

Репозиторий: rq09-smart-address

Этот пример находится в репозитории `rq09-smart-address`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq09-smart-address --template rq09-smart-address
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq09-smart-address>

Свойство `name` поля ввода очень часто используется для хранения ключа для информации, введенной в поле ввода. Делать так не обязательно, но это очень удобный способ организации форм, особенно если они очень больших размеров.

Однако этой форме (и всем формам, созданным ранее в этой главе) кое-чего не хватает. Как отправить данные этой формы? Что делать с данными при отправке? Ответим на эти вопросы в следующем разделе.

9.2.4. Отправка данных форм

Создадим очень простое приложение со списком задач — полностью готовое, выполняющее реальную работу и полнофункциональное. Это классическое упражнение при изучении веб-фреймворка, поэтому мы также займемся им.

Приложение должно предоставлять средства для создания новых задач с заголовком, категорией, продолжительностью и датой завершения. Должна быть также возможность просматривать список созданных задач и, конечно, удалять задачи по мере их завершения. В приложении предусмотрены два экрана: для вывода списка задач и для добавления новых задач.

На этой стадии важной частью становится обработка форм. Нам нужна форма с несколькими полями ввода. Когда пользователь отправляет данные формы, требуется создать новый объект данных на базе введенных данных, добавить его в список задач, очистить форму и разрешить пользователю добавить новый объект. Логика работы приложения представлена на рис. 9.12.

В этом приложении список задач хранится только в памяти, так что при перезагрузке страницы все данные будут потеряны. Длительное хранение данных для нас сейчас интереса не представляет; пока достаточно обеспечить логичную

работу с данными форм. При создании приложения будут использоваться три компонента (их структура представлена на рис. 9.13):

- `<App />` — главный компонент, содержит список задач в форме локального состояния, а также методы для добавления и удаления элементов. Компонент также знает, выполняется ли в настоящий момент добавление задачи или просмотр списка всех задач, и содержит небольшое меню для переключения между двумя представлениями.
- `<List />` — получает список задач для отображения, а также функцию, вызываемую при удалении задачи.
- `<Add />` — содержит форму для ввода информации о новой задаче и ее отправки. При отправке данных компонента происходит возврат к списку. Также можно отменить отправку и вернуться к списку, ничего не добавляя.

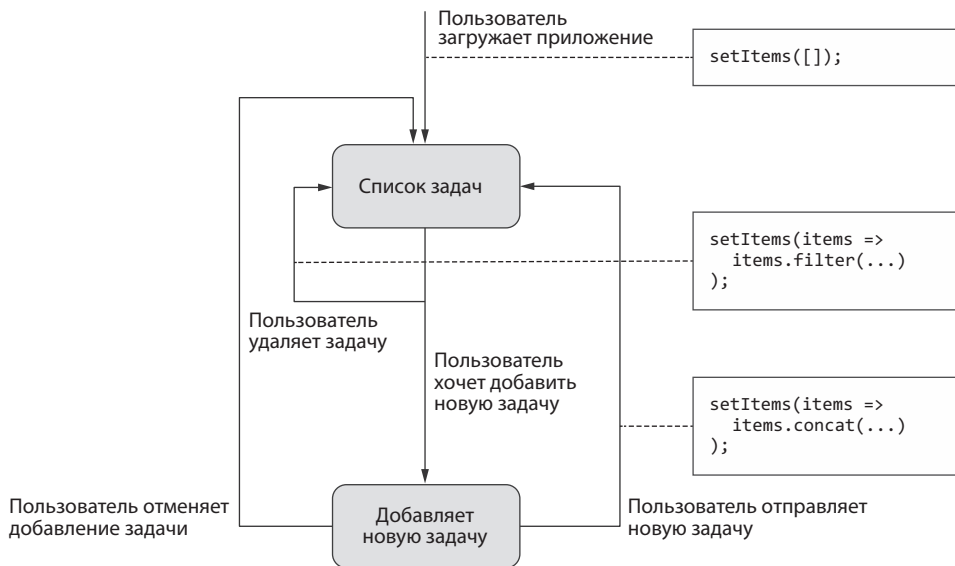


Рис. 9.12. Логика работы приложения со списком задач

Готовое приложение должно выглядеть как на рис. 9.14.

Начнем с реализации главного компонента `<App />`. В этом примере будут использоваться несколько файлов, и код компонента будет храниться в файле `App.js`. Его содержимое приведено в листинге 9.6.

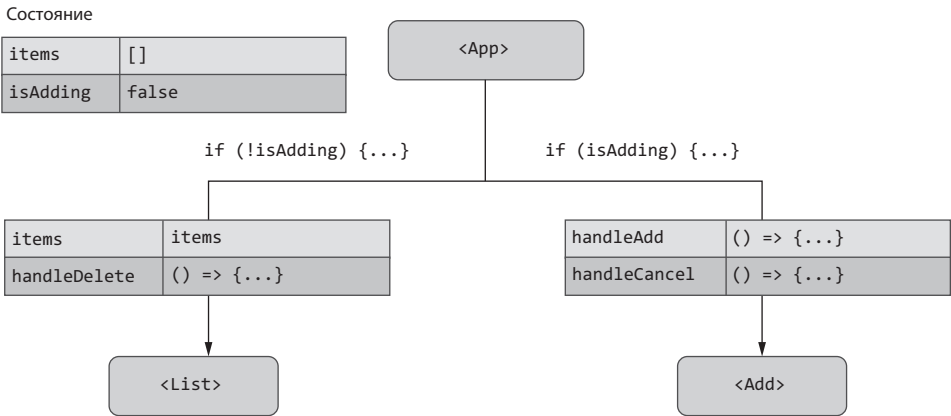


Рис. 9.13. Схема структуры трех компонентов приложения и связей между ними

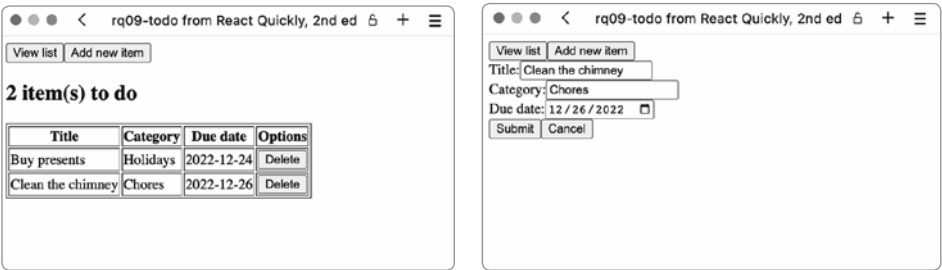


Рис. 9.14. Готовое приложение при просмотре списка задач и добавлении новой задачи

Листинг 9.6. Главный компонент в файле App.js

При добавлении задачи состояние обновляется всеми существующими задачами плюс только что созданная задача. Затем приложение возвращается к представлению списка

```
import { useState } from "react";
import List from "./List";
import Add from "./Add";
function App() {
  const [items, setItems] = useState([]);
  const [isAdding, setAdding] = useState(false);
  const handleDelete = (item) =>
    setItems(oldItems =>
      oldItems.filter((oldItem) => oldItem !== item));
  const handleAdd = (newItem) => {
    setItems(oldItems => oldItems.concat([newItem]));
    setAdding(false);
  };
}
```

Исходное состояние приложения отражает тот факт, что список задач пуст, а задача в настоящий момент не добавляется

При удалении задачи состояние обновляется всеми задачами, кроме удаляемой

Импортирует два представления из отдельных файлов


```

const handleCancel = () => setAdding(false);
return (
  <main>
    <nav>
      <button onClick={() => setAdding(false)}>
        View list
      </button>
      <button onClick={() => setAdding(true)}>
        Add new item
      </button>
    </nav>
    {isAdding ? (
      <Add
        handleAdd={handleAdd}
        handleCancel={handleCancel}
      />
    ) : (
      <List
        items={items}
        handleDelete={handleDelete}
      />
    )}
  </main>
);
}
export default App;

```

При отмене добавления задачи приложение просто возвращается к представлению списка

Меню просто переключает флаг добавления задачи в зависимости от того, добавляется новая задача или нет

Если задача добавляется, мы включаем соответствующий компонент с двумя необходимыми обратными вызовами в свойствах

Если задача не добавляется, приложение выводит список всех задач, поэтому здесь также должны передаваться необходимые свойства

Основная часть приложения зависит от текущего состояния — добавляется новая задача или нет?

Разобравшись с главным компонентом `<App />`, перейдем к компоненту `<List />` в `List.js`. Он намного проще предыдущего, потому что он только выводит таблицу со всеми задачами и размещает рядом с каждой кнопку для ее удаления. Компонент получает два свойства: список выводимых задач и обратный вызов, инициируемый при удалении задачи. Его реализация приведена в листинге 9.7.

Листинг 9.7. Компонент списка в файле `List.js`

```

function List({ items, handleDelete }) {
  if (!items.length) {
    return <h2>To-do list empty, go out and play!</h2>;
  }
  return (
    <>
      <h2>{items.length} item(s) to do</h2>
      <table border="1">
        <thead>
          <tr>
            <th>Title</th>
            <th>Category</th>

```

Важная часть этого компонента — ранний возврат при отсутствии задач. Нет необходимости выводить таблицу, если ее нечем заполнить

```

    <th>Due date</th>
    <th>Options</th>
  </tr>
</thead>
<tbody>
  {items.map((item) => (
    <tr key={JSON.stringify(item)}>
      <td>{item.title}</td>
      <td>{item.category}</td>
      <td>{item.date}</td>
      <td>
        <button
          onClick={() => handleDelete(item)}
        >
          Delete
        </button>
      </td>
    </tr>
  ))}
</tbody>
</table>
</>
);
}
export default List;

```

Если есть что-то, что можно вывести, приложение перебирает все задачи и выводит в таблице строку для каждой

Кнопка Delete вызывает функцию обратного вызова отмены, передавая всю задачу в аргументе

Наконец, реализуем важный компонент приложения: форму для добавления новой задачи в компоненте `<Add />` в `Add.js`. Воспользуемся некоторыми приемами, описанными в этой главе, включая обобщенный обработчик изменения для всех полей ввода, обновляющий состояние компонента на основании свойства `name`. Реализация приведена в листинге 9.8.

Репозиторий: rq09-todo

Этот пример находится в репозитории `rq09-todo`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq09-todo --template rq09-todo
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq09-todo>

Листинг 9.8. Компонент формы в файле Add.js

```

import { useState } from "react";
function Add({ handleAdd, handleCancel }) {
  const [data, setData] = useState({
    title: "",
    category: "",
    date: "",
  });
  const onChange = (evt) => {
    const key = evt.target.name;
    const value = evt.target.value;
    setData((oldData) =>
      ({ ...oldData, [key]: value }));
  };
  const onSubmit = (evt) => {
    handleAdd(data);
    evt.preventDefault();
  };
  return (
    <form
      onSubmit={onSubmit}
      style={{ display: "flex", flexDirection: "column" }}
    >
      <label>
        Title:
        <input
          value={data.title}
          name="title"
          onChange={onChange}
        />
      </label>
      <label>
        Category:
        <input
          value={data.category}
          name="category"
          onChange={onChange}
        />
      </label>
      <label>
        Due date:
        <input
          type="date"
          value={data.date}
          name="date"
          onChange={onChange}
        />
      </label>
      <div>
        <button>Submit</button>
      </div>
    </form>
  );
}

```

Инициализирует состояние, как и прежде

Тот же обработчик изменения, который использовался в листинге 9.5, — это универсальная конструкция!

При отправке данных формы необходимо передать данные соответствующему обратному вызову и заблокировать действие формы по умолчанию. Если вы забудете о последнем, страница перезагрузится и все данные будут потеряны

Назначает обработчик события для формы

Назначает свойства полям ввода, как обычно. Обратите внимание, что для поля завершающей даты также добавляется `type="date"`

Кнопка по умолчанию — кнопка отправки данных (Submit), если явно не указан ее тип `type="button"`, так что в данном случае это Submit. Обработчик щелчка здесь не нужен, его роль возьмет на себя обработчик отправки данных формы

```

    <button type="button" onClick={handleCancel}>
      Cancel
    </button>
  </div>
</form>
);
}
export default Add;

```

Кнопка Cancel не должна отправлять данные формы, поэтому приходится явно задать ее тип, а затем инициировать обратный вызов отмены по щелчку

Это наше первое большое приложение, и мы настоятельно советуем поэкспериментировать с исходным кодом, если вы этого еще не делаете, разбираясь в примерах. Постарайтесь понять, как все, что мы говорили о свойствах, событиях, состояниях, JSX и композиции компонентов, сводится воедино и дополняется новыми знаниями о работе с формами — и в результате получается небольшое, но мощное приложение.

Расширим пример и добавим в него новые элементы. Можно создать отдельную форму для категоризации новых задач, а затем вывести в форме раскрывающийся список категорий, в котором пользователь выберет нужную. Конечно, для этого нужно уметь пользоваться раскрывающимися списками, но это тема следующего раздела. Также можно добавить режим календаря, в котором все задачи будут отображаться в сетке. К задачам можно добавить новые свойства, например ожидаемую продолжительность, чтобы приложение ежедневно выводило ее сводку для всех задач.

Отправка данных формы или щелчок кнопки?

При использовании отправки данных формы, по сути, производится их сбор в форме и отправка удаленному сервису или в другое хранилище. Так как в этом разделе используется управляемый ввод, состояние компонента React может считаться источником истины и значение можно читать непосредственно из состояния.

Для формы нужно создать обработчик отправки данных. Как уже говорилось, обработчик отправки данных формы будет автоматически вызываться браузером в двух случаях:

- Пользователь щелкает на кнопке **Submit** внутри формы.
- Поле ввода в форме с кнопкой **Submit** обладает фокусом, а пользователь нажимает **Enter**.

Так как обработчик отправки данных формы обрабатывает оба случая, это то, что нужно. Размещение обработчика непосредственно как обработчика щелчков кнопки **Submit** приведет к неправильной обработке отправки данных, если пользователь передаст фокус полю ввода и нажмет **Enter**.

9.2.5. Другие виды полей ввода

В предыдущем примере мы кратко познакомились с другой разновидностью полей ввода. Мы включили поле ввода даты, и как видно из рис. 9.14, в браузере оно выглядит по-другому. Это не единственный специальный тип — существует много других полезных разновидностей ввода. На рис. 9.15 (копия рис. 9.1) представлены все типы полей ввода, которые могут использоваться в формах HTML вообще и в формах React в частности.

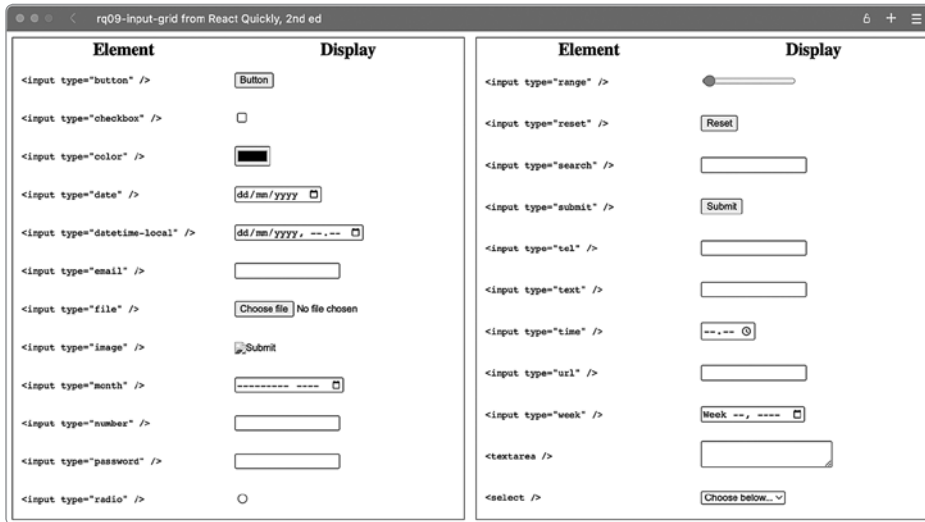


Рис. 9.15. Типы ввода, доступные для форм HTML и React. В основном это разновидности полей ввода, но есть и кнопки, и раскрывающиеся списки

Некоторые из этих типов ввода — всего лишь модификации, упрощающие ввод данных на устройствах с меньшим размером экрана. Например, поле с `type="tel"` при получении фокуса на мобильном устройстве отображает на клавиатуре только кнопки, используемые при вводе телефонных номеров (цифры, +, – и несколько других). Другие поля намного сложнее и имеют намного более сложный интерфейс.

Некоторые из этих полей в действительности не поля ввода, а кнопки. К их числу относятся кнопки с типами `reset`, `submit`, `button` и `image` на рис. 9.15.

Многие из этих полей ввода имеют такой же API, как поля для ввода текста, так что с точки зрения React работать с ними очень просто. Обратите внимание: в предыдущем примере нам ничего не пришлось изменять в React только

потому, что мы использовали поле данных. Свойства и события использовались как раньше.

Тем не менее некоторые поля ввода немного отличаются от других и требуют использовать ввод и события по-другому. В следующих подразделах мы рассмотрим все типы полей ввода с краткими примерами, поясняющими, как с ними работать.

ПРИМЕЧАНИЕ Все примеры этой главы включены в репозиторий `rq09-todo`, упомянутый выше, так как все они представляют собой разновидности формы добавления задач, используемой в примере. Однако эти файлы имеют ограниченную функциональность, так как компонент списка в этом приложении настроен для работы только с самой формой добавления, которая использовалась ранее.

Один тип полей ввода мы рассматривать не будем. Поле ввода для работы с файлами не может использоваться с управляемым вводом, поэтому мы опишем его немного позже, когда перейдем к неуправляемому вводу.

Числовые поля ввода

Числовые поля ввода почти не отличаются от текстовых полей ввода, за одним исключением. Нормальным значением свойства `value` целевого объекта является строка, но поскольку мы работаем с числами, текущее значение поля ввода нам требуется в числовом виде. Чтобы его получить, можно обратиться к свойству `evt.target.valueAsNumber` вместо обычного свойства `evt.target.value`. Не нужно вручную парсить ввод и преобразовывать его в строку в подходящей системе счисления — JavaScript сделает это автоматически.

Числовые поля ввода делятся на два вида: собственно числовые и календарные. К числовым полям ввода относятся типы `"number"` и `"range"`, а к календарным — типы `"date"`, `"datetime-local"`, `"month"`, `"time"` и `"week"`.

При использовании календарного поля ввода свойство `value` будет возвращать выбор в виде строки, зависящей от локального выбора языка и других настроек. Таким образом, для поля `type="week"` свойство `value` может вернуть строку `"2022-w52"`. С другой стороны, свойство `valueAsNumber` возвращает метку времени для многих других календарных полей ввода — количество миллисекунд от 1 января 1970 года до выбранной даты и/или времени. Таким образом, для той же 52-й недели 2022 года будет возвращено число `1672012800000`.

Поля `"month"` — особая разновидность календарных полей, так как они возвращают количество месяцев от января 1970 года до выбранного месяца. Таким образом, если выбрать декабрь 2022 года, свойство `valueAsNumber` вернет `635` — количество

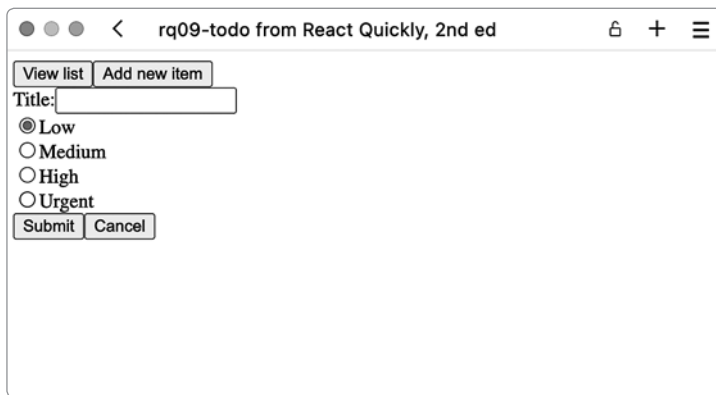
прошедших месяцев. Числовые поля ввода получают свойства `min`, `max` и `step`. Они обозначают диапазон допустимых значений и величину шага, на которую будет прирастать значение, если для его изменения используется клавиатура.

React не видит разницы, задается ли значение числового поля в виде числа или в виде строки. Перед отображением оно все равно будет преобразовано в число, так что если вы передаете данные, не подходящие для вывода в заданном поле, они будут отображаться некорректно. По этой причине, чтобы избежать проблем с преобразованиями, лучше хранить значения в виде чисел в JavaScript.

Чекбоксы и радиокнопки

Чекбоксы и радиокнопки занимают особое положение, потому что у них нет значения — или по крайней мере изменяющегося значения. Значением является идентификатор, который сообщает, что означает чекбокс или радиокнопка, но он не содержит информации о том, установлено данное состояние или сброшено.

Для примера возьмем форму, показанную на рис. 9.16. Четыре радиокнопки представляют собой независимые элементы `<input>`, но их значения статичны — это просто четыре варианта приоритета. Динамическая часть элемента содержит информацию о том, какая радиокнопка выбрана в списке. По этой причине у двух разновидностей полей ввода, чекбоксов и радиокнопок, имеется свойство `checked`, которому присваивается `true` или `false` для управления состоянием компонента.



The screenshot shows a web browser window with the title "rq09-todo from React Quickly, 2nd ed". Inside the browser, there is a form with the following elements: a "View list" button, an "Add new item" button, a "Title:" label followed by a text input field, four radio buttons labeled "Low", "Medium", "High", and "Urgent" (the "Low" radio button is selected), and "Submit" and "Cancel" buttons.

Рис. 9.16. Использование радиокнопок для выбора приоритета

Начнем с создания формы с четырьмя радиокнопками. Ее реализация приведена в листинге 9.9.

Листинг 9.9. Компонент формы с радиокнопками (фрагмент)

```

import { useState } from "react";
function Radio({ value, label, onChange, current }) {
  return (
    <label>
      <input
        type="radio"
        name="importance"
        checked={value === current}
        value={value}
        onChange={onChange}
      />
      {label}
    </label>
  );
}
function Add({ handleAdd, handleCancel }) {
  const [data, setData] = useState({ title: "", importance: "low" });
  const onChangeTitle = (evt) =>
    setData(oldData => ({ ...oldData, title: evt.target.value }));
  const onChangeImportance = (evt) =>
    setData(oldData =>
      ({ ...oldData, importance: evt.target.value }));
  ...
  <Radio
    value="low"
    label="Low"
    current={data.importance}
    onChange={onChangeImportance}
  />
  <Radio
    value="medium"
    label="Medium"
    current={data.importance}
    onChange={onChangeImportance}
  />
  <Radio
    value="high"
    label="High"
    current={data.importance}
    onChange={onChangeImportance}
  />
  <Radio
    value="urgent"
    label="Urgent"
    current={data.importance}
    onChange={onChangeImportance}
  />
  ...
}

```

Создает вспомога-
тельный компонент,
который рендерит
ярлык с внутренней
радиокнопкой

Назначает всем радиокнопкам
одинаковые имена в этом компоненте,
чтобы обозначить их принадлежность
к одной группе радиокнопок

Присваивает checked значение true только
для текущей выбранной радиокнопки

Присваивает значение, статическое для
каждого экземпляра этого компонента

Всем экземплярам
назначается один
обработчик изменения

Обработчик изменения
работает как обычно

Создает четыре экземпляра
компонента Radio

Создадим другую разновидность формы с задачами, подобными приведенной выше. На этот раз сохраняться будет только название задачи и степень ее срочности — логическое значение, которое будет храниться в объекте задачи. В форме этот признак будет представлен чекбоксом.

Листинг 9.10. Компонент формы с чекбоксом (фрагмент)

```
import { useState } from "react";
function Add({ handleAdd, handleCancel }) {
  const [data, setData] = useState({ title: "", isUrgent: false });
  const onChangeTitle = (evt) =>
    setData((oldData) =>
      ({ ...oldData, title: evt.target.value }));
  const onChangeUrgent = (evt) =>
    setData((oldData) =>
      ({ ...oldData, isUrgent: evt.target.checked }));
  ...
  <label>
    Title:
    <input
      value={data.title}
      onChange={onChangeTitle}
    />
  </label>
  <label>
    <input
      type="checkbox"
      checked={data.isUrgent}
      onChange={onChangeUrgent}
    />
    Urgent?
  </label>
  ...
}
```

Реализация обработчика изменения для поля названия основана на обращении к значению целевого свойства, как мы это уже не раз делали

Задает значения свойств value и onChange как обычно

Задает значения свойств checked и onChange для чекбокса. Обратите внимание: свойство value задавать не нужно, потому что в данном случае оно не выполняет никакой функции

Обработчик изменения для чекбокса устроен иначе. Он проверяет логическое свойство .checked для целевого свойства

Раскрывающиеся списки

Раскрывающиеся списки на первый взгляд сильно отличаются от других полей ввода в HTML. Раскрывающиеся списки состоят из нескольких элементов, а выбор в них обозначается совершенно иначе. Впрочем, React значительно упрощает работу с этим типом поля ввода. В отношении используемых свойств раскрывающиеся списки мало чем отличаются от обычных элементов ввода. Конечно, в них придется добавлять варианты выбора.

Реализуем пример с приоритетами, показанный на рис. 9.16, но на этот раз с раскрывающимся списком. Примерный вид итогового приложения показан на рис. 9.17. Получить его на удивление просто, потому что нам помогает React.

Листинг 9.11. Компонент формы с раскрывающимся списком (фрагмент)

```
import { useState } from "react";
function Add({ handleAdd, handleCancel }) {
  const [data, setData] = useState({
    title: "",
    priority: "low", ← Инициализирует priority простой строкой
  });
  const onChange = (evt) => {
    const key = evt.target.name;
    const value = evt.target.value;
    setData((oldData) =>
      ({ ...oldData, [key]: value }));
  };
  ...
  <label>
    Priority:
    <select
      value={data.priority}
      name="priority"
      onChange={onChange}
    >
      <option value="low">Low</option>
      <option value="medium">Medium</option>
      <option value="high">High</option>
      <option value="urgent">Urgent</option>
    </select>
  </label>
  ...
}
```

Один обработчик изменения может использоваться как для обычных полей ввода, так и для раскрывающихся списков

Задает значения свойств value и onChange как для обычных полей, непосредственно для элемента select

Для добавления вариантов выбора используются элементы option со значением и выводимым текстом

Если вы уже пользовались раскрывающимися списками в HTML, то знаете, что обычно для обозначения выбранного варианта приходится задавать свойство `selected` отдельных элементов `<option>`. В HTML у элемента `select` нет свойства `value`. Однако React позаботился, чтобы раскрывающиеся списки были удобны, поэтому их API совпадает с API полей ввода, и это здорово!

Можно даже добавить поля с множественным выбором, чтобы пользователь мог выбрать более одного варианта. Представьте, что у вас есть раскрывающийся список с именами и пользователь может выбрать в нем ответственных за работу над задачей. Если хранить массив в локальном состоянии, можно использовать массив вариантов в качестве значения компонента, как показано в листинге 9.12.

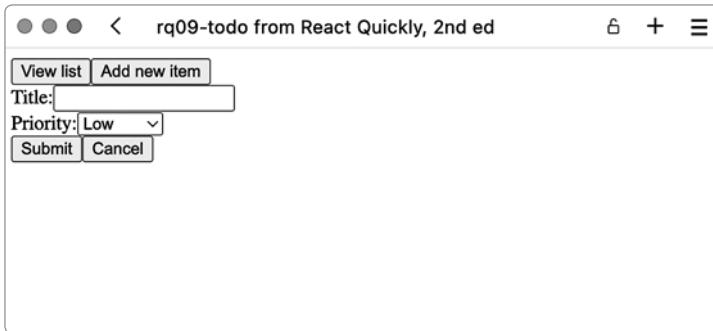


Рис. 9.17. В новой версии приоритет выбирается из раскрывающегося списка

Листинг 9.12. Компонент формы со списком с множественным выбором (фрагмент)

```
import { useState } from "react";
function Add({ handleAdd, handleCancel }) {
  const [data, setData] = useState({
    title: "",
    people: [],
  });
  const onChange = (evt) => {
    const key = evt.target.name;
    const value = evt.target.value;
    setData((oldData) => ({ ...oldData, [key]: value }));
  };
  const onChangePeople = (evt) => {
    const options =
      Array.from(evt.target.selectedOptions);
    const value = options.map((opt) => opt.value);
    setData((oldData) => ({ ...oldData, people: value }));
  };
  ...
  <label>
    People:
    <select
      value={data.people}
      name="people"
      onChange={onChangePeople}
      multiple
    >
      <option>Tinky Winky</option>
      <option>Po</option>
      <option>Laa-Laa</option>
      <option>Dipsy</option>
    </select>
  </label>
  ...
}
```

Значение состояния представляет собой обычный массив, и его можно инициализировать пустым массивом

Однако приходится создавать специальный обработчик изменения, потому что придется обращаться к списку выбранных вариантов для целевого объекта

Для каждого выбранного варианта необходимо извлечь свойство value. Полученный массив значений option сохраняется в состоянии

Свойства присваиваются как обычно, но теперь также задается свойство "multiple"

Поддержка списков с множественным выбором требует дополнительных усилий, но необходимость в таких списках возникает редко. Тем не менее это очень полезный инструмент, который стоит освоить для таких редких ситуаций.

Многострочные поля ввода

Многострочные текстовые поля в HTML называются *текстовыми областями* (text area). Текстовые области на первый взгляд очень похожи на поля ввода в HTML; главное их отличие в том, что значение текстовой области поля добавляется не как значение свойства `value` элемента, а как дочерний текстовый узел. Если полю ввода в HTML нужно задать значение `"this text"`, можно воспользоваться свойством `value`:

```
<input value="this text" />
```

Чтобы сделать то же в текстовой области, необходимо задать текст как дочерний элемент:

```
<textarea>this text</textarea>
```

Однако в React так поступать не нужно. В React текстовые области используются так же, как если бы они были текстовыми полями. Таким образом, для добавления поля с описанием в форму с задачами достаточно расширить форму.

Листинг 9.13. Компонент формы с текстовой областью (фрагмент)

```
import { useState } from "react";
function Add({ handleAdd, handleCancel }) {
  const [data, setData] = useState({
    title: "",
    description: "",
  });
  const onChange = (evt) =>
    setData((oldData) =>
      ({ ...oldData, [evt.target.name]: evt.target.value }));
  ...
  <label>
    Description:
    <textarea
      value={data.description}
      name="description"
      onChange={onChange}
    />
  </label>
  ...
```

Значение состояния снова инициализируется строкой

Задаёт свойство value непосредственно для элемента textarea — не нужно делать его дочерним узлом

Может использовать тот же обобщенный обработчик изменения, что и прежде, при условии, что будет задано свойство name

Освоив эти три дополнительных типа полей ввода, вы сможете уверенно создавать даже сложные формы и правильно обрабатывать данные в коде, пригодном для повторного использования, чтобы избежать дублирования.

9.2.6. Другие свойства

Все остальные свойства работают со всеми типами ввода как обычно. Дело в том, что большую часть дополнительной функциональности обеспечивает HTML, поэтому в React не требуется специально ничего делать, чтобы пользоваться этими возможностями. Вот некоторые свойства, которые можно добавлять к полям ввода для управления работой формы:

- **Required** — если задать это свойство для поля ввода, то поле ввода станет обязательным. Если оно не будет заполнено, отправить форму не получится — браузер не активирует обратный вызов `onSubmit`. Если поле что-то содержит, отправка данных работает как обычно. Это логическое свойство, так что его достаточно включить в виде `<input required />`.
- **min, max и step** — эти свойства, используемые при вводе числовых данных и диапазонов, управляют допустимыми диапазонами значений. Их можно использовать в поле `range` в следующем виде: `<input type="range" min="100" max="200" step="10" />`.
- **readOnly** — это свойство делает поле ввода доступным только для чтения. Пользователь не сможет изменять значение в поле ввода, и для него никогда не будет активирован обработчик `onChange`. Свойство также является логическим. Обратите внимание на прописную `O` в имени свойства.
- **Disabled** — если свойство задано, то поле ввода блокируется. Заблокированное поле отличается от поля, доступного только для чтения, тем, что ему невозможно передать фокус. Поля, доступные только для чтения, продолжают считаться частью отправляемых данных формы, тогда как заблокированные поля в отправке не участвуют. Это логическое свойство.
- **List** — если свойству присваивается идентификатор элемента `<datalist>` где-то в документе, то список данных предоставляет доступные для ввода в поле варианты. Этот режим отчасти напоминает раскрывающийся список, за исключением того, что `list` содержит только рекомендации, так что поле ввода не ограничивается этими значениями.
- **maxLength** — определяет максимальное количество символов, разрешенных в поле ввода; многие браузеры управляют этим значением автоматически. Обратите внимание на прописную `L` в имени свойства.

Существует много других свойств, которые здесь не рассмотрены. Все они работают в HTML, а не относятся к специфике React. Если вы захотите узнать о них больше, рекомендуем обратиться к документации MDN для полей ввода: <http://mng.bz/WzAg>.

9.3. КОНТРОЛЬ НЕУПРАВЛЯЕМОГО ВВОДА

Создадим еще один очень простой калькулятор — на этот раз только с одним полем ввода. Требуется построить компонент, который для заданного числа возвращает сумму всех целых чисел, не превышающих его. Таким образом, для входного значения 4 возвращается $1 + 2 + 3 + 4 = 10$.

Значение вычисляется по очень простой формуле $n*(n+1)/2$, где n — число, для которого вычисляется сумма. На этот раз мы не будем вычислять итоговое значение, пока пользователь не щелкнет на кнопке **Submit**. Дерево для такого управляемого компонента изображено на рис. 9.18.

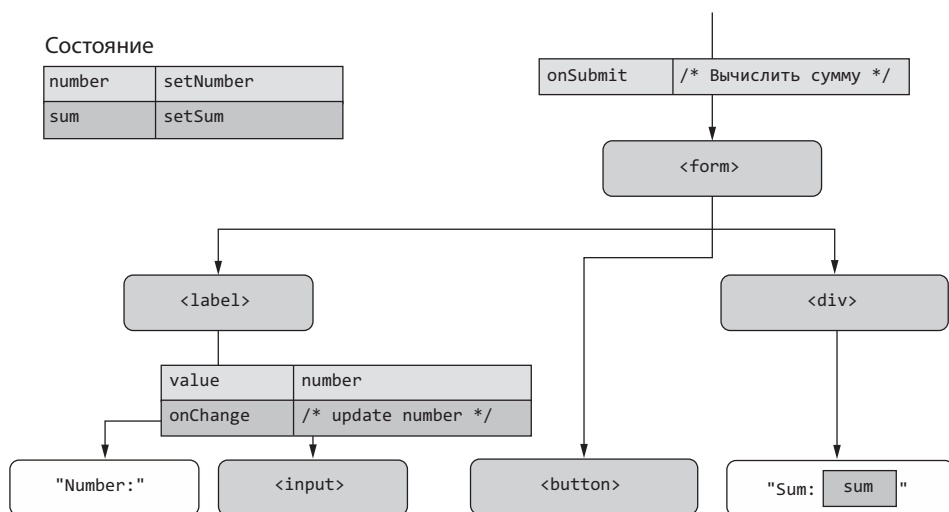


Рис. 9.18. Выходная разметка JSX для вычисления суммы целых чисел. При отправке данных формы вычисляется сумма, которая выводится в соответствующем элементе

Создадим локальную переменную состояния для хранения входящего значения, как уже не раз делали до этого. На этот раз также понадобится другая

переменная для хранения суммы, так как сумма изменяется только при отправке формы.

Для чего же нужна переменная `number`? Только для того, чтобы ее можно было передать управляемому компоненту ввода. Конечно, это очень удобно, и так мы полностью контролируем поле ввода, но на самом деле контроль не нужен, потому что пользователь может ввести любое число по своему выбору (при условии, что мы задали свойство `min="0"`, так как сумма не может вычисляться для отрицательных чисел).

То же можно сделать другим способом. Можно полностью отказаться от контроля поля ввода, просто создать элемент HTML и хранить его, пока он не понадобится. Нас интересует только значение на момент отправки формы; на самом деле не нужно перегружать компонент управлением состоянием поля ввода во время работы.

Недостатком такого подхода может быть то, что мы управляем только начальным значением компонента; после этого с ним уже ничего нельзя сделать. Но нам и не нужно контролировать значение в этом компоненте, так что все в порядке. Если реализовать эту схему, остается только поддерживать переменную `sum` в состоянии и вычислять ее в обработчике события отправки данных формы. Полученное дерево компонентов представлено на рис. 9.19.

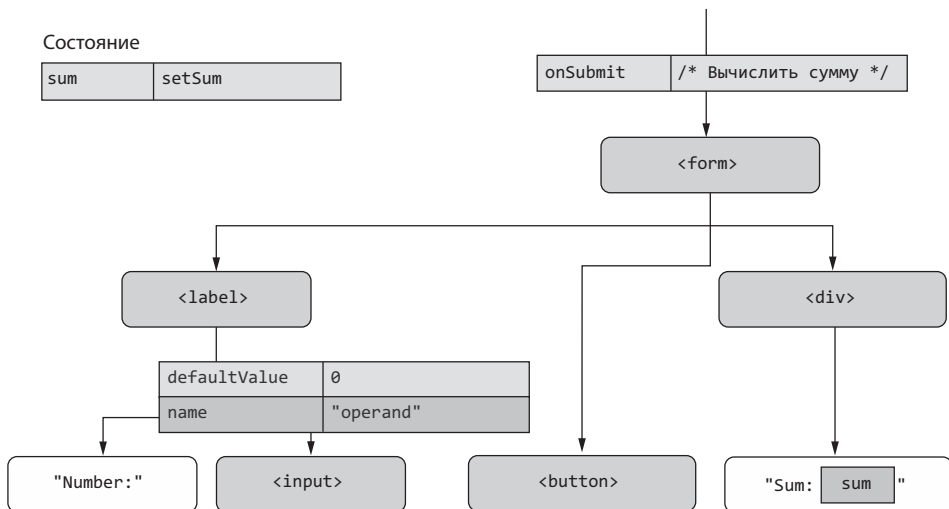


Рис. 9.19. Выходная разметка JSX для вычисления суммы целых чисел с неуправляемым компонентом ввода

Остается понять, как обратиться к числу в поле ввода. Можно создать ссылку на поле ввода и использовать запись `ref.current.valueAsNumber`, но это не обязательно. Событие отправки данных содержит свойство `target` со ссылкой на элемент формы, а элемент формы позволяет обратиться по прямой ссылке ко всем своим полям ввода при помощи коллекции `.elements`. Таким образом, если полю ввода назначено имя "operand", к нему можно обратиться через объект отправки данных формы в виде `evt.target.elements.operand.valueAsNumber`. Приемлемо. Реализация приведена в листинге 9.14.

Листинг 9.14. Сумма натуральных чисел

```
import { useState } from "react";
function NaturalSum() {
  const [sum, setSum] = useState(0);
  const onSubmit = (evt) => {
    const value =
      evt.target.elements.operand.valueAsNumber;
    const naturalSum = (value * (value + 1)) / 2;
    setSum(naturalSum);
    evt.preventDefault();
  };
  return (
    <form
      onSubmit={onSubmit}
      style={{ display: "flex", flexDirection: "column" }}
    >
      <label>
        Number:
        <input
          type="number"
          min="1"
          defaultValue="1"
          name="operand"
        />
      </label>
      <div>
        <button>Submit</button>
      </div>
      <div>Sum: {sum}</div>
    </form>
  );
}
function App() {
  return <NaturalSum />;
}
export default App;
```

← Данные ввода вообще не хранятся в состоянии

Входящие значения необходимо считывать не из состояния, а через DOM. К счастью, это очень легко сделать через элемент формы

Задаёт `defaultValue`, но не значение элемента ввода, и свойство `name`, чтобы элемент было легко найти через форму

Репозиторий: rq09-natural-sum

Этот пример находится в репозитории `rq09-natural-sum`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq09-natural-sum --template rq09-natural-sum
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq09-natural-sum>

Должно работать. Попробуйте запустить приложение в браузере. Примерный результат показан на рис. 9.20.

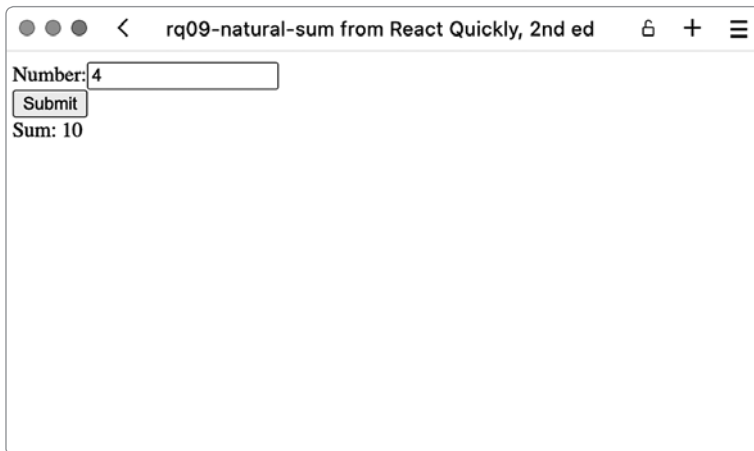


Рис. 9.20. Калькулятор вычисляет верную сумму натуральных чисел для ввода 4

Заметим, что мы также могли определить обработчик события изменения для поля ввода, читающий значение при изменении, но это не нужно.

Что же мы приобрели и что потеряли? В табл. 9.1 представлено сравнение двух решений.

Таблица 9.1. Различия между управляемым и неуправляемым вводом

	Управляемый ввод	Неуправляемый ввод
Присваивание начального значения	Да	Нет
Чтение значения при изменении	Необходимо	Можно при желании, но не нужно
Чтение значений при отправке данных	Легко, они хранятся в состоянии	Приходится действовать через DOM, но это возможно
Значения состояния	Необходимы для каждого поля ввода	Вообще не нужны
Изменение значений на ходу	Легко	Очень трудно (но можно сделать через DOM)
Источник истины	Значение состояния компонентов	Значение DOM

9.3.1. Возможности

Когда же следует применять неуправляемый ввод? Таблица 9.1 подсказывает ответы. Вспомним адресную форму из листинга 9.5: нам были нужны значения состояния и обработчики изменений, но мы с ними практически ничего не делали. Мы просто копировали значения DOM в значения состояния и обратно.

Реализовать эту же форму с неуправляемыми полями ввода на удивление просто. Вспомните, что в листинге 9.5 форма ни для чего не использовалась. У формы не было обработчика отправки данных. Мы редактировали данные, но никуда их не отправляли.

Предположим, вам нужно создать адресную форму, данные из которой при отправке должны передаваться удаленному сервису в POST-запросе по URL-адресу `//salespower.invalid/api/address`. Заметим, что это условный URL-адрес; он работать не будет (домен верхнего уровня `.invalid` также указывает на это). Если дополнить пример из листинга 9.5 обработчиком отправки данных, который отправляет данные в POST-запросе по этому URL-адресу, мы получим листинг 9.15.

Рассмотрим эту схему с неуправляемыми полями ввода. Прежде всего, в этом случае не понадобятся никакие значения состояния и обработчики изменения. Уже только это обстоятельство значительно упростит форму.

С другой стороны, усложнится обработчик отправки данных, потому что хотя в `evt.target.elements` доступен объект со значениями состояния, это не собственно список значений состояния, а объект самих элементов ввода. Однако этот объект также содержит все поля ввода формы с нумерованными индексами, так что `form.elements[0]` обозначает первый элемент формы и т. д. Кнопка `Submit` также является элементом формы, но можно ограничиться первыми шестью полями формы, потому что известно, что пригодятся только они. Необходимо перебрать список элементов, извлечь имя и значение каждого элемента и поместить их в объект.

Листинг 9.15. Управляемая адресная форма с отправкой данных

```
import { useState } from "react";
const URL = "/salespower.invalid/api/address";
function Address() {
  const [data, setData] = useState({
    address1: "",
    address2: "",
    zip: "",
    city: "",
    state: "",
    country: "",
  });
  const onChange = (evt) => {
    const key = evt.target.name;
    const value = evt.target.value;
    setData((oldData) =>
      ({ ...oldData, [key]: value }));
  };
  const onSubmit = (evt) => {
    fetch(URL, {
      method: "POST",
      body: JSON.stringify(data),
    });
    evt.preventDefault();
  };
  return (
    <form
      onSubmit={onSubmit}
      style={{ display: "flex", flexDirection: "column" }}
    >
      <label>
        Address line 1:
        <input
          value={data.address1}

```

Сначала
инициализируется
состояние

Создает обработчик
изменения, который
может обновить
состояние

Использует состояние как данные,
передаваемые обработчику
отправки данных

```

    name="address1"
    onChange={onChange}
  />
</label>
<label>
  Address line 2:
  <input
    value={data.address2}
    name="address2"
    onChange={onChange}
  />
</label>
<label>
  Zip:
  <input
    value={data.zip}
    name="zip"
    onChange={onChange}
  />
</label>
<label>
  City:
  <input
    value={data.city}
    name="city"
    onChange={onChange}
  />
</label>
<label>
  State:
  <input
    value={data.state}
    name="state"
    onChange={onChange}
  />
</label>
<label>
  Country:
  <input
    value={data.country}
    name="country"
    onChange={onChange}
  />
</label>
<button>Submit</button>
</form>
);
}
export default Address;

```

← Назначает
 обработчик onChange
 каждому полю ввода

← Кнопка Submit

Листинг 9.16. Неуправляемая адресная форма с отправкой данных (фрагмент)

```

const URL = "//salespower.invalid/api/address";
function Address() {
  const onSubmit = (evt) => {
    const data = Object.fromEntries(
      Array.from(evt.target.elements)
        .slice(0, 6)
        .map((input) => [input.name, input.value])
    );
    fetch(URL, {
      method: "POST",
      body: JSON.stringify(data),
    });
    evt.preventDefault();
  };
  return (
    <form
      onSubmit={onSubmit}
      style={{ display: "flex", flexDirection: "column" }}
    >
      <label>
        Address line 1:
        <input name="address1" />
      </label>
      <label>
        Address line 2:
        <input name="address2" />
      </label>
      <label>
        Zip:
        <input name="zip" />
      </label>
      <label>
        City:
        <input name="city" />
      </label>
      <label>
        State:
        <input name="state" />
      </label>
      <label>
        Country:
        <input name="country" />
      </label>
      <button>Submit</button>
    </form>
  );
}
export default Address;

```

Главное изменение — обработчик отправки данных, в котором текущие данные извлекаются непосредственно из формы (вместо чтения из локального состояния компонента, как в предыдущем примере)

Добавляет обработчик отправки данных к объекту формы

Кнопка Submit

Подсчитаем символы в двух листингах: управляемый вариант в листинге 9.15 содержит 1441 символ, а неуправляемый вариант в листинге 9.16 — всего 1022 символа. Объем кода сократился примерно на 30%! Кроме того, управляемый компонент рендерится каждый раз, когда пользователь что-то вводит, тогда как неуправляемый компонент вообще обходится без повторного рендеринга!

Кажется, неуправляемая форма лучше, — и это действительно так, если форма очень проста и ничего не нужно контролировать. Но если требуется нестандартный контроль над формой с такими параметрами, как проверка данных, ограничения, форматирование и т. д., то нужно сделать управляемыми хотя бы те поля, к которым они относятся. Вообще, если вам нужна простая адресная форма, которая не использует ни проверку данных, ни правила, а только управляет входящие значения по целевому URL-адресу методом POST, вам вообще не нужен React (или JavaScript). Со всем этим справится самая обычная форма HTML.

Потенциал React раскрывается только в сложных веб-приложениях и при работе со сложными формами. Если потребуется добавить проверку данных в какую-либо из двух предыдущих форм, то сделать это в управляемом примере из листинга 9.15 будет намного проще, чем в неуправляемом примере из листинга 9.16. А если ничего этого не нужно, то скорее всего, не нужно и контролировать поля ввода в компоненте React. Собственно, не нужен и сам React.

9.3.2. Поля ввода файлов

Поля ввода файлов могут быть только неуправляемыми, потому что свойство `value` в DOM защищено — эта защита является частью средств безопасности браузера. Нельзя напрямую задать значение поля ввода файлов; можно только прочесть его после того, как пользователь выберет файл для отправки. Единственное, что можно сделать, — сбросить текущее значение, но не получится вручную изменить значение поля или инициализировать его.

Таким образом, в React поля ввода файлов не могут быть управляемыми ни при каких условиях. Если попытаться создать компонент `<input type="file" value={file} />`, браузер начнет грязно ругаться, примерно так:

```
Uncaught DOMException: Failed to set the 'value' property on 'HTMLInputElement':
This input element accepts a filename, which may only be programmatically set to
the empty string1.
```

¹ Неперехваченное исключение `DOMException`: не удалось задать свойство `'value'` для `'HTMLInputElement'`: этот элемент ввода получает имя файла, которому программным путем может быть присвоена только пустая строка.

Таким образом, если вам потребуется включить в форму React поле ввода файлов, придется (по меньшей мере) сделать это поле неуправляемым. Но поскольку проверять, ограничивать или форматировать его текущее значение никогда не придется, сложности это вызвать не должно.

9.4. ВОПРОСЫ

1. Исходное значение можно задать только для управляемых полей ввода, но не для неуправляемых. *Да или нет?*
2. Какой обработчик события используется для обработки ввода в раскрывающемся списке?
 - a) `onValue`
 - b) `onChange`
 - c) `onSelect`
 - d) `onClick`
3. Какое из следующих свойств вы используете для чтения нового состояния чекбокса в обработчике события?
 - a) `evt.target.value`
 - b) `evt.target.selected`
 - c) `evt.target.checked`
 - d) `evt.target.valueAsBoolean`
4. Если нужно обратиться к неуправляемому полю ввода с именем "email" в обработчике события формы, какой из приведенных вариантов синтаксиса следует использовать?
 - a) `evt.target.inputs.email`
 - b) `evt.target.email`
 - c) `evt.target.nodes.email`
 - d) `evt.target.elements.email`
5. Какие два свойства являются обязательными для управляемого элемента ввода?
 - a) `name`
 - b) `value`
 - c) `defaultValue`
 - d) `onChange`

ОТВЕТЫ

1. Нет. Исходное значение можно задать в обоих режимах. Не получится обновить значение после того, как для неуправляемого поля будет определено исходное значение, но задать исходное значение можно.
2. Всегда используется обработчик события `onChange` независимо от того, с каким элементом ввода в форме вы работаете.
3. Состояние элемента чекабокса хранится в свойстве `checked`, поэтому для обращения к нему в обработчике события пишется `evt.target.checked`.
4. Для обращения к элементам форм через элемент `form` в DOM используется свойство `elements`. Элемент формы с именем "email" доступен через обращение `elements.email`.
5. У всех управляемых полей ввода всегда должны быть определены свойства `value` и `onChange`.

ИТОГИ

- Работа с данными форм — одна из важнейших составляющих React. Работать с формами и полями ввода в приложениях React очень удобно.
- Элементы ввода форм могут быть управляемыми или неуправляемыми.
- При работе с формами рекомендуется использовать управляемые поля ввода, которые предоставляют возможности проверки данных, изменения и оперативной фильтрации ввода.
- Чтобы использовать управляемый ввод, необходимо задать свойства `value` и `onChange` для элемента ввода в JSX и подтверждать каждое изменение обновлением свойства `value`.
- Альтернативное решение — неуправляемый ввод. Этот режим сокращает возможности модификации данных, но он также уменьшает объем кода, необходимого для работы с формами при минимальных требованиях к контролю данных.
- Для неуправляемых полей ввода можно задать исходное значение в свойстве `defaultValue`, но задать свойство `value` не удастся.

- В React могут использоваться все виды полей ввода HTML, включая текстовые поля, числовые поля, поля для выбора даты и времени, поля для ввода пароля, чекбоксы, радиокнопки, раскрывающиеся списки, поля диапазонов, текстовые области и многое другое.
- Некоторые виды полей ввода используют отличающиеся API, например, для чтения состояния чекбоксов, радиокнопок и списков с множественным выбором.
- Поля ввода файлов могут быть только неуправляемыми, потому что значение такого поля не может контролироваться в JavaScript.

10

Расширенные хуки React для масштабирования

В ЭТОЙ ГЛАВЕ

- ✓ Структурирование потока данных с использованием React Context
- ✓ Управление сложным состоянием с использованием преобразований
- ✓ Создание нестандартных хуков для повторного использования кода

Итак, вы научились строить небольшие простые приложения React. У вас есть вся необходимая информация и инструменты для создания современных интерактивных, обладающих состоянием виджетов React, включающих набор взаимосвязанных компонентов, — но только для относительно небольших.

Реальные приложения React будут намного больше и сложнее любого из рассмотренных примеров. Можно создавать небольшие виджеты для сайтов (например, калькулятор индекса массы тела), которые состоят из пары компонентов и при этом неплохо работают, но такие проекты встречаются нечасто, и в основном ими занимаются непрофессиональные разработчики.

Профессионалы React разрабатывают собственные большие приложения самостоятельно или в команде. С ростом приложения компоненты интерфейсов усложняются, и работа с кодом требует большего мастерства.

Если вы будете разрабатывать приложения, не уделяя внимания структуре или технологии, вы можете столкнуться с проблемами, например:

1. Сложная логика передачи данных может привести к размножению свойств во всех компонентах для передачи всех необходимых данных.
2. Сложные пути передачи состояния могут стать причиной появления недействительных состояний, если не уделять должного внимания синхронизации взаимосвязанных значений состояния.
3. Дублирующийся код иногда плохо обобщается, если обобщать только целые компоненты, а не их отдельные части.

Все эти проблемы возникают независимо от того, работаете вы самостоятельно или в составе большой команды, и касаются вопросов масштабирования. То, что работает в малом масштабе, не обязательно сработает в большом.

Похожие проблемы масштабирования уже встречались при создании сложных форм в главе 9. Если форма содержит одно или два поля ввода, то использовать значение состояния для каждого поля достаточно удобно. Но если в форме пять или десять полей, создание отдельного значения состояния для каждого поля становится слишком хлопотным делом и, скажем честно, признаком плохого дизайна. При применении в большем масштабе некоторые концепции адаптируются для улучшения масштабирования.

В этой главе рассматриваются инструменты, которые чрезвычайно полезны при организации и структурировании приложений и проектов React в целом, чтобы вы могли создавать более качественные продукты и получать лучший опыт разработки.

Мы рассмотрим решения трех упомянутых выше проблем масштабирования:

1. Предоставление компонентам доступа к значениям независимо от глубины при помощи React Context — превосходный механизм организации сложных потоков данных. Эта тема рассматривается в разделе 10.1.
2. Для обновления состояния с множественными независимыми значениями можно воспользоваться преобразованием — эта концепция позаимствована из функционального программирования. Об этом будет рассказано в разделе 10.2.
3. Нестандартные хуки — отличный способ обобщения мелких и крупных блоков бизнес-логики, который будет рассмотрен в разделе 10.3. Нестандартные хуки быстро становятся основным механизмом предоставления функциональности, пригодной для повторного использования, как в грани-

цах проекта, так и в библиотеках с открытым исходным кодом, доступных на GitHub и/или npm.

ПРИМЕЧАНИЕ Исходный код примеров этой главы доступен по адресу <https://rq2e.com/ch10>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке одной командой.

10.1. РАЗРЕШЕНИЕ КОМПОНЕНТАМ ОБРАЩАТЬСЯ К ЗНАЧЕНИЯМ

Давайте снова создадим приложение для решения реальной проблемы. На этот раз в нем будет использоваться информационная панель — экран, который вы видите после входа в приложение. На панели выводится сообщение, которое приветствует вас по имени, а в правом верхнем углу находится кнопка с вашим именем и ссылкой на страницу персональных настроек. Задача усложняется тем, что имя является динамическим и вы получаете его от серверной части. Примерный вид приложения показан на рис. 10.1.

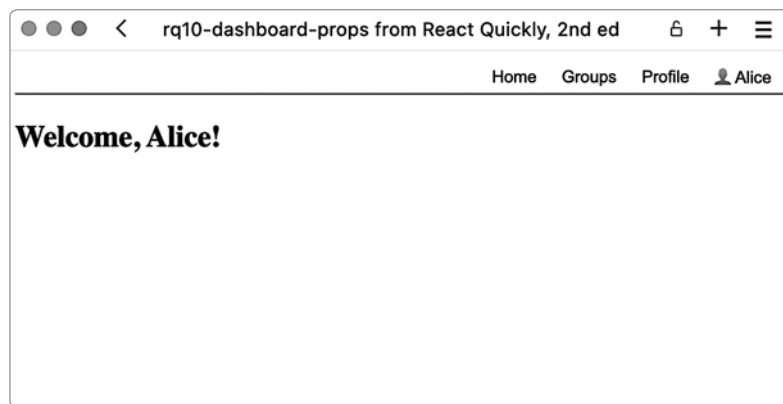


Рис. 10.1. Итоговый вид информационной панели. Имя пользователя выводится дважды, и в этом суть проблемы

Разобьем задачу на компоненты. Верхнее меню должно быть частью заголовка. Центральная страница с приветствием — всего лишь одна из многих страниц приложения. Мы знаем, что функциональность приложения будет расширяться, поэтому добавим дополнительные логические уровни на этот случай. Иерархия компонентов представлена на рис. 10.2.

Однако на рис. 10.2 не показано, как мы получаем значение `name` с самого верха дерева компонентов, откуда оно передается компоненту панели вплоть до двух меньших компонентов нижнего уровня, в которых оно должно выводиться.

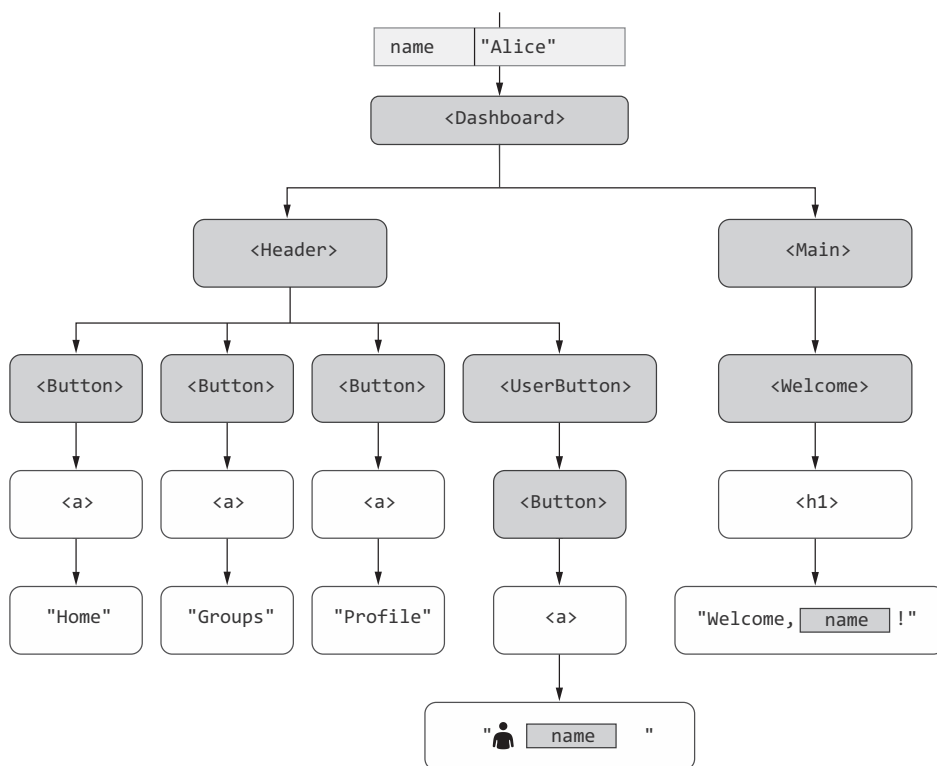


Рис. 10.2. Структура компонентов с заполнителями для имени пользователя. Обратите внимание: свойство `name` используется дважды, но нигде не передается как свойство

Если действовать так, как мы действовали до сих пор, нам пришлось бы передавать свойство через все компоненты на пути к тому, в котором оно используется. Структура компонентов выглядела бы так, как показано на рис. 10.3.

Но обратите внимание: в этом дереве компонентов свойство `name` передается как компоненту `Header`, так и компоненту `Main`. Ни одному из этих компонентов это свойство само по себе не нужно. Это свойство передается двум промежуточным компонентам, только чтобы они передали его следующему компоненту в цепочке. Однако такое решение работает, и его возможная реализация приведена в листинге 10.1.

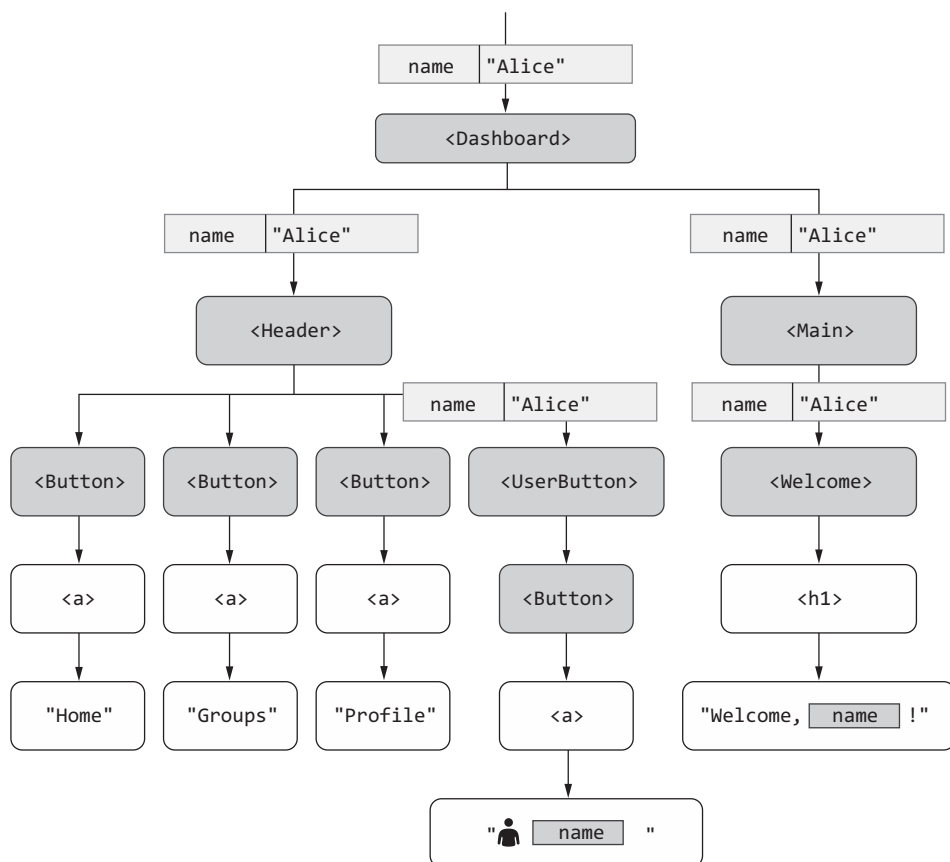


Рис. 10.3. Структура компонентов с передачей свойства всем промежуточным компонентам. Свойство пате должно присутствовать в пяти компонентах, но только два компонента выводят его

Листинг 10.1. Панель с лишними свойствами name

```
const BUTTON_STYLE = {
  display: "inline-block",
  padding: "4px 10px",
  background: "transparent",
  border: "0",
};
const HEADER_STYLE = {
  display: "flex",
  justifyContent: "flex-end",
  borderBottom: "1px solid",
```

```

};
function Button({ children }) {
  return (
    <button style={BUTTON_STYLE}>
      {children}
    </button>
  );
}
function UserButton({ name }) {
  return <Button> 🧑 {name}</Button>;
}
function Header({ name }) {
  return (
    <header style={HEADER_STYLE}>
      <Button>Home</Button>
      <Button>Groups</Button>
      <Button>Profile</Button>
      <UserButton name={name} />
    </header>
  );
}
function Welcome({ name }) {
  return (
    <section>
      <h1>Welcome, {name}!</h1>
    </section>
  );
}
function Main({ name }) {
  return (
    <main>
      <Welcome name={name} />
    </main>
  );
}
function Dashboard({ name }) {
  return (
    <>
      <Header name={name} />
      <Main name={name} />
    </>
  );
}
function App() {
  return <Dashboard name="Alice" />;
}
export default App;

```

А вы знали, что эмодзи можно использовать прямо в React? Да, это так!

Компонент получает свойство только для того, чтобы передать его другому компоненту

Передаёт свойство name компоненту, который на самом деле не будет использовать его

Репозиторий: rq10-dashboard-props

Этот пример находится в репозитории `rq10-dashboard-props`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq10-dashboard-props --template rq10-dashboard-props
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq10-dashboard-props>

Решение выглядит логично и работает. Открыв его в браузере, вы получите результат, показанный на рис. 10.1.

10.1.1. React Context

Свойства передаются компонентам только для того, чтобы быть переданными другому компоненту... Не похоже на качественную архитектуру. Должно существовать более элегантное решение. А что, если создать объект-хранилище, инкапсулирующий набор компонентов, который поставлял бы данные всем своим дочерним компонентам по запросу, без передачи лишних свойств?

Поздравляем, вы только что изобрели React Context. Контекст решает именно эту задачу — инкапсулирует компоненты со значением, которое все компоненты-потомки могут запрашивать без использования свойств.

Сквозная передача свойств (drilling)

Практика добавления свойств в компонент только для того, чтобы он передал их другим компонентам, которые, в свою очередь, просто передадут их другому уровню компонентов, называется *сквозной передачей свойств*. Свойство передается насквозь через несколько уровней компонентов, потому что его нужно передать с наружного уровня на внутренний.

Сквозная передача свойств очень быстро становится проблемой в больших базах данных, и контексты React — один из лучших инструментов для ее решения. Без применения подходящих паттернов проектирования (например, провайдеров контекста) в некоторых компонентах могут появиться десятки свойств, добавленных только для дальнейшей передачи по дереву компонентов.

Очевидно, это плохой дизайн, и он стал одной из причин популярности React Context.

Контекст в React состоит из двух частей. Ему необходим провайдер, содержащий значение, которое должно передаваться любым компонентам-потомкам, и потребитель, используемый в каждом компоненте-потомке, которому понадобится доступ к значению.

Провайдер контекста представляет собой относительно простой компонент React. Потребитель проще всего создать в форме хука `useContext`. По сути, контекст устроен примерно так, как показано на рис. 10.4.

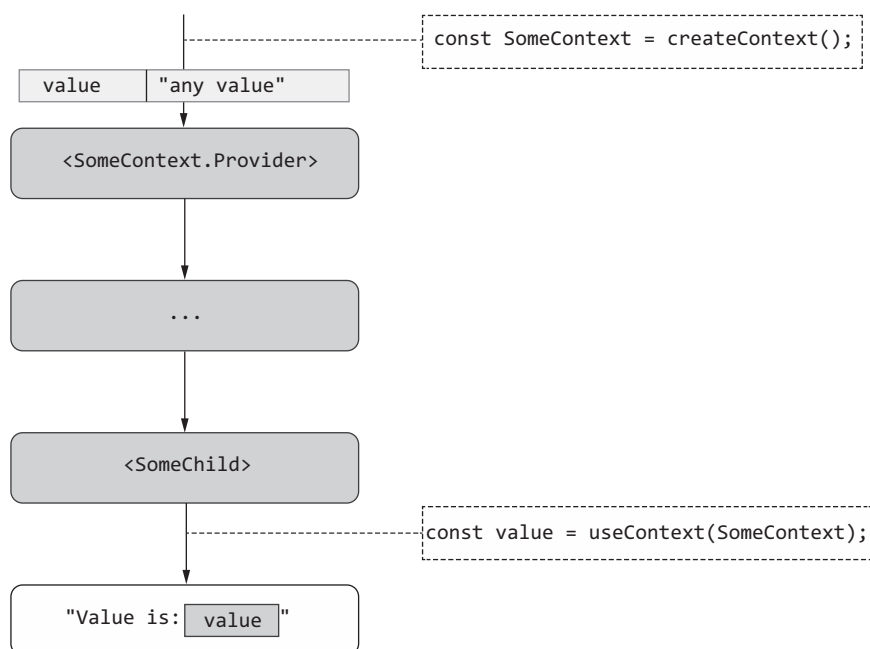


Рис. 10.4. Передача значения от провайдера к потребителю с использованием хука `useContext`

Для работы нам понадобятся два параметра React Context API. Во-первых, вызов `createContext`, который определяет контекст, сохраняемый в переменной. Переменная создается за пределами любых компонентов и существует в той же области видимости, что и другие компоненты, поэтому к ней можно обращаться как к любому другому компоненту. Во-вторых, хук `useContext`. Хук получает ссылку на контекст и возвращает текущее значение контекста. Добавим контекст `NameContext` в приложение с приведенным выше деревом компонентов, как показано на рис. 10.5.

Собственно, это все. Реализация приведена в листинге 10.2. Мы получаем тот же результат, но с намного более эффективной передачей данных.

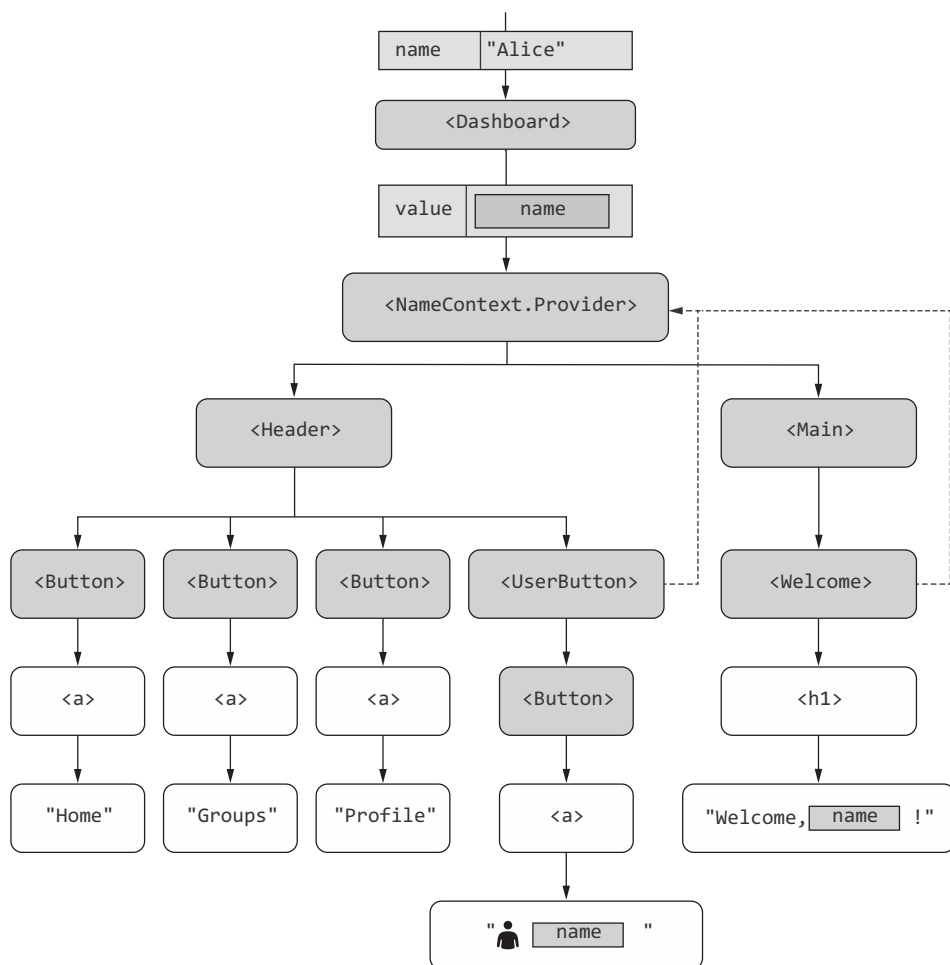


Рис. 10.5. Дерево компонентов приложения с окружающим контекстом. Стрелками обозначены компоненты, использующие контекст и обращающиеся к текущему значению, определяемому провайдером контекста

Листинг 10.2. Информационная панель с контекстом

```

import { createContext, useContext } from "react";
const BUTTON_STYLE = {
  display: "inline-block",
  padding: "4px 10px",
  background: "transparent",
  border: "0",
};
const HEADER_STYLE = {

```

← Импортирует две функции из пакета React

```

    display: "flex",
    justifyContent: "flex-end",
    borderBottom: "1px solid",
  };
  const NameContext = createContext();
  function Button({ children }) {
    return <button style={BUTTON_STYLE}>{children}</button>;
  }
  function UserButton() {
    const name = useContext(NameContext);
    return <Button> {name}</Button>;
  }
  function Header() {
    return (
      <header style={HEADER_STYLE}>
        <Button>Home</Button>
        <Button>Groups</Button>
        <Button>Profile</Button>
        <UserButton />
      </header>
    );
  }
  function Welcome() {
    const name = useContext(NameContext);
    return (
      <section>
        <h1>Welcome, {name}!</h1>
      </section>
    );
  }
  function Main() {
    return (
      <main>
        <Welcome />
      </main>
    );
  }
  function Dashboard({ name }) {
    return (
      <NameContext.Provider value={name}>
        <Header />
        <Main />
      </NameContext.Provider>
    );
  }
  function App() {
    return <Dashboard name="Alice" />;
  }
  export default App;

```

Контекст создается в глобальной области видимости, чтобы к нему можно было обратиться из любой точки

Два компонента, которым нужен доступ к имени, могут обратиться к значению, подключаясь к контексту при помощи `useContext`

В компоненте панели все дерево необходимо упаковать в провайдер контекста, с именем в качестве значения контекста

В главном компоненте приложения вся информационная панель инициализируется значением имени «Alice»

Многие компоненты теперь вообще не получают свойства

Репозиторий: rq10-dashboard-context

Этот пример находится в репозитории `rq10-dashboard-context`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq10-dashboard-context --template rq10-dashboard-context
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq10-dashboard-context>

10.1.2. Контекстные состояния

Применять контекст для хранения статического значения, которое используется во всем приложении, безусловно, удобно, но еще удобнее таким же образом хранить динамическую информацию. Хук `useContext` обладает состоянием, так что если значение контекста изменится, хук `useContext` укажет на необходимость использующему его компоненту автоматически отрендериться повторно.

Представьте ту же панель, но на этот раз вы — администратор, который хочет понимать, как будет выглядеть панель для любого пользователя из базы данных. Администратору доступен раскрывающийся список пользователей, для которых он может просмотреть внешний вид панели.

Дерево компонентов изображено на рис. 10.6. Компонент панели `dashboard` остался таким же, как прежде (мы не приводим здесь все его дочерние компоненты в целях экономии места).

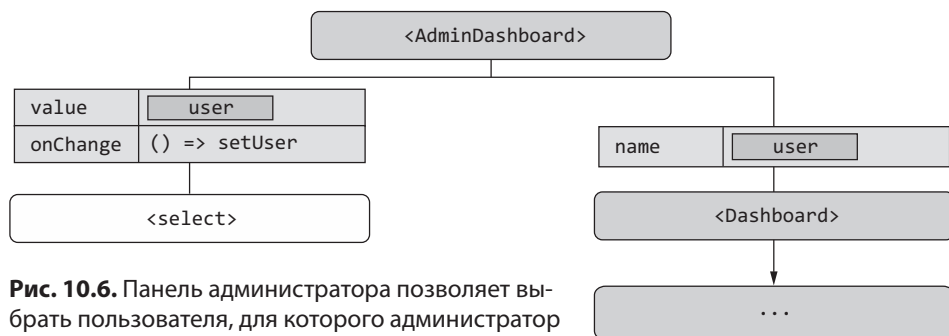


Рис. 10.6. Панель администратора позволяет выбрать пользователя, для которого администратор может увидеть внешний вид панели. Панель администратора включает раскрывающийся список и обычную панель пользователя

Мы воспользуемся простым элементом списка для выбора одного из трех пользователей в системе: Alice, Bob или Carol. Простой хук `useState` может использоваться для запоминания выбранного пользователя и передачи его компонентам при необходимости. Расширим предыдущий пример новой панелью администратора.

Листинг 10.3. Панель администратора

```
import {
  useState, ← Также необходимо импортировать хук useState
  createContext,
  useContext,
} from "react";
const BUTTON_STYLE = {
  display: "inline-block",
  padding: "4px 10px",
  background: "transparent",
  border: "0",
};
const HEADER_STYLE = {
  display: "flex",
  justifyContent: "flex-end",
  borderBottom: "1px solid",
};
const NameContext = createContext();
function Button({ children }) {
  return <button style={BUTTON_STYLE}>{children}</button>;
}
function UserButton() {
  const name = useContext(NameContext);
  return <Button>👤 {name}</Button>;
}
function Header() {
  return (
    <header style={HEADER_STYLE}>
      <Button>Home</Button>
      <Button>Groups</Button>
      <Button>Profile</Button>
      <UserButton />
    </header>
  );
}
function Welcome() {
  const name = useContext(NameContext);
  return (
    <section>
      <h1>Welcome, {name}!</h1>
    </section>
  );
}
```

```

function Main() {
  return (
    <main>
      <Welcome />
    </main>
  );
}
function Dashboard({ name }) {
  return (
    <NameContext.Provider value={name}>
      <Header />
      <Main />
    </NameContext.Provider>
  );
}
function AdminDashboard() {
  const [user, setUser] = useState("Alice");
  return (
    <>
      <select
        value={user}
        onChange={(evt) => setUser(evt.target.value)}
      >
        <option>Alice</option>
        <option>Bob</option>
        <option>Carol</option>
      </select>
      <Dashboard name={user} />
    </>
  );
}
function App() {
  return <AdminDashboard />;
}
export default App;

```

Внутри компонента dashboard
ничего не изменилось

Создает простое состояние
(по умолчанию Alice)

Использует контролируемый
элемент select для выбора
пользователя

Текущий выбранный
пользователь передается
компоненту dashboard

Репозиторий: rq10-dashboard-admin

Этот пример находится в репозитории `rq10-dashboard-admin`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq10-dashboard-admin --template rq10-dashboard-admin
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq10-dashboard-admin>

Приложение в браузере выглядит, как показано на рис. 10.7. Выберите другого пользователя из списка, и вы увидите, что имя обновится — как в меню, так и в заголовке.

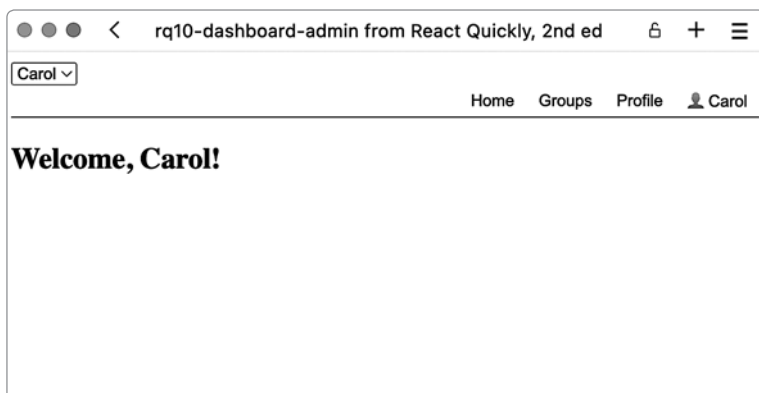


Рис. 10.7. Панель администратора с панелью пользователя Carol (имя, выбранное в раскрывающемся списке в левом верхнем углу)

10.1.3. Подробнее о React Context

А теперь сделаем шаг назад и рассмотрим React Context более подробно. Как уже говорилось, для использования React Context необходимо создать провайдер и потребитель. Потребитель можно создать двумя способами: либо в форме хука, либо с использованием *рендер-пропа*. Но прежде чем выбирать тот или иной вариант, необходимо создать сам контекст функцией `createContext` из пакета React:

```
import { createContext } from 'react';  
const MyContext = createContext(defaultValue);
```

Здесь нужно обратить внимание на две вещи:

- Имя, присваиваемое переменной контекста, обычно начинается с прописной буквы.
- `createContext` получает один аргумент — значение по умолчанию. О том, как оно работает, расскажем чуть позже.

Потребление контекста

Переменная контекста (например, `MyContext` в предыдущем примере) содержит два свойства, представляющих интерес: `MyContext.Provider` и `MyContext.`

`Consumer`. Мы уже объяснили, как потреблять контекст с использованием хука `useContext`. То же самое можно сделать с использованием свойства `MyContext`. `Consumer`, но это сложнее.

Предположим, требуется вывести абзац со значением `name`, предоставленным ближайшим контекстом `NameContext`, в компоненте `DisplayName`. Для этого можно воспользоваться хуком `useContext`:

```
function DisplayName() {
  const name = useContext(NameContext);
  return <p>{name}</p>
}
```

Здесь все просто. Мы вызываем хук и получаем текущее значение в виде переменной, которая может напрямую использоваться в компоненте.

Если попытаться сделать то же самое с компонентом `Consumer`, придется вызвать компонент-потребитель с функцией, являющейся его первым и единственным потомком, и эта функция будет вызвана со значением контекста:

```
function DisplayName() {
  return (
    <p>
      <NameContext.Consumer>
        {(name) => name}
      </NameContext.Consumer>
    </p>
  );
}
```

Передача функции, возвращающей JSX в виде дочернего узла компонента, представляет собой пример *рендер-пропа* (о котором говорилось выше), поскольку это свойство может рендерить JSX при вызове. Вероятно, вы уже заметили, что приходится писать заметно больше кода, а если с возвращаемым значением необходимо выполнить вычисления или применить к нему логику, структуру компонента придется серьезно изменять.

Компонент `Consumer` сегодня используется довольно редко. В основном он встречается в унаследованных проектах на базе классов.

Композиция контекста

Провайдер используется для создания контекста, который может затем потребляться. Потребитель используется для потребления ближайшего предоставленного контекста. Заметим, что один контекст может многократно предоставляться

в приложении и даже во вложенном виде. Также можно использовать один контекст много раз, даже вне провайдера.

При потреблении контекста вы получите значение, предоставленное ближайшим провайдером в направлении вверх по дереву документа JSX. Если над потребителем нет ни одного провайдера, вы получите значение по умолчанию, определенное при создании контекста (см. схему на рис. 10.8).

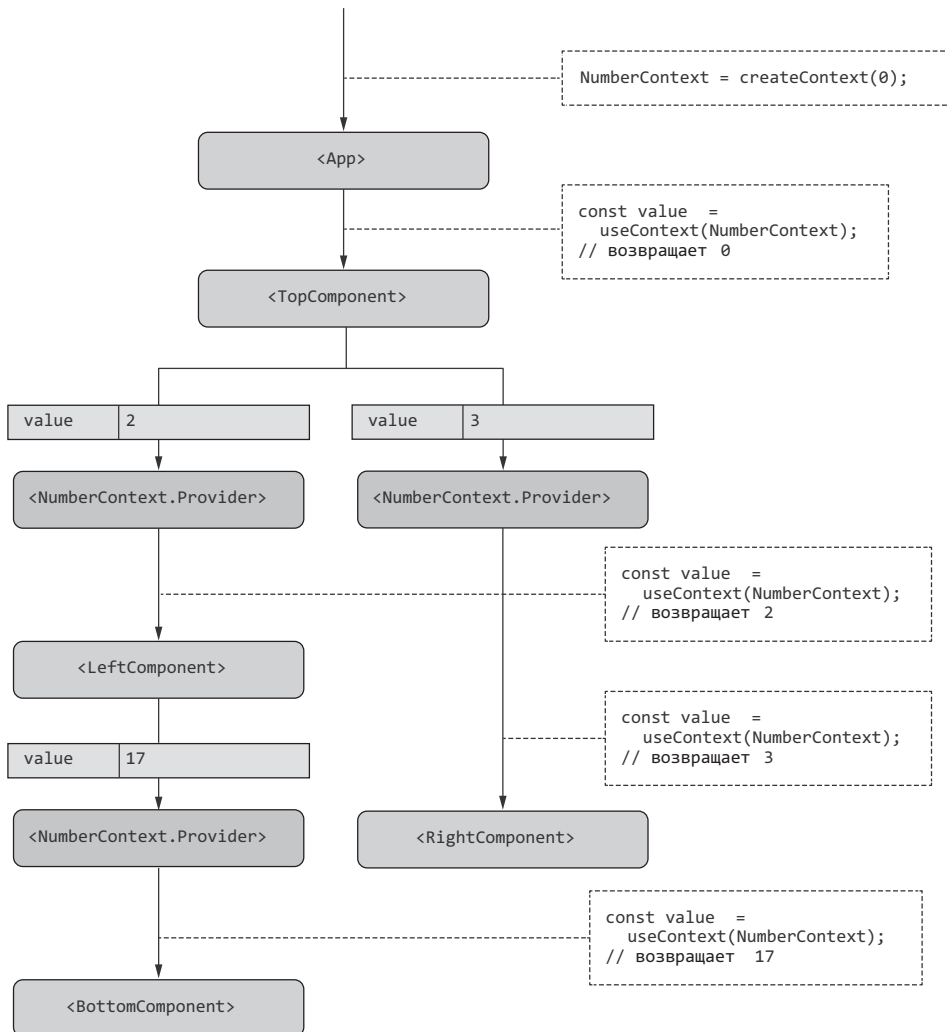


Рис. 10.8. Для одного контекста можно создать много провайдеров и потребителей

Обратите внимание на два момента:

- Если вы потребляете контекст, над которым нет провайдера, как в случае с `TopComponent`, вы получите значение по умолчанию из определения контекста (в данном случае 0).
- Если вы потребляете контекст, над которым расположено несколько провайдеров, как в случае с `BottomComponent`, вы получите значение от ближайшего провайдера вверх по дереву документа (например, 17 вместо 2 в данном случае).

Пример вложенных контекстов

Типичная ситуация применения вложенных контекстов — UI-переменные, например приложение, в котором используются кнопки с разной шириной границ. Представим интернет-магазин с товарами и информацией о компании. В заголовке и футере находятся кнопки, в том числе для открытия корзины.

По умолчанию все кнопки имеют границу шириной 1 пиксель, но в футере ширина границы всех кнопок составляет 2 пикселя. Кроме того, каждой кнопке, ведущей в корзину, всегда назначается ширина 5 пикселей, поскольку они имеют большую важность. Для начала набросаем эскиз структуры системы, изображенный на рис. 10.9.

Теперь каждый компонент кнопки должен проходить вверх по дереву компонентов, чтобы найти ближайший провайдер контекста границы и получить от него ширину границы. Если провайдер в дереве не найден, кнопка использует значение по умолчанию, определенное при создании контекста. На рис. 10.10 дерево компонентов размечено операциями поиска ближайшего провайдера.

Теперь у нас есть все вводные и мы можем написать реализацию, приведенную в листинге 10.4. На рис. 10.11 показано, как выглядит приложение в браузере.

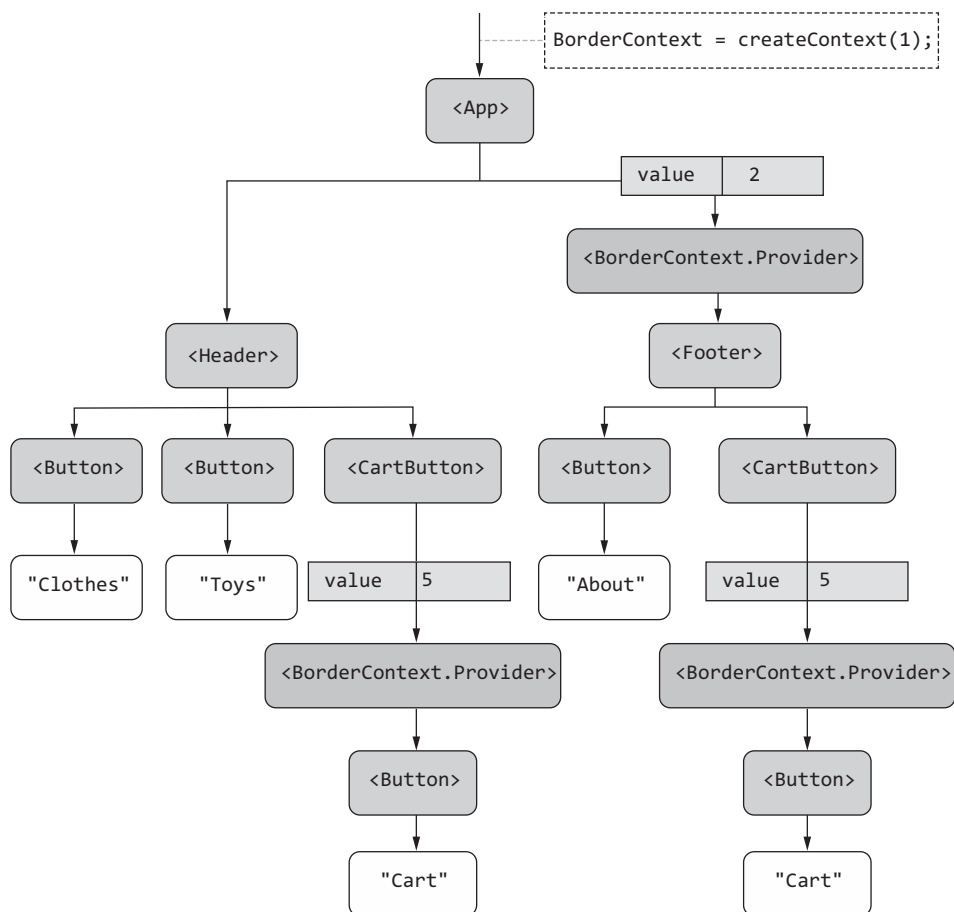


Рис. 10.9. Дерево компонентов для интернет-магазина. Обратите внимание: мы определяем как значение по умолчанию для контекста, так и несколько провайдеров контекста

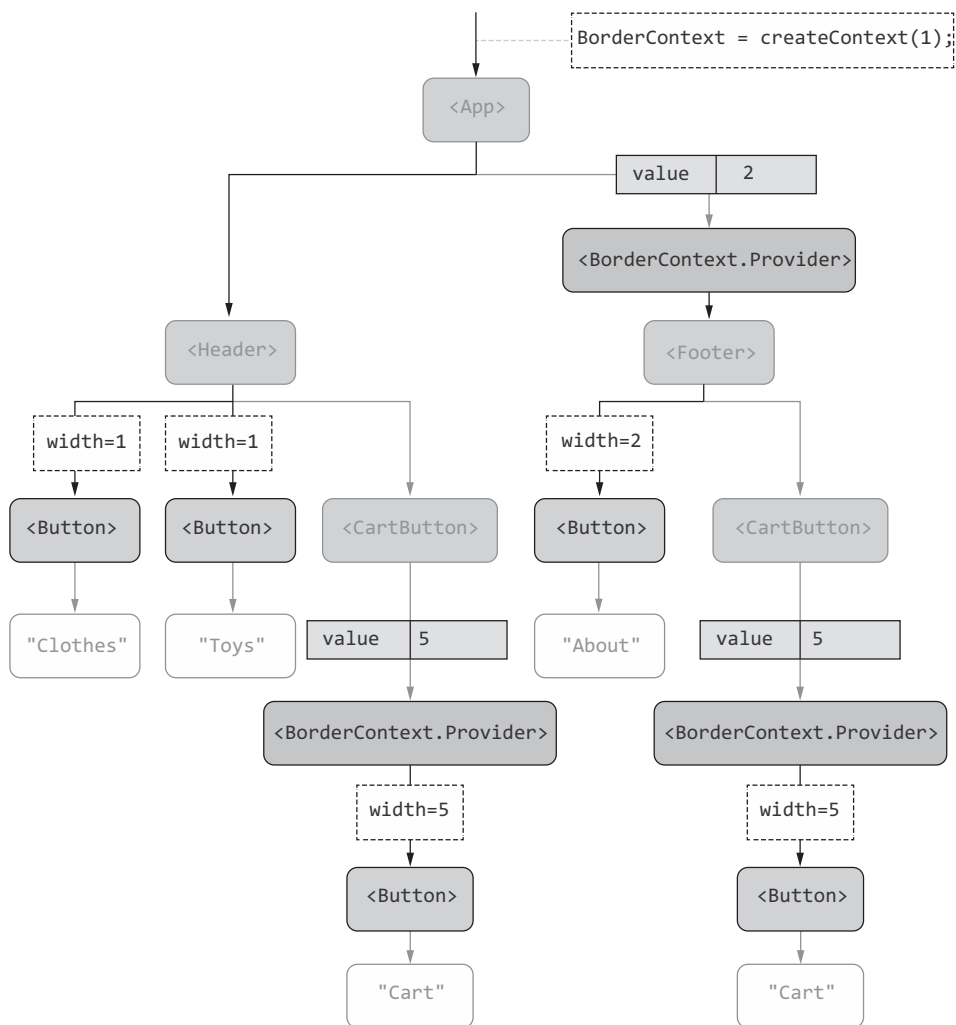


Рис. 10.10. Дерево компонентов, в котором каждый компонент кнопки соединен жирной стрелкой с ближайшим провайдером (или корнем), от которого он получает ширину границы для этого компонента

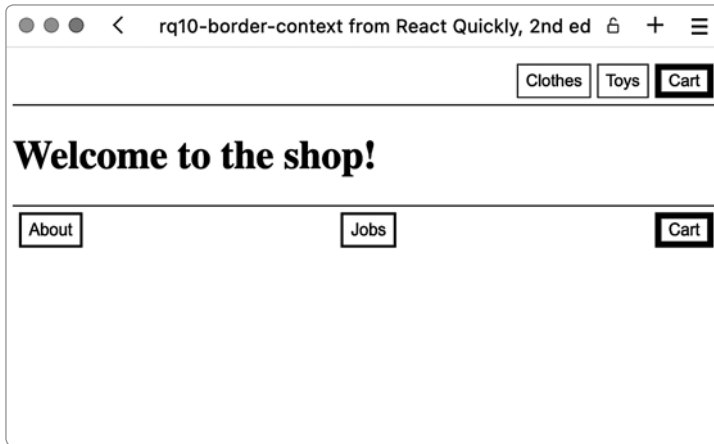


Рис. 10.11. На сайте магазина все кнопки имеют верную ширину границ. Выглядит так себе, но это желание заказчика!

Листинг 10.4. Получение ширины границы из контекста

```
import { useContext, createContext } from "react";
const BorderContext = createContext(1);
function Button({ children }) {
  const borderWidth = useContext(BorderContext);
  const style = {
    border: `${borderWidth}px solid black`,
    background: "transparent",
  };
  return <button style={style}>{children}</button>;
}
function CartButton() {
  return (
    <BorderContext.Provider value={5}>
      <Button>Cart</Button>
    </BorderContext.Provider>
  );
}
function Header() {
  const style = {
    padding: "5px",
    borderBottom: "1px solid black",
    marginBottom: "10px",
    display: "flex",
    gap: "5px",
    justifyContent: "flex-end",
  };
  return (
```

Создает исходный контекст со значением по умолчанию 1

Компонент кнопки потребляет значение, предоставленное ближайшим провайдером, и использует его как свойство ширины границы в CSS

Добавляет провайдер ширины границы внутри кнопки корзины, чтобы этой кнопке предоставлялась ширина границы 5 px

```

    <header style={style}>
      <Button>Clothes</Button>
      <Button>Toys</Button>
      <CartButton />
    </header>
  );
}
function Footer() {
  const style = {
    padding: "5px",
    borderTop: "1px solid black",
    marginTop: "10px",
    display: "flex",
    justifyContent: "space-between",
  };
  return (
    <footer style={style}>
      <Button>About</Button>
      <Button>Jobs</Button>
      <CartButton />
    </footer>
  );
}
function App() {
  return (
    <main>
      <Header />
      <h1>Welcome to the shop!</h1>
      <BorderContext.Provider value={2}> ←
        <Footer />
      </BorderContext.Provider>
    </main>
  );
}
export default App;

```

Футер заключается в провайдер, чтобы всем внутренним кнопкам по умолчанию назначалась рамка 2 px, если только особый провайдер не передаст им другое значение

Репозиторий: rq10-border-context

Этот пример находится в репозитории rq10-border-context. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq10-border-context --template rq10-border-context
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq10-border-context>

10.2. РАБОТА СО СЛОЖНЫМ СОСТОЯНИЕМ

Вернемся к нашему самому любимому примеру — счетчику с кнопками инкремента и декремента! На этот раз мы используем другой подход. Вместо того чтобы добавлять обычный хук `useState` для хранения и управления значением состояния, воспользуемся преобразованием и хуком `useReducer`.

Для начала кратко опишем, как работает хук `useReducer`, — просто чтобы перейти к работе над примером, а более подробно поговорим об этом в следующих подразделах.

API `useReducer` выглядит так:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Эти четыре элемента связаны друг с другом, как показано на рис. 10.12.

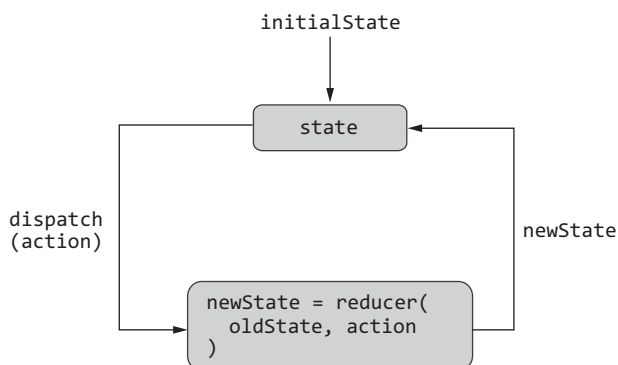


Рис. 10.12. Логика передачи данных в хуке `useReducer` отчасти напоминает обычный хук `useState`: она также начинается с исходного значения, которое обновляется в процессе работы приложения. Но способ обновления внутреннего состояния более сложен, так как новое состояние получается в результате «преобразования» старого с использованием функций и действий

Схема состоит из четырех частей. `state` и `initialState` работают точно так же, как для `useState()`. Таким образом, для счетчика исходным состоянием будет 0, а состоянием — значение счетчика на текущий момент.

Два новых параметра — функции `dispatch` и `reducer`. Функция `dispatch` работает как улучшенный сеттер, который позволяет не задавать значение напрямую, а указывает функции `reducer`, как его следует задать. Таким образом, `reducer` — функция, которая получает текущее состояние и указанное действие

и возвращает новое состояние на их основе. `dispatch` вызывается с объектом *действия*, который затем передается `reducer` с прежним состоянием; предполагается, что `reducer` вернет новое состояние. А пока реализуем эту схему для счетчика.

Листинг 10.5. Компонент-счетчик с функцией `reducer`

```
import { useReducer } from "react";
function reducer(state, { type }) {
  switch (type) {
    case "INCREMENT":
      return state + 1;
    case "DECREMENT":
      return state - 1;
    default:
      return state;
  }
}
function Counter() {
  const [counter, dispatch] = useReducer(reducer, 0);
  return (
    <section>
      <h1>Counter: {counter}</h1>
      <div>
        <button onClick={
          () => dispatch({ type: "INCREMENT" })
        }>
          Increment
        </button>
        <button onClick={
          () => dispatch({ type: "DECREMENT" })
        }>
          Decrement
        </button>
      </div>
    </section>
  );
}
function App() {
  return <Counter />;
}
export default App;
```

Создает функцию-редьюсер, которая получает
прежнее состояние (текущее значение) и объект
действия, который обладает типом

Возвращает прежнее
значение плюс или минус 1
в зависимости от типа

Инициализирует хук
функцией-редьюсером
и исходным значением 0

Вызывает функцию `dispatch`
с соответствующими
объектами действий

Репозиторий: `rq10-counter-reducer`

Этот пример находится в репозитории `rq10-counter-reducer`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq10-counter-reducer --template rq10-counter-reducer
```


На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq10-counter-reducer>

Вуаля! Мы еще раз реализовали счетчик. На этот раз гораздо более сложным и продуманным способом, но эту концепцию можно использовать для более сложных сценариев состояний, как вы увидите в следующем подразделе.

10.2.1. Взаимозависимое состояние

Другая проблема, которая может возникнуть с ростом сложности приложения, — взаимозависимое состояние. Иначе говоря, состояние содержит разные значения, которые связаны друг с другом, но не являются копиями друг друга. Для примера возьмем простой компонент, который загружает внешний контент и выводит его после завершения загрузки. Так как контент загружается из внешнего источника, загрузка может завершиться неудачей. В таком случае выводится сообщение об ошибке.

Такой компонент проще всего построить с тремя разными значениями состояния. Одно значение представляет ход загрузки (загрузка продолжается, загрузка завершилась успешно, загрузка завершилась неудачей), другое содержит объект результата, если загрузка была успешной, а третье — сообщение об ошибке в случае неудачной загрузки. Эта простая схема изображена на рис. 10.13.

Состояние содержит три взаимозависимых значения; это означает, что смысл имеют только отдельные комбинации значений. Например, в состоянии загрузки (LOADING) для `error` или `result` не имеет смысла ничего, кроме `null`, потому что еще ничего не загружено. Точно так же в случае неудачной загрузки (FAILURE) состояние `result` не может содержать значения — просто по определению.

Однако разработчику приходится постоянно держать эти связи в голове. Не получится напрямую запрограммировать эту связь с простыми значениями состояния при использовании `useState()`. Разработчик должен не забыть сбросить состояния `error` и `result` при перезагрузке внешнего ресурса, потому что в противном случае компонент окажется в некорректном состоянии и не будет знать, что нужно вывести.

В такой ситуации лучше создать одну функцию, которая позволяет перемещаться между разными семантическими состояниями системы, а не между отдельными переменными. Например, представьте, что произойдет при неудачной загрузке, если существуют три разные переменные и нужно задать значения двух из них:

```
fetch(...).catch(() => {
  setStatus(FAILURE);
  setError("Loading failed");
});
```

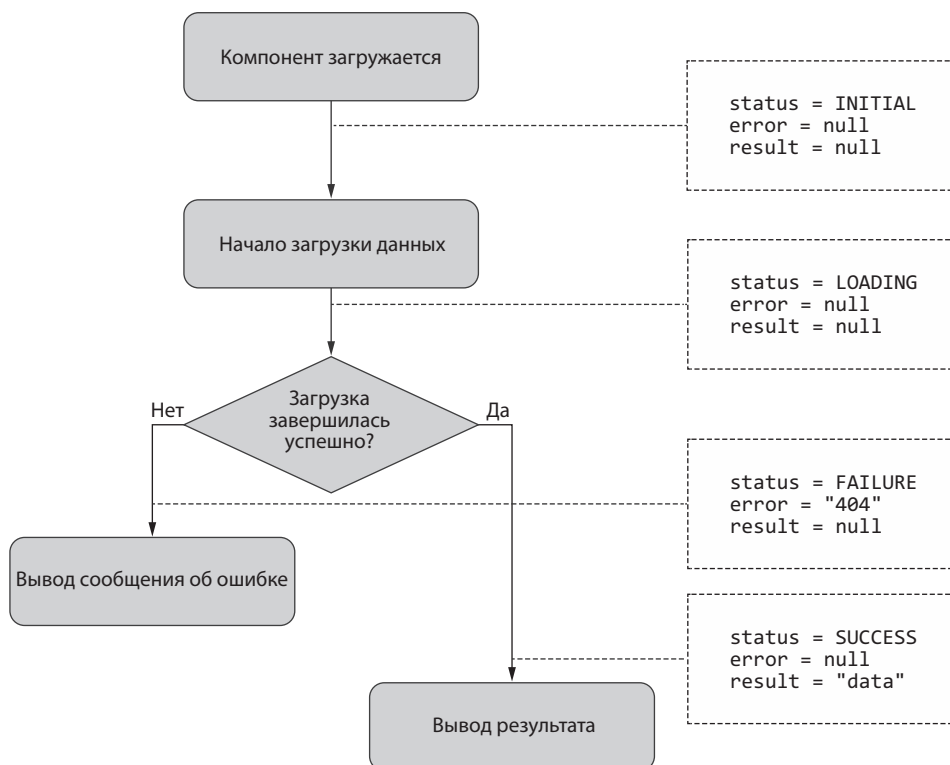


Рис. 10.13. Логика работы простого компонента загрузки. Пунктирные прямоугольники представляют желаемое состояние в заданной точке выполнения программы

Если использовать рекомендованное решение с вызовом одной функции, изменяющей семантическое состояние, получится вот что:

```
fetch(...).catch(() => {
  failureHappenedAndThisIsTheErrorMessage("Loading failed");
});
```

Заметная разница. Первый синтаксис довольно ошибкоопасный, а второй предоставляет намного более чистый API с минимальным риском ошибиться в интерпретации того, что следует делать.

useReducer приходит на помощь

Именно для таких ситуаций существует хук `useReducer`. Но вместо того чтобы использовать одно примитивное значение в качестве состояния, можно хранить объект с несколькими значениями состояния. Затем переданные объекты действий используются для манипуляций со всем объектом значений состояния по мере надобности.

Итак, вернемся к диаграмме с логикой работы и посмотрим, какие объекты действий понадобятся для продвижения состояния и какие аргументы с полезными данными потребуются для обновления значений состояния (рис. 10.14). Остается лишь определить функцию-редьюсер, которая получает существующее состояние и объект действия, а затем генерирует новое состояние на их основе.

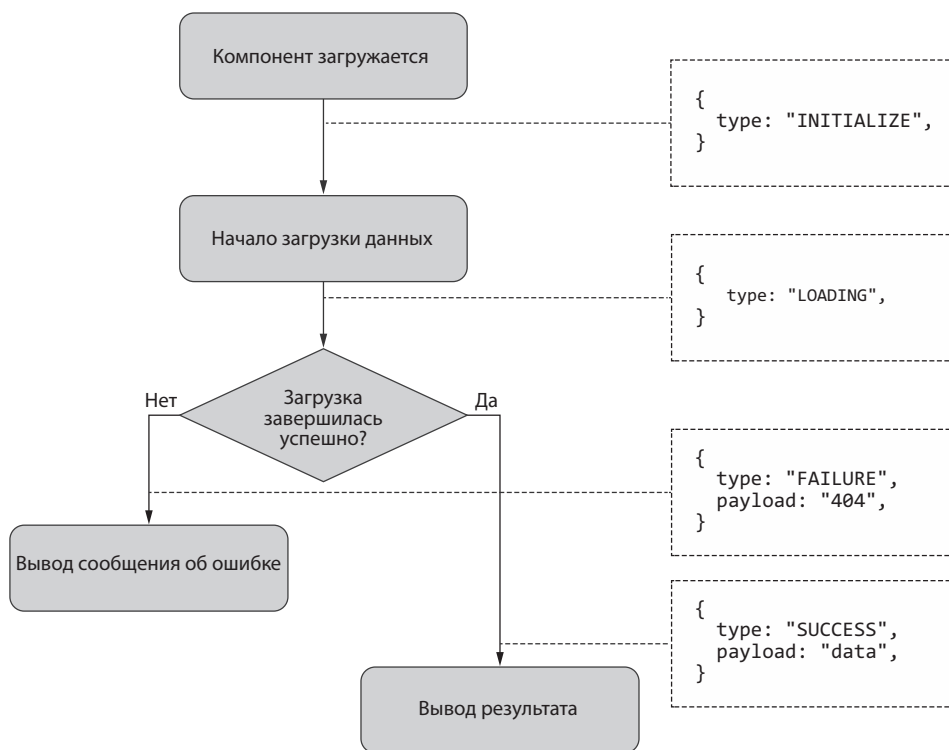


Рис. 10.14. Логика работы компонента загрузки с API действий. Пунктирные прямоугольники обозначают объект действия, который будет передаваться редьюсеру, чтобы тот произвел нужное обновление внутреннего состояния

Общая структура редьюсера обычно выглядит так:

```
function reducer(state, { type, payload }) {
  switch (type) {
    case "TYPE_A":
      // Вернуть новое состояние на базе TYPE_A
    case "TYPE_B":
      // Вернуть новое состояние на базе TYPE_B
  }
}
```

Таким образом, для нашего конкретного сценария использования обычно определяются типы действий "INITIALIZE", "LOADING", "ERROR" и "SUCCESS". Ниже перечислены состояния, к которым приводят все эти действия:

- При инициализации `status` присваивается значение "INITIALIZE", а всем остальным переменным присваивается `null` независимо от их текущего значения.
- В процессе загрузки `status` присваивается значение "LOADING", а все остальное остается без изменений.
- Если загрузка завершается неудачей, `status` присваивается значение "FAILURE", а `error` присваиваются переданные данные.
- Если загрузка завершается успешно, `status` присваивается значение "SUCCESS", а `result` присваиваются переданные данные.

Реализуем сказанное в представленной выше структуре `reducer`:

```
function reducer(state, { type, payload }) {
  switch (type) {
    case "INITIALIZE":
      return {
        status: "INITIALIZE",
        result: null,
        error: null,
      };
    case "LOADING":
      return {
        ...state,
        status: "LOADING",
      };
    case "FAILURE":
      return {
        ...state,
        status: "FAILURE",
        error: payload,
      };
    case "SUCCESS":
```

При инициализации стираются
все значения независимо
от предшествующего состояния

В процессе загрузки
изменяется только значение
status и ничего больше

При возникновении ошибки изменяется
status и задается значение error

```

return {
  ...state,
  status: "SUCCESS",
  result: payload,
};
}
}

```

При успешной загрузке изменяется status и задается значение result

Вернемся к исходной задаче. Нужен компонент, который может загружать данные и выводить информацию о статусе в процессе загрузки. Для работы с состоянием будет использоваться `reducer`, определенный ранее, но с некоторыми изменениями.

Листинг 10.6. Компонент загрузки с редьюсером

Далее ожидаются только действия типов `LOADING`, `FAILURE` и `SUCCESS`

```

import { useReducer, useEffect } from "react";
const URL = "//swapi.dev/api/films";
const INITIAL_STATE = {
  status: "INITIALIZE",
  result: null,
  error: null,
};
function reducer(state, { type, payload }) {
  switch (type) {
    case "LOADING":
      return { ...state, status: "LOADING" };
    case "FAILURE":
      return { ...state, status: "FAILURE", error: payload };
    case "SUCCESS":
      return { ...state, status: "SUCCESS", result: payload };
    default:
      return state;
  }
}
function Loader() {
  const [state, dispatch] =
    useReducer(reducer, INITIAL_STATE);
  useEffect(() => {
    dispatch({ type: "LOADING" });
    fetch(URL)
      .then((res) => res.json())
      .then(
        ({ results }) =>
          dispatch({
            type: "SUCCESS",
            payload: results,
          })
      )
  })
}

```

Исходное состояние извлекается в переменную, а не в один из вариантов внутри редьюсера

Секция `default` добавляется на случай, если будут отправлены неизвестные бессмысленные данные

Если загрузка завершается успешно, изменяем status и задаем result

В хуке эффекта начинаем с присваивания status значения `LOADING` посредством передачи соответствующего действия

Если будут возвращены результаты, они задаются в состоянии передачей действия `SUCCESS`

```

    .catch(
      ({ message }) =>
        dispatch({
          type: "FAILURE",
          payload: message,
        })
    );
  }, []);
  const { status, error, result } = state;
  if (status === "INITIALIZE") {
    return <h1>Initializing...</h1>;
  }
  if (status === "LOADING") {
    return <h1>Loading...</h1>;
  }
  if (status === "FAILURE") {
    return <h1>Error occurred: {error}</h1>;
  }
  return (
    <>
      <h1>Results are in</h1>
      <ul>
        {result.map(({ title }) => (
          <li key={title}>{title}</li>
        ))}
      </ul>
    </>
  );
}
function App() {
  return <Loader />;
}
export default App;

```

Если где-то произошла ошибка, передается действие **ERROR** с сообщением

Когда все сделано, состояние можно деструктурировать в три переменные, которые в нем содержатся

Наконец, в зависимости от значения переменной **status** выводится сообщение, в котором используются значения **error** или **result**

Репозиторий: **rq10-reducer-load**

Этот пример находится в репозитории **rq10-reducer-load**. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq10-reducer-load --template rq10-reducer-load
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq10-reducer-load>

Приложение в браузере работает! Результат показан на рис. 10.15.

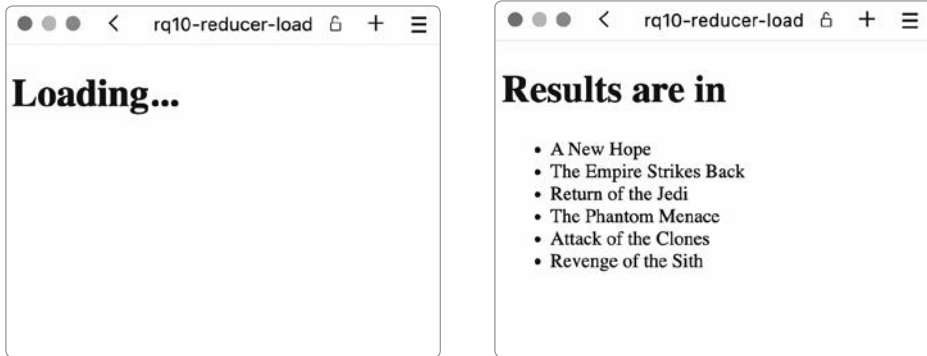


Рис. 10.15. Компонент загрузки в действии: первый снимок экрана показывает процесс загрузки, на втором загрузка успешно завершена

Если заменить URL-адрес несуществующим, например таким:

```
const URL = '//swapi.dev.invalid/api/films';
```

загрузка завершится неудачей, и вы увидите сообщение об ошибке, как показано на рис. 10.16.

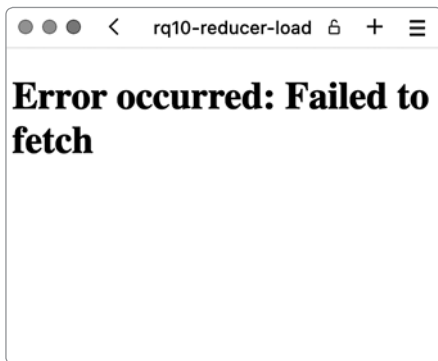


Рис. 10.16. Если загрузка завершается неудачей, выводится сообщение об ошибке

У созданного нами редьюсера есть одна приятная особенность — он полностью обобщен. Его не интересует, что мы загружаем или как мы это делаем. Он только управляет состоянием вокруг всего, что может загружаться или при загрузке чего может произойти ошибка, — данных, графики, видео, шрифтов, детенышей выдры и остального.

Что такое редьюсер?

Концепция *редьюсера* (reducer) проистекает из модели обработки данных *MapReduce* — подхода, связанного с обновлением модели данных в процессе обработки на основании записей из потока данных.

Редьюсеры обычно рассматриваются как чистые, простые и не имеющие побочных эффектов функции, детерминированные и определяемые исключительно своими параметрами. Редьюсер обычно получает текущее состояние окружения и некоторое новое действие/запись и обновляет состояние окружения в зависимости от полученной информации.

В крупномасштабных вычислениях и при анализе данных редьюсеры используются для быстрого и эффективного обхода сложных структур данных.

В React редьюсером называется функция или набор функций, преобразующих текущее состояние в новое на основании текущего действия. В React также ожидается, что редьюсер чист, детерминирован и не имеет побочных эффектов.

10.3. НЕСТАНДАРТНЫЕ ХУКИ

Вернемся к примеру с панелью, приведенному выше. Этот фрагмент кода уже не раз нам встречался:

```
const name = useContext(NameContext);
```

И хотя он очень прост, он требует правильного объединения `useContext` и `NameContext`. Но можно немного упростить задачу, переместить функциональность в новую функцию и заменить код следующим:

```
const name = useName();
```

Так можно? Да, конечно! Мы просто создаем нестандартную функцию, которая выполняет свою работу скрыто. Таким образом, эта функция использует хук `useContext` для извлечения текущего значения из контекста `NameContext`:

```
function useName() {
  return useContext(NameContext);
}
```

Вот и все — простое обобщение функциональности за счет вынесения дублируемой части кода в общую функцию.

Но есть один нюанс: мы обобщаем функциональность, основанную на хуке *React*, а использование хуков *React* должно подчиняться определенным правилам.

Во-первых, хуки React не могут использоваться за пределами функциональных компонентов. Во-вторых, хуки React могут использоваться только в одном порядке и только если это одни и те же хуки.

Новая функция, которую мы создаем, — `useName` — тоже должна подчиняться этим правилам. Это нестандартный хук, и вы только что создали свой первый нестандартный хук! Учтите, что этот хук очень прост и не получает никаких аргументов, но вообще нестандартные хуки могут получать аргумент. Примеры нестандартных хуков будут приведены в следующем подразделе. Теперь разберемся, что же именно превращает функцию в нестандартный хук, как решить, когда и где использовать нестандартные хуки и где найти больше таких хуков.

10.3.1. Как образуется нестандартный хук?

Нестандартный хук — функция, использующая хук. Хуком может быть любой встроенный или нестандартный хук, так что определение выглядит рекурсивным, но это не так. Все начинается с 10 встроенных хуков, описанных выше. Если вы создали функцию, использующую какие-либо из этих 10 хуков, значит, вы создали нестандартный хук. Если вы создали функцию, использующую какие-либо из этих 10 хуков или нестандартный хук, в котором использовался один из этих 10 хуков, вы тоже создали нестандартный хук, и т. д. Таким образом, чтобы функция стала нестандартным хуком, она должна использовать один или несколько встроенных хуков ниже в цепочке. Несколько примеров см. на рис. 10.17.

Функции, которые являются нестандартными хуками	Функции, которые не являются нестандартными хуками
☒ Используется один или несколько встроенных хуков <pre>function useToggle(default = false) { const [value, setter] = useState(default); const toggle = () => setter(v => !v); return [value, toggle]; }</pre>	☒ Хуки вообще не используются <pre>function useSomeLogic() { const value = {}; const setter = (v) => value.v = v; return [value, setter]; }</pre>
☒ Используется один или несколько нестандартных хуков <pre>function useDarkMode() { const [isDarkMode, toggleDarkMode] = useToggle(false); return { isDarkMode, toggleDarkMode }; }</pre>	☒ Две функции, но ни одна не использует встроенные хуки <pre>function useValue(v) { return useProperty(v); } function useProperty(p) { return useValue(p); }</pre>
☒ Используются как встроенные, так и нестандартные хуки <pre>function useRefToggle() { const [value, toggle] = useToggle(false); const ref = useRef(value); return [ref, toggle]; }</pre>	

Рис. 10.17. Примеры функций, которые являются или не являются нестандартными хуками

Обратите внимание на имена функций. Всем нестандартным хукам принято давать имена, начинающиеся с `use*`, и использовать эти имена только для этих функций. Тем не менее ничего волшебного в этих именах нет. Задача разработчика — следить, чтобы нестандартный хук назывался `use*`. Также стоит лишний раз проверить, что все функции с именами `use*` действительно являются нестандартными хуками.

10.3.2. Когда использовать нестандартный хук?

Нестандартный хук можно использовать, когда вы посчитаете нужным. Сделать код крайне оптимизированным и компактным почти никогда не получится (иногда он может быть таким, но для этого нужно очень сильно постараться).

Часто нестандартные хуки реализуют в одной из следующих двух ситуаций:

- Нестандартный хук создается для функциональности, которая понадобится в нескольких местах.
- Функциональность перемещается в нестандартный хук, чтобы очистить компонент и сделать его более удобочитаемым.

Обе цели абсолютно приемлемы, и такое использование часто практикуется разработчиками. Различия между этими случаями проявляются в именах хуков. Если имя хука выглядит обобщенным, вероятно, он относится к повторному использованию функциональности. Если его имя предельно конкретно, скорее всего, это просто результат вынесения сложной логики во внешний файл для упрощения представления. Начнем с нескольких примеров приложений, которые мы создавали ранее в этой книге.

useToggle

Рассмотрим один из примеров главы 6, где приводился следующий фрагмент кода компонента с интерактивным обратным отсчетом:

```
function Countdown({ from }) {
  ...
  const [isRunning, setRunning] = useState(false);
  ...
  onClick={() => setRunning((v) => !v)}
  ...
}
```

Функциональность создания флага с логическим значением в состоянии и определения функции для переключения флага в противоположное состояние — довольно общая процедура.

Обобщим ее в хук, который напоминает обычный хук `useState`, не считая того, что вместо сеттера возвращается функция переключения `toggle`. В качестве значений состояния могут использоваться только логические значения:

```
function useToggle(default = false) {
  const [value, setter] = useState(Boolean(default));
  const toggle = () => setter(v => !v);
  return [value, toggle];
}
```

Обратите внимание, что этот хук возвращает массив со значением и функцией, как обычный хук `useState`. Этот паттерн очень часто применяется при работе с нестандартными хуками, так как разработчик сразу понимает, как работать с хуком. Значение, возвращаемое хуком `useToggle`, может использоваться так же, как значение, возвращаемое хуком `useState`, но функция отличается. Если функция `setter`, возвращаемая хуком `useState`, может использоваться для присваивания `value` произвольного значения, функция `toggle`, возвращаемая хуком `useToggle`, может использоваться только для инвертирования текущего логического значения — функция `toggle` не получает аргумента.

И это все. Мы получили удобную обобщенную функциональность переключения флага. Ее можно применить к интерактивному счетчику следующим образом:

```
function Countdown({ from }) {
  ...
  const [isRunning, toggleRunning] = useToggle();
  ...
  onClick={toggleRunning}
  ...
}
```

Объем кода особо не уменьшился, но без лишней функции он выглядит гораздо проще. Также хук можно использовать в другом месте, если функциональность переключения флага с состоянием понадобится для других целей.

useForm

В главе 9 в компоненте формы использовалась следующая функциональность:

```
function Address() {
  const [data, setData] = useState({
    address1: "",
    address2: "",
    zip: "",
    city: "",
    state: "",
  });
}
```

```

    country: "",
  });
  const onChange = (evt) => {
    const key = evt.target.name;
    const value = evt.target.value;
    setData(oldData => ({ ...oldData, [key]: value }));
  };
  ...
}

```

Эту функциональность можно также обобщить в нестандартный хук, который может использоваться не только для этой формы с шестью полями ввода, но и для любой формы с любым количеством полей ввода:

```

function useForm(initialValues) {
  const [data, setData] = useState(initialValues);
  const onChange = (evt) => {
    const key = evt.target.name;
    const value = evt.target.value;
    setData(oldData => ({ ...oldData, [key]: value }));
  };
  return [data, onChange];
}

```

Вот как он используется в форме из примера:

```

function Address() {
  const [data, onChange] = useForm({
    address1: "",
    address2: "",
    zip: "",
    city: "",
    state: "",
    country: "",
  });
  ...
}

```

Такое решение выглядит элегантно и пригодно для повторного использования.

useLoader

Помните редьюсер, который мы создали в этой главе? Это был обобщенный хук для загрузки произвольного типа контента, который позволял компоненту указать, что загрузка выполняется в настоящий момент или завершилась успехом или неудачей. Однако мы не сделали редьюсер обобщенным, а просто упаковали его в компонент. Предыдущая версия компонента выглядела так:

```

import { useEffect, useReducer } from "react";
function reducer(state, { type, payload }) {
  switch (type) {
    case "LOADING":
      return { ...state, status: "LOADING" };
    case "FAILURE":
      return { ...state, status: "FAILURE", error: payload };
    case "SUCCESS":
      return { ...state, status: "SUCCESS", result: payload };
    default:
      return state;
  }
}
const INITIAL_STATE = { status: "INITIALIZE", result: null, error: null }
function Loader() {
  const [state, dispatch] = useReducer(reducer, INITIAL_STATE);
  useEffect(() => {
    dispatch({ type: "LOADING" });
    fetch(URL)
      .then((res) => res.json())
      .then(
        ({ results }) => dispatch({ type: "SUCCESS", payload: results })
      )
      .catch(
        ({ message }) => dispatch({ type: "FAILURE", payload: message })
      );
  }, []);
  ...
}

```

И снова можно извлечь обобщенные части логики во внешний хук и использовать их в компоненте. Хук приходит к следующему виду:

```

import { useReducer } from "react";
function reducer(state, { type, payload }) {
  switch (type) {
    case "LOADING":
      return { ...state, status: "LOADING" };
    case "FAILURE":
      return { ...state, status: "FAILURE", error: payload };
    case "SUCCESS":
      return { ...state, status: "SUCCESS", result: payload };
    default:
      return state;
  }
};
function useLoader(initialState) {
  return useReducer(reducer, initialState);
}
export default useLoader;

```

Если сохранить предыдущий фрагмент в файле `useLoader.js`, расположенном в одном каталоге с исходным компонентом, компонент упрощается до следующего вида:

```
import { useEffect } from "react";
import useLoader from "./useLoader";
const INITIAL_STATE = { status: "INITIALIZE", result: null, error: null };
function Loader() {
  const [state, dispatch] = useLoader(INITIAL_STATE);
  useEffect(() => {
    dispatch({ type: "LOADING" });
    fetch(URL)
      .then((res) => res.json())
      .then(
        ({ results }) => dispatch({ type: "SUCCESS", payload: results })
      )
      .catch(
        ({ message }) => dispatch({ type: "FAILURE", payload: message })
      );
  }, [actions]);
  ...
}
```

При таком разделении логики на два отдельных блока каждую из частей читать намного проще.

useCounter

В этом примере бизнес-логика, находящаяся внутри компонента, выделяется во внешний блок, просто чтобы разработчику было проще получить представление об исходном компоненте, не создавая обобщенную функциональность. Возьмем наш компонент-счетчик с кнопками инкремента и декремента. Эта конкретная функциональность не используется в других местах; нам нужно лишь разгрузить компонент.

Ранее компонент выглядел так:

```
function StyledCounter() {
  const [counter, setCounter] = useState(0);
  const update = (d) => setCounter((v) => v + d);
  const handleIncrement = () => update(1);
  const handleDecrement = () => update(-1);
  return (
    <section>
      <h1>Counter: {counter}</h1>
      <div>
        <Button handleClick={handleIncrement} label="Increment" />
        <Button handleClick={handleDecrement} label="Decrement" />
      </div>
    </section>
  );
}
```

```

    </div>
  </section>
);
}

```

Если выделить следующую часть в нестандартный хук вида:

```

import { useState } from "react";
function useCounter() {
  const [counter, setCounter] = useState(0);
  const update = (d) => setCounter((v) => v + d);
  const handleIncrement = () => update(1);
  const handleDecrement = () => update(-1);
  return {counter, handleIncrement, handleDecrement};
}
export default useCounter;

```

и сохранить предыдущий фрагмент в файле `useCounter.js` в одном каталоге с исходным компонентом, компонент принимает следующий вид:

```

import useCounter from "../useCounter";
function StyledCounter() {
  const {counter, handleIncrement, handleDecrement} = useCounter();
  return (
    <section>
      <h1>Counter: {counter}</h1>
      <div>
        <Button handleClick={handleIncrement} label="Increment" />
        <Button handleClick={handleDecrement} label="Decrement" />
      </div>
    </section>
  );
}

```

Перед нами чистый компонент! Всего один хук создает все значения состояния и обратные вызовы, необходимые для выполнения своих обязанностей, а остальной код компонента содержит JSX.

10.3.3. Где найти нестандартные хуки?

Везде! Нестандартные хуки — один из лучших способов выражения сложных логических правил, и вы увидите, что многие библиотеки и утилиты распространяются в форме нестандартных хуков. Хуки значительно превосходят компоненты по гибкости, поскольку не требуют никаких неявных представлений о семантике, пользовательском интерфейсе или элементах HTML. Нестандартные хуки — чистая функциональность, которая может применяться, как требуется

для приложения. Ниже приведен краткий список отличных нестандартных хуков, которыми вы можете пользоваться:

- *useHooks* (<https://usehooks.com>) — подборка хуков общего назначения для повседневной работы.
- *Collection of React Hooks* (<https://nikgraf.github.io/react-hooks>) — огромная библиотека, содержащая более 400 хуков, разработанных пользователями для самых разных целей.
- *React Aria* (<https://react-spectrum.adobe.com/react-aria>) — распространяемая с открытым исходным кодом библиотека хуков, ориентированных на доступность. Эти хуки предоставляют привязки для клавиатуры и указателя мыши для многих более или менее сложных виджетов, разработанных и поддерживаемых Adobe.
- *awesome-react-hooks* (<https://github.com/rehooks/awesome-react-hooks>) — список хуков React с сортировкой по категориям и кратким описанием.

10.4. ВОПРОСЫ

1. Из следующих вариантов выберите верный способ создания провайдера контекста для контекста с именем `StyleContext` со значением `style`:
 - a) `<StyleProvider value={style}>`
`...`
`</StyleProvider>`
 - b) `<StyleContext.Provider value={style}>`
`...`
`</StyleContext.Provider>`
 - c) `<StyleProvider style={style}>`
`...`
`</StyleProvider>`
 - d) `<StyleContext.Provider style={style}>`
`...`
`</StyleContext.Provider>`
2. Если хук `useContext` используется с контекстом, не имеющим провайдера над соответствующим компонентом в дереве документа JSX, выдается ошибка. Да или нет?

3. Хук `useState` превосходит хук `useReducer` во всех отношениях, и все, что можно сделать с помощью последнего, эффективнее реализовать с `useState`. *Да или нет?*
4. Определив нестандартный хук, необходимо зарегистрировать его как *официальный* нестандартный хук при помощи специальной функции React. *Да или нет?*
5. Нестандартные хуки отлично подходят для обобщения функциональности в приложениях, чтобы упростить чтение и использование компонентов. *Да или нет?*

ОТВЕТЫ

6. *Да.* Нестандартные хуки — один из основных способов совместного использования между разными приложениями. Многие библиотеки React предоставляют свою функциональность в форме нестандартных хуков.
5. *Нет.* Любая определенная функция, в которой используется другой хук, автоматически становится нестандартным хуком. Не нужно ничего делать, чтобы она заработала как хук.
4. В основном *нет*, однако существуют ситуации, в которых лучше использовать хук `useState` вместо `useReducer`. Эти два хука служат разным целям и редко напрямую конкурируют друг с другом.
3. *Нет.* Если провайдер не найден, хук `useContext` возвращает значение по умолчанию, предоставляемое при вызове `createContext`.
2. Контекст всегда предоставляется через его свойство `.Provider`, а значение контекста — через свойство `value`.

```
1. <StyleContext.Provider value={style}>
  ...
</StyleContext.Provider>
```

ИТОГИ

- React Context и хук `useContext` — чрезвычайно универсальные и полезные инструменты разработчика React.
- React Context идеально подходит для предотвращения сквозной передачи свойств — практики передачи свойств компоненту только для того, чтобы

он передал полученное свойство следующему компоненту в цепочке. Это признак плохого дизайна архитектуры, и React Context помогает избежать этой проблемы.

- Потребитель React Context (либо через свойство `.Consumer`, либо через хук `useContext`) получает текущее значение контекста от ближайшего провайдера React Context того же типа при поиске вверх по дереву компонентов или значение по умолчанию, если провайдер не найден.
- В рабочем коде всегда рекомендуется использовать хук `useContext` вместо свойства компонента `.Consumer`.
- React Context может успешно применяться для управления очень сложными данными в больших приложениях.
- Редьюсеры идеально подходят для управления сложным состоянием в приложениях. Это отличный инструмент для работы со взаимозависимыми переменными и предотвращения недопустимых конфигураций состояния.
- Редьюсеры — функции, преобразующие состояние в новое состояние в зависимости от заданного действия.
- Редьюсеры по своей природе чисты и не имеют побочных эффектов.
- Можно создавать нестандартные хуки для обобщения функциональности.
- Нестандартные хуки часто намного проще обобщить, чем целые компоненты, и они становятся основным механизмом совместного использования бизнес-логики между частями приложения.
- В Сети доступно множество нестандартных хуков, объединенных в пакеты или добавляемых простым копированием и вставкой.

11

Проект: меню сайта

В ЭТОЙ ГЛАВЕ

- ✓ Подготовка необходимых шаблонов для компонента меню
- ✓ Рендеринг меню для статического сайта
- ✓ Домашнее задание: расширение функциональности меню

Мы достигли важной точки. После главы 10 вы знаете о React все, что необходимо, чтобы создавать довольно сложные веб-приложения. В этой и двух следующих главах рассматриваются примеры проектов. Это намного более масштабные примеры, которые проиллюстрируют первые этапы создания полнофункциональных веб-приложений и подготовят к разработке продвинутых вариантов этих приложений.

Проект этой главы — меню сайта. Это компонент меню сверху страницы, используемый напрямую. Его создание включает пять шагов, как показано на рис. 11.1.

Работа над приложением начинается с создания заготовки. На каждом этапе приложения в меню будут добавляться новые расширенные возможности и будут использоваться новые механизмы React API. В этой главе мы разберем шаги 1 и 2. Остальные шаги мы опишем, но выполнить их вам придется самостоятельно.

Конечно, мы дадим справочное решение для всех шагов, включая шаги 3–5, но не скажем, как мы к нему пришли. Разумеется, ваше решение для шагов 3–5 не будет полностью совпадать с нашим, потому что одно и то же можно сделать разными способами. Если вы зайдете в тупик, загляните в наше решение, но все же сначала постарайтесь справиться самостоятельно.

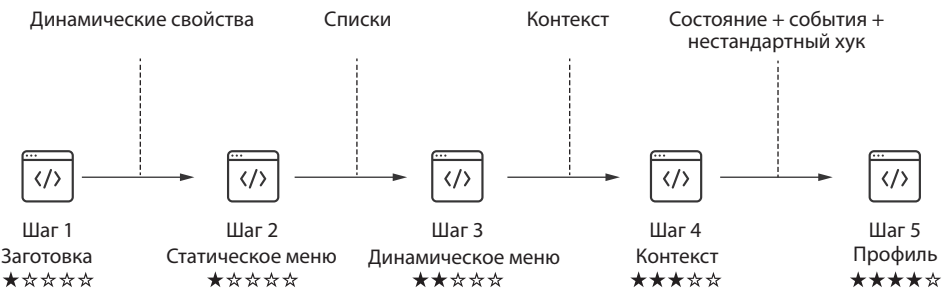


Рис. 11.1. Проект начинается с создания заготовки. Далее в этой главе мы создадим полнофункциональное динамическое меню с необязательной ссылкой на страницу профиля

В табл. 11.1 точно описано, что происходит на каждом шаге: какая функциональность создается и какие части React API при этом используются.

Таблица 11.1. Пять шагов проекта меню

Шаг	Функциональность	Используемый React API	Сложность
Шаг 1. Заготовка	Создание базовой структуры компонента для сайта с пустым меню.	Главы 1–4: функциональные компоненты с использованием JSX.	★☆☆☆☆
Шаг 2. Статическое меню	Добавление статического меню JSX с использованием нестандартного компонента для элементов меню с динамическими свойствами.	Главы 3–4: использование JSX с динамическими свойствами.	★☆☆☆☆
Шаг 3. Динамическое меню	Рендеринг меню по списку объектов, описывающих элементы меню. Примечание: это домашнее задание. Выполните этот шаг <i>самостоятельно</i> !	Главы 3–4: рендеринг списков элементов JSX.	★☆☆☆☆
Шаг 4. Получение ссылок из контекста	Получение списка элементов меню из контекста, предоставленного на уровне всего приложения.	Глава 10: контекст.	★★★★☆

Шаг	Функциональность	Используемый React API	Сложность
Шаг 5. Добавление не-обязательной ссылки	Добавление кнопки входа в систему/выхода из системы для динамического добавления и удаления ссылки на профиль из меню. Примечание: это домашнее задание. Выполните этот шаг <i>самостоятельно</i> !	Глава 5: состояние. Глава 6: события. Глава 10: нестандартный контекст и нестандартные хуки.	★★★★☆

Прежде чем начать, разберемся, что же мы создаем. Это минималистичный сайт с верхним горизонтальным меню, содержащим ссылки на разделы сайта. Оно показано на рис. 11.2.

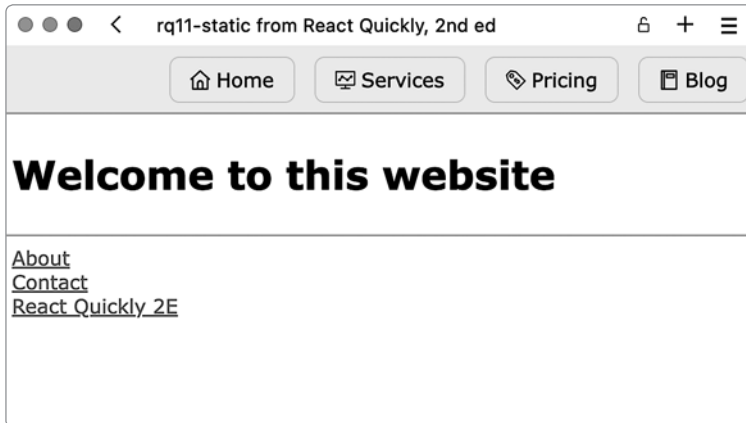


Рис. 11.2. Сайт, который мы создаем. Наша задача — спроектировать меню в его верхней части: элементы Home, Services, Pricing и Blog. При первой итерации элементы меню определяются статически

Для большей выразительности добавим в интерфейс некоторые эффекты, включая небольшой эффект при наведении указателя мыши на элемент меню. На рис. 11.3 показан желаемый результат. Итак, со структурой и целями разобрались, пора за работу!

ПРИМЕЧАНИЕ Исходный код заготовки и предлагаемых решений для всех разделов этой главы доступен по адресу <https://rq2e.com/ch11>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке одной командой.

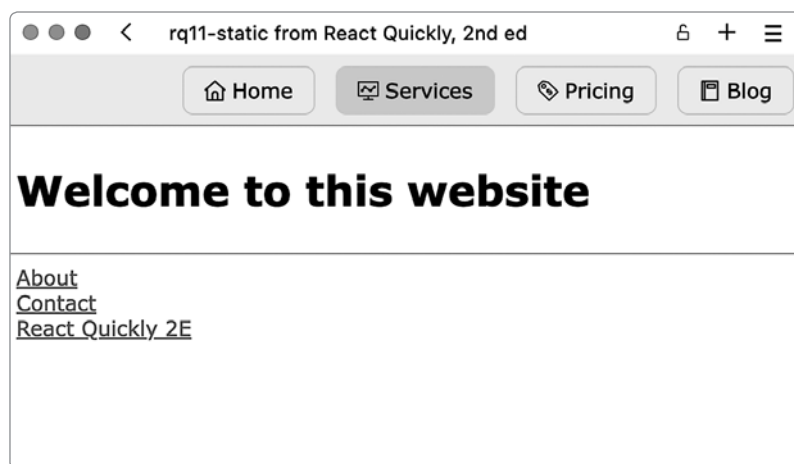


Рис. 11.3. Когда пользователь задерживает указатель над одним из элементов меню, его фон слегка затемняется. Тем самым наглядно выделяется элемент, с которым будет взаимодействовать пользователь

11.1. ЗАГОТОВКА МЕНЮ

Мы начинаем работу над первым «настоящим» проектом React. Для этого нам понадобится план. Продумаем его, как если бы это был реальный проект для клиента, работодателя или компании вашего дядюшки, специализирующейся на стрижке газонов и присмотре за детьми.

Начнем с желаемого результата, показанного на рис. 11.2 и 11.3. Будем действовать по следующей схеме:

1. Определим выходную разметку HTML, которая рендерит желаемый результат.
2. Создадим компоненты React, которые рендерят JSX для получения желаемой разметки HTML.
3. Поместим статические изображения в папку `public`, содержимое которой может загружаться во время выполнения.
4. Создадим таблицу стилей.
5. Реализуем компоненты, необходимые для получения требуемой функциональности.

Вперед!

11.1.1. Выходная разметка HTML

Начнем с создания статической страницы HTML с использованием React и JSX. Обычно React для такой задачи не используют (да это и не рекомендуется), но так как это лишь первый шаг и далее мы добавим динамическую функциональность на этой основе, будет логично начать со статического вывода. Структура выходной разметки HTML для шаблона показана на рис. 11.4.

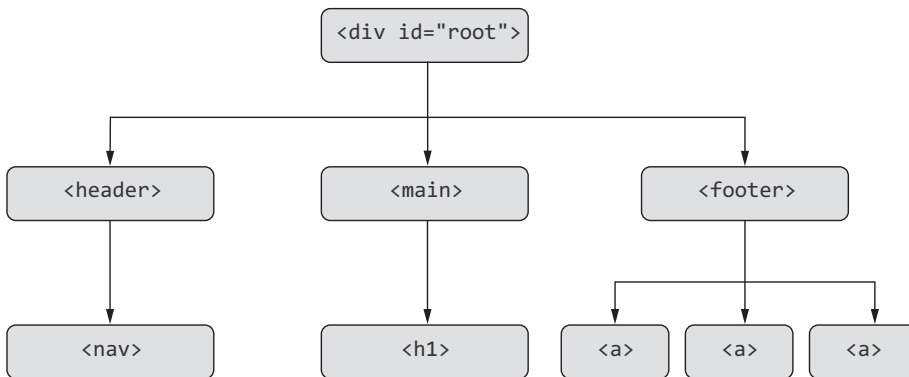


Рис. 11.4. Дерево узлов HTML для шаблона приложения меню состоит только из этих элементов

11.1.2. Иерархия компонентов

Чтобы отрендерить нужное дерево HTML, нам понадобится приложение React, которое рендерит похожий набор узлов JSX. Для реализации этой задачи может использоваться любое количество компонентов React. При желании можно создать по компоненту для каждого узла HTML или один компонент, который будет рендерить все сразу, — решать разработчику.

Ранее мы предлагали готовую структуру компонента React для всех примеров, приведенных в книге. Но скоро (позже в этой главе) вам придется создать ее самостоятельно. Вы сами будете отвечать за результат и разрабатывать дерево компонентов для его получения.

Умение разрезать торт и разбивать результат на компоненты становится важнейшим навыком разработчика React. На рис. 11.5 представлены два разных подхода к созданию компонентов, рендерящих нужный результат.

Существуют разные способы разбиения дерева компонентов, и ни один нельзя считать единственно верным. Решая, как структурировать компоненты,

постарайтесь выдержать баланс между сложностью и распределением обязанностей. В данном случае можно легко реализовать все приложение в одном компоненте, как в примере слева на рис. 11.5; но если вы знаете, что внутри элемента `<nav>` в заголовке позже будет добавлена расширенная функциональность, мы инкапсулируем его в отдельном компоненте. Пока что этот компонент очень маленький, но он станет шире.

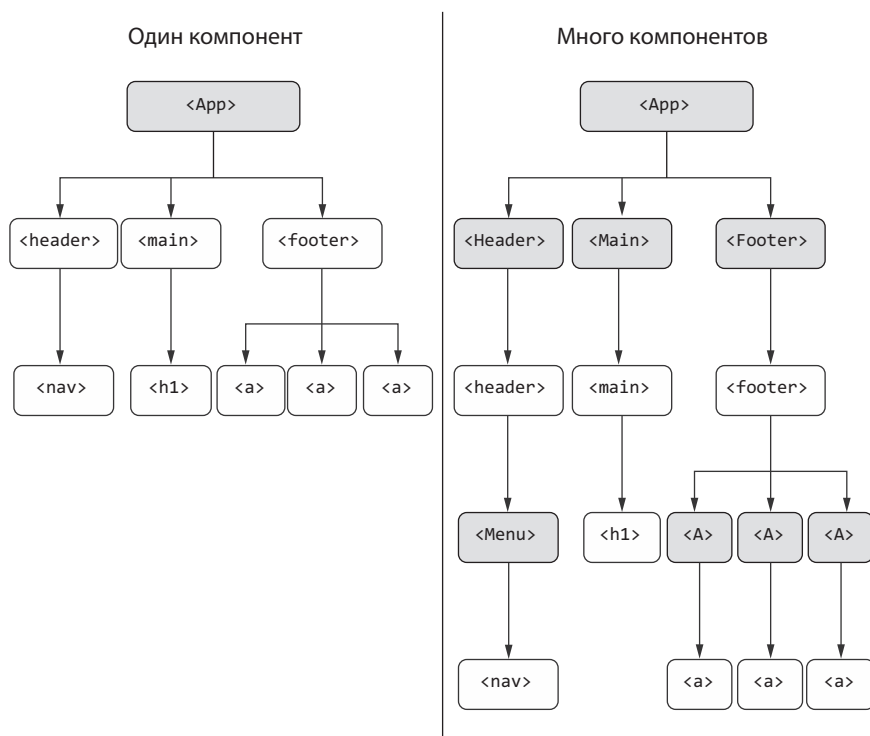


Рис. 11.5. Два подхода к созданию компонентов для заданного вывода. Вместо двух крайних вариантов мы воспользуемся промежуточным решением, потому что сейчас создаем только меню

11.1.3. Значки

Как видно из рис. 11.2, рядом с элементами меню есть значки. Посмотрим на них поближе на рис. 11.6.

Существует много способов рендеринга значков в React; мы воспользуемся простейшим — рендерингом изображений в формате SVG (Scalable Vector Graphics), загруженных из внешнего файла.

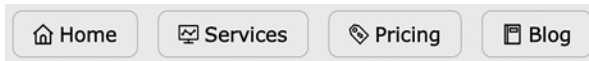


Рис. 11.6. Значки в меню (увеличенные)

Для этого можно разместить файлы в папке **public**, находящейся в папке приложения React:

public/	
icons/ ←	Создает папку icons в папке public
blog.svg	
home.svg	
pricing.svg	
profile.svg	Помещает пять файлов
services.svg	SVG в папку icons
favicon.ico	
index.html	В папке public также находятся два файла, созданных CRA
	(Create-React-App) по умолчанию. Мы их не трогаем

Если эти файлы находятся в папке **public**, их можно загрузить с указанием пути `"/icons/blog.svg"`. Например, значок блога можно отрендерить в теге `<img>`:

```

```

Также обратите внимание, что в папке **icons** находится изображение **profile.svg**. Для первых версий меню рисунок не нужен, но понадобится позже, поэтому он уже размещен здесь на случай, если вы продолжите работать над приложением. Все значки общедоступны, и их можно использовать полностью бесплатно в любом контексте на ваше усмотрение.

11.1.4. CSS

Наконец, в приложении будет использоваться CSS. До сих пор мы использовали встраиваемые стили при помощи атрибута `style` элементов JSX. Такое решение работало, потому что нам было достаточно минимального оформления. Однако сейчас нам понадобится много стилей и смена оформления по наведению указателя мыши. Отрендерить многочисленные стили с использованием встраиваемой формы можно, хотя это и не оптимальный вариант. Но реализовать таким способом оформление по наведению указателя невозможно, поэтому нам понадобится нормальная таблица стилей.

К счастью, с React и CRA эта задача решается очень просто. Эту тему мы почти не обсуждали, но в файле React таблицу стилей можно импортировать напрямую,

и компилятор React при необходимости преобразует его в обычную таблицу стилей, вставленную в HTML.

Плюсы и минусы загрузки стилей во встраиваемой форме или с использованием таблиц стилей выходят за рамки книги, поэтому пока будем придерживаться следующей схемы, поскольку она проста и хорошо подходит для небольших приложений:

1. Создать таблицу стилей `style.css` в папке `src`.
2. Загрузить таблицу стилей в главном файле `App.js`, где определяется корневое приложение.
3. Назначить имена классов элементам JSX и обеспечить их рендеринг по правилам, определенным в таблице стилей.

Вот и все!

Загрузка таблицы стилей в JavaScript означает импортирование файла. Он не импортируется *как* что-то, например компоненты, вы просто импортируете файл в следующем виде:

```
import './style.css';
```

Готово. Теперь можно применять имена классов там, где потребуется.

11.1.5. Шаблон

Мы создали заготовку приложения в виде шаблона, чтобы вы могли сразу приступить к работе.

Репозиторий: `rq11-scaffold`

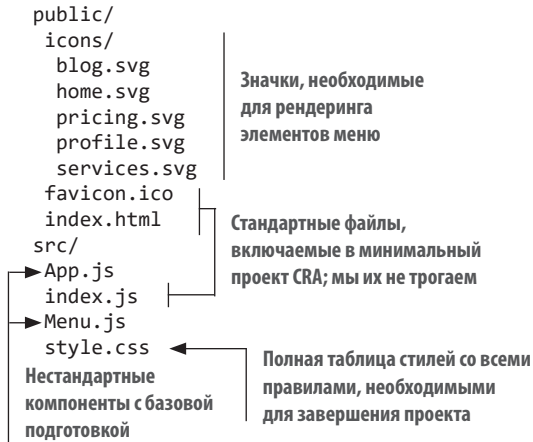
Этот пример находится в репозитории `rq11-scaffold`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq11-scaffold --template rq11-scaffold
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq11-scaffold>

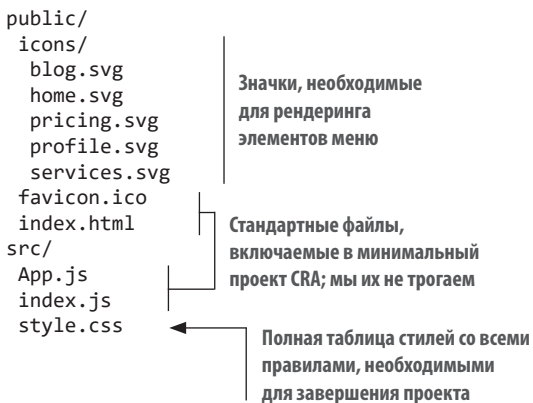
Шаблон включает следующие важные файлы:



Но если хотите, можете начать с нуля. Используйте другой шаблон, который содержит только значки и таблицу стилей, но без нестандартных компонентов. Папка `src` содержит стандартный минимальный шаблон, не считая таблицы стилей:

```
npx create-react-app web-menu --template rq11-minimal
```

Если вы пошли по второму пути, вам придется отредактировать `src/App.js`. Также не забудьте импортировать таблицу стилей. Минимальный шаблон содержит следующие файлы:



11.1.6. Исходный код

Исходный код заготовки приложения, определенной в `src/App.js`, приведен в листинге 11.1.

Листинг 11.1. Файл `src/App.js` для заготовки

```
import Menu from "../Menu";
import "../style.css";
function App() {
  return (
    <>
      <header>
        <Menu />
      </header>
      <main>
        <h1>Welcome to this website</h1>
      </main>
      <footer>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
        <a href="//reactquickly.dev">React Quickly 2E</a>
      </footer>
    </>
  );
}
export default App;
```

Меню определяется во внешнем файле и импортируется в начале

CSS определяется во внешнем файле с именем `style.css`

Рендерит компонент меню в соответствующем месте заголовка

Исходный код файла CSS `src/style.css` представлен в листинге 11.2.

Листинг 11.2. Файл `src/style.css` в заготовке

```
html,
body {
  margin: 0;
  font-family: Verdana;
}
main,
header,
footer {
  padding: 8px;
}
header {
  border-bottom: 1px solid darkgray;
  background: #eee;
}
footer {
  border-top: 1px solid darkgray;
  display: flex;
  flex-direction: column;
}
.menu {
```

```

display: flex;
gap: 16px;
padding: 0;
margin: 0;
list-style: none;
justify-content: flex-end;
}
.menu-link {
text-decoration: none;
color: inherit;
display: flex;
align-items: center;
gap: 5px;
padding: 8px 16px;
border: 1px solid lightgray;
border-radius: 8px;
}
.menu-link:hover {
background: lightgray;
}

```

Исходный код меню, определенного в файле `src/Menu.js`, приведен в листинге 11.3.

Листинг 11.3. Файл `src/Menu.js` в заготовке

```

function Menu() {
  return <nav></nav>; ← Пока что меню рендерит только пустой элемент <nav>
}
export default Menu;

```

11.1.7. В браузере

Запустив приложение в браузере, вы получите симпатичный сайт с пустым меню, представленный на рис. 11.7.



Рис. 11.7. Сайт с пустым меню, которое скоро будет заполнено ссылками

11.2. РЕНДЕРИНГ СТАТИЧЕСКОГО МЕНЮ

На этом шаге проекта мы начнем с текущего состояния после завершения первой части упражнения и добавим в него функциональность, необходимую для рендеринга статического меню с фиксированным списком элементов.

Запустите приложение из шаблона `rq11-scaffold` либо самостоятельно создайте заготовку приложения на основании сказанного в предыдущем разделе. Вы должны получить приложение, определенное в следующем репозитории.

Репозиторий: `rq11-static`

Этот пример находится в репозитории `rq11-static`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq11-static --template rq11-static
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq11-static>

11.2.1. Цель упражнения

Цель этого упражнения — заполнить пустое меню с предыдущего шага. Компонент меню был обычным пустым элементом, не содержащим пунктов меню:

```
function Menu() {  
  return <nav></nav>;  
}
```

Прежде чем начинать работу, необходимо определить нужную разметку HTML, затем решить, какое дерево компонента эффективнее всего использовать для создания этой разметки, и наконец, реализовать эти компоненты.

11.2.2. Выходная разметка HTML

Мы будем выводить HTML только внутри компонента `<nav>`. Чтобы отрендерить список ссылок в меню, воспользуемся деревом HTML на рис. 11.8.

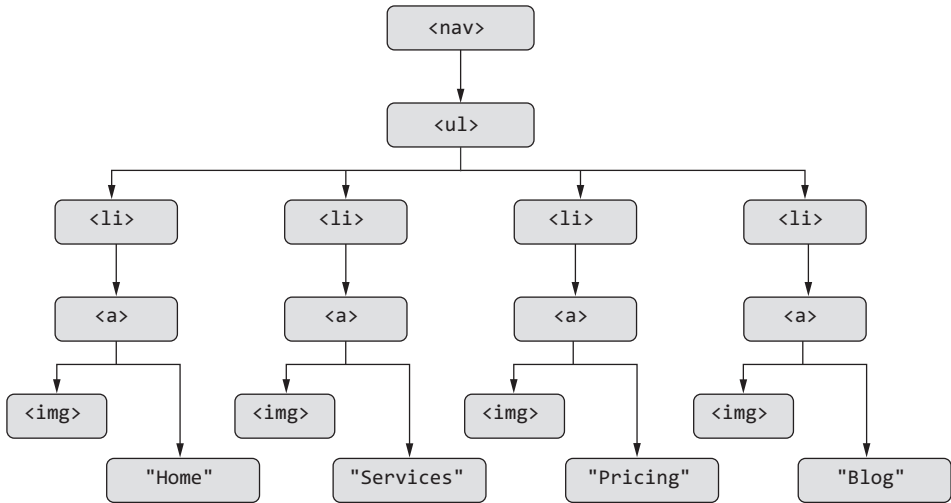


Рис. 11.8. Дерево узлов HTML для элемента навигации представляет собой неупорядоченный список ссылок

11.2.3. Дерево компонентов

Дерево компонентов для подобных структур пишется само собой. Нам нужен компонент, который инкапсулирует дублирующуюся разметку HTML в дереве HTML на рис. 11.8. Назовем его компонентом `MenuItem`, который будет содержать три свойства:

- `href` — целевой URL-адрес, на который будет указывать ссылка.
- `Icon` — имя SVG-файла, загружаемого в качестве значка.
- `Children` — текст внутри ссылки.

Структура этого дерева компонентов изображена на рис. 11.9.

Когда структура будет готова, можно переходить к реализации необходимых изменений в соответствующих компонентах.

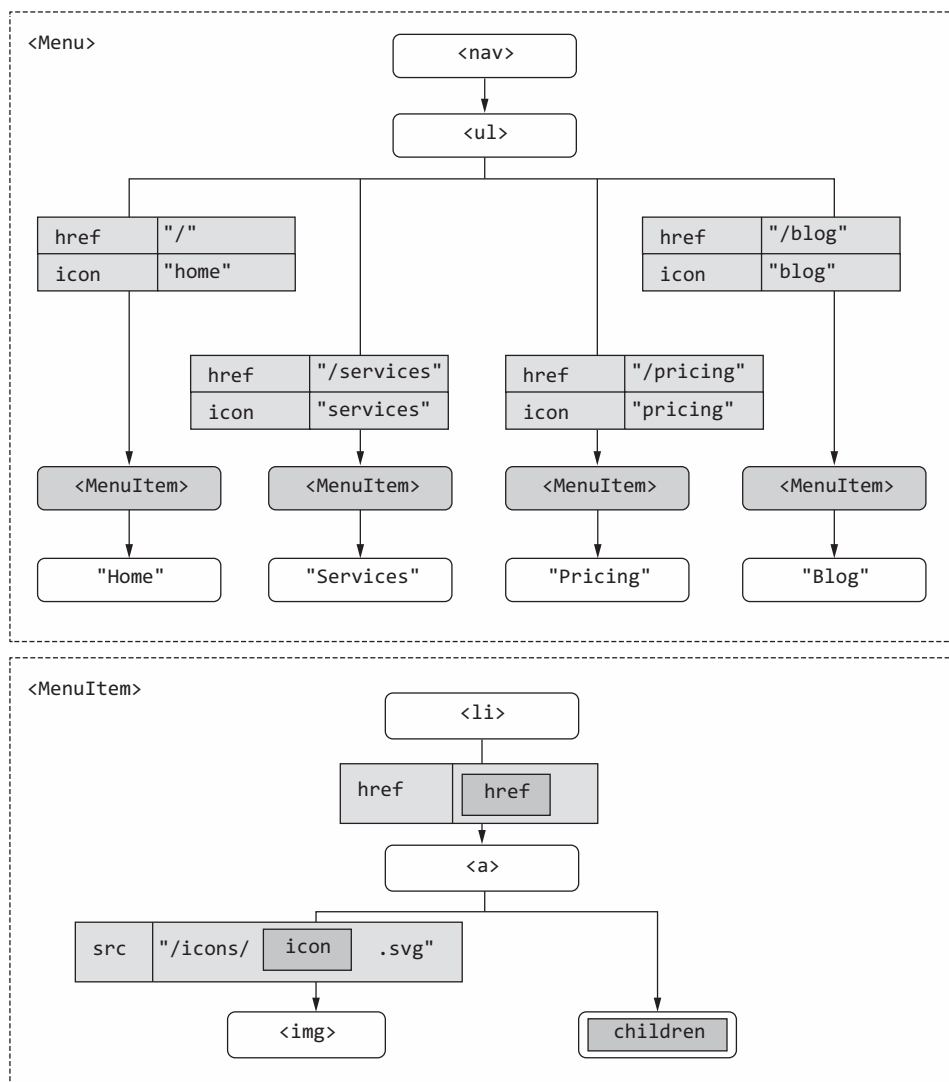


Рис. 11.9. В структуре используются два компонента. Кроме того, компоненту `MenuItem` передаются три свойства

11.2.4. Исходный код

В листинге 11.4 приведена обновленная реализация компонента `Menu.js`. Как видно из рис. 11.7, необходимо отрендерить список из четырех экземпляров `MenuItem`.

Листинг 11.4. Файл `src/MenuItem.js` для статического меню

```
import MenuItem from "./MenuItem";
function Menu() {
  return (
    <nav>
      <ul className="menu">
        <MenuItem href="/" icon="home">
          Home
        </MenuItem>
        <MenuItem href="/services" icon="services">
          Services
        </MenuItem>
        <MenuItem href="/pricing" icon="pricing">
          Pricing
        </MenuItem>
        <MenuItem href="/blog" icon="blog">
          Blog
        </MenuItem>
      </ul>
    </nav>
  );
}
export default Menu;
```

Четыре экземпляра
компонента `MenuItem`
с разными свойствами

Затем реализуем новый компонент `MenuItem.js`.

Листинг 11.5. Файл `src/MenuItem.js` для статического меню

```
function MenuItem({ href, icon, children }) {
  const iconSrc = `/icons/${icon}.svg`;
  return (
    <li>
      <a href={href} className="menu-link">
        <img src={iconSrc} width="16" alt="" />
        {children}
      </a>
    </li>
  );
}
export default MenuItem;
```

Компонент `MenuItem` получает три свойства,
одно из которых — специальное свойство `children`

Определяет источник для
значка на основании известного
местоположения файлов значков
и переданного свойства `icon`

Элементу ссылки нужно
значение `href`, которое
берется из свойств

Рендерит свойство `children`
рядом с изображением
как текст ссылки

Элементу изображения
необходим источник, который
вычисляется в переменной

11.2.5. В браузере

Запустив приложение в браузере, вы получите именно такой результат, как на рис. 11.2. Повторим его на рис. 11.10 для сравнения.



Рис. 11.10. Статическое меню работает! Попробуйте навести указатель мыши на разные элементы меню и понаблюдайте за эффектом, работа которого обеспечивается правилами таблиц стилей

11.3. ДОМАШНЕЕ ЗАДАНИЕ: ДИНАМИЧЕСКОЕ МЕНЮ

Выполнив шаг 2, мы получили красивое статическое меню. Зачем делать то же самое другим способом? Дело в том, что на следующей стадии проекта элементы меню станут динамическими (руководитель недавно говорил об этом), так что лучше подготовиться заранее.

Под «динамическими элементами меню» понимается то, что их количество, а также текст, значок и, возможно, ссылки отдельных элементов меню могут обновляться в процессе взаимодействия пользователя с сайтом. Представьте, что вы заходите на сайт как покупатель; внезапно часть элементов меню исчезает и появляются новые элементы. Разные пользователи могут видеть разный набор элементов меню после входа. Таким образом, хотя текущее решение работает, оно не идеально для динамического меню, в котором элементы должны обновляться в зависимости от состояния и внешних данных.

11.3.1. Цель шага

Цель этого шага — подготовка проекта к динамическому рендерингу за счет перехода на список объектов, которые должны рендериться как элементы меню, чтобы не вводить все элементы меню вручную в JSX. Структура проекта останется прежней, так как в нем, скорее всего, будет использоваться то же количество компонентов, но между ними будет передаваться больше данных. Однако появляется дополнительное условие: список элементов меню должен определяться в компоненте `App` (потому что так говорит сениор — архитектор проекта, а кто мы такие, чтобы с ним спорить?). Так как список используется в компоненте меню, необходимо передать этот список как свойство.

11.3.2. Подсказки для решения

Чтобы выполнить этот шаг, требуется ответить на два вопроса:

1. Как структурировать элементы списка для хранения всей информации, необходимой для рендеринга элементов меню?
2. Как рендерить элементы меню на основании списка объектов?

Для ответа на них дадим несколько подсказок.

Определение элементов списка

Элементы списка должны содержать информацию о том, куда указывает ссылка меню (`href`), какой значок должен отображаться и какой текст должен выводиться. Для этого можно воспользоваться объектом следующего вида:

```
{ title: "Home", href: "/", icon: "home" }
```

Имена свойств остаются полностью на ваше усмотрение, и если хотите, можете их изменить.

Рендеринг элементов меню

Чтобы отрендерить узлы JSX на основании списка элементов, вспомните, что говорилось в разделе 3.2.8 о рендеринге списков объектов JSX. Обычно при рендеринге используется следующая структура:

```
<parent>
  {list.map((object) => (
    <node
      key={object.id}
      otherProp={object.other}
      ...
    />
  ))}
</parent>
```

← Отображает список объектов на список узлов JSX

← Добавляет уникальный ключ для каждого узла

← Добавляет любые другие необходимые свойства из элемента отображенного списка

Не забудьте определить уникальный ключ `key` для каждого узла в отображенном ответе, это важно.

11.3.3. Иерархия компонентов

Количество компонентов не изменилось, но используются они по-другому. Если прежде в компоненте `Menu` содержался статический список из четырех экземпляров компонента `MenuItem`, теперь используется динамический список экземпляров `MenuItem`, зависящий от длины массива ссылок. Необходимо

передать этот список ссылок из компонента `App` в компонент `Menu`. Не изменился только компонент `MenuItem`, так как он уже был динамическим и мог отображать динамический контент.

Мы рекомендуем использовать иерархию компонентов, показанную на рис. 11.11, но вы можете предложить собственную структуру, если хотите. Единственно верного варианта не существует.

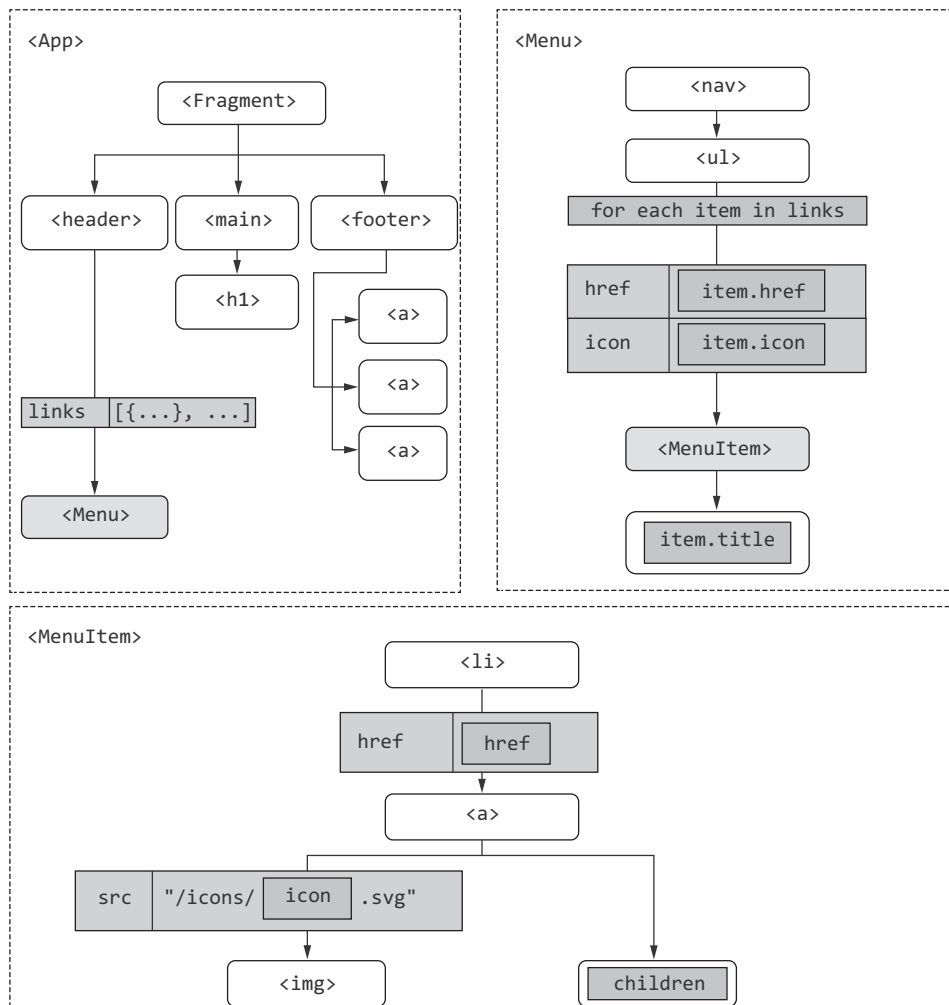


Рис. 11.11. Теперь компоненту `Menu` передается свойство из `App`. Это свойство используется для генерирования динамического количества экземпляров `MenuItem`

11.3.4. Что дальше?

Мы крайне рекомендуем попытаться решить задачу самостоятельно. Если вы справились с шагом 2 и реализовали статический список, продолжайте с этого места. Но если вы хотите начать с чистого листа, можете воспользоваться нашей реализацией шага 2 из репозитория `rq11-static`.

Завершив это упражнение, сравните свою версию с нашей — конечно, они не будут совпадать на 100 %, но вы сможете сопоставить подходы к решению.

Репозиторий: `rq11-dynamic`

Этот пример находится в репозитории `rq11-dynamic`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq11-dynamic --template rq11-dynamic
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq11-dynamic>

Если задание показалось вам интересным, проверьте свои силы на двух следующих шагах, хотя они немного сложнее.

11.4. ДОМАШНЕЕ ЗАДАНИЕ: ПОЛУЧЕНИЕ ДАННЫХ ИЗ КОНТЕКСТА

Мы сделали меню динамическим, и это отличное начало. Но чтобы меню стало действительно динамичным, необходимо иметь возможность работать со списком элементов меню из любой точки приложения. Передавать список ссылок между всеми компонентами утомительно — удобнее переместить список ссылок в окружающий контекст приложения. Так мы сможем легко получать доступ к списку из любой точки.

11.4.1. Цель шага

На этом шаге абстракция поднимается на следующий уровень. Вместо того чтобы передавать список ссылок между компонентами в виде свойства, переместим его в контекст. Пока контекст содержит только список ссылок, который используется в компоненте `Menu`. Чтобы подготовиться к следующему шагу, на

котором контекст станет доступным для других частей приложения, рекомендуем создать контекст как обертку для всего приложения.

11.4.2. Подсказки для решения

Чтобы загрузить ссылки из контекста, необходимо выполнить три условия:

1. Определить контекст React в переменной, доступной из разных компонентов.
2. Создать провайдер контекста вокруг нужной части приложения.
3. Применить хук `useContext`, когда понадобится доступ к переменной контекста.

Определение контекста

Чтобы определить контекст React, достаточно вызвать функцию `createContext()` из пакета React. Чтобы переменная была доступна из нескольких компонентов, ее можно создать в отдельном файле и экспортировать. Это самый быстрый способ достичь цели:

```
import { createContext } from 'react';
const Context = createContext([]);
export default Context;
```

← Аргументом по умолчанию в данном случае является пустой массив

Обратите внимание на передачу пустого массива в качестве значения контекста по умолчанию. Если вы попытаетесь обратиться к контексту, где он не определен, то получите пустой список ссылок в качестве значения.

Создание провайдера контекста

Чтобы создать провайдер контекста, упакуйте нужные компоненты в компонент `Context.Provider`. Этому экземпляру компонента необходимо предоставить свойство `value`, которое содержит текущее значение контекста.

Если контекст хранится в переменной с именем `MenuContext`, а список ссылок — в переменной с именем `links`, контекст для набора компонентов А, В и С можно предоставить следующим образом:

```
return (
  <MenuContext.Provider value={links}>
    <A />
    <B />
    <C />
  </MenuContext.Provider>
);
```

Обращение к значению контекста

Чтобы обратиться к значению контекста, используйте хук `useContext` в компоненте, находящемся внутри провайдера контекста. Например, если контексту присвоено имя `MenuContext`, к текущему значению можно обратиться следующим образом:

```
import { useContext } from 'react';
function SomeComponent() {
  const value = useContext(MenuContext);
  ...
}
```

11.4.3. Иерархия компонентов

Как и в предыдущем случае, у задачи может быть много решений. Мы представили свое видение решения на рис. 11.12, но это лишь одна диаграмма с деревом. Возможны и другие.

11.4.4. Что дальше?

Мы рекомендуем попытаться решить задачу самостоятельно. Если вы справились с шагом 3, продолжайте с этого места. Но если вы хотите начать с чистого листа, можете воспользоваться нашей реализацией шага 3 из репозитория `rq11-dynamic`. Завершив упражнение, для справки сравните свою версию с нашей.

Мы уже довольно далеко продвинулись, поэтому попытайтесь выполнить и следующий шаг. Он немного сложнее предыдущих, но дело того стоит — вы начнете видеть, как все составляющие образуют единый рабочий механизм.

Репозиторий: `rq11-context`

Этот пример находится в репозитории `rq11-context`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq11-context --template rq11-context
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq11-context>

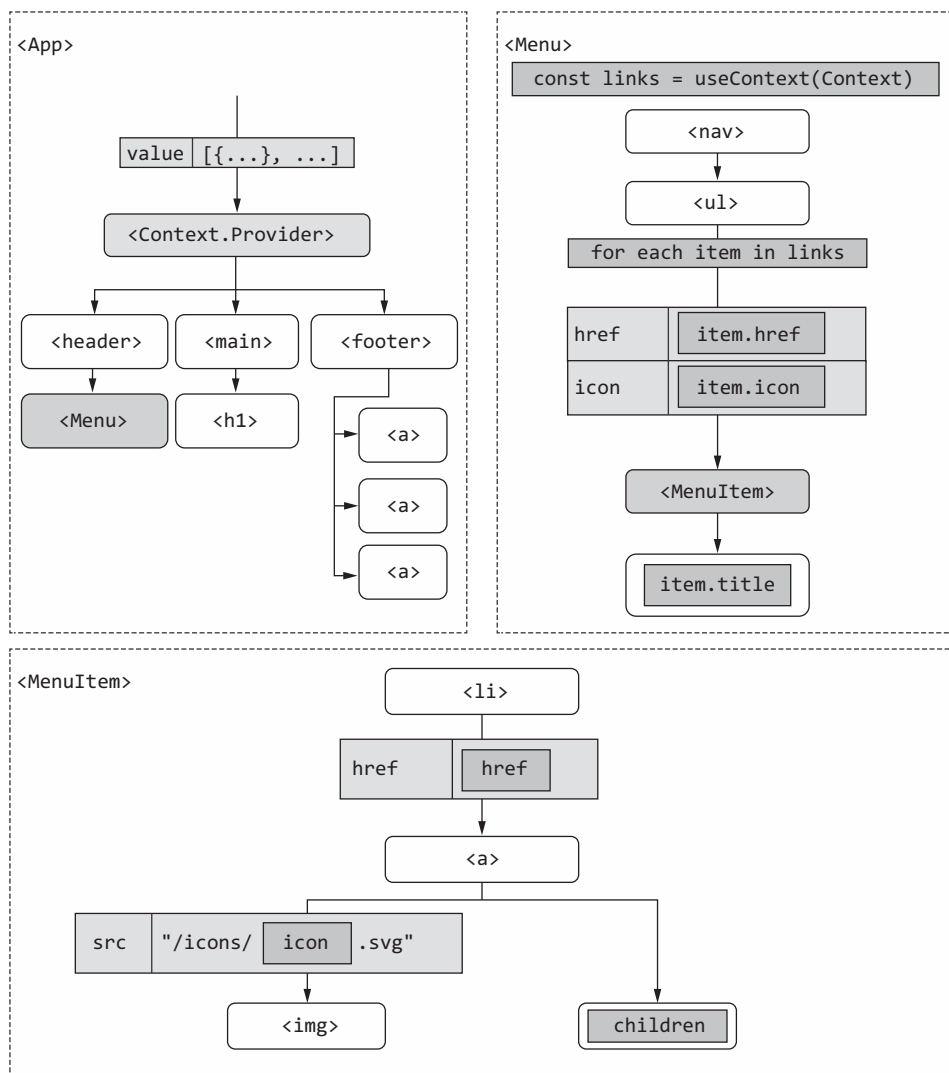


Рис. 11.12. Все приложение оборачивается в провайдер контекста, и этот же контекст используется для получения значения в компоненте меню

11.5. ДОМАШНЕЕ ЗАДАНИЕ: ДОПОЛНИТЕЛЬНАЯ ССЫЛКА

Мы добрались до заключительного шага проекта. На нем мы добавим очень простую механику аутентификации, после прохождения которой у пользователя в меню появится дополнительная ссылка.

Насколько проста схема аутентификации, спросите вы? Она основана на доверии. Если нажать кнопку **Log In**, вход считается успешно выполненным. Нажав **Log Out**, пользователь выходит из сеанса. Конечно, для реальной проверки подлинности это совершенно бесполезно, но для демонстрационных целей достаточно. Блок-схема логики приложения максимально проста и показана на рис. 11.13.

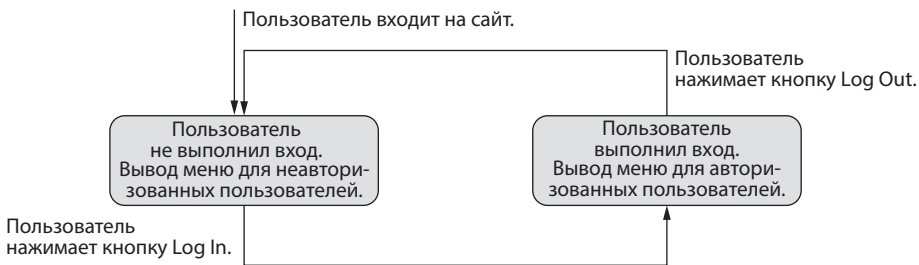


Рис. 11.13. Пользователь либо выполнил вход, либо нет; этот факт отражен в меню

Добавим очень простую кнопку **Log In** в основную секцию страницы под заголовком. На рис. 11.14 показано, как должен выглядеть сайт.

Если пользователь нажал кнопку, он считается вошедшим в систему, и в меню появляется новая ссылка на его профиль. На рис. 11.15 показан желаемый результат.

11.5.1. Цель шага

На этом шаге провайдер контекста будет дополнен новыми свойствами, необходимыми для получения информации о состоянии и операциях с ним. Кроме того, состояние необходимо где-то хранить так, чтобы оно могло обновляться.

В процессе работы вы можете произвольно менять структуру компонентов. Добавьте дополнительные компоненты, если посчитаете нужным. Подобные решения, как мы уже говорили, остаются за разработчиком.

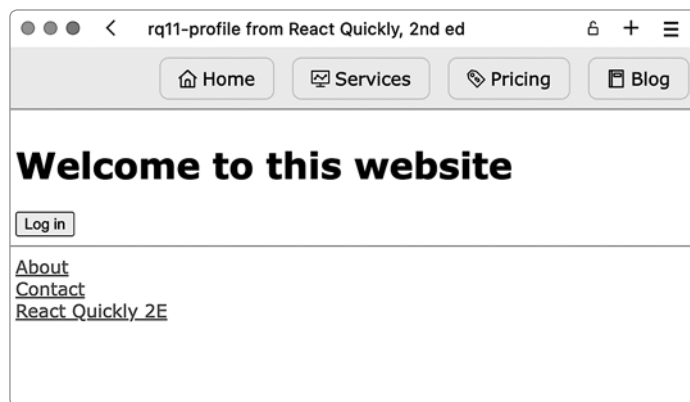


Рис. 11.14. Если пользователь не авторизован, основной раздел сайта содержит кнопку Log In и четыре элемента меню, как и прежде

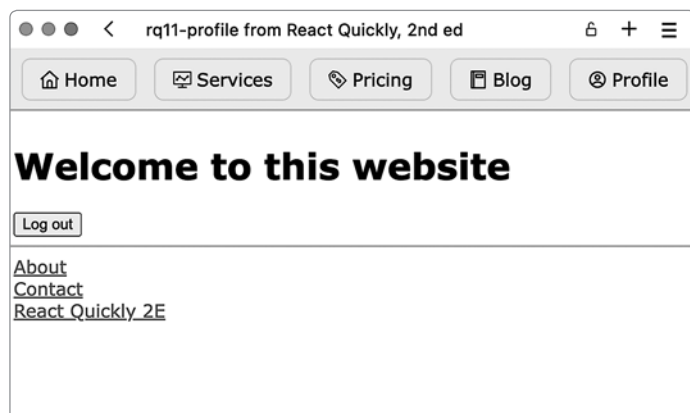


Рис. 11.15. Если пользователь прошел авторизацию, в основном разделе сайта появляется кнопка Log Out, а в меню добавляется пятый элемент

11.5.2. Подсказки для решения

Ниже приведены подсказки, которые помогут вам с решением этой задачи:

- На шаге 4 значение контекста представляет собой список ссылок. Теперь контекст должен содержать более одного значения, поэтому, скорее всего, лучше использовать объект.
- Чтобы было удобно обращаться к значению контекста, можно создать нестандартный хук.
- Чтобы отслеживать логический признак того, прошел пользователь авторизацию или нет, скорее всего, понадобится хук с состоянием.
- Перемещение провайдера контекста в отдельный компонент может упростить понимание логики приложения.

- Так как основной раздел приложения теперь содержит больше логики, может быть полезно выделить его в отдельный компонент.
- Вспомните, что специальное свойство `children` может использоваться для «сквозной» передачи компонентов через другие компоненты.

Объект как значение контекста

Хотя эта тема прямо не рассматривалась в главе 10, можно сохранить в контексте несколько значений. Для этого используется объект:

```
const value = {
  someValue,  ← В объекте провайдера можно хранить сколько угодно значений
  someFunction, ← В нем даже можно хранить функции
};
return (
  <Context.Provider value={value}>
    ...
  </Context.Provider>
);
```

Помните, что при изменении значения провайдера можно изменить исходное значение по умолчанию, передаваемое в `createContext`.

Нестандартный хук для обращения к контексту

Если у вас есть контекст, который вы хотите использовать в нескольких местах (например, контекст API для обращения к какой-то общей функциональности API), можно использовать хук `useContext`:

```
// В Component.js
import { useContext } from 'react';
import APIContext from './API';
...
function Component() {
  const value = useContext(APIContext);
  ...
}
```

Но чтобы упростить себе (и возможно, вашим коллегам) жизнь, создайте нестандартный хук, который делает это за вас. После этого остается импортировать один хук:

```
// В API.js
...
export function useAPI() {
  return useContext(APIContext);
};
```

```
// В Component.js
import { useAPI } from './API';
...
function Component() {
  const value = useAPI();
  ...
}
```

Может показаться, что создавать такие крошечные нестандартные хуки глупо, но на самом деле они весьма полезны и часто используются на практике.

Логическое значение с состоянием

Если вам потребуется, чтобы простое значение обладало состоянием в React, проще всего воспользоваться хуком `useState`, как объясняется в главе 5:

```
import { useState } from 'react';
const [isVisible, setVisible] = useState(false);
```

Возвращаемое значение `useState` можно деконструировать в значение и функцию-сеттер

Помните, что сеттер состояния не должен быть доступен напрямую. Создайте собственные функции, которые упростят работу с ним:

```
const [isVisible, setVisible] = useState(false);
const show = () => setVisible(true);
const hide = () => setVisible(false);
```

Конечно, это лишь общие примеры; вам потребуется изменить их для своего приложения.

Кроме того, не забудьте использовать переменную с состоянием. В нашем конкретном приложении значение ссылок должно изменяться в зависимости от логического значения с состоянием.

Компонент провайдера контекста

Когда в контекст добавляется новая логика и новые значения, часто появляется смысл переместить его в специальный компонент. Таким образом, вместо

```
function App() {
  ...
  const value = { a, b, c };
  return (
    <Context.Provider value={value}>
      ...
    </Context.Provider>
  );
}
```

можно создать два компонента, App и ValueProvider, и разбить их следующим образом:

```
function ValueProvider() {
  const value = { a, b, c };
  return (
    <Context.Provider value={value}>
      ...
    </Context.Provider>
  );
}
function App() {
  ...
  return (
    <ValueProvider>
      ...
    </ValueProvider>
  );
}
```

Преобразование части компонента в отдельный компонент

Если компонент становится слишком сложным, стоит выделить его часть в отдельный компонент. Предположим, что имеется компонент, состоящий из нескольких секций, и мы добавляем сложность в одну из них:

```
// В App.js
function App() {
  const onClickButton = () => { ... };
  return (
    <>
      <header>
        ...
      </header>
      <main>
        <p>This is main</p>
        <button onClick={onClickButton}>
          ...
        </button>
      </main>
      <aside>
        ...
      </aside>
    </>
  );
}
```

Эта переменная используется только в основной секции

Основная секция стала слишком большой, ее лучше выделить в новый компонент

В какой-то момент вы можете почувствовать, что основной раздел приложения становится слишком большим и лучше выделить его в отдельный компонент. Чтобы это сделать, возьмите соответствующий JSX (и связанные переменные) компонента и переместите в новый компонент:

```
// В Main.js
function Main() {
  const onClickButton = () => { ... };
  return (
    <main>
      <p>This is main</p>
      <button onClick={onClickButton}>
        ...
      </button>
    </main>
  );
}
// In App.js
function App() {
  return (
    <>
      <header>
        ...
      </header>
      <Main />
      <aside>
        ...
      </aside>
    </>
  );
}
```

Мы создали новый компонент (в новом файле), содержащий только часть предыдущего компонента

Теперь вся основная секция просто заменяется новым компонентом

Оба компонента заметно упростились, и теперь читателю кода будет проще понять их назначение.

Свойство children

Иногда новый компонент не требуется делать специализированным. В разных условиях он должен заполняться разным контентом. Предположим, у вас есть приложение из нескольких секций с одинаковым стилем, но содержимое этих секций различно:

```
Но содержимое различается
function App() {
  return (
    <main>
      <section className="section section-fancy">
        <A />
      </section>
      <section className="section section-fancy">
        <B />
      </section>
      <section className="section section-fancy">
        <C />
      </section>
    </main>
  );
}
```

Во всех секциях используется один класс

```

    </main>
  );
}

```

В этом случае имеет смысл создать компонент для разделов, который позволит передавать произвольных потомков. Для этой цели можно воспользоваться свойством `children`:

```

function Section({ children }) { ← Новый компонент с обобщенной разметкой секции
  return (
    <section className="section section-fancy">
      {children} ← Не забудьте отрендерить
    </section>      свойство children там, где
  );              должны выводиться потомки
}
function App() {
  return (
    <main>
      <Section> ←
        <A />    Теперь секции можно заменить
      </Section>  новым компонентом.
      <Section> ← Без дублирования логики
        <B />    приложение выглядит
      </Section>  намного аккуратнее
      <Section> ←
        <C />
      </Section>
    </main>
  );
}

```

Почему мы говорим об этом сейчас? Этот прием часто используется со специализированными компонентами провайдеров, поэтому можно поступить, например, так:

```

function App() {
  ...
  return (
    <ValueProvider>
      <h1>Какой-то заголовок</h1>
    </ValueProvider>
  );
}

```

11.5.3. Иерархия компонентов

На этом последнем шаге вам придется принять намного больше решений, поэтому мы не будем слишком влиять на вас. Однако на рис. 11.16 приведем возможную высокоуровневую схему компонентов в окончательной версии приложения.

11.5.4. Что дальше?

Если вы захотите проверить свои силы (а нам кажется, это стоит сделать), можете продолжить работу над приложением с последнего шага. Или можете воспользоваться нашим приложением в состоянии после завершения шага 4 — в таком случае начните с реализации из репозитория `rq11-context`. Завершив упражнение, для справки сравните свою версию с нашей.

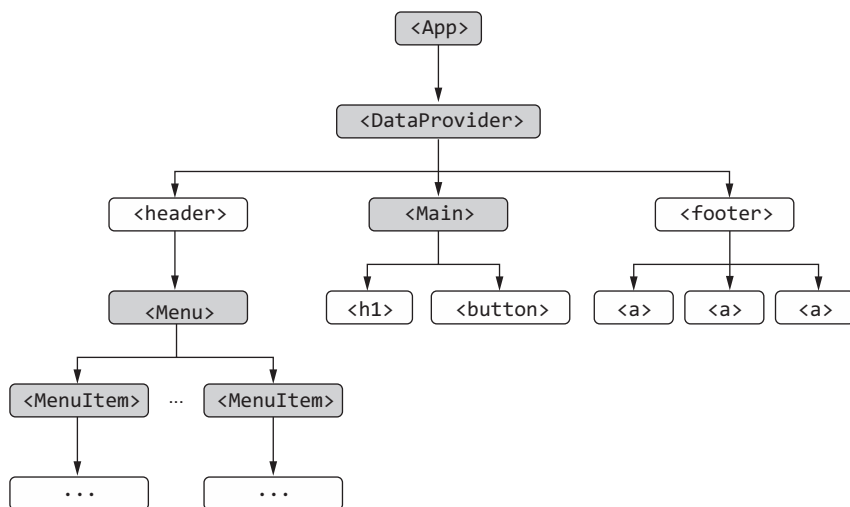


Рис. 11.16. Высокоуровневая схема возможной структуры компонентов приложения на последнем шаге

Репозиторий: `rq11-profile`

Этот пример находится в репозитории `rq11-profile`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq11-profile --template rq11-profile
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq11-profile>

Это все — работа над проектом завершена. Но никто не мешает вам расширить его, дополнить новой функциональностью и поэкспериментировать с возможностями

контекстов для управления данными. Можно добавить кнопку для входа с привилегиями администратора, при котором в меню будут рендериться новые элементы или, возможно, меню полностью изменится.

11.6. НЕСКОЛЬКО СЛОВ НАПОСЛЕДОК

В первом проекте мы взяли на себя роль наставника. Мы прошли все шаги медленно и подробно разбирали, как их завершать. В двух следующих проектах будет меньше советов и детальных разборов. При необходимости вы всегда можете вернуться к первому проекту и посмотреть, что нужно сделать, чтобы выполнить упражнение.

Финальная версия этого проекта может стать неплохой основой для сайта с динамическим меню. Впрочем, ей еще многого не хватает, чтобы она могла применяться для построения реальных сайтов. В ней нет поддержки хостинга, рендеринга на стороне сервера, аутентификации с использованием бэкенда и многого другого, но эти темы выходят за рамки книги.

ИТОГИ

- Этот проект содержит основные этапы создания реальных приложений с использованием React.
- Начните с анализа желаемого результата и попробуйте создать разметку HTML, которая обеспечит рендеринг этого результата.
- Далее спроектируйте дерево компонентов, которое создаст эквивалентную разметку JSX. На первой итерации она может быть полностью статичной и лишенной состояния.
- Затем добавляйте сложность, пока приложение не придет к нужному состоянию.
- Со временем у вас накопится опыт разработки React и достаточно уверенности, чтобы пропускать отдельные шаги этого процесса и, возможно, перейти сразу к последнему шагу, потому что вы уже хорошо знаете, как работать с контекстами, списками объектов JSX, переменными состояния и всем, что вам может понадобиться.

12

Проект: таймер

В ЭТОЙ ГЛАВЕ

- ✓ Создание заготовки для компонента таймера
- ✓ Реализация таймера с фиксированной продолжительностью
- ✓ Домашнее задание: расширение функциональности таймера

В следующем проекте мы займемся построением таймера с обратным отсчетом. Он почти не отличается от таймера на телефоне, только вы создадите его самостоятельно! В этом упражнении будем считать, что вы набрались опыта, так что сейчас будет меньше подсказок по сравнению с главой 11 и большую часть работы вы выполните самостоятельно. Но не волнуйтесь, мы верим в вас — все получится!

Мы подготовим основу проекта за вас, так что вам не придется долго возиться с проектированием или семантикой. Для этого мы создали заготовку для шага 1, содержащую статическую разметку HTML, и семантику, необходимую для шага 2, а также все изображения, значки и стили проекта. Высокоуровневая схема проекта представлена на рис. 12.1.

На шаге 2 мы общими усилиями добавим состояние и преобразуем статический вывод шага 1 в рабочий, хотя ограниченный по своим функциям таймер с фиксированной продолжительностью. На шаге 3 добавим инициализацию, чтобы

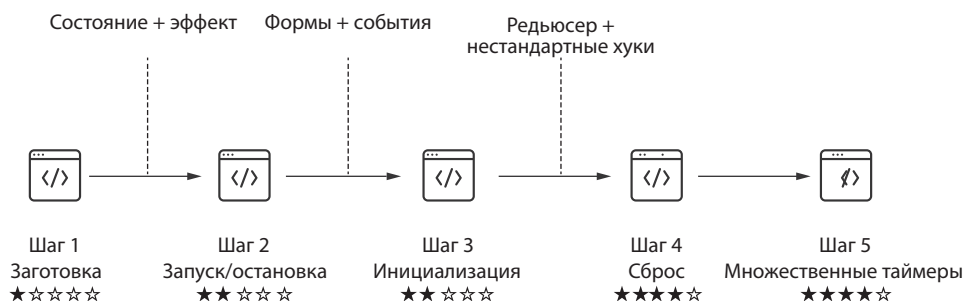


Рис. 12.1. В ходе работы над проектом мы выполним пять шагов и создадим довольно сложный таймер

таймер можно было установить на произвольный интервал, а также сбросить работающий таймер. Это потребует добавления форм и обработки событий. При добавлении сброса таймера на шаге 4 мы проведем рефакторинг хранения состояния, переместив его в редьюсер и нестандартный хук. Наконец, на шаге 5 добавим возможность одновременного запуска нескольких таймеров, работающих независимо друг от друга.

Как мы уже сказали, теперь мы доверяем вам больше, так что последние два шага будут довольно сложными. Более подробное описание процесса и используемых технологий представлено в табл. 12.1.

Таблица 12.1. Пять шагов проекта таймера

Шаг	Функциональность	Используемый React API	Сложность
Шаг 1. Заготовка	Создание базовой структуры компонента для вывода времени и кнопок	Главы 1–4: функциональные компоненты с использованием JSX	★☆☆☆☆
Шаг 2. Запуск/остановка	Реализация компонента с состоянием, который ведет обратный отсчет времени после запуска, пока не будет остановлен или не истечет время	Глава 5: состояние Глава 6: хук эффекта	★★☆☆☆
Шаг 3. Инициализация	Инициализация таймера заданным интервалом, который определяется пользователем в форме с полями ввода. Примечание: это домашнее задание. Выполните этот шаг самостоятельно!	Глава 8: прослушивание событий. Глава 10: обработка пользовательского ввода в форме	★★☆☆☆

Таблица 12.1 (окончание)

Шаг	Функциональность	Используемый React API	Сложность
Шаг 4. Сброс	Преобразование логики состояния в редьюсер, а также добавление новой логики для сброса таймера. Примечание: это домашнее задание. Выполните этот шаг <i>самостоятельно!</i>	Глава 10: редьюсер и нестандартный хук	★★★★☆
Шаг 5. Множественные таймеры	Реализация возможности создания нескольких таймеров, выполняемых независимо друг от друга. Примечание: это домашнее задание. Выполните этот шаг <i>самостоятельно!</i>		★★★★☆

А чтобы немного подогреть ваш интерес, приведем несколько снимков экрана окончательной версии. Хотя иллюстрации в книге черно-белые, настоящий таймер использует в качестве фона приятный фиолетовый градиент. Результат шага 1 представлен на рис. 12.1, а финальная версия с множественными таймерами — на рис. 12.3.

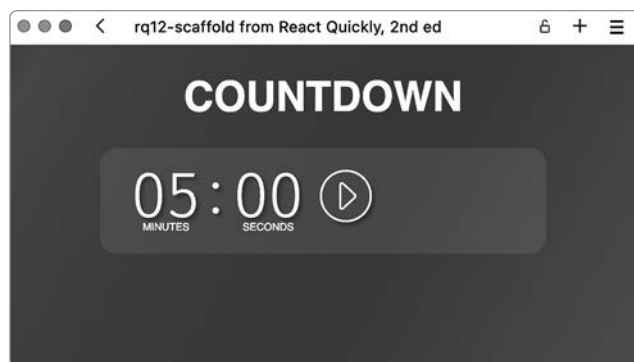


Рис. 12.2. На шаге 1 создается структура проекта и стили, необходимые для рендеринга статического таймера. На этом этапе кнопка ничего не делает и отсчет времени не ведется

ПРИМЕЧАНИЕ Исходный код заготовки и предлагаемых решений для всех разделов этой главы доступен по адресу <https://rq2e.com/ch12>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке одной командой.

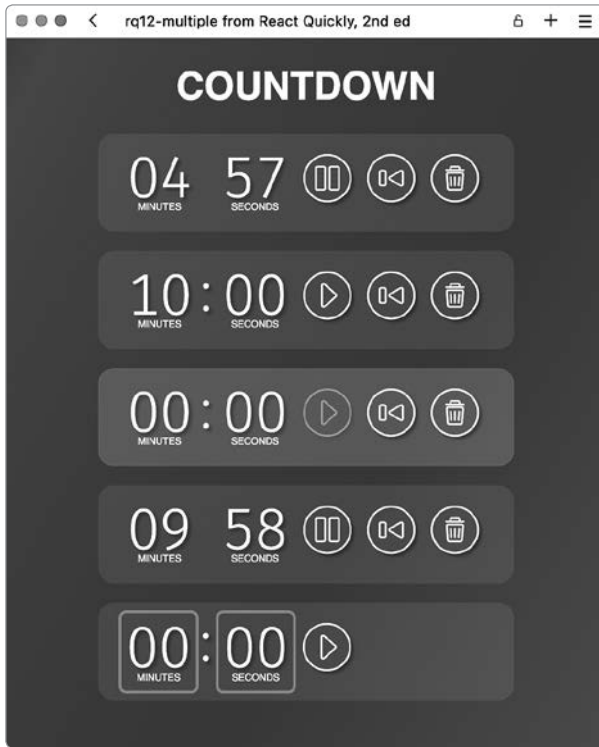


Рис. 12.3. По завершении шага 5 получается потрясающее приложение с несколькими независимо работающими таймерами. Только представьте, сколько яиц разного размера вы сможете варить одновременно!

12.1. ЗАГОТОВКА ДЛЯ ТАЙМЕРА

Чтобы вам было проще сосредоточиться на реализации кода React и не возиться с HTML и CSS, мы предоставим все необходимые стили и семантику. Все эти параметры будут настроены в заготовке приложения. Как и в предыдущем примере (а на самом деле и в любом проекте веб-разработки), работа над проектом включает несколько этапов:

1. Определить выходную разметку HTML, которая рендерит желаемый результат.
2. Создать компоненты React, которые рендерят JSX для получения желаемой разметки HTML.
3. Поместить статические изображения в папку `public`, содержимое которой может загружаться во время выполнения.
4. Создать таблицу стилей.

5. Реализовать компоненты, необходимые для получения требуемой функциональности.

Последний этап представляет для нас самый большой интерес, и мы будем заниматься им на шагах 2–5 проекта. Первые четыре этапа будут выполнены на шаге 1.

12.1.1. Выходная разметка HTML

Для выходной разметки HTML последовательно разберем части проекта и поймем, как моделировать их с использованием HTML. Затем в каждый узел добавим классы CSS, которые можно будет использовать в таблицах стилей в следующей части. Когда все части будут готовы, соберем их вместе и сформируем проект. Отдельные части этого проекта наглядно выделены на графике на рис. 12.4.



Рис. 12.4. Семь частей, образующих приложение-таймер. Они расположены в следующем порядке, начиная с внутреннего уровня: число и единица измерения, кнопка, два числа, разделенных двоеточием, полный таймер, поле ввода формы, форма с несколькими полями ввода и кнопкой и, наконец, полный список таймеров

Разберем каждую из этих частей и опишем связанную с ней структуру HTML.

Число и единица измерения

С каждым числом в индикаторе времени связана некоторая единица измерения. Текстовые поля числа и единицы измерения объединяются в элемент списка. Этот элемент списка будет использоваться для вывода времени как единое целое. Таким образом, каждое отдельное число и единица образуют элемент списка с внутренними абзацами:

```
<li class="part">
  <p class="number">05</p>
  <p class="unit">minutes</p>
</li>
```

Кнопка

Конечно, это самая обычная кнопка, но со значком, соответствующим названию `title`:

```
<button title="Play" class="toggle">
  
</button>
```

Составляющие времени

Время выводится в виде списка частей, а именно числа и единицы измерения, за которыми следует двоеточие, а затем еще одно число и единица измерения:

```
<ul class="parts">
  <!-- число + единица -->
  <li class="colon">:</li>
  <!-- число + единица -->
</ul>
```

Таймер

Таймер представляет собой секцию, содержащую список составляющих времени, за которым следует одна или несколько кнопок:

```
<section class="timer">
  <!-- составляющие времени -->
  <!-- кнопка(и) -->
</section>
```

Во время работы таймера добавляется класс `timer-ticking`, чтобы двоеточие в индикаторе времени мигало:

```
<section class="timer timer-ticking">
  <!-- составляющие времени -->
  <!-- кнопка(и) -->
</section>
```

Если таймер достигает 0, он должен начинать мигать, указывая на истечение интервала. Для этого его можно пометить добавлением класса `timer-ringing`:

```
<section class="timer timer-ringing">
  <!-- составляющие времени -->
  <!-- кнопка(и) -->
</section>
```

Поля ввода

Область ввода состоит из числа и единицы измерения, но вместо двух абзацев они представляются полем ввода и ярлыком. При этом они образуют часть, которая представлена элементом списка:

```
<li class="part">
  <input class="number" type="number" name="seconds" id="seconds" />
  <label class="unit" for="seconds">Seconds</label>
</li>
```

Форма

Поля ввода составляют форму с кнопкой. Так как кнопка внутри формы автоматически становится кнопкой отправки данных, нам не придется ничего делать, чтобы форма заработала:

```
<form class="timer timer-new">
  <ul class="parts">
    <!-- ввод -->
    <li class="colon">:</li>
    <!-- ввод -->
  </ul>
  <!-- кнопка -->
</form>
```

Таймеры

Список таймеров представляет собой обычный элемент, охватывающий все таймеры, и возможно, форму для добавления нового таймера в конце:

```
<div class="timers">
  <!-- таймер(ы) -->
  <!-- необязательная форма -->
</div>
```

Если список таймеров включает кнопку + для запуска нового таймера, ее можно добавить как кнопку с классами `timer` и `timer-add`:

```
<div class="timers">
  <!-- таймер(ы) -->
  <button class="timer timer-add">+</button>
</div>
```


12.1.2. Иерархия компонентов

В предыдущем разделе перечислены некоторые части приложения, которые легко преобразуются в компоненты React, необходимые для рендеринга приложения. Однако в заготовке компонентов будет намного меньше, и только после того, как мы начнем добавлять функциональность на шаге 2, компоненты начнут разбиваться на части. Для заготовки будут использоваться всего три компонента, как показано на рис. 12.5.

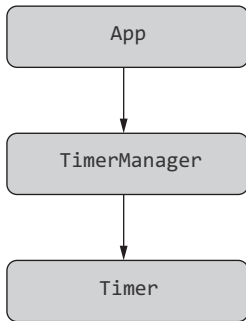


Рис. 12.5. Мы довольно долго будем работать с этим простейшим деревом компонентов. Обратите внимание: на диаграмме представлены только компоненты React, а не все простые узлы JSX, которые также будут рендериться

В дальнейшем мы добавим новую функциональность и логику, что естественно приведет к повышению сложности дерева компонентов.

12.1.3. Структура проекта

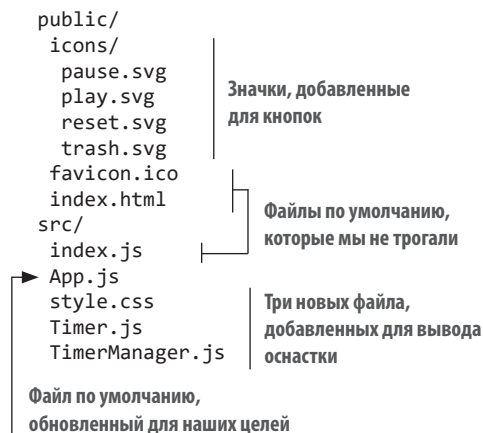
Для этого проекта нам понадобятся значки. Всего нужны четыре разновидности кнопок. Все они представлены на рис. 12.6.



Рис. 12.6. Четыре разновидности кнопок, которые нужны в приложении: воспроизведение, остановка, сброс и удаление

Как и в главе 11, мы поместим файлы значков в папку `public`, находящуюся в отдельной папке `icons`. Кроме того, нам понадобятся три компонента дерева

компонентов из предыдущего раздела и, конечно, таблица стилей. В итоге заготовка будет включать следующие файлы:



Репозиторий: rq12-scaffold

Этот пример находится в репозитории `rq12-scaffold`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq12-scaffold --template rq12-scaffold
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq12-scaffold>

Все готово для работы. Продолжив читать этот раздел, вы подробнее узнаете об исходном коде заготовки, но если вам просто нравится увиденное, можете переходить к разделу 12.2, в котором мы займемся реализацией простейшей версии таймера.

12.1.4. Исходный код

Этот раздел включает исходный код базовых компонентов, входящих в заготовку. Копировать их не нужно, так как этот код готов к выполнению в уже упомянутом шаблоне.

Таблица стилей

Мы не приводим всю таблицу стилей, поскольку она представляет собой множество простых правил CSS, однако на некоторые моменты следует обратить внимание. Прежде всего, мы загружаем из Google Fonts API шрифт с именем Fira Sans с красивыми цифрами, которые используем для отображения показаний часов. Этот шрифт также поддерживает функциональность *табулированных чисел* (tabular numbers). Мы хотим выводить числа в формате с фиксированной шириной; иначе говоря, ширина цифр не должна меняться при переходе индикатора с 10 на 09. Но во многих шрифтах 1 и 0 имеют разную ширину, и цифры будут смещаться, если только не поместить каждую цифру в отдельную область.

Некоторые семейства шрифтов позволяют указать, что числа должны выводиться в формате с фиксированной шириной, занимая одинаковое количество пикселей при заданном размере шрифта. Загрузим шрифт и активируем режим табулированных чисел в CSS при помощи следующего объявления:

```
@import url("https://fonts.googleapis.com/css2
➤ ?family=Fira+Sans:wght@300&display=swap");
.number, .colon {
  font-family: "Fira Sans", sans-serif;
  font-variant-numeric: tabular-nums;
}
```

Классы должны использовать загруженный шрифт

Включает модификацию шрифта «tabular-nums», чтобы использовать числа с фиксированной шириной

Не все шрифты поддерживают эту модификацию, и если вы захотите использовать другой шрифт для чисел, убедитесь, что он поддерживает эту возможность, благодаря чему вывод выглядит очень привлекательно.

Кроме того, глядя на таблицу стилей, можно заметить, что для создания макетов в этой таблице стилей используется Flexbox. Это просто удобный инструмент для выравнивания элементов в подобных приложениях.

Основное приложение

Компонент основного приложения — наша отправная точка. В этом упражнении он полностью аналогичен первой версии (листинг 12.1), так как содержит только строку заголовка и компонент `TimerManager`. Здесь будет размещаться вся логика таймеров, так что глобального состояния в этом приложении не будет.

Также обратите внимание, что в корневом компоненте загружается таблица стилей CSS. Загрузить ее нужно только один раз, так что есть смысл сделать это в корне. Файл `src/App.js` приведен в листинге 12.1.

Листинг 12.1. Файл `src/App.js` в заготовке

```
import "./style.css";
import TimerManager from "./TimerManager";
function App() {
  return (
    <main className="wrapper">
      <h1 className="title">Countdown</h1>
      <TimerManager />
    </main>
  );
}
export default App;
```

Импортирует таблицу стилей, чтобы можно было использовать стили с самого начала работы

Загружает компонент менеджера таймера

Рендерит его в нужном месте

Менеджер таймера

Менеджер таймера — контейнер, содержащий один или несколько таймеров, а также логику инициализации или добавления новых таймеров. Этот компонент не отвечает за реальное управление отдельными таймерами (например, возможность перевода текущего времени), а управляет разными таймерами и их начальным временем.

В заготовке менеджер таймера содержит один экземпляр таймера без свойств. Этот файл `src/TimerManager.js` приведен в листинге 12.2.

Листинг 12.2. Файл `src/TimerManager.js` в заготовке

```
import Timer from "./Timer";
function TimerManager() {
  return (
    <div className="timers">
      <Timer />
    </div>
  );
}
export default TimerManager;
```

Менеджер таймера импортирует компонент таймера, чтобы отобразить его экземпляр

Рендерит экземпляр без свойств или другой логики

Отдельный таймер

Основная часть работы будет выполняться в компоненте таймера. В исходной заготовке это чисто статический компонент с фиксированным ответом JSX, который выводит время 05 минут и 00 секунд, а также кнопку **Play**. Впрочем, пока что они не работают.

Заметим, что в реализации присутствует дублирующийся код JSX, который затем может быть оптимизирован разбиением на отдельные компоненты, включая вывод числа и единиц измерения. Кнопку также хорошо бы выделить в компонент, так как позже она будет использоваться в нескольких местах. Этот файл `src/Timer.js` приведен в листинге 12.3.

Листинг 12.3. Файл `src/Timer.js` в заготовке

```
function Timer() {
  return (
    <section className="timer">
      <ul className="parts">
        <li className="part">
          <p className="number">05</p>
          <p className="unit">minutes</p>
        </li>
        <li className="colon">:</li>
        <li className="part">
          <p className="number">00</p>
          <p className="unit">seconds</p>
        </li>
      </ul>
      <button title="Play" className="toggle">
        
      </button>
    </section>
  );
}
export default Timer;
```

Оставшееся время
05 минут и 00 секунд
пока остается жестко
закодированным

12.1.5. Запуск приложения

Открыв приложение в браузере, вы увидите результат, показанный в начале главы, на рис. 12.2, и здесь, на рис. 12.7. Приложение пока не работает, но выглядит привлекательно, правда?

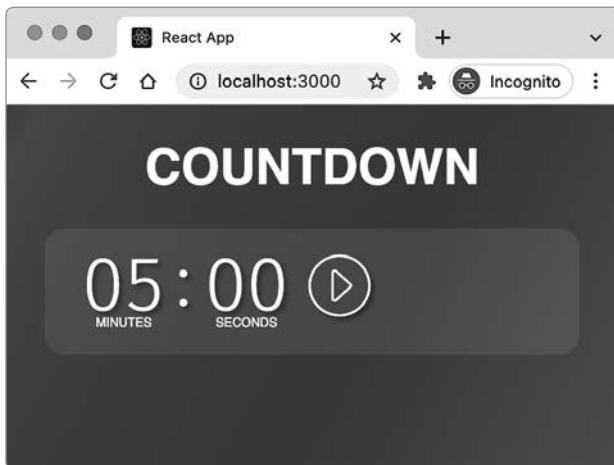


Рис. 12.7. Заготовка таймера ничего не делает... пока

12.2. ДОБАВЛЕНИЕ ПРОСТОЙ КНОПКИ ЗАПУСКА/ОСТАНОВКИ ТАЙМЕРА

На предыдущем шаге у нас получилось симпатичное, но очень скучное приложение — ведь оно ничего не делает. Пришло время его изменить. Начнем с реализации простейшего таймера с обратным отсчетом, запускаемым кнопкой, и возможностью его приостановить, если бомба должна взорваться через пару секунд или яйцо почти сварилось.

12.2.1. Цель упражнения

Цель этого упражнения — создание необходимой функциональности для ведения обратного отсчета. Мы рекомендуем следовать перечисленным шагам, но это ни в коем случае не единственный и даже не обязательно лучший подход. Просто мы предпочитаем делать так:

1. Определить, какие части таймера подходят для выделения в компоненты, чтобы упростить дальнейшую работу с таймером.
2. Создать в компоненте таймера состояние для сохранения текущего времени обратного отсчета, а также логического признака того, ведется отсчет в текущий момент или нет.
3. Добавить в таймер эффект для уменьшения оставшегося времени каждую секунду.
4. Вывести кнопку, запускающую обратный отсчет при остановленном таймере, и кнопку, останавливающую его в случае ведения отсчета.

Точность

Заметим, что нас не очень волнует точность измерения времени. Нас устроит выполнение обратного отсчета в `setInterval`, хотя `setInterval` пользуется дурной славой из-за своей ненадежности. Таким образом, на каждом такте погрешность обратного отсчета будет составлять от нескольких сотых до десятой доли секунды. Когда таймер в приложении отсчитывает полные 5 минут, это отклонение может достичь полминуты «реального времени».

Если вам требуется точность, лучше выбрать другой путь. Для нас это не приоритет, поэтому мы воспользуемся выбранным подходом во всем проекте.

Точность отсчета можно повысить в конце работы над проектом, при желании. В этом вам поможет функция `performance.now()` (обеспечивающая точность до микросекунд, если ОС поддерживает такую возможность).

12.2.2. Иерархия компонентов

Как уже сказано, наша первая задача — выделить части таймера в атомарные компоненты, которые можно объединять по своему усмотрению. Для этого вернемся к исходному списку частей HTML и разобьем компонент следующим образом:

- **Timer** — таймер состоит из индикатора времени и одной или нескольких кнопок.
- **TimeDisplay** — индикатор времени представляет собой список из числа и единицы измерения, за которыми следует двоеточие, а затем еще одно число и единица измерения.
- **Number** — число и единица измерения, с именами классов для соответствующего стилизового оформления.
- **Button** — кнопка содержит ярлык доступности и значок, облегчающий визуализацию.

Эти четыре компонента образуют таймер и соединяются друг с другом, как показано на рис. 12.8. Также мы видим, что начальный интервал передается таймеру непосредственно из менеджера таймера.

Таймер содержит только одну кнопку, но значок на кнопке, надпись и обработчик щелчка изменяются в зависимости от того, работает таймер или нет. Кроме того, обратите внимание, что индикатор времени получает только одно входящее значение — количество оставшихся секунд. Компонент индикатора времени должен разбить его на минуты и секунды, чтобы передать двум числовым компонентам.

12.2.3. Обновленная структура проекта

На этом шаге проекта добавляются новые компоненты, а также обновляются уже существующие. Обновленная структура файлов после завершения этого шага будет выглядеть так:



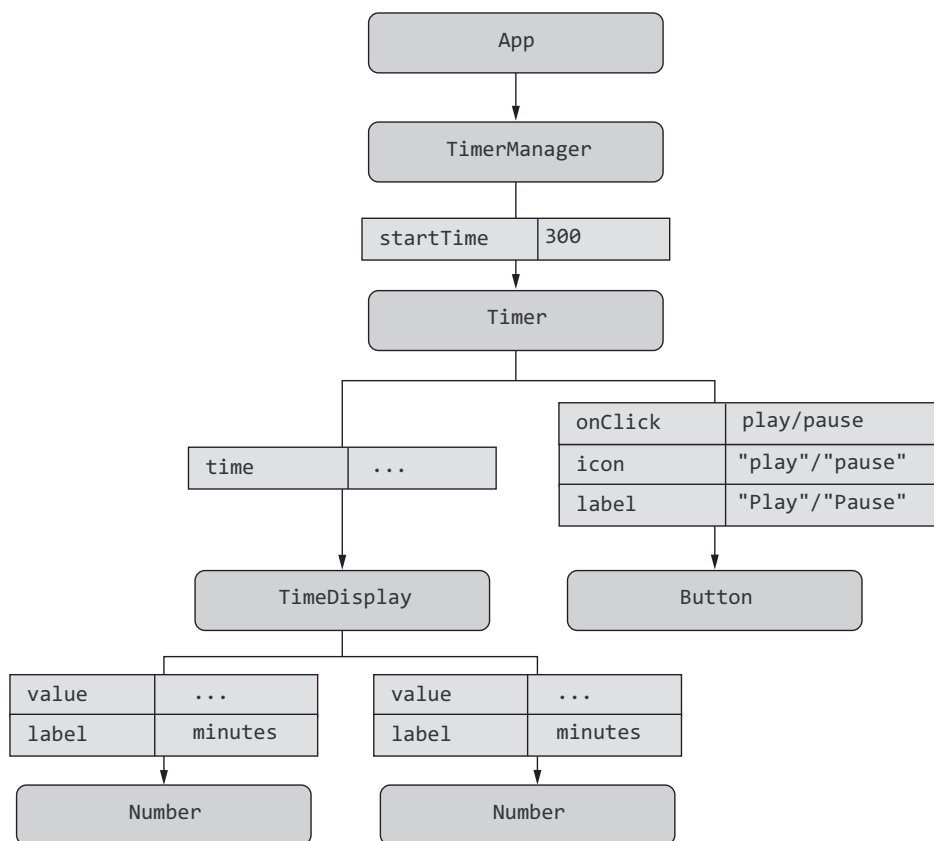


Рис. 12.8. Дерево компонентов для приложения с одним таймером состоит из нескольких экземпляров трех новых компонентов: `TimeDisplay`, `Number` и `Button`

Попробуйте реализовать этот шаг самостоятельно. Можете начать с заготовки на шаге 1 проекта либо с нашей реализации из репозитория `rq12-scaffold`. Завершив работу, сравните свое решение с нашим.

Репозиторий: `rq12-playpause`

Этот пример находится в репозитории `rq12-playpause`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq12-playpause --template rq12-playpause
```


На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq12-playpause>

12.2.4. Исходный код

Этот раздел содержит полный исходный код всех обновленных файлов и файлов, добавленных на этом шаге, а также некоторые подробности реализации, важные для понимания принятых решений.

Менеджер таймера

Менеджер таймера все еще ничего не делает. Изменение только одно: на этот раз мы явно задаем время по умолчанию для таймера. Вы можете заменить его другим значением, чтобы поэкспериментировать. Файл `src/TimerManager.js` полностью приведен в листинге 12.4.

Листинг 12.4. Файл `src/TimerManager.js` для простого таймера

```
import Timer from "../Timer";
function TimerManager() {
  return (
    <div className="timers">
      <Timer startTime={3000} />
    </div>
  );
}
export default TimerManager;
```

← Добавляет свойство с начальным интервалом по умолчанию

Обобщенный компонент кнопки

Нам понадобится новый компонент кнопки. Он будет очень простым: будет рендерить элемент кнопки с соответствующим текстом и значком. Если компоненту кнопки передаются дополнительные свойства, мы перенаправляем их элементу кнопки. Файл `src/Button.js` полностью приведен в листинге 12.5.

Листинг 12.5. Файл `src/Button.js` для простого таймера

```
function Button({ icon, label, ...rest }) {
  return (
    <button title={label} className="toggle" {...rest}>
      <img src={`/icons/${icon}.svg`} alt={label} />
    </button>
  );
}
export default Button;
```

← Здесь необходимо передать значок и ярлык, но можно передавать любые другие свойства кнопок

← Все дополнительные свойства будут добавлены к элементу кнопки. Именно здесь следует добавлять такие свойства, как `onClick`

Компонент числа и единицы измерения

Компонент `Number` просто выводит свойства. Число должно быть отформатировано так, чтобы оно всегда состояло из двух цифр, поэтому мы преобразуем его в строку и добавляем начальные нули при необходимости, как показано в листинге 12.6.

Листинг 12.6. Файл `src/Number.js` в простом таймере

```
function Number({ value, label }) {
  return (
    <li className="part">
      <p className="number">
        {String(value).padStart(2, "0")}
      </p>
      <p className="unit">{label}</p>
    </li>
  );
}
export default Number;
```

Здесь выводится число, но оно должно всегда состоять из двух символов. Для этого оно преобразуется в строку и при необходимости дополняется начальными нулями. В результате число 7 будет выводиться в виде 07

Ярлык отображается в исходном виде — позже мы преобразуем текст к верхнему регистру средствами CSS, чтобы он лучше смотрелся

Компонент индикатора времени

Этот компонент получает время, разбивает его на минуты и секунды (с использованием операций деления, округления и получения остатка), после чего передает эти значения двум экземплярам компонента числа. Исходный код файла `src/TimeDisplay.js` приведен в листинге 12.7.

Листинг 12.7. Файл `src/TimeDisplay.js` в простом таймере

```
import Number from "../Number";
function TimeDisplay({ time }) {
  const minutes = Math.floor(time / 60);
  const seconds = time % 60;
  return (
    <ul className="parts">
      <Number value={minutes} label="minutes" />
      <li className="colon">:</li>
      <Number value={seconds} label="seconds" />
    </ul>
  );
}
export default TimeDisplay;
```

Количество оставшихся минут вычисляется делением времени на 60 с округлением вниз

Количество оставшихся секунд равно остатку от деления на 60

Компонент таймера

Здесь все составляющие собираются воедино. Логика работы компонента таймера в виде диаграммы состояний представлена на рис. 12.9.

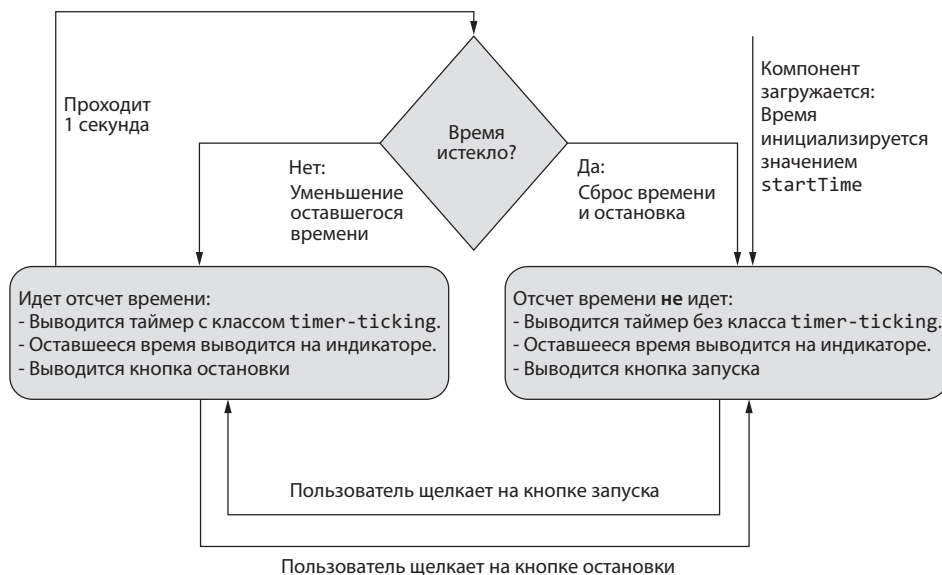


Рис. 12.9. Диаграмма состояний для простого компонента таймера — отсчет времени либо ведется, либо нет. Этот признак определяет, какая информация должна выводиться и какие эффекты должны запускаться

Помните, что таймер теперь получает одно свойство `startTime`, которое определяет, с каким интервалом будет запускаться таймер. С этим значением мы можем инициализировать локальное состояние количеством оставшихся секунд, в течение которых будет вестись отсчет, а также отдельным состоянием, которое содержит логический признак ведения отсчета таймером.

Затем выполняются разные действия в зависимости от того, ведется отсчет времени или нет. Если отсчет ведется, выполняется хук эффекта для декремента (пока значение не достигнет нуля — в этом случае все установки таймера сбрасываются). Также выводится кнопка остановки, если таймер работает.

Если отсчет времени не ведется, хук эффекта не выполняется (или выполняется, если он зачем-то нужен, но в нем ничего не происходит) и отображается кнопка запуска. Содержимое файла `src/Timer.js` приведено в листинге 12.8.

Листинг 12.8. Файл `src/Timer.js` в простом таймере

Далее необходимо знать, ведется отсчет или нет

```
import { useState, useEffect } from "react";
import Button from "../Button";
import TimeDisplay from "../TimeDisplay";
function Timer({ startTime }) {
  const [remaining, setRemaining] =
    useState(startTime);
  const [isRunning, setRunning] = useState(false);
  useEffect(() => {
    if (!isRunning) {
      return;
    }
    function tick() {
      setRemaining((oldValue) => {
        const value = oldValue - 1;
        if (value <= 0) {
          setRunning(false);
          return startTime;
        }
        return value;
      });
    }
    const interval = setInterval(tick, 1000);
    return () => clearInterval(interval);
  }, [isRunning, startTime]);
  const play = () => setRunning(true);
  const pause = () => setRunning(false);
  return (
    <section className={
      `timer ${isRunning ? "timer-ticking" : ""}`
    >
      <TimeDisplay time={remaining} />
      {isRunning ? (
        <Button
          icon="pause"
          label="Pause"
          onClick={pause}
        />
      ) : (
        <Button
          icon="play"
          label="Play"
          onClick={play}
        />
      )}
    </section>
  );
}
export default Timer;
```

Чтобы таймер работал, нам понадобятся два значения с состоянием. Прежде всего необходимо знать, сколько секунд осталось

Непрерывное движение к концу Вселенной реализуется хуком эффекта. Впрочем, хук что-то делает только в том случае, если таймер действительно работает

Если отсчет времени ведется, мы планируем интервал, который уменьшает количество оставшихся секунд каждую секунду или останавливает таймер при достижении 0 (после чего таймер сбрасывается)

Всегда убирайте за собой

Этот хук зависит от того, ведется отсчет времени или нет, а также от начального времени (к которому необходимо вернуться при сбросе таймера после завершения отсчета)

Добавляет условный класс в секцию `timer` для вывода мигающего двоеточия во время выполнения

Передает оставшееся время компоненту индикатора времени

Если отсчет ведется, выводится кнопка остановки таймера. Если отсчет не ведется, выводится кнопка запуска таймера

Выводит разные кнопки в зависимости от того, ведется отсчет времени или нет

12.2.5. Запуск приложения

Запустив это приложение в браузере, вы увидите результат, показанный на рис. 12.10.



Рис. 12.10. Таймер либо отсчитывает время, либо находится в состоянии остановки. Во время отсчета двоеточие мигает. Внешний вид и поведение кнопки меняются в зависимости от режима работы

12.3. ДОМАШНЕЕ ЗАДАНИЕ: ИНИЦИАЛИЗАЦИЯ ТАЙМЕРА ПРОИЗВОЛЬНЫМ ВРЕМЕНЕМ

Цель этого шага — инициализация таймера нестандартным временем, которое задает пользователь в форме с полями ввода. Подсказки, приведенные ниже, помогут с решением этой задачи:

1. Создайте новый компонент для добавления таймера. В таблице стилей вы найдете классы, которые пригодятся для дизайна формы и ее полей ввода.
2. Выберите, должен быть ввод с новой формы таймера контролируемым или нет. Оба варианта здесь допустимы.
3. Помните, что кнопка, добавленная к форме, становится кнопкой отправки данных, даже если у нее нет обработчика щелчков.
4. Наделите менеджер таймера состоянием. Храните информацию о том, ведет таймер отсчет или нет и какое начальное время задано.
5. Добавьте кнопку удаления (со значком мусорной корзины) к компоненту таймера.

6. Компоненту таймера нужно новое свойство — обратный вызов, который инициируется при завершении или сбросе таймера. Обратный вызов должен сбросить менеджер таймера, чтобы можно было использовать форму для нового таймера.

Конечно, мы бы предпочли, чтобы вы начали с приложения, разработанного на предыдущем шаге, но если вы хотите начать с нашего решения, обратитесь к репозиторию `rq12-playpause`. После завершения работы сравните свое решение с нашим.

Репозиторий: `rq12-initialize`

Этот пример находится в репозитории `rq12-initialize`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq12-initialize --template rq12-initialize
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq12-initialize>

12.4. ДОМАШНЕЕ ЗАДАНИЕ: СБРОС ТАЙМЕРОВ

Цель этого шага — преобразование логики состояния в редьюсер и добавление логики для сброса таймеров. Вот несколько подсказок, которые помогут решить эту задачу:

1. Преобразуйте состояние в таймере в редьюсер.
2. Добавьте в таймер новую кнопку сброса и обеспечьте перезапуск таймера по вызову соответствующего действия для редьюсера. Тщательно продумайте, что должно происходить с обоими значениями в редьюсере при сбросе.
3. Возможно, вы захотите изменить то, что происходит при завершении отсчета таймера. Можно не удалять таймер, а оставить его до тех пор, пока он не будет удален активно.

Конечно, мы бы предпочли, чтобы вы начали с приложения, разработанного на предыдущем шаге, но если вы хотите начать с нашего решения, обратитесь к репозиторию `rq12-initialize`. После завершения работы сравните свое решение с нашим.

Репозиторий: rq12-reset

Этот пример находится в репозитории `rq12-reset`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq12-reset --template rq12-reset
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq12-reset>

12.5. ДОМАШНЕЕ ЗАДАНИЕ: МНОЖЕСТВЕННЫЕ ТАЙМЕРЫ

На этом шаге мы разрешим пользователю установить несколько таймеров, которые будут работать независимо друг от друга. Вот несколько подсказок, которые помогут решить эту задачу:

1. Обновите состояние, хранящееся в менеджере таймера, чтобы могли выполняться сразу несколько таймеров одновременно (с разным начальным временем).
2. Обеспечьте возможность удаления таймеров в любой момент времени.
3. Завершение отсчета должно обозначаться миганием (в таблице стилей для этого существует специальный класс), чтобы пользователь мог сбросить или удалить таймер в любой момент.
4. В менеджере таймера предоставьте пользователю возможность добавить новый таймер нажатием кнопки, но не отображайте форму создания таймера до нажатия кнопки добавления. После добавления нового таймера снова выведите кнопку добавления.

Конечно, мы бы предпочли, чтобы вы начали с приложения, разработанного на предыдущем шаге, но если вы хотите начать с нашего решения, обратитесь к репозиторию `rq12-reset`. После завершения работы сравните свое решение с нашим.

Репозиторий: rq12-multiple

Этот пример находится в репозитории `rq12-multiple`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq12-multiple --template rq12-multiple
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq12-multiple>

ИТОГИ

- В этом проекте мы взяли за более сложное приложение, которое можно использовать на практике.
- Мы прошли все этапы, которые вы пройдете, разрабатывая реальный подобный проект — в одиночку или в команде.
- Начните с анализа структуры и предметной области; так вы будете лучше представлять структуру желаемого приложения. В этом проекте мы смогли определить, какие компоненты нам понадобятся, непосредственно из самой структуры приложения и сохранить это разделение логики в процессе работы.
- Также продумайте внутреннее состояние компонентов и не забывайте, что состояние может находиться на разных уровнях одновременно. В этом проекте состояние находится на трех уровнях, работающих независимо друг от друга, хотя существует интерфейс, обеспечивающий взаимодействие между ними.
- Список таймеров — это всего лишь список таймеров. Ему известно, с какого времени запускается каждый таймер, но он не знает, какие таймеры продолжают отсчет и не истекло ли на них время.
- Отдельный таймер работает сам по себе, пока не будет удален. При удалении он инициирует обратный вызов, предоставленный родителем, и ему не важно, что произойдет после.
- Форма для добавления нового таймера запоминает состояние формы и инициирует обратный вызов после отправки данных. Ей ничего не известно о том, что произойдет после отправки данных.
- Создавая реальные приложения, продумайте подходящую семантику (не все должно существовать в виде `<div>`) и уделите внимание доступности приложения (добавьте к элементам описания ролей и ярлыки, где необходимо).

Проект: менеджер задач

В ЭТОЙ ГЛАВЕ

- ✓ Создание заготовки для менеджера задач
- ✓ Реализация простого списка задач
- ✓ Домашнее задание: расширение функциональности менеджера задач

Мы добрались до третьего — и последнего — проекта в этой книге. В работе над ним вам пригодится все, что вы узнали к этому моменту, а также дополнительные знания JavaScript и HTML, которые, мы надеемся, у вас есть, — впрочем, они нужны лишь для продвинутого домашнего задания в конце главы.

В этом проекте мы создадим *менеджер задач*, то есть чуть более сложную разновидность списка задач. В начале менеджер задач будет представлять собой простой список задач в виде карточек, которые можно начинать и завершать. Затем мы добавим к каждой задаче возможность разделения на этапы, чтобы пользователь мог включить в задачи больше деталей. После этого добавим возможность менять порядок этапов в каждой задаче — сначала только с использованием кнопок, а затем и перетаскиванием. Именно последняя часть шага 5 — перетаскивание — может вызвать трудности. Схема работы над проектом представлена на рис. 13.1, и для последнего шага 5 мы воспользуемся инструментом расширенных событий.

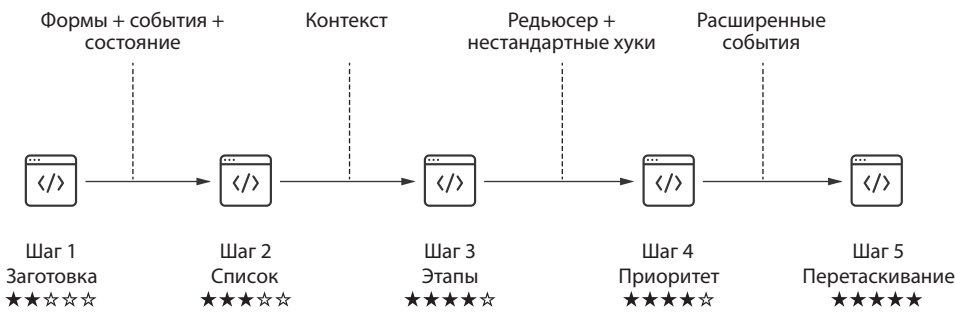


Рис. 13.1. Работа над проектом будет включать пять шагов, на каждом из которых в приложение добавляется функциональность и сложность

И снова мы подготовим основу проекта за вас, так что вам не придется долго возиться с HTML, значками и CSS. Однако она будет полностью статичной и ничего не выполняющей. Чтобы добавить в нее функциональность, мы вместе будем поэтапно повышать сложность проекта.

Необходимые шаги перечислены в табл. 13.1. В ней также подробно описано, что мы будем делать и к каким главам стоит обращаться за помощью в ходе работы над упражнениями.

Таблица 13.1. Пять шагов проекта менеджера задач

Шаг	Функциональность	Используемый React API	Сложность
Шаг 1. Заготовка	Создание базовой структуры компонентов для списка задач и формы добавления новой задачи	Главы 1-4: функциональные компоненты с использованием JSX	★★★★☆
Шаг 2. Список	Преобразование статической структуры в динамический список задач, хранящийся в состоянии, с возможностями редактирования и удаления задач	Глава 5: состояние Глава 8: события Глава 9: формы	★★★★☆
Шаг 3. Этапы	Добавление этапов и отслеживания прогресса задач, включая средства удаления и завершения этапов. Список задач представляет собой аккордеон: в любой момент раскрывается только одна задача. Примечание: это домашнее задание. Выполните этот шаг <i>самостоятельно</i> !	Глава 10: контекст	★★★★☆

Шаг	Функциональность	Используемый React API	Сложность
Шаг 4. Приоритет	Добавление возможности назначения приоритета этапам за счет их переупорядочения. Задача упрощается преобразованием состояния в редьюсер. Примечание: это домашнее задание. Выполните этот шаг <i>самостоятельно!</i>	Глава 10: редьюсер и нестандартный хук	★★★★☆
Шаг 5. Перетаскивание	Реализация возможности перетаскивания этапов для изменения приоритета (вместо перемещения на одну позицию вверх или вниз при помощи кнопок со стрелками). Примечание: это домашнее задание. Выполните этот шаг <i>самостоятельно!</i>	Глава 8: обработчики событий	★★★★★

Первая итерация этого упражнения, к которой мы придем, выполнив шаг 2, представляет собой очень простой менеджер задач, показанный на рис. 13.2.

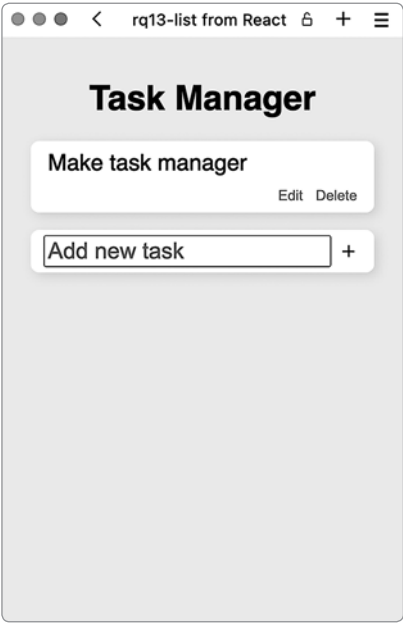


Рис. 13.2. Первая итерация менеджера задач представляет собой обычный список с возможностью добавления, удаления и редактирования задач

Однако по завершении шага 5 проекта приложение станет намного более мощным и будет иметь гораздо более широкую функциональность, как показано на рис. 13.3.



Рис. 13.3. Теперь каждая задача содержит список этапов, и пользователь может добавлять, помечать как завершённые, редактировать, удалять и изменять приоритет этих этапов внутри задачи

Этапы даже можно перетаскивать внутри задач, как показано на рис. 13.4.

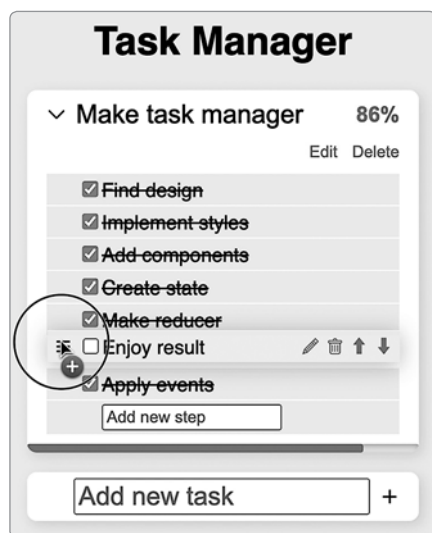


Рис. 13.4. Этапы можно перетаскивать внутри списка, чтобы упростить изменение приоритета

Нам предстоит серьёзная работа. Итак, приступим к заключительному проекту последней главы книги.

ПРИМЕЧАНИЕ Исходный код заготовки и предлагаемых решений для всех разделов этой главы доступен по адресу <https://rq2e.com/ch13>. Но как вы узнали в главе 2, все эти примеры можно создать прямо в командной строке одной командой.

13.1. ЗАГОТОВКА МЕНЕДЖЕРА ЗАДАЧ

И снова начнем с базовой заготовки приложения. Мы создадим разметку HTML для статического менеджера задач и предоставим все необходимые стили. Менеджер не будет ни динамическим, ни функциональным, но будет очень похож на окончательную версию; чтобы он работал, достаточно немного «магии React».

Мы также предоставим значки для кнопок в решении.

13.1.1. Иерархия компонентов

На этом шаге мы немного упростим себе жизнь. Создадим все (статическое) приложение в одном компоненте. В принципе, можно было выделить и несколько компонентов, но мы решили, что вам это лучше сделать самостоятельно.

С одним компонентом, возвращающим весь код JSX для приложения в статической, фиксированной конфигурации, вы будете точно видеть, как создается приложение и куда лучше двигаться. В результате схема компонентов этого приложения оказывается очень простой — она представлена на рис. 13.5.

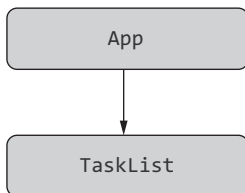


Рис. 13.5. Выше мы утверждали, что самое простое дерево изображено на рис. 12.5, но пожалуй, это дерево еще проще!

13.1.2. Структура проекта

С такой простой иерархией компонентов на этом шаге о папке с исходным кодом сказать особо нечего. В ней, как обычно, находится главный файл приложения и файл CSS, а также единственный специфический компонент — `TaskList`.

Тем не менее в проекте используются некоторые значки; мы добавили восемь файлов SVG в папку `public`, где находится содержимое, используемое в проекте. Структура файлов выглядит так:

```
public/
icons/
  caret.svg
  check.svg
  down.svg
  drag.svg
  pencil.svg
  plus.svg
  trash.svg
  up.svg
favicon.ico
index.html
src/
  App.js
  index.js
  style.css
  TaskList.js
```

Репозиторий: **rq13-scaffold**

Этот пример находится в репозитории `rq13-scaffolds`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq13-scaffold --template rq13-scaffold
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq13-scaffold>

13.1.3. Исходный код

На этом шаге приложение содержит всего два исходных файла, представляющих интерес. Это главный файл приложения, который почти не отличается от тех, что мы уже видели, и список задач, специфический для приложения.

Главное приложение

Код главного приложения содержится в файле `src/App.js` и наверняка вам уже хорошо знаком. Содержимое файла представлено в листинге 13.1.

Листинг 13.1. Файл src/App.js в заготовке

```
import "./style.css"; ← Загружает файл CSS
import TaskList from "./TaskList"; ← Загружает компонент верхнего
function App() {                                     уровня в приложении
  return (
    <div>
      <TaskList />
    </div>
  );
}
```

```

<main>
  <h1>Task Manager</h1>
  <TaskList />
</main>
);
}
export default App;

```

Рендерит этот компонент
в соответствующем дереве JSX

Список задач

Файл `src/TaskList.js` приведен в листинге 13.2. В нем содержится весь код JSX для рендеринга списка из двух задач, а также формы для добавления новой задачи в конце списка. Впрочем, пока все это неактивно, так что при нажатии кнопок не происходит ничего интересного.

Листинг 13.2. Файл `src/TaskList.js` в заготовке

Каждая задача представлена элементом списка

```

function TaskList() {
  return (
    <ol className="lane">
      <li className="card">
        <header className="card-header">
          <p className="card-title">This is a task</p>
        </header>
        <ul className="card-controls">
          <li>
            <button className="card-control">Edit</button>
          </li>
          <li>
            <button className="card-control">Delete</button>
          </li>
        </ul>
      </li>
      <li className="card">
        <header className="card-header">
          <p className="card-title">This is another task</p>
        </header>
        <ul className="card-controls">
          <li>
            <button className="card-control">Edit</button>
          </li>
          <li>
            <button className="card-control">Delete</button>
          </li>
        </ul>
      </li>
      <li className="card">
        <header

```

Серия задач представляет
собой упорядоченный список

У задачи есть
заголовок
с названием

Под заголовком
задачи располагается
неупорядоченный
список кнопок

```

      className="card-header card-header-new"
    >
    <form className="card-title-form">
      <input
        className="card-title card-title-input"
        placeholder="Add new task"
        name="title"
      />
      <button className="icon-button">
        
      </button>
    </form>
  </header>
</li>
</ol>
);
}
export default TaskList;

```

Последняя задача в списке отличается от других: она содержит форму для создания новой задачи при помощи поля ввода и кнопки со значком

13.1.4. Запуск приложения

Заготовка представляет собой симпатичное, но совершенно бесполезное приложение со статическим списком задач и формой в нижней части, как показано на рис. 13.6.

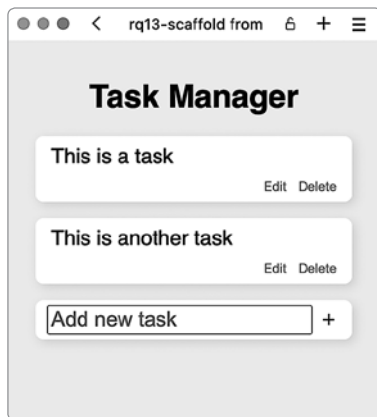


Рис. 13.6. Если бы кнопки работали, это было бы довольно полезное приложение

13.2. ПРОСТОЙ СПИСОК ЗАДАЧ

Итак, мы разобрались с самым необходимым, и теперь у нас есть код JSX, стили и значки. Можно переходить к следующему шагу — созданию приложения менеджера задач. Все, что мы сделали до этого, можно считать «обычной»

веб-разработкой, которая выходит за рамки материала книги. А сейчас мы займемся разработкой React, и у вас появится возможность применить все, что вы узнали.

13.2.1. Цель упражнения

На этом шаге проекта в структуру, намеченную в заготовке, мы добавим новую функциональность. После этого у вас должен получиться простой менеджер задач, который делает следующее:

- Выводит список задач.
- Позволяет пользователю создать новую задачу.
- Позволяет пользователю переименовать задачу.

Разделим работу на два этапа:

1. Разобьем один большой компонент на несколько меньших компонентов подходящим способом, в зависимости от размера компонентов, их обязанностей и визуального представления.
2. Наделим приложение состоянием, чтобы оно начиналось с заранее определенного списка и пользователи могли присоединять, удалять и обновлять задачи по желанию.

13.2.2. Иерархия компонентов

Хотя в заготовке используется всего один компонент `TaskList`, связанный с менеджером задач, на этом шаге мы расширим его до нескольких компонентов. Новая структура показана на рис. 13.7.

Начнем с внутреннего уровня: названием задачи может быть большой абзац или поле ввода, в котором можно отредактировать название и нажать кнопку для отправки нового названия. Мы создадим его как первый компонент `TaskHeader`. Каждая задача представляется отдельным компонентом `Task`, который хранит информацию состояния задачи, а именно находится ли название в режиме редактирования.

В конце списка задач создается новый компонент для добавления новой задачи. Здесь будет находиться третий новый компонент `TaskAdd`, который содержит форму и инициирует обратный вызов после отправки данных.

Наконец, необходимо добавить в проект кнопку со значком, и это можно сделать сейчас. Проявим творческий подход и назовем этот компонент `Button`. Сейчас

кнопка со значком понадобится только внутри формы создания новой задачи, но она будет не раз применяться в дальнейшем. Собрал все воедино, получаем дерево компонентов на рис. 13.8.

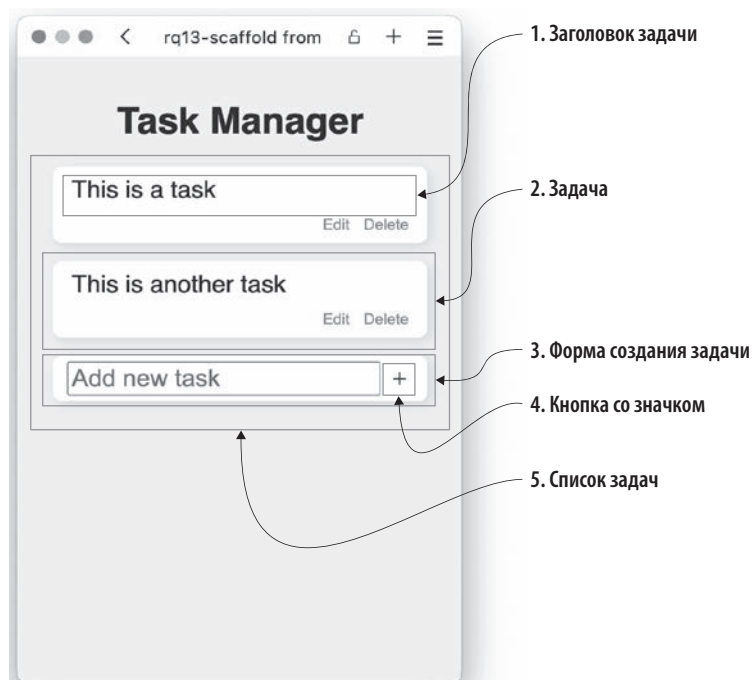


Рис. 13.7. Менеджер задач на этом шаге разбивается на пять компонентов: заголовок задачи с названием или полем ввода для редактирования названия; задача с заголовком и элементами управления; форма для добавления новых задач; кнопка со значком; и наконец, полный список задач

13.2.3. Обновленная структура проекта

Как мы уже сказали, на этом шаге проекта добавляются новые компоненты, а также обновляются некоторые существующие. Кроме того, мы добавляем простой файл JavaScript для подготовки начального значения массива задач с состоянием. Мы назвали этот файл *fixture.js*, так как термин *fixture* часто используется для обозначения «фиксированных» данных, заполняющих приложение.

Наконец, добавим еще одно улучшение: вложенную структуру файлов. В дальнейшем, скорее всего, будут добавляться новые компоненты, поэтому мы инкапсулируем четыре компонента, относящиеся к задачам, в отдельной папке. Чтобы

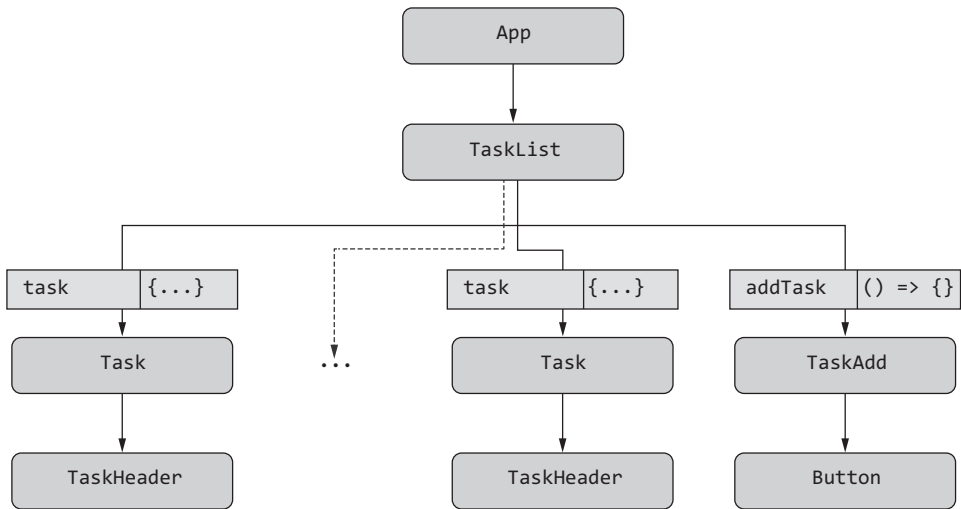


Рис. 13.8. Дерево на шаге 2 проекта состоит из пяти компонентов. В частности, компонент `Task` используется многократно в зависимости от количества отображаемых задач

их было проще импортировать, добавим в эту папку индексный файл, который экспортирует из этой папки только необходимые компоненты.

По завершении этого шага обновленная структура данных будет такой:



Мы рекомендуем постараться выполнить этот шаг самостоятельно. Начните с заготовки на шаге 1 проекта или с нашей реализации из репозитория `rq13-scaffold`. Завершив работу, сравните свое решение с нашим.

Репозиторий: rq13-list

Этот пример находится в репозитории `rq13-list`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq13-list --template rq13-list
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq13-list>

13.2.4. Исходный код

В этом разделе приведен полный исходный код всех обновленных файлов и файлов, добавленных на этом шаге, а также подробности реализации, важные для понимания принятых решений.

Главный файл приложения

Единственное изменение главного компонента приложения — расположение импортируемого списка задач. Вместо импортирования списка задач по умолчанию из файла `TaskList` мы импортируем его как именованный импорт из папки с именем `task`. Это изменение представлено в листинге 13.3.

Листинг 13.3. Файл `src/App.js` в простом списке задач

```
import "./style.css";
import { TaskList } from "./task";
function App() {
  return (
    <main>
      <h1>Task Manager</h1>
      <TaskList />
    </main>
  );
}
export default App;
```

Изменяется только место и тип импортирования компонента списка задач

Кнопка со значком

В файле `src/Button.js` создается очень простая кнопка со значком.

Листинг 13.4. Файл `src/Button.js` в простом списке задач

```
function Button({ className = "", icon, label, ...rest }) {
  return (
    <button className={`icon-button ${className}`} {...rest}>
      <img draggable={false} src={`icons/${icon}.svg`} alt={label} />
    </button>
  );
}
export default Button;
```

Можно передать несколько именованных свойств, а также другие свойства, которые могут передаваться по ссылке

Здесь заслуживает внимания только атрибут `draggable` изображения. Он необходим для подготовки к последнему шагу, чтобы кнопки со значками нельзя было перетаскивать независимо

Общедоступный интерфейс для папки `task`

Папка `task` служит своего рода отдельным модулем, в котором можно использовать компоненты во внутренней реализации так, как нам необходимо, но во внешнюю среду как «общедоступный» предоставляется только компонент списка задач. Впрочем, формально это не ограничение, так как из папки `task` можно импортировать любой компонент; это лишь первый шаг на пути к пакетной структуре проекта. В результате остается очень простой индексный файл `src/task/index.js`, приведенный в листинге 13.5.

Листинг 13.5. Файл `src/task/index.js` в очень простом списке задач

```
export { default as TaskList } from "../TaskList";
```

Из этой папки предоставляется только компонент списка задач, для чего используется именованный экспорт

Фиксированный список задач по умолчанию

Список задач хранится в локальном хранилище в браузере, но при первом запуске приложения в нем еще нет списка задач, который можно восстановить, поэтому для работы приложения необходимы данные по умолчанию. Можно вывести пустой список, но часто для заполнения используют условные данные, которые пользователь может взять за образец.

Может, наш пример и нельзя назвать образцовым, но он определенно нагляден. Файл `src/task/fixture.js` приведен в листинге 13.6.

Листинг 13.6. Файл `src/task/fixture.js` в простом списке задач

```
const initialState = [
  { id: 1, title: "Make task manager" },
  { id: 2, title: "Now add some more tasks" },
];
export default initialState;
```

Заголовок задачи

Заголовок задачи кажется простым, потому что фактически представляет собой ее название. Однако его можно редактировать, и во время редактирования заголовков преобразуется в форму со значком-галочкой, служащим кнопкой отправки данных. Реализация из файла `src/task/TaskHeader.js` приведена в листинге 13.7.

Листинг 13.7. Файл `src/task/TaskHeader.js` в простом списке задач

```
function TaskHeader({
  task,
  isEditable,
  setEditable,
  editTask,
}) {
  const { title } = task;
  const handleEditTask = (evt) => {
    evt.preventDefault();
    editTask(task.id, evt.target.title.value);
    setEditable(false);
  };
  if (isEditable) {
    return (
      <header className="card-header">
        <form
          className="card-title-form"
          onSubmit={handleEditTask}
        >
          <input
            className="card-title card-title-input"
            defaultValue={title}
            name="title"
          />
          <button className="icon-button">
            
          </button>
        </form>
      </header>
    );
  }
  return (
    <header className="card-header">
      <p className="card-title">{title}</p>
    </header>
  );
}
export default TaskHeader;
```

Этот компонент получает свойства, необходимые для редактирования заголовка

При отправке формы редактирования блокируется действие по умолчанию (то есть перезагрузка страницы), текущая задача обновляется новым значением и возможность редактирования заголовка снова отключается

Если заголовок редактируется, мы возвращаем специальный код JSX

Этот код JSX включает форму с обработчиком отправки данных, определенную выше

Если заголовок не редактируется, то возвращается статический блок JSX с текущим заголовком

Задача целиком

Приведенный выше заголовок включается в каждую задачу. Задачи можно редактировать и удалять. При редактировании задачи заголовок обновляется

соответствующими свойствами, включая обратный вызов для обновления задачи, переданный от родительского компонента. При удалении задачи мы просто напрямую инициируем обратный вызов удаления. Вся эта функциональность реализована в файле `src/task/Task.js`.

Листинг 13.8. Файл `src/task/Task.js` в простом списке задач

Задача получает всю информацию о текущей задаче, а также два обратных вызова для редактирования и удаления задачи соответственно

```
import { useState } from "react";
import TaskHeader from "../TaskHeader";
function Task({ task, editTask, deleteTask }) {
  const [isEditable, setEditable] = useState(false);
  return (
    <li className="card">
      <TaskHeader
        task={task}
        isEditable={isEditable}
        setEditable={setEditable}
        editTask={editTask}
      />
      <ul className="card-controls">
        {!isEditable && (
          <li>
            <button
              className="card-control"
              onClick={() => setEditable(true)}
            >
              Edit
            </button>
          </li>
        )}
        <li>
          <button
            className="card-control"
            onClick={() => deleteTask(task.id)}
          >
            Delete
          </button>
        </li>
      </ul>
    </li>
  );
}
```

Хранит в каждой задаче локальное состояние с информацией о том, редактируется ли в настоящий момент название задачи

Рендерит ранее определенный компонент заголовка задачи со всеми необходимыми свойствами в начале компонента

Под заголовком размещаются две кнопки; первая устанавливает локальный флаг редактирования в true

Вторая кнопка инициирует обратный вызов удаления

```
export default Task;
```

Форма создания задачи

Для добавления новой задачи используется форма с одним неуправляемым полем ввода для названия задачи и кнопкой отправки данных в форме кнопки со значком. Содержимое файла `src/task/TaskAdd.js` приведено в листинге 13.9.

Листинг 13.9. Файл `src/task/TaskAdd.js` в простом списке задач

Компонент формы задачи получает одно свойство с обратным вызовом, который должен выполняться при добавлении новой задачи

При отправке данных формы задачи выполняются три операции: отмена действия по умолчанию, инициирование обратного вызова и сброс формы, чтобы она была готова к добавлению новой задачи

```
import Button from "../Button";
function TaskAdd({ addTask }) {
  const handleAddTask = (evt) => {
    evt.preventDefault();
    addTask(evt.target.title.value);
    evt.target.reset();
  };
  return (
    <li className="card">
      <header className="card-header card-header-new">
        <form
          className="card-title-form"
          onSubmit={handleAddTask}
        >
          <input
            className="card-title card-title-input"
            placeholder="Add new task"
            name="title"
          />
          <Button icon="plus" label="Add task" />
        </form>
      </header>
    </li>
  );
}
export default TaskAdd;
```

Обработчик отправки данных добавляется в узел `form`

Полный список задач с состоянием

Последний компонент на этом шаге — список задач — одновременно и самый важный, и наделен самыми широкими обязанностями. Он решает три задачи:

1. Управление состоянием всех задач и предоставление обратных вызовов для добавления, редактирования и удаления задач.
2. Инициализация списка задач из локального хранилища или из фиксированного списка по умолчанию.
3. Вывод всех задач в списке, в конце которого добавлена форма для создания новой задачи.

Один компонент наделяется множеством обязанностей, поэтому часть их (особенно пункты 1 и 2) стоит выделить в нестандартный хук, чтобы проще понять. Пока мы оставим весь код в одном файле, но рекомендуем внести это изменение в будущих итерациях проекта. Содержимое файла `src/task/TaskList.js` приведено в листинге 13.10.

Листинг 13.10. Файл `src/task/TaskList.js` в простом списке задач

Мы используем функцию, которая возвращает исходное состояние как аргумент для хука состояния. Напомним, что эта функция будет вызываться только при первом рендеринге компонента, но не при повторных рендерингах

```
import { useState, useEffect } from "react";
import Task from "../Task";
import TaskAdd from "../TaskAdd";
import initialState from "../fixture";
function getInitialState() {
  return (
    JSON.parse(localStorage.getItem("task-manager-items-list")) ||
    initialState
  );
}
```

Исходное состояние списка задач представляет собой парсированное значение из локального хранилища, если оно существует, или исходное состояние, возвращаемое фиксированным списком

```
function TaskList() {
  const [tasks, setTasks] = useState(getInitialState());
```

```
  useEffect(() => {
    localStorage.setItem(
      "task-manager-items-list",
      JSON.stringify(tasks)
    );
  }, [tasks]);
```

Добавляет эффект для сохранения списка задач в локальном хранилище при каждом изменении списка задач

```
  const addTask = (title) =>
    setTasks((ts) => ts.concat(
      [{ id: Math.random() * 1000000, title }]
    ));
```

Первый из трех обратных вызовов представляет собой функцию для добавления новой задачи. Она присоединяет аргумент к списку задач при помощи функции `update`

```
  const editTask = (id, title) =>
    setTasks((ts) => ts.map(
      (task) =>
        (task.id === id ? { ...task, title } : task)
    ));
```

Обратный вызов для редактирования задачи отображает весь список задач на новый массив и обновляет соответствующую задачу в процессе перебора всех элементов

```
  const deleteTask = (id) => setTasks(
    (ts) => ts.filter((task) => task.id !== id)
  );
```

Наконец, обратный вызов для удаления задачи фильтрует существующий список задач, чтобы удалить из него лишнюю задачу

```
  return (
    <ol className="lane">
      {tasks.map((task) => (
        <Task
          key={task.id}
          task={task}
          editTask={editTask}
          deleteTask={deleteTask}
        />
      ))}
      <TaskAdd addTask={addTask} />
    </ol>
  );
}
```

Два обратных вызова передаются компоненту задачи для каждой задачи из списка

Наконец, форма создания задачи добавляется в конец списка

```
export default TaskList;
```

13.2.5. Запуск приложения

На рис. 13.9 показано, как должно выглядеть приложение. Мы создадим несколько задач, которые можно будет удалить позже.

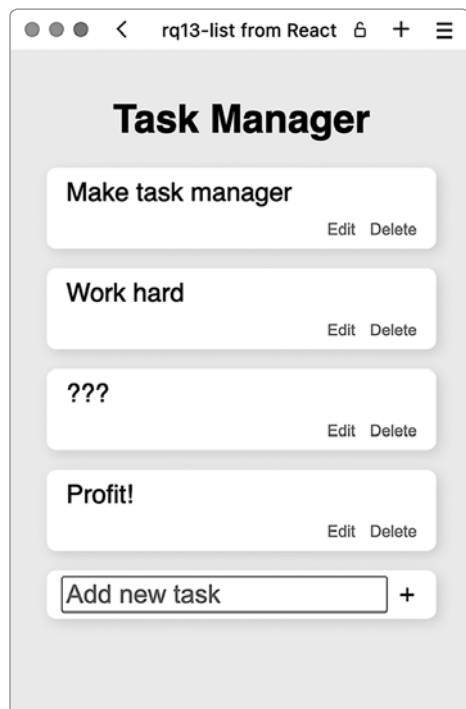


Рис. 13.9. Первая рабочая итерация приложения. В ней можно создавать, удалять и редактировать задачи

13.3. ДОМАШНЕЕ ЗАДАНИЕ: ЭТАПЫ И ПРОГРЕСС ВЫПОЛНЕНИЯ ЗАДАЧ

На этом шаге необходимо выполнить следующее:

- Добавить упорядоченный список «завершаемых» шагов внутри каждой задачи.
- В конец списка добавить поле ввода для нового элемента, помещаемого в конец списка.
- Для каждого этапа в списке добавить чекбокс, помечающий этап как заверченный или незавершенный, а также кнопку для удаления этапа.

- Разрешить пользователю скрывать и отображать этапы задачи (по умолчанию они скрыты).
- Вывести данные о ходе выполнения задачи на индикаторе прогресса, показывающем долю завершенных этапов задачи. Индикатор прогресса должен оставаться видимым, даже если список этапов скрыт.

Вот несколько подсказок, которые помогут решить эту задачу:

1. Хотя в этом примере можно обойтись хранением состояния в простом массиве, которым управляет хук `useState`, сейчас нам нужен более точный контроль состояния, поэтому состояние преобразуется в редьюсер и добавляются действия для нескольких необходимых обновлений: `addTask`, `editTask`, `deleteTask`, `addStep`, `editStep` и `deleteStep`.
2. Также можно упаковать список задач в провайдер контекста, чтобы упростить доступ к этим действиям внутри вложенных компонентов.
3. Для добавления индикатора прогресса используйте элемент HTML `<progress />`. Он довольно простой, и для него уже добавлено стилевое оформление в файле CSS заготовки.
4. Чтобы вывести список этапов с чекбоксами, используйте элементы HTML с подходящей семантикой (отлично подойдут элементы `<ol />`, `<li />`, `<label />` и `<input type="checkbox" />`).
5. Добавление нового шага требует формы с полем ввода и кнопкой. С этим вы уже должны легко справиться.

Конечно, мы бы предпочли, чтобы вы начали с приложения, разработанного на предыдущем шаге, но если вы хотите начать с нашего решения, обратитесь к репозиторию `rq13-list`. После завершения работы сравните свое решение с нашим.

Репозиторий: `rq13-steps`

Этот пример находится в репозитории `rq13-steps`. Вы можете использовать этот репозиторий, создав новое приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq13-steps --template rq13-steps
```

Другой способ: по адресу, приведенному ниже, зайдите на сайт, просмотрите код и приложение непосредственно в браузере либо загрузите исходный код в виде `zip`-файла:

<https://rq2e.com/rq13-steps>

13.4. ДОМАШНЕЕ ЗАДАНИЕ: ПРИОРИТЕТЫ

На этом шаге нам предстоит решить две задачи:

- Добавить кнопку для переименования этапа внутри задачи.
- Добавить кнопку для изменения приоритета этапов внутри каждой задачи.

Ниже несколько подсказок, которые помогут с ними справиться:

1. Если вы еще не преобразовали структуру данных в редьюсер (вместо простого массива состояния с предыдущего шага), обязательно сделайте это. Перемещать элементы в массиве не так сложно, но помните, что каждый раз придется создавать новый массив; изменить существующий массив невозможно. Именно поэтому лучше использовать централизованную и организованную функциональность редьюсера.
2. Переименование этапа работает точно так же, как переименование всей задачи: вы задаете значение локального состояния, которым должен быть заменен текст, и обновляете данные при помощи провайдера после отправки данных формы.
3. В остальном добавление трех кнопок рядом с каждым этапом и вызов нужных функций в редьюсере к этому моменту уже не должны представлять сложностей.
4. Чтобы задание было более интересным, попробуйте сделать следующее: всем функциям, которые должны вызываться внутри одной задачи (добавление этапов, их перемещение и удаление и т. д.), должен быть доступен идентификатор задачи, который будет использоваться для обращения к нужному объекту внутри объекта задачи. Возможно, вам удастся использовать дополнительный провайдер вокруг каждой задачи, который абстрагирует идентификатор этой задачи от отдельных вызовов внутри самой задачи.

Конечно, мы бы предпочли, чтобы вы начали с приложения, разработанного на предыдущем шаге, но если вы хотите начать с нашего решения, обратитесь к репозиторию `rq13-steps`. После завершения работы сравните свое решение с нашим.

Репозиторий: `rq13-priority`

Этот пример находится в репозитории `rq13-priority`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq13-priority --template rq13-priority
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде zip-файла:

<https://rq2e.com/rq13-priority>

13.5. ДОМАШНЕЕ ЗАДАНИЕ: ПЕРЕТАСКИВАНИЕ

На этом шаге необходимо реализовать возможность перетаскивания этапов внутри каждой задачи. Ниже дадим подсказки, которые помогут решить эту задачу:

1. Перетаскивание может быть реализовано в HTML двумя способами. Либо используйте встроенную функциональность HTML5 с атрибутом `draggable` и событиями `dragstart`, `dragover`, `dragenter`, `dragleave` и `drop` (все они поддерживаются в React), либо реализуйте эту функциональность самостоятельно с использованием «чистых» событий мыши (`mousedown`, `mousemove` и `mouseup`).
2. Какой бы вариант вы ни выбрали, задача непростая. Приходится учитывать много разных факторов. Например, если вы начинаете перетаскивать элемент с номером 3 в списке, необходимо иметь возможность сбросить его в любой другой позиции списка, в том числе перед первым или после последнего элемента. Проследите, чтобы в приложении эта возможность работала правильно.
3. Кроме того, придется обновить редьюсер, чтобы обеспечить возможность перемещения этапа в произвольную позицию списка этапов задачи. Возможный вариант интерфейса — `moveStepTo({taskId, step, position})`. Также необходимо учитывать различия между перемещением этапа в более раннюю или более позднюю позицию списка.
4. В репозитории приложения `rq13-dragging` мы использовали встроенную функциональность перетаскивания HTML5. Чтобы выделить место для сброса элемента при перетаскивании этапов, мы отображаем новые элементы между всеми существующими этапами в списке с присоединенным обработчиком `onDrop`. Учтите, что вам также необходимо присоединить обработчики событий `onDragEnter`, `onDragLeave` и `onDragOver` (где вы блокируете действие по умолчанию, а именно запрет на сброс) для элемента, служащего допустимой целью перетаскивания.
5. Если это упражнение покажется вам трудным, не волнуйтесь. Оно действительно такое! Наша реализация не блещет элегантностью, но она работает и выглядит нормально.

Конечно, мы бы предпочли, чтобы вы начали с приложения, разработанного на предыдущем шаге, но если вы хотите начать с нашего решения, обратитесь к репозиторию `rq13-priority`. После завершения работы сравните свое решение с нашим.

Репозиторий: `rq13-dragging`

Этот пример находится в репозитории `rq13-dragging`. Вы можете использовать этот репозиторий, создав новое веб-приложение на основе соответствующего шаблона:

```
$ npx create-react-app rq13-dragging --template rq13-dragging
```

На следующем сайте можно просмотреть код, увидеть работу приложения непосредственно в браузере или загрузить исходный код в виде `zip`-файла:

<https://rq2e.com/rq13-dragging>

13.6. ЗАКЛЮЧЕНИЕ

Третий проект в книге — самый сложный. Мы дали много информации, но вам все равно придется потрудиться и использовать здравый смысл и техническое чутье.

Этот проект содержит реальный кейс, включая пошаговое нарастание сложности и применение практичных и переносимых паттернов программирования React. Чтобы дойти до конца и завершить шаг 5, потребуется приложить все силы. Иногда для этого нас нужно хорошенько подтолкнуть... Впрочем, кажется, это не совсем подходящая метафора.

ИТОГИ

- Разбиение структуры на визуальные части и последующее преобразование каждой части в отдельный компонент React — очень полезный прием, который часто применяется на практике.
- Выбрать между `useState` и `useReducer` иногда бывает сложно, потому что граница между ними размыта и выбор отчасти зависит от личных предпочтений разработчика. В этом проекте мы начали с `useState`, но переключились на редьюсер, когда структура начала усложняться, хотя могли этого и не делать.

- Паттерн провайдера очень гибкий, так что нам удалось снова применить его в этом проекте — пусть и другим способом. Запомните этот паттерн, он часто оказывается полезным.
- Записывать обновления состояния сложнее, если приходится обеспечивать неизменяемость объектов и массивов. Особенно сложны манипуляции с массивами, так как при этом требуется перемещать элементы копированием всех элементов в новый массив в нужном порядке.
- Работать с одиночными событиями в React достаточно просто, но сложные паттерны событий (например, перетаскивание) доставляют массу хлопот даже в таких удобных системах, как React. Возможно, в будущем эти задачи станут проще, но пока что они требуют от разработчика значительных усилий.

Азат Мардан, Мортен Барклунд

React быстро

2-е международное издание

Перевел с английского Е. Матвеев

Руководитель дивизиона

Ю. Сергиенко

Ведущий редактор

Е. Строганова

Литературный редактор

М. Трусковская

Художественный редактор

В. Мостипан

Корректоры

С. Беяева, Н. Викторова

Верстка

Л. Егорова

Изготовлено в России. Изготовитель: ООО «Прогресс книга». Место нахождения и фактический адрес:
194044, Россия, г. Санкт-Петербург, Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 03.2024. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные
профессиональные, технические и научные.

Импортёр в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 27.02.24. Формат 70×100/16. Бумага офсетная. Усл. п. л. 41,280. Тираж 700. Заказ 0000.